

pixrep onepdf
repo: New folder
generated: 2026-08-11 01:03:16
files: 161

== tree ==

```
New folder/
|-- README.md
|-- mobile/
|   |-- App.tsx
|   |-- src/
|       |-- simAuth.ts
|-- src/
|   |-- db/
|   |   |-- index.ts
|   |   |-- schema.ts
|   |-- instrumentation.ts
|   |-- proxy.ts
|   |-- app/
|   |   |-- layout.tsx
|   |   |-- page.tsx
|   |   |-- admin/
|   |   |   |-- layout.tsx
|   |   |   |-- page.tsx
|   |   |   |-- activity/
|   |   |   |   |-- page.tsx
|   |   |   |-- backup/
|   |   |   |   |-- page.tsx
|   |   |   |-- columns/
|   |   |   |   |-- page.tsx
|   |   |   |-- homes/
|   |   |   |   |-- page.tsx
|   |   |   |-- leaves/
|   |   |   |   |-- page.tsx
|   |   |   |-- live/
|   |   |   |   |-- page.tsx
|   |   |   |-- map/
|   |   |   |   |-- page.tsx
|   |   |   |-- messengers/
|   |   |   |   |-- page.tsx
|   |   |   |-- notifications/
|   |   |   |   |-- page.tsx
|   |   |   |-- options/
|   |   |   |   |-- page.tsx
|   |   |   |-- reports/
|   |   |   |   |-- page.tsx
|   |   |   |-- sync/
|   |   |   |   |-- page.tsx
|   |   |   |-- targets/
|   |   |   |   |-- page.tsx
|   |   |   |-- trips/
|   |   |   |   |-- page.tsx
|   |   |   |-- users/
|   |   |   |   |-- page.tsx
|   |   |   |-- records/
|   |   |   |   |-- [type]/
|   |   |   |       |-- page.tsx
|   |   |-- apple-touch-icon.png/
|   |   |-- route.ts
|   |   |-- diagnostics/
|   |   |   |-- page.tsx
|   |   |-- install/
|   |   |   |-- page.tsx
|   |   |-- login/
|   |   |   |-- LoginForm.tsx
|   |   |   |-- page.tsx
|   |   |-- offline/
|   |   |   |-- page.tsx
|   |   |-- panel/
|   |   |   |-- layout.tsx
|   |   |   |-- page.tsx
|   |   |   |-- doctors/
|   |   |   |   |-- page.tsx
|   |   |   |-- home/
|   |   |   |   |-- page.tsx
|   |   |   |-- leaves/
|   |   |   |   |-- page.tsx
|   |   |   |-- map/
|   |   |   |   |-- page.tsx
|   |   |   |-- notifications/
|   |   |   |   |-- page.tsx
|   |   |   |-- options/
|   |   |   |   |-- page.tsx
|   |   |   |-- orders/
|   |   |   |   |-- page.tsx
```

```

|-- pharmacies/
|   |-- page.tsx
|-- reports/
|   |-- page.tsx
|-- trip/
|   |-- page.tsx
|-- ping/
|   |-- route.ts
|-- api/
|   |-- activity/
|   |   |-- route.ts
|   |-- attachments/
|   |   |-- route.ts
|   |   |-- [id]/
|   |   |   |-- route.ts
|   |-- backup/
|   |   |-- route.ts
|   |   |-- restore/
|   |   |   |-- route.ts
|   |-- export/
|   |   |-- route.ts
|   |-- geocode/
|   |   |-- route.ts
|   |-- health/
|   |   |-- route.ts
|   |-- homes/
|   |   |-- route.ts
|   |-- leaves/
|   |   |-- route.ts
|   |-- live/
|   |   |-- route.ts
|   |-- messengers/
|   |   |-- route.ts
|   |   |-- updates/
|   |   |   |-- route.ts
|   |-- notifications/
|   |   |-- route.ts
|   |-- options/
|   |   |-- route.ts
|   |-- ping/
|   |   |-- route.ts
|   |-- reports/
|   |   |-- route.ts
|   |-- roles/
|   |   |-- route.ts
|   |-- settings/
|   |   |-- route.ts
|   |-- source/
|   |   |-- route.ts
|   |-- targets/
|   |   |-- route.ts
|   |-- trips/
|   |   |-- route.ts
|   |   |-- points/
|   |   |   |-- route.ts
|   |-- users/
|   |   |-- route.ts
|   |-- auth/
|   |   |-- check-username/
|   |   |   |-- route.ts
|   |   |-- forgot/
|   |   |   |-- route.ts
|   |   |-- login/
|   |   |   |-- route.ts
|   |   |-- logout/
|   |   |   |-- route.ts
|   |   |-- me/
|   |   |   |-- route.ts
|   |   |-- otp/
|   |   |   |-- route.ts
|   |-- mobile/
|   |   |-- login-with-phone/
|   |   |   |-- route.ts
|   |   |-- nonce/
|   |   |   |-- route.ts
|   |-- records/
|   |   |-- [type]/
|   |   |   |-- route.ts
|   |   |-- send/
|   |   |   |-- route.ts
|   |   |-- lookup/
|   |   |   |-- route.ts
|-- sync/
|   |-- pull/
|   |   |-- route.ts

```

```

|-- push/
|   |-- route.ts
|   |-- run/
|   |   |-- route.ts
|   |   |-- status/
|   |   |   |-- route.ts
|   |-- tiles/
|   |   |-- [z]/
|   |   |   |-- [x]/
|   |   |   |   |-- [y]/
|   |   |   |   |-- route.ts
|   |-- map/
|   |   |-- tiles/
|   |   |   |-- [z]/
|   |   |   |   |-- [x]/
|   |   |   |   |   |-- [y]/
|   |   |   |   |   |-- route.ts
|-- icons/
|   |-- [file]/
|   |   |-- route.ts
|-- screenshots/
|   |-- [file]/
|   |   |-- route.ts
|-- globals.css
|-- components/
|   |-- BackButton.tsx
|   |-- Charts.tsx
|   |-- Combobox.tsx
|   |-- Confirm.tsx
|   |-- ConnectionStatus.tsx
|   |-- FileUploader.tsx
|   |-- JalaliDateInput.tsx
|   |-- LocationPicker.tsx
|   |-- MapBox.tsx
|   |-- NavButton.tsx
|   |-- NotificationBell.tsx
|   |-- RecordScreen.tsx
|   |-- ServiceWorker.tsx
|   |-- SessionProvider.tsx
|   |-- ShareBox.tsx
|   |-- Shell.tsx
|   |-- ui.tsx
|   |-- screens/
|   |   |-- ChartsPanel.tsx
|   |   |-- HomeScreen.tsx
|   |   |-- LeaveScreen.tsx
|   |   |-- MapExplorer.tsx
|   |   |-- NotificationScreen.tsx
|   |   |-- OptionsScreen.tsx
|   |   |-- ReportScreen.tsx
|   |   |-- TargetPanel.tsx
|-- lib/
|   |-- auth.ts
|   |-- bootstrap.ts
|   |-- constants.ts
|   |-- defaults.ts
|   |-- device.ts
|   |-- download.ts
|   |-- endpoints.ts
|   |-- excel.ts
|   |-- geo.ts
|   |-- iran.ts
|   |-- jalali.ts
|   |-- messaging.ts
|   |-- mobile-auth.ts
|   |-- notify.ts
|   |-- offline-session.ts
|   |-- otp.ts
|   |-- pwa-image.ts
|   |-- retry.ts
|   |-- settings-server.ts
|   |-- sort.ts
|   |-- sound.ts
|   |-- sync-config.ts
|   |-- sync.ts
|   |-- tracker.ts
|   |-- useLive.ts
|   |-- zip.ts
|-- package.json
|-- next-env.d.ts
|-- next.config.ts
|-- public/
|   |-- cloudflare-worker.js
|   |-- sw-template.js
|   |-- sw.js

```

```
| |-- manifest.webmanifest
| |-- data/
| |   |-- iran-provinces-lite.geojson
| |   |-- world-countries.geojson
|-- .env.ndcohub.example
|-- .gitlab-ci.yml
|-- drizzle.config.json
|-- eslint.config.mjs
|-- postcss.config.mjs
|-- render.yaml
|-- tsconfig.json
`-- scripts/
    |-- build-sw.mjs
    |-- generate-assets.mjs
    |-- simplify-geojson.mjs
    `-- start.mjs
```

== index ==

```
0001 README.md
0002 mobile/App.tsx
0003 src/db/index.ts
0004 package.json
0005 next-env.d.ts
0006 next.config.ts
0007 public/cloudflare-worker.js
0008 public/sw-template.js
0009 public/sw.js
0010 src/instrumentation.ts
0011 src/proxy.ts
0012 mobile/src/simAuth.ts
0013 src/app/layout.tsx
0014 src/app/page.tsx
0015 src/components/BackButton.tsx
0016 src/components/Charts.tsx
0017 src/components/Combobox.tsx
0018 src/components/Confirm.tsx
0019 src/components/ConnectionStatus.tsx
0020 src/components/FileUploader.tsx
0021 src/components/JalaliDateInput.tsx
0022 src/components/LocationPicker.tsx
0023 src/components/MapBox.tsx
0024 src/components/NavButton.tsx
0025 src/components/NotificationBell.tsx
0026 src/components/RecordScreen.tsx
0027 src/components/ServiceWorker.tsx
0028 src/components/SessionProvider.tsx
0029 src/components/ShareBox.tsx
0030 src/components/Shell.tsx
0031 src/components/ui.tsx
0032 src/db/schema.ts
0033 src/lib/auth.ts
0034 src/lib/bootstrap.ts
0035 src/lib/constants.ts
0036 src/lib/default.ts
0037 src/lib/device.ts
0038 src/lib/download.ts
0039 src/lib/endpoints.ts
0040 src/lib/excel.ts
0041 src/lib/geo.ts
0042 src/lib/iran.ts
0043 src/lib/jalali.ts
0044 src/lib/messaging.ts
0045 src/lib/mobile-auth.ts
0046 src/lib/notify.ts
0047 src/lib/offline-session.ts
0048 src/lib/otp.ts
0049 src/lib/pwa-image.ts
0050 src/lib/retry.ts
0051 src/lib/settings-server.ts
0052 src/lib/sort.ts
0053 src/lib/sound.ts
0054 src/lib/sync-config.ts
0055 src/lib/sync.ts
0056 src/lib/tracker.ts
0057 src/lib/useLive.ts
0058 src/lib/zip.ts
0059 src/app/admin/layout.tsx
0060 src/app/admin/page.tsx
0061 src/app/apple-touch-icon.png/route.ts
0062 src/app/diagnostics/page.tsx
0063 src/app/install/page.tsx
0064 src/app/login/LoginForm.tsx
0065 src/app/login/page.tsx
0066 src/app/offline/page.tsx
0067 src/app/panel/layout.tsx
```

0068 src/app/panel/page.tsx
0069 src/app/ping/route.ts
0070 src/components/screens/ChartsPanel.tsx
0071 src/components/screens/HomeScreen.tsx
0072 src/components/screens/LeaveScreen.tsx
0073 src/components/screens/MapExplorer.tsx
0074 src/components/screens/NotificationScreen.tsx
0075 src/components/screens/OptionsScreen.tsx
0076 src/components/screens/ReportScreen.tsx
0077 src/components/screens/TargetPanel.tsx
0078 src/app/admin/activity/page.tsx
0079 src/app/admin/backup/page.tsx
0080 src/app/admin/columns/page.tsx
0081 src/app/admin/homes/page.tsx
0082 src/app/admin/leaves/page.tsx
0083 src/app/admin/live/page.tsx
0084 src/app/admin/map/page.tsx
0085 src/app/admin/messengers/page.tsx
0086 src/app/admin/notifications/page.tsx
0087 src/app/admin/options/page.tsx
0088 src/app/admin/reports/page.tsx
0089 src/app/admin/sync/page.tsx
0090 src/app/admin/targets/page.tsx
0091 src/app/admin/trips/page.tsx
0092 src/app/admin/users/page.tsx
0093 src/app/api/activity/route.ts
0094 src/app/api/attachments/route.ts
0095 src/app/api/backup/route.ts
0096 src/app/api/export/route.ts
0097 src/app/api/geocode/route.ts
0098 src/app/api/health/route.ts
0099 src/app/api/homes/route.ts
0100 src/app/api/leaves/route.ts
0101 src/app/api/live/route.ts
0102 src/app/api/messengers/route.ts
0103 src/app/api/notifications/route.ts
0104 src/app/api/options/route.ts
0105 src/app/api/ping/route.ts
0106 src/app/api/reports/route.ts
0107 src/app/api/roles/route.ts
0108 src/app/api/settings/route.ts
0109 src/app/api/source/route.ts
0110 src/app/api/targets/route.ts
0111 src/app/api/trips/route.ts
0112 src/app/api/users/route.ts
0113 src/app/icons/[file]/route.ts
0114 src/app/panel/doctors/page.tsx
0115 src/app/panel/home/page.tsx
0116 src/app/panel/leaves/page.tsx
0117 src/app/panel/map/page.tsx
0118 src/app/panel/notifications/page.tsx
0119 src/app/panel/options/page.tsx
0120 src/app/panel/orders/page.tsx
0121 src/app/panel/pharmacies/page.tsx
0122 src/app/panel/reports/page.tsx
0123 src/app/panel/trip/page.tsx
0124 src/app/screenshots/[file]/route.ts
0125 src/app/admin/records/[type]/page.tsx
0126 src/app/api/attachments/[id]/route.ts
0127 src/app/api/auth/check-username/route.ts
0128 src/app/api/auth/forgot/route.ts
0129 src/app/api/auth/login/route.ts
0130 src/app/api/auth/logout/route.ts
0131 src/app/api/auth/me/route.ts
0132 src/app/api/auth/otp/route.ts
0133 src/app/api/backup/restore/route.ts
0134 src/app/api/messengers/updates/route.ts
0135 src/app/api/mobile/login-with-phone/route.ts
0136 src/app/api/mobile/nonce/route.ts
0137 src/app/api/records/[type]/route.ts
0138 src/app/api/records/lookup/route.ts
0139 src/app/api/sync/pull/route.ts
0140 src/app/api/sync/push/route.ts
0141 src/app/api/sync/run/route.ts
0142 src/app/api/sync/status/route.ts
0143 src/app/api/trips/points/route.ts
0144 src/app/api/records/[type]/send/route.ts
0145 src/app/api/tiles/[z]/[x]/[y]/route.ts
0146 src/app/api/map/tiles/[z]/[x]/[y]/route.ts
0147 .env.ndcohub.example
0148 .gitlab-ci.yml
0149 drizzle.config.json
0150 eslint.config.mjs
0151 postcss.config.mjs
0152 render.yaml

```
0153 tsconfig.json
0154 public/manifest.webmanifest
0155 scripts/build-sw.mjs
0156 scripts/generate-assets.mjs
0157 scripts/simplify-geojson.mjs
0158 scripts/start.mjs
0159 public/data/iran-provinces-lite.geojson
0160 public/data/world-countries.geojson
0161 src/app/globals.css
```

== files ==

@@ README.md markdown
۰۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰

۰۰۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰۰۰ ۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰۰۰۰ ۰۰۰۰.
Next.js 16 (App Router) + PostgreSQL (Drizzle ORM) + PWA (۰۰۰۰ ۰۰۰ ۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰ ۰ iOS).

۰۰۰۰۰۰ ۰۰۰۰۰۰۰۰

```
| ۰۰۰ | ۰۰۰ ۰۰۰۰۰۰ | ۰۰۰ ۰۰۰۰ | ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ |
| --- | --- | --- | --- |
| ۰۰۰۰ | `admin` | `admin1234` | `09120000000` |
| ۰۰۰۰۰۰۰ ۰۰۰۰۰ | `rep1` | `rep1234` | `09121111111` |
```

> ۰ ۰۰۰۰۰۰۰۰ ۰۰۰۰۰ `ADMIN\_PASSWORD` ۰ `ADMIN\_PHONE` ۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰ ۰۰۰۰۰ ۰۰۰۰
> (۰۰۰ ۰ ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰). ۰۰ ۰ ۰۰۰۰۰ ۰ ۰۰۰۰ «۰۰۰۰۰۰۰۰۰ ۰ ۰۰۰۰۰۰» ۰۰ ۰ ۰۰ ۰۰۰۰.

۰۰۰۰۰ ۰۰۰۰ (VS Code)

```
```bash
npm install
۰۰۰۰ .env
DATABASE_URL=postgresql://user:pass@host/db?sslmode=require
npm run dev
```
```

۰۰۰۰۰۰۰ ۰ ۰۰۰۰۰ ۰۰۰۰ **۰ ۰۰۰۰ ۰۰۰۰۰۰** ۰۰۰۰۰ ۰۰۰۰۰۰۰ (۰۰۰۰۰ ۰ ۰۰۰۰۰ migration ۰۰۰۰).

۰۰۰۰۰۰۰ ۰۰۰ Render + Neon

۰۰۰ ۰۰۰۰ «Could not find a production build» ۰۰۰۰۰۰۰۰۰

۰۰ ۰۰۰ Render ۰۰۰ ۰۰۰ ۰ ۰۰۰۰ ۰۰۰۰۰۰۰:

```
...
==> Running 'npm start'
...
```

۰ **۰۰۰ ۰۰ ۰ `npm run build` ۰۰۰۰ ۰۰۰۰۰** → ۰۰۰۰ ۰۰۰۰ **Build Command** ۰۰۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰.
`next start` ۰۰ ۰۰۰۰ ۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰۰۰۰ ۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰ `next` ۰۰۰۰ ۰۰۰۰۰.

۰۰۰۰۰۰ ۰۰۰۰ (۰ ۰۰۰۰۰۰)

Render → ۰۰۰۰۰ ۰۰۰ → **Settings** → ۰۰۰ **Build & Deploy**:

```
۰۰۰۰	۰۰۰۰۰ ۰۰۰۰
Root Directory	۰۰۰۰ (۰۰۰ ۰۰۰۰۰ ۰ ۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰)
Build Command	`npm install --include=dev && npm run build`
Start Command	`npm run start`
Health Check Path	`/api/health`
```

۰۰۰ **Environment** → ۰۰۰۰۰۰ ۰۰۰۰۰۰۰:

```
Key	Value
`DATABASE_URL`	۰۰۰۰ ۰۰۰۰۰ Neon ۰ ۰۰۰۰ `postgresql://user:pass@ep-xxx.neon.tech/neondb?sslmode=require`
`APP_SECRET`	۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰
`NODE_VERSION`	`20.18.0`
`NPM_CONFIG_PRODUCTION`	`false`
```

۰۰ ۰۰۰ **Manual Deploy → Clear build cache & deploy**.

> ۰۰۰۰: `--include=dev` ۰۰۰۰ ۰۰ ۰۰۰ Render ۰۰۰۰۰ `NODE\_ENV=production` ۰۰۰۰۰۰۰۰ ۰ ۰ ۰۰۰ ۰۰۰۰
> `npm install` ۰۰۰۰۰۰۰۰ toolchain (typescript ۰ tailwind) ۰ ۰۰۰ ۰۰۰۰۰۰۰ ۰ ۰۰۰۰ ۰۰۰۰۰۰۰.

۰۰۰۰۰۰ ۰۰۰۰۰ (۰۰۰ ۰۰۰ ۰ ۰ Build Command ۰ ۰۰۰۰۰۰ ۰۰۰۰۰)

۰۰۰۰۰۰۰ `npm start` ۰۰۰ ۰۰۰۰۰ ۰ ۰ `scripts/start.mjs` ۰۰۰ ۰۰۰ ۰۰۰: ۰۰ ۰۰۰۰۰۰ `next` ۰۰۰۰۰ ۰۰۰۰۰۰
۰۰۰۰۰ ۰۰ ۰ ۰۰۰۰۰ ۰۰۰۰ ۰ ۰۰۰ `next build` ۰۰۰۰۰۰۰ ۰ ۰۰۰ ۰۰۰ `PORT` ۰۰۰۰۰ ۰ ۰ Render ۰۰۰۰ ۰۰۰۰۰۰.
۰ ۰۰۰ ۰ ۰۰۰۰۰۰۰ ۰۰۰۰ ۰ ۰۰۰۰۰ live ۰۰۰۰۰۰ (۰۰۰ ۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰ ۰۰).

۰۰۰۰۰۰۰ ۰۰ Blueprint

0000000000 000 000000 HTML 00 00 000000 000 **000000000 000000000000 (0.0 0000000) 00 00**.
000 000000 00 00 React 0000 HTML 0000 00000000 JS 00000000 0000 000 0000000 0000
0000000 00000000 000 000 0000000 0000000 00000000 000000000 → 0000 0000.

0000000 00000000 `scripts/build-sw.mjs` 00 00 00 build 0000 00000000 000000 000000
000 0000000000 `/\_next/static` 00 000000000 0 00 000000000000 000 0000000. 0000 **00 0000
00000000 + 00 0000** 0000000 0000 0000000 00000000.

0000 000000000
0000 `diagnostics` — 00 000000 0000: 000 000000 0000000000000 000000 0000000000 00000000
0000 0000 000000 0000000000 0000 00000000 000000 00000000 GPS 0 00 0000000.
0000 «000000000 0 000 00000» 0 «000 000000» 00 0000.

0 00000 000000000 — 0000 0000

0000 0000 **00 00000 00000000** 00 0000 «00000000» 000 00 000 00000000 00 000000 0000000:

```
0000	00000
`0000 00000`	000 0000000 000000 00000000
`00000 00000`	00000 00000000 0000 00 00000 00000000 0000 000 0000
`000 0000` (00000000)	00000 00 0000000 0000 00 00000 0000 000 00000000 0000 0000 000000
`00000 000000`	00000000000000000000: 00 00 000000 000000 00000000 00000 0000 00000 000 000
```

0000 00 0000 «000000000 0000» 000000000 000000 000000 0000000 0000.

0 000000000000 0000 00 000000 0000 0000

0) Keep-Alive — 00000000 00 00000000 000000
00 0000 0000 000000 0000 0 0000 0000 00 00000000:

```
...  
GET /ping → 200 "OK" (0000 0000 ~0ms)  
GET /ping?json → {"status":"OK","uptimeMs":...}  
GET /api/ping → 0000  
HEAD /ping → 200  
...
```

000000 UptimeRobot: New Monitor → HTTP(s) → URL: `https://<app>.onrender.com/ping`
→ Interval: **0 00000** 00 000 000 000000 0000000 Render 0000 000000 00000000.

0) Retry 00 Exponential Backoff + Jitter
000000 `src/lib/retry.ts`:

```
API	0000000
`withRetry(fn, opts)`	000000 00 0000000 00 0000 0000
`withRetrySafe(fn, fallback)`	0000 000 00 000000 00000000
`fetchWithRetry(url, init)`	00000000 HTTP 000000 (00000000000000 0000000 00000000)
`dbRetry(fn, label)`	000000 00000000 00 0000 0000
`dbTransaction(fn)`	0000000 00 BEGIN/COMMIT/ROLLBACK 0 retry
```

000000 000000 (Full Jitter): 0s → 0s → 0s → 0s → 00s` 00 000 **00 000000**0
00 000000 000000000 0000000 000 000 00 00 00000 000000 0000000 0000000 00 000000 00000000 000000 00000.

0000000 0000 000000: `ECONNRESET`, `ETIMEDOUT`, `ECONNREFUSED`, `EAI\_AGAIN`, `EPIPE` ...
00000000 PostgreSQL `08xxx` (00000)0 `53xxx` (00000)0 `57Pxx` (000000)0 `40001/40P01` (00000)0
000000000 HTTP `5xx`, `429`, `408` 0 0000 «endpoint is disabled» 000000 Neon.

0000000 000000000 000000 (000000 000000 000000000): `4xx`0 000 0000 0000 `23505`0 `AbortError`.

0) 000000 000000000 Neon
`src/db/index.ts` 0000 000000 00 0000000 000000 00000000:

```
...  
000000 : postgresql://u:p@ep-cool-fire-123.eu-central-1.aws.neon.tech/neondb  
000000 : postgresql://u:p@ep-cool-fire-123-pooler.eu-central-1.aws.neon.tech/neondb  
          ?sslmode=require&connect_timeout=15&application_name=sabt-etelaat-kol  
...
```

- 000000 0000000 00 endpoint 000 **`-pooler`** (PgBouncer)
- **`connect\_timeout=15`** 0000 0000 000000 000 000000000
- `statement\_timeout` 0 `query\_timeout` 000000 00 000000
- `keepAlive` 000000 `max=8`, `min=0`, `idleTimeout=30s`
- 00000000 `pool.on("error")` 00 0000 000000 00000000 00000000 00 00 00 0000000000

`GET /api/health` 000000 0000 00 00000000000000:
`{"ok":true,"db":true,"latencyMs":8,"pooler":true,"connectTimeoutSec":15}`

0 000 «This page couldn't load» 0 0000 00000000 0000000

0000 0000
0000000000000 0000 00000000 **00-000** 0000 000 000000000 0000000 000000 00 0000 00 000000
000000 0000 0000000 000 000000 0 000000 00 0000 00000000 000 00000000.


```
### ❶) HTML 0000 00 00000000
00 `next.config.ts` 0000 000 00000000:
`Cache-Control: no-cache, no-store, must-revalidate, max-age=0` + `Pragma: no-cache` + `Expires: 0`
00 00000000 `/_next/static/*` 00 `immutable` 00000000 (000000 0000000 000).
```

```
### ❷) 0000 0000 0000000 0000 0000000
000000000 `ConnectionStatus`: 0000 000000 0000 00 0000
**«0000000 00 0000 0000000 0000 — 000000 000000 000000000 000 00 000000 0000»**
000000000 000000000 «000000 0000» 0 «000000000» 0 «0000». 000000 0000000000 000 000 00 000000 000.
000000 0000000 00 **000000 000000 00000** 000000 00000000 (00 `navigator.onLine` 00 00 000000 000000000000000 000)
0 000 00 00 00 00000 000000 000000 0000000.
```

```
### ❸) 000000000000: 0000000 «0000-000»
```js
if (req.mode === "navigate") {
 0 0000 00 0000 0 000000 0 cache: "no-store"
 0 000 0000 0000 → 0000 0000 00 00 (0000000 0000 000 0000000)
 0 000 0000 → 000000 00 (/panel 00 /) 00 0000000000 0000000 000000 0000
 0 000000 000 → 0000 0000000 00 0000 0000 0000
}
```
```

000000000000 RSC (000000 00000 Next) 00 0000000 00000000 00 0000000000 000 000000 0000000 000 0000.

```
### ❹) 000 0000000 + 000000 0000
- 00000000 000000000 00 00 (000000 00000 000000000).
- 00 000/0000000 00 **IndexedDB** 00 0000000 0 0000
  «00000000 000000 00 0 00 00 000 000 00000000 000000 0000000» 000000 0000 0000000.
- 0000 «N 0000 00 00000000 000000» 00 0000 **«000000 000000»** 0000 000000 0000.
- 000000 0000000 00 Background Sync 0000000 0000000000 000000 00000 0 00 00 000000.
- 0000 `offline` 0000 00000000 0000: 00 0 00000 0000000 000000 0000000 0 0000
  «000000 00 0000 0000000» 000000 00 0000 000 0000000.
```

0 000000 000000 0000000 000000 00 00000000000000

```
> **00000000 000000 Laravel:** 000000 `bot-generator-bale-telegram-laravel-10` 00 PHP/MySQL 000000 000
> 0 0000 «00000 0000» 0000 00 000000 000000 000000. 0000000000000 00 000000 PHP 000000000 000 0000
> 000000 00 0000 000000 0 000000000 000000 0000. 0000000000 0000 00000 (Bot API 0000000/000 + 000000000)
> 000000000 0000 00 TypeScript 000000 000 000000 000000 0 000000000 000000 000000000 000.
```

```
### 000 00000000 — 0000 `messengers`
0000	00000
`platform`	telegram / bale / eitaa / whatsapp
`label`	000000 0000000 0000 000000000
`targetType`	group / channel / phone
`target`	chat_id 0000 000000 00 000000 0000
`token`	0000 0000 00 0000 API
`provider`	000000 0000 0000000: whatsiplus / ultramsg / cloudapi / custom
`apiUrl`	0000 00000000 00 `{phone}` 0 `{text}` 0 `{token}`
`enabled`	0000/00000000 0000 000 0000
`lastStatus`, `lastOkAt`, `lastErrorAt`	000000 000000 000000 0000 000000 00 000
```

0000 `message\_logs` 00000000 0000 00 000000 (0000/0000000 + 000 000) 00 000 00000000.

```
### 000000 0000 (`src/lib/messaging.ts`)
```ts
formatOrderMessage(order) // 0000 000 000000
sendOne(target, text) // 000000 00 00 0000 (0000000 → 0000000)
dispatchText(text, orderId?) // 000000 00 000 000000000 0000
dispatchOrder(order) // 000000 00 000000
fetchUpdates(platform, token) // 000 0000000 chat_id
```
```

```
### 0000000 000
- 00 00000000 00 **0 000** 00 000000 0 → 0 → 0 000000 (Full Jitter) 000000 0000000.
- 0000000000 00 `Promise.allSettled` 000000 000000 00000000000 **00000 000 0000 00 000000 000000000**.
- 000 000000 0000000 0000 00000000 0000000 00 **00000000 Cloudflare** 0000 00000000.
- 0000 0000000000000 00 00 000 00 00 000 00000000 (000 whatsiplus) 00 000 **00000** 000000 0000000.
```

```
### 00000000 0000000 0000
00000000000	0000	0000
**0000000**	@BotFather → `/newbot`	chat_id 000000 0000 00 `-100...`
**0000**	@BotFather 0000 0000 → `/newbot`	chat_id 00000
**000000**	00000000 00 `eitaayar.ir`	0000 000000 00 `@`
**00000000**	0000 API 00 `whatsiplus.ir` (00 UltraMsg / Cloud API)	000000 0000
```

0000 0000000 0 000 0000 **«0 0000000 0000000 chat_id»** 00 000 0000 0000: 0000 00 00 0000 000
000000 00 0000 000000000 0 0000 00 000000 00 000000 000000 00 00 0000 00000000 000.

0 0000 00 0000000000 0000000 00 0000000000

```
> **00000000 000:** `getLine1Number()` 00000 MSISDN 00 00 00000000000 00000000000 000 000000 000 0
> 000000000 000 0000 00 0000 00000000000 → 000000000 `null` 000000000000. 00 iOS 00 0000 API00 0000 000000.
> 0000000000 00000000 000000000 000000 00.
```

```
### 00000000 API
0000	000
`POST /api/mobile/nonce`	0000 nonce 0000000000 (00 Replay)
`POST /api/mobile/login-with-phone`	0000 00 00000 0000000000 + 00000 HMAC
`POST /api/auth/otp`	0000 00000000 00 00 000000
```

```
### 000000
1. **HMAC-SHA256** 000 `nonce|timestamp|deviceId|phone|simFingerprint` 00 `MOBILE_APP_SECRET`
2. **nonce 0000000000** 00 000000 0 00000 — 0000 000 00 0000000
3. **0000 000000** 00 0000 0 00000 0000000 0000
4. ICCID/IMSI 0000 000 000000 0000000000 000 **00 SHA-256** 00
```

```
### 000000 0000 (00000000)
`
`
`
200 TOKEN ← 0000 0000
401 BAD_SIGNATURE ← 000000 0000
401 BAD_NONCE ← Replay Attack
404 PHONE_NOT_REGISTERED ← 00000 00 000000 0000
422 SIM_NUMBER_UNAVAILABLE ← 000000 000 000 0000 → 0000 00 0000000
403 DEVICE_MISMATCH ← 0000000000 00 0000 000000
403 SIM_CHANGED ← 0000000000 0000 000 000
403 PASSWORD_LOGIN_REQUIRED ← 000000 000000 0000 000 000000 000000 000
`
`
`
```

```
### 00 0000000000
0000 `mobile/` 0000 `App.tsx` 0 `src/simAuth.ts` (React Native) 0 000000 Flutter
000000000 00000000 0000 0000 00000000 0 000 000. 000000 `MOBILE_APP_SECRET` 0000 00
Render 0 00 000000 0000.
```

```
## 0 000000 000000 000000 0000000 0 0000000 0000000
```

```
### 0) 000000 00 000000 000000000 0 0000
00000 `targets` 00 0000000000 `userId`,`period` (000 00000), `productKey`,
`quantity`,`priceDistributor`,`pricePharmacy`.
```

```
- 0000 **0000 → 0 000000 0000** 00000000 00000000 0 000000 0000 000000 0 0000 0000/0000000000
0000 00 000000 00000000 00000000 0000 00000000 0 0000 «000000 0000000000 0000 0000 000000000000».
- `GET /api/targets` 00000000 00 000000000000000: 000000 0000 0000 000000000000 (0000 000000000000 0000
0000)0 000000000000 0000 0000 0 0000 000000.
- **00000 00 0000 00 000 000000** 0000000 0000 0 00000000000 000000 0000 0000000 0 00 0000 0000000
00000000000 000000000 0000 00 0000000.
- 0000 «0000000 000000» 00 0000 0000 000000000 0 000000 0000000.
```

```
### 0) 000000 000000 00000000
`GET /api/records/lookup?type=pharmacies|doctors&name=...&phone=...`
00000000 00000000000000 00000000 (0000 «00000000000/00000»0 0000000000000 0/0 00000) 00
«00000000000 000» 0 «0000» 000000 000000 0000 00000. 000000 0000 0000 0000 000000 000
00 0000000 000000 0000 0 000 00000000000 0000 0000000 0 000 00 000 00 00000000 000000 0000000.
```

```
### 0) 0000000 0000000 000 000000
00 00000 000 00000000000 00 000 000000:
- 00000 0000 «000000 0000 00000000000» 00 000000 000000000000 000000 0000 00000 0 00000000
000000 0000000 000000 0 000 000000 000000000000.
- 00000 **«0 0000000 0000000 00000000»** 00 000000 0000000 000000 000000 000000000
0000 0 00000000 00 00 000000 000000 00 0000000.
```

```
## 0 0000000 00 000000: Render 0 NdcoHub.ir
```

```
0000000 000 00 000000 0000 0000000 0 00000000 000 000000 **00000000** 000000 000000000:
```

```
0000	000000	000
NdcoHub (0000000000)	`https://ndcohub.ir`	0000 0000 000000 — 0000 0 0000 000000
Render	`https://namayandeelmi-javad.onrender.com`	0000 000000000 0000 00 0000
```

```
### 000) 0000000 0000000 0000 00 000 000000
`src/lib/endpoints.ts` — 0000000 0000000000000 000000 00 000000 00 0000000 0000000 0 000
0000000000 000 0000 0000 0000 0000000 **0000000 000 0000 0000 000000 0000000** 00
0000 0000 000000 00 000000 00000000.
```

```
### 0) 000000000000 00000
00 00000 00 0000 0000 0000:
- `uid` — 000000 00000000 (UUID) 00 000000000000 0000 00 0000 000000 000000
- `updated_at` — 00 000000 00000000 0000000 0000000 0000000
- `origin` — 000 000000 00 000000 00 00 000000 000
```

```
**000000** 00 0 000000 (0 00 00000 0000) 00 0000 000000000 0000 00 `pull` 0 00000000
000 00 `push` 0000000. 00 000000: **Last-Write-Wins** 00 0000 `updated_at`.
```

```
**00 0000000000:** 000000 000000 0000000000 000000 0000 `sek.sync=on` 000000 0000000 00
000000 0000 0000 000000 00 000 00000 0000000000 000000 0000000000 00000000 «000000 000000»
0000 000000000. 0000 `updated_at < excluded.updated_at` 00 0000000000 000000 00 00 0000000.
```

```
### □) 00000000 000000000000
0000	000
`POST /api/sync/pull`	0000 00000000 00 00 00000000 (00 000000 HMAC)
`POST /api/sync/push`	0000 00000000 000 00 000000000
`POST /api/sync/run`	00000 0000/0000 (`x-sync-key`)
`GET /api/sync/status`	000000 00000000000 0 00000
```

0000 **0000 → 0 00000000000 000000** 00000 00 00000 0000 000000/000000
0000 «00000000000 00000»□ «000000000000 0000» 0 «0000 0000 0000000».

□) 000000000 000000

0000 Render:

```
```\n\nNODE_NAME=render\nSYNC_SECRET=<0000 00000 000 00 0000>\nSYNC_PEERS=ndcohub|https://ndcohub.ir\nSYNC_INTERVAL_MINUTES=5\nNEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeeelmi-javad.onrender.com\nPUBLIC_BASE_URL=https://namayandeeelmi-javad.onrender.com\n```\n\n
```

\*\*0000 NdcoHub\*\* (00000 `env.ndcohub.example` 00 000 0000):

```
```\n\nNODE_NAME=ndcohub\nSYNC_SECRET=<0000 0000>\nSYNC_PEERS=render|https://namayandeeelmi-javad.onrender.com\nNEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeeelmi-javad.onrender.com\nPUBLIC_BASE_URL=https://ndcohub.ir\n```\n\n
```

□) CI/CD 000 00 00 0000
-`.github/workflows/deploy.yml` — 0000000 000000 **00000000000 0000 GitLab**□ 0000000 000 00 00 0000
-`.gitlab-ci.yml` — 0000 000000 00 **00000000000 0000 GitHub**

000000 0000000000: `GITLAB\_REPO\_URL`, `GITLAB\_TOKEN`, `RENDER\_DEPLOY\_HOOK`,
`NDCOHUB\_DEPLOY\_HOOK`, ``SYNC_SECRET` .

0 0000000 00 00 0000 00 GitHub / GitLab

000 `git status` 000 000000 000 «nothing to commit» 00000000 0000 0000000000 0000 0000
000 000000 000 0000000 (0000 000 0000 0000000 0000000000 00 000 00000 0000 0000).

0000000: 0000000 0000 00 000 0000000

1. 00 0000 **00000** 0000 0000.
2. 00 **00000000 → 0 000000000000 0000000** 000000.
3. 0000 **«□ 0000000 0000 0000 (ZIP)»** 00 000000.
4. 0000000 ZIP 00 00 0000 000000 000 000000 000 0000 (00000000 0000000000 000000).
5. 000:

```
```\nbash\n\n git add -A\n git status          # 0000 0000000000 000000 000000 00 0000 0000000\n git commit -m "000000000000 0000 0000000"\n git push origin main\n git push gitlab main      # 000 00000 GitLab 00 000000\n\n```\n\n
```

> 000000 00000 `node\_modules`□ `next` □ `env` 0000 (000000 00000 0 00000000 000).  
> 00000000 000000000000 00000 00000 `HOW-TO-UPDATE.md` 00 0000 ZIP 0000 0000.

00000 API: `GET /api/source` (000 00000) — □ `GET /api/source?list=1` 0000 0000 000000 000000000.

## 0 00 000000 000

### □) 000000 00000 0000000000 0000000/0000  
00000000 0000000000 00000 0000000 00 00000000 000000 0000000. 00000 00000 000000  
(0000 «0000000») 00 0000000 000000 0000000 00 00000 00000000000 00000 000000 0  
00000 000000 00000 00000000.

\*\*00000000\*\* 0000000000 000000000000 000000 0000000000 00 00000000000000 00000000 \*\*0000000\*\*  
0000000000 00 00000 00000 0000000 00 0000000 000000 0000000 000000000 (00000 00 000 00000  
0000000 00 000000000 00000 00000).

### □) 0000 000000 000 00000  
00000 `/api/geocode` 000000 000 (0000000 000 00000 00 CORS □ 0000000000 0000000000 000000):

- 0000000 00000000 00 00 00000 00000 + 00000 «□ 000000» + 00000 Enter
- 00000 □ 00000 0000000 00 0000 00000 000000 (0000 0000000000/00000 + 0000)
- 0000000000 00 00000 00000000 00000000: `35.7219, 51.4089`
- 00000 «□ 00000000 00000 000 00000» (00000000 0000000)
- 00 000000000000000 0000 00 00 (Photon 0000 Nominatim) 00 0000000 0000000 00000000

```
> 0000 00000000: `Number(null)` 00 00000000000000 000000 **0000** 00000 0000000000 00
> 00000000 00000 00000000000 «00000000 000000 00 00000000 0000» 00000 000000 0 0000000 00000
> 000000000000000. 00000 00000 0000000000 0000000000 00000000 00000000.
```

```
0 00000 00000 + 0000 00000000 00000
```

```
00000000000 00000000000
00 00000 00000 00 00000 00000 000000000000 00000 0000000 00000:
```

```
0000	000
`src/lib/geo.ts`	00000000 00 00000000 00 000 00000 0 00 000000 000000 + 000000 `resolveArea`, `provinceOf`, `distanceKm`
`src/components/screens/MapExplorer.tsx`	00000 00000 00 000000 000000/0000/00000000 0 000000
`src/app/panel/map` 0 `src/app/admin/map`	000000 «0 0000 00000»
```

```
`fetchJson` 0 `fetchRows` 00 00 `src/lib/useLive.ts` 000000 00000 0 `MapBox`
00000 00 `MapArea` 0000000000 00000000 (000000 00000 000 00000000 000000000000).
```

```
000 00 000000 00000
```

```
0) `Custom Cache-Control headers detected for /_next/static`
000 00000 000 `/_next/static` 000 00 (000 Next.js 00 00 `immutable` 000 00000000)
0 000000 `no-store` 00 000000 `/:path((?!_next/).*)` 000 00 000000 000000 00.
```

```
0) `Encountered unexpected file in NFT list`
0000 `api/source` 00 `fs` 0 `path` 000 000000000 import 00000 000000 00000 000000
000000 00000000 Turbopack 00 000000 00 0000000000 00000 0000. 00000 000 0000000000
0000000000 **00000000000 0 000 00 00000 00000** 00000000000 0000000000.
```

```
000000: `npm run build` 00000 000 000000 00 000000 00000 00000000.
```

```
@@ mobile/App.tsx typescript
/**
 * 00000 00000 00000000000 00000000 «0000 0000000000 00»
 * React Native — 00 00000000 00000000 0000000000 0 000000 00 00 00000000
 */
import React, { useEffect, useState } from "react";
import {
 ActivityIndicator,
 Alert,
 I18nManager,
 SafeAreaView,
 ScrollView,
 StyleSheet,
 Text,
 TextInput,
 TouchableOpacity,
 View,
} from "react-native";
import { WebView } from "react-native-webview";
import {
 API_BASE,
 getSavedToken,
 loginWithSim,
 readSimInfo,
 requestOtp,
 verifyOtp,
 type SimInfo,
} from "../src/simAuth";

I18nManager.allowRTL(true);
I18nManager.forceRTL(true);

type Screen = "loading" | "sim" | "credentials" | "otp" | "app";
```

```
export default function App() {
 const [screen, setScreen] = useState<Screen>("loading");
 const [sim, setSim] = useState<SimInfo | null>(null);
 const [msg, setMsg] = useState("");
 const [busy, setBusy] = useState(false);
 const [token, setToken] = useState<string | null>(null);

 const [username, setUsername] = useState("");
 const [password, setPassword] = useState("");
 const [code, setCode] = useState("");
 const [masked, setMasked] = useState("");

 /* ----- 00000: 00000 00000 00000 00000000 00 0000000000 ----- */
 useEffect(() => {
 (async () => {
 const saved = await getSavedToken();
 if (saved) {
 setToken(saved);
 setScreen("app");
 return;
 }
 })();
 });
```

```

 }
 const info = await readSimInfo();
 setSim(info);

 if (!info.hasSim) {
 setMsg("☹️ 📶📶📶📶📶📶 📶 📶📶 📶📶📶📶 📶📶. 📶📶📶 📶📶📶📶📶 📶 📶📶📶 📶📶📶.");
 setScreen("credentials");
 return;
 }

 if (!info.phoneNumber) {
 // 📶📶📶📶 📶📶📶 📶 📶📶 📶📶📶📶 📶📶📶 → 📶📶 📶 📶📶📶
 setMsg(
`📶📶📶📶 ${info.carrier || ""} 📶📶📶📶 📶📶 📶📶 📶📶📶📶 📶📶📶 📶 📶📶 📶📶📶📶 📶📶📶 📶📶📶 📶📶.\n` +
"📶📶 📶📶📶 📶 📶📶 📶📶📶 📶📶 📶📶 📶 📶 📶📶📶 📶📶📶📶 📶📶📶 📶📶.",
);
 setScreen("credentials");
 return;
 }

 setBusy(true);
 const res = await loginWithSim();
 setBusy(false);

 if (res.status === "success") {
 setToken(res.token);
 setScreen("app");
 } else if (res.status === "need-otp") {
 setMsg(res.message);
 setMasked(res.masked ?? "");
 setScreen("credentials");
 } else {
 setMsg(res.message);
 setScreen("credentials");
 }
 })();
}, []);

const doRequestOtp = async () => {
if(!username || !password) return Alert.alert("☹️", "📶📶 📶📶📶 📶 📶📶 📶📶 📶 📶📶 📶📶");
 setBusy(true);
 const r = await requestOtp(username, password);
 setBusy(false);
 setMsg(r.message);
 if (r.masked) setMasked(r.masked);
 if (r.ok) setScreen("otp");
};

const doVerifyOtp = async () => {
 setBusy(true);
 const r = await verifyOtp(username, password, code);
 setBusy(false);
 if (r.ok && r.token) {
 setToken(r.token);
 setScreen("app");
 } else setMsg(r.message ?? "☹️ 📶📶📶📶 📶📶");
};

/* ----- 📶📶📶📶 📶📶 📶📶📶 WebView 📶 📶📶 ----- */
if (screen === "app" && token) {
 return (
 <SafeAreaView style={styles.flex}>
 <WebView
 source={{ uri: `${API_BASE}/panel` }}
 injectedJavaScriptBeforeContentLoaded={`
 try {
 sessionStorage.setItem('sek_token', ${JSON.stringify(token)});
 localStorage.setItem('sek_token_p', ${JSON.stringify(token)});
 } catch (e) {}
 `}
 true;
 </>
 <startInLoadingState
 geolocationEnabled
 domStorageEnabled
 javaScriptEnabled
 allowsBackForwardNavigationGestures
 </SafeAreaView>
);
}

if (screen === "loading" || busy) {
 return (
 <SafeAreaView style={[styles.flex, styles.center]}>
 <ActivityIndicator size="large" color="#0f766e" />
);
}

```

```

<Text style={styles.hint}>□□ □□□ □□□□□□□□□□...</Text>
</SafeAreaView>
);
}

return (
 <SafeAreaView style={styles.flex}>
 <ScrollView contentContainerStyle={styles.container}>
 <Text style={styles.title}>□□□ □□□□□□□□ □□</Text>
 <Text style={styles.sub}>□□□□ □□□□□□□□ □□□□</Text>

 {sim ? (
 <View style={styles.simBox}>
 <Text style={styles.simText}>
 {sim.hasSim ? `□ : "□□" □□□□□□□□: {sim.carrier} || "□□□□□□□□"
 {sim.phoneNumber ? `— ${sim.phoneNumber}` : " (□□□□□□ □□□ □□□□□□□□□ □□□□□ □□□□)"
 </Text>
 </View>
) : null}

 {msg ? <Text style={styles.msg}>{msg}</Text> : null}

 {screen === "credentials" ? (
 <>
 <TextInput
 style={styles.input}
 placeholder="□□□ □□□□□□"
 value={username}
 onChangeText={setUsername}
 autoCapitalize="none"
 />
 <TextInput
 style={styles.input}
 placeholder="□□□ □□□□"
 value={password}
 onChangeText={setPassword}
 secureTextEntry
 />
 <TouchableOpacity style={styles.btn} onPress={doRequestOtp}>
 <Text style={styles.btnText}>□ □□□□□ □□ □□□□ □□□□□□</Text>
 </TouchableOpacity>
 <TouchableOpacity
 style={[styles.btn, styles.btnGhost]}
 onPress={async () => {
 setBusy(true);
 const r = await loginWithSim();
 setBusy(false);
 if (r.status === "success") {
 setToken(r.token);
 setScreen("app");
 } else setMsg(r.message);
 }}
 >
 <Text style={styles.btnGhostText}>□ □□□□ □□□□ □□ □□□□□□□□</Text>
 </TouchableOpacity>
 </>
) : null}

 {screen === "otp" ? (
 <>
 <Text style={styles.hint}>□□ □□□□□□□□ □□ {masked} □□ □□□□ □□□□</Text>
 <TextInput
 style={[styles.input, styles.code]}
 placeholder="— — — — —"
 value={code}
 onChangeText={setCode}
 keyboardType="number-pad"
 maxLength={6}
 />
 <TouchableOpacity style={styles.btn} onPress={doVerifyOtp}>
 <Text style={styles.btnText}>□ □□□□□ □ □□□□</Text>
 </TouchableOpacity>
 </>
) : null}
 </ScrollView>
 </SafeAreaView>
);
}

const styles = StyleSheet.create({
 flex: { flex: 1, backgroundColor: "#f1f5f9" },
 center: { justifyContent: "center", alignItems: "center" },
 container: { padding: 24, gap: 12 },
 title: { fontSize: 24, fontWeight: "bold", color: "#0f766e", textAlign: "center", marginTop: 40 },
 sub: { fontSize: 14, color: "#64748b", textAlign: "center", marginBottom: 12 },

```

```

simBox: { backgroundColor: "#fff", borderRadius: 14, padding: 12, borderWidth: 1, borderColor: "#e2e8f0" },
simText: { fontSize: 12, color: "#334155", textAlign: "right" },
msg: { fontSize: 12, color: "#b45309", backgroundColor: "#fffbeb", padding: 12, borderRadius: 12, textAlign: "right",
lineHeight: 22 },
input: { backgroundColor: "#fff", borderRadius: 14, borderWidth: 1, borderColor: "#cbd5e1", padding: 14, fontSize: 14,
textAlign: "right" },
code: { textAlign: "center", fontSize: 22, letterSpacing: 8 },
btn: { backgroundColor: "#0f766e", borderRadius: 14, padding: 15, alignItems: "center" },
btnText: { color: "#fff", fontWeight: "bold", fontSize: 14 },
btnGhost: { backgroundColor: "#fff", borderWidth: 1, borderColor: "#0f766e" },
btnGhostText: { color: "#0f766e", fontWeight: "bold", fontSize: 14 },
hint: { fontSize: 12, color: "#64748b", textAlign: "center", marginTop: 8 },
});

```

```

@@ src/db/index.ts typescript

```

```

import { drizzle } from "drizzle-orm/node-postgres";
import { Pool, type PoolClient, type PoolConfig } from "pg";
import { isTransientError, withRetry, type RetryOptions } from "@lib/retry";

```

```

/* =====
* 数据库 数据库 数据库 PostgreSQL (Neon / Render / 数据库)
*
* 1) 数据库 数据库 数据库 endpoint 数据库 «pooler» 数据库 Neon
* 2) connect_timeout=15 数据库 数据库 数据库 数据库 数据库
* 3) Retry 数据库 Exponential Backoff 数据库 数据库 数据库
* ===== */

```

```

const rawUrl = process.env.DATABASE_URL;
export const hasDatabaseUrl = Boolean(rawUrl);

```

```

if (!rawUrl) {
 console.error("DATABASE_URL 数据库 数据库 数据库. 数据库 Render → Environment 数据库 数据库 Neon 数据库 数据库.");
}

```

```

const FALLBACK_URL = "postgresql://invalid:invalid@127.0.0.1:1/invalid";

```

```

/** 数据库 数据库 数据库 数据库 数据库 数据库 */

```

```

function isLocalHost(url: string) {
 return /^@(localhost|127\.0\.0\.1|host\.docker\.internal)/.test(url);
}

```

```

/**

```

```

* 数据库 数据库 数据库 数据库 数据库:
* - 数据库 Neon 数据库 数据库 数据库 `pooler` 数据库 数据库 (PgBouncer 数据库 数据库 数据库)
* - `sslmode=require` 数据库 数据库 数据库 数据库
* - `connect_timeout=15` 数据库 数据库 数据库 数据库 数据库
* - `application_name` 数据库 数据库 数据库 数据库 Neon
*/

```

```

export function normalizeConnectionString(input: string): { url: string; usedPooler: boolean; local: boolean } {
 const local = isLocalHost(input);
 let usedPooler = false;

```

```

 let parsed: URL;
 try {
 parsed = new URL(input);
 } catch {
 return { url: input, usedPooler: false, local };
 }

```

```

 if (!local) {
 // Neon: ep-xxx.region.aws.neon.tech → ep-xxx-pooler.region.aws.neon.tech
 const host = parsed.hostname;
 if (/\.neon\.tech$/i.test(host) && !host.includes("-pooler")) {
 const [first, ...rest] = host.split(".");
 parsed.hostname = [`${first}-pooler`, ...rest].join(".");
 usedPooler = true;
 } else if (host.includes("-pooler")) {
 usedPooler = true;
 }
 if (!parsed.searchParams.has("sslmode")) parsed.searchParams.set("sslmode", "require");
 }

```

```

// 数据库 数据库 数据库 数据库 — 数据库 数据库 数据库 cold start
if (!parsed.searchParams.has("connect_timeout")) parsed.searchParams.set("connect_timeout", "15");
if (!parsed.searchParams.has("application_name")) parsed.searchParams.set("application_name", "sabt-etelaat-kol");

return { url: parsed.toString(), usedPooler, local };
}

```

```

const { url: connectionString, usedPooler, local } = normalizeConnectionString(rawUrl ?? FALLBACK_URL);

```

```

if (rawUrl) {
 console.log(
 `数据库 数据库 数据库: ${local ? "数据库" : "usedPooler ? "Neon pooler" : "数据库 (数据库 pooler)"} | connect_timeout=15s`
);
}

```

```

 });
}

const poolConfig: PoolConfig = {
 connectionString,
 ssl: local ? undefined : { rejectUnauthorized: false },
 // PgBouncer 连接池 配置
 max: Number(process.env.DB_POOL_MAX ?? (local ? 5 : 8)),
 min: 0,
 idleTimeoutMillis: 30_000,
 connectionTimeoutMillis: 15_000,
 // 数据库连接池 配置
 statement_timeout: 20_000,
 query_timeout: 20_000,
 keepAlive: true,
 keepAliveInitialDelayMillis: 10_000,
 allowExitOnIdle: false,
};

const globalForDb = globalThis as typeof globalThis & {
 __sekPool?: Pool;
};

function createPool() {
 const p = new Pool(poolConfig);
 // 数据库连接池 配置
 p.on("error", (err) => {
 console.error("数据库连接池 配置错误 (数据库连接池 配置):", err.message);
 });
 return p;
}

export const pool = globalForDb.__sekPool ?? createPool();
if (process.env.NODE_ENV !== "production") globalForDb.__sekPool = pool;

export const db = drizzle(pool);

/* ----- */
/* 数据库连接池 配置 */
/* ----- */

const DB_RETRY: RetryOptions = {
 retries: 4,
 baseDelayMs: 1_000,
 maxDelayMs: 30_000,
 timeoutMs: 25_000,
 isRetryable: isTransientError,
};

/**
 * 数据库连接池 配置
 *
 * @example
 * const rows = await dbRetry(() => db.select().from(users), "users:list");
 */
export function dbRetry<T>(operation: () => Promise<T>, label = "db"): Promise<T> {
 return withRetry(() => operation(), { ...DB_RETRY, label });
}

/** 数据库连接池 配置 */
export async function dbRetrySafe<T>(operation: () => Promise<T>, fallback: T, label = "db"): Promise<T> {
 try {
 return await dbRetry(operation, label);
 } catch (err) {
 console.error(`数据库连接池 配置错误: ${err instanceof Error ? err.message : err}`);
 return fallback;
 }
}

/** 数据库连接池 配置 */
export function dbTransaction<T>(fn: (client: PoolClient) => Promise<T>, label = "tx"): Promise<T> {
 return dbRetry(async () => {
 const client = await pool.connect();
 try {
 await client.query("BEGIN");
 const result = await fn(client);
 await client.query("COMMIT");
 return result;
 } catch (err) {
 await client.query("ROLLBACK").catch(() => undefined);
 throw err;
 } finally {
 client.release();
 }
 }, label);
}

```



```

}

/**
 * @param {Object} options - options for the database check
 * @returns {Promise<{ ok: boolean; latencyMs: number; detail: string }>}
 */
export async function checkDatabase(): Promise<{ ok: boolean; latencyMs: number; detail: string }> {
 const started = Date.now();
 try {
 await withRetry(() => pool.query("select 1"), {
 retries: 2,
 baseDelayMs: 700,
 maxDelayMs: 3_000,
 timeoutMs: 12_000,
 label: "health",
 });
 return { ok: true, latencyMs: Date.now() - started, detail: usedPooler ? "pooler" : local ? "local" : "direct" };
 } catch (err) {
 return {
 ok: false,
 latencyMs: Date.now() - started,
 detail: err instanceof Error ? err.message.slice(0, 200) : "unknown",
 };
 }
}

```

```

export const dbInfo = { usedPooler, local, connectTimeoutSec: 15 };

```

```

@@ package.json json
{
 "name": "sabt-etelaat-kol",
 "private": true,
 "engines": {
 "node": ">=20.9.0"
 },
 "scripts": {
 "dev": "next dev",
 "build": "next build",
 "start": "node scripts/start.mjs",
 "start:raw": "next start",
 "lint": "eslint .",
 "typecheck": "tsc --noEmit",
 "prebuild": "node scripts/generate-assets.mjs",
 "gen:assets": "node scripts/generate-assets.mjs",
 "postbuild": "node scripts/build-sw.mjs"
 },
 "dependencies": {
 "@types/leaflet": "^1.9.21",
 "dotenv": "17.3.1",
 "drizzle-orm": "0.45.2",
 "leaflet": "^1.9.4",
 "next": "16.2.6",
 "pg": "8.20.0",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 "sharp": "^0.35.3"
 },
 "devDependencies": {
 "@tailwindcss/postcss": "4.1.17",
 "@types/node": "22.19.15",
 "@types/pg": "8.18.0",
 "@types/react": "19.2.14",
 "@types/react-dom": "19.2.3",
 "drizzle-kit": "0.31.10",
 "eslint": "9.39.4",
 "eslint-config-next": "16.2.6",
 "postcss": "8.5.8",
 "tailwindcss": "4.1.17",
 "typescript": "5.9.3"
 }
}

```

```

@@ next-env.d.ts typescript
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "next-dev-types/routes.d.ts";

```

```

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.

```

```

@@ next.config.ts typescript
import type { NextConfig } from "next";

```

```

/**
 * @param {Object} options - options for the database check
 * @returns {Promise<{ ok: boolean; latencyMs: number; detail: string }>}
 */
export async function checkDatabase(): Promise<{ ok: boolean; latencyMs: number; detail: string }> {
 const started = Date.now();
 try {
 await withRetry(() => pool.query("select 1"), {
 retries: 2,
 baseDelayMs: 700,
 maxDelayMs: 3_000,
 timeoutMs: 12_000,
 label: "health",
 });
 return { ok: true, latencyMs: Date.now() - started, detail: usedPooler ? "pooler" : local ? "local" : "direct" };
 } catch (err) {
 return {
 ok: false,
 latencyMs: Date.now() - started,
 detail: err instanceof Error ? err.message.slice(0, 200) : "unknown",
 };
 }
}

```

```

*/
const NO_STORE = "no-cache, no-store, must-revalidate, max-age=0";

const nextConfig: NextConfig = {
 poweredByHeader: false,
 compress: true,
 reactStrictMode: true,
 experimental: {
 optimizePackageImports: ["leaflet"],
 },
 productionBrowserSourceMaps: false,

 async headers() {
 return [
 {
 // 在 HTML 文档头中设置 Cache-Control 和 Pragma 头
 // (在 _next/ 目录下的文件)
 source: "/*:path(?!_next/.*)",
 headers: [
 { key: "Cache-Control", value: NO_STORE },
 { key: "Pragma", value: "no-cache" },
 { key: "Expires", value: "0" },
 { key: "X-Content-Type-Options", value: "nosniff" },
],
 },
 // 在 _next/static 目录下的文件
 // immutable 头表示该文件在 30 天内不会改变
 {
 source: "/sw.js",
 headers: [
 { key: "Content-Type", value: "application/javascript; charset=utf-8" },
 { key: "Cache-Control", value: NO_STORE },
 { key: "Service-Worker-Allowed", value: "/" },
],
 },
 {
 source: "/manifest.webmanifest",
 headers: [
 { key: "Content-Type", value: "application/manifest+json; charset=utf-8" },
 { key: "Cache-Control", value: "public, max-age=3600" },
 { key: "Access-Control-Allow-Origin", value: "*" },
],
 },
 {
 source: "/logo.svg",
 headers: [
 { key: "Cache-Control", value: "public, max-age=604800" },
 { key: "Access-Control-Allow-Origin", value: "*" },
],
 },
 {
 source: "/apple-touch-icon.png",
 headers: [
 { key: "Cache-Control", value: "public, max-age=31536000, immutable" },
 { key: "Access-Control-Allow-Origin", value: "*" },
],
 },
 {
 source: "/icons/*:path*",
 headers: [
 { key: "Cache-Control", value: "public, max-age=31536000, immutable" },
 { key: "Access-Control-Allow-Origin", value: "*" },
],
 },
 {
 source: "/screenshots/*:path*",
 headers: [
 { key: "Cache-Control", value: "public, max-age=604800" },
 { key: "Access-Control-Allow-Origin", value: "*" },
],
 },
 {
 source: "/data/*:path*",
 headers: [
 { key: "Cache-Control", value: "public, max-age=2592000, stale-while-revalidate=31536000" },
 { key: "Access-Control-Allow-Origin", value: "*" },
],
 },
 // API 端点
 {
 source: "/*api/*:path*",
 headers: [{ key: "Cache-Control", value: NO_STORE }],
 },
];
 },
};

```

```

};

export default nextConfig;

@@ public/cloudflare-worker.js javascript
/**
 * =====
 * 000000 00000 0000 0000 000000 «000 00000000 00»
 * Cloudflare Worker — 0000 0000 (00000000 0000 00000 0000000)
 * -----
 * 0000 000 0000 0000 00 0000 0000:
 * 0) 0000 0 chat_id 00 00 000 0000000 00000000 → 0000 0000 0000
 * 0000 00 00000 0000 00 000000 0000000 0 00000 0000.
 * 0) 000 0000 000000 000000 00 0000000 00000000 000000 00000000 0000000.
 * 0) 0000000000 00 0000000 (WhatsApp Cloud API).
 * 0) 0000 00000000 JSON + CORS + 000 00000000 (x-proxy-secret).
 * 0) 0000 000 000 000000 00 0000000 0000000 (tapi 0 api).
 * =====
 */

const CONFIG = {
 // 00 0000 000000 00000000000 0000000 00000000 00 00000 00000000 (00000000)
 TELEGRAM_BOT_TOKEN: "",
 TELEGRAM_CHAT_ID: "",
 BALE_BOT_TOKEN: "",
 BALE_CHAT_ID: "",
 ITA_BOT_TOKEN: "",
 ITA_CHAT_ID: "",
 WHATSAPP_TOKEN: "", // 00 0000 phoneNumberId:accessToken
 WHATSAPP_TO: "",
 // 000 000000 00000000 000 0000000000000000 00 000 x-proxy-secret 00000 00000 0000000 00000000
 PROXY_SECRET: "",
};

const CORS = {
 "Access-Control-Allow-Origin": "",
 "Access-Control-Allow-Methods": "POST, OPTIONS",
 "Access-Control-Allow-Headers": "Content-Type, x-proxy-secret",
};

const json = (obj, status = 200) =>
 new Response(JSON.stringify(obj), {
 status,
 headers: { "Content-Type": "application/json; charset=utf-8", ...CORS },
 });

export default {
 async fetch(request) {
 if (request.method === "OPTIONS") return new Response(null, { headers: CORS });
 if (request.method === "GET") {
 return json({ ok: true, service: "sabt-etelaat-kol proxy", messengers: ["telegram", "bale", "ita", "whatsapp"] });
 }
 if (request.method !== "POST") return json({ success: false, error: "0000 POST 0000 000", 405 });

 if (CONFIG.PROXY_SECRET && request.headers.get("x-proxy-secret") !== CONFIG.PROXY_SECRET) {
 return json({ success: false, error: "0000 00000000 0000000 000", 401 });
 }

 let body;
 try {
 body = await request.json();
 } catch {
 return json({ success: false, error: "00000 000000000 00000 JSON 00000", 400 });
 }

 // 0000000000 00 getUpdates 0000 0000 chat_id
 if (body.action === "getUpdates") {
 const mg = String(body.messenger || body.platform || "").toLowerCase();
 const tk = body.token || (mg === "telegram" ? CONFIG.TELEGRAM_BOT_TOKEN : CONFIG.BALE_BOT_TOKEN);
 if (!tk) return json({ ok: false, error: "00000 000000 00000 000", 400 });
 const bases = mg === "telegram" ? ["https://api.telegram.org"] : ["https://tapi.bale.ai", "https://api.bale.ai"];
 for (const base of bases) {
 try {
 const r = await fetch(`${base}/bot${tk}/getUpdates?limit=100`);
 const raw = await r.text();
 if (r.ok) return new Response(raw, { headers: { "Content-Type": "application/json", ...CORS } });
 } catch {
 /* next host */
 }
 }
 }
 return json({ ok: false, error: "00000000 00000 00 00000000 000", 502 });
 }
};

const messenger = String(body.messenger || body.platform || "").toLowerCase();
const text = body.text;

```

```

const chatId = body.chatId || body.chat_id;
const token = body.token;

if (!messenger) return json({ success: false, error: 'مessenger ناموجود است', 400});
if (!text) return json({ success: false, error: 'text ناموجود است', 400});

try {
 let result;
 switch (messenger) {
 case "telegram":
 result = await sendTelegram(text, chatId, token);
 break;
 case "bale":
 result = await sendBale(text, chatId, token);
 break;
 case "ita":
 case "eita":
 result = await sendEita(text, chatId, token);
 break;
 case "whatsapp":
 result = await sendWhatsapp(text, chatId, token, body.provider);
 break;
 default:
 return json({ success: false, error: "مessenger ناموجود است", 400});
 }
 return json({ success: true, ok: true, messenger, data: result });
} catch (error) {
 return json({ success: false, ok: false, messenger, error: String(error.message || error) }, 502);
}
};

async function postJson(url, payload) {
 const res = await fetch(url, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(payload),
 });
 const raw = await res.text();
 let data;
 try {
 data = JSON.parse(raw);
 } catch {
 data = { raw };
 }
 return { res, data, raw };
}

async function sendTelegram(text, chatId, token) {
 const tk = token || CONFIG.TELEGRAM_BOT_TOKEN;
 const cid = chatId || CONFIG.TELEGRAM_CHAT_ID;
 if (!tk || !cid) throw new Error("chat_id یا token ناموجود است");
 const { data } = await postJson(`https://api.telegram.org/bot${tk}/sendMessage`, { chat_id: cid, text });
 if (data.ok === false) throw new Error(data.description || "ارسال پیام ناموفق است");
 return data;
}

async function sendBale(text, chatId, token) {
 const tk = token || CONFIG.BALE_BOT_TOKEN;
 const cid = chatId || CONFIG.BALE_CHAT_ID;
 if (!tk || !cid) throw new Error("chat_id یا token ناموجود است");
 const hosts = ["https://tapi.bale.ai", "https://api.bale.ai"];
 let last = "";
 for (const host of hosts) {
 try {
 const { data } = await postJson(`${host}/bot${tk}/sendMessage`, { chat_id: cid, text });
 if (data.ok !== false) return data;
 last = data.description || JSON.stringify(data);
 } catch (e) {
 last = String(e.message || e);
 }
 }
 throw new Error(last || "ارسال پیام ناموفق است");
}

async function sendEita(text, chatId, token) {
 const tk = token || CONFIG.ITA_BOT_TOKEN;
 const cid = chatId || CONFIG.ITA_CHAT_ID;
 if (!tk || !cid) throw new Error("chat_id یا token ناموجود است");
 // فرمت فرم-اوریچد (form-urlencoded)
 const res = await fetch(`https://eitaayar.ir/api/${tk}/sendMessage`, {
 method: "POST",
 headers: { "Content-Type": "application/x-www-form-urlencoded" },
 body: new URLSearchParams({ chat_id: cid, text }).toString(),
 });
};

```

```
const raw = await res.text();
let data;
try {
 data = JSON.parse(raw);
} catch {
 data = { raw };
}
if (!res.ok || data.ok === false) throw new Error(data.description || raw.slice(0, 200));
return data;
}

async function sendWhatsapp(text, to, token, provider) {
 const p = (provider || "cloudapi").toLowerCase();
 const digits = String(to || CONFIG.WHATSAPP_TO || "").replace(/\D/g, "");
 const tk = token || CONFIG.WHATSAPP_TOKEN;

 if (p === "whatsiplus") {
 const r = await fetch(
 `https://api.whatsiplus.com/sendMsg/${tk}?phoneNumber=${encodeURIComponent(digits)}&message=${encodeURIComponent(text)}`
);
 const raw = await r.text();
 let d;
 try { d = JSON.parse(raw); } catch { d = { raw }; }
 const okFlag = String(d.success ?? d.status ?? "").toLowerCase();
 if (!r.ok || okFlag === "false" || okFlag === "error") throw new Error(d.message || raw.slice(0, 180));
 return d;
 }

 if (p === "ultramsg") {
 const [instance, apiToken] = String(tk).split(":");
 if (!instance || !apiToken) throw new Error("❌❌❌ UltraMsg ❌❌❌ instanceId:token ❌❌❌");
 const r = await fetch(`https://api.ultramsg.com/${instance}/messages/chat`, {
 method: "POST",
 headers: { "Content-Type": "application/x-www-form-urlencoded" },
 body: new URLSearchParams({ token: apiToken, to: digits, body: text }).toString(),
 });
 const raw = await r.text();
 if (!r.ok) throw new Error(raw.slice(0, 180));
 try { return JSON.parse(raw); } catch { return { raw }; }
 }

 return sendWhatsappCloud(text, digits, tk);
}

async function sendWhatsappCloud(text, to, token) {
 const tk = token || CONFIG.WHATSAPP_TOKEN;
 const target = String(to || CONFIG.WHATSAPP_TO || "").replace(/\D/g, "");
 const i = String(tk).indexOf(":");
 const phoneNumberId = i > 0 ? tk.slice(0, i) : "";
 const accessToken = i > 0 ? tk.slice(i + 1) : "";
 if (!phoneNumberId || !accessToken) throw new Error("❌❌❌ ❌❌❌❌❌ ❌❌❌ phoneNumberId:accessToken ❌❌❌");
 if (!target) throw new Error("❌❌❌❌❌ ❌❌❌ ❌❌❌❌❌❌ ❌❌❌ ❌❌❌❌❌❌");
 const res = await fetch(`https://graph.facebook.com/v21.0/${phoneNumberId}/messages`, {
 method: "POST",
 headers: { "Content-Type": "application/json", Authorization: `Bearer ${accessToken}` },
 body: JSON.stringify({
 messaging_product: "whatsapp",
 recipient_type: "individual",
 to: target,
 type: "text",
 text: { preview_url: false, body: text },
 }),
 });
 const raw = await res.text();
 if (!res.ok) throw new Error(raw.slice(0, 200));
 try {
 return JSON.parse(raw);
 } catch {
 return { raw };
 }
}

@@ public/sw-template.js javascript
/* =====
 * 📄📄📄📄📄 «📄📄 📄📄📄📄📄 📄» — 📄📄 📄 (📄📄📄📄📄 📄📄)
 *
 * 📄📄📄 📄📄📄: 📄📄📄 📄 📄📄📄 HTML📄 *📄📄 📄📄📄📄📄 📄📄📄📄📄📄 📄 CSS*
 * 📄📄📄📄 📄 📄📄📄📄 📄📄📄📄. 📄📄📄 📄📄 📄📄 📄📄📄📄📄 📄📄📄📄 📄📄
 * 📄📄 📄📄📄📄 📄📄 📄📄📄📄📄 JS 📄📄📄📄📄📄 📄📄📄📄📄📄 📄 📄📄 📄📄📄 📄📄📄📄📄.
 * 📄📄 📄📄📄 📄📄 scripts/build-sw.mjs 📄📄 📄📄 build 📄📄📄📄 📄📄📄📄.
 * ===== */

const BUILD = "__BUILD_ID__";
const SHELL = `sek-shell-${BUILD}`;
```

```

const DATA = `sek-data-${BUILD}`;

const API_TIMEOUT = 6000;
const ASSET_TIMEOUT = 20000;
const NAV_TIMEOUT = 5000; // 0000 00000 0000 HTML 00 00000 000 00000000 000 0000 00000

/* 0000 0000000 00 00 000 000000000000 000000 00 0000 0000000 00000 0000000 0000 */
function offlineFlagged(res) {
 const h = new Headers(res.headers);
 h.set("x-sek-offline", "1");
 return new Response(res.body, { status: res.status, statusText: res.statusText, headers: h });
}

const PAGES = __PAGES__;
const ASSETS = __ASSETS__;
const EXTRAS = __EXTRAS__;
const RSC_PAGES = __RSC_PAGES__;
void RSC_PAGES;

/* ----- 000: 000000 0000 ----- */
self.addEventListener("install", (e) => {
 self.skipWaiting();
 e.waitUntil(
 (async () => {
 const cache = await caches.open(SHELL);
 // 00000000 00000 (JS/CSS) 000 — 0000 00000 000000 0000 00000000
 await addAll(cache, ASSETS, 6);
 await addAll(cache, PAGES, 4);
 await addAll(cache, EXTRAS, 3);
 const cs = await self.clients.matchAll();
 cs.forEach((c) => c.postMessage({ type: "precache-done", assets: ASSETS.length, pages: PAGES.length }));
 })()
);
});

async function addAll(cache, urls, concurrency) {
 let i = 0;
 const workers = Array.from({ length: concurrency }, async () => {
 while (i < urls.length) {
 const url = urls[i++];
 try {
 const res = await fetch(new Request(url, { cache: "reload", credentials: "same-origin" }));
 if (res && (res.ok || res.type === "opaque")) await cache.put(url, res.clone());
 } catch {}
 /* 00000 00000 00000000 00 000000 */
 }
 });
 await Promise.all(workers);
}

/* ----- 0000000000 ----- */
self.addEventListener("activate", (e) => {
 e.waitUntil(
 (async () => {
 if (self.registration.navigationPreload) {
 try {
 await self.registration.navigationPreload.enable();
 } catch {}
 }
 const keys = await caches.keys();
 await Promise.all(keys.filter((k) => !k.endsWith(BUILD)).map((k) => caches.delete(k)));
 await self.clients.claim();
 })()
);
});

function timeoutFetch(req, ms) {
 return new Promise((resolve, reject) => {
 const t = setTimeout(() => reject(new Error("timeout")), ms);
 fetch(req).then(
 (r) => {
 clearTimeout(t);
 resolve(r);
 },
 (err) => {
 clearTimeout(t);
 reject(err);
 }
);
 });
}

/* ----- 00 0000000 ----- */
const DB_NAME = "sek-queue";

```

```

function idb() {
 return new Promise((res, rej) => {
 const r = indexedDB.open(DB_NAME, 1);
 r.onupgradeneeded = () => r.result.createObjectStore("q", { keyPath: "id", autoIncrement: true });
 r.onsuccess = () => res(r.result);
 r.onerror = () => rej(r.error);
 });
}
}
async function queueAdd(item) {
 const db = await idb();
 return new Promise((res) => {
 const tx = db.transaction("q", "readwrite");
 tx.objectStore("q").add(item);
 tx.oncomplete = () => res(true);
 tx.onerror = () => res(false);
 });
}
}
async function queueAll() {
 const db = await idb();
 return new Promise((res) => {
 const tx = db.transaction("q", "readonly");
 const rq = tx.objectStore("q").getAll();
 rq.onsuccess = () => res(rq.result || []);
 rq.onerror = () => res([]);
 });
}
}
async function queueDel(id) {
 const db = await idb();
 return new Promise((res) => {
 const tx = db.transaction("q", "readwrite");
 tx.objectStore("q").delete(id);
 tx.oncomplete = () => res(true);
 tx.onerror = () => res(false);
 });
}
}
async function flushQueue() {
 const items = await queueAll();
 let sent = 0;
 for (const it of items) {
 try {
 const r = await fetch(it.url, { method: it.method, headers: it.headers, body: it.body, credentials: "include" });
 if (r.ok || (r.status >= 400 && r.status < 500)) {
 await queueDel(it.id);
 if (r.ok) sent++;
 }
 } catch {
 break;
 }
 }
 const rest = await queueAll();
 const cs = await self.clients.matchAll();
 cs.forEach(c => c.postMessage({ type: "queue-status", pending: rest.length, sent }));
 return sent;
}
}

self.addEventListener("message", (e) => {
 if (e.data === "flush") e.waitUntil(flushQueue());
 if (e.data === "skipWaiting") self.skipWaiting();
 if (e.data === "queue-count")
 e.waitUntil(
 queueAll().then(async (q) => {
 const cs = await self.clients.matchAll();
 cs.forEach(c => c.postMessage({ type: "queue-status", pending: q.length, sent: 0 }));
 })
);
 if (e.data === "cache-status")
 e.waitUntil(
 caches.open(SHELL).then(async (c) => {
 const keys = await c.keys();
 const cs = await self.clients.matchAll();
 cs.forEach(x =>
 x.postMessage({ type: "cache-status", cached: keys.length, expected: ASSETS.length + PAGES.length + EXTRAS.len
gth })
);
 })
);
});
self.addEventListener("sync", (e) => {
 if (e.tag === "sek-sync") e.waitUntil(flushQueue());
});

/* ----- 0000 000 00000000000 ----- */
self.addEventListener("notificationclick", (event) => {
 const data = event.notification.data || {};
 event.notification.close();
});

```

```

event.waitUntil(
 (async () => {
 const all = await self.clients.matchAll({ type: "window", includeUncontrolled: true });
 // 通知を送るクライアントをすべて取得
 for (const c of all) {
 c.postMessage({ type: "notification-click", id: data.id });
 if ("focus" in c) {
 await c.focus();
 if (data.link) c.navigate?.(data.link);
 return;
 }
 }
 if (self.clients.openWindow) await self.clients.openWindow(data.link || "/panel/notifications");
 })()
);
});

/* ----- Web Push (通知を送る) ----- */
self.addEventListener("push", (event) => {
 let payload = { title: "通知", body: "", id: 0, link: "/panel/notifications" };
 try {
 payload = { ...payload, ...(event.data ? event.data.json() : {}) };
 } catch {}
 if (event.data) payload.body = event.data.text();
}
event.waitUntil(
self.registration.showNotification(`${payload.title}`, {
 body: payload.body,
 icon: "/icons/icon-192.png",
 badge: "/icons/icon-192.png",
 dir: "rtl",
 lang: "fa",
 tag: `sek-${payload.id}`,
 requireInteraction: true,
 vibrate: [200, 100, 200],
 data: payload,
}))
);
});

/* ----- 通知を送る ----- */
self.addEventListener("fetch", (event) => {
 const req = event.request;
 const url = new URL(req.url);
 if (url.origin !== self.location.origin) return;
 if (url.pathname === "/api/ping") return;

 /* 通知を送る ----- */
 if (req.method !== "GET") {
 if (!url.pathname.startsWith("/api/")) return;
 event.respondWith(
 (async () => {
 try {
 return await fetch(req.clone());
 } catch {}
 if (url.pathname.startsWith("/api/auth")) {
 return new Response(JSON.stringify({ error: "通知を送るエラー", status: 503, headers: { "Content-Type": "application/json; charset=utf-8" } }));
 }
 const body = await req.clone().text();
 const headers = {};
 req.headers.forEach((v, k) => (headers[k] = v));
 await queueAdd({ url: req.url, method: req.method, headers, body, at: Date.now() });
 if (self.registration.sync) {
 try {
 await self.registration.sync.register("sek-sync");
 } catch {}
 }
 const cs = await self.clients.matchAll();
 cs.forEach((c) => c.postMessage({ type: "queued" }));
 return new Response(
 JSON.stringify({
 ok: true,
 queued: true,
 offline: true,
 message: "通知を送る成功",
 }),
 { status: 202, headers: { "Content-Type": "application/json; charset=utf-8" } }
);
 })()
);
 }
});
return;

```



```

}

/* 000000000000 RSC (00000000 000000 Next) → 0000 00 000000 0000 00
* 0000 000 0000 0000000000 000 000000 00 0000 00000000 000 00000000. */
const isRsc = url.searchParams.has("_rsc") || req.headers.get("RSC") === "1";
if (isRsc) {
 event.respondWith(
 (async () => {
 try {
 const res = await timeoutFetch(req, NAV_TIMEOUT);
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 const cached = await caches.match(req);
 if (cached) return cached;
 }
 // 00000 RSC0 Next 0000 00 00000000 0000 0000 0000000000 (00 00 00 000 00000000)
 return new Response("", { status: 504 });
 })()
);
 return;
}

/* 0000000000 00000000 Next → 00-0000 0 0000000 (0000000000 000000) */
if (url.pathname.startsWith("/_next/static/")) {
 event.respondWith(
 (async () => {
 const cached = await caches.match(req);
 if (cached) return cached;
 try {
 const res = await timeoutFetch(req, ASSET_TIMEOUT);
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 return new Response("", { status: 504 });
 }
 })()
);
 return;
}

/* API → 0000 00 0000 00000000 0000 00 */
if (url.pathname.startsWith("/api/")) {
 if (
 url.pathname.startsWith("/api/auth/login") ||
 url.pathname.startsWith("/api/auth/logout") ||
 url.pathname.startsWith("/api/auth/forgot") ||
 url.pathname.startsWith("/api/auth/otp") ||
 url.pathname.startsWith("/api/auth/check-username")
) {
 return;
 }

 event.respondWith(
 (async () => {
 try {
 const res = await timeoutFetch(req, API_TIMEOUT);
 if (res.ok) {
 const c = await caches.open(DATA);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 const cached = await caches.match(req);
 if (cached) {
 const h = new Headers(cached.headers);
 h.set("x-from-cache", "1");
 return new Response(await cached.blob(), { status: 200, headers: h });
 }
 return new Response(
 JSON.stringify({
 offline: true,
 rows: [],
 logs: [],
 reps: [],
 error: "00000000 00 0000 00000000 0000",
 }),
 { status: 200, headers: { "Content-Type": "application/json; charset=utf-8", "x-sek-offline": "1" } }
);
 }
 })()
);
}

```

```

 })()
);
 return;
}

/* =====
* 000000 00000 → «0000-000» (Network First)
*
* 0000 000 HTML 000000 00 00000 00000000000 00000 0000 000 0000
* 000000 000000 00 000000. 00 000000 000 00000 00000000 00000000
* 000 0000 00000 00 000000 000000 000000 00 00 00 00000000000 00
* 0000000 0000000 00 00000 00000 000000 (00 00000 0000000).
* ===== */
if (req.mode === "navigate") {
 event.respondWith(
 (async () => {
 // 0 00000 00000 00000 00 00000 000000
 try {
 const preload = await event.preloadResponse;
 const res = preload || (await timeoutFetch(new Request(req, { cache: "no-store" }), NAV_TIMEOUT));
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(url.pathname, res.clone()); // 000 0000000000 00000000 00000000
 return res;
 }
 if (res) return res; // 00000 000000 00000 (000/000) 00 0000000000 00000 0000
 } catch {
 /* 00000 00 000000 00000 → 000000 00 00 */
 }

 // 0 0000000: 00000 00000 00 00 (0000000 00000 0000 00000000)
 const cachedPage =
 (await caches.match(url.pathname, { ignoreSearch: true })) ||
 (await caches.match(req, { ignoreSearch: true }));
 if (cachedPage) return offlineFlagged(cachedPage);

 // 0 000 00 00000 00 000000 000000 00 00 000 00 0000000000 000 00000000 000000 00000
 const shell =
 (await caches.match("/panel")) || (await caches.match("/")) || (await caches.match("/login"));
 if (shell) return offlineFlagged(shell);

 // 0 000000 0000: 00000 0000000000 00000000
 return (
 (await caches.match("/offline")) ||
 new Response(
 "<html dir=rtl><meta charset=utf-8><body style='font-family:Tahoma;text-align:center;padding:40px'>" +
 "<h2>0000000 00 00000 00000000 0000</h2><p>000000 000000 0000000000 000 00 000000 00000.</p>" +
 "<button onclick='location.reload()' style='padding:10px 24px;border-radius:10px;background:#0f766e;color:" +
 "#fff;border:0'>00000 00000</button>" +
 "</body></html>",
 { status: 200, headers: { "Content-Type": "text/html; charset=utf-8" } }
)
);
 })()
);
 return;
}

/* 00000 000000 → 00-000 */
event.respondWith(
 (async () => {
 const cached = await caches.match(req);
 if (cached) return cached;
 try {
 const res = await timeoutFetch(req, ASSET_TIMEOUT);
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 return new Response("", { status: 504 });
 }
 })()
);
});

@@ public/sw.js javascript
/* =====
* 000000000000 «0000 00000000 00» — 00000 0 (00000000 00000)
*
* 00000 000000: 000000 00 000000 HTML0 *0000 00000000000 000000000000 0 CSS*
* 0000000 00 00000000 000000000. 00000 0000 0000 000 0000000000 00000000 000000
* 000 0000000 0000 00000000000 JS 00000000000 00000000000 0 00000 00000 000000000.
* 000 00000 00000 scripts/build-sw.mjs 00 00 00 build 000000 00000000.
```

```

* ===== */

const BUILD = "msmdifhz-48";
const SHELL = `sek-shell-${BUILD}`;
const DATA = `sek-data-${BUILD}`;

const API_TIMEOUT = 6000;
const ASSET_TIMEOUT = 20000;
const NAV_TIMEOUT = 5000; // 0000 00000 0000 HTML 00 00000 000 00000000 000 0000 000000

/** 0000 000000 00 00 000 000000000000 000000 00 0000 000000 000000 000000 0000 */
function offlineFlagged(res) {
 const h = new Headers(res.headers);
 h.set("x-sek-offline", "1");
 return new Response(res.body, { status: res.status, statusText: res.statusText, headers: h });
}

const PAGES = [
 "/",
 "/login",
 "/offline",
 "/install",
 "/diagnostics",
 "/panel",
 "/panel/pharmacies",
 "/panel/doctors",
 "/panel/orders",
 "/panel/trip",
 "/panel/map",
 "/panel/home",
 "/panel/leaves",
 "/panel/notifications",
 "/panel/options",
 "/panel/reports",
 "/admin",
 "/admin/activity",
 "/admin/records/pharmacies",
 "/admin/records/doctors",
 "/admin/records/orders",
 "/admin/map",
 "/admin/live",
 "/admin/trips",
 "/admin/homes",
 "/admin/leaves",
 "/admin/notifications",
 "/admin/reports",
 "/admin/options",
 "/admin/columns",
 "/admin/users",
 "/admin/messengers",
 "/admin/backup"
];

const ASSETS = [
 "/_next/static/chunks/0.f806x69l1dq.js",
 "/_next/static/chunks/01xlw8hd842-c.js",
 "/_next/static/chunks/01yarxs5j9-pm.js",
 "/_next/static/chunks/025h39a4hxy-7.js",
 "/_next/static/chunks/02a3un8vt.4z0.js",
 "/_next/static/chunks/02es.zsatljwd.js",
 "/_next/static/chunks/03h9eo09svijh.js",
 "/_next/static/chunks/03-yq9q893hmn.js",
 "/_next/static/chunks/04zs0174xt4cf.js",
 "/_next/static/chunks/06in8m2ln4m7x.js",
 "/_next/static/chunks/06r9_3ub2r-4z.js",
 "/_next/static/chunks/07lhk_q6pmm3r.js",
 "/_next/static/chunks/07uz2g0_38qia.js",
 "/_next/static/chunks/08.gvqj-o7ww6.js",
 "/_next/static/chunks/09pnscjn-jklc.js",
 "/_next/static/chunks/09u-zy9mfz~2_.js",
 "/_next/static/chunks/0_uvbk1-ua0pn.js",
 "/_next/static/chunks/0a59dbudyzsi4.js",
 "/_next/static/chunks/0d3shmw5_nmn.js",
 "/_next/static/chunks/0e8p5l2oiw7kt.js",
 "/_next/static/chunks/0evn0xx7qmz14.js",
 "/_next/static/chunks/0h7njwe2.s3ht.js",
 "/_next/static/chunks/0hoxp6v6.ghn5.js",
 "/_next/static/chunks/0n.0ufr5-u.u3.js",
 "/_next/static/chunks/0n1bpogj.il-n.js",
 "/_next/static/chunks/0o9kx41n71_k~.js",
 "/_next/static/chunks/0oudi9rq3kf0y.js",
 "/_next/static/chunks/0rjm_co8bzrp0.js",
 "/_next/static/chunks/0rz3o0743-gv3.js",
 "/_next/static/chunks/0t48hzs_6fshe.css",
 "/_next/static/chunks/0u-69.dj98a1y.js",
 "/_next/static/chunks/0v4dehyglgch.js",

```

```

"/_next/static/chunks/0v7ff1k50ig4a.js",
"/_next/static/chunks/0vy~5p61tj63m.js",
"/_next/static/chunks/0we42v7.l-4bo.js",
"/_next/static/chunks/0wxhwo2i~7vvo.js",
"/_next/static/chunks/0xnf3du~wj5.7.css",
"/_next/static/chunks/0zafsy1z7iub1.js",
"/_next/static/chunks/0ze4gu236oq96.js",
"/_next/static/chunks/11bf.0jc0_nuk.js",
"/_next/static/chunks/11p2tj5w~k_4s.js",
"/_next/static/chunks/13niyj~yq3tn~.js",
"/_next/static/chunks/17ij8w.0v9eei.js",
"/_next/static/chunks/17zu884hptsv9.js",
"/_next/static/chunks/turbopack-0ngnbt.drh_yz.js",
"/_next/static/mVg2gUquQcbk3idaw9VXG/_buildManifest.js",
"/_next/static/mVg2gUquQcbk3idaw9VXG/_clientMiddlewareManifest.js",
"/_next/static/mVg2gUquQcbk3idaw9VXG/_ssgManifest.js"
];
const EXTRAS = [
 "/manifest.webmanifest",
 "/logo.svg",
 "/icons/icon-192.png",
 "/icons/icon-512.png",
 "/apple-touch-icon.png",
 "/data/iran-provinces.geojson"
];
const RSC_PAGES = [
 "/?_rsc=offline",
 "/login?_rsc=offline",
 "/offline?_rsc=offline",
 "/install?_rsc=offline",
 "/diagnostics?_rsc=offline",
 "/panel?_rsc=offline",
 "/panel/pharmacies?_rsc=offline",
 "/panel/doctors?_rsc=offline",
 "/panel/orders?_rsc=offline",
 "/panel/trip?_rsc=offline",
 "/panel/map?_rsc=offline",
 "/panel/home?_rsc=offline",
 "/panel/leaves?_rsc=offline",
 "/panel/notifications?_rsc=offline",
 "/panel/options?_rsc=offline",
 "/panel/reports?_rsc=offline",
 "/admin?_rsc=offline",
 "/admin/activity?_rsc=offline",
 "/admin/records/pharmacies?_rsc=offline",
 "/admin/records/doctors?_rsc=offline",
 "/admin/records/orders?_rsc=offline",
 "/admin/map?_rsc=offline",
 "/admin/live?_rsc=offline",
 "/admin/trips?_rsc=offline",
 "/admin/homes?_rsc=offline",
 "/admin/leaves?_rsc=offline",
 "/admin/notifications?_rsc=offline",
 "/admin/reports?_rsc=offline",
 "/admin/options?_rsc=offline",
 "/admin/columns?_rsc=offline",
 "/admin/users?_rsc=offline",
 "/admin/messengers?_rsc=offline",
 "/admin/backup?_rsc=offline"
];
void RSC_PAGES;

/* ----- 0000: 00000000 0000 ----- */
self.addEventListener("install", (e) => {
 self.skipWaiting();
 e.waitUntil(
 (async () => {
 const cache = await caches.open(SHELL);
 // 0000000000 000000 (JS/CSS) 0000 — 00000 000000 000000 0000000000
 await addAll(cache, ASSETS, 6);
 await addAll(cache, PAGES, 4);
 await addAll(cache, EXTRAS, 3);
 const cs = await self.clients.matchAll();
 cs.forEach((c) => c.postMessage({ type: "precache-done", assets: ASSETS.length, pages: PAGES.length }));
 })()
);
});

async function addAll(cache, urls, concurrency) {
 let i = 0;
 const workers = Array.from({ length: concurrency }, async () => {
 while (i < urls.length) {
 const url = urls[i++];
 try {
 const res = await fetch(new Request(url, { cache: "reload", credentials: "same-origin" }));
 }
 }
 });
}

```

```

 if (res && (res.ok || res.type === "opaque")) await cache.put(url, res.clone());
 } catch {
 }
 /* 000000 000000 00000000 00 000000 */
 }
 });
 await Promise.all(workers);
}

/* ----- 0000000000 ----- */
self.addEventListener("activate", (e) => {
 e.waitUntil(
 (async () => {
 if (self.registration.navigationPreload) {
 try {
 await self.registration.navigationPreload.enable();
 } catch {}
 }
 const keys = await caches.keys();
 await Promise.all(keys.filter((k) => !k.endsWith(BUILD)).map((k) => caches.delete(k)));
 await self.clients.claim();
 })()
);
});

function timeoutFetch(req, ms) {
 return new Promise((resolve, reject) => {
 const t = setTimeout(() => reject(new Error("timeout")), ms);
 fetch(req).then(
 (r) => {
 clearTimeout(t);
 resolve(r);
 },
 (err) => {
 clearTimeout(t);
 reject(err);
 }
);
 });
}

/* ----- 00 000000 ----- */
const DB_NAME = "sek-queue";
function idb() {
 return new Promise((res, rej) => {
 const r = indexedDB.open(DB_NAME, 1);
 r.onupgradeneeded = () => r.result.createObjectStore("q", { keyPath: "id", autoIncrement: true });
 r.onsuccess = () => res(r.result);
 r.onerror = () => rej(r.error);
 });
}

async function queueAdd(item) {
 const db = await idb();
 return new Promise((res) => {
 const tx = db.transaction("q", "readwrite");
 tx.objectStore("q").add(item);
 tx.oncomplete = () => res(true);
 tx.onerror = () => res(false);
 });
}

async function queueAll() {
 const db = await idb();
 return new Promise((res) => {
 const tx = db.transaction("q", "readonly");
 const rq = tx.objectStore("q").getAll();
 rq.onsuccess = () => res(rq.result || []);
 rq.onerror = () => res([]);
 });
}

async function queueDel(id) {
 const db = await idb();
 return new Promise((res) => {
 const tx = db.transaction("q", "readwrite");
 tx.objectStore("q").delete(id);
 tx.oncomplete = () => res(true);
 tx.onerror = () => res(false);
 });
}

async function flushQueue() {
 const items = await queueAll();
 let sent = 0;
 for (const it of items) {
 try {
 const r = await fetch(it.url, { method: it.method, headers: it.headers, body: it.body, credentials: "include" });
 if (r.ok || (r.status >= 400 && r.status < 500)) {

```

```

 await queueDel(it.id);
 if (r.ok) sent++;
 }
} catch {
 break;
}
}
const rest = await queueAll();
const cs = await self.clients.matchAll();
cs.forEach((c) => c.postMessage({ type: "queue-status", pending: rest.length, sent }));
return sent;
}

self.addEventListener("message", (e) => {
 if (e.data === "flush") e.waitUntil(flushQueue());
 if (e.data === "skipWaiting") self.skipWaiting();
 if (e.data === "queue-count")
 e.waitUntil(
 queueAll().then(async (q) => {
 const cs = await self.clients.matchAll();
 cs.forEach((c) => c.postMessage({ type: "queue-status", pending: q.length, sent: 0 }));
 })
);
 if (e.data === "cache-status")
 e.waitUntil(
 caches.open(SHELL).then(async (c) => {
 const keys = await c.keys();
 const cs = await self.clients.matchAll();
 cs.forEach((x) =>
 x.postMessage({ type: "cache-status", cached: keys.length, expected: ASSETS.length + PAGES.length + EXTRAS.len
gth })
);
 })
);
});
self.addEventListener("sync", (e) => {
 if (e.tag === "sek-sync") e.waitUntil(flushQueue());
});

/* ----- 0000 000 000000000000 ----- */
self.addEventListener("notificationclick", (event) => {
 const data = event.notification.data || {};
 event.notification.close();
 event.waitUntil(
 (async () => {
 const all = await self.clients.matchAll({ type: "window", includeUncontrolled: true });
 // 000 000000 000 0000 0000 00000 00 00000 00
 for (const c of all) {
 c.postMessage({ type: "notification-click", id: data.id });
 if ("focus" in c) {
 await c.focus();
 if (data.link) c.navigate?.(data.link);
 return;
 }
 }
 if (self.clients.openWindow) await self.clients.openWindow(data.link || "/panel/notifications");
 })()
);
});

/* ----- Web Push (00 0000 0000000000) ----- */
self.addEventListener("push", (event) => {
 let payload = { title: "000000 00000", body: "", id: 0, link: "/panel/notifications" };
 try {
 payload = { ...payload, ...(event.data ? event.data.json() : {}) };
 } catch {
 if (event.data) payload.body = event.data.text();
 }
 event.waitUntil(
 self.registration.showNotification(`0 ${payload.title}`, {
 body: payload.body,
 icon: "/icons/icon-192.png",
 badge: "/icons/icon-192.png",
 dir: "rtl",
 lang: "fa",
 tag: `sek-${payload.id}`,
 requireInteraction: true,
 vibrate: [200, 100, 200],
 data: payload,
 })
);
});

/* ----- 000000 000000 ----- */
self.addEventListener("fetch", (event) => {

```

```

const req = event.request;
const url = new URL(req.url);
if (url.origin !== self.location.origin) return;
if (url.pathname === "/api/ping") return;

/* 000000 → 00 000000 */
if (req.method !== "GET") {
 if (url.pathname.startsWith("/api/")) return;
 event.respondWith(
 (async () => {
 try {
 return await fetch(req.clone());
 } catch {
 if (url.pathname.startsWith("/api/auth")) {
 return new Response(JSON.stringify({ error: "0 0000 00000 0000000 0000 000." }), {
 status: 503,
 headers: { "Content-Type": "application/json; charset=utf-8" },
 });
 }
 const body = await req.clone().text();
 const headers = {};
 req.headers.forEach((v, k) => (headers[k] = v));
 await queueAdd({ url: req.url, method: req.method, headers, body, at: Date.now() });
 if (self.registration.sync) {
 try {
 await self.registration.sync.register("sek-sync");
 } catch {}
 }
 const cs = await self.clients.matchAll();
 cs.forEach((c) => c.postMessage({ type: "queued" }));
 return new Response(
 JSON.stringify({
 ok: true,
 queued: true,
 offline: true,
 message: "0 0000000 000 0000 0000000 00000 00 0 000000 00000 0000000 00000 000000.",
 }),
 { status: 202, headers: { "Content-Type": "application/json; charset=utf-8" } }
);
 }
 })()
);
 return;
}

/* 000000000000 RSC (000000 00000 Next) → 0000 00 00000 000 00
 * 0000 000 0000 0000000000 000 00000 00 0000 000000 000 00000000. */
const isRsc = url.searchParams.has("_rsc") || req.headers.get("RSC") === "1";
if (isRsc) {
 event.respondWith(
 (async () => {
 try {
 const res = await timeoutFetch(req, NAV_TIMEOUT);
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 const cached = await caches.match(req);
 if (cached) return cached;
 }
 // 0000 RSC0 Next 0000 00 000000 0000 0000 0000000000 (00 00 00 000 0000000)
 return new Response("", { status: 504 });
 })()
);
 return;
}

/* 0000000000 00000000 Next → 00-000 0 000000 (000000000 000000) */
if (url.pathname.startsWith("/_next/static/")) {
 event.respondWith(
 (async () => {
 const cached = await caches.match(req);
 if (cached) return cached;
 try {
 const res = await timeoutFetch(req, ASSET_TIMEOUT);
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 return new Response("", { status: 504 });
 }
 })()
);
}

```

```

 })()
);
 return;
}

/* API → 0000 00 0000 000000 000 00 */
if (url.pathname.startsWith("/api/")) {
 if (
 url.pathname.startsWith("/api/auth/login") ||
 url.pathname.startsWith("/api/auth/logout") ||
 url.pathname.startsWith("/api/auth/forgot") ||
 url.pathname.startsWith("/api/auth/otp") ||
 url.pathname.startsWith("/api/auth/check-username")
)
 return;

 event.respondWith(
 (async () => {
 try {
 const res = await timeoutFetch(req, API_TIMEOUT);
 if (res.ok) {
 const c = await caches.open(DATA);
 c.put(req, res.clone());
 }
 return res;
 } catch {
 const cached = await caches.match(req);
 if (cached) {
 const h = new Headers(cached.headers);
 h.set("x-from-cache", "1");
 return new Response(await cached.blob(), { status: 200, headers: h });
 }
 return new Response(
 JSON.stringify({
 offline: true,
 rows: [],
 logs: [],
 reps: [],
 error: "000000 00 0000 000000 0000",
 }),
 { status: 200, headers: { "Content-Type": "application/json; charset=utf-8", "x-sek-offline": "1" } }
);
 }
 })()
);
 return;
}

/* =====
* 000000 00000 → «0000-000» (Network First)
*
* 0000 000 HTML 000000 00 00000 0000000000 00000 0000 000 0000
* 000000 000000 00 00000. 00 00000 000 0000 0000000 00000000
* 000 000 0000 00 00000 00000 00000 00 00 00 0000000000 00
* 000000 000000 00 0000 0000 00000 (00 0000 000000).
* ===== */
if (req.mode === "navigate") {
 event.respondWith(
 (async () => {
 // 0 0000 0000 0000 00 0000 000000
 try {
 const preload = await event.preloadResponse;
 const res = preload || (await timeoutFetch(new Request(req, { cache: "no-store" }), NAV_TIMEOUT));
 if (res && res.ok) {
 const c = await caches.open(SHELL);
 c.put(url.pathname, res.clone()); // 000 00000000 0000000 000000
 return res;
 }
 }
 if (res) return res; // 0000 00000 0000 (000/000) 00 00000000 0000 000
 }) catch {}
);
 /* 0000 00 00000 0000 → 00000 00 00 */
}

// 0 000000: 0000 0000 00 00 (000000 0000 000 000000)
const cachedPage =
 (await caches.match(url.pathname, { ignoreSearch: true })) ||
 (await caches.match(req, { ignoreSearch: true }));
if (cachedPage) return offlineFlagged(cachedPage);

// 0 000 00 0000 00 00000 000000 00 00 000 00 00000000 000 000000 00000 0000
const shell =
 (await caches.match("/panel")) || (await caches.match("/")) || (await caches.match("/login"));
if (shell) return offlineFlagged(shell);

// 0 00000 000: 0000 0000000 000000

```



[illegible]

```

.split(",")
.map((s) => s.trim().replace(/\$/ , ""))
.filter(Boolean);

function isAllowed(origin: string) {
 if (!origin) return false;
 const clean = origin.replace(/\$/ , "");
 if (ALLOWED.includes(clean)) return true;
 // 000000000000 0000 00000 0000 0 0000 00000
 return /^https?:\/\/(localhost|127\.\0\.\0\.\0)(:\d+)?$/ .test(clean) || /\.ndcohub\.ir$/ .test(clean);
}

export default function proxy(req: NextRequest) {
 const origin = req.headers.get("origin") ?? "";
 const isApi = req.nextUrl.pathname.startsWith("/api/") || req.nextUrl.pathname === "/ping";

 if (!isApi) return NextResponse.next();

 // 0000 00 preflight
 if (req.method === "OPTIONS") {
 const res = new NextResponse(null, { status: 204 });
 applyCors(res, origin);
 return res;
 }

 const res = NextResponse.next();
 applyCors(res, origin);
 return res;
}

function applyCors(res: NextResponse, origin: string) {
 if (isAllowed(origin)) {
 res.headers.set("Access-Control-Allow-Origin", origin);
 res.headers.set("Access-Control-Allow-Credentials", "true");
 } else if (!origin) {
 res.headers.set("Access-Control-Allow-Origin", "");
 }
 res.headers.set("Vary", "Origin");
 res.headers.set("Access-Control-Allow-Methods", "GET,POST,PATCH,PUT,DELETE,OPTIONS,HEAD");
 res.headers.set(
 "Access-Control-Allow-Headers",
 "Content-Type, x-auth-token, x-sync-node, x-sync-timestamp, x-sync-signature, x-sync-key, x-proxy-secret",
);
 res.headers.set("Access-Control-Max-Age", "86400");
}

export const config = {
 matcher: ["/api/:path*", "/ping"],
};

@@ mobile/src/simAuth.ts typescript
/**
 * =====
 * 00000 00000 0000 00 00000000 — React Native
 * -----
 * 00 000000 000 000:
 * 000000 «00000» 000000000 000 00000000 0000000 000000 0000 0000000000
 * (00000000 000000 0000 0 00000000) 00000 MSISDN 00 0000 0000000000 00000000000.
 * 00 iOS 00 0000 0000 API 00000 0000 0000 000000.
 *
 * 0000000000 0000 000000 00 00000 00000:
 * 0) 00000 00000 00000000 00000000 000000 (000 000000000 000000 00000)
 * 0) 0000000 «0000 0000000 0000000000» (ICCID/IMSI/0000000000) 00 00000000 0000
 * 0 0000000 00000000000 00 0000000 00000000
 * 0) 00 00000 00000 000000 → 00000 00 00 00000000 00 00000000 00000000 00000000
 * (00000 00 00000000 0 00000000 00 0000000000 0000000000)
 * =====
 */

import { NativeModules, PermissionsAndroid, Platform } from "react-native";
import DeviceInfo from "react-native-device-info";
import * as Keychain from "react-native-keychain";
import { HmacSHA256, enc } from "crypto-js";

// 00000 00000 0000
export const API_BASE = "https://namayandeelmi-javad.onrender.com";

/** 00000 00000000 00 MOBILE_APP_SECRET 0000 00000 000000 00000 */
const APP_SECRET = "REPLACE_WITH_MOBILE_APP_SECRET";

const DEVICE_ID_KEY = "sek_device_id";

/* ----- */
/* 00000000 */
/* ----- */

```

```

export async function requestSimPermissions(): Promise<boolean> {
 if (Platform.OS !== "android") return false;
 try {
 const perms = [
 PermissionsAndroid.PERMISSIONS.READ_PHONE_STATE,
 // 00 00000000 00+ 0000 000000 000000 0000 0000
 "android.permission.READ_PHONE_NUMBERS" as never,
];
 const result = await PermissionsAndroid.requestMultiple(perms);
 return Object.values(result).some((v) => v === PermissionsAndroid.RESULTS.GRANTED);
 } catch {
 return false;
 }
}

/* ----- */
/* 000000 00000000 000000000 */
/* ----- */

export type SimInfo = {
 /** 000000 — 0000 000 0000 0000 (0000 000000000000 0000000) */
 phoneNumber: string;
 /** 000000 00000000 0000000000 0000 000000 000000 0000 */
 fingerprint: string;
 carrier: string;
 simCount: number;
 hasSim: boolean;
 source: "line1" | "subscription" | "fallback" | "none";
};

function normalizePhone(raw?: string | null): string {
 if (!raw) return "";
 let d = String(raw).replace(/\\D/g, "");
 if (d.startsWith("0098")) d = "0" + d.slice(4);
 else if (d.startsWith("98") && d.length === 12) d = "0" + d.slice(2);
 else if (d.length === 10 && d.startsWith("9")) d = "0" + d;
 return /^0\\d{10}$/.test(d) ? d : "";
}

/** 000000 00000000 0000000000 00 0000000000 00 000000 */
export async function readSimInfo(): Promise<SimInfo> {
 const empty: SimInfo = {
 phoneNumber: "",
 fingerprint: "",
 carrier: "",
 simCount: 0,
 hasSim: false,
 source: "none",
 };

 if (Platform.OS !== "android") {
 // iOS: 0000000 000000 00000 000000 0000 000 0000000000 00 000000 0000
 const carrier = await DeviceInfo.getCarrier().catch(() => "");
 return { ...empty, carrier, hasSim: !!carrier, fingerprint: carrier ? `ios:${carrier}` : "" };
 }

 await requestSimPermissions();

 let phoneNumber = "";
 let fingerprint = "";
 let carrier = "";
 let simCount = 0;
 let source: SimInfo["source"] = "none";

 // 0 react-native-sim-data (000 000 0000) — 0000000000 00000000
 try {
 const SimData = NativeModules.RNSimData ?? require("react-native-sim-data").default;
 const info = await Promise.resolve(SimData?.getSimInfo?.());
 if (info) {
 const list = Array.isArray(info) ? info : Object.values(info);
 const first = (list[0] ?? info) as Record<string, unknown>;
 simCount = Array.isArray(list) ? list.length : 1;
 phoneNumber = normalizePhone(String(first.phoneNumber ?? first.number ?? ""));
 carrier = String(first.carrierName ?? first.displayName ?? "");
 const iccid = String(first.simSerialNumber ?? first.iccId ?? "");
 const imsi = String(first.subscriberId ?? "");
 const mccmnc = `${first.mcc ?? ""}${first.mnc ?? ""}`;
 fingerprint = iccid || imsi || `${carrier}:${mccmnc}`;
 if (phoneNumber) source = "subscription";
 }
 } catch {
 // 000000 000 00000 00 0000 000000 */
 }
}

```

```
// @react-native-device-info
import { DeviceInfo } from 'react-native-device-info';

if (!carrier) carrier = await DeviceInfo.getCarrier().catch(() => "");
if (!phoneNumber) {
 const p = await DeviceInfo.getPhoneNumber?.().catch(() => "");
 const n = normalizePhone(p);
 if (n) {
 phoneNumber = n;
 source = "line1";
 }
}
if (!fingerprint && carrier) {
 fingerprint = `${carrier}:${await DeviceInfo.getUniqueId().catch(() => "")}`;
 source = source === "none" ? "fallback" : source;
}

return {
 phoneNumber,
 fingerprint,
 carrier,
 simCount: simCount || (carrier ? 1 : 0),
 hasSim: !!carrier || !!fingerprint,
 source,
};
}

/* ----- */
/* 设备信息 设备信息 (Keychain 设备信息 设备信息) */
/* ----- */

export async function getDeviceId(): Promise<string> {
 try {
 const saved = await Keychain.getGenericPassword({ service: DEVICE_ID_KEY });
 if (saved && saved.password) return saved.password;
 } catch {
 /* ignore */
 }
 const fresh = (await DeviceInfo.getUniqueId().catch(() => "")) || `${Date.now().toString(36)}${Math.random().toString(36).slice(2)}`;
 const id = String(fresh).replace(/^[a-zA-Z0-9]/g, "").slice(0, 60);
 try {
 await Keychain.setGenericPassword(DEVICE_ID_KEY, id, { service: DEVICE_ID_KEY });
 } catch {
 /* ignore */
 }
 return id;
}

export async function getDeviceInfoLabel(): Promise<string> {
 const brand = await DeviceInfo.getBrand().catch(() => "");
 const model = await DeviceInfo.getModel().catch(() => "");
 const os = `${Platform.OS} ${await DeviceInfo.getSystemVersion().catch(() => "")}`;
 return `${brand} ${model} — ${os}.trim().slice(0, 200);
}

/* ----- */
/* 设备信息 设备信息 */
/* ----- */

function base64url(input: string) {
 return input.replace(/\+/g, "-").replace(/\//g, "_").replace(/=+$/, "");
}

function sign(canonical: string) {
 return base64url(HmacSHA256(canonical, APP_SECRET).toString(enc.Base64));
}

async function fetchJson(path: string, body: unknown, timeoutMs = 20000) {
 const ctrl = new AbortController();
 const timer = setTimeout(() => ctrl.abort(), timeoutMs);
 try {
 const res = await fetch(`${API_BASE}${path}`, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(body),
 signal: ctrl.signal,
 });
 const data = await res.json().catch(() => ({}));
 return { ok: res.ok, status: res.status, data };
 } finally {
 clearTimeout(timer);
 }
}

/* ----- */
/* 设备信息 设备信息 设备信息 */
/* ----- */
```

```

/* ----- */

export type LoginResult =
 | { status: "success"; token: string; user: Record<string, unknown> }
 | { status: "need-otp"; message: string; masked?: string }
 | { status: "error"; message: string; code?: string };

export async function loginWithSim(): Promise<LoginResult> {
 const sim = await readSimInfo();
 const deviceId = await getDeviceId();
 const deviceInfo = await getDeviceInfoLabel();

 if (!sim.hasSim) {
 return { status: "error", message: "SIM卡未检测到或已失效。请检查SIM卡是否插入正确。", code: "NO_SIM" };
 }

 // 防止 nonce 被重放攻击 (Replay Attack)
 const nonceRes = await fetchJson("/api/mobile/nonce", { deviceId }).catch(() => null);
 if (!nonceRes?.ok) {
 return { status: "error", message: "获取 nonce 失败。请重试。", code: "NONCE_ERROR" };
 }
 const nonce = String(nonceRes.data.nonce);
 // 验证 nonce 是否过期 (nonceRes.data.serverTime 与当前时间对比)
 const timestamp = Number(nonceRes.data.serverTime ?? Date.now());

 // 生成 canonical 字符串
 const canonical = [nonce, String(timestamp), deviceId, sim.phoneNumber, sim.fingerprint].join("|");
 const signature = sign(canonical);

 // 发送登录请求
 const res = await fetchJson("/api/mobile/login-with-phone", {
 deviceId,
 deviceInfo,
 nonce,
 timestamp,
 signature,
 phoneNumber: sim.phoneNumber,
 simFingerprint: sim.fingerprint,
 simCarrier: sim.carrier,
 simCount: sim.simCount,
 }).catch(() => null);

 if (!res) return { status: "error", message: "登录失败。请重试。", code: "LOGIN_ERROR" };

 if (res.ok && res.data.token) {
 await Keychain.setGenericPassword("token", String(res.data.token), { service: "sek_token" });
 return { status: "success", token: String(res.data.token), user: res.data.user };
 }

 if (res.data.needOtp) {
 return { status: "need-otp", message: String(res.data.error ?? ""), masked: res.data.masked };
 }

 return { status: "error", message: String(res.data.error ?? "未知错误"), code: res.data.code };
}

/* ----- */
/* 防止 token 被重放攻击 (Replay Attack) */
/* ----- */

export async function requestOtp(username: string, password: string) {
 const deviceId = await getDeviceId();
 const res = await fetchJson("/api/auth/otp", { action: "request", username, password, deviceId });
 return { ok: res.ok, message: String(res.data.message ?? res.data.error ?? ""), masked: res.data.masked };
}

export async function verifyOtp(username: string, password: string, code: string) {
 const deviceId = await getDeviceId();
 const deviceInfo = await getDeviceInfoLabel();
 const res = await fetchJson("/api/auth/otp", {
 action: "verify",
 username,
 password,
 code,
 deviceId,
 deviceInfo,
 });
 if (res.ok && res.data.token) {
 await Keychain.setGenericPassword("token", String(res.data.token), { service: "sek_token" });
 return { ok: true, token: String(res.data.token), user: res.data.user };
 }
 return { ok: false, message: String(res.data.error ?? "验证失败") };
}

```

```

export async function getSavedToken(): Promise<string | null> {
 try {
 const c = await Keychain.getGenericPassword({ service: "sek_token" });
 return c ? c.password : null;
 } catch {
 return null;
 }
}

@@ src/app/layout.tsx typescript
import type { Metadata, Viewport } from "next";
import type { ReactNode } from "react";
import "@/globals.css";
import ServiceWorker from "@/components/ServiceWorker";
import ConfirmProvider from "@/components/Confirm";
import ConnectionStatus from "@/components/ConnectionStatus";

export const metadata: Metadata = {
 title: "📄 📄📄📄📄 📄",
 description: "📄📄📄📄 📄📄 📄📄📄📄📄 📄📄📄📄📄 📄📄 📄📄📄📄📄 📄📄",
 applicationName: "📄📄 📄📄📄📄 📄",
 manifest: "/manifest.webmanifest",
 icons: {
 icon: [
 { url: "/icons/icon-192.png", sizes: "192x192", type: "image/png" },
 { url: "/icons/icon-512.png", sizes: "512x512", type: "image/png" },
 { url: "/favicon-32.png", sizes: "32x32", type: "image/png" },
],
 apple: [{ url: "/apple-touch-icon.png", sizes: "180x180", type: "image/png" }],
 shortcut: ["/favicon-32.png"],
 },
 appleWebApp: { capable: true, title: "📄📄 📄📄📄📄 📄", statusBarStyle: "default" },
 other: { "mobile-web-app-capable": "yes", "format-detection": "telephone=no" },
};

export const viewport: Viewport = {
 width: "device-width",
 initialScale: 1,
 maximumScale: 5,
 themeColor: "#0f766e",
};

export default function RootLayout({ children }: { children: ReactNode }) {
 return (
 <html lang="fa" dir="rtl">
 <head>
 /* 📄📄 📄📄📄📄📄📄 📄📄 📄📄📄📄📄📄 📄📄 PWABuilder 📄📄 📄📄📄📄 📄📄📄📄 */
 <script
 dangerouslySetInnerHTML={{
 __html:
 "if('serviceWorker' in navigator){navigator.serviceWorker.register('/sw.js',{scope:'/'}).catch(function(){",
 }}
 />
 </head>
 <body className="min-h-screen bg-slate-100 text-slate-900 antialiased">
 <ConfirmProvider>{children}</ConfirmProvider>
 <ServiceWorker />
 <ConnectionStatus />
 </body>
 </html>
);
}

@@ src/app/page.tsx typescript
"use client";

import { useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { patchFetch } from "@/components/SessionProvider";
import { getCachedUser } from "@/lib/offline-session";

export default function Home() {
 const router = useRouter();
 const [slow, setSlow] = useState(false);

 useEffect(() => {
 patchFetch();

 // 📄 📄📄 📄📄📄📄📄📄 📄📄📄📄 📄📄📄📄 📄📄📄📄 📄📄📄📄 📄📄📄📄📄 (📄📄 📄📄 📄📄📄📄📄)
 const cached = getCachedUser();
 if (cached) {
 router.replace(cached.role === "rep" ? "/panel" : "/admin");
 return;
 }
 });
}

```

```

// () 000 0000000 000000 0000000 00 0000 0000
if (!navigator.onLine) {
 router.replace("/login");
 return;
}

const slowTimer = setTimeout(() => setSlow(true), 5000);
// 000 000 00 00000 000000 000000 00 0000 0000 00000000 00 00000 0000 000000
const bail = setTimeout(() => router.replace("/login"), 12000);
// 00000000000 0000 (Render 0000000 000 00 00000000 000000 00000000)
fetch("/api/ping", { cache: "no-store" }).catch(() => undefined);

fetch("/api/auth/me", { cache: "no-store" })
 .then((r) => (r.ok ? r.json() : null))
 .then((d) => router.replace(!d?.user ? "/login" : d.user.role === "rep" ? "/panel" : "/admin"))
 .catch(() => router.replace("/login"))
 .finally(() => {
 clearTimeout(slowTimer);
 clearTimeout(bail);
 });

return () => {
 clearTimeout(slowTimer);
 clearTimeout(bail);
};
}, [router]);

return (
 <main className="flex min-h-screen flex-col items-center justify-center gap-3 bg-slate-100 p-6 text-center">
 <div className="size-10 animate-spin rounded-full border-4 border-teal-200 border-t-teal-600" />
 <p className="text-sm font-bold text-slate-600">00 000 000000000 «000 00000000 00»...</p>
 {slow ? (
 <p className="max-w-xs text-xs leading-6 text-slate-500">
 000000 00 0000 000 000 00 0000 00 000 000000 000. 000 0000 000 0000 — 0000000 0000000 000000 0000000.
 </p>
) : null}

 0000 0000000 00 0000 0000

 </main>
);
}

@@ src/components/BackButton.tsx typescript
"use client";

import { usePathname, useRouter } from "next/navigation";

/** 0000 00000 — 00 000 000000 000000 0000 0000 00 000 000000 0000 0000000 */
export default function BackButton() {
 const router = useRouter();
 const pathname = usePathname();
 const isRoot = pathname === "/panel" || pathname === "/admin" || pathname === "/login" || pathname === "/";

 const home = pathname.startsWith("/admin") ? "/admin" : "/panel";

 return (
 <div className="mb-3 flex items-center gap-2">
 <button
 onClick={() => {
 if (window.history.length > 1) router.back();
 else router.push(home);
 }}
 className="inline-flex items-center gap-1 rounded-xl bg-white px-3 py-2 text-xs font-bold text-slate-700 shadow-sm ring-1 ring-slate-200 hover:bg-slate-50"
 title="0000000 00 0000 000"
 >
 → 0000000
 </button>
 {!isRoot ? (
 <button
 onClick={() => router.push(home)}
 className="inline-flex items-center gap-1 rounded-xl bg-white px-3 py-2 text-xs font-bold text-teal-700 shadow-sm ring-1 ring-slate-200 hover:bg-teal-50"
 title="0000 0000"
 >
 0 0000 0000
 </button>
) : null}
 </div>
);
}

@@ src/components/Charts.tsx typescript

```

```

"use client";

import { toPersianDigits } from "@lib/jalali";

export type Series = { label: string; value: number; color?: string };

const PALETTE = ["#0d9488", "#0ea5e9", "#6366f1", "#f59e0b", "#e11d48", "#8b5cf6", "#10b981", "#f97316"];

export function BarChart({ data, height = 180, unit = "" }: { data: Series[]; height?: number; unit?: string }) {
 if (data.length === 0) return <Empty />;
 const max = Math.max(...data.map((d) => d.value), 1);
 return (
 <div className="scroll-x">
 <div className="flex items-end gap-2" style={{ height, minWidth: data.length * 52 }}>
 {data.map((d, i) => {
 const h = Math.max(4, (d.value / max) * (height - 34));
 return (
 <div key={` ${d.label}-${i} `} className="flex flex-1 flex-col items-center justify-end gap-1">
 {toPersianDigits(d.value)}
 <div
 className="w-full rounded-t-lg transition-all"
 style={{ height: h, background: d.color ?? PALETTE[i % PALETTE.length], minWidth: 26 }}
 title={` ${d.label}: ${d.value}${unit} `}
 />

 {d.label}

 </div>
);
 })}
 </div>
 </div>
);
}

/**
 * نمودار خطی (Line Chart)
 * @param labels لیست برچسب‌ها
 * @param series آرایه از سری‌ها (نام، مقادیر، رنگ)
 * @param height ارتفاع نمودار
 */
export function LineChart({
 labels,
 series,
 height = 200,
}: {
 labels: string[];
 series: { name: string; values: number[]; color?: string }[];
 height?: number;
}) {
 if (labels.length === 0 || series.length === 0) return <Empty />;
 const W = Math.max(320, labels.length * 46);
 const H = height;
 const pad = { t: 12, r: 8, b: 24, l: 34 };
 const max = Math.max(1, ...series.flatMap((s) => s.values));
 const x = (i: number) => pad.l + (i * (W - pad.l - pad.r)) / Math.max(1, labels.length - 1);
 const y = (v: number) => H - pad.b - (v / max) * (H - pad.t - pad.b);

 return (
 <div className="scroll-x">
 <svg width={W} height={H} className="min-w-full">
 {[0, 0.5, 1].map((f) => (
 <g key={f}>
 <line x1={pad.l} x2={W - pad.r} y1={y(max * f)} y2={y(max * f)} stroke="#e2e8f0" strokeWidth={1} />
 <text x={4} y={y(max * f) + 4} fontSize={9} fill="#94a3b8">
 {toPersianDigits(Math.round(max * f))}
 </text>
 </g>
))}
 {series.map((s, si) => {
 const color = s.color ?? PALETTE[si % PALETTE.length];
 const pts = s.values.map((v, i) => ` ${x(i)}, ${y(v)} `).join(" ");
 return (
 <g key={s.name}>
 <polyline points={pts} fill="none" stroke={color} strokeWidth={2.5} strokeLinejoin="round" />
 {s.values.map((v, i) => (
 <circle key={i} cx={x(i)} cy={y(v)} r={3} fill={color}>
 <title>{s.name} — {labels[i]}: ${v}</title>
 </circle>
))}
 </g>
);
 })}
 </svg>
 <div className="mt-1 flex flex-wrap gap-2">
 {labels.map((l, i) => (
 <text key={i} x={x(i)} y={H - 6} fontSize={9} fill="#64748b" textAnchor="middle">
 {l}
 </text>
))}
 </div>
 </div>
);
}

```





```
export default function Combobox({
 value,
 onChange,
 options,
 category,
 onAdded,
 canAdd = true,
 placeholder = "□□□□□□ □□ □□□□ □□□□...",
 parent = "",
 parentLabel = "",
 requireParent = false,
 disabled = false,
 selectOnly = false,
}): {
 value: string;
 onChange: (v: string) => void;
 options: string[];
 category?: string;
 onAdded?: (v: string) => void;
 canAdd?: boolean;
 placeholder?: string;
 /** □□□□□ □□□□ (□□□□□ □□□ □□□□□ □□□□ □□□□ □□□) */
 parent?: string;
 parentLabel?: string;
 requireParent?: boolean;
 disabled?: boolean;
 /** □□□ □□□ □□□□□□ □□ □□□□□ — □□□□□ □□□□□□□ □□ □□□ □□□□□□ □□□□□ □□□□□ □□□□□ */
 selectOnly?: boolean;
} {
 const [open, setOpen] = useState(false);
 const [query, setQuery] = useState(value);
 const [saving, setSaving] = useState(false);
 const [typing, setTyping] = useState(false);
 const [note, setNote] = useState("");
 const ref = useRef<HTMLDivElement>(null);
 const typingTimer = useRef<ReturnType<typeof setTimeout> | null>(null);
 const confirm = useConfirm();

 const blocked = disabled || (requireParent && !parent.trim());

 useEffect(() => setQuery(value), [value]);

 useEffect(() => {
 if (!open) return;
 const handler = (e: MouseEvent) => {
 if (ref.current && !ref.current.contains(e.target as Node)) setOpen(false);
 };
 document.addEventListener("mousedown", handler);
 return () => document.removeEventListener("mousedown", handler);
 }, [open]);

 useEffect(
 () => () => {
 if (typingTimer.current) clearTimeout(typingTimer.current);
 },
 [],
);

 const filtered = useMemo(() => {
 const q = query.trim().toLowerCase();
 if (!q) return options.slice(0, 80);
 const words = q.split(/\s+/);
 return options.filter((o) => words.every((w) => o.toLowerCase().includes(w))).slice(0, 80);
 }, [query, options]);

 const trimmed = query.trim();
 const exact = options.some((o) => o.trim() === trimmed);
 // □□□□ □□□□□□ □□□ □□ □ □□□ □□ □□□□□ □ □□ □□ □□□□ □□□□ □□□□ □□□□ □□□□□□
 const showAdd = !selectOnly && canAdd && !!category && !blocked && trimmed.length >= 2 && !exact && !typing;

 const markTyping = () => {
 setTyping(true);
 if (typingTimer.current) clearTimeout(typingTimer.current);
 typingTimer.current = setTimeout(() => setTyping(false), 700);
 };

 const addNow = async () => {
 if (!trimmed || !category || saving) return;
 const ok = await confirm({
 title: "□□□□□□ □□□□□ □□□□",
 message: `«${trimmed}»${parent ? ` □□□□ «${parent}»` : ""} □□ □□□□ «□□□□□ □□□□\n□□□□ □□□□□ □□□□ □□□ □□□ □□□□□□ □□□□ □□□□ □□□□□□ □□□.`,
 confirmText: "□□□□□□ □□ □□□□",
 });
 if (!ok) return;
 };
}
```

```

setSaving(true);
const res = await fetch("/api/options", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ category, value: trimmed, parent }),
}).catch(() => null);
setSaving(false);

const data = await res?.json().catch(() => ({}));
if (res?.ok) {
 onChange(trimmed);
 onAdded?.(trimmed);
 setOpen(false);
 setNote(data?.duplicate ? "⬅️ ⬅️⬅️⬅️ ⬅️ ⬅️ ⬅️⬅️ ⬅️⬅️" : "⬅️ ⬅️ ⬅️⬅️ ⬅️⬅️ ⬅️");
 setTimeout(() => setNote(""), 2500);
} else {
 setNote(data?.error ?? "⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️ ⬅️⬅️");
 setTimeout(() => setNote(""), 3500);
}
};

return (
 <div className="relative" ref={ref}>
 <div className="flex gap-1">
 <input
 value={query}
 placeholder={blocked ? `⬅️⬅️⬅️⬅️ ${parentLabel || "⬅️⬅️⬅️ ⬅️⬅️"} ⬅️ ⬅️⬅️⬅️⬅️ ⬅️⬅️` : placeholder}
 disabled={blocked}
 onFocus={() => !blocked && setOpen(true)}
 onChange={(e) => {
 setQuery(e.target.value);
 setOpen(true);
 markTyping();
 // ⬅️ ⬅️⬅️ ⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️ ⬅️ ⬅️⬅️⬅️⬅️ ⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️
 if (!selectOnly) onChange(e.target.value);
 }}
 onBlur={() => {
 // ⬅️ ⬅️⬅️ ⬅️⬅️ ⬅️⬅️ ⬅️⬅️ ⬅️⬅️ ⬅️ ⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️ ⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️⬅️
 if (selectOnly) setTimeout(() => setQuery(value), 150);
 }}
 onKeyDown={(e) => {
 // Enter ⬅️⬅️⬅️ ⬅️⬅️ ⬅️ ⬅️⬅️⬅️ ⬅️ ⬅️⬅️⬅️ ⬅️ ⬅️⬅️⬅️ ⬅️⬅️
 if (e.key === "Enter") e.preventDefault();
 if (e.key === "Escape") setOpen(false);
 }}
 className="w-full rounded-xl border border-slate-300 bg-white px-3 py-2.5 text-sm outline-none transition focus:
s:border-teal-500 focus:ring-2 focus:ring-teal-100 disabled:cursor-not-allowed disabled:bg-slate-100 disabled:text-slate
-400"
 />
 {showAdd ? (
 <button
 type="button"
 onClick={addNow}
 disabled={saving}
 title={`⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️ ⬅️ ⬅️⬅️⬅️⬅️`}
 className="shrink-0 rounded-xl bg-emerald-600 px-3 text-xs font-bold text-white hover:bg-emerald-700 disable
d:opacity-50"
 >
 {saving ? "...": "⬅️ ⬅️⬅️⬅️⬅️"}
 </button>
) : null}
 </div>

 {note ? <p className="mt-1 text-[10px] font-bold text-emerald-700">{note}</p> : null}
 {selectOnly && !blocked && options.length > 0 ? (
 <p className="mt-1 text-[10px] text-slate-400">{options.length.toLocaleString("fa-IR")} ⬅️⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️</p>
) : null}

 {open && !blocked ? (
 <div className="fade-in absolute z-30 mt-1 max-h-56 w-full overflow-y-auto rounded-xl bg-white py-1 shadow-xl ri
ng-1 ring-slate-200">
 {filtered.length === 0 ? (
 <div className="px-3 py-2 text-[11px] text-slate-400">
 {selectOnly
 ? "⬅️⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️ — ⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️"
 : trimmed.length >= 2
 ? "⬅️⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️ — ⬅️ ⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️⬅️"
 : "⬅️⬅️⬅️ ⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️⬅️"}
 </div>
) : null}
 {filtered.map((o) => (
 <button
 key={o}
 type="button"

```

```

 onMouseDown={e => e.preventDefault()}
 onClick={() => {
 onChange(o);
 setQuery(o);
 setOpen(false);
 }}
 className={`block w-full px-3 py-2 text-right text-sm hover:bg-teal-50 ${
 o === value ? "bg-teal-50 font-bold text-teal-700" : "text-slate-700"
 }`}
 >
 {o}
 </button>
)}
</div>
) : null}
</div>
);
}

```

```

@@ src/components/Confirm.tsx typescript
"use client";

```

```

import { createContext, useCallback, useContext, useState, type ReactNode } from "react";

```

```

type Opts = {
 title: string;
 message?: string;
 confirmText?: string;
 cancelText?: string;
 danger?: boolean;
};

```

```

type Ctx = (o: Opts) => Promise<boolean>;

```

```

const ConfirmCtx = createContext<Ctx>(async () => true);

```

```

/**
 * 确认对话框
 */
const confirm = useConfirm();
if (!confirm({ title: '...' })) return;
export function useConfirm() {
 return useContext(ConfirmCtx);
}

```

```

export default function ConfirmProvider({ children }: { children: ReactNode }) {
 const [state, setState] = useState<(Opts & { resolve: (v: boolean) => void }) | null>(null);

```

```

 const confirm = useCallback<Ctx>(
 (o) => new Promise<boolean>((resolve) => setState({ ...o, resolve })),
 [],
);

```

```

 const close = (v: boolean) => {
 state?.resolve(v);
 setState(null);
 };

```

```

 return (
 <ConfirmCtx.Provider value={confirm}>
 {children}
 {state ? (

```

```

 <div
 className="fixed inset-0 z-[100] flex items-end justify-center bg-slate-900/60 p-0 sm:items-center sm:p-4"
 onClick={() => close(false)}
 >
 <div
 className="fade-in w-full max-w-sm rounded-t-3xl bg-white p-4 shadow-2xl sm:rounded-2xl"
 onClick={e => e.stopPropagation()}
 >
 <div className="mb-2 flex items-start gap-3">
 <span
 className={`text-2xl ${state.danger ? "" : ""}`}
 >{state.danger ? "⚠️" : ""}
 <div className="min-w-0 flex-1">
 <h3
 className="text-sm font-black text-slate-800"
 >{state.title}</h3>
 {state.message ? (
 <p
 className="mt-1 whitespace-pre-wrap text-xs leading-6 text-slate-600"
 >{state.message}</p>
) : null}
 </div>
 </div>
 <div className="mt-4 flex gap-2">
 <button
 onClick={() => close(true)}
 className={`flex-1 rounded-xl px-4 py-2.5 text-sm font-bold text-white ${
 state.danger ? "bg-rose-600 hover:bg-rose-700" : "bg-teal-600 hover:bg-teal-700"
 }`}
 >
 {state.confirmText ?? "确认"}
 </button>
 <button

```

```

 onClick={() => close(false)}
 className="flex-1 rounded-xl bg-white px-4 py-2.5 text-sm font-bold text-slate-700 ring-1 ring-slate-300
hover:bg-slate-50"
 >
 {state.cancelText ?? "□□□□□□"}
 </button>
</div>
</div>
</div>
) : null}
</ConfirmCtx.Provider>
);
}

@@ src/components/ConnectionStatus.tsx typescript
"use client";

import { useCallback, useEffect, useRef, useState } from "react";

/**
 * □□□□ □□□□ □□□□ + □□□□□□ □□ □□□□ □□□□□□.
 *
 * □□□□□□:
 * - □□□□□□ «□□□□ □□□□□□ □□□□□□»□ □□□ □□ □□□□ □□□□ □□□□ □□□□ □□□□□□
 * □ □□□□□ □□□□□□□□□□ □□ □□□ □□□□ □□□□□ □□□□.
 * - «□□□□□□» □□□□ □□□□□ □□□□□ □□□□□□ □□ □□□□□□ □□ □□□□ □□□□□□
 * (□□ □□□□□□ navigator.onLine □□ □□□ □□□□□□□□ □□□□□ □□□□□□□□□□□□ □□□□).
 * - □□□□ □□□□□□□□ □□□□□□□□ □□□□□ □□□□□ «□□□□□□ □□□□□□□□ □□□□□□□□□□» □□ □□□□□□□□
 * □ □□□□□□□□□ □□□□ □□□□□ □□□□ □□□□□ □□□□□□ □□ □□□□□ □□□□□□□□.
 */

type State = "online" | "checking" | "offline";

const PROBE_URL = "/ping";
const CHECK_INTERVAL = 25_000;

export default function ConnectionStatus() {
 const [state, setState] = useState<State>("online");
 const [pending, setPending] = useState(0);
 const [flash, setFlash] = useState("");
 const [sending, setSending] = useState(false);
 const [dismissed, setDismissed] = useState(false);
 const failures = useRef(0);

 /** □□□ □□□□ □□□□□□ □□ □□□□□ □□□□ (□□□□ □□□□ □□ □□□□□□□□) */
 const probe = useCallback(async (): Promise<boolean> => {
 if (typeof navigator !== "undefined" && !navigator.onLine) return false;
 try {
 const ctrl = new AbortController();
 const timer = setTimeout(() => ctrl.abort(), 8000);
 const res = await fetch(`${PROBE_URL}?t=${Date.now()}`, {
 cache: "no-store",
 signal: ctrl.signal,
 credentials: "omit",
 });
 clearTimeout(timer);
 return res.ok;
 } catch {
 return false;
 }
 }, []);

 const check = useCallback(async () => {
 const ok = await probe();
 if (ok) {
 failures.current = 0;
 setState("online");
 setDismissed(false);
 // □□ □□□□□□ □□□□□□□□ □□ □□□□□□ □□□□□ □□□□□□□□
 navigator.serviceWorker?.controller?.postMessage("flush");
 } else {
 failures.current += 1;
 // □□ □□□□ □□□□□ □□ □□ □□□□□ □□□□ □□□ □□□ □□□□□□□□ □□□□□□□□ □□□□
 setState(failures.current >= 2 ? "offline" : "checking");
 }
 navigator.serviceWorker?.controller?.postMessage("queue-count");
 }, [probe]);

 useEffect(() => {
 check();
 const iv = setInterval(() => {
 if (!document.hidden) check();
 }, CHECK_INTERVAL);

 const onOnline = () => check();

```

```

const onFocus = () => check();
window.addEventListener("online", onOnline);
window.addEventListener("offline", () => setState("offline"));
window.addEventListener("focus", onFocus);
document.addEventListener("visibilitychange", onFocus);

const onMsg = (e: MessageEvent) => {
 if (e.data?.type === "queue-status") {
 setPending(e.data.pending ?? 0);
 if (e.data.sent > 0) {
 setFlash(`🔴 ${e.data.sent} 🟡🟡🟡🟡🟡🟡 🟢 🟢🟢🟢🟢 🟢🟢🟢🟢`);
 setTimeout(() => setFlash(""), 6000);
 }
 }
 if (e.data?.type === "queued") {
 setPending((p) => p + 1);
 setFlash("🟢 🟢🟢🟢🟢🟢 🟢🟢🟢🟢 🟢 🟢 🟢 🟢🟢 🟢🟢🟢🟢🟢 🟢🟢🟢🟢🟢🟢");
 setTimeout(() => setFlash(""), 6000);
 }
};
navigator.serviceWorker?.addEventListener("message", onMsg);

return () => {
 clearInterval(iv);
 window.removeEventListener("online", onOnline);
 window.removeEventListener("focus", onFocus);
 document.removeEventListener("visibilitychange", onFocus);
 navigator.serviceWorker?.removeEventListener("message", onMsg);
};
}, [check]);

const sendNow = async () => {
 setSending(true);
 setFlash("🟢 🟢 🟢🟢 🟢🟢🟢🟢 🟢🟢🟢🟢🟢...");
 const ok = await probe();
 if (!ok) {
 setSending(false);
 setFlash("🟢 🟢🟢🟢 🟢🟢🟢🟢 🟢 🟢🟢🟢 🟢🟢🟢🟢 🟢🟢 🟢🟢 🟢🟢🟢🟢 🟢🟢 🟢🟢🟢");
 setTimeout(() => setFlash(""), 6000);
 return;
 }
 navigator.serviceWorker?.controller?.postMessage("flush");
 setTimeout(() => {
 navigator.serviceWorker?.controller?.postMessage("queue-count");
 setSending(false);
 }, 2500);
};

const offline = state === "offline";
const showBar = (offline && !dismissed) || pending > 0 || !!flash;
if (!showBar) return null;

return (
 <div className="pointer-events-none fixed inset-x-0 bottom-0 z-[90] flex flex-col items-center gap-2 p-3">
 {flash ? (
 <div className="pointer-events-auto w-full max-w-md rounded-xl bg-slate-900/95 px-3 py-2 text-center text-xs font-bold text-white shadow-lg">
 {flash}
 </div>
) : null}

 {offline && !dismissed ? (
 <div className="pointer-events-auto w-full max-w-md rounded-2xl bg-amber-500 px-3 py-2.5 text-white shadow-xl">
 <div className="flex items-start gap-2">
 🔴
 <div className="min-w-0 flex-1">
 <div className="text-xs font-black">🟢🟢🟢🟢 🟢 🟢🟢🟢 🟢🟢🟢🟢🟢🟢</div>
 <div className="mt-0.5 text-[11px] leading-5 text-amber-50">
 🟢🟢🟢🟢 🟢🟢🟢🟢 🟢🟢🟢🟢🟢 🟢🟢 🟢 🟢🟢🟢 🟢🟢🟢. 🟢🟢🟢🟢🟢🟢🟢 🟢 🟢🟢 🟢🟢🟢🟢 🟢🟢🟢 🟢🟢🟢🟢🟢 🟢🟢🟢🟢 🟢 🟢🟢
 🟢🟢 🟢🟢 🟢🟢🟢🟢 🟢🟢🟢🟢 🟢.
 </div>
 <div className="mt-2 flex flex-wrap gap-2">
 <button
 onClick={check}
 className="rounded-lg bg-white px-3 py-1 text-[11px] font-bold text-amber-700">
 >
 🟢 🟢🟢🟢 🟢🟢🟢
 </button>
 <a
 href="/diagnostics"
 className="rounded-lg bg-white/20 px-3 py-1 text-[11px] font-bold text-white ring-1 ring-white/40">
 >
 🟢 🟢🟢🟢🟢🟢

 <button

```

```

 onClick={() => setDismissed(true)}
 className="rounded-lg bg-white/20 px-3 py-1 text-[11px] font-bold text-white ring-1 ring-white/40"
 >
 □□□□
 </button>
 </div>
 </div>
</div>
) : null}

{pending > 0 ? (
 <div className="pointer-events-auto w-full max-w-md rounded-2xl bg-teal-700 px-3 py-2.5 text-white shadow-xl">
 <div className="flex flex-wrap items-center gap-2">
 □
 <div className="min-w-0 flex-1">
 <div className="text-xs font-black">
 {pending.toLocaleString("fa-IR")} □□□□ □□ □□□□□□ □□□□□□
 </div>
 <div className="text-[11px] text-teal-50">
 {offline ? "□□□□□□ □□□ □□□ □□□□□□□□ □□□□□ □□□□□□" : "□□□□□□□□ □□□ □□□ — □□□□□□□□□□ □□□□ □□□□ □□□□□"}
 </div>
 </div>
 <button
 onClick={sendNow}
 disabled={sending}
 className="rounded-lg bg-white px-3 py-1.5 text-[11px] font-black text-teal-800 disabled:opacity-60"
 >
 {sending ? "□□ □□□ □□□□□..." : "□□□□□□ □□□□□□"}
 </button>
 </div>
 </div>
) : null}
</div>
);
}

@@ src/components/FileUploader.tsx typescript
"use client";

import { useCallback, useEffect, useRef, useState } from "react";
import { Badge, Button } from "@/ui";
import { toPersianDigits } from "@/lib/jalali";

export type Att = {
 id: number;
 fileName: string;
 mimeType: string;
 size: number;
 repName?: string;
 ownerId?: number | null;
};

const fmtSize = (n: number) =>
 n > 1024 * 1024 ? `${toPersianDigits((n / 1024 / 1024).toFixed(1))} □□□□□□□` : `${toPersianDigits(Math.round(n / 1024))} □□□□□□□`;

export function printFile(id: number, mime: string) {
 const url = `/api/attachments/${id}`;
 const w = window.open("", "_blank", "width=900,height=700");
 if (!w) return;
 const content = mime.startsWith("image/")
 ? ``
 : `<iframe src="${url}" style="border:0;width:100%;height:95vh" onload="window.focus();setTimeout(()=>window.print(),600)"></iframe>`;
 w.document.write(
 `<html dir="rtl"><head><meta charset="utf-8"><title>□□□ □□□□</title></head><body style="margin:0">${content}</body></html>`,
);
 w.document.close();
}

export function FileList({
 files,
 onDelete,
 compact = false,
}: {
 files: Att[];
 onDelete?: (id: number) => void;
 compact?: boolean;
}) {
 if (files.length === 0) return <p className="text-xs text-slate-400">□□□□□□ □□□□□□□□ □□□□ □□□</p>;
 return (
 <div className={`grid gap-2 ${compact ? "grid-cols-2 sm:grid-cols-3" : "grid-cols-1 sm:grid-cols-2"}`>
 {files.map((f) => (

```

```

 <div key={f.id} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
 {f.mimeType.startsWith("image/") ? (
 // eslint-disable-next-line @next/next/no-img-element
 <img
 src={`\api/attachments/${f.id}`}
 alt={f.fileName}
 className="mb-2 h-28 w-full cursor-pointer rounded-lg object-cover"
 onClick={() => window.open(`\api/attachments/${f.id}`, "_blank")}
 />
) : (
 <div className="mb-2 flex h-28 items-center justify-center rounded-lg bg-white text-3xl"></div>
)}
 <div className="truncate text-[11px] font-bold text-slate-700" title={f.fileName}>
 {f.fileName}
 </div>
 <div className="text-[10px] text-slate-400">
 {fmtSize(f.size)} {f.repName ? `— ${f.repName}` : ""}
 </div>
 <div className="mt-1 flex flex-wrap gap-1">
 <a
 href={`\api/attachments/${f.id}`}
 target="_blank"
 rel="noreferrer"
 className="rounded-lg bg-sky-100 px-2 py-1 text-[10px] font-bold text-sky-700"
 >
 □ □ □ □ □ □

 <a
 href={`\api/attachments/${f.id}?download=1`}
 className="rounded-lg bg-emerald-100 px-2 py-1 text-[10px] font-bold text-emerald-700"
 >
 □ □ □ □ □ □ □ □

 <button
 onClick={() => printFile(f.id, f.mimeType)}
 className="rounded-lg bg-slate-200 px-2 py-1 text-[10px] font-bold text-slate-700"
 >
 □ □ □ □ □ □
 </button>
 {onDelete ? (
 <button
 onClick={() => onDelete(f.id)}
 className="rounded-lg bg-rose-100 px-2 py-1 text-[10px] font-bold text-rose-700"
 >
 □ □ □
 </button>
) : null}
 </div>
 </div>
))}
</div>
);
};

/** □□□□□□□□ □□□□/□□□□. □□ □□□□ □□□ □□□□□□ □□□□□□ □□□□ □□□□ □□□□□ □ □□□ □□□□ □□□□□□□□. */
export default function FileUploader({
 ownerType = "doctor",
 ownerId = null,
 onChangeIds,
}: {
 ownerType?: string;
 ownerId?: number | null;
 onChangeIds?: (ids: number[]) => void;
}) {
 const [files, setFiles] = useState<Att[]>([]);
 const [busy, setBusy] = useState(false);
 const [err, setErr] = useState("");
 const inputRef = useRef<HTMLInputElement>(null);
 const cb = useRef(onChangeIds);
 cb.current = onChangeIds;

 useEffect(() => {
 cb.current?.(files.map((f) => f.id));
 }, [files]);

 const load = useCallback(async () => {
 if (!ownerId) return;
 const res = await fetch(`\api/attachments?ownerType=${ownerType}&ownerId=${ownerId}`, { cache: "no-store" });
 if (res.ok) setFiles((await res.json()).rows ?? []);
 }, [ownerType, ownerId]);

 useEffect(() => {
 load();
 }, [load]);
}

```



```

const upload = async (list: FileList | null) => {
 if (!list?.length) return;
 setErr("");
 setBusy(true);
 for (const file of Array.from(list).slice(0, 5)) {
 if (file.size > 3 * 1024 * 1024) {
 setErr(`«${file.name}» بزرگتر از ۳ مگابایت است`);
 continue;
 }
 const data = await new Promise<string>((resolve) => {
 const r = new FileReader();
 r.onload = () => resolve(String(r.result));
 r.readAsDataURL(file);
 });
 const res = await fetch("/api/attachments", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ ownerType, ownerId, fileName: file.name, data }),
 });
 if (res.ok) {
 const d = await res.json();
 setFiles((p) => [d.row, ...p]);
 } else {
 const d = await res.json().catch(() => ({}));
 setErr(d.error ?? "خطای نامشخص در آپلود فایل");
 }
 }
 setBusy(false);
 if (inputRef.current) inputRef.current.value = "";
};

const remove = async (id: number) => {
 await fetch(`/api/attachments?id=${id}`, { method: "DELETE" });
 setFiles((p) => p.filter((f) => f.id !== id));
};

return (
 <div className="space-y-2">
 <div className="flex flex-wrap items-center gap-2">
 <input
 ref={inputRef}
 type="file"
 multiple
 accept="image/*, application/pdf, .doc, .docx, .xls, .xlsx"
 onChange={(e) => upload(e.target.files)}
 className="hidden"
 id={`file-${ownerType}`}
 />
 <Button variant="soft" onClick={() => inputRef.current?.click()} disabled={busy}>
 {busy ? "در حال آپلود فایل‌هاست..." : "آپلود فایل‌ها"}
 </Button>
 <Badge tone="slate">{toPersianDigits(files.length)} فایل</Badge>
 فایل PDF بزرگتر از ۳ مگابایت نیست
 </div>
 {err ? <p className="text-xs font-bold text-rose-600">{err}</p> : null}
 <FileList files={files} onDelete={remove} compact />
 </div>
);
}

```

```

@@ src/components/JalaliDateInput.tsx typescript
"use client";

```

```

import { useEffect, useMemo, useRef, useState } from "react";
import {
 JALALI_MONTHS,
 WEEK_DAYS,
 formatJalali,
 jalaliMonthLength,
 jalaliWeekDay,
 maskJalaliInput,
 parseJalali,
 toPersianDigits,
 todayJalali,
} from "@lib/jalali";

export default function JalaliDateInput({
 value,
 onChange,
 placeholder = "YYYY/MM/DD",
}: {
 value: string;
 onChange: (v: string) => void;
 placeholder?: string;
}) {

```

```

const [open, setOpen] = useState(false);
const boxRef = useRef<HTMLDivElement>(null);
const parsed = parseJalali(value) ?? parseJalali(todayJalali());!;
const [view, setView] = useState<[number, number]>([parsed[0], parsed[1]]);

useEffect(() => {
 const p = parseJalali(value);
 if (p) setView([p[0], p[1]]);
}, [value]);

useEffect(() => {
 if (!open) return;
 const handler = (e: MouseEvent) => {
 if (boxRef.current && !boxRef.current.contains(e.target as Node)) setOpen(false);
 };
 document.addEventListener("mousedown", handler);
 return () => document.removeEventListener("mousedown", handler);
}, [open]);

const grid = useMemo(() => {
 const [jy, jm] = view;
 const len = jalaliMonthLength(jy, jm);
 const first = jalaliWeekDay(jy, jm, 1);
 const cells: (number | null)[] = Array(first).fill(null);
 for (let d = 1; d <= len; d++) cells.push(d);
 while (cells.length % 7 !== 0) cells.push(null);
 return cells;
}, [view]);

const selected = parseJalali(value);

return (
 <div className="relative" ref={boxRef}>
 <div className="flex gap-2">
 <input
 value={value}
 inputMode="numeric"
 placeholder={placeholder}
 onChange={(e) => onChange(maskJalaliInput(e.target.value))}
 className="w-full rounded-xl border border-slate-300 bg-white px-3 py-2.5 text-sm tracking-widest outline-none
transition focus:border-teal-500 focus:ring-2 focus:ring-teal-100"
 />
 <button
 type="button"
 onClick={() => setOpen((v) => !v)}
 className="shrink-0 rounded-xl bg-teal-50 px-3 text-lg ring-1 ring-teal-200 hover:bg-teal-100"
 title="□□□□□□ □□ □□□□□□"
 >
 □
 </button>
 </div>

 {open ? (
 <div className="fade-in absolute z-30 mt-2 w-[19rem] max-w-[92vw] rounded-2xl bg-white p-3 shadow-xl ring-1 ring
-slate-200">
 <div className="mb-2 flex items-center justify-between gap-1">
 <button
 type="button"
 className="rounded-lg px-2 py-1 text-slate-600 hover:bg-slate-100"
 onClick={() => setView(([y, m]) => (m === 1 ? [y - 1, 12] : [y, m - 1]))}
 >
 </button>
 <div className="flex items-center gap-2 text-sm font-bold text-slate-700">
 <select
 value={view[1]}
 onChange={(e) => setView(([y]) => [y, Number(e.target.value)])}
 className="rounded-lg border border-slate-200 px-2 py-1 text-xs"
 >
 {JALALI_MONTHS.map((m, i) => (
 <option key={m} value={i + 1}>
 {m}
 </option>
))}
 </select>
 <input
 type="number"
 value={view[0]}
 onChange={(e) => setView(([, m]) => [Number(e.target.value) || 1400, m])}
 className="w-20 rounded-lg border border-slate-200 px-2 py-1 text-xs"
 />
 </div>
 <button
 type="button"
 className="rounded-lg px-2 py-1 text-slate-600 hover:bg-slate-100"

```

```

 onClick={() => setView([y, m]) => (m === 12 ? [y + 1, 1] : [y, m + 1])}}
 >
 </button>
</div>
<div className="grid grid-cols-7 gap-1 text-center text-[11px] font-bold text-slate-400">
 {WEEK_DAYS.map((d) => (
 <div key={d}>{d}</div>
))}
</div>
<div className="mt-1 grid grid-cols-7 gap-1">
 {grid.map((d, i) => {
 const isSel =
 d !== null && selected && selected[0] === view[0] && selected[1] === view[1] && selected[2] === d;
 return (
 <button
 key={i}
 type="button"
 disabled={d === null}
 onClick={() => {
 if (d === null) return;
 onChange(formatJalali(view[0], view[1], d));
 setOpen(false);
 }}
 className={`h-8 rounded-lg text-xs font-semibold transition ${
 d === null
 ? "cursor-default"
 : isSel
 ? "bg-teal-600 text-white"
 : "text-slate-700 hover:bg-teal-50"
 }`}
 >
 {d === null ? "" : toPersianDigits(d)}
 </button>
);
 })}
</div>
<button
 type="button"
 onClick={() => {
 onChange(todayJalali());
 setOpen(false);
 }}
 className="mt-2 w-full rounded-xl bg-slate-100 py-2 text-xs font-bold text-slate-700 hover:bg-slate-200"
>
 □□□□□: {toPersianDigits(todayJalali())}
</button>
</div>
) : null}
</div>
);
}

```

```

@@ src/components/LocationPicker.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useRef, useState } from "react";
import dynamic from "next/dynamic";
import { Badge, Button, Input } from "../ui";
import { toPersianDigits } from "@lib/jalali";

const MapBox = dynamic(() => import("../MapBox"), {
 ssr: false,
 loading: () => (
 <div className="flex h-[240px] items-center justify-center rounded-2xl bg-slate-100 text-sm text-slate-400">
 □□ □□ □□□□□□ □□□□...
 </div>
),
});

```

```

export type LatLng = { lat: number | null; lng: number | null; accuracy: number | null };
type Hit = { label: string; lat: number; lng: number; type: string };

```

```

/**
 * □□□□□□ □□□□□□ □□ □□ □□□:
 * □ □□□□□ □□□□ □□ □□□□ (□□□□□□ + □□□□ □□□□□□)
 * □ □□□□□ □□□□□□ □□□□ GPS (□□□□□□□□□□□□ □□ □□□□□□□□□□)
 * □ □□□□ □□ □□□□□ □□□□□□ □□□ □□□□ / □□□□ □□□□ □□□□□□□□
 */

```

```

export default function LocationPicker({
 value,
 onChange,
 label,
 height = 260,
 /** □□□ □□□□□□□□ □□□□ □□□□□□ (□□□□□ □□□ □□□□□□□□□□ + □□□) */
}) {

```

```

 suggestQuery = "",
 }: {
 value: LatLng;
 onChange: (v: LatLng) => void;
 label?: string;
 height?: number;
 suggestQuery?: string;
 }) {
 const [status, setStatus] = useState("");
 const [busy, setBusy] = useState(false);

 // --- ۰۰۰۰۰۰ ۰۰۰۰ ---
 const [query, setQuery] = useState("");
 const [hits, setHits] = useState<Hit[]>([]);
 const [searching, setSearching] = useState(false);
 const [searchMsg, setSearchMsg] = useState("");
 const [openList, setOpenList] = useState(false);
 const [address, setAddress] = useState("");

 const watchRef = useRef<number | null>(null);
 const timerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
 const autoRef = useRef<ReturnType<typeof setTimeout> | null>(null);
 const bestRef = useRef<number>(Number.POSITIVE_INFINITY);
 const boxRef = useRef<HTMLDivElement>(null);

 /* ----- ۰۰۰۰۰۰ ۰۰۰۰ ----- */

 const runSearch = useCallback(async (q: string, silent = false) => {
 const term = q.trim();
 if (term.length < 2) {
 setHits([]);
 if (!silent) setSearchMsg("۰۰۰۰۰۰ ۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰");
 return;
 }
 setSearching(true);
 if (!silent) setSearchMsg("۰۰ ۰۰۰ ۰۰۰۰۰۰...");
 try {
 const res = await fetch(`/api/geocode?q=${encodeURIComponent(term)}`, { cache: "no-store" });
 const d = await res.json();
 const list: Hit[] = d.results ?? [];
 setHits(list);
 setOpenList(list.length > 0);
 setSearchMsg(
 list.length ? `${toPersianDigits(list.length)} ۰۰۰۰۰ — ۰۰۰ ۰۰ ۰۰۰۰۰۰ ۰۰۰۰` : "۰۰۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰",
);
 } catch {
 setSearchMsg("۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰ — ۰۰۰۰۰۰۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰۰۰۰ ۰۰۰۰");
 } finally {
 setSearching(false);
 }
 }, []);

 /** ۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰ */
 useEffect(() => {
 if (autoRef.current) clearTimeout(autoRef.current);
 const term = query.trim();
 if (term.length < 3) {
 setHits([]);
 return;
 }
 autoRef.current = setTimeout(() => void runSearch(term, true), 800);
 return () => {
 if (autoRef.current) clearTimeout(autoRef.current);
 };
 }, [query, runSearch]);

 /** ۰۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰۰ */
 useEffect(() => {
 if (!openList) return;
 const h = (e: MouseEvent) => {
 if (boxRef.current && !boxRef.current.contains(e.target as Node)) setOpenList(false);
 };
 document.addEventListener("mousedown", h);
 return () => document.removeEventListener("mousedown", h);
 }, [openList]);

 const pickHit = (h: Hit) => {
 onChange({ lat: h.lat, lng: h.lng, accuracy: 0 });
 setQuery(h.label);
 setAddress(h.label);
 setOpenList(false);
 setSearchMsg("۰ ۰۰۰۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰ — ۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰");
 };

 /** ۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰۰ */

```

```

const reverseLookup = useCallback(async (lat: number, lng: number) => {
 try {
 const res = await fetch(`/api/geocode?lat=${lat}&lng=${lng}`, { cache: "no-store" });
 const d = await res.json();
 if (d.results?.[0]?.label) setAddress(d.results[0].label);
 } catch {
 /* ignore */
 }
}, []);

/* ----- GPS ----- */

const stop = useCallback(() => {
 if (watchRef.current !== null) {
 navigator.geolocation.clearWatch(watchRef.current);
 watchRef.current = null;
 }
 if (timerRef.current) {
 clearTimeout(timerRef.current);
 timerRef.current = null;
 }
 setBusy(false);
}, []);

useEffect(() => () => stop(), [stop]);

const locate = () => {
 if (!navigator.geolocation) {
 setStatus("GPS не поддерживается вашим браузером");
 return;
 }
 if (!window.isSecureContext) setStatus("GPS доступен только в HTTPS");
 stop();
 bestRef.current = Number.POSITIVE_INFINITY;
 setBusy(true);
 setStatus("Поиск местоположения... (поиск)");
 watchRef.current = navigator.geolocation.watchPosition(
 (pos) => {
 const acc = pos.coords.accuracy ?? 9999;
 if (acc < bestRef.current) {
 bestRef.current = acc;
 onChange({ lat: pos.coords.latitude, lng: pos.coords.longitude, accuracy: acc });
 }
 setStatus(`${Math.round(acc)} м — точность местоположения`);
 if (acc <= 8) {
 stop();
 setStatus(`${Math.round(acc)} м`);
 void reverseLookup(pos.coords.latitude, pos.coords.longitude);
 }
 },
 (err) => {
 stop();
 setStatus(
 err.code === 1
 ? "Ваш браузер не поддерживает геолокацию"
 : "Геолокация недоступна: " +
 {
 0: "Вы не разрешили доступ к местоположению",
 1: "Ваш браузер не поддерживает геолокацию",
 2: "Геолокация недоступна в этом режиме",
 }[err.code]
);
 },
 { enableHighAccuracy: true, maximumAge: 0, timeout: 20000 },
);
 timerRef.current = setTimeout(() => {
 stop();
 if (bestRef.current === Number.POSITIVE_INFINITY) {
 setStatus("Не удалось определить местоположение");
 } else {
 setStatus(`${Math.round(bestRef.current)} м`);
 if (value.lat && value.lng) void reverseLookup(value.lat, value.lng);
 }
 }, 15000);
};

const acc = value.accuracy ?? null;
const accTone = acc === null ? "amber" : acc <= 20 ? "green" : acc <= 60 ? "amber" : "slate";

return (
 <div className="space-y-2">
 /* ----- GPS ----- */
 <div className="relative" ref={boxRef}>
 <div className="flex gap-1">
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 onFocus={() => hits.length && setOpenList(true)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {

```

```

 e.preventDefault();
 void runSearch(query);
 }
}
placeholder="📍 📍📍📍📍 📍📍📍 📍 📍📍 📍📍... (📍📍📍📍: 📍📍📍📍 📍📍 📍📍📍📍)"
/>
<Button variant="soft" onClick={() => void runSearch(query)} disabled={searching}>
{searching ? "...": "📍 📍📍📍📍"}
</Button>
{suggestQuery.trim() ? (
 <Button
 variant="ghost"
 onClick={() => {
 setQuery(suggestQuery);
 void runSearch(suggestQuery);
 }}
 >
 title="📍📍📍📍 📍 📍📍 📍📍📍 📍📍📍📍"
 </Button>
) : null}
</div>

{searchMsg ? <p className="mt-1 text-[11px] text-slate-500">{searchMsg}</p> : null}

{openList && hits.length > 0 ? (
 <div className="fade-in absolute z-30 mt-1 max-h-56 w-full overflow-y-auto rounded-xl bg-white py-1 shadow-xl
ring-1 ring-slate-200">
 {hits.map((h, i) => (
 <button
 key={` ${h.lat}-${h.lng}-${i}`}
 type="button"
 onMouseDown={(e) => e.preventDefault()}
 onClick={() => pickHit(h)}
 className="block w-full px-3 py-2 text-right text-xs leading-5 text-slate-700 hover:bg-teal-50"
 >
 📍 {h.label}
 </button>
))}
 </div>
) : null}
</div>

/* ----- GPS 📍 📍📍📍 ----- */
<div className="flex flex-wrap items-center gap-2">
 <Button variant="soft" onClick={locate} disabled={busy}>
{busy ? "📍 📍 📍📍 📍📍📍📍📍📍...": "📍 📍📍📍📍 📍📍 📍"}
 </Button>
 {busy ? (
 <Button
 variant="ghost"
 onClick={() => {
 stop();
 setStatus("📍📍📍📍📍📍📍📍 📍📍📍 📍");
 }}
 >
 📍📍📍
 </Button>
) : null}
 {value.lat && value.lng ? (
 <
 <Badge tone={accTone as "green" | "amber" | "slate"}>
{acc ? 📍📍📍 ≈ ${toPersianDigits(Math.round(acc))} 📍📍📍 : "📍📍📍 📍📍📍"}
 </Badge>
 <Button variant="ghost" onClick={() => void reverseLookup(value.lat!, value.lng!)}>
📍 📍📍📍📍📍 📍📍📍 📍📍 📍📍📍
 </Button>
 </>
) : (
 <Badge tone="amber">📍📍📍📍📍 📍📍 📍📍📍</Badge>
)}
</div>

{status ? <p className="text-[11px] text-slate-500">{status}</p> : null}
{address ? (
 <p className="rounded-lg bg-slate-50 px-2 py-1.5 text-[11px] text-slate-600 ring-1 ring-slate-200">
📍 {address}
 </p>
) : null}
<p className="text-[11px] text-slate-400">
📍📍📍📍📍 📍📍 📍📍📍 📍📍📍 📍📍📍📍 📍📍📍 📍📍📍 📍📍 📍📍📍 📍📍 📍📍📍 📍📍📍 📍📍📍.
</p>

<MapBox
 height={height}

```

```

 draggable
 accuracy={acc}
points={value.lat && value.lng ? [{ lat: value.lat, lng: value.lng, label: label || "□□□□□□" }]: []}
 onPick={({p}) => onChange({ lat: p.lat, lng: p.lng, accuracy: 0 })}
 />

 <div className="grid grid-cols-2 gap-2">
 <Input
placeholder="□□□ □□□□□□□□ (lat)"
 value={value.lat ?? ""}
 onChange={(e) => onChange({ ...value, lat: Number(e.target.value) || null })}
 className="text-left text-xs"
 />
 <Input
placeholder="□□□ □□□□□□□□ (lng)"
 value={value.lng ?? ""}
 onChange={(e) => onChange({ ...value, lng: Number(e.target.value) || null })}
 className="text-left text-xs"
 />
 </div>
</div>
);
}

@@ src/components/MapBox.tsx typescript
"use client";

import { useEffect, useRef, useState } from "react";
import "leaflet/dist/leaflet.css";
import type { Map as LeafletMap, Layer } from "leaflet";
import { PROVINCE_EN_FA } from "@lib/geo";

export type MapPoint = { lat: number; lng: number; label?: string; color?: string };

/** □□□□□□□□ □□ □□□□ □□□□ □□ □□ □□□□□□ □□ (□□□□□□/□□□□□□□□) */
export type MapArea = { lat: number; lng: number; zoom?: number; radiusKm?: number; name?: string };

function esc(s: string) {
 return s
 .replace(/&/g, "&")
 .replace(/</g, "<")
 .replace(/>/g, ">")
 .replace(/"/g, """);
}

/**
 * □□□□ □□□□□ □□□□ □□□□□□□□ □□□□□□.
 *
 * □□□□ □□□: □□□□□□□□ □□□□ □□□□□□ □□ □□□□□□ □□ OpenStreetMap □□□□□ □□□□□□□□□□
 * □□ `api/map/tiles` □□□ □□□□□ □□□ □□□□□□ □□□ □□□□□□ □□ □□□□ 403 □□□ □□□.
 */
export default function MapBox({
 points = [],
 path = [],
 onPick,
 height = 260,
 center,
 zoom = 14,
 follow = false,
 accuracy,
 draggable = false,
 area,
 showIranProvinces = false,
 selectedProvince = "",
 onProvinceSelect,
}: {
 points?: MapPoint[];
 path?: { lat: number; lng: number }[];
 onPick?: (p: { lat: number; lng: number }) => void;
 height?: number;
 center?: { lat: number; lng: number } | null;
 zoom?: number;
 follow?: boolean;
 accuracy?: number | null;
 draggable?: boolean;
 area?: MapArea | null;
 /** □□□□□ □□□ □□ □□□□□ □□□□□□□□□□□□ □□ □□□□□□□□ */
 showIranProvinces?: boolean;
 selectedProvince?: string;
 onProvinceSelect?: (province: string) => void;
}) {
 const divRef = useRef<HTMLDivElement>(null);
 const mapRef = useRef<LeafletMap | null>(null);
 const layersRef = useRef<Layer[]>([]);
 const provinceLayerRef = useRef<Layer | null>(null);

```

```

const tileLayerRef = useRef<Layer | null>(null);
const pickRef = useRef<onPick>;
const provinceCbRef = useRef<onProvinceSelect>;
const [tileState, setTileState] = useState<"loading" | "ready" | "fallback">("loading");
const [mapError, setMapError] = useState("");

pickRef.current = onPick;
provinceCbRef.current = onProvinceSelect;

const pointsKey = points.map((p) => `${p.lat.toFixed(6)},${p.lng.toFixed(6)},${p.label ?? ""}`).join(";");
const pathKey = `${path.length}:${path.length ? `${path[path.length - 1].lat.toFixed(5)},${path[path.length - 1].lng.toFixed(5)}` : ""}`;
const centerKey = center ? `${center.lat.toFixed(6)},${center.lng.toFixed(6)}` : "";
const areaKey = area ? `${area.lat.toFixed(4)},${area.lng.toFixed(4)},${area.zoom ?? ""}` : "";

useEffect(() => {
 let disposed = false;
 (async () => {
 try {
 const L = (await import("leaflet")).default;
 if (disposed || !divRef.current || mapRef.current) return;

 const start = center ?? points[0] ?? path[0] ?? area ?? { lat: 32.4279, lng: 53.688 };
 const startZoom = points.length === 0 && path.length === 0 && area?.zoom ? area.zoom : zoom;
 const map = L.map(divRef.current, {
 attributionControl: true,
 zoomControl: true,
 preferCanvas: true,
 minZoom: 4,
 maxZoom: 19,
 }).setView([start.lat, start.lng], startZoom);

 map.attributionControl.setPrefix(false);

 // 403 404 500 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000
 const tiles = L.tileLayer("/api/map/tiles/{z}/{x}/{y}.png", {
 maxZoom: 19,
 minZoom: 4,
 updateWhenIdle: true,
 keepBuffer: 3,
 crossOrigin: false,
 attribution: '© OpenStreetMap',
 });
 let loaded = 0;
 let failed = 0;
 tiles.on("tileload", () => {
 loaded++;
 if (loaded >= 1) setTileState(failed > loaded ? "fallback" : "ready");
 });
 tiles.on("tileerror", () => {
 failed++;
 if (failed >= 3 && loaded === 0) setTileState("fallback");
 });
 tiles.addTo(map);
 tileLayerRef.current = tiles;

 map.on("click", (e: { latlng: { lat: number; lng: number } }) => {
 pickRef.current?.({ lat: e.latlng.lat, lng: e.latlng.lng });
 });
 mapRef.current = map;
 setTimeout(() => map.invalidateSize(), 150);
 drawRecords(L, map);
 if (showIranProvinces) await drawProvinces(L, map, selectedProvince);
 } catch (err) {
 setMapError(err instanceof Error ? err.message : "403 404 500 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 82
```



```

 if (!map || cancelled) return;
 drawRecords(L, map);

 if (area && points.length === 0 && path.length === 0) {
 map.setView([area.lat, area.lng], area.zoom ?? 10);
 return;
 }
 const target = center ?? points[points.length - 1] ?? path[path.length - 1];
 if (follow && target) map.setView([target.lat, target.lng], map.getZoom());
 }());
 return () => {
 cancelled = true;
 };
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [pointsKey, pathKey, centerKey, areaKey, accuracy]);

useEffect(() => {
 if (!showIranProvinces || !mapRef.current) return;
 let cancelled = false;
 (async () => {
 const L = (await import("leaflet")).default;
 if (!cancelled && mapRef.current) await drawProvinces(L, mapRef.current, selectedProvince);
 })();
 return () => {
 cancelled = true;
 };
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [showIranProvinces, selectedProvince]);

// eslint-disable-next-line @typescript-eslint/no-explicit-any
function drawRecords(L: any, map: LeafletMap) {
 for (const layer of layersRef.current) layer.remove();
 layersRef.current = [];

 if (path.length > 1) {
 const line = L.polyline(
 path.map((p) => [p.lat, p.lng]),
 { color: "#0d9488", weight: 4, opacity: 0.85 },
).addTo(map);
 layersRef.current.push(line);
 }

 if (accuracy && points[0]) {
 const circle = L.circle([points[0].lat, points[0].lng], {
 radius: accuracy,
 color: "#0ea5e9",
 weight: 1,
 fillColor: "#38bdf8",
 fillOpacity: 0.12,
 }).addTo(map);
 layersRef.current.push(circle);
 }

 for (const p of points) {
 const icon = L.divIcon({
 className: "sek-map-marker",
 html: `<div style="background:${p.color ?? "#0f766e"};width:18px;height:18px;border-radius:50%;border:3px solid #fff;box-shadow:0 2px 8px rgba(15,23,42,.45)"></div>`,
 iconSize: [18, 18],
 iconAnchor: [9, 9],
 });
 const marker = L.marker([p.lat, p.lng], { icon, draggable }).addTo(map);
 if (p.label) marker.bindTooltip(esc(p.label), { direction: "top", className: "sek-point-tooltip" });
 if (draggable) {
 marker.on("dragend", (e: { target: { getLatLng: () => { lat: number; lng: number } } }) => {
 const ll = e.target.getLatLng();
 pickRef.current?.({ lat: ll.lat, lng: ll.lng });
 });
 }
 layersRef.current.push(marker);
 }

 const all = [...points, ...path];
 if (all.length > 1) {
 map.fitBounds(
 all.map((p) => [p.lat, p.lng] as [number, number]),
 { padding: [28, 28], maxZoom: 17 },
);
 } else if (all.length === 1) {
 map.setView([all[0].lat, all[0].lng], Math.max(map.getZoom(), 16));
 }
}

// eslint-disable-next-line @typescript-eslint/no-explicit-any
async function drawProvinces(L: any, map: LeafletMap, selected: string) {

```

```

provinceLayerRef.current?.remove();
try {
 const res = await fetch("/data/iran-provinces.geojson", { cache: "force-cache" });
 if (!res.ok) throw new Error(`GeoJSON ${res.status}`);
 const geo = await res.json();
 const layer = L.geoJSON(geo, {
 // eslint-disable-next-line @typescript-eslint/no-explicit-any
 style: (feature: any) => {
 const en = String(feature?.properties?.shapeName ?? "");
 const fa = PROVINCE_EN_FA[en] ?? en;
 const active = Boolean(selected && fa === selected);
 return {
 color: active ? "#0f766e" : "#64748b",
 weight: active ? 3 : 1.3,
 opacity: 0.95,
 fillColor: active ? "#14b8a6" : "#e2e8f0",
 fillOpacity: active ? 0.28 : 0.1,
 };
 },
 // eslint-disable-next-line @typescript-eslint/no-explicit-any
 onEachFeature: (feature: any, polygon: any) => {
 const en = String(feature?.properties?.shapeName ?? "");
 const fa = PROVINCE_EN_FA[en] ?? en;
 polygon.bindTooltip(esc(fa), {
 permanent: true,
 direction: "center",
 className: `sek-province-label${selected === fa ? " is-active" : ""}`,
 });
 polygon.on("mouseover", () => polygon.setStyle({ fillOpacity: 0.32, weight: 2.5 }));
 polygon.on("mouseout", () => {
 const active = selected === fa;
 polygon.setStyle({ fillOpacity: active ? 0.28 : 0.1, weight: active ? 3 : 1.3 });
 });
 polygon.on("click", (e: { originalEvent?: { stopPropagation?: () => void } }) => {
 e.originalEvent?.stopPropagation?.();
 provinceCbRef.current?.(fa);
 try {
 map.fitBounds(polygon.getBounds(), { padding: [20, 20], maxZoom: 9 });
 } catch {
 /* ignore */
 }
 });
 },
 }).addTo(map);
 provinceLayerRef.current = layer;
 // □□□□ □□ □□□□□□□□ □□□ □□□□□
 layer.bringToBack?.();
 if (!selected && points.length === 0 && path.length === 0) {
 map.fitBounds(layer.getBounds(), { padding: [8, 8] });
 }
} catch (err) {
 setMapError(□□□ □□□□□□□ □□□□□□□ □□□: ${err instanceof Error ? err.message : "□□□"});
}
}

return (
 <div className="relative w-full overflow-hidden rounded-2xl bg-[#eef2f1] ring-1 ring-slate-200">
 <div ref={divRef} style={{ height }} className="w-full" role="application" aria-label="□□□□" />

 {tileState === "loading" ? (
 <div className="pointer-events-none absolute left-3 top-3 z-[500] rounded-lg bg-white/90 px-2 py-1 text-[10px] font-bold text-slate-500 shadow">
 □ □□ □□□ □□□□□□□□ □□□□...
 </div>
) : null}
 {tileState === "fallback" ? (
 <div className="pointer-events-none absolute left-3 top-3 z-[500] rounded-lg bg-amber-50/95 px-2 py-1 text-[10px] font-bold text-amber-700 shadow ring-1 ring-amber-200">
 □□□□ □□□□ □□□□ □□□□ □ □□□□□ □□□□□□□□ □□□□□□□□
 </div>
) : null}
 {mapError ? (
 <div className="absolute inset-x-3 bottom-3 z-[500] rounded-xl bg-rose-50/95 px-3 py-2 text-center text-[11px] font-bold text-rose-700 shadow ring-1 ring-rose-200">
 {mapError}
 </div>
) : null}
 </div>
);
}

@@ src/components/NavButton.tsx typescript
"use client";

import { useEffect, useState } from "react";

```

```

import { createPortal } from "react-dom";

type App = {
 key: string;
 label: string;
 icon: string;
 /** 000000 00 00000000 0000 0000 */
 scheme?: (lat: number, lng: number, name: string) => string;
 /** 00000 00 0000000000 00000000 */
 web: (lat: number, lng: number, name: string) => string;
 platforms?: ("android" | "ios" | "desktop")[];
};

const APPS: App[] = [
 {
 key: "neshan",
 label: "00000",
 icon: "0",
 scheme: (la, ln) => `neshan://maps?lat=${la}&lng=${ln}`,
 web: (la, ln) => `https://neshan.org/maps/routing/car/${la},${ln}#c${la}-${ln}-16z-0p`,
 platforms: ["android", "ios", "desktop"],
 },
 {
 key: "balad",
 label: "0000",
 icon: "00",
 scheme: (la, ln) => `balad://maps?lat=${la}&lng=${ln}`,
 web: (la, ln) => `https://balad.ir/navigate?destination=${la},${ln}`,
 platforms: ["android", "ios", "desktop"],
 },
 {
 key: "google",
 label: "00000 00",
 icon: "0",
 scheme: (la, ln) => `google.navigation:q=${la},${ln}&mode=d`,
 web: (la, ln) => `https://www.google.com/maps/dir/?api=1&destination=${la},${ln}&travelmode=driving`,
 platforms: ["android", "ios", "desktop"],
 },
 {
 key: "waze",
 label: "0000 (Waze)",
 icon: "0",
 scheme: (la, ln) => `waze://?ll=${la},${ln}&navigate=yes`,
 web: (la, ln) => `https://waze.com/ul?ll=${la},${ln}&navigate=yes`,
 platforms: ["android", "ios", "desktop"],
 },
 {
 key: "apple",
 label: "00000 0000",
 icon: "0",
 scheme: (la, ln, n) => `maps://?daddr=${la},${ln}&q=${n}`,
 web: (la, ln, n) => `https://maps.apple.com/?daddr=${la},${ln}&q=${n}`,
 platforms: ["ios", "desktop"],
 },
 {
 key: "osm",
 label: "OpenStreetMap",
 icon: "0",
 web: (la, ln) => `https://www.openstreetmap.org/directions?to=${la},${ln}`,
 platforms: ["android", "ios", "desktop"],
 },
 {
 key: "yandex",
 label: "00000000",
 icon: "0",
 scheme: (la, ln) => `yandexnavi://build_route_on_map?lat_to=${la}&lon_to=${ln}`,
 web: (la, ln) => `https://yandex.com/maps/?rtext=~${la},${ln}&rtt=auto`,
 platforms: ["android", "ios", "desktop"],
 },
 {
 key: "geo",
 label: "000000000 00000000 00000000",
 icon: "0",
 scheme: (la, ln, n) => `geo:${la},${ln}?q=${la},${ln}(${n})`,
 web: (la, ln) => `https://www.google.com/maps?q=${la},${ln}`,
 platforms: ["android"],
 },
];

function platform(): "android" | "ios" | "desktop" {
 const ua = navigator.userAgent;
 if (/Android/i.test(ua)) return "android";
 if (/iPhone|iPad|iPod/i.test(ua)) return "ios";
 return "desktop";
}

```

```

/** 0000 00000000 — 000 00000000000 00000 000 000000 00 00000 000000 */
export default function NavButton({
 lat,
 lng,
 label = "0000",
 compact = true,
}): {
 lat?: number | null;
 lng?: number | null;
 label?: string;
 compact?: boolean;
}() {
 const [open, setOpen] = useState(false);
 const [plat, setPlat] = useState<"android" | "ios" | "desktop">("desktop");
 const [canShare, setCanShare] = useState(false);
 const [pos, setPos] = useState<{ top: number; left: number } | null>(null);
 const [mobileMenu, setMobileMenu] = useState(false);
 const [mounted, setMounted] = useState(false);

 useEffect(() => {
 setMounted(true);
 setPlat(platform());
 setCanShare(typeof navigator !== "undefined" && !!navigator.share);
 const update = () => setMobileMenu(window.innerWidth < 640);
 update();
 window.addEventListener("resize", update);
 return () => window.removeEventListener("resize", update);
 }, []);

 if (!lat || !lng) return —;

 const name = encodeURIComponent(label || "0000");
 const list = APPS.filter((a) => !a.platforms || a.platforms.includes(plat));

 const go = (a: App) => {
 setOpen(false);
 const webUrl = a.web(lat, lng, name);
 // 000 000000 00000 0000 00 000 000000
 if (plat === "desktop" || !a.scheme) {
 const w = window.open(webUrl, "_blank", "noopener,noreferrer");
 if (!w) window.location.href = webUrl; // 000 000000 00000 00
 return;
 }
 // 000 000000 00000 00 0000000 000000 0000000 00 0000 0000 0000 00 000 000000
 const started = Date.now();
 const fb = setTimeout(() => {
 if (!document.hidden && Date.now() - started < 2500) window.open(webUrl, "_blank", "noopener");
 }, 1100);
 window.addEventListener("pagehide", () => clearTimeout(fb), { once: true });
 window.location.href = a.scheme(lat, lng, name);
 };

 const shareLocation = async () => {
 setOpen(false);
 const text = `${label}\\nhttps://www.google.com/maps?q=${lat},${lng}`;
 if (navigator.share) {
 try {
 await navigator.share({ title: label, text });
 return;
 } catch {
 /* 000 00 */
 }
 }
 await navigator.clipboard?.writeText(text).catch(() => undefined);
 };

 const copyCoords = async () => {
 setOpen(false);
 await navigator.clipboard?.writeText(`${lat},${lng}`).catch(() => undefined);
 };

 const menu = open ? (
 <div
 className="fixed inset-0 z-[9998] bg-slate-900/45 backdrop-blur-[1px]"
 onClick={(e) => {
 e.preventDefault();
 e.stopPropagation();
 setOpen(false);
 }}
 >
 <div
 className={
 mobileMenu
 ? "fade-in fixed inset-x-3 bottom-3 max-h-[78vh] overflow-y-auto rounded-3xl bg-white p-3 shadow-2xl ring-1

```

```

ring-white/80"
 : "fade-in fixed w-60 max-h-[75vh] overflow-y-auto rounded-2xl bg-white p-2 shadow-2xl ring-1 ring-slate-200
"
 }
 style={mobileMenu ? undefined : { top: pos?.top ?? 80, left: pos?.left ?? 16 }}
 onClick={(e) => {
 e.preventDefault();
 e.stopPropagation();
 }}
 >
 <div className="mb-2 flex items-center justify-between gap-2 px-1">
 <div>
 <div className="text-xs font-black text-slate-800">□ □□□□□□ □□□□□□□□</div>
 <div className="mt-0.5 max-w-[190px] truncate text-[10px] text-slate-400">{label}</div>
 </div>
 <button
 type="button"
 onClick={() => setOpen(false)}
 className="flex size-8 items-center justify-center rounded-full bg-slate-100 text-sm font-bold text-slate-60
0"
aria-label="□□□□"
 >
 □
 </button>
 </div>

 <div className={mobileMenu ? "grid grid-cols-2 gap-2" : "space-y-1"}>
 {list.map((a) => (
 <button
 key={a.key}
 type="button"
 onClick={() => go(a)}
 className={`flex items-center gap-2 rounded-xl bg-slate-50 px-3 text-right font-bold text-slate-700 ring-1
ring-slate-200 transition hover:bg-teal-50 hover:text-teal-800 ${
 mobileMenu ? "min-h-12 py-3 text-xs" : "w-full py-2 text-[11px]"
 }`}
 >
 {a.icon}
 {a.label}
 </button>
)})}
 </div>

 <div className="my-2 h-px bg-slate-100" />
 <div className={mobileMenu ? "grid grid-cols-2 gap-2" : "space-y-1"}>
 {canShare ? (
 <button
 type="button"
 onClick={shareLocation}
 className="rounded-xl bg-sky-50 px-3 py-2.5 text-right text-[11px] font-bold text-sky-700 ring-1 ring-sky-
200"
 >
 □ □□□□□□□□□□□□□
 </button>
) : null}
 <button
 type="button"
 onClick={copyCoords}
 className="rounded-xl bg-slate-100 px-3 py-2.5 text-right text-[11px] font-bold text-slate-700 ring-1 ring-s
late-200"
 >
 □ □□□ □□□□□□
 </button>
 </div>
 </div>
) : null;

return (
 <>

 <button
 type="button"
 onClick={(e) => {
 e.preventDefault();
 e.stopPropagation();
 const r = (e.currentTarget as HTMLElement).getBoundingClientRect();
 const menuWidth = 240;
 const left = Math.min(Math.max(8, r.left), Math.max(8, window.innerWidth - menuWidth - 8));
 const estimatedHeight = 390;
 const top =
 r.bottom + estimatedHeight < window.innerHeight
 ? r.bottom + 6
 : Math.max(8, r.top - estimatedHeight - 6);
 setPos({ top, left });
 }}
 >

```

```

 setOpen(true);
 }}
 title="🔔🔔🔔🔔🔔🔔🔔🔔"
 className={`rounded-xl bg-gradient-to-l from-teal-600 to-emerald-600 font-bold text-white shadow-sm transition
hover:from-teal-700 hover:to-emerald-700 active:scale-95 ${
 compact ? "px-2.5 py-1.5 text-[11px]" : "px-4 py-2.5 text-xs"
 }`}
 >
 🔔🔔🔔🔔🔔🔔🔔🔔
 </button>

{mounted && menu ? createPortal(menu, document.body) : null}
</>
);
}

```

```

@@ src/components/NotificationBell.tsx typescript
"use client";

```

```

import Link from "next/link";
import { useCallback, useEffect, useRef, useState } from "react";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { playNotificationSound, vibrate } from "@lib/sound";

```

```

type N = {
 id: number;
 title: string;
 body: string;
 fromName: string;
 kind: string;
 link: string;
 readAt: string | null;
 createdAt: string;
};

```

```

export default function NotificationBell({ basePath }: { basePath: string }) {
 const [rows, setRows] = useState<N[]>([]);
 const [unread, setUnread] = useState(0);
 const [open, setOpen] = useState(false);
 const [toast, setToast] = useState<N | null>(null);
 const [muted, setMuted] = useState(false);
 const lastId = useRef<number>(0);
 const initialized = useRef(false);

 const load = useCallback(async () => {
 // 🔔🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔🔔
 const res = await fetch("/api/notifications?unread=1", { cache: "no-store" }).catch(() => null);
 if (!res?.ok) return;
 const d = await res.json();
 const list: N[] = d.rows ?? [];
 setRows(list);
 setUnread(list.length);
 const top = list[0];
 if (top && initialized.current && top.id > lastId.current && !top.readAt) {
 setToast(top);
 setTimeout(() => setToast(null), 10000);
 if (localStorage.getItem("sek_mute") !== "1") {
 playNotificationSound();
 vibrate();
 }
 showSystemNotification(top);
 }
 if (top) lastId.current = Math.max(lastId.current, top.id);
 initialized.current = true;
 }, []);

 useEffect(() => {
 setMuted(localStorage.getItem("sek_mute") === "1");
 load();
 const t = setInterval(load, 20000);

 // 🔔🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔 (🔔🔔🔔🔔🔔🔔🔔🔔)
 const ask = () => {
 if (typeof Notification !== "undefined" && Notification.permission === "default") {
 Notification.requestPermission().catch(() => undefined);
 }
 window.removeEventListener("click", ask);
 window.removeEventListener("touchstart", ask);
 };
 window.addEventListener("click", ask, { once: true });
 window.addEventListener("touchstart", ask, { once: true });

 // 🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔 🔔🔔🔔🔔🔔🔔🔔
 const onMsg = (e: MessageEvent) => {
 if (e.data?.type === "notification-click") {

```

```

 if (e.data.id) markOne(Number(e.data.id));
 load();
 }
};
navigator.serviceWorker?.addEventListener("message", onMsg);

return () => {
 clearInterval(t);
 window.removeEventListener("click", ask);
 window.removeEventListener("touchstart", ask);
 navigator.serviceWorker?.removeEventListener("message", onMsg);
};
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [load]);

/**
 * 通知管理
 * 通知管理
 * 通知管理
 * 通知管理
 * 通知管理
 */
const showSystemNotification = async (n: N) => {
 if (typeof Notification === "undefined" || Notification.permission !== "granted") return;
 const payload = {
 body: n.body || n.title,
 icon: "/icons/icon-192.png",
 badge: "/icons/icon-192.png",
 tag: `sek-${n.id}`,
 dir: "rtl" as const,
 lang: "fa",
 requireInteraction: true,
 data: { id: n.id, link: n.link || `${basePath}/notifications` },
 vibrate: [200, 100, 200],
 };
 try {
 // 注册通知
 const reg = await navigator.serviceWorker?.getRegistration();
 if (reg) {
 await reg.showNotification(`${n.title}`, payload as NotificationOptions);
 return;
 }
 const note = new Notification(`${n.title}`, payload as NotificationOptions);
 note.onclick = () => {
 window.focus();
 markOne(n.id);
 window.location.href = payload.data.link;
 note.close();
 };
 } catch {
 /* ignore */
 }
};

const markOne = async (id: number) => {
 await fetch("/api/notifications", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ ids: [id] }),
 });
 load();
};

const toggleMute = () => {
 const next = !muted;
 setMuted(next);
 localStorage.setItem("sek_mute", next ? "1" : "0");
 if (!next) playNotificationSound();
};

const askPermission = async () => {
 if (typeof Notification !== "undefined" && Notification.permission === "default") {
 await Notification.requestPermission().catch(() => undefined);
 }
};

const markAll = async () => {
 await fetch("/api/notifications", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ all: true }),
 });
 load();
};

return (
 <>
 <div className="relative">
 <button
 onClick={() => {
 setOpen((v) => !v);
 }}
 />
 </div>
 </>
);

```

```

 askPermission();
 }}
 className="relative rounded-lg bg-white/15 px-2 py-1.5 text-sm hover:bg-white/25"
title="□□□□□□□□"
>
□
{unread > 0 ? (

 {toPersianDigits(unread > 99 ? "99+" : unread)}

) : null}
</button>

{open ? (
 <>
 <div className="fixed inset-0 z-30" onClick={() => setOpen(false)} />
 <div className="fade-in absolute left-0 z-40 mt-2 max-h-80 w-72 overflow-y-auto rounded-2xl bg-white p-2 tex
t-slate-800 shadow-2xl ring-1 ring-slate-200">
 <div className="mb-1 flex items-center justify-between px-2">
 □□□□□□□□
 <div className="flex items-center gap-2">
 <button onClick={toggleMute} className="text-[11px] font-bold text-slate-500" title="□□□□ □□□□">
 {muted ? "□ □□□ □□□□" : "□ □□□ □□□□"}
 </button>
 <button onClick={markAll} className="text-[11px] font-bold text-teal-700">
 □□□□□□ □□□
 </button>
 </div>
 </div>
 {rows.length === 0 ? (
 <p className="px-2 py-6 text-center text-xs text-slate-400">
 □□□□□ □□□□□□□□□□□□□□ □□□□□□ □
 </p>
) : (
 rows.slice(0, 12).map((n) => (
 <div
 key={n.id}
 onClick={() => !n.readAt && markOne(n.id)}
 className={`mb-1 cursor-pointer rounded-xl px-2.5 py-2 text-xs ${n.readAt ? "bg-slate-50" : "bg-teal
-50 ring-1 ring-teal-200"}`>
 <div className="font-bold text-slate-800">
 {n.title}
 </div>
 {!n.readAt ? (□□□□ □□□□□□ □□□□ □□□□) : null}
 </div>
 {n.body ? <div className="mt-0.5 text-[11px] text-slate-600">{n.body}</div> : null}
 <div className="mt-1 text-[10px] text-slate-400">
 {n.fromName ? `${n.fromName} — ` : ""}
 {tehranDateTime(n.createdAt)}
 </div>
 </div>
))
)}
 <Link
 href={`/${basePath}/notifications`}
 onClick={() => setOpen(false)}
 className="block rounded-xl bg-slate-100 py-2 text-center text-xs font-bold text-slate-700"
 >
 □□□□□□ □□□ □□□□□□□□
 </Link>
 </div>
</>
) : null}
</div>

{toast ? (
 <div className="fade-in fixed bottom-4 left-4 right-4 z-50 mx-auto max-w-sm rounded-2xl bg-slate-900/95 p-3 text
-white shadow-2xl sm:right-auto">
 <div className="text-xs font-black text-amber-300">□ □□□□□ □□□□</div>
 <div className="mt-1 text-sm font-bold">{toast.title}</div>
 {toast.body ? <div className="text-xs text-slate-200">{toast.body}</div> : null}
 <div className="mt-2 flex gap-3">
 <button
 onClick={() => {
 markOne(toast.id);
 setToast(null);
 }}
 className="text-[11px] font-bold text-emerald-300"
 >
 □ □□□□□□□
 </button>
 <button onClick={() => setToast(null)} className="text-[11px] font-bold text-teal-300">
 □□□□
 </button>
 </div>
 </div>
) : null}

```



```

 </div>
 </div>
) : null}
 </>
);
}

@@ src/components/RecordScreen.tsx typescript
"use client";

import { useCallback, useEffect, useMemo, useState } from "react";
import dynamicImport from "next/dynamic";
import JalaliDateInput from "../JalaliDateInput";
import Combobox from "../Combobox";
import LocationPicker, { type LatLng } from "../LocationPicker";
import FileUploader, { FileList, type Att } from "../FileUploader";
import ShareBox from "../ShareBox";
import NavButton from "../NavButton";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "../ui";
import {
 DEFAULT_PRODUCTS,
 DEFAULT_COLUMNS,
 DEFAULT_FORM_FIELDS,
 bonusKeyOf,
 type ProductConfig,
 type ColumnConfig,
 type FormFieldConfig,
} from "@lib/defaults";
import { isValidJalali, tehranDateTime, toPersianDigits, todayJalali } from "@lib/jalali";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";

const MapBox = dynamicImport(() => import("../MapBox"), { ssr: false });

export type RecordType = "pharmacies" | "doctors" | "orders";

export type Row = {
 id: number;
 repName: string;
 dateShamsi: string;
 province?: string;
 city?: string;
 region?: string;
 name?: string;
 landline?: string;
 pharmacyName?: string;
 managerName?: string;
 managerPhone?: string;
 specialty?: string;
 phone?: string;
 secretaryName?: string;
 secretaryPhone?: string;
 otherAddresses?: string;
 address?: string;
 isPercent?: boolean;
 percentValue?: string;
 lat?: number | null;
 lng?: number | null;
 accuracy?: number | null;
 locationLabel?: string;
 items?: Record<string, number>;
 distributor?: string;
 visitor?: string;
 notes?: string;
 sent?: boolean;
 sendStatus?: string;
 createdAt?: string;
};

type Form = Record<string, string> & { items?: never };

type DupRow = {
 id: number;
 name: string;
 specialty?: string;
 phone?: string;
 city?: string;
 province?: string;
 region?: string;
 landline?: string;
 managerName?: string;
 managerPhone?: string;
 address?: string;
 lat?: number | null;

```

```

 lng?: number | null;
 accuracy?: number | null;
 isPercent?: boolean;
 percentValue?: string;
 repName: string;
 dateShamsi: string;
 mine: boolean;
 };

 type OrderHistory = {
 orderCount: number;
 lastDate: string;
 lastRep: string;
 totalUnits: number;
 totalBonus: number;
 items: { key: string; label: string; qty: number; bonus: number }[];
 recent: { id: number; dateShamsi: string; repName: string; units: number }[];
 };

 type TargetRow = {
 productKey: string;
 productLabel: string;
 quantity: number;
 sold: number;
 remaining: number;
 percent: number;
 };

 const emptyForm = (): Record<string, string> => ({
 dateShamsi: todayJalali(),
 province: "",
 city: "",
 region: "",
 name: "",
 landline: "",
 pharmacyName: "",
 managerName: "",
 managerPhone: "",
 address: "",
 specialty: "",
 phone: "",
 secretaryName: "",
 secretaryPhone: "",
 otherAddresses: "",
 distributor: "",
 visitor: "",
 notes: "",
 });

 const TITLES: Record<RecordType, { title: string; icon: string; nameLabel: string }> = {
 pharmacies: { title: "فهرست داروخانه ها", icon: "🏠", nameLabel: "نام داروخانه" },
 doctors: { title: "فهرست پزشکان", icon: "👨", nameLabel: "نام پزشک" },
 orders: { title: "فهرست سفارشات", icon: "📦", nameLabel: "نام سفارش" },
 };

 const PAGE = 25;

 export default function RecordScreen({ type, isAdmin = false }: { type: RecordType; isAdmin?: boolean }) {
 const meta = TITLES[type];
 const [tab, setTab] = useState<"form" | "list">(isAdmin ? "list" : "form");
 const [form, setForm] = useState<Record<string, string>>(emptyForm);
 const [items, setItems] = useState<Record<string, number>>({});
 const [loc, setLoc] = useState<LatLng>({ lat: null, lng: null, accuracy: null });
 const [rows, setRows] = useState<Row[]>([]);
 const [opts, setOpts] = useState<Record<string, string[]>>({});
 const [optTree, setOptTree] = useState<Record<string, Record<string, string[]>>>({});
 const [msg, setMsg] = useState<{ kind: "error" | "success" | "info"; text: string } | null>(null);
 const [busy, setBusy] = useState(false);
 const [selected, setSelected] = useState<number[]>([]);
 const [detail, setDetail] = useState<Row | null>(null);
 const [search, setSearch] = useState("");
 const [period, setPeriod] = useState("");
 const [page, setPage] = useState(1);
 const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
 const [isPercent, setIsPercent] = useState(false);
 const [percentValue, setPercentValue] = useState("");
 const [fileIds, setFileIds] = useState<number[]>([]);
 const [detailFiles, setDetailFiles] = useState<Att[]>([]);
 const [dups, setDups] = useState<DupRow[]>([]);
 const [history, setHistory] = useState<OrderHistory | null>(null);
 const [suggestion, setSuggestion] = useState<Record<string, unknown> | null>(null);
 const [autofilled, setAutofilled] = useState(false);
 const [targets, setTargets] = useState<TargetRow[]>([]);
 const [editRow, setEditRow] = useState<Row | null>(null);
 const [editDraft, setEditDraft] = useState<Record<string, string>>({});
 }

```

```

const [editItems, setEditItems] = useState<Record<string, number>>({});
const [columns, setColumns] = useState<ColumnConfig[]>(DEFAULT_COLUMNS[type]);
const [formFields, setFormFields] = useState<FormFieldConfig[]>(DEFAULT_FORM_FIELDS[type]);

const confirm = useConfirm();
const set = (k: string, v: string) => setForm((f) => ({ ...f, [k]: v }));

const loadRows = useCallback(async () => {
 const res = await fetch(`/api/records/${type}`, { cache: "no-store" });
 if (res.ok) setRows((await res.json()).rows ?? []);
}, [type]);

const loadOptions = useCallback(async () => {
 const res = await fetch("/api/options", { cache: "no-store" });
 if (!res.ok) return;
 const data = await res.json();
 const rows = (data.rows ?? []) as { category: string; value: string; parent?: string }[];
 const grouped: Record<string, string[]> = {};
 const tree: Record<string, Record<string, string[]>> = {};
 for (const o of rows) {
 (grouped[o.category] ??= []).push(o.value);
 const par = o.parent ?? "";
 ((tree[o.category] ??= {})[par] ??= []).push(o.value);
 }
 setOpts(grouped);
 setOptTree(tree);
}, []);

const loadSettings = useCallback(async () => {
 const res = await fetch("/api/settings", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 const v = d.values ?? {};
 setProducts((v.products as ProductConfig[])?.filter((p) => p.enabled) ?? DEFAULT_PRODUCTS);

 /**
 * 00000 00000000 000000000 00 0000000000.
 * 000 0000 0000 00000000 00000000 000000 00 00000000 000000 00000000
 * 000 00000 0000 00000 0000 (000 «000000») 00 0000000 00000 00000
 * 00 0000 0000000000 0000 00000 0 0000 000000 0000 0000000.
 */
 const stored = v[`columns.${type}`] as ColumnConfig[] | undefined;
 const defaults = DEFAULT_COLUMNS[type];
 if (Array.isArray(stored) && stored.length) {
 const have = new Set(stored.map((c) => c.key));
 // 000 0000 00000 0000000 00 00000000 00000 0000000 00000 00000000
 // 000 0000 0000 00000000 0000 0000 000000 000000 0000000.
 const mandatory = defaults.filter((c) => c.key === "actions" && !have.has(c.key));
 setColumns([...stored, ...mandatory]);
 } else {
 setColumns(defaults);
 }

 const storedFields = v[`fields.${type}`] as FormFieldConfig[] | undefined;
 if (Array.isArray(storedFields) && storedFields.length) {
 const have = new Set(storedFields.map((f) => f.key));
 // 000000 000 00000 00 00 000 000 0 00 00000000 00000 0000000 00000000 0000000.
 const mandatory = DEFAULT_FORM_FIELDS[type].filter((f) => f.key === "location" && !have.has(f.key));
 setFormFields([...storedFields, ...mandatory]);
 } else {
 setFormFields(DEFAULT_FORM_FIELDS[type]);
 }
}, [type]);

useEffect(() => {
 loadOptions();
 loadSettings();
}, [loadOptions, loadSettings]);

// 0000000000 00000000 0000 0000 0000 00 0000 0000
useLive(loadRows, 15000, tab === "list");

const recordName = type === "orders" ? form.pharmacyName : form.name;
const showField = (key: string) => formFields.some((f) => f.key === key && f.visible);

const submit = async () => {
 if (!isValidJalali(form.dateShamsi)) {
 setMsg({ kind: "error", text: "000000 0000 000000 0000 (00000: 0000/00/00)" });
 return;
 }
 if (!recordName.trim()) {
 setMsg({ kind: "error", text: `${meta.nameLabel} 000000 000` });
 return;
 }
 // 000000 0000000 0000 000 00 000

```

```
if (type !== "orders" && dupes.length > 0) {
 const ok = await confirm({
 title: "□□ □□□□ □□□□□□",
 message:
 `«${dupes[0].name}» □□□□ ${dupes[0].mine ? "□□□□ □□□□□□" : "□□□□ ${dupes[0].repName}`} □□ □□□□` +
 `${toPersianDigits(dupes[0].dateShamsi)} □□ □□ □□.\n□□□ □□□□□□ □□□□□□□□ □□□□ □□ □□□□`,
 confirmText: "□□□□ □□ □□",
 cancelText: "□□□□□□",
 danger: true,
 });
 if (!ok) return;
}
if (
 !(await confirm({
 title: "□□□ □□□□□□□",
 message: `«${recordName.trim()}» □□ □□□□ ${toPersianDigits(form.dateShamsi)} □□ □□□□`,
 confirmText: "□□ □□",
 }))
)
 return;
setBusy(true);
setMsg(null);
const res = await fetch(`/api/records/${type}`, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({
 ...form,
 items,
 isPercent,
 percentValue,
 fileIds,
 lat: loc.lat,
 lng: loc.lng,
 accuracy: loc.accuracy,
 locationLabel: recordName.trim(),
 }),
});
setBusy(false);
if (res.ok) {
 setForm(emptyForm());
 setItems({});
 setIsPercent(false);
 setPercentValue("");
 setFileIds([]);
 setLoc({ lat: null, lng: null, accuracy: null });
 setMsg({ kind: "success", text: "□ □□□ □□. □□□□ □□□□□ □□ □□□□ □□ □□ «□□□□ □□□□□□□□□□□□» □□□□□." });
 setDupes([]);
 setHistory(null);
 loadRows();
 loadTargets();
 setTab("list");
} else {
 const d = await res.json().catch(() => ({}));
 setMsg({ kind: "error", text: d.error ?? "□□□ □□ □□ □□□□□□□" });
}
};

const sendSelected = async () => {
 const ids = selected.length ? selected : rows.filter((r) => !r.sent).map((r) => r.id);
 if (!ids.length) {
 setMsg({ kind: "info", text: "□□□□□ □□□□□□□□□□□□□□ □□□□ □□□□□" });
 return;
 }
 if (
 !(await confirm({
 title: "□□□□□ □□□□□□□ □□ □□□□",
 message: `${toPersianDigits(ids.length)} □□□□□ □□□□□ □□□□${type === "orders" ? " □□□□□□□□ □□ □□□□□□□□□□□□ □□□□ □□ □□□□ □□□□□□□□." : ""}`,
 confirmText: "□□□□□",
 }))
)
 return;
 setBusy(true);
 const res = await fetch(`/api/records/${type}/send`, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ ids }),
 });
 setBusy(false);
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setSelected([]);
 setMsg({
 kind: "success",
 text:

```

```

 type === "orders"
 ? `□ □□□□□ □□. □□□□□ □□□□□□□□□□: ${d.status ?? "—"}`
 : `□ ${toPersianDigits(ids.length)} □□□□□ □□□□ □□□□ □□□□ □□`,
 });
 loadRows();
} else setMsg({ kind: "error", text: d.error ?? "□□□ □□ □□□□□ □□□□□□□" });
};

const periods = useMemo(() => {
 const s = new Set(rows.map((r) => r.dateShamsi.slice(0, 7)).filter(Boolean));
 return [...s].sort().reverse();
}, [rows]);

const filtered = useMemo(() => {
 const q = search.trim().toLowerCase();
 return rows.filter((r) => {
 if (period && !r.dateShamsi.startsWith(period)) return false;
 if (!q) return true;
 const hay = `${r.repName} ${r.name ?? ""} ${r.pharmacyName ?? ""} ${r.managerName ?? ""} ${r.specialty ?? ""} ${r.city ?? ""} ${r.province ?? ""} ${r.managerPhone ?? ""} ${r.phone ?? ""}`;
 return hay.toLowerCase().includes(q);
 });
}, [rows, search, period]);

const pageRows = filtered.slice(0, page * PAGE);

useEffect(() => {
 if (detail && type !== "orders") {
 const ot = type === "doctors" ? "doctor" : "pharmacy";
 fetch(`/api/attachments?ownerType=${ot}&ownerId=${detail.id}`, { cache: "no-store" })
 .then((r) => (r.ok ? r.json() : null))
 .then((d) => setDetailFiles(d?.rows ?? []))
 .catch(() => setDetailFiles([]));
 } else setDetailFiles([]);
}, [detail, type]);

const orderText = (r: Row) => {
 const lines = [
 "□ □□□□□ □□□□□□□□",
 `□□□□□ □□□□□: ${toPersianDigits(r.dateShamsi)}`,
 `□□□□□□□ □□□□: ${r.repName}`,
 `□□□ □□□□□□□□: ${r.pharmacyName ?? ""}`,
 `□□□□□ □□□□□: ${r.managerName ?? ""}`,
 `□□□□□ □□□□□: ${toPersianDigits(r.managerPhone ?? "")}`,
 `□□□□: ${r.address ?? ""}`,
];
 // □□ □□□□□ □□□□□ □□□□□□ □□ □□□□□□□□ (□□□ □□□□□□□□)
 lines.push("— □□□□□ □□□□□ —");
 for (const p of products) {
 const q = Number(r.items?.[p.key] ?? 0);
 const b = Number(r.items?.[bonusKeyOf(p.key)] ?? 0);
 if (q || b) lines.push(`${p.label}: ${toPersianDigits(q)} | □□□□□: ${toPersianDigits(b)}`);
 }
 lines.push(`□□□ □□□: ${r.distributor ?? ""}`);
 lines.push(`□□□ □□□□□□□□: ${r.visitor ?? ""}`);
 if (r.notes) lines.push(`□□□□□□□□: ${r.notes}`);
 return lines.join("\n");
};

/* ---- □□□□□□□□ □□□□□□□□ □□□□ □□□□□ □□□□ □□ □□□□ ---- */
const loadTargets = useCallback(async () => {
 if (type !== "orders") return;
 const res = await fetch("/api/targets", { cache: "no-store" });
 if (res.ok) {
 const d = await res.json();
 setTargets(d.progress ?? []);
 }
}, [type]);

useEffect(() => {
 loadTargets();
}, [loadTargets]);

/* ---- □□□□□ □□□□□□□ + □□□□□□□ □□□□□□□ □□ □□□□□ ---- */
const lookupName = type === "orders" ? form.pharmacyName : form.name;
const lookupPhone = type === "doctors" ? form.phone : form.managerPhone;

useEffect(() => {
 if (tab !== "form") return;
 const n = (lookupName ?? "").trim();
 if (n.length < 2) {
 setDuples([]);
 setHistory(null);
 setSuggestion(null);
 return;
 }

```

```

 }
 const t = setTimeout(async () => {
 const q = new URLSearchParams({ type, name: n, phone: lookupPhone ?? "" });
 const res = await fetch(`/api/records/lookup?${q}`, { cache: "no-store" }).catch(() => null);
 if (!res?.ok) return;
 const d = await res.json();
 setDuples(d.duplicates ?? []);
 setHistory(d.history ?? null);
 setSuggestion(d.suggestion ?? null);
 }, 600);
 return () => clearTimeout(t);
 }, [lookupName, lookupPhone, type, tab]);

 /** 〇〇〇〇〇〇 〇〇〇〇〇〇 〇〇〇〇〇〇 〇〇 〇〇〇〇〇 〇〇〇〇〇〇〇〇〇 */
 const applySuggestion = (src?: Record<string, unknown> | DupRow) => {
 const s = (src ?? suggestion) as Record<string, unknown> | null;
 if (!s) return;
 setForm((f) => ({
 ...f,
 pharmacyName: String(s.pharmacyName ?? s.name ?? f.pharmacyName),
 name: type === "orders" ? f.name : String(s.name ?? f.name),
 managerName: String(s.managerName ?? f.managerName),
 managerPhone: String(s.managerPhone ?? f.managerPhone),
 address: String(s.address ?? f.address),
 province: String(s.province ?? f.province),
 city: String(s.city ?? f.city),
 region: String(s.region ?? f.region),
 landline: String(s.landline ?? f.landline),
 distributor: String(s.distributor ?? f.distributor),
 visitor: String(s.visitor ?? f.visitor),
 }));
 const la = Number(s.lat);
 const ln = Number(s.lng);
 if (Number.isFinite(la) && Number.isFinite(ln) && la && ln) {
 setLoc({ lat: la, lng: ln, accuracy: Number(s.accuracy) || null });
 }
 setAutofilled(true);
 setTimeout(() => setAutofilled(false), 4000);
 };

 /* ----- 〇〇〇〇〇〇 〇 〇〇〇 〇〇〇〇〇 ----- */
 const openEdit = (r: Row) => {
 setEditRow(r);
 setEditDraft({
 dateShamsi: r.dateShamsi ?? "",
 name: r.name ?? "",
 pharmacyName: r.pharmacyName ?? "",
 province: r.province ?? "",
 city: r.city ?? "",
 region: r.region ?? "",
 landline: r.landline ?? "",
 managerName: r.managerName ?? "",
 managerPhone: r.managerPhone ?? "",
 specialty: r.specialty ?? "",
 phone: r.phone ?? "",
 secretaryName: r.secretaryName ?? "",
 secretaryPhone: r.secretaryPhone ?? "",
 address: r.address ?? "",
 otherAddresses: r.otherAddresses ?? "",
 distributor: r.distributor ?? "",
 visitor: r.visitor ?? "",
 notes: r.notes ?? "",
 percentValue: r.percentValue ?? "",
 });
 setEditItems({ ...(r.items ?? {}) });
 };

 const saveEdit = async () => {
 if (!editRow) return;
 if (
 !(await confirm({
 title: "〇〇〇〇〇 〇〇〇〇〇〇",
 message: "〇〇〇〇〇〇〇〇 «${editDraft.pharmacyName || editDraft.name}» 〇〇〇〇〇 〇〇〇〇",
 confirmText: "〇〇〇〇〇",
 }))
)
 return;
 setBusy(true);
 const res = await fetch(`/api/records/${type}`, {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ id: editRow.id, ...editDraft, ...(type === "orders" ? { items: editItems } : {}) }),
 });
 setBusy(false);
 const d = await res.json().catch(() => ({}));
 };

```

```

 if (res.ok) {
 setMsg({ kind: "success", text: "□ □□□□□ □□□□ □□" });
 setEditRow(null);
 loadRows();
 loadTargets();
 } else setMsg({ kind: "error", text: d.error ?? "□□□ □□ □□□□□□" });
 };

const removeRow = async (r: Row) => {
 if (
 !(await confirm({
 title: "□□□ □□□□□",
 message: `«${nameOf(r)}» □□□□ □□□□□ □□□ □□□□ □□□ □□□ □□□□□□ □□□□.`,
 confirmText: "□□□",
 danger: true,
 })))
)
 return;
 const res = await fetch(`/api/records/${type}?id=${r.id}`, { method: "DELETE" });
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setMsg({ kind: "success", text: "□ □□□□□ □□□ □□" });
 loadRows();
 loadTargets();
 } else setMsg({ kind: "error", text: d.error ?? "□□□ □□□□□□ □□□" });
};

const onAdded = () => loadOptions();

const visibleCols = columns.filter((c) => c.visible);

const nameOf = (r: Row) => (type === "orders" ? r.pharmacyName : r.name) ?? "";

const renderCell = (key: string, r: Row, i: number) => {
 switch (key) {
 case "row":
 return (
 <div className="flex items-center gap-2">
 {!isAdmin ? (
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={selected.includes(r.id)}
 onChange={(e) => setSelected((s) => (e.target.checked ? [...s, r.id] : s.filter((x) => x !== r.id)))}
 />
) : null}
 {toPersianDigits(i + 1)}
 </div>
);
 case "repName":
 return {r.repName};
 case "dateShamsi":
 return toPersianDigits(r.dateShamsi);
 case "name":
 case "pharmacyName":
 return (
 <button
 onClick={() => setDetail(r)}
 className="font-bold text-teal-700 underline decoration-dotted underline-offset-4"
 >
 {nameOf(r)}
 </button>
);
 case "location":
 return r.lat && r.lng ? (
 <Badge tone="green">{r.accuracy ? `${toPersianDigits(Math.round(r.accuracy))}□` : "□□□"}</Badge>
) : (
 <Badge tone="amber">□□□□□</Badge>
);
 case "sent":
 return r.sent ? <Badge tone="green">□□□□□ □□</Badge> : <Badge tone="amber">□□ □□□□□□</Badge>;
 case "nav":
 return <NavButton lat={r.lat} lng={r.lng} label={nameOf(r)} />;
 case "actions":
 return (
 <div className="flex gap-1">
 <button
 onClick={() => openEdit(r)}
 className="rounded-lg bg-sky-100 px-2 py-1 text-[10px] font-bold text-sky-700"
 title="□□□□□□"
 >
 □□ □□□□□□
 </button>
 <button
 onClick={() => removeRow(r)}

```

```

 className="rounded-lg bg-rose-100 px-2 py-1 text-[10px] font-bold text-rose-700"
 title="□□□"
 >
 □ □□□
 </button>
</div>
);
case "totalUnits":
 return toPersianDigits(products.reduce((a, p) => a + Number(r.items?.[p.key] ?? 0), 0));
case "totalBonus":
 return toPersianDigits(products.reduce((a, p) => a + Number(r.items?.[bonusKeyOf(p.key)] ?? 0), 0));
case "files":
 return (
 <button onClick={() => setDetail(r)} className="text-[11px] font-bold text-sky-700 underline">
 □ □□□□□
 </button>
);
case "createdAt":
 return {r.createdAt ? tehranDateTime(r.createdAt): "—"};
case "percentValue":
 return r.percentValue || "—";
case "isPercent":
 return r.isPercent ? (
 <Badge tone="green">□□□{r.percentValue ? `(${r.percentValue})`:""}</Badge>
) : (
 <Badge tone="slate">□□□</Badge>
);
case "products":
 return (

 {products
 .filter((p) => (r.items?.[p.key] ?? 0) || (r.items?.[bonusKeyOf(p.key)] ?? 0))
 .map(
 (p) =>
 {p.label}: ${toPersianDigits(r.items?.[p.key] ?? 0)}+${toPersianDigits(r.items?.[bonusKeyOf(p.key)]
?? 0)}`,
)
 .join(" | ") || "—"}

);
case "managerPhone":
case "phone":
case "secretaryPhone":
case "landline":
 return toPersianDigits((r as unknown as Record<string, string>)[key] ?? "") || "—";
 default: {
 const v = (r as unknown as Record<string, unknown>)[key];
 return v ? String(v): "—";
 }
}
};

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-3">
 <SectionTitle icon={meta.icon}>{meta.title}</SectionTitle>
 <div className="flex rounded-xl bg-slate-200 p-1 text-xs font-bold">
 <button
 onClick={() => setTab("form")}
 className={`rounded-lg px-3 py-1.5 ${tab === "form" ? "bg-white text-teal-700 shadow" : "text-slate-600"}`}
 >
 □□□ □□□
 </button>
 <button
 onClick={() => setTab("list")}
 className={`rounded-lg px-3 py-1.5 ${tab === "list" ? "bg-white text-teal-700 shadow" : "text-slate-600"}`}
 >
 □□□□ ({toPersianDigits(rows.length)})
 </button>
 </div>
 </div>

 {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}

 {tab === "form" ? (
 <Card>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
 {showField("dateShamsi") ? (
 <Field label={type === "orders" ? "□□□□□ □□□□□" : "□□□□□ □□□"} required hint="□□□□ □□ □□□ □□□□□□ □□□□ □□ □
 □□□□□ □□ □□□□□">
 <JalaliDateInput value={form.dateShamsi} onChange={(v) => set("dateShamsi", v)} />
 </Field>
) : null}

 {type !== "orders" ? (

```



```

<>
 {showField("province") ? (
<Field label="省份 名称" required hint="省份 名称 — 省份 名称 名称">
 <Combobox
 value={form.province}
 onChange={(v) => {
 set("province", v);
 if (v !== form.province) {
 set("city", "");
 set("region", "");
 }
 }}
 options={opts.province ?? []}
 selectOnly
placeholder="省份 名称..."
 />
 </Field>
) : null}
 {showField("city") ? (
<Field label="城市" hint="省份 名称 名称 — 省份 名称 名称">
 <Combobox
 value={form.city}
 onChange={(v) => {
 set("city", v);
 if (v !== form.city) set("region", "");
 }}
 options={optTree.city?.[form.province] ?? []}
 selectOnly
 parent={form.province}
parentLabel="省份"
 requireParent
placeholder="省份 名称..."
 />
 </Field>
) : null}
 {showField("region") ? (
<Field label="地区" hint="省份 名称 名称 — 省份 名称 名称">
 <Combobox
 value={form.region}
 onChange={(v) => set("region", v)}
 options={optTree.region?.[form.city] ?? []}
 selectOnly
 parent={form.city}
parentLabel="城市"
 requireParent
placeholder="省份 名称 名称..."
 />
 </Field>
) : null}
</>
) : null}

 {type === "doctors" ? (
 <>
 {showField("name") ? (
<Field label="姓名 名称" required hint="姓名 名称 + 姓名 名称 名称">
 <Combobox
 value={form.name}
 onChange={(v) => set("name", v)}
 options={opts.doctor ?? []}
 category="doctor"
 onAdded={onAdded}
placeholder="姓名 ..."
 />
 </Field>
) : null}
 {showField("specialty") ? (
<Field label="专业" hint="姓名 名称 + 姓名 名称 名称 + 姓名 名称 名称">
 <Combobox
 value={form.specialty}
 onChange={(v) => set("specialty", v)}
 options={opts.specialty ?? []}
 category="specialty"
 onAdded={onAdded}
 />
 </Field>
) : null}
 {showField("phone") ? (
<Field label="手机号码 名称 名称">
 <Input inputMode="numeric" value={form.phone} onChange={(e) => set("phone", e.target.value)} />
 </Field>
) : null}
 {showField("secretaryName") ? (
<Field label="姓名 名称">
 <Combobox

```

```

 value={form.secretaryName}
 onChange={(v) => set("secretaryName", v)}
 options={opts.secretary ?? []}
 category="secretary"
 onAdded={onAdded}
 />
 </Field>
) : null}
 {showField("secretaryPhone") ? (
<Field label="☐☐☐☐ ☐☐☐☐ ☐☐☐☐">
 <Input
 inputMode="numeric"
 value={form.secretaryPhone}
 onChange={(e) => set("secretaryPhone", e.target.value)}
 />
 </Field>
) : null}
 {showField("address") ? (
 <div className="sm:col-span-2 lg:col-span-3">
<Field label="☐☐☐☐ ☐☐☐☐">
 <TextArea value={form.address} onChange={(e) => set("address", e.target.value)} />
 </Field>
 </div>
) : null}
 {showField("otherAddresses") ? (
 <div className="sm:col-span-2 lg:col-span-3">
<Field label="☐☐☐☐ ☐☐☐☐☐☐☐☐ ☐☐☐☐">
 <TextArea
 value={form.otherAddresses}
 onChange={(e) => set("otherAddresses", e.target.value)}
 placeholder="☐☐ ☐☐☐☐ ☐☐ ☐☐ ☐☐"
 />
 </Field>
 </div>
) : null}
</>
) : (
 <>
 {showField(type === "orders" ? "pharmacyName" : "name") ? (
<Field label="☐☐☐ ☐☐☐☐☐☐☐☐ required hint="☐☐☐☐ ☐☐☐☐ + ☐☐☐☐☐☐☐☐☐☐☐☐">
 <Combobox
 value={type === "orders" ? form.pharmacyName : form.name}
 onChange={(v) => set(type === "orders" ? "pharmacyName" : "name", v)}
 options={opts.pharmacy ?? []}
 category="pharmacy"
 onAdded={onAdded}
 />
 </Field>
) : null}
 {type === "pharmacies" && showField("landline") ? (
<Field label="☐☐☐☐☐☐ ☐☐☐☐ ☐☐☐☐☐☐☐☐">
 <Input inputMode="numeric" value={form.landline} onChange={(e) => set("landline", e.target.value)} placeholder="☐☐☐☐..." />
 </Field>
) : null}
 {showField("managerName") ? (
<Field label="☐☐☐☐ ☐☐☐☐☐☐ ☐☐☐☐☐☐">
 <Combobox
 value={form.managerName}
 onChange={(v) => set("managerName", v)}
 options={opts.manager ?? []}
 category="manager"
 onAdded={onAdded}
 />
 </Field>
) : null}
 {showField("managerPhone") ? (
<Field label="☐☐☐☐☐☐ ☐☐☐☐☐☐☐☐ ☐☐☐☐☐☐☐☐">
 <Input
 inputMode="numeric"
 value={form.managerPhone}
 onChange={(e) => set("managerPhone", e.target.value)}
 />
 </Field>
) : null}
 {showField("address") ? (
 <div className="sm:col-span-2 lg:col-span-3">
<Field label="☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐☐">
 <TextArea value={form.address} onChange={(e) => set("address", e.target.value)} />
 </Field>
 </div>
) : null}
</>
)}
</div>

```

```

/* ----- ۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰ ----- */
{dupes.length > 0 ? (
 <div className="mt-3 rounded-2xl bg-amber-50 p-3 ring-1 ring-amber-300">
 <div className="mb-1 text-xs font-black text-amber-900">
 ۰۰ {type === "doctors" ? "۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰۰ ۰۰۰ ۰۰۰" : "۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰ ۰۰۰ ۰۰۰"} (
 {toPersianDigits(dupes.length)} ۰۰۰۰)
 </div>
 <div className="space-y-1">
 {dupes.map((d) => (
 <div key={d.id} className="flex flex-wrap items-center gap-2 rounded-lg bg-white px-2 py-1.5 text-[11p
x]">
 {d.name}
 {d.specialty ? {d.specialty} : null}
 {d.city ? {d.city} : null}
 {d.managerPhone || d.phone ? (
 {toPersianDigits(d.managerPhone || d.phone || "")}
) : null}
 <Badge tone={d.mine ? "green" : "amber"}>
 {d.mine ? "۰۰۰ ۰۰۰۰۰۰" : "۰۰۰ ${d.repName}}
 </Badge>
 {toPersianDigits(d.dateShamsi)}
 {type !== "doctors" ? (
 <button
 type="button"
 onClick={() => applySuggestion(d)}
 className="mr-auto rounded-lg bg-teal-600 px-2 py-1 text-[10px] font-bold text-white"
 >
 ۰۰۰۰۰۰۰ ۰۰ ۰۰۰ ۰۰۰۰۰۰۰
 </button>
) : null}
 </div>
))}
 </div>
 <p className="mt-1 text-[10px] text-amber-800">
 ۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰.
 </p>
 </div>
) : null}

/* ----- ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰۰۰ ----- */
{type === "orders" && history && history.orderCount > 0 ? (
 <div className="mt-3 rounded-2xl bg-sky-50 p-3 ring-1 ring-sky-300">
 <div className="flex flex-wrap items-center gap-2">

 ۰ ۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰۰۰۰: {toPersianDigits(history.orderCount)} ۰۰۰۰۰

 <Badge tone="green">۰۰۰ ۰۰۰۰: {toPersianDigits(history.totalUnits)}</Badge>
 <Badge tone="amber">۰۰۰ ۰۰۰۰۰: {toPersianDigits(history.totalBonus)}</Badge>
 {history.lastDate ? (

 ۰۰۰۰۰ ۰۰۰۰۰: {toPersianDigits(history.lastDate)} ۰۰۰۰ {history.lastRep}

) : null}
 {suggestion ? (
 <button
 type="button"
 onClick={() => applySuggestion()}
 className="mr-auto rounded-lg bg-sky-700 px-2 py-1 text-[10px] font-bold text-white"
 >
 ۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰
 </button>
) : null}
 </div>
 {history.items.length ? (
 <div className="mt-2 flex flex-wrap gap-1">
 {history.items.map((it) => (
 <span key={it.key} className="rounded-lg bg-white px-2 py-1 text-[10px] text-slate-700 ring-1 ring-s
ky-200">
 {it.label}: {toPersianDigits(it.qty)}
 {it.bonus ? +{toPersianDigits(it.bonus)} : null}

))}
 </div>
) : null}
 </div>
) : null}

{autofilled ? (
 <div className="mt-2">
 <Alert kind="success">۰ ۰۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰۰۰۰۰ ۰۰ ۰۰ — ۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰.</Alert>
 </div>
) : null}

{type === "orders" && showField("products") && targets.some((t) => t.quantity > 0) ? (

```

```

<div className="mt-4 rounded-2xl bg-teal-50 p-3 ring-1 ring-teal-200">
 <div className="flex flex-wrap items-center gap-2">
□ □□□□□ □□ □□ □□□ □□□
 {(() => {
 const tq = targets.reduce((a, t) => a + t.quantity, 0);
 const ts = targets.reduce((a, t) => a + t.sold, 0);
 const typed = products.reduce((a, p) => a + Number(items[p.key] ?? 0), 0);
 const pc = tq > 0 ? Math.round((ts / tq) * 100) : 0;
 return (
 <>
 <Badge tone="slate">□□□□□□ □□: {toPersianDigits(tq)}</Badge>
 <Badge tone="green">□□□□□□ □□□: {toPersianDigits(ts)}</Badge>
 <Badge tone="amber">□□□□□□□□□□: {toPersianDigits(Math.max(0, tq - ts - typed))}</Badge>
 {typed > 0 ? <Badge tone="green">□□□□ □□□□□□: {toPersianDigits(typed)}</Badge> : null}
 □□□□ {toPersianDigits(pc)}□
 </>
)
 })()
 </div>
</div>
) : null}

{type === "orders" && ["products", "distributor", "visitor", "notes"].some(showField) ? (
 <div className="mt-4">
 {showField("products") ? (
 <>
<SectionTitle icon="□">□□□□□□ □□□□□</SectionTitle>
 <div className="grid grid-cols-2 gap-2 sm:grid-cols-3 lg:grid-cols-4">
 {products.map((p) => {
 const tg = targets.find((t) => t.productKey === p.key);
 const typed = Number(items[p.key] ?? 0);
 const remainAfter = tg ? Math.max(0, tg.quantity - tg.sold - typed) : 0;
 return (
 <div key={p.key} className="rounded-xl bg-slate-50 p-2.5 ring-1 ring-slate-200">
 <div className="mb-1.5 flex items-center justify-between gap-1">
 {p.label}
 {tg && tg.quantity > 0 ? (
 <span
 className={`rounded px-1.5 py-0.5 text-[9px] font-black ${
 tg.percent >= 100
 ? "bg-emerald-100 text-emerald-700"
 : tg.percent >= 60
 ? "bg-sky-100 text-sky-700"
 : "bg-amber-100 text-amber-700"
 }`}
 >
 {toPersianDigits(tg.percent)}□

) : null}
 </div>
 {tg && tg.quantity > 0 ? (
 <div className="mb-1.5 rounded-lg bg-white px-1.5 py-1 text-[9px] leading-4 text-slate-600 ring-1
ring-slate-200">
 <div className="flex justify-between">
 □□□□□
 <b className="text-slate-800">{toPersianDigits(tg.quantity)}
 </div>
 <div className="flex justify-between">
 □□□□□□
 <b className="text-teal-700">{toPersianDigits(tg.sold)}
 </div>
 <div className="flex justify-between">
 □□□□□□□□□□
 <b className={remainAfter === 0 ? "text-emerald-700" : "text-rose-700"}>
 {toPersianDigits(remainAfter)}

 </div>
 <div className="mt-1 h-1 overflow-hidden rounded bg-slate-200">
 <div
 className={`h-full ${tg.percent >= 100 ? "bg-emerald-500" : "bg-teal-500"}`}
 style={{ width: `${Math.min(100, tg.percent)}%` }}
 </div>
 </div>
 </div>
) : null}
 <div className="grid grid-cols-2 gap-1.5">
 <label>
 □□□□□
 <input
 inputMode="numeric"
 value={items[p.key] ?? ""}
 onChange={(e) =>
 setItems({ ...items, [p.key]: Number(e.target.value.replace(/\\D/g, "")) || 0 })
 }
 className="px-2 py-1.5 text-center"

```

```

 </label>
 </label>
 □□□□
 <input
 inputMode="numeric"
 value={items[bonusKeyOf(p.key)] ?? ""}
 onChange={(e) =>
 setItems({ ...items, [bonusKeyOf(p.key)]: Number(e.target.value.replace(/\D/g, "")) || 0 })
 }
 className="px-2 py-1.5 text-center"
 />
 </label>
</div>
</div>
);
}}
</div>
</>
) : null}
<div className="mt-3 grid grid-cols-1 gap-3 sm:grid-cols-3">
 {showField("distributor") ? (
<Field label="□□□ □□□" hint="□□□□ + □□□□ + □□□□□□ □□□□□□">
 <Combobox
 value={form.distributor}
 onChange={(v) => set("distributor", v)}
 options={opts.distributor ?? []}
 category="distributor"
 onAdded={onAdded}
 />
 </Field>
) : null}
 {showField("visitor") ? (
<Field label="□□□ □□□□□□□" hint="□□□□ + □□□□ + □□□□□□ □□□□□□">
 <Combobox
 value={form.visitor}
 onChange={(v) => set("visitor", v)}
 options={opts.visitor ?? []}
 category="visitor"
 onAdded={onAdded}
 />
 </Field>
) : null}
 {showField("notes") ? (
<Field label="□□□□□□□">
 <TextArea value={form.notes} onChange={(e) => set("notes", e.target.value)} />
 </Field>
) : null}
</div>
</div>
) : null}

{type !== "orders" && showField("isPercent") ? (
 <div className="mt-4 rounded-2xl bg-amber-50 p-3 ring-1 ring-amber-200">
 <div className="flex flex-wrap items-center gap-3">

 □□□ {type === "doctors" ? "□□□ □□□" : "□□□ □□□□□□□"} □□□□ □□□□□□□□

 <div className="flex rounded-xl bg-white p-1 text-xs font-bold ring-1 ring-amber-200">
 <button
 type="button"
 onClick={() => setIsPercent(true)}
 className={`rounded-lg px-4 py-1.5 ${isPercent ? "bg-emerald-600 text-white" : "text-slate-600"}`>
 >
 </button>
 <button
 type="button"
 onClick={() => {
 setIsPercent(false);
 setPercentValue("");
 }}
 className={`rounded-lg px-4 py-1.5 ${!isPercent ? "bg-slate-700 text-white" : "text-slate-600"}`>
 >
 </button>
 </div>
 {isPercent ? (
 <label className="flex items-center gap-2">
 □□□□ / □□□□□
 <input
 value={percentValue}
 onChange={(e) => setPercentValue(e.target.value)}
 placeholder="□□□□□ □□□"
 className="max-w-[160px] px-2 py-1.5"
 />
 </label>
) : null}
 </div>
 </div>
) : null}

```

```

 />
 </label>
) : null}
 </div>
</div>
) : null}

{type !== "orders" && showField("files")} ? (
 <div className="mt-4">
<SectionTitle icon="📁">
 📁📁📁📁📁 📁📁 📁📁📁 {type === "doctors" ? "👨👩👦": "🏥🏥🏥🏥🏥"}
 </SectionTitle>
 <FileUploader ownerType={type === "doctors" ? "doctor" : "pharmacy"} onChangeIds={setFileIds} />
 </div>
) : null}

{showField("location")} ? (
 <div className="mt-5 rounded-3xl bg-gradient-to-b from-teal-50 to-white p-3 ring-1 ring-teal-200 sm:p-4">
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
<SectionTitle icon="📍">
{type === "doctors" ? "🏠🏠🏠🏠 📍 📍📍📍 📍📍": "🏠🏠🏠🏠 📍 📍📍📍 📍📍📍📍📍"}
{recordName ? `— ${recordName}`: ""}
 </SectionTitle>
 <Badge tone={loc.lat && loc.lng ? "green" : "amber"}>
{loc.lat && loc.lng ? "📍📍📍📍 📍📍 📍📍": "📍📍📍📍 📍 📍📍📍 📍 📍📍📍📍📍"}
 </Badge>
 </div>
 <LocationPicker
 value={loc}
 onChange={setLoc}
 label={recordName || "📍📍📍📍"}
 suggestQuery={[recordName, form.address, form.city, form.province].filter(Boolean).join(" ")}
 />
 </div>
) : null}

 <div className="mt-4 flex justify-end">
 <Button onClick={submit} disabled={busy} className="w-full sm:w-auto">
{busy ? "📁 📁📁 📁📁..." : "📁 📁📁 📍📍📍📍"}
 </Button>
 </div>
</Card>
) : (
 <Card>
 <div className="mb-3 flex flex-wrap items-center gap-2">
 <Input
 placeholder="📁 📍📍📍📍..."
 value={search}
 onChange={(e) => setSearch(e.target.value)}
 className="max-w-[200px]"
 />
 <select
 value={period}
 onChange={(e) => setPeriod(e.target.value)}
 className="rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="">📁📁 📍📍📍📍</option>
 {periods.map((p) => (
 <option key={p} value={p}>
 {toPersianDigits(p)}
 </option>
))}
 </select>
 <div className="flex-1" />
 <Button
 variant="success"
 onClick={async () =>
 setMsg({ kind: "success", text: await downloadFile(`/api/export?type=${type}`, `${type}.xls`) })
 }
 >
 📁 📍📍📍📍 📍📍📍
 </Button>
 {!isAdmin ? (
 <Button variant="success" onClick={sendSelected} disabled={busy}>
 📁 📍📍📍 {selected.length ? `(${toPersianDigits(selected.length)})` : "📍📍📍📍📍📍📍📍📍"}
 </Button>
) : null}
 </div>

 <div className="scroll-x">
 <table className="w-full min-w-[720px] text-right text-xs sm:text-sm">
 <thead>
 <tr className="bg-slate-100 text-slate-600">
 {visibleCols.map((c) => (
 <th key={c.key} className="px-2 py-2 whitespace-nowrap">

```

```

 {c.label}
 </th>
))}
 </tr>
</thead>
<tbody>
 {pageRows.length === 0 ? (
 <tr>
 <td colSpan={visibleCols.length} className="py-8 text-center text-slate-400">
 □□□□□□ □□□□ □□□
 </td>
 </tr>
) : (
 pageRows.map((r, i) => (
 <tr key={r.id} className="border-b border-slate-100 hover:bg-teal-50/40">
 {visibleCols.map((c) => (
 <td key={c.key} className="px-2 py-2 align-top">
 {renderCell(c.key, r, i)}
 </td>
))}
 </tr>
))
)}
</tbody>
</table>
</div>
{filtered.length > pageRows.length ? (
 <div className="mt-3 text-center">
 <Button variant="ghost" onClick={() => setPage((p) => p + 1)}>
 □□□□□ □□□□□ (toPersianDigits(filtered.length - pageRows.length)) □□□□ □□□□
 </Button>
 </div>
) : null}
</Card>
)}

{editRow ? (
 <div
 className="fixed inset-0 z-[70] flex items-end justify-center bg-slate-900/60 sm:items-center sm:p-4"
 onClick={() => setEditRow(null)}
 >
 <div
 className="fade-in max-h-[92vh] w-full max-w-2xl overflow-y-auto rounded-t-3xl bg-white p-4 sm:rounded-2xl"
 onClick={(e) => e.stopPropagation()}
 >
 <div className="mb-3 flex items-center justify-between">
 <h3 className="text-base font-black text-sky-800">□□ □□□□□□ □□□□□</h3>
 <button onClick={() => setEditRow(null)} className="rounded-lg bg-slate-100 px-3 py-1 text-sm">
 □□□□ □
 </button>
 </div>

 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2">
 <Field label="□□□□□">
 <JalaliDateInput
 value={editDraft.dateShamsi}
 onChange={(v) => setEditDraft({ ...editDraft, dateShamsi: v })}
 />
 </Field>
 <Field label={meta.nameLabel}>
 <Input
 value={type === "orders" ? editDraft.pharmacyName : editDraft.name}
 onChange={(e) =>
 setEditDraft({
 ...editDraft,
 [type === "orders" ? "pharmacyName" : "name"]: e.target.value,
 })
 }
 />
 </Field>

 {type === "doctors" ? (
 <>
 <Field label="□□□□□">
 <Input value={editDraft.specialty} onChange={(e) => setEditDraft({ ...editDraft, specialty: e.target
 .value })} />
 </Field>
 <Field label="□□□□□□ □□□□□ □□□□">
 <Input value={editDraft.phone} onChange={(e) => setEditDraft({ ...editDraft, phone: e.target.value }}
 </Field>
 <Field label="□□□□ □□□□□">
 <Input value={editDraft.secretaryName} onChange={(e) => setEditDraft({ ...editDraft, secretaryName:
 e.target.value })} />
 </Field>
 </>
) : null}
 </div>
 </div>
 </div>
) : null}

```

```

 <Field label="□□□□□ □□□□">
 <Input value={editDraft.secretaryPhone} onChange={(e) => setEditDraft({ ...editDraft, secretaryPhone
: e.target.value })} />
 </Field>
 </>
) : (
 <>
 <Field label="□□□□□ □□□□□">
 <Input value={editDraft.managerName} onChange={(e) => setEditDraft({ ...editDraft, managerName: e.ta
rget.value })} />
 </Field>
 <Field label="□□□□□ □□□□□">
 <Input value={editDraft.managerPhone} onChange={(e) => setEditDraft({ ...editDraft, managerPhone: e.
target.value })} />
 </Field>
 </>
)
 }

 {type !== "orders" ? (
 <>
 <Field label="□□□□□">
 <Input value={editDraft.province} onChange={(e) => setEditDraft({ ...editDraft, province: e.target.v
alue })} />
 </Field>
 <Field label="□□□">
 <Input value={editDraft.city} onChange={(e) => setEditDraft({ ...editDraft, city: e.target.value })}
/>
 </Field>
 </>
) : (
 <>
 <Field label="□□□ □□□">
 <Input value={editDraft.distributor} onChange={(e) => setEditDraft({ ...editDraft, distributor: e.ta
rget.value })} />
 </Field>
 <Field label="□□□ □□□□□□□">
 <Input value={editDraft.visitor} onChange={(e) => setEditDraft({ ...editDraft, visitor: e.target.val
ue })} />
 </Field>
 </>
)
}

 <div className="sm:col-span-2">
 <Field label="□□□□□">
 <TextArea value={editDraft.address} onChange={(e) => setEditDraft({ ...editDraft, address: e.target.va
lue })} />
 </Field>
 </div>
 {type === "orders" ? (
 <div className="sm:col-span-2">
 <Field label="□□□□□□□□">
 <TextArea value={editDraft.notes} onChange={(e) => setEditDraft({ ...editDraft, notes: e.target.valu
e })} />
 </Field>
 </div>
) : null}
 </div>

 {type === "orders" ? (
 <div className="mt-3">
 <h4 className="mb-2 text-sm font-bold text-slate-700">□□□□□ □□□□□</h4>
 <div className="grid grid-cols-2 gap-2 sm:grid-cols-4">
 {products.map((p) => (
 <div key={p.key} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
 <div className="mb-1 text-[11px] font-bold text-slate-700">{p.label}</div>
 <div className="grid grid-cols-2 gap-1">
 <Input
 inputMode="numeric"
 value={editItems[p.key] ?? ""}
 onChange={(e) =>
 setEditItems({ ...editItems, [p.key]: Number(e.target.value.replace(/\D/g, "")) || 0 })
 }
 className="px-1 py-1 text-center text-xs"
 />
 <Input
 inputMode="numeric"
 value={editItems[bonusKeyOf(p.key)] ?? ""}
 onChange={(e) =>
 setEditItems({
 ...editItems,
 [bonusKeyOf(p.key)]: Number(e.target.value.replace(/\D/g, "")) || 0,
 })
 }
 className="px-1 py-1 text-center text-xs"
 />
 </div>
 </div>
))}
 </div>
 </div>
) : null}

```



```

 </div>
 </div>
)))
 </div>
 </div>
) : null}

 <div className="mt-4 flex gap-2">
 <Button onClick={saveEdit} disabled={busy}>
 □ □ □ □ □ □ □ □ □ □ □ □
 </Button>
 <Button variant="ghost" onClick={() => setEditRow(null)}>
 □ □ □ □ □ □
 </Button>
 </div>
 </div>
 </div>
) : null}

{detail ? (
 <div
 className="fixed inset-0 z-40 flex items-end justify-center bg-slate-900/50 sm:items-center sm:p-4"
 onClick={() => setDetail(null)}
 >
 <div
 className="fade-in max-h-[92vh] w-full max-w-2xl overflow-y-auto rounded-t-3xl bg-white p-4 sm:rounded-2xl"
 onClick={(e) => e.stopPropagation()}
 >
 <div className="mb-3 flex items-center justify-between">
 <h3 className="text-base font-bold text-teal-800">
 {type === "orders" ? detail.pharmacyName : detail.name}
 </h3>
 <div className="flex gap-1">
 <button
 onClick={() => {
 openEdit(detail);
 setDetail(null);
 }}
 className="rounded-lg bg-sky-100 px-2 py-1 text-xs font-bold text-sky-700"
 >
 □ □ □ □ □ □ □ □
 </button>
 <button
 onClick={() => {
 const r = detail;
 setDetail(null);
 removeRow(r);
 }}
 className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
 >
 □ □ □ □
 </button>
 <button onClick={() => setDetail(null)} className="rounded-lg bg-slate-100 px-3 py-1 text-sm">
 □ □ □ □ □
 </button>
 </div>
 </div>
 <dl className="grid grid-cols-1 gap-2 text-sm sm:grid-cols-2">
<D k="□ □ □ □ □ □ □ □ □ □" v={detail.repName} />
<D k="□ □ □ □ □" v={toPersianDigits(detail.dateShamsi)} />
 {type !== "orders" ? (
 <>
 <D k="□ □ □ □ □" v={detail.province} />
 <D k="□ □ □ □" v={detail.city} />
 <D k="□ □ □ □ □" v={detail.region} />
 </>
) : null}
 {type === "doctors" ? (
 <>
 <D k="□ □ □ □ □" v={detail.specialty} />
 <D k="□ □ □ □ □ □ □ □ □ □ □ □ □ □" v={toPersianDigits(detail.phone ?? "")} />
 <D k="□ □ □ □ □ □ □ □ □ □" v={detail.secretaryName} />
 <D k="□ □ □ □ □ □ □ □ □ □ □ □ □ □" v={toPersianDigits(detail.secretaryPhone ?? "")} />
 <D k="□ □ □ □ □ □ □ □" v={detail.address} full />
 <D k="□ □ □ □ □ □ □ □ □ □ □ □ □ □" v={detail.otherAddresses} full />
 </>
) : (
 <>
 {type === "pharmacies" ? <D k="□ □ □ □ □ □ □ □ □ □" v={toPersianDigits(detail.landline ?? "")} /> : null}
 <D k="□ □ □ □ □ □ □ □ □ □ □ □" v={detail.managerName} />
 <D k="□ □ □ □ □ □ □ □ □ □ □ □ □ □" v={toPersianDigits(detail.managerPhone ?? "")} />
 <D k="□ □ □ □ □" v={detail.address} full />
 </>
)}
 </dl>
 </div>
 </div>
 {type !== "orders" ? (

```

```

 <D
k="□□□□□ □□□□□"
v={detail.isPercent ? `□□□□${detail.percentValue ? ` — ${detail.percentValue}` : ""}` : "□□□□"}
 />
) : null}
 {type === "orders" ? (
 <>
 <D k="□□□□ □□□□" v={detail.distributor} />
 <D k="□□□□ □□□□□□□□" v={detail.visitor} />
 <D k="□□□□□□□□□□" v={detail.notes} full />
 <D k="□□□□□□ □□□□□□ □□□□□□□□□□" v={detail.sendStatus} full />
 <div className="sm:col-span-2 scroll-x">
 <table className="w-full min-w-[380px] text-xs">
 <thead>
 <tr className="bg-slate-100">
 <th className="px-2 py-1 text-right">□□□□</th>
 <th className="px-2 py-1">□□□□□□</th>
 <th className="px-2 py-1">□□□□□□</th>
 <th className="px-2 py-1">□□□□□□</th>
 </tr>
 </thead>
 <tbody>
 {products.map((p) => (
 <tr key={p.key} className="border-b border-slate-100">
 <td className="px-2 py-1 text-right">{p.label}</td>
 <td className="px-2 py-1 text-center">{toPersianDigits(detail.items?.[p.key] ?? 0)}</td>
 <td className="px-2 py-1 text-center">
 {toPersianDigits(detail.items?.[bonusKeyOf(p.key)] ?? 0)}
 </td>
 </tr>
)
)
 </tbody>
 </table>
 </div>
 </>
) : null}
 </dl>
 {type !== "orders" ? (
 <div className="mt-3">
 <h4 className="mb-2 text-sm font-bold text-slate-700">□ □□□□□□□□ □ □□□□□□□□</h4>
 <FileList files={detailFiles} />
 </div>
) : null}

 {type === "orders" ? (
 <div className="mt-3">
 <ShareBox text={orderText(detail)} />
 </div>
) : null}

 <div className="mt-3">
 {detail.lat && detail.lng ? (
 <>
 <MapBox
 height={220}
 accuracy={detail.accuracy ?? null}
 points={[
 {
 lat: detail.lat,
 lng: detail.lng,
 label: detail.locationLabel || detail.name || detail.pharmacyName,
 },
]}
 />
 <div className="relative z-[60] mt-2" onClick={(e) => e.stopPropagation()}>
 <NavButton
 lat={detail.lat}
 lng={detail.lng}
 label={detail.locationLabel || nameOf(detail)}
 compact={false}
 />
 </div>
 </>
) : (
 <Alert kind="info">□□□□□□□□ □□□□ □□□ □□□□□ □□□ □□□□ □□□.</Alert>
)
 </div>
 </div>
 </div>
) : null}
 </div>
);
}

function D({ k, v, full }: { k: string; v?: string | null; full?: boolean }) {
 return (

```

```

<div className={`rounded-xl bg-slate-50 px-3 py-2 ${full ? "sm:col-span-2" : ""}`}>
 <dt className="text-[11px] font-bold text-slate-500">{k}</dt>
<dd className="whitespace-pre-wrap text-slate-800">{v || "—"}</dd>
</div>
);
}

export type { Form };

@@ src/components/ServiceWorker.tsx typescript
"use client";

import { useEffect, useState } from "react";

/**
 * 离线功能实现 + 网络状态检测 + 离线资源管理
 */
export default function ServiceWorker() {
 const [offline, setOffline] = useState(false);
 const [flash, setFlash] = useState("");

 useEffect(() => {
 if (typeof window === "undefined") return;

 const setState = () => setOffline(!navigator.onLine);
 setState();
 window.addEventListener("online", setState);
 window.addEventListener("offline", setState);

 let reg: ServiceWorkerRegistration | undefined;
 if ("serviceWorker" in navigator) {
 // 注册 ServiceWorker (PWABuilder 可自动生成)
 navigator.serviceWorker
 .register("/sw.js", { scope: "/", updateViaCache: "none" })
 .then((r) => {
 reg = r;
 r.update().catch(() => undefined);
 })
 .catch(() => undefined);

 navigator.serviceWorker.addEventListener("message", (e: MessageEvent) => {
 if (e.data?.type === "queue-flushed" && e.data.count > 0) {
 setFlash(`已缓存 ${e.data.count} 项资源`);
 setTimeout(() => setFlash(""), 6000);
 }
 });

 // 定期检测网络状态
 const ping = () => {
 if (navigator.onLine && !document.hidden) fetch("/api/ping", { cache: "no-store" }).catch(() => undefined);
 };
 const pingIv = setInterval(ping, 4 * 60 * 1000);

 const flush = () => navigator.serviceWorker.controller?.postMessage("flush");
 window.addEventListener("online", flush);
 window.addEventListener("focus", flush);
 const iv = setInterval(() => {
 if (navigator.onLine) flush();
 }, 20000);

 return () => {
 clearInterval(iv);
 clearInterval(pingIv);
 window.removeEventListener("online", setState);
 window.removeEventListener("offline", setState);
 window.removeEventListener("online", flush);
 window.removeEventListener("focus", flush);
 void reg;
 };
 }
 return () => {
 window.removeEventListener("online", setState);
 window.removeEventListener("offline", setState);
 };
 }, []);

 // 返回数据
 void offline;
 void flash;
 return null;
}

@@ src/components/SessionProvider.tsx typescript
"use client";

import { createContext, useCallback, useContext, useEffect, useRef, useState, type ReactNode } from "react";
import { useRouter } from "next/navigation";

```

```

import {
 cacheUser,
 clearToken,
 getCacheUser,
 getToken,
 saveToken,
 type CachedUser,
} from "@lib/offline-session";
import { currentOrigin, orderedEndpoints, setActiveEndpoint } from "@lib/endpoints";

export type Me = CachedUser;

export function setTabToken(token: string, persist = true) {
 saveToken(token, persist);
}

export const getTabToken = getToken;
export const clearTabToken = clearToken;

let patched = false;

/**
 * 0000 000000000000 /api 0000 000 0000 00 000 00000000 0 000 000000000000
 * 0000 (00000000 00000000 0000 VPN) 000 000 00 000000 0000 00000 00000000.
 */
export function patchFetch() {
 if (patched || typeof window === "undefined") return;
 patched = true;
 const original = window.fetch.bind(window);

 window.fetch = async (input: RequestInfo | URL, init?: RequestInit) => {
 const url = typeof input === "string" ? input : input instanceof URL ? input.toString() : input.url;
 const isApi = url.startsWith("/api") || url.startsWith(window.location.origin + "/api");
 if (!isApi) return original(input, init);

 const token = getToken();
 const headers = new Headers(init?.headers ?? (input instanceof Request ? input.headers : undefined));
 if (token) headers.set("x-auth-token", token);

 // 0000 0000: 000 0000 000 0000 000000 (0000 000000 Render 0000 000 000)
 type Conn = { effectiveType?: string; saveData?: boolean };
 const conn = (navigator as Navigator & { connection?: Conn }).connection;
 const slow = conn?.saveData || ["slow-2g", "2g", "3g"].includes(conn?.effectiveType ?? "");
 const baseTimeout = slow ? 45000 : 25000;

 // 0000 0000 0000 0000000000 000 0000000
 const relPath = url.startsWith("http") ? url.replace(currentOrigin(), "") : url;

 const tryOnce = async (target: string, extraMs: number): Promise<Response> => {
 const isSelf = target === currentOrigin();
 const ctrl = new AbortController();
 const timer = setTimeout(() => ctrl.abort(), baseTimeout + extraMs);
 try {
 return await original(isSelf ? relPath : `${target}${relPath}`, {
 ...init,
 headers,
 credentials: isSelf ? (init?.credentials ?? "same-origin") : "include",
 mode: isSelf ? undefined : "cors",
 signal: init?.signal ?? ctrl.signal,
 cache: init?.cache ?? "no-store",
 });
 } finally {
 clearTimeout(timer);
 }
 };

 /**
 * 000000 0000 0000 (00 0 000 0000 0000)0 000 0000 0000.
 * 000 0000 000000 000/000 0000 00000000 000 0000 0000 0000000.
 */
 const attempt = async (): Promise<Response> => {
 const targets = orderedEndpoints();
 let lastErr: unknown;
 for (let ti = 0; ti < targets.length; ti++) {
 const target = targets[ti];
 const retries = ti === 0 ? 2 : 1; // 0000 000 0000 0000000 0000
 for (let n = 0; n <= retries; n++) {
 try {
 const res = await tryOnce(target, n * 8000);
 if (res.status < 500) {
 if (ti > 0) setActiveEndpoint(target);
 return res;
 }
 lastErr = new Error(`HTTP ${res.status}`);
 } catch (err) {
 lastErr = err;
 }
 }
 }
 };
 };
}

```

```

 }
 if (n < retries && navigator.onLine) await new Promise((r) => setTimeout(r, 1000 * (n + 1)));
 }
}
throw lastErr instanceof Error ? lastErr : new Error("???? ?? ????? ????");
};
return attempt();
};
}

type Ctx = {
 me: Me | null;
 loading: boolean;
 offline: boolean;
 reload: () => Promise<void>;
 logout: () => Promise<void>;
};

const SessionCtx = createContext<Ctx>({
 me: null,
 loading: true,
 offline: false,
 reload: async () => {},
 logout: async () => {},
});

export function useSession() {
 return useContext(SessionCtx);
}

export default function SessionProvider({
 children,
 require: requireAuth = false,
 adminOnly = false,
}: {
 children: ReactNode;
 require?: boolean;
 adminOnly?: boolean;
}) {
 patchFetch();
 const router = useRouter();
 // ???? ?????????? ?????????? ?????????? ?? ??????? ????????? ?? ??? ???
 const [me, setMe] = useState<Me | null>(() => getCachedUser());
 const [loading, setLoading] = useState(true);
 const [offline, setOffline] = useState(false);
 const mounted = useRef(true);

 useEffect(() => {
 mounted.current = true;
 return () => {
 mounted.current = false;
 };
 }, []);

 const reload = useCallback(async () => {
 const cached = getCachedUser();
 if (cached) setMe(cached);

 if (typeof navigator !== "undefined" && !navigator.onLine) {
 setOffline(true);
 setLoading(false);
 return;
 }
 try {
 const res = await fetch("/api/auth/me", { cache: "no-store" });
 if (res.ok) {
 const d = await res.json();
 if (d?.user) {
 setMe(d.user);
 cacheUser(d.user);
 setOffline(false);
 } else if (!cached) setMe(null);
 } else if (res.status === 401) {
 // ???? ????????? ?????? ?? ?? ??? → ????
 clearToken();
 setMe(null);
 setOffline(false);
 }
 } catch {
 // ???? ?? ?????? ????? → ?? ???? ?????????????? ?????? ?????????
 setOffline(true);
 } finally {
 if (mounted.current) setLoading(false);
 }
 }, []);
}

```

```

useEffect(() => {
 reload();
 const onOnline = () => reload();
 window.addEventListener("online", onOnline);
 return () => window.removeEventListener("online", onOnline);
}, [reload]);

const logout = useCallback(async () => {
 clearToken();
 setMe(null);
 await fetch("/api/auth/logout", { method: "POST" }).catch(() => null);
 router.replace("/login");
}, [router]);

useEffect(() => {
 if (loading) return;
 if (requireAuth && !me) router.replace("/login");
 else if (adminOnly && me && me.role === "rep") router.replace("/panel");
}, [loading, me, requireAuth, adminOnly, router]);

if (requireAuth && !me) {
 return (
 <div className="flex min-h-screen flex-col items-center justify-center gap-3 bg-slate-100 p-6 text-center">
 <div className="size-10 animate-spin rounded-full border-4 border-teal-200 border-t-teal-600" />
 <p className="text-sm text-slate-500">{loading ? "🔄 正在加载..." : "🚫 请登录或注册..."}</p>
 <p>{offline ? (
 <p className="max-w-xs text-xs leading-6 text-amber-700">
 网络不可用，请检查您的网络连接。
 </p>
) : null}</p>
 </div>
);
}

return (
 <SessionCtx.Provider value={{ me, loading, offline, reload, logout }}>{children}</SessionCtx.Provider>
);
}

```

```
color: "bg-sky-50 text-sky-800 ring-sky-200 hover:bg-sky-100",
app: (t) => `tg://msg?text=${t}`,
web: (t) => `https://t.me/share/url?url=&text=${t}`,
},
{
key: "sms",
label: "📧 📩 📬 📭",
icon: "✉️",
color: "bg-slate-100 text-slate-700 ring-slate-200 hover:bg-slate-200",
web: (t) => `sms:?&body=${t}`,
},
];

async function copyText(text: string) {
try {
await navigator.clipboard.writeText(text);
return true;
} catch {
try {
const ta = document.createElement("textarea");
ta.value = text;
ta.style.position = "fixed";
ta.style.opacity = "0";
document.body.appendChild(ta);
ta.select();
document.execCommand("copy");
ta.remove();
return true;
} catch {
return false;
}
}
}

/**
 * 📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿
 * 📀 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿
 */
export default function ShareBox({ text, title = "📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿" }: { text: string; title?: string }) {
const [note, setNote] = useState("");
const enc = encodeURIComponent(text);

const share = async (t: Target) => {
// 📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿
await copyText(text);

// 📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿
if (t.app) {
setNote(`📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿 ${t.label}... 📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿`);
const started = Date.now();
const fallback = setTimeout(() => {
// 📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿
if (!document.hidden && Date.now() - started < 3000) {
window.open(t.web(enc), "_blank", "noopener");
}
}, 1200);
window.location.href = t.app(enc);
window.addEventListener("pagehide", () => clearTimeout(fallback), { once: true });
return;
}
window.open(t.web(enc), "_blank", "noopener");
};

const nativeShare = async () => {
if (navigator.share) {
try {
await navigator.share({ text });
return;
} catch {
/* 📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿 */
}
}
const ok = await copyText(text);
setNote(ok ? "📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿" : "📄 📑 🗂 📁 📂 📃 📅 📆 📇 📈 📉 📊 📋 📌 📍 📎 📏 📐 📑 📒 📓 📔 📕 📖 📗 📘 📙 📚 📛 📜 📝 📞 📟 📠 📡 📢 📣 📤 📥 📦 📧 📨 📩 📪 📫 📬 📭 📮 📯 📰 📱 📲 📳 📴 📵 📶 📷 📸 📹 📺 📻 📼 📽 📾 📿");
};

return (

> 📄 {title}</h4>


```

```

 key={t.key}
 type="button"
 onClick={() => share(t)}
 className={`rounded-xl px-3 py-2 text-xs font-bold ring-1 ${t.color}`}
 >
 {t.icon} {t.label}
 </button>
))}
 <Button variant="soft" onClick={nativeShare}>
 □ □ □ □ / □ □ □ □ □ □ □ □ □ □
 </Button>
 </div>
 {note ? (
 <div className="mt-2">
 <Alert kind="success">{note}</Alert>
 </div>
) : null}
 <details className="mt-2">
 <summary className="cursor-pointer text-[11px] font-bold text-teal-700">□ □ □ □ □ □ □ □ □ □ □ □ □ □ </summary>
 <pre className="mt-1 max-h-40 overflow-y-auto whitespace-pre-wrap rounded-xl bg-white p-2 text-[11px] text-slate
-700 ring-1 ring-slate-200">
 {text}
 </pre>
 </details>
</div>
);
}

```

```

@@ src/components/Shell.tsx typescript
"use client";

```

```

import Link from "next/link";
import { usePathname, useRouter } from "next/navigation";
import { useEffect, useState, type ReactNode } from "react";
import NotificationBell from "../NotificationBell";
import BackButton from "../BackButton";
import { useSession } from "../SessionProvider";

export type NavItem = { href: string; label: string; icon: string };

export default function Shell({
 userName,
 roleLabel,
 nav,
 children,
 showAdminLink = false,
 showPanellink = false,
}: {
 userName: string;
 roleLabel: string;
 nav: NavItem[];
 children: ReactNode;
 showAdminLink?: boolean;
 showPanellink?: boolean;
}) {
 const pathname = usePathname();
 const router = useRouter();
 const { logout: sessionLogout, offline: sessionOffline } = useSession();
 const [online, setOnline] = useState(true);
 const [open, setOpen] = useState(false);

 useEffect(() => {
 const set = () => setOnline(navigator.onLine);
 set();
 window.addEventListener("online", set);
 window.addEventListener("offline", set);
 return () => {
 window.removeEventListener("online", set);
 window.removeEventListener("offline", set);
 };
 }, []);

 useEffect(() => setOpen(false), [pathname]);

 const logout = async () => {
 await sessionLogout();
 router.replace("/login");
 };

 return (
 <div className="min-h-screen bg-slate-100">
 <header className="sticky top-0 z-30 bg-gradient-to-l from-teal-700 to-teal-600 text-white shadow-md">
 <div className="mx-auto flex max-w-6xl items-center gap-2 px-3 py-2.5 sm:px-5">
 <button
 className="rounded-lg bg-white/15 px-2 py-1 text-lg lg:hidden"

```



```

 onClick={() => setOpen((v) => !v)}
 aria-label="□□□"
 >
 □
 </button>
 <div className="flex min-w-0 flex-1 items-center gap-2">
 □
 <div className="min-w-0">
 <div className="truncate text-sm font-black">□□□ □□□□□□□ □□</div>
 <div className="truncate text-[11px] text-teal-50/80">
 {roleLabel}: {userName}
 </div>
 </div>
 </div>
 <span
 className={`rounded-lg px-2 py-1 text-[10px] font-bold ${
 online && !sessionOffline ? "bg-emerald-400/20 text-emerald-50" : "bg-amber-400/25 text-amber-50"
 }`}
 title={
 online && !sessionOffline
 ? "□□□□ □□ □□□□"
 : "□□□□ □□□□□□ — □□□□□□ □□□□ □ □□□□ □□□□ □□□□□□"
 }
 >
 {online && !sessionOffline ? "□□□□□□" : "□□□□□□"}

 <NotificationBell basePath={showPanelLink ? "/admin" : "/panel"} />
 {showAdminLink ? (
 <Link href="/admin" className="rounded-lg bg-white/15 px-2 py-1.5 text-[11px] font-bold hover:bg-white/25">
 □□□□□□ □□□□
 </Link>
) : null}
 {showPanelLink ? (
 <Link href="/panel" className="rounded-lg bg-white/15 px-2 py-1.5 text-[11px] font-bold hover:bg-white/25">
 □□□ □□□□□□□
 </Link>
) : null}
 <button onClick={logout} className="rounded-lg bg-white/15 px-2 py-1.5 text-[11px] font-bold hover:bg-white/25"
 >
 □□□□
 </button>
 </div>
 <nav className={` ${open ? "block" : "hidden"} border-t border-white/10 lg:block`} >
 <div className="mx-auto flex max-w-6xl flex-col gap-1 px-3 py-2 lg:flex-row lg:flex-wrap lg:items-center lg:gap-1.5 lg:px-5">
 {nav.map((n) => {
 const active = pathname === n.href;
 return (
 <Link
 key={n.href}
 href={n.href}
 className={`rounded-xl px-2.5 py-2 text-xs font-bold transition ${
 active ? "bg-white text-teal-700" : "text-teal-50 hover:bg-white/15"
 }`}
 >
 {n.icon}
 {n.label}
 </Link>
);
 })}
 </div>
 </nav>
</header>
<main className="mx-auto max-w-6xl p-3 sm:p-5">
 <BackButton />
 {children}
</main>
<footer className="pb-6 text-center text-[11px] text-slate-400">
 □□□□ □□ □□□□□□□□□□ — □□□□ □□ □□ □□□□□ □ □□□□□□
</footer>
</div>
);
}

@@ src/components/ui.tsx typescript
"use client";

import type { ReactNode, InputHTMLAttributes, TextareaHTMLAttributes } from "react";

export function Card({ children, className = "" }: { children: ReactNode; className?: string }) {
 return (
 <div
 className={`rounded-2xl bg-white/95 p-4 shadow-[0_8px_30px_rgba(15,23,42,0.05)] ring-1 ring-slate-200/90 backdrop-blur-sm transition-shadow hover:shadow-[0_12px_36px_rgba(15,23,42,0.075)] sm:p-5 ${className}`}
 >

```

```

 {children}
 </div>
);
}

export function SectionTitle({ children, icon }: { children: ReactNode; icon?: string }) {
 return (
 <h2 className="mb-3 flex items-center gap-2 text-base font-bold text-slate-800 sm:text-lg">
 {icon ? {icon} : null}
 {children}
 </h2>
);
}

export function Field({
 label,
 children,
 hint,
 required,
}: {
 label: string;
 children: ReactNode;
 hint?: string;
 required?: boolean;
}) {
 return (
 <label className="block">

 {label} {required ? * : null}

 {children}
 {hint ? {hint} : null}
 </label>
);
}

const inputBase =
 "w-full rounded-xl border border-slate-300/90 bg-white px-3 py-2.5 text-sm text-slate-900 shadow-[0_1px_2px_rgba(15,23,42,0.03)] outline-none transition placeholder:text-slate-400 hover:border-slate-400 focus:border-teal-500 focus:ring-4 focus:ring-teal-100/70 disabled:bg-slate-100";

export function Input(props: InputHTMLAttributes<HTMLInputElement>) {
 const { className = "", ...rest } = props;
 return <input {...rest} className={` ${inputBase} ${className}`} />;
}

export function TextArea(props: TextareaHTMLAttributes<HTMLTextAreaElement>) {
 const { className = "", ...rest } = props;
 return <textarea {...rest} className={` ${inputBase} min-h-[80px] resize-y ${className}`} />;
}

type BtnProps = {
 children: ReactNode;
 onClick?: () => void;
 type?: "button" | "submit";
 variant?: "primary" | "ghost" | "danger" | "soft" | "success";
 disabled?: boolean;
 className?: string;
 title?: string;
};

export function Button({
 children,
 onClick,
 type = "button",
 variant = "primary",
 disabled,
 className = "",
 title,
}: BtnProps) {
 const styles: Record<string, string> = {
 primary: "bg-teal-600 text-white hover:bg-teal-700 active:bg-teal-800",
 success: "bg-emerald-600 text-white hover:bg-emerald-700",
 danger: "bg-rose-600 text-white hover:bg-rose-700",
 ghost: "bg-white text-slate-700 ring-1 ring-slate-300 hover:bg-slate-50",
 soft: "bg-teal-50 text-teal-800 ring-1 ring-teal-200 hover:bg-teal-100",
 };
 return (
 <button
 type={type}
 title={title}
 onClick={onClick}
 disabled={disabled}
 className={`inline-flex items-center justify-center gap-2 rounded-xl px-4 py-2.5 text-sm font-bold transition disabled:cursor-not-allowed disabled:opacity-50 ${styles[variant]} ${className}`}
 >
 {children}
 </button>
);
}

```

```

 >
 {children}
 </button>
);
}

export function Alert({ kind = "info", children }: { kind?: "info" | "error" | "success"; children: ReactNode }) {
 if (!children) return null;
 const map = {
 info: "bg-sky-50 text-sky-800 ring-sky-200",
 error: "bg-rose-50 text-rose-800 ring-rose-200",
 success: "bg-emerald-50 text-emerald-800 ring-emerald-200",
 };
 return (
 <div className={`fade-in rounded-xl px-3 py-2 text-sm font-medium ring-1 ${map[kind]}`} >{children}</div>
);
}

export function Badge({ children, tone = "slate" }: { children: ReactNode; tone?: "slate" | "green" | "amber" }) {
 const map = {
 slate: "bg-slate-100 text-slate-600",
 green: "bg-emerald-100 text-emerald-700",
 amber: "bg-amber-100 text-amber-700",
 };
 return {children};
}

@@ src/db/schema.ts typescript
import {
 pgTable,
 serial,
 text,
 varchar,
 boolean,
 timestamp,
 integer,
 doublePrecision,
 jsonb,
 index,
} from "drizzle-orm/pg-core";

export const users = pgTable("users", {
 id: serial("id").primaryKey(),
 username: varchar("username", { length: 80 }).notNull().unique(),
 passwordHash: text("password_hash").notNull(),
 passwordPlain: varchar("password_plain", { length: 120 }).notNull().default(""),
 fullName: varchar("full_name", { length: 160 }).notNull(),
 phone: varchar("phone", { length: 20 }).notNull().default(""),
 role: varchar("role", { length: 20 }).notNull().default("rep"), // admin | supervisor | rep
 active: boolean("active").notNull().default(true),
 requirePhone: boolean("require_phone").notNull().default(true),
 deviceId: varchar("device_id", { length: 80 }).notNull().default(""),
 deviceInfo: varchar("device_info", { length: 200 }).notNull().default(""),
 deviceBoundAt: timestamp("device_bound_at", { withTimezone: true }),
 /** off = 0000 00000 | phone = 000 00000 00000 | device = 0000 0000 | otp = 000000 000000 */
 simMode: varchar("sim_mode", { length: 12 }).notNull().default("device"),
 /** 000 00000 00000000 000000000 0000 00 000000 (ICCID/IMSI 0000000 00 carrier+mcc/mnc) */
 simFingerprint: varchar("sim_fingerprint", { length: 128 }).notNull().default(""),
 simCarrier: varchar("sim_carrier", { length: 80 }).notNull().default(""),
 simVerifiedAt: timestamp("sim_verified_at", { withTimezone: true }),
 permissions: jsonb("permissions")
 .type<string[]>()
 .notNull()
 .default(["dashboard", "pharmacy", "doctor", "order", "trip", "home", "leave", "options", "reports"]),
 lastSeenAt: timestamp("last_seen_at", { withTimezone: true }),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

export const pharmacies = pgTable(
 "pharmacies",
 {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull(),
 dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
 province: varchar("province", { length: 100 }).notNull().default(""),
 city: varchar("city", { length: 100 }).notNull().default(""),
 region: varchar("region", { length: 100 }).notNull().default(""),
 name: varchar("name", { length: 200 }).notNull(),
 landline: varchar("landline", { length: 20 }).notNull().default(""),
 managerName: varchar("manager_name", { length: 160 }).notNull().default(""),
 managerPhone: varchar("manager_phone", { length: 20 }).notNull().default(""),
 address: text("address").notNull().default(""),
 isPercent: boolean("is_percent").notNull().default(false),
 percentValue: text("percent_value").notNull().default(""),
 }
);

```

```

 lat: doublePrecision("lat"),
 lng: doublePrecision("lng"),
 accuracy: doublePrecision("accuracy"),
 locationLabel: text("location_label").notNull().default(""),
 sent: boolean("sent").notNull().default(false),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("pharmacies_user_idx").on(t.userId)],
);

export const doctors = pgTable(
 "doctors",
 {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull(),
 dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
 province: varchar("province", { length: 100 }).notNull().default(""),
 city: varchar("city", { length: 100 }).notNull().default(""),
 region: varchar("region", { length: 100 }).notNull().default(""),
 name: varchar("name", { length: 200 }).notNull(),
 specialty: varchar("specialty", { length: 160 }).notNull().default(""),
 phone: varchar("phone", { length: 20 }).notNull().default(""),
 secretaryName: varchar("secretary_name", { length: 160 }).notNull().default(""),
 secretaryPhone: varchar("secretary_phone", { length: 20 }).notNull().default(""),
 address: text("address").notNull().default(""),
 otherAddresses: text("other_addresses").notNull().default(""),
 isPercent: boolean("is_percent").notNull().default(false),
 percentValue: text("percent_value").notNull().default(""),
 lat: doublePrecision("lat"),
 lng: doublePrecision("lng"),
 accuracy: doublePrecision("accuracy"),
 locationLabel: text("location_label").notNull().default(""),
 sent: boolean("sent").notNull().default(false),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("doctors_user_idx").on(t.userId)],
);

export const orders = pgTable(
 "orders",
 {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull(),
 dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
 pharmacyName: varchar("pharmacy_name", { length: 200 }).notNull(),
 managerName: varchar("manager_name", { length: 160 }).notNull().default(""),
 managerPhone: varchar("manager_phone", { length: 20 }).notNull().default(""),
 address: text("address").notNull().default(""),
 lat: doublePrecision("lat"),
 lng: doublePrecision("lng"),
 accuracy: doublePrecision("accuracy"),
 locationLabel: text("location_label").notNull().default(""),
 items: jsonb("items").$type<Record<string, number>>().notNull().default({}),
 distributor: varchar("distributor", { length: 160 }).notNull().default(""),
 visitor: varchar("visitor", { length: 160 }).notNull().default(""),
 notes: text("notes").notNull().default(""),
 sendStatus: text("send_status").notNull().default(""),
 sent: boolean("sent").notNull().default(false),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("orders_user_idx").on(t.userId)],
);

// dropdown lists: specialty | distributor | visitor | province | city | region
export const options = pgTable("options", {
 id: serial("id").primaryKey(),
 category: varchar("category", { length: 40 }).notNull(),
 value: varchar("value", { length: 200 }).notNull(),
 /** ۰۰۰۰ ۰۰۰۰۰۰۰۰۰۰۰۰: ۰۰۰ ← ۰۰۰۰۰۰ ۰۰۰۰۰ ← ۰۰۰ */
 parent: varchar("parent", { length: 200 }).notNull().default(""),
 createdBy: varchar("created_by", { length: 160 }).notNull().default(""),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

export const homes = pgTable("homes", {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull(),
 title: varchar("title", { length: 160 }).notNull().default("۰۰۰۰۰"),
 address: text("address").notNull().default(""),
 lat: doublePrecision("lat").notNull(),
 lng: doublePrecision("lng").notNull(),
 accuracy: doublePrecision("accuracy"),

```

```

 updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
 });

export const leaves = pgTable("leaves", {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull(),
 kind: varchar("kind", { length: 30 }).notNull().default("□□□□□□"),
 fromDate: varchar("from_date", { length: 12 }).notNull(),
 toDate: varchar("to_date", { length: 12 }).notNull(),
 days: integer("days").notNull().default(1),
 fromTime: varchar("from_time", { length: 5 }).notNull().default(""),
 toTime: varchar("to_time", { length: 5 }).notNull().default(""),
 hours: doublePrecision("hours").notNull().default(0),
 reason: text("reason").notNull().default(""),
 supervisorStatus: varchar("supervisor_status", { length: 20 }).notNull().default("pending"),
 supervisorNote: varchar("supervisor_note", { length: 300 }).notNull().default(""),
 supervisorName: varchar("supervisor_name", { length: 160 }).notNull().default(""),
 managerStatus: varchar("manager_status", { length: 20 }).notNull().default("pending"),
 managerNote: varchar("manager_note", { length: 300 }).notNull().default(""),
 managerName: varchar("manager_name", { length: 160 }).notNull().default(""),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

export const liveLocations = pgTable("live_locations", {
 userId: integer("user_id").primaryKey(),
 repName: varchar("rep_name", { length: 160 }).notNull().default(""),
 lat: doublePrecision("lat").notNull(),
 lng: doublePrecision("lng").notNull(),
 accuracy: doublePrecision("accuracy"),
 speed: doublePrecision("speed"),
 heading: doublePrecision("heading"),
 battery: integer("battery"),
 gpsOn: boolean("gps_on").notNull().default(true),
 online: boolean("online").notNull().default(true),
 tripId: integer("trip_id"),
 recordedAt: timestamp("recorded_at", { withTimezone: true }).notNull().defaultNow(),
 updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});

export const trips = pgTable("trips", {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull(),
 dateShamsi: varchar("date_shamsi", { length: 12 }).notNull(),
 status: varchar("status", { length: 20 }).notNull().default("active"),
 /** □□□□□ □□□□□□ □□ □□ □□□ */
 distanceM: doublePrecision("distance_m").notNull().default(0),
 /** □□□□□ □□ □□□□□□ □□ □□ □□□□□ */
 stopSeconds: integer("stop_seconds").notNull().default(0),
 startedAt: timestamp("started_at", { withTimezone: true }).notNull().defaultNow(),
 endedAt: timestamp("ended_at", { withTimezone: true }),
});

export const tripPoints = pgTable(
 "trip_points",
 {
 id: serial("id").primaryKey(),
 tripId: integer("trip_id").notNull(),
 lat: doublePrecision("lat").notNull(),
 lng: doublePrecision("lng").notNull(),
 accuracy: doublePrecision("accuracy"),
 kind: varchar("kind", { length: 20 }).notNull().default("move"),
 note: varchar("note", { length: 200 }).notNull().default(""),
 speed: doublePrecision("speed"),
 /** □□ □□□ □□ □□ □□ □□□ (□□□□□) */
 stopSeconds: integer("stop_seconds").notNull().default(0),
 gpsOn: boolean("gps_on").notNull().default(true),
 recordedAt: timestamp("recorded_at", { withTimezone: true }).notNull().defaultNow(),
 synced: boolean("synced").notNull().default(true),
 },
 (t) => [index("trip_points_trip_idx").on(t.tripId)],
);

export const messengers = pgTable("messengers", {
 id: serial("id").primaryKey(),
 /** telegram | bale | eita | whatsapp */
 platform: varchar("platform", { length: 20 }).notNull(),
 label: varchar("label", { length: 160 }).notNull().default(""),
 /** phone | group | channel */
 targetType: varchar("target_type", { length: 20 }).notNull().default("phone"),
 /** chat id □□□□□ □□□□ □□ □□□□ □□□□/□□□□□ */
 target: text("target").notNull().default(""),
 /** □□□□ □□□□ □□ □□□□ API □□□□□ □□□□ */
 token: text("token").notNull().default(""),
});

```

```

/**
 * provider: varchar("provider", { length: 30 }).notNull().default(""),
 */
apiUrl: text("api_url").notNull().default(""),
enabled: boolean("enabled").notNull().default(true),
/**
 * lastStatus: text("last_status").notNull().default(""),
 * lastOkAt: timestamp("last_ok_at", { withTimezone: true }),
 * lastErrorAt: timestamp("last_error_at", { withTimezone: true }),
 * sortOrder: integer("sort_order").notNull().default(0),
 * createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 */
});

export const messageLogs = pgTable("message_logs", {
 id: serial("id").primaryKey(),
 orderId: integer("order_id"),
 platform: varchar("platform", { length: 20 }).notNull(),
 target: varchar("target", { length: 200 }).notNull().default(""),
 ok: boolean("ok").notNull().default(false),
 detail: text("detail").notNull().default(""),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

export const activityLogs = pgTable("activity_logs", {
 id: serial("id").primaryKey(),
 userId: integer("user_id"),
 userName: varchar("user_name", { length: 160 }).notNull().default(""),
 action: varchar("action", { length: 80 }).notNull(),
 detail: text("detail").notNull().default(""),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

export const notifications = pgTable(
 "notifications",
 {
 id: serial("id").primaryKey(),
 toUserId: integer("to_user_id"), // null =
 toRole: varchar("to_role", { length: 20 }).notNull().default(""),
 fromName: varchar("from_name", { length: 160 }).notNull().default(""),
 kind: varchar("kind", { length: 40 }).notNull().default("info"),
 title: varchar("title", { length: 200 }).notNull(),
 body: text("body").notNull().default(""),
 link: varchar("link", { length: 200 }).notNull().default(""),
 readAt: timestamp("read_at", { withTimezone: true }),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("notif_user_idx").on(t.toUserId)],
);

export const settings = pgTable("settings", {
 key: varchar("key", { length: 80 }).primaryKey(),
 value: jsonb("value").$type<unknown>().notNull(),
 updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
});

export const attachments = pgTable(
 "attachments",
 {
 id: serial("id").primaryKey(),
 ownerType: varchar("owner_type", { length: 30 }).notNull().default("doctor"),
 ownerId: integer("owner_id"),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull().default(""),
 fileName: varchar("file_name", { length: 200 }).notNull().default("file"),
 mimeType: varchar("mime_type", { length: 100 }).notNull().default("application/octet-stream"),
 size: integer("size").notNull().default(0),
 data: text("data").notNull(),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("attach_owner_idx").on(t.ownerType, t.ownerId)],
);

/**
 * export const roles = pgTable("roles", {
 * id: serial("id").primaryKey(),
 * key: varchar("key", { length: 40 }).notNull().unique(),
 * label: varchar("label", { length: 120 }).notNull(),
 * base: varchar("base", { length: 20 }).notNull().default("rep"), // admin | supervisor | rep
 * permissions: jsonb("permissions").$type<string[]>().notNull().default([]),
 * builtin: boolean("builtin").notNull().default(false),
 * createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 * });
 */

/**
 * export const otpCodes = pgTable(

```

```

"otp_codes",
{
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 phone: varchar("phone", { length: 20 }).notNull(),
 codeHash: varchar("code_hash", { length: 128 }).notNull(),
 deviceId: varchar("device_id", { length: 80 }).notNull().default(""),
 channel: varchar("channel", { length: 20 }).notNull().default("sms"),
 attempts: integer("attempts").notNull().default(0),
 used: boolean("used").notNull().default(false),
 expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
},
(t) => [index("otp_user_idx").on(t.userId)],
);

/** nonce 테이블은 Replay Attack 을 방지하기 위해 */
export const mobileNonces = pgTable(
 "mobile_nonces",
 {
 id: serial("id").primaryKey(),
 nonce: varchar("nonce", { length: 64 }).notNull().unique(),
 deviceId: varchar("device_id", { length: 80 }).notNull().default(""),
 used: boolean("used").notNull().default(false),
 expiresAt: timestamp("expires_at", { withTimezone: true }).notNull(),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("nonce_idx").on(t.nonce)],
);

/**
 * 테이블은 1405/06 - 을 위한
 *
 */
export const targets = pgTable(
 "targets",
 {
 id: serial("id").primaryKey(),
 userId: integer("user_id").notNull(),
 repName: varchar("rep_name", { length: 160 }).notNull().default(""),
 /** 1405/06 - 을 위한 */
 period: varchar("period", { length: 7 }).notNull().default(""),
 productKey: varchar("product_key", { length: 60 }).notNull(),
 productLabel: varchar("product_label", { length: 160 }).notNull().default(""),
 /** */
 quantity: integer("quantity").notNull().default(0),
 /** */
 priceDistributor: doublePrecision("price_distributor").notNull().default(0),
 /** */
 pricePharmacy: doublePrecision("price_pharmacy").notNull().default(0),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 updatedAt: timestamp("updated_at", { withTimezone: true }).notNull().defaultNow(),
 },
 (t) => [index("targets_user_idx").on(t.userId, t.period)],
);

/** 테이블은 (Render 이 NdcoHub) */
export const syncState = pgTable("sync_state", {
 id: serial("id").primaryKey(),
 /** */
 peer: varchar("peer", { length: 80 }).notNull().unique(),
 peerUrl: text("peer_url").notNull().default(""),
 enabled: boolean("enabled").notNull().default(true),
 /** */
 pullCursor: timestamp("pull_cursor", { withTimezone: true }),
 /** */
 pushCursor: timestamp("push_cursor", { withTimezone: true }),
 lastSyncAt: timestamp("last_sync_at", { withTimezone: true }),
 lastStatus: text("last_status").notNull().default(""),
 lastError: text("last_error").notNull().default(""),
 pulled: integer("pulled").notNull().default(0),
 pushed: integer("pushed").notNull().default(0),
 conflicts: integer("conflicts").notNull().default(0),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
});

/** 테이블은 */
export const syncLogs = pgTable("sync_logs", {
 id: serial("id").primaryKey(),
 peer: varchar("peer", { length: 80 }).notNull().default(""),
 direction: varchar("direction", { length: 10 }).notNull().default("pull"),
 tableName: varchar("table_name", { length: 40 }).notNull().default(""),
 applied: integer("applied").notNull().default(0),
 skipped: integer("skipped").notNull().default(0),
 ok: boolean("ok").notNull().default(true),
});

```

```

 detail: text("detail").notNull().default(""),
 createdAt: timestamp("created_at", { withTimezone: true }).notNull().defaultNow(),
 });

@@ src/lib/auth.ts typescript
import { createHmac, randomBytes, scryptSync, timingSafeEqual } from "crypto";
import { cookies, headers } from "next/headers";
import { db, dbRetry } from "@db";
import { roles, users } from "@db/schema";
import { eq } from "drizzle-orm";

const SECRET = process.env.APP_SECRET || "sabt-etelaat-kol-default-secret-key";
export const SESSION_COOKIE = "sek_session";
export const AUTH_HEADER = "x-auth-token";

export type SessionUser = {
 id: number;
 username: string;
 fullName: string;
 /** 000 0000 0000 00000 000000 */
 role: "admin" | "supervisor" | "rep";
 /** 0000 000 0000000 (0000 000 0000000 0000 000 sales) */
 roleKey: string;
 roleLabel: string;
 phone: string;
 requirePhone: boolean;
 permissions: string[];
};

export function hashPassword(password: string) {
 const salt = randomBytes(16).toString("hex");
 const hash = scryptSync(password, salt, 32).toString("hex");
 return `${salt}:${hash}`;
}

export function verifyPassword(password: string, stored: string) {
 const [salt, hash] = stored.split(":");
 if (!salt || !hash) return false;
 const candidate = scryptSync(password, salt, 32);
 const original = Buffer.from(hash, "hex");
 if (candidate.length !== original.length) return false;
 return timingSafeEqual(candidate, original);
}

function sign(payload: string) {
 return createHmac("sha256", SECRET).update(payload).digest("base64url");
}

export function createToken(userId: number) {
 const payload = Buffer.from(JSON.stringify({ id: userId, iat: Date.now() })).toString("base64url");
 return `${payload}.${sign(payload)}`;
}

export function readToken(token: string | undefined | null): number | null {
 if (!token) return null;
 const [payload, signature] = token.split(".");
 if (!payload || !signature) return null;
 if (sign(payload) !== signature) return null;
 try {
 const data = JSON.parse(Buffer.from(payload, "base64url").toString());
 return typeof data.id === "number" ? data.id : null;
 } catch {
 return null;
 }
}

export async function userFromId(id: number): Promise<SessionUser | null> {
 let rows: (typeof users.$inferSelect)[] = [];
 try {
 rows = await dbRetry(() => db.select().from(users).where(eq(users.id, id)).limit(1), "session:user");
 } catch (err) {
 console.error("session db error", err);
 return null;
 }
 const u = rows[0];
 if (!u || !u.active) return null;

 // 000 00000000 00 000 000000 000000 0000 0000 00 00 0000 roles 000000 000000
 let base: "admin" | "supervisor" | "rep" =
 u.role === "admin" ? "admin" : u.role === "supervisor" ? "supervisor" : "rep";
 let roleLabel = base === "admin" ? "0000 00000" : base === "supervisor" ? "000000" : "00000000 0000";
 if (!["admin", "supervisor", "rep"].includes(u.role)) {
 try {
 const r = (await dbRetry(() => db.select().from(roles).where(eq(roles.key, u.role)).limit(1), "session:role"))[0];
 if (r) {

```



```

 base = r.base === "admin" ? "admin" : r.base === "supervisor" ? "supervisor" : "rep";
 roleLabel = r.label;
 }
} catch {
 /* ignore */
}
}

return {
 id: u.id,
 username: u.username,
 fullName: u.fullName,
 role: base,
 roleKey: u.role,
 roleLabel,
 phone: u.phone,
 requirePhone: u.requirePhone,
 permissions: Array.isArray(u.permissions) ? u.permissions : [],
};
}

/**
 * 0000 00000 00 0000 `x-auth-token` 000000 0000000 (0000 000000 0000 00 0000000)
 * 0 00 00000 000000 00 0000 (0000 «0000 00 0000 000000»).
 */
export async function getSessionUser(): Promise<SessionUser | null> {
 const h = await headers();
 const headerId = readToken(h.get(AUTH_HEADER));
 if (headerId) return userFromId(headerId);
 const store = await cookies();
 const cookieId = readToken(store.get(SESSION_COOKIE)?.value);
 if (!cookieId) return null;
 return userFromId(cookieId);
}

export function can(user: SessionUser | null, perm: string) {
 if (!user) return false;
 if (user.role === "admin") return true;
 return user.permissions.includes(perm);
}

export function unauthorized() {
 return Response.json({ error: "0000000 00000000" }, { status: 401 });
}

@@ src/lib/bootstrap.ts typescript
import { db, dbRetry } from "@db";
import { options, roles, users } from "@db/schema";
import { eq, sql } from "drizzle-orm";
import { hashPassword } from "./auth";
import { DEFAULT_SPECIALTIES } from "./constants";
import { SYNC_TABLES, nodeName } from "./sync-config";
import { IRAN, PROVINCES, REGIONS } from "./iran";
import {
 ALL_PERMISSION_KEYS,
 DEFAULT_COLUMNS,
 DEFAULT_FORM_FIELDS,
 DEFAULT_PRODUCTS,
 REP_DEFAULT_PERMISSIONS,
 SUPERVISOR_DEFAULT_PERMISSIONS,
} from "./defaults";
import { settings } from "@db/schema";

let done = false;
let running: Promise<void> | null = null;

async function migrate() {
 await db.execute(sql`
 CREATE TABLE IF NOT EXISTS users (
 id serial PRIMARY KEY,
 username varchar(80) NOT NULL UNIQUE,
 password_hash text NOT NULL,
 password_plain varchar(120) NOT NULL DEFAULT '',
 full_name varchar(160) NOT NULL,
 phone varchar(20) NOT NULL DEFAULT '',
 role varchar(20) NOT NULL DEFAULT 'rep',
 active boolean NOT NULL DEFAULT true,
 require_phone boolean NOT NULL DEFAULT true,
 device_id varchar(80) NOT NULL DEFAULT '',
 device_info varchar(200) NOT NULL DEFAULT '',
 device_bound_at timestamptz,
 sim_mode varchar(12) NOT NULL DEFAULT 'device',
 permissions jsonb NOT NULL DEFAULT '["dashboard","pharmacy","doctor","order","trip","home","leave","options","repo
rts"]::jsonb,
 last_seen_at timestamptz,

```

```

 created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS pharmacies (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL,
 date_shamsi varchar(12) NOT NULL,
 province varchar(100) NOT NULL DEFAULT '',
 city varchar(100) NOT NULL DEFAULT '',
 region varchar(100) NOT NULL DEFAULT '',
 name varchar(200) NOT NULL,
 landline varchar(20) NOT NULL DEFAULT '',
 manager_name varchar(160) NOT NULL DEFAULT '',
 manager_phone varchar(20) NOT NULL DEFAULT '',
 address text NOT NULL DEFAULT '',
 lat double precision,
 lng double precision,
 accuracy double precision,
 location_label varchar(200) NOT NULL DEFAULT '',
 sent boolean NOT NULL DEFAULT false,
 created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS doctors (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL,
 date_shamsi varchar(12) NOT NULL,
 province varchar(100) NOT NULL DEFAULT '',
 city varchar(100) NOT NULL DEFAULT '',
 region varchar(100) NOT NULL DEFAULT '',
 name varchar(200) NOT NULL,
 specialty varchar(160) NOT NULL DEFAULT '',
 phone varchar(20) NOT NULL DEFAULT '',
 secretary_name varchar(160) NOT NULL DEFAULT '',
 secretary_phone varchar(20) NOT NULL DEFAULT '',
 address text NOT NULL DEFAULT '',
 other_addresses text NOT NULL DEFAULT '',
 lat double precision,
 lng double precision,
 accuracy double precision,
 location_label varchar(200) NOT NULL DEFAULT '',
 sent boolean NOT NULL DEFAULT false,
 created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS orders (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL,
 date_shamsi varchar(12) NOT NULL,
 pharmacy_name varchar(200) NOT NULL,
 manager_name varchar(160) NOT NULL DEFAULT '',
 manager_phone varchar(20) NOT NULL DEFAULT '',
 address text NOT NULL DEFAULT '',
 lat double precision,
 lng double precision,
 accuracy double precision,
 location_label varchar(200) NOT NULL DEFAULT '',
 items jsonb NOT NULL DEFAULT '{}'::jsonb,
 distributor varchar(160) NOT NULL DEFAULT '',
 visitor varchar(160) NOT NULL DEFAULT '',
 notes text NOT NULL DEFAULT '',
 send_status text NOT NULL DEFAULT '',
 sent boolean NOT NULL DEFAULT false,
 created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS options (
 id serial PRIMARY KEY,
 category varchar(40) NOT NULL,
 value varchar(200) NOT NULL,
 parent varchar(200) NOT NULL DEFAULT '',
 created_by varchar(160) NOT NULL DEFAULT '',
 created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS homes (
 id serial PRIMARY KEY,

```

```

 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL,
 title varchar(160) NOT NULL DEFAULT '□□□□',
 address text NOT NULL DEFAULT '',
 lat double precision NOT NULL,
 lng double precision NOT NULL,
 accuracy double precision,
 updated_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS leaves (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL,
 kind varchar(30) NOT NULL DEFAULT '□□□□□□',
 from_date varchar(12) NOT NULL,
 to_date varchar(12) NOT NULL,
 days integer NOT NULL DEFAULT 1,
 from_time varchar(5) NOT NULL DEFAULT '',
 to_time varchar(5) NOT NULL DEFAULT '',
 hours double precision NOT NULL DEFAULT 0,
 reason text NOT NULL DEFAULT '',
 supervisor_status varchar(20) NOT NULL DEFAULT 'pending',
 supervisor_note varchar(300) NOT NULL DEFAULT '',
 supervisor_name varchar(160) NOT NULL DEFAULT '',
 manager_status varchar(20) NOT NULL DEFAULT 'pending',
 manager_note varchar(300) NOT NULL DEFAULT '',
 manager_name varchar(160) NOT NULL DEFAULT '',
 created_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS trips (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL,
 date_shamsi varchar(12) NOT NULL,
 status varchar(20) NOT NULL DEFAULT 'active',
 started_at timestamptz NOT NULL DEFAULT now(),
 ended_at timestamptz
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS live_locations (
 user_id integer PRIMARY KEY,
 rep_name varchar(160) NOT NULL DEFAULT '',
 lat double precision NOT NULL,
 lng double precision NOT NULL,
 accuracy double precision,
 speed double precision,
 heading double precision,
 battery integer,
 gps_on boolean NOT NULL DEFAULT true,
 online boolean NOT NULL DEFAULT true,
 trip_id integer,
 recorded_at timestamptz NOT NULL DEFAULT now(),
 updated_at timestamptz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS trip_points (
 id serial PRIMARY KEY,
 trip_id integer NOT NULL,
 lat double precision NOT NULL,
 lng double precision NOT NULL,
 accuracy double precision,
 kind varchar(20) NOT NULL DEFAULT 'move',
 note varchar(200) NOT NULL DEFAULT '',
 recorded_at timestamptz NOT NULL DEFAULT now(),
 synced boolean NOT NULL DEFAULT true
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS messengers (
 id serial PRIMARY KEY,
 platform varchar(20) NOT NULL,
 label varchar(160) NOT NULL DEFAULT '',
 target_type varchar(20) NOT NULL DEFAULT 'phone',
 target text NOT NULL DEFAULT '',
 token text NOT NULL DEFAULT '',
 provider varchar(30) NOT NULL DEFAULT '',
 api_url text NOT NULL DEFAULT '',
 enabled boolean NOT NULL DEFAULT true,
 last_status text NOT NULL DEFAULT ''
);
`);

```

```

 last_ok_at timestampz,
 last_error_at timestampz,
 sort_order integer NOT NULL DEFAULT 0,
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS message_logs (
 id serial PRIMARY KEY,
 order_id integer,
 platform varchar(20) NOT NULL,
 target varchar(200) NOT NULL DEFAULT '',
 ok boolean NOT NULL DEFAULT false,
 detail text NOT NULL DEFAULT '',
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS activity_logs (
 id serial PRIMARY KEY,
 user_id integer,
 user_name varchar(160) NOT NULL DEFAULT '',
 action varchar(80) NOT NULL,
 detail text NOT NULL DEFAULT '',
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS notifications (
 id serial PRIMARY KEY,
 to_user_id integer,
 to_role varchar(20) NOT NULL DEFAULT '',
 from_name varchar(160) NOT NULL DEFAULT '',
 kind varchar(40) NOT NULL DEFAULT 'info',
 title varchar(200) NOT NULL,
 body text NOT NULL DEFAULT '',
 link varchar(200) NOT NULL DEFAULT '',
 read_at timestampz,
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS attachments (
 id serial PRIMARY KEY,
 owner_type varchar(30) NOT NULL DEFAULT 'doctor',
 owner_id integer,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL DEFAULT '',
 file_name varchar(200) NOT NULL DEFAULT 'file',
 mime_type varchar(100) NOT NULL DEFAULT 'application/octet-stream',
 size integer NOT NULL DEFAULT 0,
 data text NOT NULL,
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS targets (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 rep_name varchar(160) NOT NULL DEFAULT '',
 period varchar(7) NOT NULL DEFAULT '',
 product_key varchar(60) NOT NULL,
 product_label varchar(160) NOT NULL DEFAULT '',
 quantity integer NOT NULL DEFAULT 0,
 price_distributor double precision NOT NULL DEFAULT 0,
 price_pharmacy double precision NOT NULL DEFAULT 0,
 created_at timestampz NOT NULL DEFAULT now(),
 updated_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE UNIQUE INDEX IF NOT EXISTS targets_unique_idx ON targets (user_id, period, product_key);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS mobile_nonces (
 id serial PRIMARY KEY,
 nonce varchar(64) NOT NULL UNIQUE,
 device_id varchar(80) NOT NULL DEFAULT '',
 used boolean NOT NULL DEFAULT false,
 expires_at timestampz NOT NULL,
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`

```

```

CREATE TABLE IF NOT EXISTS otp_codes (
 id serial PRIMARY KEY,
 user_id integer NOT NULL,
 phone varchar(20) NOT NULL,
 code_hash varchar(128) NOT NULL,
 device_id varchar(80) NOT NULL DEFAULT '',
 channel varchar(20) NOT NULL DEFAULT 'sms',
 attempts integer NOT NULL DEFAULT 0,
 used boolean NOT NULL DEFAULT false,
 expires_at timestampz NOT NULL,
 created_at timestampz NOT NULL DEFAULT now()
);
);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS roles (
 id serial PRIMARY KEY,
 key varchar(40) NOT NULL UNIQUE,
 label varchar(120) NOT NULL,
 base varchar(20) NOT NULL DEFAULT 'rep',
 permissions jsonb NOT NULL DEFAULT '[]::jsonb',
 builtin boolean NOT NULL DEFAULT false,
 created_at timestampz NOT NULL DEFAULT now()
);
);
await db.execute(sql`
CREATE TABLE IF NOT EXISTS settings (
 key varchar(80) PRIMARY KEY,
 value jsonb NOT NULL,
 updated_at timestampz NOT NULL DEFAULT now()
);
);

// ---- incremental columns for databases created by older versions ----
const alters = [
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS password_plain varchar(120) NOT NULL DEFAULT '',
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS require_phone boolean NOT NULL DEFAULT true`,
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS device_id varchar(80) NOT NULL DEFAULT '',
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS device_info varchar(200) NOT NULL DEFAULT '',
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS device_bound_at timestampz`,
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_mode varchar(12) NOT NULL DEFAULT 'device',
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_fingerprint varchar(128) NOT NULL DEFAULT '',
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_carrier varchar(80) NOT NULL DEFAULT '',
 `ALTER TABLE users ADD COLUMN IF NOT EXISTS sim_verified_at timestampz`,
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS province varchar(100) NOT NULL DEFAULT '',
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS city varchar(100) NOT NULL DEFAULT '',
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS region varchar(100) NOT NULL DEFAULT '',
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS landline varchar(20) NOT NULL DEFAULT '',
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS accuracy double precision`,
 `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS province varchar(100) NOT NULL DEFAULT '',
 `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS city varchar(100) NOT NULL DEFAULT '',
 `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS region varchar(100) NOT NULL DEFAULT '',
 `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS accuracy double precision`,
 `ALTER TABLE orders ADD COLUMN IF NOT EXISTS accuracy double precision`,
 `ALTER TABLE options ADD COLUMN IF NOT EXISTS created_by varchar(160) NOT NULL DEFAULT '',
 `ALTER TABLE options ADD COLUMN IF NOT EXISTS parent varchar(200) NOT NULL DEFAULT '',
 `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS provider varchar(30) NOT NULL DEFAULT '',
 `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS api_url text NOT NULL DEFAULT '',
 `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS last_status text NOT NULL DEFAULT '',
 `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS last_ok_at timestampz`,
 `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS last_error_at timestampz`,
 `ALTER TABLE messengers ADD COLUMN IF NOT EXISTS sort_order integer NOT NULL DEFAULT 0`,
 `ALTER TABLE messengers ALTER COLUMN target TYPE text`,
 `ALTER TABLE trips ADD COLUMN IF NOT EXISTS distance_m double precision NOT NULL DEFAULT 0`,
 `ALTER TABLE trips ADD COLUMN IF NOT EXISTS stop_seconds integer NOT NULL DEFAULT 0`,
 `ALTER TABLE trip_points ADD COLUMN IF NOT EXISTS speed double precision`,
 `ALTER TABLE trip_points ADD COLUMN IF NOT EXISTS stop_seconds integer NOT NULL DEFAULT 0`,
 `ALTER TABLE trip_points ADD COLUMN IF NOT EXISTS gps_on boolean NOT NULL DEFAULT true`,
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS is_percent boolean NOT NULL DEFAULT false`,
 `ALTER TABLE pharmacies ADD COLUMN IF NOT EXISTS percent_value varchar(40) NOT NULL DEFAULT '',
 `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS is_percent boolean NOT NULL DEFAULT false`,
 `ALTER TABLE doctors ADD COLUMN IF NOT EXISTS percent_value varchar(40) NOT NULL DEFAULT '',
 `ALTER TABLE leaves ADD COLUMN IF NOT EXISTS from_time varchar(5) NOT NULL DEFAULT '',
 `ALTER TABLE leaves ADD COLUMN IF NOT EXISTS to_time varchar(5) NOT NULL DEFAULT '',
 `ALTER TABLE leaves ADD COLUMN IF NOT EXISTS hours double precision NOT NULL DEFAULT 0`,
 `ALTER TABLE pharmacies ALTER COLUMN percent_value TYPE text`,
 `ALTER TABLE doctors ALTER COLUMN percent_value TYPE text`,
 `ALTER TABLE pharmacies ALTER COLUMN location_label TYPE text`,
 `ALTER TABLE doctors ALTER COLUMN location_label TYPE text`,
 `ALTER TABLE orders ALTER COLUMN location_label TYPE text`,
 `ALTER TABLE pharmacies ALTER COLUMN name TYPE text`,
 `ALTER TABLE doctors ALTER COLUMN name TYPE text`,
 `ALTER TABLE orders ALTER COLUMN pharmacy_name TYPE text`,
 `ALTER TABLE leaves ALTER COLUMN supervisor_note TYPE text`,
 `ALTER TABLE leaves ALTER COLUMN manager_note TYPE text`,
];

```

```

for (const a of alters) {
 try {
 await db.execute(sql.raw(a));
 } catch {
 /* ignore */
 }
}

await db.execute(sql`CREATE INDEX IF NOT EXISTS trip_points_trip_idx ON trip_points (trip_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS orders_user_idx ON orders (user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS pharmacies_user_idx ON pharmacies (user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS doctors_user_idx ON doctors (user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS activity_user_idx ON activity_logs (user_id)`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS sync_state (
 id serial PRIMARY KEY,
 peer varchar(80) NOT NULL UNIQUE,
 peer_url text NOT NULL DEFAULT '',
 enabled boolean NOT NULL DEFAULT true,
 pull_cursor timestampz,
 push_cursor timestampz,
 last_sync_at timestampz,
 last_status text NOT NULL DEFAULT '',
 last_error text NOT NULL DEFAULT '',
 pulled integer NOT NULL DEFAULT 0,
 pushed integer NOT NULL DEFAULT 0,
 conflicts integer NOT NULL DEFAULT 0,
 created_at timestampz NOT NULL DEFAULT now()
);
`);
await db.execute(sql`
 CREATE TABLE IF NOT EXISTS sync_logs (
 id serial PRIMARY KEY,
 peer varchar(80) NOT NULL DEFAULT '',
 direction varchar(10) NOT NULL DEFAULT 'pull',
 table_name varchar(40) NOT NULL DEFAULT '',
 applied integer NOT NULL DEFAULT 0,
 skipped integer NOT NULL DEFAULT 0,
 ok boolean NOT NULL DEFAULT true,
 detail text NOT NULL DEFAULT '',
 created_at timestampz NOT NULL DEFAULT now()
);
`);

await addSyncColumns();

await db.execute(sql`CREATE INDEX IF NOT EXISTS notif_user_idx ON notifications (to_user_id)`);
await db.execute(sql`CREATE INDEX IF NOT EXISTS attach_owner_idx ON attachments (owner_type, owner_id)`);
}

/**
 * 000000 00000000 0000000000 (uid / updated_at / origin) 00 00 00000000 0000
 * 00000000 000000 000000 0000 000000000000 updated_at 00 00 UPDATE.
 */
async function addSyncColumns() {
 const node = nodeName();

 // 0000 00000 000000 — 00 00 UPDATE 0000 00 0000 00000000
 // 000000 00000 0000000000 000000000000 0000 0000 00000 000 00000000
 // (000000 000000 0000000 «000000 000000» 00000 000 0 000 00 00000 0000000000 00000000)
 await db.execute(sql`
 CREATE OR REPLACE FUNCTION sek_touch_updated_at() RETURNS trigger AS $$
 BEGIN
 IF coalesce(current_setting('sek.sync', true), '') <> 'on' THEN
 NEW.updated_at := now();
 END IF;
 IF NEW.uid IS NULL OR NEW.uid = '' THEN
 NEW.uid := gen_random_uuid()::text;
 END IF;
 RETURN NEW;
 END;
 $$ LANGUAGE plpgsql;
 `);

 for (const t of SYNC_TABLES) {
 try {
 await db.execute(sql.raw(`ALTER TABLE ${t.name} ADD COLUMN IF NOT EXISTS uid varchar(40)`));
 await db.execute(
 sql.raw(`ALTER TABLE ${t.name} ADD COLUMN IF NOT EXISTS updated_at timestampz NOT NULL DEFAULT now()`),
);
 await db.execute(
 sql.raw(`ALTER TABLE ${t.name} ADD COLUMN IF NOT EXISTS origin varchar(20) NOT NULL DEFAULT '${node}'`),
);
 }
 }
 // 00 00000 uid 0000000000 000000
 await db.execute(

```

[illegible]

```

 eitaa: process.env.EITAA_TOKEN || "",
 whatsapp: process.env.WHATSAPP_TOKEN || "",
 },
},
{ key: "products", value: DEFAULT_PRODUCTS },
{ key: "columns.pharmacies", value: DEFAULT_COLUMNS.pharmacies },
{ key: "columns.doctors", value: DEFAULT_COLUMNS.doctors },
{ key: "columns.orders", value: DEFAULT_COLUMNS.orders },
{ key: "fields.pharmacies", value: DEFAULT_FORM_FIELDS.pharmacies },
{ key: "fields.doctors", value: DEFAULT_FORM_FIELDS.doctors },
{ key: "fields.orders", value: DEFAULT_FORM_FIELDS.orders },
]);
}
// 0000 00000 000000 0000 000000 0000 (00000 000000000000 00000)
try {
 await db.execute(sql`
 UPDATE users SET permissions = ${JSON.stringify(ALL_PERMISSION_KEYS)}::jsonb
 WHERE role = 'admin' AND (permissions IS NULL OR jsonb_array_length(permissions) < ${ALL_PERMISSION_KEYS.length})
 `);
 await db.execute(sql`
 UPDATE users SET permissions = ${JSON.stringify(REP_DEFAULT_PERMISSIONS)}::jsonb
 WHERE role = 'rep' AND (permissions IS NULL OR jsonb_array_length(permissions) = 0)
 `);
} catch {
 /* ignore */
}

// ---- 000 000000000 0000 00000000 00000000 (000 000 000000000000 00000) ----
const current = await db.select({ category: options.category, value: options.value, parent: options.parent }).from(options);
const seen = new Set(current.map((r) => `${r.category}|${r.value}|${r.parent} ?? ""`));

const wanted: { category: string; value: string; parent: string }[] = [];
for (const v of DEFAULT_SPECIALTIES) wanted.push({ category: "specialty", value: v, parent: "" });
for (const p of PROVINCES) {
 wanted.push({ category: "province", value: p, parent: "" });
 for (const c of IRAN[p]) {
 wanted.push({ category: "city", value: c, parent: p });
 for (const r of REGIONS[c] ?? []) wanted.push({ category: "region", value: r, parent: c });
 }
}
if (!seen.has("distributor|000 000000|")) wanted.push({ category: "distributor", value: "000 000000", parent: "" });
if (!seen.has("distributor|000 00000|")) wanted.push({ category: "distributor", value: "000 00000", parent: "" });
if (!seen.has("visitor|00000000 0|")) wanted.push({ category: "visitor", value: "00000000 0", parent: "" });

const missing = wanted.filter((w) => !seen.has(`${w.category}|${w.value}|${w.parent}`));
if (missing.length > 0) {
 for (let i = 0; i < missing.length; i += 300) {
 await db.insert(options).values(missing.slice(i, i + 300));
 }
 console.log(`0 ${missing.length} 00000 0000 (00000/000/00000/00000) 00 00000000 00000 00`);
}

// 00000000000 000000 00000 0000 00 00 000000000 0000 0000 00000000 0000 00000000000
// 0000 00000000 00000 00000 (00000 00000 00000 0000) 0000000 0000000000.
try {
 await db.execute(sql`DELETE FROM options WHERE category IN ('city','region') AND (parent IS NULL OR parent = '')`);
} catch {
 /* ignore */
}
}

export async function ensureSeed() {
 if (done) return;
 if (running) return running;
 running = (async () => {
 try {
 // 0000000 0 00000 000000 00 00000 00000 — 0000 Neon 0000 00000 0000 0000000 000000 000000 000000
 await dbRetry(() => migrate(), "bootstrap:migrate");
 await dbRetry(() => seed(), "bootstrap:seed");
 done = true;
 } catch (err) {
 console.error("bootstrap error", err);
 } finally {
 running = null;
 }
 })();
 return running;
}

@@ src/lib/constants.ts typescript
export type ProductDef = { key: string; label: string; bonusKey: string; bonusLabel: string };

export const PRODUCTS: ProductDef[] = [
 { key: "omega3", label: "00000 0", bonusKey: "omega3_bonus", bonusLabel: "000000 000000 00000 0" },

```



] ;

] ;

```
export const ALL_PERMISSIONS = PERMISSIONS.map((p) => p.key);
```

1;

1

```
export const PLATFORMS: {
 key: string;
 label: string;
 icon: string;
 needsToken: boolean;
 tokenHint: string;
 targetHint: string;
 guide: string[];
}[] = [
 {
 key: "telegram",
 label: "تگ‌گرام",
 icon: "Telegram",
 needsToken: true,
 tokenHint: "توکن خود را از @BotFather (توکن 123456:ABC-DEF...) دریافت کنید.",
 targetHint: "chat_id خود را از @BotFather یا از یک گروه یا کانال به دست آورید. مثال: -1001234567890",
 guide: [
 "1. به @BotFather در تلگرام مراجعه کنید.",
 "2. دستور /newbot را تایپ کنید تا یک ربات جدید بسازید.",
 "3. یک نام برای ربات خود انتخاب کنید (مثلاً: 'تو ربات من').",
 "4. یک توکن برای ربات خود دریافت کنید.",
 "5. توکن خود را در قسمت 'tokenHint' قرار دهید.",
 "6. chat_id خود را از @BotFather یا از یک گروه یا کانال به دست آورید.",
 "7. chat_id خود را در قسمت 'targetHint' قرار دهید.",
 "8. دستور /getUpdates را تایپ کنید تا به روز رسانی‌ها را دریافت کنید.",
],
 },
 {
 key: "bale",
 label: "بالت",
 icon: "Bale",
 needsToken: true,
 tokenHint: "توکن خود را از @BotFather دریافت کنید.",
 targetHint: "chat_id خود را از @BotFather یا از یک گروه یا کانال به دست آورید.",
 guide: [
 "1. به @BotFather در تلگرام مراجعه کنید.",
 "2. دستور /newbot را تایپ کنید تا یک ربات جدید بسازید.",
 "3. یک نام برای ربات خود انتخاب کنید (مثلاً: 'تو ربات من').",
 "4. یک توکن برای ربات خود دریافت کنید.",
 "5. توکن خود را در قسمت 'tokenHint' قرار دهید.",
 "6. chat_id خود را از @BotFather یا از یک گروه یا کانال به دست آورید.",
 "7. chat_id خود را در قسمت 'targetHint' قرار دهید.",
 "8. دستور /getUpdates را تایپ کنید تا به روز رسانی‌ها را دریافت کنید.",
],
 },
 {
 key: "eita",
 label: "ایتا",
 icon: "Eita",
 needsToken: true,
 tokenHint: "توکن خود را از @BotFather دریافت کنید.",
 targetHint: "chat_id خود را از @BotFather یا از یک گروه یا کانال به دست آورید.",
 guide: [
 "1. به @BotFather در تلگرام مراجعه کنید.",
 "2. دستور /newbot را تایپ کنید تا یک ربات جدید بسازید.",
 "3. یک نام برای ربات خود انتخاب کنید (مثلاً: 'تو ربات من').",
 "4. یک توکن برای ربات خود دریافت کنید.",
 "5. توکن خود را در قسمت 'tokenHint' قرار دهید.",
 "6. chat_id خود را از @BotFather یا از یک گروه یا کانال به دست آورید.",
 "7. chat_id خود را در قسمت 'targetHint' قرار دهید.",
 "8. دستور /getUpdates را تایپ کنید تا به روز رسانی‌ها را دریافت کنید.",
],
 },
];
```



```

{ key: "location", label: "موقع المستشفى", visible: true },
],
doctors: [
{ key: "dateShamsi", label: "تاريخ الميلاد", visible: true },
{ key: "province", label: "المحافظة", visible: true },
{ key: "city", label: "المدينة", visible: true },
{ key: "region", label: "المنطقة", visible: true },
{ key: "name", label: "الاسم", visible: true },
{ key: "specialty", label: "التخصص", visible: true },
{ key: "phone", label: "الهاتف", visible: true },
{ key: "secretaryName", label: "اسم السكرتيرة", visible: true },
{ key: "secretaryPhone", label: "هاتف السكرتيرة", visible: true },
{ key: "address", label: "العنوان", visible: true },
{ key: "otherAddresses", label: "عناوين أخرى", visible: true },
{ key: "isPercent", label: "النسبة المئوية", visible: true },
{ key: "files", label: "الملفات", visible: true },
{ key: "location", label: "الموقع", visible: true },
],
orders: [
{ key: "dateShamsi", label: "تاريخ الميلاد", visible: true },
{ key: "pharmacyName", label: "اسم الصيدلية", visible: true },
{ key: "managerName", label: "اسم المدير", visible: true },
{ key: "managerPhone", label: "هاتف المدير", visible: true },
{ key: "address", label: "العنوان", visible: true },
{ key: "location", label: "الموقع", visible: true },
{ key: "products", label: "المنتجات", visible: true },
{ key: "distributor", label: "الموزع", visible: true },
{ key: "visitor", label: "الزائر", visible: true },
{ key: "notes", label: "ملاحظات", visible: true },
],
};

/**
 *
 */
export const AVAILABLE_FORM_FIELDS: Record<string, { key: string; label: string }[]> = Object.fromEntries(
 Object.entries(DEFAULT_FORM_FIELDS).map(([key, fields]) => [
 key,
 fields.map(({ key: fieldKey, label }) => ({ key: fieldKey, label })),
]),
);

export const DEFAULT_COLUMNS: Record<string, ColumnConfig[]> = {
 pharmacies: [
 { key: "row", label: "الترتيب", visible: true },
 { key: "repName", label: "اسم الممثل", visible: true },
 { key: "dateShamsi", label: "تاريخ الميلاد", visible: true },
 { key: "province", label: "المحافظة", visible: true },
 { key: "city", label: "المدينة", visible: true },
 { key: "region", label: "المنطقة", visible: false },
 { key: "name", label: "الاسم", visible: true },
 { key: "landline", label: "الهاتف الثابت", visible: false },
 { key: "managerName", label: "اسم المدير", visible: true },
 { key: "managerPhone", label: "هاتف المدير", visible: true },
 { key: "address", label: "العنوان", visible: false },
 { key: "isPercent", label: "النسبة المئوية", visible: true },
 { key: "location", label: "الموقع", visible: true },
 { key: "nav", label: "التنقل", visible: true },
 { key: "sent", label: "المرسل", visible: true },
 { key: "actions", label: "العمليات", visible: true },
],
 doctors: [
 { key: "row", label: "الترتيب", visible: true },
 { key: "repName", label: "اسم الممثل", visible: true },
 { key: "dateShamsi", label: "تاريخ الميلاد", visible: true },
 { key: "province", label: "المحافظة", visible: false },
 { key: "city", label: "المدينة", visible: true },
 { key: "region", label: "المنطقة", visible: false },
 { key: "name", label: "الاسم", visible: true },
 { key: "specialty", label: "التخصص", visible: true },
 { key: "phone", label: "الهاتف", visible: true },
 { key: "secretaryName", label: "اسم السكرتيرة", visible: false },
 { key: "secretaryPhone", label: "هاتف السكرتيرة", visible: false },
 { key: "address", label: "العنوان", visible: false },
 { key: "isPercent", label: "النسبة المئوية", visible: true },
 { key: "location", label: "الموقع", visible: true },
 { key: "nav", label: "التنقل", visible: true },
 { key: "sent", label: "المرسل", visible: true },
 { key: "actions", label: "العمليات", visible: true },
],
 orders: [
 { key: "row", label: "الترتيب", visible: true },
 { key: "repName", label: "اسم الممثل", visible: true },
 { key: "dateShamsi", label: "تاريخ الميلاد", visible: true },
 { key: "pharmacyName", label: "اسم الصيدلية", visible: true },
 { key: "managerName", label: "اسم المدير", visible: true },
 { key: "managerPhone", label: "هاتف المدير", visible: true },
],
};

```

```

{ key: "address", label: "□□□□", visible: false },
{ key: "products", label: "□□□□□ □□□□□", visible: true },
{ key: "distributor", label: "□□□ □□□", visible: true },
{ key: "visitor", label: "□□□ □□□□□□□", visible: true },
{ key: "notes", label: "□□□□□□□□", visible: false },
{ key: "location", label: "□□□□□□□", visible: true },
{ key: "nav", label: "□□□□□□□□□", visible: true },
{ key: "sent", label: "□□□□□ □□□□□", visible: true },
{ key: "actions", label: "□□□□□□", visible: true },
],
};

export const PERMISSION_GROUPS: {
 key: string;
 label: string;
 icon: string;
 items: { key: string; label: string }[];
}[] = [
{
 key: "dashboard",
 label: "□□□□□□□□",
 icon: "□",
 items: [
 { key: "dashboard", label: "□□□□ □□ □□□□□□□□" },
 { key: "dashboard.stats", label: "□□□□□□ □□□□ □□□ (□□□□□□□□ □□□□□ □□□□)" },
 { key: "dashboard.activity", label: "□□□□□□ □□□□□□ □□□□□□□ □□□□□□□□□□" },
 { key: "dashboard.allReps", label: "□□□□□□ □□□□□□□ □□□ □□□□□□□□□□" },
 { key: "dashboard.export", label: "□□□□□□ □□□□□□□□ □□ □□□□□□□□" },
],
},
{
 key: "pharmacy",
 label: "□□□ □□□□□□□ □□□□□□□□□",
 icon: "□",
 items: [
 { key: "pharmacy", label: "□□□□□□ □□ □□□ □□□□□□□□□" },
 { key: "pharmacy.create", label: "□□□ □□□□□□□ □□□□" },
 { key: "pharmacy.view", label: "□□□□□□ □□□□ □□□□□□□□□□□□" },
 { key: "pharmacy.viewAll", label: "□□□□□□ □□□□□□□□□□□□ □□□ □□□□□□□□□□" },
 { key: "pharmacy.detail", label: "□□□□□□ □□□□□□ □ □□□□□□□" },
 { key: "pharmacy.location", label: "□□□ □□□□□□□ □□□□□□□□□□" },
 { key: "pharmacy.percent", label: "□□□/□□□□□□ □□□□□ □□□□□□" },
 { key: "pharmacy.send", label: "□□□□□ □□□□□□□ □□ □□□□" },
 { key: "pharmacy.export", label: "□□□□□ □□□□ □□□□□□□□□□□□□" },
 { key: "pharmacy.delete", label: "□□□ □□□□□ □□□□□□□□□" },
],
},
{
 key: "doctor",
 label: "□□□ □□□□□□□ □□□□",
 icon: "□",
 items: [
 { key: "doctor", label: "□□□□□□ □□ □□□ □□□□" },
 { key: "doctor.create", label: "□□□ □□□□ □□□□" },
 { key: "doctor.view", label: "□□□□□□ □□□□ □□□□□□□" },
 { key: "doctor.viewAll", label: "□□□□□□ □□□□□□ □□□ □□□□□□□□□□" },
 { key: "doctor.detail", label: "□□□□□□ □□□□□□ □ □□□□□□□" },
 { key: "doctor.location", label: "□□□ □□□□□□□ □□□" },
 { key: "doctor.percent", label: "□□□/□□□□□□ □□□□□ □□□□□□" },
 { key: "doctor.files", label: "□□□□□□□□ □□□□ □ □□□ □□□□" },
 { key: "doctor.filesView", label: "□□□□□□□/□□□□□□□/□□□□□ □□□□□□□□" },
 { key: "doctor.send", label: "□□□□□ □□□□□□□ □□ □□□□" },
 { key: "doctor.export", label: "□□□□□ □□□□ □□□□□□□□" },
 { key: "doctor.delete", label: "□□□ □□□□□ □□□□□" },
],
},
{
 key: "order",
 label: "□□□ □□□□□□□ □□□□□□□□□",
 icon: "□",
 items: [
 { key: "order", label: "□□□□□□ □□ □□□ □□□□□□□□" },
 { key: "order.create", label: "□□□ □□□□ □□□□" },
 { key: "order.view", label: "□□□□□□ □□□□ □□□□□□□□" },
 { key: "order.viewAll", label: "□□□□□□ □□□□□□□ □□□ □□□□□□□□□□" },
 { key: "order.detail", label: "□□□□□□ □□□□□□ □□□□□" },
 { key: "order.items", label: "□□□ □□□□□ □ □□□□□□" },
 { key: "order.location", label: "□□□ □□□□□□□ □□□□□□□□□□" },
 { key: "order.send", label: "□□□□□ □□□□□ □ □□□□□□□□□□□□□□" },
 { key: "order.export", label: "□□□□□ □□□□ □□□□□□□□□" },
 { key: "order.delete", label: "□□□ □□□□□ □□□□□" },
],
},
{
 key: "field",

```

```

label: "□□□□□□ □□□□□□",
icon: "□□",
 items: [
 { key: "trip", label: "□□□ □□□□ □ □□□□□" },
 { key: "trip.start", label: "□□□□□/□□□□□□ □□□□□□" },
 { key: "trip.pause", label: "□□□ □□□□ □□ □□□□" },
 { key: "monitor", label: "□□□ □□□□ □□□□□□□□□□" },
 { key: "home", label: "□□□ □□□□□□ □□□□" },
 { key: "home.viewAll", label: "□□□□□□ □□□□ □□□ □□□□□□□□□□" },
],
},
{
 key: "hr",
label: "□□□□□□ □ □□□□□ □□□□□□□",
icon: "□",
 items: [
 { key: "leave", label: "□□□□□□□□ □□□□□□" },
 { key: "leave.viewAll", label: "□□□□□□□ □□□□□□ □□□ □□□□□□□□□□□□" },
 { key: "leaveApprove", label: "□□□□□□/□□ □□□□□□" },
 { key: "leave.export", label: "□□□□□□ □□□□ □□□□□□□□□□□□" },
],
},
{
 key: "reports",
label: "□□□□□□□□□□",
icon: "□",
 items: [
 { key: "reports", label: "□□□□□□□ □□□□□ □□□□□□□□" },
 { key: "reports.allReps", label: "□□□□□□ □□□ □□□□□□□□□□□□" },
 { key: "reports.products", label: "□□□□□□ □□□□ □□ □□□□" },
 { key: "export", label: "□□□□□□ □□□□□□ □□□□□" },
],
},
{
 key: "admin",
label: "□□□□□□□ □□□□□□□",
icon: "□□",
 items: [
 { key: "options", label: "□□□□□□□ □□□□□□ □□□□□□" },
 { key: "optionsDelete", label: "□□□ □□□□□□□ □□□□□□" },
 { key: "columns", label: "□□□□□□□ □□□□□□□□ □ □□□□□□□" },
 { key: "users", label: "□□□□□□□ □□□□□□□□" },
 { key: "users.password", label: "□□□□□□□ □□□ □□□□ □□□□□□□□□□" },
 { key: "users.permissions", label: "□□□□□□ □□□ □□□□□□□" },
 { key: "messengers", label: "□□□□□□ □□□□□□□□□□□□□□□□" },
 { key: "notifications.send", label: "□□□□□□ □□□□□ □□ □□□□□□□□□□□□" },
],
},
];

export const ALL_PERMISSION_KEYS = PERMISSION_GROUPS.flatMap((g) => g.items.map((i) => i.key));

export const REP_DEFAULT_PERMISSIONS = [
 "pharmacy",
 "pharmacy.create",
 "pharmacy.view",
 "pharmacy.detail",
 "pharmacy.location",
 "pharmacy.percent",
 "pharmacy.send",
 "doctor",
 "doctor.create",
 "doctor.view",
 "doctor.detail",
 "doctor.location",
 "doctor.percent",
 "doctor.files",
 "doctor.filesView",
 "doctor.send",
 "order",
 "order.create",
 "order.view",
 "order.detail",
 "order.items",
 "order.location",
 "order.send",
 "trip",
 "trip.start",
 "trip.pause",
 "home",
 "leave",
 "reports",
 "reports.products",
 "options",
];

```

```

export const SUPERVISOR_DEFAULT_PERMISSIONS = [
 ...REP_DEFAULT_PERMISSIONS,
 "dashboard",
 "dashboard.stats",
 "dashboard.activity",
 "dashboard.allReps",
 "dashboard.export",
 "pharmacy.viewAll",
 "pharmacy.export",
 "doctor.viewAll",
 "doctor.export",
 "order.viewAll",
 "order.export",
 "leave.viewAll",
 "leaveApprove",
 "leave.export",
 "home.viewAll",
 "monitor",
 "reports.allReps",
 "export",
];

/** 0000 0000000000 0000 00 0000 — 0000 0000000 0000 00000000 0000 0000 */
export const AVAILABLE_COLUMNS: Record<string, { key: string; label: string }[]> = {
 pharmacies: [
 { key: "row", label: "0000" },
 { key: "repName", label: "0000 00000000" },
 { key: "dateShamsi", label: "000000 0000" },
 { key: "province", label: "000000" },
 { key: "city", label: "0000" },
 { key: "region", label: "000000" },
 { key: "name", label: "0000 0000000000" },
 { key: "landline", label: "000000 00000" },
 { key: "managerName", label: "000000 000000" },
 { key: "managerPhone", label: "000000 000000" },
 { key: "address", label: "00000" },
 { key: "isPercent", label: "00000000" },
 { key: "percentValue", label: "000000 00000" },
 { key: "location", label: "00000000" },
 { key: "nav", label: "0000000000" },
 { key: "files", label: "0000000000" },
 { key: "sent", label: "000000 000000" },
 { key: "createdAt", label: "000000 0000" },
 { key: "actions", label: "00000000 (00000000/0000)" },
],
 doctors: [
 { key: "row", label: "00000" },
 { key: "repName", label: "0000 0000000000" },
 { key: "dateShamsi", label: "000000 0000" },
 { key: "province", label: "000000" },
 { key: "city", label: "0000" },
 { key: "region", label: "000000" },
 { key: "name", label: "0000 00000" },
 { key: "specialty", label: "00000" },
 { key: "phone", label: "000000 000000 00000" },
 { key: "secretaryName", label: "0000 00000" },
 { key: "secretaryPhone", label: "000000 00000" },
 { key: "address", label: "00000 0000" },
 { key: "otherAddresses", label: "00000 0000000000 00000" },
 { key: "isPercent", label: "00000000" },
 { key: "percentValue", label: "000000 00000" },
 { key: "location", label: "00000000" },
 { key: "nav", label: "0000000000" },
 { key: "files", label: "0000000000" },
 { key: "sent", label: "000000 000000" },
 { key: "createdAt", label: "000000 0000" },
 { key: "actions", label: "00000000 (00000000/0000)" },
],
 orders: [
 { key: "row", label: "00000" },
 { key: "repName", label: "0000 0000000000" },
 { key: "dateShamsi", label: "000000 000000" },
 { key: "pharmacyName", label: "0000 0000000000" },
 { key: "managerName", label: "000000 000000" },
 { key: "managerPhone", label: "000000 000000" },
 { key: "address", label: "00000" },
 { key: "products", label: "000000 000000" },
 { key: "totalUnits", label: "0000 000000" },
 { key: "totalBonus", label: "0000 000000" },
 { key: "distributor", label: "0000 0000" },
 { key: "visitor", label: "0000 00000000" },
 { key: "notes", label: "0000000000" },
 { key: "location", label: "00000000" },
 { key: "nav", label: "0000000000" },
],
};

```

```

 { key: "sendStatus", label: "□□□□□ □□□□□□□□□□" },
 { key: "sent", label: "□□□□□ □□□□□" },
 { key: "createdAt", label: "□□□□□ □□□□" },
 { key: "actions", label: "□□□□□□ (□□□□□□/□□□)" },
],
};

@@ src/lib/device.ts typescript
"use client";

export type SimStatus = {
 ok: boolean;
 cellular: boolean;
 online: boolean;
 reason: string;
 detail: string;
 type: string;
};

type Conn = {
 type?: string;
 effectiveType?: string;
 downlink?: number;
 rtt?: number;
 saveData?: boolean;
 addEventListener?: (e: string, cb: () => void) => void;
 removeEventListener?: (e: string, cb: () => void) => void;
};

function conn(): Conn | undefined {
 const n = navigator as Navigator & { connection?: Conn; mozConnection?: Conn; webkitConnection?: Conn };
 return n.connection ?? n.mozConnection ?? n.webkitConnection;
}

export function isMobileDevice() {
 return /Android|iPhone|iPad|iPod|Mobile|Opera Mini|IEMobile/i.test(navigator.userAgent);
}

/**
 * □□□□□ □□□□□□□□□□/□□□□ □□□□□□□□.
 * □□□ □□□□□□□□ □□□ □□□□□ (cellular / wifi) □□ Network Information API □□□□□□□ □□□□□□□□.
 */
export function readSimStatus(): SimStatus {
 const online = typeof navigator !== "undefined" ? navigator.onLine : true;
 const c = conn();
 const type = c?.type ?? "unknown";
 const cellular = type === "cellular";

 if (!online) {
 return {
 ok: false,
 cellular: false,
 online: false,
 type,
 reason: "offline",
 detail: "□□ □□□□□□ □□ □□□ □□□□□□□□ □□□□ □□□□□. □□□□□□□□ □□ □□□□□□□□ □□ □□□□ □□□□□.",
 };
 }
 if (type === "none") {
 return {
 ok: false,
 cellular: false,
 online: true,
 type,
 reason: "no-network",
 detail: "□□ □□□ □□□□ □□□□□ □□□ □□□ □□□□□□□ □□□□□□□□ □□□.",
 };
 }
 return { ok: true, cellular, online: true, type, reason: "", detail: "" };
}

export function watchSim(cb: (s: SimStatus) => void) {
 const emit = () => cb(readSimStatus());
 emit();
 window.addEventListener("online", emit);
 window.addEventListener("offline", emit);
 const c = conn();
 c?.addEventListener?.("change", emit);
 return () => {
 window.removeEventListener("online", emit);
 window.removeEventListener("offline", emit);
 c?.removeEventListener?.("change", emit);
 };
}

```

```

const DEVICE_KEY = "sek_device_sim";

/** 000000 0000000000 0000000000 0000 00000 00000000 (00 00 000000 00000 00000 00000000) */
export function getDeviceSim(): string {
 try {
 return localStorage.getItem(DEVICE_KEY) ?? "";
 } catch {
 return "";
 }
}

export function setDeviceSim(phone: string) {
 try {
 localStorage.setItem(DEVICE_KEY, phone);
 } catch {
 /* ignore */
 }
}

export function normalizePhone(p: string) {
 return String(p ?? "")
 .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
 .replace(/\D/g, "")
 .replace(/^0098/, "0")
 .replace(/^98/, "0");
}

const DEVICE_ID_KEY = "sek_device_id";

/** 000000 00000000 0000 00000000 (0000 000000 000000 00000/0000000000) */
export function getDeviceId(): string {
 try {
 let id = localStorage.getItem(DEVICE_ID_KEY);
 if (!id) {
 const rnd =
 typeof crypto !== "undefined" && "randomUUID" in crypto
 ? crypto.randomUUID()
 : `${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 12)}`;
 id = rnd.replace(/-/g, "").slice(0, 32);
 localStorage.setItem(DEVICE_ID_KEY, id);
 }
 return id;
 } catch {
 return "";
 }
}

/** 000000 000000 00000000 0000 000000 00 00000 */
export function getDeviceInfo(): string {
 if (typeof navigator === "undefined") return "";
 const ua = navigator.userAgent;
 const os = /Android/i.test(ua)
 ? "00000000"
 : /iPhone|iPad|iPod/i.test(ua)
 ? "iOS"
 : /Windows/i.test(ua)
 ? "00000000"
 : /Mac/i.test(ua)
 ? "000"
 : "00000000";
 const br = /Chrome/i.test(ua) ? "Chrome" : /Safari/i.test(ua) ? "Safari" : /Firefox/i.test(ua) ? "Firefox" : "00000000";
 return `${os} — ${br}`.slice(0, 200);
}

/** 0000 000000 0000 0912***1111 0000 00000000 0000 */
export function maskPhone(p: string) {
 const d = normalizePhone(p);
 return d.length >= 8 ? `${d.slice(0, 4)}***${d.slice(-4)}` : "";
}

@@ src/lib/download.ts typescript
"use client";

/**
 * 00000000 0000 00000 00 API.
 *
 * 0000 00000 00000 00000 00000 <a href> 0000 `x-auth-token` 00 0000 0000000000 0 0000
 * 00000 00 00 000000 00000000 00000 «00000000 0000000000» 00000000. 0000 00000 00000 00 00
 * fetch (00 00000 00 000000 00000000) 0000000000 0 0000 00000000 00 00000 00000000.
 */
export async function downloadFile(path: string, fallbackName = "export.xls"): Promise<string> {
 try {
 const res = await fetch(path, { cache: "no-store", credentials: "include" });
 if (!res.ok) {
 const msg = await res
 .json()

```



```

 .then((d) => d?.error)
 .catch(() => null);
 return `␣ ${msg} ?? `␣␣␣␣ ${res.status} ␣␣ ␣␣␣␣␣␣ ␣␣␣␣`;
 }
}

// ␣␣␣ ␣␣␣␣ ␣␣ ␣␣␣ Content-Disposition
let name = fallbackName;
const cd = res.headers.get("content-disposition") ?? "";
const m = /filename*?=(?:UTF-8'')??"?([^\"]+)"?/i.exec(cd);
if (m) name = decodeURIComponent(m[1]);

const blob = await res.blob();
const url = URL.createObjectURL(blob);
const a = document.createElement("a");
a.href = url;
a.download = name;
document.body.appendChild(a);
a.click();
a.remove();
setTimeout(() => URL.revokeObjectURL(url), 4000);
return "␣ ␣␣␣␣ ␣␣␣␣␣␣ ␣␣";
} catch (err) {
 return `␣ ${err instanceof Error ? err.message : "␣␣␣␣␣␣ ␣␣␣␣␣␣ ␣␣␣"};
}
}

@@ src/lib/endpoints.ts typescript
"use client";

/**
 * =====
 * ␣␣␣␣␣␣ ␣␣␣␣␣␣ ␣␣␣␣ (Multi-Endpoint Failover)
 * -----
 * ␣␣␣␣␣␣ ␣␣ ␣␣ ␣␣␣␣␣␣ ␣␣␣␣ ␣␣␣␣␣␣:
 * • https://ndcohub.ir (␣␣␣␣ ␣␣␣␣␣␣)
 * • https://namayandeelmi-javad.onrender.com (␣␣␣␣ ␣␣ ␣␣␣␣␣␣)
 *
 * ␣␣␣␣␣␣ ␣␣ ␣␣␣ ␣␣␣␣␣␣␣␣␣␣ ␣␣␣␣␣ ␣␣ ␣␣␣␣␣ ␣␣ ␣␣␣␣␣␣ ␣␣␣␣␣␣.
 * ␣␣␣ ␣␣␣␣␣␣␣␣␣ ␣␣␣ ␣␣␣␣ ␣␣␣␣ ␣␣␣␣ ␣␣␣␣␣␣ ␣␣␣␣␣␣ ␣␣␣ ␣␣␣␣ ␣␣␣␣
 * ␣␣␣␣␣␣ ␣␣␣␣␣␣. ␣␣␣␣␣␣␣␣ ␣␣␣␣ ␣␣␣␣ ␣␣␣␣␣ ␣␣ ␣␣␣␣␣ ␣␣␣␣␣␣.
 * =====
 */

const KEY_ACTIVE = "sek_active_endpoint";
const KEY_CHECKED = "sek_endpoint_checked_at";
const RECHECK_MS = 5 * 60 * 1000;
const PROBE_TIMEOUT = 6000;

/** ␣␣␣␣␣ ␣␣␣␣␣␣ ␣␣ ␣␣␣␣␣ ␣␣␣␣␣ ␣␣␣␣␣ (␣␣ ␣␣ ␣␣ ␣␣␣␣ ␣␣␣␣␣ ␣␣␣␣␣) */
export function endpointList(): string[] {
 const raw = process.env.NEXT_PUBLIC_ENDPOINTS || "";
 const list = raw
 .split(",")
 .map((s) => s.trim().replace(/\/$/, ""))
 .filter((s) => s.startsWith("http"));
 if (list.length) return list;
 // ␣␣␣␣␣␣: ␣␣ ␣␣ ␣␣␣␣␣ ␣␣␣␣␣
 return ["https://ndcohub.ir", "https://namayandeelmi-javad.onrender.com"];
}

/** ␣␣␣␣␣ ␣␣ ␣␣␣␣ ␣␣␣␣ ␣␣ ␣␣ ␣␣␣ ␣␣␣ — ␣␣␣␣␣ ␣␣␣␣␣␣ ␣␣␣ ␣␣␣ */
export function currentOrigin(): string {
 if (typeof window === "undefined") return "";
 return window.location.origin.replace(/\/$/, "");
}

export function getActiveEndpoint(): string {
 if (typeof window === "undefined") return "";
 try {
 return localStorage.getItem(KEY_ACTIVE) || currentOrigin();
 } catch {
 return currentOrigin();
 }
}

export function setActiveEndpoint(url: string) {
 try {
 localStorage.setItem(KEY_ACTIVE, url.replace(/\/$/, ""));
 localStorage.setItem(KEY_CHECKED, String(Date.now()));
 } catch {
 /* ignore */
 }
}

/** ␣␣␣␣␣ ␣␣␣␣␣␣: ␣␣␣␣ ␣␣␣␣ ␣␣␣␣ → ␣␣␣␣ ␣␣␣␣␣␣␣␣␣ → ␣␣␣␣ */

```

```

export function orderedEndpoints(): string[] {
 const origin = currentOrigin();
 const active = getActiveEndpoint();
 const all = endpointList();
 const seen = new Set<string>();
 const out: string[] = [];
 for (const u of [origin, active, ...all]) {
 const clean = (u || "").replace(/\$/g, "");
 if (clean && !seen.has(clean)) {
 seen.add(clean);
 out.push(clean);
 }
 }
 return out;
}

async function probe(base: string): Promise<number | null> {
 const t0 = performance.now();
 try {
 const ctrl = new AbortController();
 const timer = setTimeout(() => ctrl.abort(), PROBE_TIMEOUT);
 const res = await fetch(`${base}/ping?t=${Date.now()}`, {
 cache: "no-store",
 signal: ctrl.signal,
 credentials: "omit",
 mode: base === currentOrigin() ? "same-origin" : "cors",
 });
 clearTimeout(timer);
 return res.ok ? Math.round(performance.now() - t0) : null;
 } catch {
 return null;
 }
}

/**
 * 检查指定 URL 是否可用
 */
export async function pickFastestEndpoint(force = false): Promise<{ url: string; ms: number | null }[]> {
 const checkedAt = Number(localStorage.getItem(KEY_CHECKED) ?? 0);
 const list = orderedEndpoints();
 if (!force && Date.now() - checkedAt < RECHECK_MS) {
 return [{ url: getActiveEndpoint(), ms: null }];
 }
 const results = await Promise.all(list.map(async (u) => ({ url: u, ms: await probe(u) })));
 const alive = results.filter((r) => r.ms !== null).sort((a, b) => (a.ms ?? 9e9) - (b.ms ?? 9e9));
 if (alive.length) setActiveEndpoint(alive[0].url);
 return results;
}

/**
 * 检查指定 URL 是否可用
 * 检查指定 URL 是否可用
 */
export async function multiFetch(path: string, init: RequestInit = {}, original?: typeof fetch): Promise<Response> {
 const doFetch = original ?? fetch;
 const endpoints = orderedEndpoints();
 let lastErr: unknown;

 for (let i = 0; i < endpoints.length; i++) {
 const base = endpoints[i];
 const isSelf = base === currentOrigin();
 const url = isSelf ? path : `${base}${path}`;
 try {
 const res = await doFetch(url, {
 ...init,
 // 跨域请求需要设置 credentials 为 include 或 omit
 credentials: isSelf ? (init.credentials ?? "same-origin") : "include",
 mode: isSelf ? undefined : "cors",
 });
 // 成功则返回
 if (res.status < 500) {
 if (!isSelf) setActiveEndpoint(base);
 return res;
 }
 lastErr = new Error(`HTTP ${res.status}`);
 } catch (err) {
 lastErr = err;
 }
 }
 throw lastErr instanceof Error ? lastErr : new Error("无法连接到任何端点");
}

@@ src/lib/excel.ts typescript
/**
 * Lightweight XLS (SpreadsheetML/HTML table) writer.
 * 生成 Excel 文件
 * 生成 Excel 文件
 */

```

```

*/
function esc(v: unknown) {
 return String(v ?? "")
 .replace(/&/g, "&")
 .replace(/</g, "<")
 .replace(/>/g, ">");
}

export function buildXls(sheetName: string, rows: (string | number)[][]): {
 const head = rows[0] ?? [];
 const body = rows.slice(1);
 const html = `<html xmlns:o="urn:schemas-microsoft-com:office:office" xmlns:x="urn:schemas-microsoft-com:office:excel"
>
<head><meta charset="UTF-8" />
<!--[if gte mso 9]><xml><x:ExcelWorkbook><x:ExcelWorksheets><x:ExcelWorksheet>
<x:Name>${esc(sheetName)}</x:Name><x:WorksheetOptions><x:DisplayRightToLeft/></x:WorksheetOptions>
</x:ExcelWorksheet></x:ExcelWorksheets></x:ExcelWorkbook></xml><![endif]>-->
<style>
table{border-collapse:collapse;font-family:Tahoma,sans-serif;font-size:11pt}
th{background:#0f766e;color:#fff;border:1px solid #0b5c55;padding:6px;white-space:nowrap}
td{border:1px solid #cbd5e1;padding:5px;mso-number-format:"\\@"}
</style></head>
<body dir="rtl"><table><thead><tr>${head.map((h) => `<th>${esc(h)}</th>`).join("")}</tr></thead>
<tbody>${body
 .map((r) => `<tr>${r.map((c) => `<td>${esc(c)}</td>`).join("")}</tr>`)
 .join("")}</tbody></table></body></html>`;
 return "\uFEFF" + html;
}

```

```

export function xlsResponse(fileName: string, sheetName: string, rows: (string | number)[][]): {
 return new Response(buildXls(sheetName, rows), {
 headers: {
 "Content-Type": "application/vnd.ms-excel; charset=utf-8",
 "Content-Disposition": `attachment; filename="${esc(fileName)}.xls"`,
 "Cache-Control": "no-store",
 },
 });
}

```

@@ src/lib/geo.ts typescript

```

/**
 * =====
 * 00000000 00000000 00000 0000 0000
 * -----
 * 000000 0000 0 0000 000000 00 000000 0000:
 * • 000000 0000 000 000000/000 000000000000
 * • 000000 000000 00 0000 00 0000 000000 0000 0000
 * =====
 */

```

```

export type GeoArea = {
 /** 000 000000 00 000 */
 name: string;
 lat: number;
 lng: number;
 /** 00000000000 000000 0000 */
 zoom: number;
 /** 0000 0000000 000000 00 000 00000000 */
 radiusKm: number;
};

```

/\*\* 000 00000000 000000000 00 GeoJSON → 000 000000 0000000 \*/

```

export const PROVINCE_EN_FA: Record<string, string> = {
 Tehran: "تهران",
 Alborz: "آلبرز",
 Isfahan: "اصفهان",
 Fars: "فارس",
 "Razavi Khorasan": "مازندران",
 "East Azerbaijan": "آذربایجان شرقی",
 "West Azerbaijan": "آذربایجان غربی",
 Khuzestan: "خوزستان",
 Gilan: "گیلان",
 Mazandaran: "مازندران",
 Kerman: "کerman",
 Qom: "قم",
 Yazd: "یزد",
 Hamadan: "همدان",
 Kermanshah: "کermanshah",
 Golestan: "گلستان",
 Ardabil: "اردبیل",
 Qazvin: "قزوین",
 Zanjān: "زنجان",
 Markazi: "مرکزی",
 Lorestan: "لورستان",
 Kurdistan: "کردستان",
}

```

```

Hormozgan: " Hormozgan",
Bushehr: " Bushehr",
Semnan: " Semnan",
"Sistan and Baluchestan": " Sistan and Baluchestan",
"Chaharmahal and Bakhtiari": " Chaharmahal and Bakhtiari",
"Kohgiluyeh and Boyer-Ahmad": " Kohgiluyeh and Boyer-Ahmad",
"North Khorasan": " North Khorasan",
"South Khorasan": " South Khorasan",
Ilam: " Ilam",
};

/**
 * Province GeoArea — Province GeoArea
 */
export const IRAN_CENTER: GeoArea = {
 name: " Iran",
 lat: 32.4279,
 lng: 53.688,
 zoom: 5,
 radiusKm: 900,
};

/**
 * Province GeoArea — Province GeoArea
 */
export const PROVINCE_GEO: Record<string, GeoArea> = {
 "Ardabil": { name: "Ardabil", lat: 35.6892, lng: 51.389, zoom: 10, radiusKm: 70 },
 "Almas": { name: "Almas", lat: 35.8327, lng: 50.9916, zoom: 10, radiusKm: 45 },
 "Almas": { name: "Almas", lat: 32.6546, lng: 51.668, zoom: 8, radiusKm: 180 },
 "Almas": { name: "Almas", lat: 29.5918, lng: 52.5837, zoom: 8, radiusKm: 200 },
 "Almas": { name: "Almas", lat: 36.2605, lng: 59.6168, zoom: 8, radiusKm: 200 },
 "Almas": { name: "Almas", lat: 38.0962, lng: 46.2738, zoom: 8, radiusKm: 130 },
 "Almas": { name: "Almas", lat: 37.5527, lng: 45.0761, zoom: 8, radiusKm: 140 },
 "Almas": { name: "Almas", lat: 31.3183, lng: 48.6706, zoom: 8, radiusKm: 160 },
 "Almas": { name: "Almas", lat: 37.2808, lng: 49.5832, zoom: 9, radiusKm: 80 },
 "Almas": { name: "Almas", lat: 36.5659, lng: 53.06, zoom: 8, radiusKm: 130 },
 "Almas": { name: "Almas", lat: 30.2839, lng: 57.0834, zoom: 7, radiusKm: 250 },
 "Almas": { name: "Almas", lat: 34.6416, lng: 50.8746, zoom: 10, radiusKm: 50 },
 "Almas": { name: "Almas", lat: 31.8974, lng: 54.3569, zoom: 8, radiusKm: 170 },
 "Almas": { name: "Almas", lat: 34.7992, lng: 48.5146, zoom: 9, radiusKm: 90 },
 "Almas": { name: "Almas", lat: 34.3142, lng: 47.065, zoom: 9, radiusKm: 100 },
 "Almas": { name: "Almas", lat: 36.8427, lng: 54.4342, zoom: 9, radiusKm: 90 },
 "Almas": { name: "Almas", lat: 38.2498, lng: 48.2933, zoom: 9, radiusKm: 90 },
 "Almas": { name: "Almas", lat: 36.2688, lng: 50.0041, zoom: 9, radiusKm: 70 },
 "Almas": { name: "Almas", lat: 36.6736, lng: 48.4787, zoom: 9, radiusKm: 90 },
 "Almas": { name: "Almas", lat: 34.0917, lng: 49.6892, zoom: 9, radiusKm: 100 },
 "Almas": { name: "Almas", lat: 33.4878, lng: 48.3558, zoom: 9, radiusKm: 100 },
 "Almas": { name: "Almas", lat: 35.3219, lng: 46.9862, zoom: 9, radiusKm: 100 },
 "Almas": { name: "Almas", lat: 27.1832, lng: 56.2666, zoom: 8, radiusKm: 200 },
 "Almas": { name: "Almas", lat: 28.9234, lng: 50.82, zoom: 8, radiusKm: 150 },
 "Almas": { name: "Almas", lat: 35.5729, lng: 53.3971, zoom: 8, radiusKm: 180 },
 "Almas": { name: "Almas", lat: 29.4963, lng: 60.8629, zoom: 7, radiusKm: 300 },
 "Almas": { name: "Almas", lat: 32.3266, lng: 50.8644, zoom: 9, radiusKm: 80 },
 "Almas": { name: "Almas", lat: 30.6682, lng: 51.5876, zoom: 9, radiusKm: 80 },
 "Almas": { name: "Almas", lat: 37.4747, lng: 57.329, zoom: 8, radiusKm: 110 },
 "Almas": { name: "Almas", lat: 32.8649, lng: 59.2211, zoom: 7, radiusKm: 250 },
 "Almas": { name: "Almas", lat: 33.6374, lng: 46.4227, zoom: 9, radiusKm: 90 },
};

/**
 * City GeoArea (City GeoArea)
 */
export const CITY_GEO: Record<string, GeoArea> = {
 "Ardabil": { name: "Ardabil", lat: 35.6892, lng: 51.389, zoom: 11, radiusKm: 25 },
 "Almas": { name: "Almas", lat: 36.2605, lng: 59.6168, zoom: 11, radiusKm: 20 },
 "Almas": { name: "Almas", lat: 32.6546, lng: 51.668, zoom: 11, radiusKm: 18 },
 "Almas": { name: "Almas", lat: 29.5918, lng: 52.5837, zoom: 11, radiusKm: 18 },
 "Almas": { name: "Almas", lat: 38.0962, lng: 46.2738, zoom: 11, radiusKm: 16 },
 "Almas": { name: "Almas", lat: 35.8327, lng: 50.9916, zoom: 11, radiusKm: 16 },
 "Almas": { name: "Almas", lat: 31.3183, lng: 48.6706, zoom: 11, radiusKm: 16 },
 "Almas": { name: "Almas", lat: 34.6416, lng: 50.8746, zoom: 11, radiusKm: 14 },
 "Almas": { name: "Almas", lat: 34.3142, lng: 47.065, zoom: 11, radiusKm: 12 },
 "Almas": { name: "Almas", lat: 37.2808, lng: 49.5832, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 37.5527, lng: 45.0761, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 30.2839, lng: 57.0834, zoom: 12, radiusKm: 12 },
 "Almas": { name: "Almas", lat: 29.4963, lng: 60.8629, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 31.8974, lng: 54.3569, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 38.2498, lng: 48.2933, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 27.1832, lng: 56.2666, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 36.5659, lng: 53.06, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 36.2688, lng: 50.0041, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 36.6736, lng: 48.4787, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 34.0917, lng: 49.6892, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 33.4878, lng: 48.3558, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 35.3219, lng: 46.9862, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 28.9234, lng: 50.82, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 36.8427, lng: 54.4342, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 34.7992, lng: 48.5146, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 32.3266, lng: 50.8644, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 37.4747, lng: 57.329, zoom: 12, radiusKm: 10 },
 "Almas": { name: "Almas", lat: 32.8649, lng: 59.2211, zoom: 12, radiusKm: 10 },
};

```

```

 000000: { name: "000000", lat: 33.6374, lng: 46.4227, zoom: 12, radiusKm: 10 },
 000000: { name: "000000", lat: 30.6682, lng: 51.5876, zoom: 12, radiusKm: 10 },
 000000: { name: "000000", lat: 35.5729, lng: 53.3971, zoom: 12, radiusKm: 10 },
};

```

```

/** 000000 0000000 000000 00000000 00000000 000000 */
export const TEHRAN_REGIONS: Record<string, GeoArea> = {
 "000000 1": { name: "000000 0", lat: 35.8046, lng: 51.4324, zoom: 13, radiusKm: 6 },
 "000000 2": { name: "000000 0", lat: 35.7576, lng: 51.3347, zoom: 13, radiusKm: 6 },
 "000000 3": { name: "000000 0", lat: 35.7647, lng: 51.4059, zoom: 13, radiusKm: 5 },
 "000000 4": { name: "000000 0", lat: 35.7614, lng: 51.5108, zoom: 13, radiusKm: 7 },
 "000000 5": { name: "000000 0", lat: 35.7576, lng: 51.3018, zoom: 13, radiusKm: 6 },
 "000000 6": { name: "000000 0", lat: 35.7219, lng: 51.4089, zoom: 13, radiusKm: 4 },
 "000000 7": { name: "000000 0", lat: 35.7315, lng: 51.4462, zoom: 13, radiusKm: 4 },
 "000000 8": { name: "000000 0", lat: 35.7275, lng: 51.4869, zoom: 13, radiusKm: 4 },
 "000000 9": { name: "000000 0", lat: 35.6959, lng: 51.3162, zoom: 13, radiusKm: 4 },
 "000000 10": { name: "000000 00", lat: 35.6892, lng: 51.3565, zoom: 13, radiusKm: 4 },
 "000000 11": { name: "000000 00", lat: 35.6875, lng: 51.3889, zoom: 13, radiusKm: 4 },
 "000000 12": { name: "000000 00", lat: 35.6836, lng: 51.4275, zoom: 13, radiusKm: 4 },
 "000000 13": { name: "000000 00", lat: 35.7008, lng: 51.4831, zoom: 13, radiusKm: 4 },
 "000000 14": { name: "000000 00", lat: 35.6708, lng: 51.4675, zoom: 13, radiusKm: 5 },
 "000000 15": { name: "000000 00", lat: 35.6459, lng: 51.4675, zoom: 13, radiusKm: 6 },
 "000000 16": { name: "000000 00", lat: 35.6398, lng: 51.4008, zoom: 13, radiusKm: 4 },
 "000000 17": { name: "000000 00", lat: 35.6553, lng: 51.3555, zoom: 13, radiusKm: 4 },
 "000000 18": { name: "000000 00", lat: 35.6553, lng: 51.3018, zoom: 13, radiusKm: 6 },
 "000000 19": { name: "000000 00", lat: 35.6252, lng: 51.3555, zoom: 13, radiusKm: 5 },
 "000000 20": { name: "000000 00", lat: 35.5892, lng: 51.4324, zoom: 13, radiusKm: 6 },
 "000000 21": { name: "000000 00", lat: 35.6959, lng: 51.2261, zoom: 13, radiusKm: 7 },
 "000000 22": { name: "000000 00", lat: 35.7576, lng: 51.2072, zoom: 13, radiusKm: 8 },
};

```

```

/** 000 0000 0000000 0000 000000 0000000 */
function norm(s: string) {
 return String(s ?? "")
 .replace(/[\u200c\u200f\u200e]/g, "")
 .replace(/[\u002d]/g, "-")
 .replace(/ /g, " ")
 .replace(/\\s+/g, " ")
 .trim();
}

```

```

/** 000000 000000 000000 00 000000 (00000 «000000 0») */
function toEn(s: string) {
 return s.replace(/[\u06f0-\u06f9]/g, (c) => String(c.charCodeAt(0) - 0x06f0));
}

```

```

/**
 * 000000 0000000 00000 00 0000 000000 / 0000 / 000000.
 * 00000000000 000 000000 00000000000 0000000 (000000 ← 0000 ← 000000 ← 000000).
 */
export function resolveArea(province?: string, city?: string, region?: string): GeoArea {
 const r = toEn(norm(region ?? ""));
 const c = norm(city ?? "");
 const p = norm(province ?? "");

 if (c === "000000" && r && TEHRAN_REGIONS[r]) return TEHRAN_REGIONS[r];
 if (c && CITY_GEO[c]) return CITY_GEO[c];
 if (p && PROVINCE_GEO[p]) return PROVINCE_GEO[p];
 return IRAN_CENTER;
}

```

```

/** 000000 00 0000 00 000 00000000 */
export function distanceKm(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
 const R = 6371;
 const dLat = ((b.lat - a.lat) * Math.PI) / 180;
 const dLng = ((b.lng - a.lng) * Math.PI) / 180;
 const s =
 Math.sin(dLat / 2) ** 2 +
 Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
 return 2 * R * Math.asin(Math.sqrt(s));
}

```

```

/** 000000 0000000 000000 00 00 00000 */
export function provinceOf(point: { lat: number; lng: number }): string {
 let best = "";
 let bestD = Number.POSITIVE_INFINITY;
 for (const [name, g] of Object.entries(PROVINCE_GEO)) {
 const d = distanceKm(point, g);
 if (d < bestD) {
 bestD = d;
 best = name;
 }
 }
 return best;
}

```



```

"00000000", "0000000", "000000", "0000 00", "0000000", "0000000000", "0000000", "0000000000",
"000 0000", "0000000000",
},
000000: [
"0000000", "0000000000", "0000000000", "000000", "00000", "0000000000", "00000", "000",
"000000", "00000", "0000000000", "0000", "0000000", "00000",
],
000000: ["000000", "000000", "00000000", "00000", "000000000000", "0000000", "00000000", "0000", "0000", "0000000000", "0000000000"],
000000: ["000000", "00000", "0000000", "000000", "00000", "00000000", "0000000000", "00000", "000000", "0000000000", "00000"],
000000: [
"00000", "00000", "00000", "000000", "0000000", "000000", "00000", "000000", "00000000", "00000000",
"0000000", "0000000", "0000000", "000000", "00000000", "000000000",
],
000000: [
"0000000000", "0000000", "000000", "0000000000", "0000000", "00000", "00000000", "0000000",
"000000", "000000", "00000000", "00000", "00000", "00000000",
],
00000000: [
"000000", "0000", "0000000", "00000", "00000", "000000", "0000000000", "0000000000",
"0000000", "00000000", "000000000", "00000 000000", "000",
],
00000000: [
"0000000000", "0000000", "0000", "0000 0000", "000000", "0000", "0000000000", "00000",
"00000000", "00000", "000000", "0000000", "00000", "00000000", "0000000", "0000 00000",
],
000000: [
"000000", "00000000", "000000", "0000000", "00000", "000000", "00", "0000000", "0000",
"00000", "00000", "00000000", "00000000", "0000 0000", "000000000", "000000", "00000000",
],
000000: ["000000", "0000000", "0000000", "0000000", "0000000000", "00000", "000000", "00000000", "0000000", "000000", "000000000"],
"0000000 0 0000000000": [
"0000000", "00000", "0000000", "0000000000", "0000000", "0000", "00000000", "000000",
"00000000", "0000000", "0000000", "0000000", "0000000", "000000", "00000000", "0000", "00000000",
"0000000", "000000", "00000", "000000",
],
"0000000000 0 00000000": [
"0000000", "000000", "0000000", "0000000", "000000", "00000", "00000", "00",
"0000000", "0000000000", "000000000", "00000000", "000000", "0000000", "00000000",
],
"0000000000 0 000000000": ["000000", "0000000000", "000000", "0000000", "00000", "00000", "00000", "0000000", "00000", "00000 000000"]
,
"0000000 00000": ["0000000", "0000000", "00000000", "0000000", "000000", "00000", "000000", "0000", "000000000", "0000", "00000000"]
],
"0000000 00000": [
"0000000", "00000", "000", "000000", "00000000", "0000000", "0000000", "00000",
"0000000", "0000000", "0000000", "0000000000", "000000", "00000000000",
],
000000: ["000000", "0000000", "000000000", "0000000", "000000", "000000000", "0000000", "00000", "00000000", "00000", "0000000", "00000
00"],
};

```

```

/** 000000 000000000 0000000 00000 */
export const REGIONS: Record<string, string[]> = {
000000: Array.from({ length: 22 }, (_, i) => `000000 ${i + 1}`),
00000: Array.from({ length: 13 }, (_, i) => `000000 ${i + 1}`),
0000000: Array.from({ length: 15 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 11 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 10 }, (_, i) => `000000 ${i + 1}`),
0000: Array.from({ length: 12 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 8 }, (_, i) => `000000 ${i + 1}`),
00: Array.from({ length: 8 }, (_, i) => `000000 ${i + 1}`),
0000000000: Array.from({ length: 8 }, (_, i) => `000000 ${i + 1}`),
0000: Array.from({ length: 5 }, (_, i) => `000000 ${i + 1}`),
00000000: Array.from({ length: 5 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 4 }, (_, i) => `000000 ${i + 1}`),
00000000: Array.from({ length: 5 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 4 }, (_, i) => `000000 ${i + 1}`),
0000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),
00000000: Array.from({ length: 4 }, (_, i) => `000000 ${i + 1}`),
0000000000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
00000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 4 }, (_, i) => `000000 ${i + 1}`),
00000: Array.from({ length: 5 }, (_, i) => `000000 ${i + 1}`),
0000000000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 4 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),
00000000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),
00000000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
00000000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
000000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
0000000000: Array.from({ length: 3 }, (_, i) => `000000 ${i + 1}`),

```

```

 00000: Array.from({ length: 3 }, (_, i) => `00000 ${i + 1}`),
 000: Array.from({ length: 2 }, (_, i) => `00000 ${i + 1}`),
 000000: Array.from({ length: 2 }, (_, i) => `000000 ${i + 1}`),
};

export const PROVINCES = Object.keys(IRAN);

/** 000000 00 0000000 00000000 */
export const CITY_COUNT = Object.values(IRAN).reduce((a, c) => a + c.length, 0);

@@ src/lib/jalali.ts typescript
// Lightweight Jalali (Shamsi) date utilities - no external dependency.

export const JALALI_MONTHS = [
 "00000000",
 "00000000",
 "000000",
 "0000",
 "000000",
 "00000000",
 "0000",
 "000000",
 "0000",
 "0000",
 "00",
 "000000",
 "000000",
];

export const WEEK_DAYS = ["0", "0", "0", "0", "0", "0", "0"];

function div(a: number, b: number) {
 return Math.floor(a / b);
}

export function gregorianToJalali(gy: number, gm: number, gd: number): [number, number, number] {
 const g_d_m = [0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334];
 const gy2 = gm > 2 ? gy + 1 : gy;
 let days =
 355666 +
 365 * gy +
 div(gy2 + 3, 4) -
 div(gy2 + 99, 100) +
 div(gy2 + 399, 400) +
 gd +
 g_d_m[gm - 1];
 let jy = -1595 + 33 * div(days, 12053);
 days %= 12053;
 jy += 4 * div(days, 1461);
 days %= 1461;
 if (days > 365) {
 jy += div(days - 1, 365);
 days = (days - 1) % 365;
 }
 const jm = days < 186 ? 1 + div(days, 31) : 7 + div(days - 186, 30);
 const jd = 1 + (days < 186 ? days % 31 : (days - 186) % 30);
 return [jy, jm, jd];
}

export function jalaliToGregorian(jy: number, jm: number, jd: number): [number, number, number] {
 jy += 1595;
 let days =
 -355668 + 365 * jy + div(jy, 33) * 8 + div((jy % 33) + 3, 4) + jd + (jm < 7 ? (jm - 1) * 31 : (jm - 7) * 30 + 186);
 let gy = 400 * div(days, 146097);
 days %= 146097;
 if (days > 36524) {
 gy += 100 * div(--days, 36524);
 days %= 36524;
 if (days >= 365) days++;
 }
 gy += 4 * div(days, 1461);
 days %= 1461;
 if (days > 365) {
 gy += div(days - 1, 365);
 days = (days - 1) % 365;
 }
 let gd = days + 1;
 const sal_a = [
 0,
 31,
 (gy % 4 === 0 && gy % 100 !== 0) || gy % 400 === 0 ? 29 : 28,
 31,
 30,
 31,
 30,
 31,
 30,
 31,
];

```



```

 31,
 30,
 31,
 30,
 31,
];
 let gm = 0;
 for (gm = 0; gm < 13 && gd > sal_a[gm]; gm++) gd -= sal_a[gm];
 return [gy, gm, gd];
}

export function isJalaliLeap(jy: number) {
 const r = jy % 33;
 return [1, 5, 9, 13, 17, 22, 26, 30].includes(r);
}

export function jalaliMonthLength(jy: number, jm: number) {
 if (jm <= 6) return 31;
 if (jm <= 11) return 30;
 return isJalaliLeap(jy) ? 30 : 29;
}

export function pad2(n: number) {
 return n < 10 ? `0${n}` : String(n);
}

export function formatJalali(jy: number, jm: number, jd: number) {
 return `${jy}/${pad2(jm)}/${pad2(jd)}`;
}

/** Today's date in Tehran timezone as `YYYY/MM/DD` shamsi string. */
export function todayJalali(): string {
 const now = new Date();
 const parts = new Intl.DateTimeFormat("en-CA", {
 timeZone: "Asia/Tehran",
 year: "numeric",
 month: "2-digit",
 day: "2-digit",
 }).format(now);
 const [gy, gm, gd] = parts.split("-").map(Number);
 const [jy, jm, jd] = gregorianToJalali(gy, gm, gd);
 return formatJalali(jy, jm, jd);
}

export function toEnglishDigits(input: string) {
 return input
 .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
 .replace(/[\u0660-\u0669]/g, (c) => String(c.charCodeAt(0) - 0x0660));
}

export function toPersianDigits(input: string | number) {
 return String(input).replace(/d/g, (d) => String.fromCharCode(0x06f0 + Number(d)));
}

/** Auto-mask typed digits into `YYYY/MM/DD`. */
export function maskJalaliInput(raw: string) {
 const digits = toEnglishDigits(raw).replace(/d/g, "").slice(0, 8);
 if (digits.length <= 4) return digits;
 if (digits.length <= 6) return `${digits.slice(0, 4)}/${digits.slice(4)}`;
 return `${digits.slice(0, 4)}/${digits.slice(4, 6)}/${digits.slice(6)}`;
}

export function parseJalali(value: string): [number, number, number] | null {
 const m = toEnglishDigits(value).match(/^(\d{4})\/(\d{1,2})\/(\d{1,2})$/);
 if (!m) return null;
 const jy = Number(m[1]);
 const jm = Number(m[2]);
 const jd = Number(m[3]);
 if (jm < 1 || jm > 12) return null;
 if (jd < 1 || jd > jalaliMonthLength(jy, jm)) return null;
 return [jy, jm, jd];
}

export function isValidJalali(value: string) {
 return parseJalali(value) !== null;
}

/** Jalali weekday index (0 = □□□□) for a jalali date */
export function jalaliWeekDay(jy: number, jm: number, jd: number) {
 const [gy, gm, gd] = jalaliToGregorian(jy, jm, jd);
 const day = new Date(Date.UTC(gy, gm - 1, gd)).getUTCDay(); // 0=Sunday
 return (day + 1) % 7;
}

export function tehranTime(date: Date | string | number) {

```

```

const d = new Date(date);
return new Intl.DateTimeFormat("fa-IR", {
 timeZone: "Asia/Tehran",
 hour: "2-digit",
 minute: "2-digit",
 second: "2-digit",
}).format(d);
}

export function tehranDateTime(date: Date | string | number) {
 const d = new Date(date);
 const parts = new Intl.DateTimeFormat("en-CA", {
 timeZone: "Asia/Tehran",
 year: "numeric",
 month: "2-digit",
 day: "2-digit",
 }).format(d);
 const [gy, gm, gd] = parts.split("-").map(Number);
 const [jy, jm, jd] = gregorianToJalali(gy, gm, gd);
 return `${formatJalali(jy, jm, jd)} - ${tehranTime(d)}`;
}

@@ src/lib/messaging.ts typescript
import { db, dbRetrySafe } from "@db";
import { messageLogs, messengers, settings } from "@db/schema";
import { bonusKeyOf } from "./defaults";
import { getProducts } from "./settings-server";
import { toPersianDigits } from "./jalali";
import { fetchWithRetry } from "./retry";
import { eq } from "drizzle-orm";

/* =====
* 000000 000000 0000 00 00000000000000
*
* 0000000000: 00000000 0000 000000 00000000 (0000 000000 000000)
*
* 0000000000:
* - 0000000000 00 000000 0000000000 00 0000000000 00000000 00000000
* - 000000 000000 00000000 (Exponential Backoff) 0000 00000000 000000
* - 000000 00 00000000000000 000000 00 00000000 0000000000
* - 00000000 000000 00 00000000 (000000 0000000000 000000 00 00000000)
* - 0000 000000 0000 00000000/0000
* ===== */

export type Platform = "telegram" | "bale" | "eitaa" | "whatsapp";

export type MessengerTarget = {
 id?: number;
 platform: string;
 target: string;
 token: string;
 provider?: string;
 apiUrl?: string;
 label?: string;
};

export type SendResult = { ok: boolean; detail: string; via: "direct" | "proxy" | "none" };

export const PLATFORM_LABEL: Record<string, string> = {
 telegram: "00000000",
 bale: "0000",
 eitaa: "000000",
 whatsapp: "00000000",
};

/** 000000000000 00000 00000000 00 00000000000 0000000000 */
export const WHATSAPP_PROVIDERS = [
 {
 key: "whatsiplus",
 label: "0000000000000000 (whatsiplus.ir)",
 hint: "000000 API 00 0000 whatsiplus.ir | 000000: 000000 00 0 0000 00000000000000",
 },
 {
 key: "ultramsg",
 label: "UltraMsg",
 hint: "000000 00 000000 instanceld:token | 000000: 000000 00 00 000000 0000 989121111111",
 },
 {
 key: "cloudapi",
 label: "WhatsApp Cloud API (0000)",
 hint: "000000 00 000000 phoneNumberId:accessToken | 000000: 000000 00 00 000000",
 },
 {
 key: "custom",
 label: "000000 00000000",
 },
];

```

```

 hint: "شماره API رو وارد کنید {phone} و {text} و {token}",
 },
] as const;

/* ----- */
/* شماره ها را وارد کنید */
/* ----- */

export type ProxyConfig = { url: string; enabled: boolean; secret?: string };

export const DEFAULT_PROXY: ProxyConfig = {
 url: process.env.MESSAGING_PROXY_URL || "https://namayandeelmi-javad.javadalamar-mehraeen.workers.dev/",
 enabled: true,
 secret: "",
};

export async function getProxy(): Promise<ProxyConfig> {
 const rows = await dbRetrySafe(
 () => db.select().from(settings).where(eq(settings.key, "messagingProxy")).limit(1),
 [],
 "proxy:get",
);
 const v = rows[0]?.value as ProxyConfig | undefined;
 return v && typeof v.url === "string" ? { ...DEFAULT_PROXY, ...v } : DEFAULT_PROXY;
}

export type TokenMap = Record<string, string>;

const FALLBACK_TOKENS: TokenMap = {
 bale: process.env.BALE_BOT_TOKEN || "1199464939:uhHpJVtcy__qdtFfN7iuzr4AH7bZBKPG85A",
 telegram: process.env.TELEGRAM_BOT_TOKEN || "",
 eitaa: process.env.EITAA_TOKEN || "",
 whatsapp: process.env.WHATSAPP_TOKEN || "",
};

/* ----- */
export async function getTokens(): Promise<TokenMap> {
 const rows = await dbRetrySafe(
 () => db.select().from(settings).where(eq(settings.key, "messengerTokens")).limit(1),
 [],
 "tokens:get",
);
 const v = rows[0]?.value as TokenMap | undefined;
 return v && typeof v === "object" ? { ...FALLBACK_TOKENS, ...v } : FALLBACK_TOKENS;
}

export async function resolveToken(platform: string, rowToken?: string) {
 const t = (rowToken ?? "").trim();
 if (t) return t;
 return ((await getTokens())[platform] ?? "").trim();
}

/* ----- */
/* شماره ها را وارد کنید */
/* ----- */

type OrderLike = {
 id: number;
 repName: string;
 dateShamsi: string;
 pharmacyName: string;
 managerName: string;
 managerPhone: string;
 address: string;
 lat: number | null;
 lng: number | null;
 items: Record<string, number>;
 distributor: string;
 visitor: string;
 notes: string;
};

export async function formatOrderMessage(o: OrderLike, withLocation = true) {
 const PRODUCTS = await getProducts();
 const L: string[] = [];
 L.push("شماره سفارش: " + o.id);
 L.push("-----");
 L.push("شماره سفارش: " + o.id);
 L.push("تاریخ شمسی: " + o.dateShamsi);
 L.push("نام نماینده: " + o.repName);
 L.push("نام داروخانه: " + o.pharmacyName);
 if (o.managerName) L.push("نام مدیر: " + o.managerName);
 if (o.managerPhone) L.push("شماره مدیر: " + o.managerPhone);
 if (o.address) L.push("آدرس: " + o.address);
 if (withLocation && o.lat && o.lng) L.push("موقعیت: " + "https://www.google.com/maps?q=" + o.lat + "," + o.lng);
}

```

```

L.push("———— □□□□□ □□□□□ ———");
let totalUnits = 0;
let totalBonus = 0;
for (const p of PRODUCTS) {
 const q = Number(o.items?.[p.key] || 0);
 const b = Number(o.items?.[bonusKeyOf(p.key)] || 0);
 if (q || b) {
 totalUnits += q;
 totalBonus += b;
 }
 L.push(`• ${p.label}: ${toPersianDigits(q)}${b ? ` (□□□□□ ${toPersianDigits(b)})` : ""}`);
}
L.push(`□□□ □□: ${toPersianDigits(totalUnits)} □□□ | □□□□□: ${toPersianDigits(totalBonus)} □□□`);
L.push("————");
if (o.distributor) L.push(`□□□ □□□: ${o.distributor}`);
if (o.visitor) L.push(`□□□ □□□□□□□: ${o.visitor}`);
if (o.notes) L.push(`□□□□□□□□: ${o.notes}`);
return L.join("\n");
}

/* ----- */
/* □□□□□ □□□□□□ □□ □□ □□□□□□□□□□ */
/* ----- */

const RETRY = { retries: 3, baseDelayMs: 1_000, maxDelayMs: 8_000, timeoutMs: 15_000 };

function friendly(platform: string, status: number, raw: string) {
 const s = raw.replace(/\s+/g, " ").slice(0, 220);
 if (status === 401 || status === 403) return `□□□□ □${platform} □□□□□□□ □□ □□□□□ □□□ (${status}) — ${s}`;
 if (status === 404) return `□□□□□/□□□□ □${platform} □□□□ □□□ (□□□) — ${s}`;
 if (status === 400) return `□□□□ □□ □□□□□□□ □□□□□□□: ${s}`;
 return `□□□□ □${status}: ${s}`;
}

/** □□□□ □□□□□□□□ □□□□ □□□□□ □□ □□□□□□ □□□□□□□□□□□□ ("true" / "false") */
function isTruthy(v: unknown): boolean | null {
 if (typeof v === "boolean") return v;
 if (typeof v === "string") {
 const s = v.trim().toLowerCase();
 if (["true", "success", "ok", "1", "sent"].includes(s)) return true;
 if (["false", "error", "failed", "0"].includes(s)) return false;
 }
 if (typeof v === "number") return v === 1 || v === 200;
 return null;
}

async function readJson(res: Response) {
 const raw = await res.text();
 try {
 return { raw, json: JSON.parse(raw) as Record<string, unknown> };
 } catch {
 return { raw, json: null };
 }
}

/** □□□□□□ □ □□□ — □□ □□ Bot API □□ □□□□□□ □□□□□ □□□□□ */
async function sendBotApi(base: string, token: string, chatId: string, text: string, label: string) {
 const res = await fetchWithRetry(
 `${base}/bot${token}/sendMessage`,
 {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ chat_id: chatId, text, disable_web_page_preview: false }),
 },
 { ...RETRY, label },
);
 const { raw, json } = await readJson(res);
 if (res.ok && json?.ok !== false) return { ok: true, detail: "□□□□□ □□" };
 const desc = (json?.description as string) ?? raw;
 return { ok: false, detail: friendly(label, res.status, desc) };
}

/** □□□□□ — □□□□□ □□□□□□□□ (form-urlencoded) */
async function sendEitaa(token: string, chatId: string, text: string) {
 const res = await fetchWithRetry(
 `https://eitaayar.ir/api/${token}/sendMessage`,
 {
 method: "POST",
 headers: { "Content-Type": "application/x-www-form-urlencoded" },
 body: new URLSearchParams({ chat_id: chatId, text }).toString(),
 },
 { ...RETRY, label: "□□□□□" },
);
 const { raw, json } = await readJson(res);
}

```

```

if (res.ok && json?.ok !== false) return { ok: true, detail: "📩📩📩📩" };
return { ok: false, detail: friendly("📩📩📩", res.status, (json?.description as string) ?? raw) };
}

/** 📩📩📩📩 — 📩📩 📩📩📩📩 📩📩📩 */
async function sendWhatsapp(m: MessengerTarget, token: string, text: string) {
 const provider = m.provider || "whatsiplus";
 const digits = m.target.replace(/\D/g, "");

 if (provider === "whatsiplus") {
 // 📩📩📩📩📩: https://api.whatsiplus.com/sendMsg/{apikey}?phonenumber=...&message=...
 const base = (m.apiUrl || "https://api.whatsiplus.com/sendMsg").replace(/\/$/, "");
 const url = `${base}/${token}?phonenumber=${encodeURIComponent(digits)}&message=${encodeURIComponent(text)}`;
 const res = await fetchWithRetry(url, { method: "GET" }, { ...RETRY, label: "📩📩📩📩📩" });
 const { raw, json } = await readJson(res);
 // 📩📩📩📩📩 📩📩 📩📩 📩📩 📩📩 📩📩 📩📩📩📩📩📩 📩📩 📩📩 📩📩📩 📩📩
 const okFlag = json ? isTruthy(json.success) ?? isTruthy(json.status) : null;
 const failed = okFlag === false || (json?.message && okFlag !== true);
 return res.ok && !failed
 ? { ok: true, detail: "📩📩📩📩📩" }
 : { ok: false, detail: `📩📩📩📩📩: ${(json?.message as string) ?? raw.slice(0, 180)}` };
 }

 if (provider === "ultramsg") {
 const [instance, apiToken] = token.split(":");
 if (!instance || !apiToken) return { ok: false, detail: "📩📩📩 UltraMsg 📩📩📩 instanceId:token 📩📩📩" };
 const res = await fetchWithRetry(
 `https://api.ultramsg.com/${instance}/messages/chat`,
 {
 method: "POST",
 headers: { "Content-Type": "application/x-www-form-urlencoded" },
 body: new URLSearchParams({ token: apiToken, to: digits, body: text }).toString(),
 },
 { ...RETRY, label: "📩📩📩📩📩" },
);
 const { raw, json } = await readJson(res);
 const sentOk = json ? isTruthy(json.sent) ?? isTruthy(json.success) : null;
 if (!res.ok || sentOk === false || json?.error) {
 return { ok: false, detail: `📩📩📩📩📩: ${(json?.error as string) ?? raw.slice(0, 180)}` };
 }
 return { ok: true, detail: "📩📩📩📩📩" };
 }

 if (provider === "cloudapi") {
 const i = token.indexOf(":");
 const phoneNumberId = i > 0 ? token.slice(0, i) : "";
 const accessToken = i > 0 ? token.slice(i + 1) : "";
 if (!phoneNumberId || !accessToken) {
 return { ok: false, detail: "📩📩📩📩 Cloud API 📩📩📩📩📩 phoneNumberId:accessToken 📩📩📩" };
 }
 const res = await fetchWithRetry(
 `https://graph.facebook.com/v21.0/${phoneNumberId}/messages`,
 {
 method: "POST",
 headers: { "Content-Type": "application/json", Authorization: `Bearer ${accessToken}` },
 body: JSON.stringify({
 messaging_product: "whatsapp",
 recipient_type: "individual",
 to: digits,
 type: "text",
 text: { preview_url: false, body: text },
 }),
 },
 { ...RETRY, label: "📩📩📩📩📩" },
);
 const { raw } = await readJson(res);
 return res.ok ? { ok: true, detail: "📩📩📩📩📩" } : { ok: false, detail: friendly("📩📩📩📩📩", res.status, raw) };
 }
}

// 📩📩📩📩 📩📩📩📩📩
if (!m.apiUrl) return { ok: false, detail: "📩📩📩📩 📩📩📩📩 📩📩📩📩📩 📩📩📩 📩📩📩 📩📩" };
const url = m.apiUrl
 .replace("{phone}", encodeURIComponent(digits))
 .replace("{text}", encodeURIComponent(text))
 .replace("{token}", encodeURIComponent(token));
const res = await fetchWithRetry(url, { method: "GET" }, { ...RETRY, label: "📩📩📩📩📩" });
const { raw, json } = await readJson(res);
const okFlag = json ? isTruthy(json.success) ?? isTruthy(json.status) : null;
if (!res.ok || okFlag === false) return { ok: false, detail: `📩📩📩📩📩: ${raw.slice(0, 180)}` };
return { ok: true, detail: "📩📩📩📩📩" };
}

async function sendDirect(m: MessengerTarget, token: string, text: string): Promise<{ ok: boolean; detail: string }> {
 try {
 switch (m.platform) {

```

```

 case "telegram":
 return await sendBotApi("https://api.telegram.org", token, m.target, text, "000000");
 case "bale":
 return await sendBotApi("https://tapi.bale.ai", token, m.target, text, "000");
 case "eitaa":
 return await sendEitaa(token, m.target, text);
 case "whatsapp":
 return await sendWhatsapp(m, token, text);
 default:
 return { ok: false, detail: "0000000000 0000000000 00000000" };
 }
} catch (err) {
const msg = err instanceof Error ? err.message : "0000 0000";
return {
 ok: false,
 detail: msg.toLowerCase().includes("abort")
? "000000 000000 000 (Timeout) — 0000000000 0000 00 0000 000000 00000000 00000000 00 0000 0000"
: `0000 0000: ${msg}`,
};
}
}

```

```

/** 000000 00 0000 0000000 Cloudflare Worker (0000 00 00000000 00000000 000000) */
async function sendViaProxy(proxy: ProxyConfig, m: MessengerTarget, token: string, text: string) {
const alias = m.platform === "eitaa" ? "ita" : m.platform;
try {
const res = await fetchWithRetry(
 proxy.url.trim(),
 {
 method: "POST",
 headers: {
 "Content-Type": "application/json",
 ...(proxy.secret ? { "x-proxy-secret": proxy.secret } : {}),
 },
 body: JSON.stringify({
 messenger: alias,
 platform: m.platform,
 provider: m.provider,
 text,
 chatId: m.target,
 chat_id: m.target,
 token,
 }),
 },
 { ...RETRY, label: "proxy" },
);
const { raw, json } = await readJson(res);
const failed = json && (json.success === false || json.ok === false);
if (res.ok && !failed) return { ok: true, detail: "000000 00 (00 00000 0000000)" };
const detail = (json?.error as string) ?? (json?.description as string) ?? raw.slice(0, 200);
return { ok: false, detail: `000000: ${detail}` };
} catch (err) {
return { ok: false, detail: `000000 00 000000 0000: ${err instanceof Error ? err.message : "0000"}` };
}
}

```

```

/**
* 000000 00 0000 00 00 0000.
* 000000 00000000 0 00 0000 0000 00 0000 00000000 0000 00000000.
*/
export async function sendOne(
 input: MessengerTarget,
 text: string,
 proxyOverride?: ProxyConfig,
): Promise<SendResult> {
if (!input.target.trim()) return { ok: false, detail: "000000 0000 0000 0000 0000", via: "none" };

const token = await resolveToken(input.platform, input.token);
const proxy = proxyOverride ?? (await getProxy());

if (!token && input.platform !== "whatsapp") {
if (!proxy.enabled) return { ok: false, detail: "0000 0000 0000 0000 0000", via: "none" };
}

// 0) 0000 0000000 (00000000 0 0000 0000 00000000)
if (token) {
const direct = await sendDirect(input, token, text);
if (direct.ok) return { ...direct, via: "direct" };
}

// 0) 00 0000 000000 0000 00 0000 0000000
if (proxy.enabled && proxy.url) {
const viaProxy = await sendViaProxy(proxy, input, token, text);
if (viaProxy.ok) return { ...viaProxy, via: "proxy" };
return { ok: false, detail: `000000: ${direct.detail} | ${viaProxy.detail}`, via: "none" };
}
}

```

```

 return { ...direct, via: "none" };
}

// 消息发送失败 (消息内容 消息平台 消息目标 消息状态 消息时间)
if (proxy.enabled && proxy.url) {
 const viaProxy = await sendViaProxy(proxy, input, "", text);
 return { ...viaProxy, via: viaProxy.ok ? "proxy" : "none" };
}
return { ok: false, detail: "消息发送失败: 消息内容 消息平台 消息目标 消息状态 消息时间", via: "none" };
}

/* ----- */
/* 消息发送失败: 消息内容 消息平台 消息目标 消息状态 消息时间 */
/* ----- */

export type DispatchReport = {
 total: number;
 sent: number;
 failed: number;
 summary: string;
 results: { platform: string; label: string; target: string; ok: boolean; detail: string }[];
};

/**
 * 消息发送失败: 消息内容 消息平台 消息目标 消息状态 消息时间
 * 消息发送失败: 消息内容 消息平台 消息目标 消息状态 消息时间 (Promise.allSettled).
 */
export async function dispatchText(text: string, orderId?: number): Promise<DispatchReport> {
 const targets = await dbRetrySafe(
 () => db.select().from(messengers).where(eq(messengers.enabled, true)),
 [],
 "messengers:list",
);

 if (targets.length === 0) {
 return { total: 0, sent: 0, failed: 0, summary: "消息发送失败: 消息内容 消息平台 消息目标 消息状态 消息时间", results: [] };
 }

 const proxy = await getProxy();

 const settled = await Promise.allSettled(
 targets.map(async (t) => {
 const r = await sendOne(
 {
 id: t.id,
 platform: t.platform,
 target: t.target,
 token: t.token,
 provider: t.provider,
 apiUrl: t.apiUrl,
 label: t.label,
 },
 text,
 proxy,
);
 })
);

 // 消息发送失败: 消息内容 消息平台 消息目标 消息状态 消息时间
 await dbRetrySafe(
 () =>
 db.insert(messageLogs).values({
 orderId: orderId ?? null,
 platform: t.platform,
 target: t.target,
 ok: r.ok,
 detail: r.detail,
 }),
 undefined,
 "messageLog:insert",
);

 await dbRetrySafe(
 () =>
 db
 .update(messengers)
 .set({
 lastStatus: `${r.ok ? "□": "□"} ${r.detail}`.slice(0, 400),
 ...(r.ok ? { lastOkAt: new Date() } : { lastErrorAt: new Date() }),
 })
 .where(eq(messengers.id, t.id)),
 undefined,
 "messenger:status",
);

 return {
 platform: t.platform,
 label: t.label || PLATFORM_LABEL[t.platform] || t.platform,
 };
}

```

```

 target: t.target,
 ok: r.ok,
 detail: r.detail,
 });
 });
});

const results = settled.map((s, i) =>
 s.status === "fulfilled"
 ? s.value
 : {
 platform: targets[i].platform,
 label: targets[i].label || PLATFORM_LABEL[targets[i].platform],
 target: targets[i].target,
 ok: false,
 detail: "???? ??????????? ?? ?????",
 },
);

const sent = results.filter((r) => r.ok).length;
const summary = results
 .map((r) => `${PLATFORM_LABEL[r.platform] ?? r.platform}: ${r.ok ? "?? ????? ??": "?? ${r.detail}}`)
 .join(" | ");

return { total: results.length, sent, failed: results.length - sent, summary, results };
}

/** ????? ?? ????? ?? ?? ?????????????????? */
export async function dispatchOrder(order: OrderLike): Promise<string> {
 const text = await formatOrderMessage(order);
 const report = await dispatchText(text, order.id);
 return report.summary;
}

/* ----- */
/* ??? chat_id (????? / ???) */
/* ----- */

export type ChatInfo = { id: string; title: string; type: string };

export async function fetchUpdates(platform: string, rowToken?: string) {
 const token = await resolveToken(platform, rowToken);
 if (!token) return { ok: false, detail: "???? ???? ?????", chats: [] as ChatInfo[] };

 const bases =
 platform === "telegram"
 ? ["https://api.telegram.org"]
 : platform === "bale"
 ? ["https://tapi.bale.ai", "https://api.bale.ai"]
 : [];
 if (bases.length === 0) return { ok: false, detail: "??? ?????????? ?? ?????", chats: [] as ChatInfo[] };

 const chats = new Map<string, ChatInfo>();
 let lastErr = "";

 const consume = (raw: string) => {
 try {
 const j = JSON.parse(raw);
 for (const u of j?.result ?? []) {
 const c = u?.message?.chat ?? u?.channel_post?.chat ?? u?.edited_message?.chat;
 if (!c?.id) continue;
 chats.set(String(c.id), {
 id: String(c.id),
 title: c.title || [c.first_name, c.last_name].filter(Boolean).join(" ") || c.username || String(c.id),
 type: c.type === "private" ? "????": c.type === "group" || c.type === "supergroup" ? "????": "????",
 });
 }
 return j?.ok !== false;
 } catch {
 return false;
 }
 };

 for (const base of bases) {
 try {
 const res = await fetchWithRetry(`${base}/bot${token}/getUpdates?limit=100`, { method: "GET" }, { retries: 2, time
outMs: 15_000 });
 const raw = await res.text();
 if (res.ok && consume(raw)) return { ok: true, detail: "", chats: [...chats.values()] };
 lastErr = raw.slice(0, 200);
 } catch (err) {
 lastErr = err instanceof Error ? err.message : "????";
 }
 }
}

```



```

const proxy = await getProxy();
if (proxy.enabled && proxy.url) {
 try {
 const res = await fetchWithRetry(
 proxy.url,
 {
 method: "POST",
 headers: { "Content-Type": "application/json", ...(proxy.secret ? { "x-proxy-secret": proxy.secret } : {}) },
 body: JSON.stringify({ action: "getUpdates", messenger: platform === "eitaa" ? "ita" : platform, token }),
 },
 { retries: 2, timeoutMs: 15_000 },
);
 const raw = await res.text();
 if (res.ok && consume(raw)) return { ok: true, detail: "📱 📱📱 📱📱📱", chats: [...chats.values()] };
 lastErr = raw.slice(0, 200);
 } catch (err) {
 lastErr = err instanceof Error ? err.message : lastErr;
 }
}

return {
 ok: false,
 detail: `📱📱📱📱 📱📱 📱📱📱📱: ${lastErr}. 📱📱📱 📱 📱📱 📱📱📱📱 📱📱📱📱 📱 📱📱 📱📱 📱📱📱.`,
 chats: [] as ChatInfo[],
};
}

@@ src/lib/mobile-auth.ts typescript
import { createHash, createHmac, randomBytes, timingSafeEqual } from "crypto";
import { db, dbRetrySafe } from "@db";
import { mobileNonces } from "@db/schema";
import { and, eq, gt, lt } from "drizzle-orm";

/* =====
* 📱📱 📱📱📱📱 📱📱 📱 📱📱📱📱📱 📱📱📱📱
*
* 📱 📱📱 📱📱📱📱:
* 📱) 📱📱📱📱 HMAC-SHA256 📱📱 📱📱📱 📱📱📱📱 📱 📱📱 📱📱📱 📱
* 📱) nonce 📱📱📱📱📱📱 📱 📱📱📱📱 📱📱📱 (📱 Replay Attack)
* 📱) 📱📱 📱📱📱 📱📱 📱 📱📱📱📱📱📱📱 📱📱📱
* ===== */

/** 📱📱 📱📱📱 📱 📱📱📱 — 📱 Render 📱📱📱📱📱 MOBILE_APP_SECRET 📱📱📱 📱📱 */
const APP_SECRET = process.env.MOBILE_APP_SECRET || process.env.APP_SECRET || "sek-mobile-shared-secret";

export const NONCE_TTL_MS = 3 * 60 * 1000; // 📱 📱📱📱
export const MAX_CLOCK_SKEW_MS = 5 * 60 * 1000; // 📱📱📱📱 📱📱 📱📱📱 📱📱

/** 📱📱📱 nonce 📱 📱📱📱 📱 📱📱 📱📱 📱📱📱📱 */
export async function issueNonce(deviceId: string) {
 const nonce = randomBytes(24).toString("base64url");
 const expiresAt = new Date(Date.now() + NONCE_TTL_MS);

 await dbRetrySafe(
 () => db.insert(mobileNonces).values({ nonce, deviceId: deviceId.slice(0, 80), expiresAt }),
 undefined,
 "nonce:issue",
);

 // 📱📱📱📱 nonce📱📱 📱📱📱 (📱📱📱📱 📱📱 📱📱📱)
 void dbRetrySafe(
 () => db.delete(mobileNonces).where(lt(mobileNonces.expiresAt, new Date(Date.now() - NONCE_TTL_MS))),
 undefined,
 "nonce:cleanup",
);

 return { nonce, expiresAt, ttlMs: NONCE_TTL_MS };
}

/** 📱📱 nonce — 📱📱 📱 📱📱 📱📱📱 📱📱 */
export async function consumeNonce(nonce: string): Promise<boolean> {
 if (!nonce) return false;
 const rows = await dbRetrySafe(
 () =>
 db
 .select()
 .from(mobileNonces)
 .where(and(eq(mobileNonces.nonce, nonce), eq(mobileNonces.used, false), gt(mobileNonces.expiresAt, new Date()))))
 .limit(1),
 [],
 "nonce:consume",
);
 if (rows.length === 0) return false;
 await dbRetrySafe(
 () => db.update(mobileNonces).set({ used: true }).where(eq(mobileNonces.id, rows[0].id)),
);
}

```

```

 undefined,
 "nonce:mark",
);
 return true;
}

/** 00000000 00 00 0 0000 00 00 00000 000000000 0 0000 00000000 */
export function canonicalPayload(input: {
 nonce: string;
 timestamp: number;
 deviceId: string;
 phoneNumber?: string;
 simFingerprint?: string;
}) {
 return [
 input.nonce,
 String(input.timestamp),
 input.deviceId,
 input.phoneNumber ?? "",
 input.simFingerprint ?? "",
].join("|");
}

export function signPayload(canonical: string) {
 return createHmac("sha256", APP_SECRET).update(canonical).digest("base64url");
}

export function verifySignature(canonical: string, signature: string) {
 if (!signature) return false;
 const expected = signPayload(canonical);
 const a = Buffer.from(expected);
 const b = Buffer.from(signature);
 if (a.length !== b.length) return false;
 return timingSafeEqual(a, b);
}

/** 00 0000 00000 000000000 00 00000 000 (ICCID/IMSI) 00 00000000 00000 0000 */
export function hashSim(value: string) {
 if (!value) return "";
 return createHash("sha256").update(`${value}|${APP_SECRET}`).digest("hex").slice(0, 64);
}

export function normalizePhone(p: string) {
 return String(p ?? "")
 .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
 .replace(/\D/g, "")
 .replace(/^0098/, "0")
 .replace(/^98/, "0")
 .replace(/^(?!0)(9\d{9})$/, "0$1");
}

export function maskPhone(p: string) {
 const d = normalizePhone(p);
 return d.length >= 8 ? `${d.slice(0, 4)}***${d.slice(-4)}` : "";
}

export type AttestationResult =
 | { ok: true }
 | { ok: false; status: number; error: string; code: string };

/**
 * 00000000000 0000 00000000 00 000000: 0000 + nonce + 000 000000.
 * 00 0000 000000 00000000 00 MOBILE_ALLOW_UNSIGNED=1 0000000 00000000 0000.
 */
export async function verifyAttestation(body: {
 nonce?: string;
 timestamp?: number;
 signature?: string;
 deviceId?: string;
 phoneNumber?: string;
 simFingerprint?: string;
}): Promise<AttestationResult> {
 const allowUnsigned = process.env.MOBILE_ALLOW_UNSIGNED === "1";

 const nonce = String(body.nonce ?? "");
 const timestamp = Number(body.timestamp ?? 0);
 const deviceId = String(body.deviceId ?? "");

 if (!deviceId) {
 return { ok: false, status: 400, error: "000000 00000000 000000 0000 0000", code: "NO_DEVICE_ID" };
 }

 if (allowUnsigned) return { ok: true };

 if (!Number.isFinite(timestamp) || Math.abs(Date.now() - timestamp) > MAX_CLOCK_SKEW_MS) {

```

```

 return {
 ok: false,
 status: 401,
 error: "Қаталықпен бірігіп қолдануға болмайды. Қаталықпен бірігіп қолдануға болмайды.",
 code: "CLOCK_SKEW",
 };
 }

 const canonical = canonicalPayload({
 nonce,
 timestamp,
 deviceId,
 phoneNumber: body.phoneNumber,
 simFingerprint: body.simFingerprint,
 });

 if (!verifySignature(canonical, String(body.signature ?? ""))) {
 return { ok: false, status: 401, error: "Қаталықпен бірігіп қолдануға болмайды", code: "BAD_SIGNATURE" };
 }

 if (!(await consumeNonce(nonce))) {
 return {
 ok: false,
 status: 401,
 error: "Қаталықпен бірігіп қолдануға болмайды. Қаталықпен бірігіп қолдануға болмайды.",
 code: "BAD_NONCE",
 };
 }

 return { ok: true };
}

```

```

@@ src/lib/notify.ts typescript
import { db } from "@/db";
import { notifications } from "@/db/schema";

type NotifyInput = {
 toUserId?: number | null;
 toRole?: "admin" | "supervisor" | "rep" | "";
 fromName?: string;
 kind?: string;
 title: string;
 body?: string;
 link?: string;
};

/** Қаталықпен бірігіп қолдануға болмайды. Қаталықпен бірігіп қолдануға болмайды. */
export async function notify(input: NotifyInput) {
 try {
 await db.insert(notifications).values({
 toUserId: input.toUserId ?? null,
 toRole: input.toRole ?? "",
 fromName: input.fromName ?? "",
 kind: input.kind ?? "info",
 title: input.title.slice(0, 200),
 body: (input.body ?? "").slice(0, 2000),
 link: (input.link ?? "").slice(0, 200),
 });
 } catch {
 /* ignore */
 }
}

```

```

@@ src/lib/offline-session.ts typescript
"use client";

/**
 * Қаталықпен бірігіп қолдануға болмайды. Қаталықпен бірігіп қолдануға болмайды.
 * Қаталықпен бірігіп қолдануға болмайды.
 * sessionStorage → Қаталықпен бірігіп қолдануға болмайды. Қаталықпен бірігіп қолдануға болмайды.
 * localStorage → Қаталықпен бірігіп қолдануға болмайды. Қаталықпен бірігіп қолдануға болмайды.
 */

export type CachedUser = {
 id: number;
 username: string;
 fullName: string;
 role: "admin" | "supervisor" | "rep";
 roleKey: string;
 roleLabel: string;
 phone: string;
 requirePhone: boolean;
 permissions: string[];
};

```

```

const TOKEN_TAB = "sek_token";
const TOKEN_PERSIST = "sek_token_p";
const USER_CACHE = "sek_user_cache";
const CACHE_AT = "sek_user_cache_at";
/** 000000 0000 000000: 00 000 */
const MAX_AGE = 14 * 24 * 60 * 60 * 1000;

const safeGet = (store: Storage | undefined, k: string) => {
 try {
 return store?.getItem(k) ?? null;
 } catch {
 return null;
 }
};

const safeSet = (store: Storage | undefined, k: string, v: string) => {
 try {
 store?.setItem(k, v);
 } catch {
 /* ignore */
 }
};

const safeDel = (store: Storage | undefined, k: string) => {
 try {
 store?.removeItem(k);
 } catch {
 /* ignore */
 }
};

export function saveToken(token: string, persist: boolean) {
 if (typeof window === "undefined") return;
 safeSet(sessionStorage, TOKEN_TAB, token);
 if (persist) safeSet(localStorage, TOKEN_PERSIST, token);
}

export function getToken(): string | null {
 if (typeof window === "undefined") return null;
 return safeGet(sessionStorage, TOKEN_TAB) ?? safeGet(localStorage, TOKEN_PERSIST);
}

export function clearToken() {
 if (typeof window === "undefined") return;
 safeDel(sessionStorage, TOKEN_TAB);
 safeDel(localStorage, TOKEN_PERSIST);
 safeDel(localStorage, USER_CACHE);
 safeDel(localStorage, CACHE_AT);
}

export function cacheUser(u: CachedUser) {
 if (typeof window === "undefined") return;
 safeSet(localStorage, USER_CACHE, JSON.stringify(u));
 safeSet(localStorage, CACHE_AT, String(Date.now()));
}

/** 00000 0000000000 0000 00000 00 0000 00 00000 0000 */
export function getCachedUser(): CachedUser | null {
 if (typeof window === "undefined") return null;
 const raw = safeGet(localStorage, USER_CACHE);
 const at = Number(safeGet(localStorage, CACHE_AT) ?? 0);
 if (!raw) return null;
 if (at && Date.now() - at > MAX_AGE) return null;
 // 0000 000000 000 0000 00000 000 00 00000 00 00000 000 0000
 if (!getToken()) return null;
 try {
 return JSON.parse(raw) as CachedUser;
 } catch {
 return null;
 }
}

@@ src/lib/otp.ts typescript
import { createHash, randomInt } from "crypto";
import { db } from "@db";
import { messengers, otpCodes, settings } from "@db/schema";
import { and, desc, eq, gt } from "drizzle-orm";
import { sendOne } from "../messaging";
import { fetchWithRetry } from "../retry";

export const OTP_TTL_MS = 3 * 60 * 1000; // 0 00000
export const OTP_MAX_ATTEMPTS = 5;

export function hashCode(code: string, phone: string) {
 const secret = process.env.APP_SECRET || "sabt-etelaat-kol-default-secret-key";
 return createHash("sha256").update(`${code}|${phone}|${secret}`).digest("hex");
}

```

```

export function generateCode() {
 return String(randomInt(10000, 100000)); // 0 0000
}

type SmsConfig = { provider: string; apiKey: string; sender: string; pattern: string; enabled: boolean };

export const DEFAULT_SMS: SmsConfig = {
 provider: "kavenegar",
 apiKey: "",
 sender: "",
 pattern: "",
 enabled: false,
};

export async function getSmsConfig(): Promise<SmsConfig> {
 try {
 const rows = await db.select().from(settings).where(eq(settings.key, "sms")).limit(1);
 const v = rows[0]?.value as Partial<SmsConfig> | undefined;
 if (v) return { ...DEFAULT_SMS, ...v };
 } catch {
 /* ignore */
 }
 return {
 ...DEFAULT_SMS,
 provider: process.env.SMS_PROVIDER || DEFAULT_SMS.provider,
 apiKey: process.env.SMS_API_KEY || "",
 sender: process.env.SMS_SENDER || "",
 pattern: process.env.SMS_PATTERN || "",
 enabled: Boolean(process.env.SMS_API_KEY),
 };
}

/** 00000 00000 00 00000 00000 00 0000 0000 000000 */
async function post(url: string, init?: RequestInit, ms = 12000) {
 return fetchWithRetry(url, init ?? {}, { timeoutMs: ms, retries: 3, baseDelayMs: 900, maxDelayMs: 6_000, label: "sms"
});
}

/** 00000 00000 00 0000 0000000000000000 0000 000000 */
async function sendSms(phone: string, code: string): Promise<{ ok: boolean; detail: string }> {
 const cfg = await getSmsConfig();
 if (!cfg.enabled || !cfg.apiKey) return { ok: false, detail: "00000 00000 00000 0000 0000" };
 const text = `00 00000 0000 000 00 «000 00000000 00»: ${code}\n0000000: 0 00000`;

 try {
 if (cfg.provider === "kavenegar") {
 const base = `https://api.kavenegar.com/v1/${cfg.apiKey}`;
 const url = cfg.pattern
 ? `${base}/verify/lookup.json?receptor=${phone}&token=${code}&template=${cfg.pattern}`
 : `${base}/sms/send.json?receptor=${phone}&sender=${cfg.sender}&message=${encodeURIComponent(text)}`;
 const r = await post(url, { method: "GET" });
 const raw = (await r.text()).slice(0, 200);
 return r.ok ? { ok: true, detail: "00000 00" } : { ok: false, detail: raw };
 }
 if (cfg.provider === "smsir") {
 const r = await post("https://api.sms.ir/v1/send/verify", {
 method: "POST",
 headers: { "Content-Type": "application/json", "x-api-key": cfg.apiKey },
 body: JSON.stringify({
 mobile: phone,
 templateId: Number(cfg.pattern) || 100000,
 parameters: [{ name: "CODE", value: code }],
 }),
 });
 const raw = (await r.text()).slice(0, 200);
 return r.ok ? { ok: true, detail: "00000 00" } : { ok: false, detail: raw };
 }
 if (cfg.provider === "melipayamak") {
 const r = await post("https://console.melipayamak.com/api/send/simple/" + cfg.apiKey, {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ from: cfg.sender, to: phone, text }),
 });
 const raw = (await r.text()).slice(0, 200);
 return r.ok ? { ok: true, detail: "00000 00" } : { ok: false, detail: raw };
 }
 // 000000000000 000000: 0000 00 000000000000 {phone} 0 {code}
 if (cfg.provider === "custom" && cfg.sender) {
 const url = cfg.sender.replace("{phone}", phone).replace("{code}", code);
 const r = await post(url, { method: "GET" });
 return r.ok ? { ok: true, detail: "00000 00" } : { ok: false, detail: `00 ${r.status}` };
 }
 } catch (e) {
 return { ok: false, detail: "000000000000 00000 000000000 00000000" };
 }
}

```

```

 return { ok: false, detail: e instanceof Error ? e.message : "???? ???? " };
 }
}

/** ?????? ?? ?? ????? ??????????? (??/??????) ?? ????? ????????? ????? ?????? */
async function sendViaMessenger(phone: string, code: string): Promise<{ ok: boolean; detail: string }> {
 try {
 const rows = await db.select().from(messengers);
 const target = rows.find((m) => m.enabled && m.target.replace(/\D/g, "").endsWith(phone.slice(-9)));
 if (!target) return { ok: false, detail: "???? ??????????? ????? ?? ?????? ?? ?????? " };
 const r = await sendOne(
 { platform: target.platform, target: target.target, token: target.token },
 `?? ?????? ????? ?? ?? «???? ????????? ??»: ${code}\n????????: ? ??????`,
);
 return r;
 } catch {
 return { ok: false, detail: "?????? ?? ????? ??????????? ??????? ?? " };
 }
}

export async function issueOtp(userId: number, phone: string, deviceId: string) {
 const code = generateCode();
 const expiresAt = new Date(Date.now() + OTP_TTL_MS);

 let channel = "sms";
 let res = await sendSms(phone, code);
 if (!res.ok) {
 const alt = await sendViaMessenger(phone, code);
 if (alt.ok) {
 channel = "messenger";
 res = alt;
 }
 }

 await db.insert(otpCodes).values({
 userId,
 phone,
 codeHash: hashCode(code, phone),
 deviceId,
 channel: res.ok ? channel : "manual",
 expiresAt,
 });

 return { sent: res.ok, channel: res.ok ? channel : "manual", detail: res.detail, code };
}

export async function verifyOtp(userId: number, phone: string, code: string) {
 const rows = await db
 .select()
 .from(otpCodes)
 .where(and(eq(otpCodes.userId, userId), eq(otpCodes.used, false), gt(otpCodes.expiresAt, new Date()))))
 .orderBy(desc(otpCodes.id))
 .limit(1);
 const row = rows[0];
 if (!row) return { ok: false, error: "?? ?????? ?? ???. ?????? ?????????? " };
 if (row.attempts >= OTP_MAX_ATTEMPTS) return { ok: false, error: "????? ?????? ?? ?? ?? ???. ?? ?????? " };

 if (row.codeHash !== hashCode(String(code).trim(), phone)) {
 await db.update(otpCodes).set({ attempts: row.attempts + 1 }).where(eq(otpCodes.id, row.id));
 return { ok: false, error: `?? ?????? ?? ?????? ?? (${OTP_MAX_ATTEMPTS - row.attempts - 1} ?????? ?????? )` };
 }
 await db.update(otpCodes).set({ used: true }).where(eq(otpCodes.id, row.id));
 return { ok: true, deviceId: row.deviceId };
}

@@ src/lib/pwa-image.ts typescript
import { readFile } from "node:fs/promises";
import path from "node:path";

const PUB = path.join(process.cwd(), "public");

/** ?????? ?????????? ?? ?????? ?????? ?????? ?? ?????? */
const FALLBACK_LOGO = `<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512" width="512" height="512">
<defs>
 <linearGradient id="pill" x1="0" y1="1" x2="1" y2="0">
 <stop offset="0%" stop-color="#F9A03F"/><stop offset="35%" stop-color="#F7714B"/>
 <stop offset="70%" stop-color="#EC3B75"/><stop offset="100%" stop-color="#C81E63"/>
 </linearGradient>
 <linearGradient id="glossTop" x1="0" y1="0" x2="0" y2="1">
 <stop offset="0%" stop-color="ffffff" stop-opacity=".45"/>
 <stop offset="55%" stop-color="ffffff" stop-opacity="0"/>
 </linearGradient>
</defs>
<g transform="translate(256 256) rotate(-45)">
 <rect x="-168" y="-84" width="336" height="168" rx="84" ry="84" fill="url(#pill)"/>
</g>
</svg>`;

```

```

<line x1="0" y1="-84" x2="0" y2="84" stroke="#ffffff" stroke-opacity=".35" stroke-width="6"/>
<rect x="-160" y="-76" width="320" height="80" rx="40" ry="40" fill="url(#glossTop)"/>
<circle cx="104" cy="-30" r="18" fill="#ffffff" fill-opacity=".92"/>
</g>
</svg>`;

const cache = new Map<string, Uint8Array>();

async function loadSvg(rel: string, fallback: string) {
 try {
 return await readFile(path.join(PUB, rel), "utf8");
 } catch {
 return fallback;
 }
}

// eslint-disable-next-line @typescript-eslint/no-explicit-any
async function getSharp(): Promise<any | null> {
 try {
 const mod = await import("sharp");
 return mod.default;
 } catch {
 return null;
 }
}

const plate = (size: number) =>
 Buffer.from(
 `<svg xmlns="http://www.w3.org/2000/svg" width="${size}" height="${size}">
 <defs><linearGradient id="g" x1="0" y1="0" x2="1" y2="1">
 <stop offset="0%" stop-color="#ffffff"/><stop offset="100%" stop-color="#eef2f6"/></linearGradient></defs>
 <rect width="${size}" height="${size}" fill="url(#g)"/></svg>`,
);

/**
 * PNG
 */
export async function renderIcon(size: number, maskable = false): Promise<Uint8Array | null> {
 const key = `${maskable ? "m" : "i"}-${size}`;
 const hit = cache.get(key);
 if (hit) return hit;

 const sharp = await getSharp();
 const svg = await loadSvg("logo.svg", FALLBACK_LOGO);
 if (!sharp) return null;

 let out: Buffer;
 if (maskable) {
 const inner = await sharp(Buffer.from(svg), { density: 500 })
 .resize(Math.round(size * 0.62), Math.round(size * 0.62))
 .png()
 .toBuffer();
 out = await sharp(plate(size)).composite([
 { input: inner, gravity: "center" }
]).png().toBuffer();
 } else {
 out = await sharp(Buffer.from(svg), { density: 500 })
 .resize(size, size, { fit: "contain", background: { r: 0, g: 0, b: 0, alpha: 0 } })
 .png({ compressionLevel: 9 })
 .toBuffer();
 }
 const bytes = new Uint8Array(out);
 cache.set(key, bytes);
 return bytes;
}

/**
 * PNG
 */
export async function renderScreenshot(name: string): Promise<Uint8Array | null> {
 const hit = cache.get(`s-${name}`);
 if (hit) return hit;
 const sharp = await getSharp();
 if (!sharp) return null;
 const svg = await loadSvg(`src-svg/${name}.svg`, "");
 if (!svg) return null;
 const out = await sharp(Buffer.from(svg)).png().toBuffer();
 const bytes = new Uint8Array(out);
 cache.set(`s-${name}`, bytes);
 return bytes;
}

export function pngResponse(bytes: Uint8Array) {
 return new Response(bytes as unknown as BodyInit, {
 headers: {
 "Content-Type": "image/png",
 "Content-Length": String(bytes.length),
 "Cache-Control": "public, max-age=31536000, immutable",
 "Access-Control-Allow-Origin": "",
 },
 });
}

```

```

}

@@ src/lib/retry.ts typescript
/**
 * =====
 * Retry 指数 Exponential Backoff + Jitter
 * -----
 * 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时
 * Neon (cold start) 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时。
 *
 * 数据库:
 * - 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 (数据库连接池在冷启动时 5x429)。
 * - 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 (unique violation 4xx syntax) 数据库连接池在冷启动时，数据库连接池在冷启动时
 * 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时。
 * - 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时。
 * =====
 */

export type RetryOptions = {
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 retries?: number;
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 baseDelayMs?: number;
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 maxDelayMs?: number;
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 timeoutMs?: number;
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 label?: string;
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 isRetryable?: (error: unknown) => boolean;
 /** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
 onRetry?: (info: { attempt: number; delayMs: number; error: unknown }) => void;
 signal?: AbortSignal;
};

const DEFAULTS: Required<Omit<RetryOptions, "isRetryable" | "onRetry" | "signal" | "label">> = {
 retries: 4,
 baseDelayMs: 1_000,
 maxDelayMs: 30_000,
 timeoutMs: 20_000,
};

/** 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时 */
export const TRANSIENT_NETWORK_CODES = new Set([
 "ECONNRESET",
 "ETIMEDOUT",
 "ECONNREFUSED",
 "EPIPE",
 "EHOSTUNREACH",
 "ENETUNREACH",
 "ENETDOWN",
 "EAI_AGAIN",
 "ENOTFOUND",
 "EADDRNOTAVAIL",
 "ECONNABORTED",
 "UND_ERR_CONNECT_TIMEOUT",
 "UND_ERR_HEADERS_TIMEOUT",
 "UND_ERR_SOCKET",
]);

/**
 * 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时
 * 57P = 数据库连接池在冷启动时，数据库连接池在冷启动时 40001/40P01 = 数据库连接池在冷启动时
 */
export const TRANSIENT_PG_CODES = new Set([
 "08000", "08001", "08003", "08004", "08006", "08007", "08P01",
 "53300", "53400", "57P01", "57P02", "57P03",
 "40001", "40P01",
 "XX000", // internal error — 数据库连接池在冷启动时，数据库连接池在冷启动时，数据库连接池在冷启动时
]);

const TRANSIENT_MESSAGE_HINTS = [
 "timeout",
 "timed out",
 "connection terminated",
 "connection closed",
 "connection reset",
 "socket hang up",
 "server closed the connection",
 "terminating connection",
 "too many connections",
 "fetch failed",
 "network",
 "temporarily unavailable",
];

```



```

"endpoint is disabled", // 接口 Neon 数据库 连接 失败 提示信息
"compute time exceeded",
"could not connect",
];

function errCode(error: unknown): string {
 if (!error || typeof error !== "object") return "";
 const e = error as { code?: unknown; errno?: unknown; cause?: unknown };
 if (typeof e.code === "string") return e.code;
 if (typeof e.errno === "string") return e.errno;
 if (e.cause) return errCode(e.cause);
 return "";
}

function errMsg(error: unknown): string {
 if (error instanceof Error) return `${error.message} ${errMsg((error as Error & { cause?: unknown }).cause ?? "")}`;
 if (typeof error === "string") return error;
 return "";
}

/** 网络 错误 类型 判断 函数 */
export function isTransientError(error: unknown): boolean {
 if (!error) return false;

 // 网络 错误 类型 判断 函数
 if (error instanceof Error && error.name === "AbortError") return false;

 const code = errCode(error).toUpperCase();
 if (TRANSIENT_NETWORK_CODES.has(code)) return true;
 if (TRANSIENT_PG_CODES.has(errCode(error))) return true;

 // 网络 HTTP: 5xx 或 429 或 408
 const status = (error as { status?: number; statusCode?: number })?.status ??
 (error as { statusCode?: number })?.statusCode;
 if (typeof status === "number") {
 if (status >= 500 || status === 429 || status === 408) return true;
 return false;
 }

 const msg = errMsg(error).toLowerCase();
 return TRANSIENT_MESSAGE_HINTS.some((h) => msg.includes(h));
}

/** 网络 错误 类型 判断 函数 (Full Jitter) */
export function backoffDelay(attempt: number, baseDelayMs: number, maxDelayMs: number): number {
 const exponential = Math.min(maxDelayMs, baseDelayMs * 2 ** (attempt - 1));
 // Full Jitter: 网络 错误 类型 判断 函数 网络 错误 类型 判断 函数 网络 错误 类型 判断 函数
 const half = exponential / 2;
 return Math.round(half + Math.random() * half);
}

const sleep = (ms: number, signal?: AbortSignal) =>
 new Promise<void>((resolve, reject) => {
 const t = setTimeout(resolve, ms);
 signal?.addEventListener(
 "abort",
 () => {
 clearTimeout(t);
 reject(new DOMException("Aborted", "AbortError"));
 },
 { once: true },
);
 });

function withTimeout<T>(fn: (signal: AbortSignal) => Promise<T>, ms: number, outer?: AbortSignal): Promise<T> {
 if (!ms) return fn(outer ?? new AbortController().signal);
 const ctrl = new AbortController();
 const timer = setTimeout(() => ctrl.abort(new DOMException("Timeout", "TimeoutError")), ms);
 outer?.addEventListener("abort", () => ctrl.abort(), { once: true });
 return fn(ctrl.signal).finally(() => clearTimeout(timer));
}

/**
 * 数据库 连接 失败 提示信息
 *
 * @example
 * const rows = await withRetry(() => db.select().from(users), { label: "users:list" });
 */
export async function withRetry<T>(
 operation: (attempt: number, signal: AbortSignal) => Promise<T>,
 options: RetryOptions = {},
): Promise<T> {
 const cfg = { ...DEFAULTS, ...options };
 const retryable = options.isRetryable ?? isTransientError;

```

```

let lastError: unknown;

for (let attempt = 1; attempt <= cfg.retries; attempt++) {
 try {
 return await withTimeout((signal) => operation(attempt, signal), cfg.timeoutMs, options.signal);
 } catch (error) {
 lastError = error;

 const isLast = attempt >= cfg.retries;
 // TimeoutError 是否 是否 是否 是否
 const timedOut = error instanceof Error && error.name === "TimeoutError";
 if (isLast || !(timedOut || retryable(error))) throw error;

 const delayMs = backoffDelay(attempt, cfg.baseDelayMs, cfg.maxDelayMs);
 options.onRetry?.({ attempt, delayMs, error });
 console.warn(
 `[retry${options.label ? `:${options.label}` : ``}] 尝试 ${attempt}/${cfg.retries} 失败 失败` +
 `(${errCode(error) || errMsg(error).slice(0, 80)}) — 等待 ${delayMs}ms 等待`
);
 await sleep(delayMs, options.signal);
 }
}
throw lastError;
}

/** 重试 重试 重试 重试 重试 重试 重试 重试 重试 重试 */
export async function withRetrySafe<T>({
 operation: (attempt: number, signal: AbortSignal) => Promise<T>,
 fallback: T,
 options: RetryOptions = {},
}): Promise<T> {
 try {
 return await withRetry(operation, options);
 } catch (error) {
 console.error(`[retry${options.label ? `:${options.label}` : ``}] 失败 失败: ${errMsg(error).slice(0, 200)}`);
 return fallback;
 }
}

/** fetch 重试 重试 重试 重试 重试 重试 重试 重试 重试 重试 */
export async function fetchWithRetry({
 input: string | URL,
 init: RequestInit = {},
 options: RetryOptions = {},
}): Promise<Response> {
 return withRetry(
 async (_attempt, signal) => {
 const res = await fetch(input, { ...init, signal: init.signal ?? signal, cache: init.cache ?? "no-store" });
 // 5xx / 429 / 408 是否 是否 是否 是否 是否 是否 是否 是否 是否 是否
 if (res.status >= 500 || res.status === 429 || res.status === 408) {
 const err = new Error(`HTTP ${res.status} 错误 ${typeof input === "string" ? input : input.toString()}`);
 (err as Error & { status: number }).status = res.status;
 throw err;
 }
 return res;
 },
 { retries: 3, baseDelayMs: 800, maxDelayMs: 8_000, timeoutMs: 15_000, ...options },
);
}

@@ src/lib/settings-server.ts typescript
import { db } from "@db";
import { settings } from "@db/schema";
import { DEFAULT_PRODUCTS, type ProductConfig } from "../defaults";
import { eq } from "drizzle-orm";

/** 重试 重试 重试 重试 重试 重试 重试 重试 重试 重试 */
export async function getProducts(): Promise<ProductConfig[]> {
 try {
 const rows = await db.select().from(settings).where(eq(settings.key, "products")).limit(1);
 const v = rows[0]?.value as ProductConfig[] | undefined;
 if (Array.isArray(v) && v.length) return v.filter((p) => p.enabled !== false);
 } catch {
 /* ignore */
 }
 return DEFAULT_PRODUCTS;
}

@@ src/lib/sort.ts typescript
/** 重试 重试 重试 重试 重试 重试 重试 重试 重试 重试 */
const collator = new Intl.Collator("fa", {
 numeric: true,
 sensitivity: "base",
 ignorePunctuation: true,
});

```

```

export function faCompare(a: string, b: string) {
 return collator.compare(String(a ?? ""), String(b ?? ""));
}

export function faSort(values: string[]) {
 return [...new Set(values.filter(Boolean))].sort(faCompare);
}

/** ۰۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰ */
export function toEnglishDigits(value: string) {
 return String(value)
 .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06F0))
 .replace(/[\u0660-\u0669]/g, (c) => String(c.charCodeAt(0) - 0x0660));
}

@@ src/lib/sound.ts typescript
/** ۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰ (WebAudio) — ۰۰۰ ۰ ۰۰۰۰ ۰۰۰۰۰۰ */
export function playNotificationSound() {
 try {
 const Ctx =
 window.AudioContext ||
 (window as unknown as { webkitAudioContext: typeof AudioContext }).webkitAudioContext;
 if (!Ctx) return;
 const ctx = new Ctx();
 const now = ctx.currentTime;
 const notes = [880, 1174.7];
 notes.forEach((f, i) => {
 const osc = ctx.createOscillator();
 const gain = ctx.createGain();
 osc.type = "sine";
 osc.frequency.value = f;
 gain.gain.setValueAtTime(0.0001, now + i * 0.18);
 gain.gain.exponentialRampToValueAtTime(0.25, now + i * 0.18 + 0.03);
 gain.gain.exponentialRampToValueAtTime(0.0001, now + i * 0.18 + 0.22);
 osc.connect(gain).connect(ctx.destination);
 osc.start(now + i * 0.18);
 osc.stop(now + i * 0.18 + 0.24);
 });
 setTimeout(() => ctx.close().catch(() => undefined), 900);
 } catch {
 /* ignore */
 }
}

export function vibrate(pattern: number | number[] = [120, 60, 120]) {
 try {
 navigator.vibrate?.(pattern);
 } catch {
 /* ignore */
 }
}

@@ src/lib/sync-config.ts typescript
/**
 * =====
 * ۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰
 * Render (namayandeelmi-javad.onrender.com) ۰ NdcoHub.ir
 * -----
 * ۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰:
 * uid → ۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰ (UUID) ۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰
 * updated_at → ۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ (۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰)
 * origin → ۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰۰ ۰۰ ۰۰ ۰۰۰۰۰ ۰۰۰
 *
 * ۰۰۰۰۰۰۰۰: Last-Write-Wins ۰۰ ۰۰۰۰ updated_at
 * =====
 */

export type SyncTable = {
 /** ۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰ */
 name: string;
 /** ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰ */
 label: string;
 /** ۰۰۰۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ (۰۰۰۰۰ ۰۰۰۰) */
 skipColumns?: string[];
 /** ۰۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰ (۰۰۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰۰) */
 defaultOn: boolean;
 /** ۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰ ۰۰۰ ۰۰۰۰۰۰۰۰۰ */
 batch: number;
};

export const SYNC_TABLES: SyncTable[] = [
 { name: "users", label: "۰۰۰۰۰۰۰", defaultOn: true, batch: 500 },
 { name: "roles", label: "۰۰۰۰۰۰", defaultOn: true, batch: 200 },
 { name: "options", label: "۰۰۰۰۰۰۰۰ ۰۰۰۰", defaultOn: true, batch: 2000 },

```

```

{ name: "settings", label: "⚙️", defaultOn: true, batch: 200 },
{ name: "pharmacies", label: "🏪", defaultOn: true, batch: 1000 },
{ name: "doctors", label: "👨‍⚕️", defaultOn: true, batch: 1000 },
{ name: "orders", label: "📦", defaultOn: true, batch: 1000 },
{ name: "targets", label: "🎯", defaultOn: true, batch: 500 },
{ name: "homes", label: "🏠", defaultOn: true, batch: 500 },
{ name: "leaves", label: "🌿", defaultOn: true, batch: 500 },
{ name: "messengers", label: "📬", defaultOn: true, batch: 200 },
{ name: "notifications", label: "🔔", defaultOn: true, batch: 500 },
{ name: "activity logs", label: "📝", defaultOn: false, batch: 500 },
{ name: "trips", label: "🗺️", defaultOn: true, batch: 500 },
{ name: "trip_points", label: "📍", defaultOn: false, batch: 2000 },
{ name: "attachments", label: "📎", defaultOn: false, batch: 20 },
];

/** ⚙️ LOCAL ONLY COLUMNS (LOCAL ONLY) */
export const LOCAL_ONLY_COLUMNS = new Set(["id"]);

/** 🏠 Node Name — Node Render & NdcHub ⚙️ Node Name ⚙️ Node Name */
export function nodeName(): string {
 return (
 process.env.NODE_NAME ||
 process.env.SYNC_NODE_NAME ||
 (process.env.RENDER ? "render" : "") ||
 (process.env.VERCEL ? "vercel" : "") ||
 "primary"
);
};

/** 🌐 Public Base URL ⚙️ Node Name */
export function selfUrl(): string {
 return (
 process.env.PUBLIC_BASE_URL ||
 process.env.NEXT_PUBLIC_BASE_URL ||
 (process.env.RENDER_EXTERNAL_URL ?? "") ||
 ""
);
};

/** 🌐 Public Base URL ⚙️ Node Name: name|url,name|url */
export function configuredPeers(): { peer: string; url: string }[] {
 const raw = process.env.SYNC_PEERS || "";
 return raw
 .split(",")
 .map((s) => s.trim())
 .filter(Boolean)
 .map((entry) => {
 const [peer, url] = entry.split("|").map((x) => x.trim());
 return { peer: peer || "peer", url: url || peer };
 })
 .filter((p) => p.url.startsWith("http"));
}

/** 🌐 Public Base URL ⚙️ Node Name: name|url,name|url */
export function syncSecret(): string {
 return process.env.SYNC_SECRET || process.env.APP_SECRET || "";
}

/** 🌐 Public Base URL ⚙️ Node Name: name|url,name|url */
export const PUBLIC_ENDPOINTS: string[] = (
 process.env.NEXT_PUBLIC_ENDPOINTS || "https://ndcuhub.ir,https://namayandeelmi-javad.onrender.com"
)
 .split(",")
 .map((s) => s.trim().replace(/\/$/, ""))
 .filter(Boolean);

@@ src/lib/sync.ts typescript
import { createHmac, timingSafeEqual } from "crypto";
import { db, dbRetrySafe, pool } from "@db";
import { syncLogs, syncState } from "@db/schema";
import { fetchWithRetry } from "../retry";
import { LOCAL_ONLY_COLUMNS, SYNC_TABLES, configuredPeers, nodeName, selfUrl, syncSecret } from "../sync-config";
import { eq } from "drizzle-orm";

/* =====
* ⚙️ LOCAL ONLY COLUMNS (LOCAL ONLY)
*
* ⚙️ Node Name:
* 0) PULL → ⚙️ Node Name ⚙️ Node Name pullCursor ⚙️ Node Name ⚙️ Node Name
* 0) PUSH → ⚙️ Node Name ⚙️ Node Name pushCursor ⚙️ Node Name ⚙️ Node Name
*
* ⚙️ Node Name: Last-Write-Wins ⚙️ Node Name updated_at
* ⚙️ Node Name: ⚙️ Node Name ⚙️ Node Name origin ⚙️ Node Name ⚙️ Node Name ⚙️ Node Name ⚙️ Node Name
* ===== */

```

```

export type ChangeSet = Record<string, Record<string, unknown>[]>;

export type SyncReport = {
 peer: string;
 ok: boolean;
 pulled: number;
 pushed: number;
 conflicts: number;
 detail: string;
 durationMs: number;
};

const MAX_BODY_ROWS = 800;

/* ----- */
/* 000000 00000000 0000 00 00000 */
/* ----- */

export function signSync(body: string, timestamp: number) {
 return createHmac("sha256", syncSecret()).update(`${timestamp}.${body}`).digest("base64url");
}

export function verifySync(body: string, timestamp: number, signature: string) {
 if (!syncSecret()) return false;
 if (!signature) return false;
 if (Math.abs(Date.now() - timestamp) > 10 * 60 * 1000) return false;
 const expected = signSync(body, timestamp);
 const a = Buffer.from(expected);
 const b = Buffer.from(signature);
 if (a.length !== b.length) return false;
 return timingSafeEqual(a, b);
}

/* ----- */
/* 0000000 00000000 00000 */
/* ----- */

async function tableColumns(table: string): Promise<string[]> {
 const res = await pool.query(
 `select column_name from information_schema.columns where table_schema='public' and table_name=${table}`,
 [table],
);
 return res.rows.map((r) => String(r.column_name));
}

/**
 * 00000000 00000 00 00 00 00000 0000.
 *
 * 00000: 00 00000 origin 000000 000000000000 000 0000000 00 00000 00000 0 000000
 * 00000000 000 000000 00000 00000000. 00000000 00 00000 00 00 00000000 000000 00000000:
 * 0) 000000 00000000 00000 00000 000000 0000 00000000 (sek.sync = on)
 * 0) 0000 `updated_at` < excluded.updated_at` 000000 00000000 00 00 00000000
 */
export async function collectChanges(since: Date | null, excludeOrigin?: string): Promise<ChangeSet> {
 void excludeOrigin;
 const out: ChangeSet = {};
 const sinceIso = since ? since.toISOString() : "1970-01-01T00:00:00Z";

 for (const t of SYNC_TABLES) {
 if (!t.defaultOn) continue;
 try {
 const cols = await tableColumns(t.name);
 if (!cols.includes("uid") || !cols.includes("updated_at")) continue;

 const res = await pool.query(
 `select * from ${t.name} where updated_at > ${sinceIso} order by updated_at asc limit $2`,
 [sinceIso, t.batch],
);
 if (res.rows.length) out[t.name] = res.rows as Record<string, unknown>[];
 } catch (err) {
 console.warn(`[sync] 0000000 ${t.name} 0000000:`, err instanceof Error ? err.message : err);
 }
 }
 return out;
}

/* ----- */
/* 000000 000000000 000000000 */
/* ----- */

/** UPSERT 00 00000 uid 00 000000 Last-Write-Wins */
export async function applyChanges(changes: ChangeSet, fromPeer: string) {
 let applied = 0;
 let skipped = 0;
 // 00 000000 000000000 00 set_config 00 0000 00 00000000 000000 000000

```

```

const client = await pool.connect();
try {
 await client.query(`select set_config('sek.sync','on',false)`);
} catch {
 /* ignore */
}
const perTable: Record<string, { applied: number; skipped: number }> = {};

for (const [table, rows] of Object.entries(changes)) {
 const cfg = SYNC_TABLES.find((t) => t.name === table);
 if (!cfg || !Array.isArray(rows) || rows.length === 0) continue;

 let cols: string[] = [];
 try {
 cols = await tableColumns(table);
 } catch {
 continue;
 }
 if (!cols.includes("uid")) continue;

 perTable[table] = { applied: 0, skipped: 0 };

 for (const raw of rows.slice(0, MAX_BODY_ROWS)) {
 const row = { ...raw } as Record<string, unknown>;
 const uid = String(row.uid ?? "");
 if (!uid) {
 skipped++;
 perTable[table].skipped++;
 continue;
 }

 // 0000 0000000000 00 00 0000 0000 00000000
 for (const k of Object.keys(row)) {
 if (!cols.includes(k) || LOCAL_ONLY_COLUMNS.has(k)) delete row[k];
 }
 if (!row.origin) row.origin = fromPeer;

 const keys = Object.keys(row);
 if (keys.length === 0) continue;

 const placeholders = keys.map((_, i) => `${$${i + 1}}`).join(", ");
 const updates = keys
 .filter((k) => k !== "uid")
 .map((k) => `${k} = excluded.${k}`)
 .join(", ");

 try {
 // 0000 0000 00000000 00000000 0000 00000000 00000000.
 // set_config 0000 0000 000000 00 0000 00000000 00 000000 00 00 0000 0000.
 const q = `
 insert into ${table} (${keys.join(", ")}) values (${placeholders})
 on conflict (uid) do update set ${updates}
 where ${table}.updated_at < excluded.updated_at
 `;
 const res = await client.query(q, keys.map((k) => row[k]));
 if (res.rowCount && res.rowCount > 0) {
 applied++;
 perTable[table].applied++;
 } else {
 skipped++;
 perTable[table].skipped++;
 }
 } catch (err) {
 skipped++;
 perTable[table].skipped++;
 console.warn(`[sync] 000000 ${table}/${uid} 00000000`, err instanceof Error ? err.message.slice(0, 120) : err);
 }
 }
}

// 000000000000 0000 00 0000 000000 00 0000 0000 00000000 00000000 00 000000
// 0000 00000000 000000000000 0000 00000000 0000000000 0000 000000.
try {
 await client.query(`select set_config('sek.sync','off',false)`);
} catch {
 /* ignore */
}
client.release();

for (const [table, st] of Object.entries(perTable)) {
 void dbRetrySafe(
 () =>
 db.insert(syncLogs).values({
 peer: fromPeer,
 direction: "pull",
 })
);
}

```

```

 tableName: table,
 applied: st.applied,
 skipped: st.skipped,
 ok: true,
 }},
 undefined,
 "sync:log",
);
}

return { applied, skipped, perTable };
}

/* ----- */
/* */
/* ----- */

async function callPeer(url: string, path: string, payload: unknown) {
 const body = JSON.stringify(payload);
 const ts = Date.now();
 const res = await fetchWithRetry(
 `${url.replace(/\$/g, "")}${path}`,
 {
 method: "POST",
 headers: {
 "Content-Type": "application/json",
 "x-sync-node": nodeName(),
 "x-sync-timestamp": String(ts),
 "x-sync-signature": signSync(body, ts),
 },
 body,
 },
 { retries: 3, baseDelayMs: 1200, maxDelayMs: 10_000, timeoutMs: 45_000, label: "sync" },
);
 const text = await res.text();
 let data: Record<string, unknown> = {};
 try {
 data = JSON.parse(text);
 } catch {
 data = { raw: text.slice(0, 200) };
 }
 if (!res.ok) throw new Error(String(data.error ?? `HTTP ${res.status}`));
 return data;
}

/* ----- */
/* */
/* ----- */

export async function ensurePeers() {
 const peers = configuredPeers();
 for (const p of peers) {
 const existing = await dbRetrySafe(
 () => db.select().from(syncState).where(eq(syncState.peer, p.peer)).limit(1),
 [],
 "sync:peer",
);
 if (existing.length === 0) {
 await dbRetrySafe(
 () => db.insert(syncState).values({ peer: p.peer, peerUrl: p.url }),
 undefined,
 "sync:addPeer",
);
 } else if (existing[0].peerUrl !== p.url) {
 await dbRetrySafe(
 () => db.update(syncState).set({ peerUrl: p.url }).where(eq(syncState.id, existing[0].id)),
 undefined,
 "sync:updPeer",
);
 }
 }
 return dbRetrySafe(() => db.select().from(syncState), [], "sync:list");
}

/* ----- */
/* */
/* ----- */

export async function syncWithPeer(peerRow: typeof syncState.$inferSelect): Promise<SyncReport> {
 const started = Date.now();
 const me = nodeName();
 let pulled = 0;
 let pushed = 0;
 let conflicts = 0;
 const notes: string[] = [];

```

```

try {
 if (!syncSecret()) throw new Error("SYNC_SECRET 配置错误");
 if (!peerRow.peerUrl) throw new Error("peerUrl 配置错误");

 /* ----- PULL: 从对等节点拉取数据 ----- */
 const pullRes = await callPeer(peerRow.peerUrl, "/api/sync/pull", {
 node: me,
 since: peerRow.pullCursor ? new Date(peerRow.pullCursor).toISOString() : null,
 excludeOrigin: me, // 排除自己的数据
 });

 const changes = (pullRes.changes ?? {}) as ChangeSet;
 const applyRes = await applyChanges(changes, peerRow.peer);
 pulled = applyRes.applied;
 conflicts += applyRes.skipped;

 const newPullCursor = pullRes.cursor ? new Date(String(pullRes.cursor)) : new Date();
 notes.push(`${pulled} 已拉取`);

 /* ----- PUSH: 向对等节点推送数据 ----- */
 const localChanges = await collectChanges(
 peerRow.pushCursor ? new Date(peerRow.pushCursor) : null,
 peerRow.peer, // 推送给对等节点
);
 const rowCount = Object.values(localChanges).reduce((a, r) => a + r.length, 0);

 let newPushCursor = peerRow.pushCursor ? new Date(peerRow.pushCursor) : new Date(0);
 if (rowCount > 0) {
 const pushRes = await callPeer(peerRow.peerUrl, "/api/sync/push", { node: me, changes: localChanges });
 pushed = Number(pushRes.applied ?? 0);
 conflicts += Number(pushRes.skipped ?? 0);
 // 更新本地游标
 for (const rows of Object.values(localChanges)) {
 for (const r of rows) {
 const u = new Date(String(r.updated_at));
 if (!Number.isNaN(u.getTime()) && u > newPushCursor) newPushCursor = u;
 }
 }
 }
 notes.push(`${pushed} 已推送`);
} else {
 newPushCursor = new Date();
 notes.push("无数据可推送");
}

await dbRetrySafe(
 () =>
 db
 .update(syncState)
 .set({
 pullCursor: newPullCursor,
 pushCursor: newPushCursor,
 lastSyncAt: new Date(),
 lastStatus: `同步完成: ${notes.join(" | ")}`,
 lastError: "",
 pulled: (peerRow.pulled ?? 0) + pulled,
 pushed: (peerRow.pushed ?? 0) + pushed,
 conflicts: (peerRow.conflicts ?? 0) + conflicts,
 })
 .where(eq(syncState.id, peerRow.id)),
 undefined,
 "sync:ok",
);

return {
 peer: peerRow.peer,
 ok: true,
 pulled,
 pushed,
 conflicts,
 detail: notes.join(" | "),
 durationMs: Date.now() - started,
};
} catch (err) {
 const detail = err instanceof Error ? err.message : "未知错误";
 await dbRetrySafe(
 () =>
 db
 .update(syncState)
 .set({ lastSyncAt: new Date(), lastStatus: `同步失败: ${detail.slice(0, 400) }`, lastError: detail })
 .where(eq(syncState.id, peerRow.id)),
 undefined,
 "sync:fail",
);
 await dbRetrySafe(

```



```

 () => db.insert(syncLogs).values({ peer: peerRow.peer, direction: "both", ok: false, detail: detail.slice(0, 400)
 })),
 undefined,
 "sync:logFail",
);
 return {
 peer: peerRow.peer,
 ok: false,
 pulled,
 pushed,
 conflicts,
 detail,
 durationMs: Date.now() - started,
 };
}
}

/** 000000000000 00 000 00000000 0000 */
export async function syncAll(): Promise<SyncReport[]> {
 const peers = await ensurePeers();
 const active = peers.filter((p) => p.enabled && p.peerUrl);
 if (active.length === 0) return [];
 const out: SyncReport[] = [];
 for (const p of active) out.push(await syncWithPeer(p));
 return out;
}

export function syncInfo() {
 return {
 node: nodeName(),
 url: selfUrl(),
 peers: configuredPeers(),
 hasSecret: Boolean(syncSecret()),
 tables: SYNC_TABLES.filter((t) => t.defaultOn).map((t) => t.name),
 };
}

@@ src/lib/tracker.ts typescript
"use client";

/**
 * =====
 * 000000 00000 000000 (Offline-First GPS Tracker)
 * -----
 * 00000:
 * • 0000 00000 000000 00 IndexedDB 00000 000000 000000000
 * • 000000 00 00000 00 0000000000 0 00 00000 00000 00000000
 * • 00000 000000000 0000 0000000000 00 00 0000 000000000 0 000000 000000000
 * • 0000000 0000 GPS0 00000000 00 000000 00000000000 000000 00000000 000000
 * • 0000 0000000 000000 0 00 00000 0000 00000000 000000 000000000
 * • 000000 0000 00 0 000000 00 00 00000 0000000000 «00000» 0000 00000000
 * =====
 */

export type TrackPoint = {
 lat: number;
 lng: number;
 accuracy?: number | null;
 speed?: number | null;
 heading?: number | null;
 kind: "start" | "move" | "pause" | "end";
 note?: string;
 stopSeconds?: number;
 gpsOn?: boolean;
 recordedAt: string;
};

const DB_NAME = "sek-track";
const STORE = "points";
const MIN_DISTANCE_M = 12; // 00000 00 0000 000000000000 0000 0000000000 (00000 GPS)
const STOP_THRESHOLD_MS = 2 * 60 * 1000; // 00000 0000 00 0 0000000

/* ----- IndexedDB ----- */

function idb(): Promise<IDBDatabase> {
 return new Promise((res, rej) => {
 const r = indexedDB.open(DB_NAME, 1);
 r.onupgradeneeded = () => {
 const d = r.result;
 if (!d.objectStoreNames.contains(STORE)) d.createObjectStore(STORE, { keyPath: "id", autoIncrement: true });
 };
 r.onsuccess = () => res(r.result);
 r.onerror = () => rej(r.error);
 });
}
}

```

```

export async function bufferPoint(tripId: number, p: TrackPoint) {
 try {
 const d = await idb();
 await new Promise<void>((res) => {
 const tx = d.transaction(STORE, "readwrite");
 tx.objectStore(STORE).add({ tripId, ...p });
 tx.oncomplete = () => res();
 tx.onerror = () => res();
 });
 } catch {
 /* 数据库操作失败 - 可能是数据库已满，也可能是其他原因 */
 }
}

export async function bufferedPoints(): Promise<(TrackPoint & { id: number; tripId: number })[]> {
 try {
 const d = await idb();
 return await new Promise((res) => {
 const tx = d.transaction(STORE, "readonly");
 const rq = tx.objectStore(STORE).getAll();
 rq.onsuccess = () => res(rq.result ?? []);
 rq.onerror = () => res([]);
 });
 } catch {
 return [];
 }
}

async function removePoints(ids: number[]) {
 if (!ids.length) return;
 try {
 const d = await idb();
 const tx = d.transaction(STORE, "readwrite");
 const st = tx.objectStore(STORE);
 for (const id of ids) st.delete(id);
 } catch {
 /* ignore */
 }
}

export async function bufferedCount() {
 return (await bufferedPoints()).length;
}

/* ----- 数据库操作失败 ----- */

export function haversine(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
 const R = 6371000;
 const dLat = ((b.lat - a.lat) * Math.PI) / 180;
 const dLng = ((b.lng - a.lng) * Math.PI) / 180;
 const s =
 Math.sin(dLat / 2) ** 2 +
 Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
 return 2 * R * Math.asin(Math.sqrt(s));
}

export function formatDistance(m: number) {
 return m >= 1000 ? `${(m / 1000).toFixed(1)} 千米` : `${Math.round(m)} 米`;
}

export function formatDuration(sec: number) {
 const h = Math.floor(sec / 3600);
 const m = Math.floor((sec % 3600) / 60);
 if (h > 0) return `${h} 小时 ${m} 分钟`;
 if (m > 0) return `${m} 分钟`;
 return `${Math.round(sec)} 秒`;
}

/* ----- 数据库操作失败 ----- */

/** 数据库操作失败，可能是数据库已满，也可能是其他原因 */
export async function flushPoints(): Promise<{ sent: number; pending: number }> {
 const all = await bufferedPoints();
 if (all.length === 0) return { sent: 0, pending: 0 };

 const byTrip = new Map<number, typeof all>();
 for (const p of all) {
 if (!byTrip.has(p.tripId)) byTrip.set(p.tripId, []);
 byTrip.get(p.tripId)!.push(p);
 }

 let sent = 0;
 for (const [tripId, points] of byTrip) {
 const chunk = points.slice(0, 200);

```

```

 try {
 const res = await fetch("/api/trips/points", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({
 tripId,
 points: chunk.map(({ id, tripId: _t, ...rest }) => {
 void id;
 void _t;
 return rest;
 }),
 }));
 if (res.ok) {
 await removePoints(chunk.map((c) => c.id));
 sent += chunk.length;
 }
 } catch {
 break; // 数据库连接失败，退出循环
 }
 }
 return { sent, pending: (await bufferedPoints()).length };
}

/** 数据库操作函数，用于推送实时位置数据 */
export async function pushLive(p: {
 lat: number;
 lng: number;
 accuracy?: number | null;
 speed?: number | null;
 gpsOn?: boolean;
 tripId?: number | null;
 battery?: number | null;
}) {
 try {
 await fetch("/api/live", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ ...p, recordedAt: new Date().toISOString() }),
 });
 } catch {
 // 数据库操作失败，记录日志
 }
}

/** ----- 数据库操作 ----- */

export type TrackerState = {
 lastPoint: TrackPoint | null;
 distanceM: number;
 stopSeconds: number;
 gpsOn: boolean;
 pending: number;
 points: TrackPoint[];
};

export type TrackerOptions = {
 tripId: number;
 onUpdate: (s: TrackerState) => void;
};

/**
 * 数据库操作函数，用于推送实时位置数据。
 * 该函数接收一个 GPS 位置对象，并将其推送到数据库。
 */
export function createTracker({ tripId, onUpdate }: TrackerOptions) {
 let watchId: number | null = null;
 let flushTimer: ReturnType<typeof setInterval> | null = null;
 let liveTimer: ReturnType<typeof setInterval> | null = null;

 const state: TrackerState = {
 lastPoint: null,
 distanceM: 0,
 stopSeconds: 0,
 gpsOn: true,
 pending: 0,
 points: [],
 };

 let lastMoveAt = Date.now();
 let lastSavedAt = 0;

 const emit = () => onUpdate({ ...state, points: [...state.points] });

 const record = async (p: TrackPoint) => {

```

```

state.points.push(p);
if (state.points.length > 2000) state.points.shift();
state.lastPoint = p;
await bufferPoint(tripId, p);
state.pending = await bufferedCount();
emit();
};

const onPosition = async (pos: GeolocationPosition) => {
 state.gpsOn = true;
 const now = Date.now();
 const cur = { lat: pos.coords.latitude, lng: pos.coords.longitude };

 // 开始记录
 if (!state.lastPoint) {
 lastMoveAt = now;
 lastSavedAt = now;
 await record({
 ...cur,
 accuracy: pos.coords.accuracy,
 speed: pos.coords.speed,
 heading: pos.coords.heading,
 kind: "start",
 note: "开始记录",
 gpsOn: true,
 recordedAt: new Date().toISOString(),
 });
 return;
 }

 const dist = haversine(state.lastPoint, cur);

 // 判断是否停止
 if (dist < MIN_DISTANCE_M) {
 const stopped = now - lastMoveAt;
 if (stopped >= STOP_THRESHOLD_MS && now - lastSavedAt >= STOP_THRESHOLD_MS) {
 lastSavedAt = now;
 state.stopSeconds += Math.round(stopped / 1000);
 await record({
 ...cur,
 accuracy: pos.coords.accuracy,
 kind: "pause",
 note: `暂停 ${formatDuration(stopped / 1000)}`,
 stopSeconds: Math.round(stopped / 1000),
 gpsOn: true,
 recordedAt: new Date().toISOString(),
 });
 lastMoveAt = now;
 }
 return;
 }

 // 继续记录
 state.distanceM += dist;
 lastMoveAt = now;
 lastSavedAt = now;
 await record({
 ...cur,
 accuracy: pos.coords.accuracy,
 speed: pos.coords.speed,
 heading: pos.coords.heading,
 kind: "move",
 gpsOn: true,
 recordedAt: new Date().toISOString(),
 });
};

const onError = () => {
 // GPS 信号丢失，停止记录
 state.gpsOn = false;
 emit();
};

const start = () => {
 if (!("geolocation" in navigator)) return false;
 if (watchId !== null) return true;

 watchId = navigator.geolocation.watchPosition(onPosition, onError, {
 enableHighAccuracy: true,
 maximumAge: 15000,
 timeout: 30000,
 });
};

// 定时刷新
flushTimer = setInterval(async () => {

```

```

 const r = await flushPoints();
 state.pending = r.pending;
 emit();
 }, 20000);

// 000000 00000000 0000 0000 0000 GPS 0000 00000000
liveTimer = setInterval(() => {
 if (state.lastPoint) {
 void pushLive({
 lat: state.lastPoint.lat,
 lng: state.lastPoint.lng,
 accuracy: state.lastPoint.accuracy,
 speed: state.lastPoint.speed,
 gpsOn: state.gpsOn,
 tripId,
 });
 }
}, 30000);

return true;
};

const stop = async (note = "000000 000000") => {
 if (watchId !== null) {
 navigator.geolocation.clearWatch(watchId);
 watchId = null;
 }
 if (flushTimer) clearInterval(flushTimer);
 if (liveTimer) clearInterval(liveTimer);
 if (state.lastPoint) {
 await record({ ...state.lastPoint, kind: "end", note, recordedAt: new Date().toISOString() });
 }
 await flushPoints();
};

const markPause = async (note: string) => {
 if (!state.lastPoint) return;
 await record({ ...state.lastPoint, kind: "pause", note, recordedAt: new Date().toISOString() });
 await flushPoints();
};

return { start, stop, markPause, state, flush: flushPoints };
}

@@ src/lib/useLive.ts typescript
"use client";

import { useCallback, useEffect, useRef, useState } from "react";

/**
 * 000000 JSON 00 00000000 0000 — 0000 throw 000000000.
 * 000 00000000 000000000 0000000 0000 000000 00000000000000.
 */
export async function fetchJson<T = unknown>(url: string, fallback: T, init?: RequestInit): Promise<T> {
 try {
 const res = await fetch(url, { cache: "no-store", ...init });
 if (!res.ok) return fallback;
 return (await res.json()) as T;
 } catch {
 return fallback;
 }
}

/** 000000 000000 00000000 00 API00000 00 `{rows:[...]}` 000000000000000 */
export async function fetchRows<T = Record<string, unknown>>(url: string): Promise<T[]> {
 const d = await fetchJson<{ rows?: T[] }>(url, {});
 return Array.isArray(d.rows) ? d.rows : [];
}

/**
 * 000000000000 0000000000 00000000:
 * - 00 0000 00000000 00 `interval` 000000000000 0000 00 0000 00000000
 * - 0000 00 0000 000000 00000000 000000 00000000 (0000 0000 000000 0 00000)
 * - 00 00000000 00 00 00 0000 0000 0000000000 000000 00 0000 000000000000 00000000
 * - 0000 0000000000 000000 (2G / save-data) 000000 00 00000000 000000 00000000
 */
export function useLive(fn: () => void | Promise<void>, interval = 15000, enabled = true) {
 const saved = useRef(fn);
 saved.current = fn;
 const [paused, setPaused] = useState(false);

 const run = useCallback(() => {
 void saved.current();
 }, []);

```

```

useEffect(() => {
 if (!enabled || paused) return;

 type Conn = { effectiveType?: string; saveData?: boolean };
 const conn = (navigator as Navigator & { connection?: Conn }).connection;
 const slow = conn?.saveData || ["slow-2g", "2g"].includes(conn?.effectiveType ?? "");
 const ms = slow ? Math.max(interval * 3, 45000) : interval;

 run();
 let timer: ReturnType<typeof setInterval> | null = null;

 const start = () => {
 if (timer) clearInterval(timer);
 timer = setInterval(() => {
 if (!document.hidden && navigator.onLine) run();
 }, ms);
 };
 const onVisible = () => {
 if (!document.hidden) {
 run();
 start();
 } else if (timer) {
 clearInterval(timer);
 timer = null;
 }
 };

 start();
 document.addEventListener("visibilitychange", onVisible);
 window.addEventListener("online", run);
 window.addEventListener("focus", run);
 return () => {
 if (timer) clearInterval(timer);
 document.removeEventListener("visibilitychange", onVisible);
 window.removeEventListener("online", run);
 window.removeEventListener("focus", run);
 };
}, [enabled, paused, interval, run]);

return { refresh: run, paused, setPaused };
}

@@ src/lib/zip.ts typescript
import { deflateRawSync } from "node:zlib";

/**
 * =====
 * ZIP 0000 0000 0000 0000 0000 0000 0000 0000
 * -----
 * 0000 0000 0000 0000 0000 00 0000 0000 0000 0000 0000 0000
 * 00 0000 0000 0000 0000 0000 00 00 0000 00 0000 0 00
 * GitHub / GitLab 0000 000.
 * =====
 */

const CRC_TABLE = (() => {
 const t = new Uint32Array(256);
 for (let n = 0; n < 256; n++) {
 let c = n;
 for (let k = 0; k < 8; k++) c = c & 1 ? 0xedb88320 ^ (c >>> 1) : c >>> 1;
 t[n] = c >>> 0;
 }
 return t;
})();

function crc32(buf: Buffer): number {
 let c = 0xffffffff;
 for (let i = 0; i < buf.length; i++) c = CRC_TABLE[(c ^ buf[i]) & 0xff] ^ (c >>> 8);
 return (c ^ 0xffffffff) >>> 0;
}

/** 0000 0000 00 0000 DOS 0000 0000 ZIP */
function dosDateTime(d: Date) {
 const time = ((d.getHours() < 11) | (d.getMinutes() < 5) | (d.getSeconds() / 2)) & 0xffff;
 const date = (((d.getFullYear() - 1980) < 9) | ((d.getMonth() + 1) < 5) | d.getDate()) & 0xffff;
 return { time, date };
}

export type ZipEntry = { path: string; content: Buffer | string };

/** 0000 0000 ZIP 00 0000 0000 0000 */
export function createZip(entries: ZipEntry[], modified = new Date()): Buffer {
 const { time, date } = dosDateTime(modified);
 const chunks: Buffer[] = [];
 const central: Buffer[] = [];

```

```

let offset = 0;

for (const entry of entries) {
 const nameBuf = Buffer.from(entry.path.replace(/\\/g, "/"), "utf8");
 const raw = Buffer.isBuffer(entry.content) ? entry.content : Buffer.from(entry.content, "utf8");
 const crc = crc32(raw);

 // 0000 000000000000 0000 000000 0000 0000 000000 000000 000000
 const deflated = deflateRawSync(raw, { level: 9 });
 const useDeflate = deflated.length < raw.length;
 const data = useDeflate ? deflated : raw;
 const method = useDeflate ? 8 : 0;

 const local = Buffer.alloc(30);
 local.writeUInt32LE(0x04034b50, 0); // 000000 0000 0000
 local.writeUInt16LE(20, 4); // 0000 00000000
 local.writeUInt16LE(0x0800, 6); // 0000: 0000 0000 UTF-8
 local.writeUInt16LE(method, 8);
 local.writeUInt16LE(time, 10);
 local.writeUInt16LE(date, 12);
 local.writeUInt32LE(crc, 14);
 local.writeUInt32LE(data.length, 18);
 local.writeUInt32LE(raw.length, 22);
 local.writeUInt16LE(nameBuf.length, 26);
 local.writeUInt16LE(0, 28);

 chunks.push(local, nameBuf, data);

 const cd = Buffer.alloc(46);
 cd.writeUInt32LE(0x02014b50, 0); // 000000 0000000000 000000
 cd.writeUInt16LE(20, 4);
 cd.writeUInt16LE(20, 6);
 cd.writeUInt16LE(0x0800, 8);
 cd.writeUInt16LE(method, 10);
 cd.writeUInt16LE(time, 12);
 cd.writeUInt16LE(date, 14);
 cd.writeUInt32LE(crc, 16);
 cd.writeUInt32LE(data.length, 20);
 cd.writeUInt32LE(raw.length, 24);
 cd.writeUInt16LE(nameBuf.length, 28);
 cd.writeUInt16LE(0, 30);
 cd.writeUInt16LE(0, 32);
 cd.writeUInt16LE(0, 34);
 cd.writeUInt16LE(0, 36);
 cd.writeUInt32LE(0, 38);
 cd.writeUInt32LE(offset, 42);
 central.push(cd, nameBuf);

 offset += local.length + nameBuf.length + data.length;
}

const centralBuf = Buffer.concat(central);
const end = Buffer.alloc(22);
end.writeUInt32LE(0x06054b50, 0); // 000000 0000000000 000000
end.writeUInt16LE(0, 4);
end.writeUInt16LE(0, 6);
end.writeUInt16LE(entries.length, 8);
end.writeUInt16LE(entries.length, 10);
end.writeUInt32LE(centralBuf.length, 12);
end.writeUInt32LE(offset, 16);
end.writeUInt16LE(0, 20);

return Buffer.concat([...chunks, centralBuf, end]);
}

```

```

@@ src/app/admin/layout.tsx typescript
"use client";

```

```

import type { ReactNode } from "react";
import Shell from "@components/Shell";
import SessionProvider, { useSession } from "@components/SessionProvider";

function Inner({ children }: { children: ReactNode }) {
 const { me } = useSession();
 if (!me) return null;
 const has = (p: string) => me.role === "admin" || me.permissions.includes(p);

 const nav = [
 { href: "/admin", label: "00000000", icon: "☐", perm: "dashboard" },
 { href: "/admin/records/pharmacies", label: "000000000000", icon: "☐", perm: "pharmacy" },
 { href: "/admin/records/doctors", label: "00000000", icon: "☐", perm: "doctor" },
 { href: "/admin/records/orders", label: "00000000", icon: "☐", perm: "order" },
 { href: "/admin/activity", label: "00000000 00000000", icon: "☐", perm: "dashboard" },
 { href: "/admin/map", label: "000000 000000", icon: "☐", perm: "monitor" },
 { href: "/admin/live", label: "00000000 000000", icon: "☐", perm: "monitor" },
];
}

```

```

{ href: "/admin/trips", label: "📅 📅 📅", icon: "📅", perm: "monitor" },
{ href: "/admin/homes", label: "🏠 🏠 🏠 🏠 🏠", icon: "🏠", perm: "home" },
{ href: "/admin/leaves", label: "📅 📅 📅 📅", icon: "📅", perm: "leave" },
{ href: "/admin/notifications", label: "📢 📢 📢", icon: "📢" },
{ href: "/admin/reports", label: "📊 📊 📊 📊", icon: "📊", perm: "reports" },
{ href: "/admin/targets", label: "🎯 🎯 🎯", icon: "🎯", perm: "users" },
{ href: "/admin/options", label: "⚙️ ⚙️ ⚙️", icon: "⚙️", perm: "options" },
{ href: "/admin/columns", label: "📏 📏 📏 📏 📏", icon: "📏", perm: "columns" },
{ href: "/admin/users", label: "👤 👤 👤 👤", icon: "👤", perm: "users" },
{ href: "/admin/messengers", label: "💬 💬 💬 💬 💬", icon: "💬", perm: "messengers" },
{ href: "/admin/backup", label: "💾 💾 💾 💾 💾", icon: "💾", perm: "users" },
{ href: "/admin/sync", label: "🔄 🔄 🔄 🔄 🔄", icon: "🔄", perm: "users" },
{ href: "/install", label: "🔧 🔧", icon: "🔧" },
{ href: "/diagnostics", label: "🔍 🔍 🔍", icon: "🔍" },
]

.filter((n) => !n.perm || has(n.perm))
.map(({ href, label, icon }) => ({ href, label, icon }));

return (
 <Shell
 userName={me.fullName}
 roleLabel={me.role === "admin" ? "👤 📅 📅 📅" : "👤 📅 📅 📅"}
 nav={nav}
 showPanelLink
 >
 {children}
 </Shell>
);
}

export default function AdminLayout({ children }: { children: ReactNode }) {
 return (
 <SessionProvider require adminOnly>
 <Inner>{children}</Inner>
 </SessionProvider>
);
}

@@ src/app/admin/page.tsx typescript
"use client";

import Link from "next/link";
import { useCallback, useState } from "react";
import { Badge, Card, SectionTitle } from "@components/ui";
import dynamic from "next/dynamic";

const ChartsPanel = dynamic(() => import("@/components/screens/ChartsPanel"), {
 ssr: false,
 loading: () => <div className="rounded-2xl bg-white p-6 text-center text-xs text-slate-400">📊 📊 📊 📊 📊 📊 📊..
</div>,
});
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";

type Counts = {
 pharmacies: number;
 doctors: number;
 orders: number;
 trips: number;
 activeTrips: number;
 users: number;
 leaves: number;
};

type Log = { id: number; userName: string; action: string; detail: string; createdAt: string };
type Rep = {
 id: number;
 fullName: string;
 role: string;
 active: boolean;
 lastSeenAt: string | null;
 pharmacies: number;
 doctors: number;
 orders: number;
 trips: number;
 activeTrip: number;
 lastAction: string | null;
 lastActionAt: string | null;
};

export default function AdminDashboard() {
 const [counts, setCounts] = useState<Counts | null>(null);
 const [reps, setReps] = useState<Rep[]>([]);
 const [byRep, setByRep] = useState<Record<string, Log[]>>({});
 const [openRep, setOpenRep] = useState<string | null>(null);

```



```

const [live, setLive] = useState(true);
const [dl, setDl] = useState("");

const load = useCallback(async () => {
 const res = await fetch("/api/activity", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 setCounts(d.counts);
 setReps(d.reps ?? []);
 setByRep(d.byRep ?? {});
}, []);

useLive(load, 20000, live);

const tiles = [
 { label: "🏠", value: counts?.pharmacies ?? 0, icon: "🏠", href: "/admin/records/pharmacies" },
 { label: "👨‍⚕️", value: counts?.doctors ?? 0, icon: "👨‍⚕️", href: "/admin/records/doctors" },
 { label: "📄", value: counts?.orders ?? 0, icon: "📄", href: "/admin/records/orders" },
 { label: "🗺️", value: counts?.activeTrips ?? 0, icon: "🗺️", href: "/admin/trips" },
 { label: "🌿", value: counts?.leaves ?? 0, icon: "🌿", href: "/admin/leaves" },
 { label: "👤", value: counts?.users ?? 0, icon: "👤", href: "/admin/users" },
];

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <SectionTitle icon="🏠">🏠🏠🏠🏠🏠🏠🏠🏠</SectionTitle>
 <button
 onClick={() => setLive((v) => !v)}
 className={`rounded-lg px-3 py-1.5 text-xs font-bold ${live ? "bg-emerald-100 text-emerald-700" : "bg-slate-200 text-slate-600"}>
 </button>
 </div>

 <div className="grid grid-cols-2 gap-2 sm:grid-cols-3 lg:grid-cols-6">
 {tiles.map((t) => (
 <Link
 key={t.label}
 href={t.href}
 className="rounded-2xl bg-white p-3 text-center shadow-sm ring-1 ring-slate-200 hover:ring-teal-300">
 <div className="text-xl">{t.icon}</div>
 <div className="text-lg font-black text-teal-700">{toPersianDigits(t.value)}</div>
 <div className="text-[10px] font-bold text-slate-500">{t.label}</div>
 </Link>
))}
 </div>

 <Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="🏠">🏠🏠🏠🏠🏠🏠🏠🏠 — 🏠🏠🏠🏠🏠🏠🏠🏠</SectionTitle>
 <Link
 href="/admin/activity"
 className="rounded-xl bg-teal-600 px-3 py-2 text-xs font-bold text-white hover:bg-teal-700">
 🏠🏠🏠🏠🏠🏠🏠🏠🏠🏠
 </Link>
 </div>
 <div className="grid grid-cols-1 gap-2 lg:grid-cols-2">
 {reps.map((r) => {
 const logs = byRep[r.fullName] ?? [];
 const isOpen = openRep === r.fullName;
 return (
 <div key={r.id} className="rounded-2xl bg-slate-50 p-3 ring-1 ring-slate-200">
 <div className="flex flex-wrap items-center gap-2">
 {r.fullName}
 <Badge tone={r.role === "admin" ? "green" : r.role === "supervisor" ? "amber" : "slate"}>
 {r.role === "admin" ? "🏠🏠🏠🏠" : r.role === "supervisor" ? "🏠🏠🏠🏠" : "🏠🏠🏠🏠🏠🏠"}
 </Badge>
 {r.activeTrip > 0 ? <Badge tone="green">🏠🏠🏠🏠🏠🏠</Badge> : null}
 {!r.active ? <Badge tone="amber">🏠🏠🏠🏠🏠🏠</Badge> : null}
 <button
 onClick={() => setOpenRep(isOpen ? null : r.fullName)}
 className="mr-auto rounded-lg bg-white px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-teal-200">
 </button>
 </div>
 <div className="mt-2 grid grid-cols-4 gap-1 text-center text-[11px]">
 {
 ["🏠", r.pharmacies],
 ["🏠", r.doctors],
 }
 </div>
 </div>
)
 })}
 </div>
 </Card>
 </div>
)

```

```

["", r.orders],
["", r.trips],
].map(([i, v], idx) => (
 <div key={idx} className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
 {i as string} <b className="text-teal-700">{toPersianDigits(v as number)}
 </div>
))
</div>
<div className="mt-2 text-[11px] text-slate-500">
 □□□□ □□□□: {r.lastAction ?? "—"}
 {r.lastActionAt ? ` (${tehranDateTime(r.lastActionAt)})` : ""}
</div>
<div className="text-[11px] text-slate-400">
 □□□□ □□□□: {r.lastSeenAt ? tehranDateTime(r.lastSeenAt) : "—"}
</div>
 {isOpen ? (
 <div className="mt-2 max-h-52 space-y-1 overflow-y-auto">
 {logs.length === 0 ? (
 <p className="text-xs text-slate-400">□□□□□□ □□ □□□□ □□</p>
) : (
 logs.map((l) => (
 <div key={l.id} className="rounded-lg bg-white px-2 py-1 text-[11px] ring-1 ring-slate-100">
 {l.action}
 {l.detail ? — {l.detail} : null}
 {tehranDateTime(l.createdAt)}
 </div>
))
)}
 </div>
) : null}
</div>
);
 })}
</div>
</Card>

<Card>
<SectionTitle icon="">□□□□ □□□□</SectionTitle>
{dl ? <p className={mb-2 text-xs font-bold ${dl.startsWith("") ? "text-rose-600" : "text-emerald-600"}}><dl>
/ps : null}
 <div className="flex flex-wrap gap-2">
 <a
 href="/admin/targets"
 className="rounded-xl bg-amber-50 px-3 py-2 text-xs font-bold text-amber-800 ring-1 ring-amber-200 hover:bg-amber-100"
 >
 □ □□□□ □□□□

 <a
 href="/admin/backup"
 className="rounded-xl bg-indigo-50 px-3 py-2 text-xs font-bold text-indigo-800 ring-1 ring-indigo-200 hover:bg-indigo-100"
 >
 □ □□□□□□□□□□□

 {
 ["pharmacies", "□□□□□□□□□□"],
 ["doctors", "□□□□□□"],
 ["orders", "□□□□□□□□"],
 ["trips", "□□□□□□"],
 ["leaves", "□□□□□□□□"],
 ["homes", "□□□□□□"],
 ["users", "□□□□□□□□"],
].map(([k, l]) => (
 <button
 key={k}
 onClick={async () => setDl(await downloadFile(`\api/export?type=${k}`, `${k}.xls`))}
 className="rounded-xl bg-emerald-50 px-3 py-2 text-xs font-bold text-emerald-800 ring-1 ring-emerald-200 h
over:bg-emerald-100"
 >
 □□ {l}
 </button>
))}
 </div>
</Card>

<ChartsPanel scope="admin" />
</div>
);
}

```

```

@@ src/app/apple-touch-icon.png/route.ts typescript
import { pngResponse, renderIcon } from "@lib/pwa-image";

export const dynamic = "force-static";

```

```

export const revalidate = false;

export async function GET() {
 const bytes = await renderIcon(180, true);
 if (!bytes) return new Response("not available", { status: 404 });
 return pngResponse(bytes);
}

@@ src/app/diagnostics/page.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import Link from "next/link";
import { Alert, Badge, Button, Card, SectionTitle } from "@components/ui";
import { getDeviceId, getDeviceInfo, readSimStatus } from "@lib/device";
import { getCachedUser, getToken } from "@lib/offline-session";
import { toPersianDigits } from "@lib/jalali";

type Row = { key: string; label: string; state: "ok" | "warn" | "fail" | "run"; detail: string };

export default function DiagnosticsPage() {
 const [rows, setRows] = useState<Row[]>([]);
 const [busy, setBusy] = useState(false);
 const [cacheInfo, setCacheInfo] = useState("");
 const [queue, setQueue] = useState<number | null>(null);

 const put = (r: Row) => setRows((p) => [...p.filter((x) => x.key !== r.key), r]);

 const run = useCallback(async () => {
 setBusy(true);
 setRows([]);

 // () اتصال اینترنت را بررسی می‌کند
 const sim = readSimStatus();
 type Conn = { effectiveType?: string; type?: string; downlink?: number; rtt?: number; saveData?: boolean };
 const conn = (navigator as Navigator & { connection?: Conn }).connection;
 put({
 key: "net",
 label: "وضعیت اینترنت",
 state: navigator.onLine ? "ok" : "fail",
 detail: navigator.onLine
 ? `نوع: ${conn?.type ?? "نوع اینترنت"} | پهنای: ${conn?.effectiveType ?? "پهنای اینترنت"} | پهنای: ${conn?.downlink ?? "پهنای اینترنت"}
 | پهنای: ${conn?.rtt ?? "پهنای اینترنت"}ms | ذخیره داده: ${conn?.saveData ?? "ذخیره داده"} | پهنای: ${conn?.downlink ?? "پهنای اینترنت"}
 : "وضعیت اینترنت را بررسی کنید",
 });

 // () وضعیت سرویس ورکر را بررسی می‌کند
 if ("serviceWorker" in navigator) {
 const reg = await navigator.serviceWorker.getRegistration();
 put({
 key: "sw",
 label: "وضعیت سرویس ورکر (وضعیت سرویس ورکر)",
 state: reg?.active ? "ok" : "warn",
 detail: reg?.active
 ? "سرویس ورکر — وضعیت سرویس ورکر: وضعیت سرویس ورکر"
 : "وضعیت سرویس ورکر: وضعیت سرویس ورکر",
 });
 } else {
 put({ key: "sw", label: "وضعیت سرویس ورکر", state: "fail", detail: "وضعیت سرویس ورکر: وضعیت سرویس ورکر" });
 }

 // () حجم حافظه کش را بررسی می‌کند
 try {
 const names = await caches.keys();
 let total = 0;
 for (const n of names) {
 const c = await caches.open(n);
 total += (await c.keys()).length;
 }
 put({
 key: "cache",
 label: "وضعیت حافظه کش",
 state: total > 30 ? "ok" : total > 0 ? "warn" : "fail",
 detail:
 total > 30
 ? `${toPersianDigits(total)} حافظه کش — وضعیت حافظه کش: وضعیت حافظه کش`
 : `حافظه کش: ${toPersianDigits(total)} حافظه کش: وضعیت حافظه کش: وضعیت حافظه کش`,
 });
 setCacheInfo(`${total} حافظه کش: ${names.length} حافظه کش`);
 } catch {
 put({ key: "cache", label: "وضعیت حافظه کش", state: "fail", detail: "وضعیت حافظه کش: وضعیت حافظه کش" });
 }

 // () زمان اجرای کد را بررسی می‌کند
 const t0 = performance.now();

```

```

try {
 const ctrl = new AbortController();
 const to = setTimeout(() => ctrl.abort(), 20000);
 const r = await fetch(`/api/ping?t=${Date.now()}`, { cache: "no-store", signal: ctrl.signal });
 clearTimeout(to);
 const ms = Math.round(performance.now() - t0);
 put({
 key: "ping",
 label: "📶📶📶📶 📶 📶📶📶",
 state: r.ok ? (ms < 3000 ? "ok" : "warn") : "fail",
 detail: r.ok
 ? `📶📶 📶 ${toPersianDigits(ms)} 📶📶📶📶📶📶${ms > 3000 ? " (📶 — 📶📶 📶 📶📶 📶📶📶 📶)" : ""}`
 : `📶 📶 ${r.status}`,
 });
} catch {
 put({
 key: "ping",
 label: "📶📶📶📶 📶 📶📶📶",
 state: "fail",
 detail: "📶📶 📶 📶📶📶 📶📶. 📶📶 📶 📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶📶",
 });
}

// 📶 📶📶📶📶
try {
 const ctrl = new AbortController();
 const to = setTimeout(() => ctrl.abort(), 25000);
 const r = await fetch(`/api/health?t=${Date.now()}`, { cache: "no-store", signal: ctrl.signal });
 clearTimeout(to);
 const d = await r.json().catch(() => ({}));
 put({
 key: "db",
 label: "📶📶📶📶 📶📶📶",
 state: d?.ok ? "ok" : "fail",
 detail: d?.ok ? "📶📶📶 📶📶📶📶 📶📶" : d?.reason ?? "📶 📶📶📶 📶📶",
 });
} catch {
 put({ key: "db", label: "📶📶📶📶 📶📶📶", state: "fail", detail: "📶📶 📶📶 (Timeout)" });
}

// 📶 📶📶
const cached = getCacheUser();
put({
 key: "session",
 label: "📶📶 📶📶📶📶📶📶 (📶📶 📶📶📶📶)",
 state: cached ? "ok" : "warn",
 detail: cached
 ? `${cached.fullName} — 📶📶📶📶📶📶 📶📶 📶📶📶📶 📶📶 📶📶📶`
 : "📶📶 📶📶📶 📶📶📶 📶📶 📶 📶📶 📶 📶📶📶📶 📶📶 📶📶",
});

// 📶 📶📶📶 📶📶📶
const id = getDeviceId();
put({
 key: "device",
 label: "📶📶📶 📶📶📶 (📶 📶📶📶📶)",
 state: id ? "ok" : "fail",
 detail: id ? `${getDeviceInfo()} | 📶📶📶: ${id.slice(0, 10)}...` : "📶📶📶 📶📶📶📶 📶📶📶 📶",
});

// 📶 📶📶📶📶📶
put({
 key: "geo",
 label: "📶📶📶📶📶📶 (GPS)",
 state: "geolocation" in navigator ? (window.isSecureContext ? "ok" : "warn") : "fail",
 detail:
 "geolocation" in navigator
 ? window.isSecureContext
 ? "📶 📶📶📶 📶📶"
 : "📶📶 http 📶📶 📶📶📶📶📶 📶📶 📶📶📶 https 📶📶"
 : "📶📶📶📶 📶📶📶📶📶 📶📶📶📶",
});

// 📶 📶📶📶 📶📶📶📶
put({
 key: "sim",
 label: "📶📶📶 📶📶📶📶📶 / 📶📶",
 state: sim.ok ? "ok" : "warn",
 detail: sim.ok ? `📶📶 📶📶📶: ${sim.type}` : sim.detail,
});

// 📶 📶📶
put({
 key: "token",

```

```

label: "🔴🔴🔴🔴 📶📶📶",
 state: getToken() ? "ok" : "warn",
 detail: getToken() ? "🔴🔴🔴🔴 📶📶📶" : "🔴🔴🔴🔴 📶📶📶📶📶",
});

setBusy(false);
}, []);

useEffect(() => {
 run();
 const onMsg = (e: MessageEvent) => {
 if (e.data?.type === "queue-status") setQueue(e.data.pending);
 if (e.data?.type === "cache-status") setCacheInfo(`${e.data.cached} 📶 ${e.data.expected} 📶📶📶`);
 };
 navigator.serviceWorker?.addEventListener("message", onMsg);
 navigator.serviceWorker?.controller?.postMessage("queue-count");
 navigator.serviceWorker?.controller?.postMessage("cache-status");
 return () => navigator.serviceWorker?.removeEventListener("message", onMsg);
}, [run]);

const icon = (s: Row["state"]) => (s === "ok" ? "🟢" : s === "warn" ? "🟡" : s === "run" ? "🔴" : "🟠");
const tone = (s: Row["state"]) =>
 s === "ok" ? "bg-emerald-50 ring-emerald-200" : s === "warn" ? "bg-amber-50 ring-amber-200" : "bg-rose-50 ring-rose-200";

const fails = rows.filter((r) => r.state === "fail").length;

return (
 <main className="mx-auto max-w-2xl space-y-4 p-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="📶">🔴🔴🔴🔴🔴🔴 📶📶📶 📶📶📶📶📶</SectionTitle>
 <div className="flex gap-2">
 <Button onClick={run} disabled={busy}>
 {busy ? "🔴 📶📶 📶📶📶..." : "📶 📶📶📶 📶📶📶"}
 </Button>
 <Link href="/" className="rounded-xl bg-white px-3 py-2.5 text-xs font-bold text-slate-700 ring-1 ring-slate-300">
 📶📶📶📶
 </Link>
 </div>
 </div>

 {fails > 0 ? (
 <Alert kind="error">
 {toPersianDigits(fails)} 📶📶📶 📶📶📶. 📶📶📶📶 📶📶📶 📶📶 📶📶📶📶 📶📶📶📶 📶📶📶📶.
 </Alert>
) : rows.length > 0 && !busy ? (
 <Alert kind="success">📶 📶📶📶📶📶 📶📶📶 📶📶 — 📶📶📶📶 📶📶 📶📶📶📶📶 📶 📶📶 📶📶📶📶📶 📶📶📶 📶📶📶📶📶.</Alert>
) : null}

 <Card>
 <div className="space-y-2">
 {rows.map((r) => (
 <div key={r.key} className={`rounded-xl px-3 py-2 ring-1 ${tone(r.state)} `}>
 <div className="flex items-start gap-2">
 {icon(r.state)}
 <div className="min-w-0 flex-1">
 <div className="text-xs font-black text-slate-800">{r.label}</div>
 <div className="mt-0.5 text-[11px] leading-5 text-slate-600">{r.detail}</div>
 </div>
 </div>
 </div>
))}
 {rows.length === 0 ? <p className="py-6 text-center text-xs text-slate-400">📶📶 📶📶 📶📶📶...</p> : null}
 </div>
 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">📶 📶📶📶📶📶</h3>
 <div className="flex flex-wrap gap-2">
 <Button
 variant="soft"
 onClick={() => {
 navigator.serviceWorker?.controller?.postMessage("flush");
 setTimeout(run, 1500);
 }}
 >
 📶 📶📶📶 📶📶📶 📶📶📶📶 {queue ? `${toPersianDigits(queue)} ` : ""}
 </Button>
 <Button
 variant="ghost"
 onClick={async () => {
 const regs = await navigator.serviceWorker.getRegistrations();
 await Promise.all(regs.map((r) => r.unregister()));
 const keys = await caches.keys();

```

```

 await Promise.all(keys.map((k) => caches.delete(k)));
 location.reload();
 }}
 >
 □ □□□□□□□ □ □□ □□□ □□□□□
 </Button>
 <Button
 variant="ghost"
 onClick={() => {
 const text = rows.map((r) => `${icon(r.state)} ${r.label}: ${r.detail}`).join("\n");
 navigator.clipboard?.writeText(`□□□□ □□□□□□□□\n${text}\n${navigator.userAgent}`);
 }}
 >
 □ □□ □□□□
 </Button>
</div>
{cacheInfo ? <p className="mt-2 text-[11px] text-slate-400">□□□□ □□□□: {cacheInfo}</p> : null}
</Card>

<Card>
<h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□□□ □□ □□□</h3>
 <ul className="space-y-2 text-[11px] leading-6 text-slate-600">
 <li className="rounded-lg bg-slate-50 px-3 py-2">
 □□□ «□□□□□ □□ □□□» □□ □□ □□□□□□ □□ □□□□□ □□□□□□ □□□□□ □□ □□□□ □□ □□□□ □□ □□□□
 □□□□ □□. □□□□□□: □□ □□ □□ □□□□□□ □□□□□□ □□ □□□ □□ □□□□ □□□□ □□ □□ □□ □□□□□□ □□□□□□ □□
 □□ □□□□□ □ □□□□□□ □□ □□ □□□□□□ □□ □□□. □□□□□ □□□□□: □□□□□ □□□□□ □□ □□□□ □□□□ □□□□□.

 <li className="rounded-lg bg-slate-50 px-3 py-2">
 □□□ «□□□□□□□ □□□□□□□□□» □□ □□□ □□ □□ □□ □□□□□□ □□□□□□ □□ □□□□ □□ □□□ □□□ □□ □□□□
 «□□□□□□□ □ □□ □□□□» □□ □□□□□.

 <li className="rounded-lg bg-slate-50 px-3 py-2">
 □□□□ □□□□□ □□□□□ □□□□□ □□ □□ □□□ «□□□ □□» □□□ □□□□ □□ □□□□ □□ □□□□□ □□□□ □□□.

</Card>
</main>
);
}

@@ src/app/install/page.tsx typescript
"use client";

import { useEffect, useState } from "react";
import { Alert, Button, Card, SectionTitle } from "@/components/ui";

type BIP = Event & { prompt: () => Promise<void>; userChoice: Promise<{ outcome: string }> };

export default function InstallPage() {
 const [deferred, setDeferred] = useState<BIP | null>(null);
 const [installed, setInstalled] = useState(false);
 const [msg, setMsg] = useState("");

 useEffect(() => {
 const handler = (e: Event) => {
 e.preventDefault();
 setDeferred(e as BIP);
 };
 window.addEventListener("beforeinstallprompt", handler);
 window.addEventListener("appinstalled", () => setInstalled(true));
 if (window.matchMedia("(display-mode: standalone)").matches) setInstalled(true);
 return () => window.removeEventListener("beforeinstallprompt", handler);
 }, []);

 const install = async () => {
 if (!deferred) {
 setMsg("□□□ □□□□ □□ □□□□ □□□□ □□ □□□□ □□□□□ «Install app» □□ «□□□□□□ □□ □□□□ □□□□» □□ □□□□.");
 return;
 }
 await deferred.prompt();
 const res = await deferred.userChoice;
 if (res.outcome === "accepted") setInstalled(true);
 setDeferred(null);
 };

 return (
 <main className="mx-auto max-w-2xl space-y-4 p-4">
 <SectionTitle icon="□">□□□□□ □ □□□ □□□□□□□</SectionTitle>
 {installed ? <Alert kind="success">□ □□□□□□□ □□ □□ □□□□□ □□ □□ □□□□ □□□□ □□□□ □□□□ □□□□ □□□□□</Alert> : null}
 {msg ? <Alert kind="info">{msg}</Alert> : null}

 <Card>
 <p className="mb-3 text-sm text-slate-600">
 □□□ □□□□□ □□ □□□□□□□ □□□□□□□ (PWA) □□□□ □□ □□ □□□ □□ □□□□□ □□□□□ □□□□ □□□□ □□□□ □□□□ □□□□□
 □□ □□□□□ □ □□□□□□ □□ □□□ □□□□□.

```

```

 </p>
 <Button onClick={install} className="w-full sm:w-auto">
 00 000 0000000000 000 000 0000000
 </Button>
 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-black text-slate-800">0 00000000</h3>
 <ol className="list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
 0000000 00 00 0000000 Chrome 0000 0000.
 00000 «0000 0000000000» 0000 00 0000000 00 00 00000 0 000000 «Install app / 0000000 00 00000 00000».
 00000 0000000 000 00000 00000 00000 0000000.

 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-black text-slate-800">0 000000 (iOS)</h3>
 <ol className="list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
 0000000 00 00 Safari 0000 0000.
 00000 0000000000000000 (00000 00 0000) 00 000000.
 000000 «Add to Home Screen» 00 00000000 00000.

 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-black text-slate-800">0 0000000</h3>
 <ol className="list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">
 0000000 00 00 Chrome 00 Edge 0000 0000.
 00000 0000 0000 (Ⓢ) 00 00000 000000 00 0000 0 → «Install».
 0000000 00 00000 Start 00000000 0000 0000000 0 0000 00000000000 0000000 00000 0000000.

 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-black text-slate-800">0 00000 00000 APK 00000000</h3>
 <Alert kind="success">
 0 000 0000000000000000 PWABuilder 000000 0000: 00000000000 0000 0 0000 000000 (PNG)0 0000000 maskable0 00000000000000
 00000000000000000000 00000000 0 000000000 0 0000000 (id) 00 manifest.
 </Alert>
 <ol className="mt-2 list-inside list-decimal space-y-1 text-xs leading-6 text-slate-600">

 00000{" "}

 PWABuilder.com
 {" "}
 0000 0 00000 00000 00000 00000 (00 00000 login) 00 00000 00000.

 00000 Refresh/Retest 00 000000 00 manifest 00000 00000000 0000 — 00000000 0000000 0000000000.
 0000 «Package For Stores» → Android 000000 0 00000 APK/AAB 00 00000000 00000.
 00000 0000 00000 000000000000 «Download Test Package» 00 000000 0 APK 00 00000000 0000 00000 000 00000.

 <div className="mt-3 flex flex-wrap gap-2">
 <a
 href="https://www.pwabuilder.com/reportcard?site="
 target="_blank"
 rel="noreferrer"
 className="rounded-xl bg-teal-600 px-3 py-2 text-xs font-bold text-white"
 onClick={(e) => {
 e.preventDefault();
 window.open(`https://www.pwabuilder.com/reportcard?site=${window.location.origin}/`, "_blank");
 }}
 >
 0 00000 APK 00 PWABuilder

 <a
 href="/manifest.webmanifest"
 target="_blank"
 rel="noreferrer"
 className="rounded-xl bg-white px-3 py-2 text-xs font-bold text-slate-700 ring-1 ring-slate-300"
 >
 00000000 manifest

 </div>
 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-black text-slate-800">0 00000 0 000000000000 0000000000</h3>
 <div className="flex flex-wrap items-center gap-4">
 {/* eslint-disable-next-line @next/next/no-img-element */}

 {/* eslint-disable-next-line @next/next/no-img-element */}

 <div className="text-xs leading-6 text-slate-600">

```

```

 00000000 00 00 0000 00000000 0000 maskable 0 apple-touch-icon 00 000000 000000 00000000.
 </div>
</div>
</Card>
</main>
);
}

```



```

 }
 // 0) 0000000 00 0000000000 00 0000 0000 000 000000 000 00000
 if (regMask && maskPhone(entered) !== regMask) {
 setSimMismatch(
 `0 0000000 0000000000: 000000 0000000000 0000 0000 0000 000 000000 «${regMask}» 0000 000 000 «${maskPhone(entered)}» 0000
 000000000. 0000 000 00 0000000000 00000000 0000 000.`
);
 return;
 }
 }
 // 0) 0000000 00 0000000000 00 000000 000 0000 0000 000000 000
 const deviceSim = getDeviceSim();
 if (deviceSim && entered !== deviceSim) {
 setSimMismatch(
 `0 000 000 0000 0000000000 «${maskPhone(deviceSim)}» 0000 0 0000000000 000. 0000 00 000000 0000 0000 0000.`
);
 return;
 }
 setSimMismatch("");
}, [phone, mode, needPhone, regMask]);

// 00 00000 000 000000000 00 0000 0 0000 00 000000 000000 000000 0000 00000000
useEffect(() => {
 const u = username.trim();
 if (u.length < 2) {
 setHint("");
 setNeedPhone(true);
 return;
 }
 const t = setTimeout(async () => {
 const res = await fetch("/api/auth/check-username", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ username: u }),
 }).catch(() => null);
 if (!res?.ok) return;
 const d = await res.json();
 if (!d.exists) {
 setHint("");
 setNeedPhone(true);
 setRegMask("");
 setDeviceBound(false);
 return;
 }
 setNeedPhone(Boolean(d.requirePhone));
 setRegMask(d.phoneMask ?? "");
 setDeviceBound(Boolean(d.deviceBound));
 if (d.kind !== mode) {
 setHint(
 d.kind === "admin"
 ? "0000 0000 0000/00000000 000 — 000000 00 «0000 0000» 00 0000000 0000."
 : "0000 0000 00000000 0000 000 — 000000 00 «0000 00000000 0000» 00 0000000 0000.",
);
 } else setHint("");
 }, 400);
 return () => clearTimeout(t);
}, [username, mode]);

const requestOtp = async () => {
 setOtpBusy(true);
 const res = await fetch("/api/auth/otp", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ action: "request", username, password, deviceId: getDeviceId() }),
 }).catch(() => null);
 setOtpBusy(false);
 const d = await res?.json().catch(() => ({}));
 setOtpMsg(d?.message ?? d?.error ?? "000000 00 000000 000");
 if (d?.ok) {
 setOtpLeft(180);
 const iv = setInterval(() => {
 setOtpLeft((v) => {
 if (v <= 1) {
 clearInterval(iv);
 return 0;
 }
 return v - 1;
 });
 }, 1000);
 }
};

const verifyOtp = async () => {
 if (otpCode.trim().length < 4) {
 setOtpMsg("00 000000 00 00000 00000 00000");
 return;
 }

```

```

 }
 setOtpBusy(true);
 const res = await fetch("/api/auth/otp", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({
 action: "verify",
 username,
 password,
 code: otpCode.trim(),
 deviceId: getDeviceId(),
 deviceInfo: getDeviceInfo(),
 remember: true,
 }),
 });
 setOtpBusy(false);
 const d = await res.json().catch(() => ({}));
 if (!res.ok) {
 setOtpMsg(d.error ?? "❌ 验证失败");
 return;
 }
 if (d.token) setTabToken(d.token, true);
 if (d.user) cacheUser(d.user);
 if (phone) {
 localStorage.setItem("sek_phone", phone);
 setDeviceSim(normalizePhone(phone));
 }
 router.replace(d.user?.role === "rep" ? "/panel" : "/admin");
 };

const submit = async (e: React.FormEvent) => {
 e.preventDefault();
 setError("");
 if (mode === "rep" && needPhone && !simOk) {
 setError(simWarn || "❌ 请输入正确的手机号码");
 return;
 }
 if (mode === "rep" && needPhone && simMismatch) {
 setError(simMismatch);
 return;
 }
 if (needPhone && !/^0\d{10}$/.test(phone.replace(/\D/g, ""))) {
 setError("❌ 请输入正确的手机号码(00000: 00000000000)");
 return;
 }
 // 0000 000000: 000 00000000 0000 000 00000 00 0000 000000 0000 00000000
 if (!navigator.onLine) {
 const cached = getCachedUser();
 if (cached && cached.username === username.trim().toLowerCase()) {
 router.replace(cached.role === "rep" ? "/panel" : "/admin");
 return;
 }
 }
 setError("❌ 网络异常 请检查网络。 0000 000000 0000 000 000 00000000 000000 00000000 0000 000.");
 return;
}
setBusy(true);
const res = await fetch("/api/auth/login", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({
 username,
 password,
 phone,
 mode,
 remember,
 simActive: navigator.onLine,
 simActiveOnDevice: simOk && !simMismatch,
 deviceSim: getDeviceSim(),
 deviceId: getDeviceId(),
 deviceInfo: getDeviceInfo(),
 isMobile: isMobileDevice(),
 }),
});
setBusy(false);
const data = await res.json().catch(() => ({}));
if (!res.ok) {
 if (data.needOtp) {
 setError("");
 setOtpMsg(data.error ?? "");
 setOtpStep(true);
 requestOtp();
 return;
 }
}
setError(data.error ?? "❌ 登录失败");
if (data.simError) setSimMismatch(data.error ?? "");

```

```

 return;
 }
 if (data.token) setTabToken(data.token, true);
 if (data.user) cacheUser(data.user);
 if (phone) {
 localStorage.setItem("sek_phone", phone);
 // 0000 000000 0000000000 0000000000 0000 0000000 000 00000000
 if (data.user?.role === "rep") setDeviceSim(normalizePhone(phone));
 }
 router.replace(data.user?.role === "rep" ? "/panel" : "/admin");
};

return (
 <main className="flex min-h-screen items-center justify-center bg-gradient-to-b from-teal-700 via-teal-600 to-slate-
100 p-4">
 <div className="w-full max-w-md">
 {otpStep ? (
 <div className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/70 p-4">
 <div className="fade-in w-full max-w-sm rounded-2xl bg-white p-4">
 <h3 className="mb-1 text-sm font-black text-slate-800">□ □□□□□ □□□□□□□□□□</h3>
 <p className="mb-3 text-[11px] leading-6 text-slate-600">
 □□□□ □□□□□□□□ □□ □□□□ □□□□□□□□ □□□□□□ □□□□ □□□□ □□□□□□□□ □□□□ □□□□ □□□□ □□ □□□□□□□□□□ □□ □□□□
 □□□□. □□□ □□□□□ □□ □□□□□□ □□□□□□ □□□□ □□□□□□□□□□ □□ □□□ □□□□ □□□□ □□□□.
 </p>
 {otpMsg ? (
 <div className="mb-2">
 <Alert kind={otpMsg.includes("□□□□□□ □□") ? "success" : "info"}>{otpMsg}</Alert>
 </div>
) : null}
 <Field label="□□ □□□□□ required">
 <Input
 inputMode="numeric"
 value={otpCode}
 onChange={(e) => setOtpCode(e.target.value.replace(/\D/g, "").slice(0, 6))}
 placeholder="_ _ _ _ _"
 className="text-center text-lg tracking-[0.5em]"
 autoFocus
 />
 </Field>
 <div className="mt-3 flex flex-wrap gap-2">
 <Button onClick={verifyOtp} disabled={otpBusy}>
 {otpBusy ? "...": "□ □□□□□ □ □□□□□"}
 </Button>
 <Button variant="soft" onClick={requestOtp} disabled={otpBusy || otpLeft > 0}>
 {otpLeft > 0 ? `□□□□□ □□□□ ($ {toPersianDigits(otpLeft)})` : "□ □□□□□ □□□□ □□"}
 </Button>
 <Button
 variant="ghost"
 onClick={() => {
 setOtpStep(false);
 setOtpCode("");
 setOtpMsg("");
 }}
 >
 □□□□□□
 </Button>
 </div>
 </div>
 </div>
) : null}

 {forgot ? (
 <div
 className="fixed inset-0 z-50 flex items-center justify-center bg-slate-900/60 p-4"
 onClick={() => setForgot(false)}
 >
 <div
 className="fade-in w-full max-w-sm rounded-2xl bg-white p-4"
 onClick={(e) => e.stopPropagation()}
 >
 <h3 className="mb-2 text-sm font-black text-slate-800">□ □□□□□□□ □□□ □□□□</h3>
 <p className="mb-3 text-[11px] text-slate-500">
 □□□ □□□□□□ □ □□□□□ □□□□□ □□□ □□ □□□□ □□□□. □□□□□□□□ □□□□ □□□□ □□□□□ □ □□□ □□□□ □□ □□□
 □□□□□ □□□□□ □□.
 </p>
 {info ? (
 <div className="mb-2">
 <Alert kind="success">{info}</Alert>
 </div>
) : null}
 <div className="space-y-2">
 <Field label="□□□□ □□□□□□ required">
 <Input value={username} onChange={(e) => setUsername(e.target.value)} className="text-left" />
 </Field>
 <Field label="□□□□□□ □□□□□">

```

```

 <Input
 inputMode="numeric"
 value={phone}
 onChange={(e) => setPhone(e.target.value.replace(/\D/g, "").slice(0, 11))}
 />
 </Field>
 </div>
 <div className="mt-3 flex gap-2">
 <Button
 onClick={async () => {
 const res = await fetch("/api/auth/forgot", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ username, phone }),
 });
 const d = await res.json().catch(() => ({}));
 setInfo(d.message ?? d.error ?? "□□□□□□ □□");
 }}
 >
 □□□□□ □□□□□□□□
 </Button>
 <Button variant="ghost" onClick={() => setForgot(false)}>
 □□□□
 </Button>
 </div>
 </div>
 </div>
) : null}

 <div className="mb-5 text-center text-white">
 <div className="relative mx-auto mb-3 size-24">
 <div className="absolute inset-1 rounded-[1.75rem] bg-white/20 blur-lg" />
 <div className="relative flex size-24 items-center justify-center overflow-hidden rounded-[1.75rem] bg-white shadow-2xl ring-1 ring-white/70">
 /* □□□□□ □□□□ □□□□□ □□□□□□□□ */
 { /* eslint-disable-next-line @next/next/no-img-element */ }

 </div>

 □

 </div>
 <h1 className="text-2xl font-black drop-shadow-sm">□□□ □□□□□□□□ □□</h1>
 <p className="mt-1 text-xs font-medium text-teal-50/90">□□□□□□ □□□□□□ □□□□□□□□□□ □□□□</p>
 <div className="mx-auto mt-2 flex w-fit items-center gap-2 rounded-full bg-white/10 px-3 py-1 text-[10px] text-teal-50 ring-1 ring-white/20">
 □□ • □□□□□□□□ •
 </div>
 <div>
 □□□□□□ • □□□□□□
 </div>

 <form onSubmit={submit} className="space-y-3 rounded-3xl bg-white p-5 shadow-xl ring-1 ring-slate-200">
 <div className="flex rounded-xl bg-slate-100 p-1 text-xs font-bold">
 <button
 type="button"
 onClick={() => {
 setMode("rep");
 setUsername("");
 setPassword("");
 setError("");
 }}
 className={`flex-1 rounded-lg py-2 ${mode === "rep" ? "bg-teal-600 text-white" : "text-slate-600"}`
 >
 □□□□ □□□□ □□□□□□□□ □□□□
 </button>
 <button
 type="button"
 onClick={() => {
 setMode("admin");
 setUsername("");
 setPassword("");
 setError("");
 }}
 className={`flex-1 rounded-lg py-2 ${mode === "admin" ? "bg-slate-800 text-white" : "text-slate-600"}`
 >
 □ □□□□ □□□□ / □□□□□□□□
 </button>
 </div>

```

```

 {mode === "rep" && simWarn ? <Alert kind="error">{simWarn}</Alert> : null}
 {mode === "rep" && simMismatch ? <Alert kind="error">{simMismatch}</Alert> : null}
 {hint ? <Alert kind="info">{hint}</Alert> : null}
 {error ? <Alert kind="error">{error}</Alert> : null}

<Field label="□□□ □□□□□ required">
 <Input
 value={username}
 onChange={(e) => setUsername(e.target.value)}
 placeholder={mode === "admin" ? "□□□ □□□□□ □□□□" : "□□□ □□□□□ □□□□□□□□"}
 autoComplete="off"
 />
</Field>

<Field label="□□□ □□□□ required">
 <div className="relative">
 <Input
 type={showPass ? "text" : "password"}
 value={password}
 onChange={(e) => setPassword(e.target.value)}
 placeholder="....."
 autoComplete="off"
 className="pl-12"
 />
 <button
 type="button"
 onClick={() => setShowPass((v) => !v)}
 className="absolute left-2 top-1/2 -translate-y-1/2 rounded-lg bg-slate-100 px-2 py-1 text-xs"
 title={showPass ? "□□□□□ □□□□ □□□" : "□□□□□ □□□□"}
 >
 {showPass ? "□" : "□"}
 </button>
 </div>
</Field>

{mode === "rep" && needPhone ? (
 <div
 className={`rounded-xl px-3 py-2 text-[11px] font-bold ring-1 ${
 simMismatch
 ? "bg-rose-50 text-rose-800 ring-rose-300"
 : !simOk
 ? "bg-amber-50 text-amber-800 ring-amber-300"
 : "bg-emerald-50 text-emerald-800 ring-emerald-200"
 }`}
 >
 <div className="flex flex-wrap items-center gap-2">
 {simMismatch ? "□" : simOk ? "□" : "□□"} □□□□□ □□□□□□□□ □□ □□□□□□
 {netType ? □□□□: {netType} : null}
 {regMask ? □□□□□ □□□□: {regMask} : null}
 {deviceBound ? □ □□□□ □□ □□ □□□□ : null}
 </div>
 {!simMismatch && simOk ? (
 <div className="mt-1 font-normal">
 □□□□ □□□ □□ □□□□ □□□□ □□□□□□□□ □□□□□□ □□□□ □□□□ □□□□□□□□□□ □□□.
 </div>
) : null}
 </div>
) : null}

{needPhone ? (
 <Field label="□□□□□ □□□□□ □□□□ □□□ □□□ □□□□" required hint="□□ □□□□ □□□□□□□□ □□□□ □□□□ □□□□□ □□□□□ □□□□□□□□"
 >
 <Input
 inputMode="numeric"
 value={phone}
 onChange={(e) => setPhone(e.target.value.replace(/[\^d]/g, "").slice(0, 11))}
 placeholder="09xxxxxxxxx"
 />
 </Field>
) : (
 <Alert kind="info">□□□□ □□□ □□□□□ □□□□ □□□□ □□□□□ □□□□□ □□□□ □□□ □□□.</Alert>
)}

<div className="flex items-center justify-between">
 <button
 type="button"
 onClick={() => {
 setForgot(true);
 setInfo("");
 }}
 className="text-[11px] font-bold text-teal-700 underline"
 >
 □ □□□ □□□□ □□ □□□□□□ □□□□□□□□
 </button>

```

```

</div>

<label className="flex items-center gap-2 text-xs font-bold text-slate-600">
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={remember}
 onChange={(e) => setRemember(e.target.checked)}
 />
 0000 00 00000 000000 (00 0000 0000 00 00 00000 00000 00 00000 00000 00000)
</label>

<Button type="submit" disabled={busy || (mode === "rep" && needPhone && !!simMismatch)} className="w-full">
{busy ? "00 0000 00000...": mode === "admin" ? "0 00000 00 000000000 00000": "0 00000 00 000 000000000"}
</Button>

<div className="pt-1 text-center">

 0 00000 00 00000 00 0000000000 0000000000 00000

</div>

<p className="pt-1 text-center text-[11px] leading-5 text-slate-400">
0000: admin / admin1234 — 0000000: sarparast / sar1234 — 00000000: rep1 / rep1234
</p>
</form>
</div>
</main>
);
}

@@ src/app/login/page.tsx typescript
import LoginForm from "../LoginForm";

export default function LoginPage() {
 return <LoginForm />;
}

@@ src/app/offline/page.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import { getCachedUser } from "@lib/offline-session";

/**
 * 00000 «00000000 00000000 00000» — 00000000 00000:
 * - 00 0 000000 00000000 000000 00 000000 00000000 0 0000000 000000000 00000 00000000 00000000
 * - 00000 «000000 00 00000 00000000» 000000 00 00000 0000 00000000 00 00000 00 0000000 0000
 */
export default function OfflinePage() {
 const [checking, setChecking] = useState(false);
 const [tries, setTries] = useState(0);
 const [target, setTarget] = useState("/login");

 useEffect(() => {
 const u = getCachedUser();
 if (u) setTarget(u.role === "rep" ? "/panel" : "/admin");
 }, []);

 const check = useCallback(
 async (auto = false) => {
 setChecking(true);
 if (!auto) setTries((t) => t + 1);
 try {
 const ctrl = new AbortController();
 const timer = setTimeout(() => ctrl.abort(), 8000);
 const res = await fetch(`/ping?t=${Date.now()}`, { cache: "no-store", signal: ctrl.signal });
 clearTimeout(timer);
 if (res.ok) {
 window.location.replace(target);
 return true;
 }
 } catch {
 /* 00000 00000000 */
 } finally {
 setChecking(false);
 }
 return false;
 },
 [target],
);

 useEffect(() => {
 const iv = setInterval(() => check(true), 8000);
 const onOnline = () => check(true);
 }, []);
}

```

```

window.addEventListener("online", onOnline);
return () => {
 clearInterval(iv);
 window.removeEventListener("online", onOnline);
};
}, [check]);

return (
 <main className="flex min-h-screen flex-col items-center justify-center gap-4 bg-slate-100 p-6 text-center">
 <div className="text-5xl"></div>
 <h1 className="text-lg font-black text-slate-800">
 <p className="max-w-sm text-sm leading-7 text-slate-600">
 </p>

 <div className="flex flex-wrap items-center justify-center gap-2">
 <button
 onClick={() => check(false)}
 disabled={checking}
 className="rounded-xl bg-teal-600 px-5 py-2.5 text-sm font-bold text-white disabled:opacity-60"
 >
 {checking ? "⏳ ⏳⏳ ⏳⏳⏳⏳..." : "⏳ ⏳⏳⏳ ⏳⏳⏳"}
 </button>
 <a href={target} className="rounded-xl bg-white px-5 py-2.5 text-sm font-bold text-teal-700 ring-1 ring-teal-300"
 >
 ⏳ ⏳⏳⏳ ⏳ ⏳⏳⏳ ⏳⏳⏳⏳⏳

 ⏳ ⏳⏳⏳⏳⏳⏳

 </div>

 <p className="text-[11px] text-slate-400">
 ⏳⏳⏳⏳ ⏳⏳⏳⏳ ⏳ ⏳ ⏳⏳⏳⏳ ⏳⏳⏳⏳ ⏳⏳⏳⏳⏳⏳(tries > 0 ? ` — ${tries.toLocaleString("fa-IR")} ⏳⏳⏳ ⏳⏳⏳` : "")
 </p>

 <div className="mt-2 max-w-sm rounded-xl bg-white p-3 text-right text-[11px] leading-6 text-slate-500 ring-1 ring-slate-200">
 <b className="text-slate-700">⏳⏳ ⏳ ⏳⏳⏳⏳⏳⏳ ⏳⏳ ⏳⏳⏳⏳⏳ ⏳⏳ ⏳ ⏳⏳⏳⏳⏳⏳ ⏳⏳⏳⏳ ⏳⏳: ⏳⏳⏳⏳⏳⏳ ⏳⏳ ⏳⏳⏳⏳⏳ ⏳⏳
 ⏳⏳⏳ ⏳ ⏳⏳⏳⏳ ⏳⏳⏳ ⏳⏳. ⏳⏳⏳⏳⏳ ⏳ ⏳⏳⏳ ⏳⏳⏳⏳⏳⏳⏳⏳ ⏳⏳ ⏳⏳⏳⏳⏳ ⏳ ⏳⏳⏳⏳⏳⏳⏳ ⏳ ⏳ ⏳⏳⏳⏳⏳⏳ ⏳⏳ ⏳⏳ ⏳⏳⏳⏳⏳⏳ ⏳⏳ ⏳⏳⏳
 ⏳ ⏳⏳⏳⏳⏳ ⏳⏳⏳.
 </div>
 </main>
);
 }
}

```

```
@@ src/app/panel/layout.tsx typescript
"use client";
```

```
import type { ReactNode } from "react";
import Shell, { type NavItem } from "@components/Shell";
import SessionProvider, { useSession } from "@components/SessionProvider";

function Inner({ children }: { children: ReactNode }) {
 const { me } = useSession();
 if (!me) return null;
 const has = (p: string) => me.role === "admin" || me.permissions.includes(p);

 const all: (NavItem & { perm?: string })[] = [
 { href: "/panel", label: "🏠🏠🏠🏠", icon: "🏠" },
 { href: "/panel/pharmacies", label: "🏪🏪🏪🏪🏪🏪", icon: "🏪", perm: "pharmacy" },
 { href: "/panel/doctors", label: "👨👨👨👨", icon: "👨", perm: "doctor" },
 { href: "/panel/orders", label: "📦📦📦📦📦", icon: "📦", perm: "order" },
 { href: "/panel/trip", label: "🗺️🗺️ 🗺️🗺️", icon: "🗺️", perm: "trip" },
 { href: "/panel/map", label: "📍📍📍📍", icon: "📍" },
 { href: "/panel/home", label: "🏠🏠🏠🏠🏠", icon: "🏠", perm: "home" },
 { href: "/panel/leaves", label: "📅📅📅", icon: "📅", perm: "leave" },
 { href: "/panel/notifications", label: "🔔🔔🔔🔔", icon: "🔔" },
 { href: "/panel/options", label: "⚙️⚙️⚙️⚙️", icon: "⚙️", perm: "options" },
 { href: "/panel/reports", label: "📊📊📊📊", icon: "📊", perm: "reports" },
 { href: "/install", label: "🔧🔧🔧", icon: "🔧" },
 { href: "/diagnostics", label: "🔍🔍🔍🔍", icon: "🔍" },
];

 const nav = all.filter((n) => !n.perm || has(n.perm)).map(({ href, label, icon }) => ({ href, label, icon }));

 return (
 <Shell
 userName={me.fullName}
 roleLabel={me.role === "admin" ? "👑👑👑" : me.role === "supervisor" ? "👮👮👮" : "👷👷👷👷👷"}
 nav={nav}
 showAdminLink={me.role !== "rep"}
 >
 {children}
 </Shell>
);
}
```

```

</Shell>
);
}

export default function PanelLayout({ children }: { children: ReactNode }) {
 return (
 <SessionProvider require>
 <Inner>{children}</Inner>
 </SessionProvider>
);
}

@@ src/app/panel/page.tsx typescript
"use client";

import Link from "next/link";
import { useEffect, useState } from "react";
import { useSession } from "@components/SessionProvider";
import dynamic from "next/dynamic";

const TargetPanel = dynamic(() => import("@components/screens/TargetPanel"), { ssr: false });

const ChartsPanel = dynamic(() => import("@components/screens/ChartsPanel"), {
 ssr: false,
 loading: () => <div className="rounded-2xl bg-white p-6 text-center text-xs text-slate-400">□□ □□□ □□□□□□□□ □□□□□□□□..</div>,
});
import { toPersianDigits, todayJalali } from "@lib/jalali";

export default function PanelHome() {
 const { me } = useSession();
 const [counts, setCounts] = useState({ ph: 0, dr: 0, or: 0 });

 useEffect(() => {
 fetch("/api/reports", { cache: "no-store" })
 .then((r) => (r.ok ? r.json() : null))
 .then((d) => {
 const rows = (d?.rows ?? []) as { pharmacies: number; doctors: number; orders: number }[];
 setCounts({
 ph: rows.reduce((a, r) => a + r.pharmacies, 0),
 dr: rows.reduce((a, r) => a + r.doctors, 0),
 or: rows.reduce((a, r) => a + r.orders, 0),
 });
 })
 .catch(() => undefined);
 }, []);

 if (!me) return null;
 const has = (p: string) => me.role === "admin" || me.permissions.includes(p);

 const main = [
 { href: "/panel/pharmacies", title: "□□□□ □□□□□□□□ □□□□□□□□", icon: "□", n: counts.ph, perm: "pharmacy", color: "from-teal-600 to-teal-500" },
 { href: "/panel/doctors", title: "□□□□ □□□□□□□□ □□□□", icon: "□", n: counts.dr, perm: "doctor", color: "from-sky-600 to-sky-500" },
 { href: "/panel/orders", title: "□□□□ □□□□□□□□ □□□□□□□□", icon: "□", n: counts.or, perm: "order", color: "from-indigo-600 to-indigo-500" },
].filter((c) => has(c.perm));

 const more = [
 { href: "/panel/trip", title: "□□□□□ □ □□□□□□", icon: "□□", perm: "trip" },
 { href: "/panel/home", title: "□□□□□□□□ □□□□□", icon: "□", perm: "home" },
 { href: "/panel/leaves", title: "□□□□□□□□ □□□□□□", icon: "□", perm: "leave" },
 { href: "/panel/notifications", title: "□□□□□□□□□□", icon: "□", perm: "" },
 { href: "/panel/options", title: "□□□□□□□ □□□□□□", icon: "□", perm: "options" },
 { href: "/panel/reports", title: "□□□□□□ □□□□□□", icon: "□", perm: "reports" },
].filter((c) => !c.perm || has(c.perm));

 return (
 <div className="space-y-4">
 <div className="rounded-2xl bg-white p-4 shadow-sm ring-1 ring-slate-200">
 <h1 className="text-lg font-black text-slate-800">□□□□ {me.fullName} □</h1>
 <p className="mt-1 text-sm text-slate-500">□□□□□ {toPersianDigits(todayJalali())}</p>
 </div>

 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
 {main.map((c) => (
 <Link
 key={c.href}
 href={c.href}
 className={`flex items-center gap-4 rounded-2xl bg-gradient-to-l ${c.color} p-4 text-white shadow-md active:scale-[0.99]`}
 >

 {c.icon}

 </Link>
))}
 </div>
 </div>
);
}

```



```


 {c.title}
 {toPersianDigits(c.n)} □□□□□

 </Link>
)}
 </div>

 <div className="grid grid-cols-2 gap-3 sm:grid-cols-3 lg:grid-cols-6">
 {more.map((c) => (
 <Link
 key={c.href}
 href={c.href}
 className="flex flex-col items-center gap-2 rounded-2xl bg-white p-4 text-center shadow-sm ring-1 ring-slate
-200 hover:ring-teal-300"
 >
 {c.icon}
 {c.title}
 </Link>
)}
 </div>

 {has("order") ? <TargetPanel /> : null}

 {has("reports") ? <ChartsPanel scope="rep" /> : null}

 {main.length + more.length === 0 ? (
 <div className="rounded-2xl bg-amber-50 p-4 text-sm text-amber-800 ring-1 ring-amber-200">
 □□□□ □□□□□□□□□□ □□□□ □□□ □□□□□ □□□□ □□□□ □□□□ □□□□ □□□□□□□□.
 </div>
) : null}
</div>
);
}

@@ src/app/ping/route.ts typescript
/**
 * □□□□ □□□□□ /ping — Keep-Alive □□□□ □□□□□ □□□□ □ □□□□ □□□□ □□ □□□□□□□□.
 *
 * □□□□ □□□□□□□□□□ □□□□□□□□□□ (UptimeRobot□ BetterStack□ cron-job.org):
 * URL : https://<your-app>.onrender.com/ping
 * Interval: □□ □ □□□□□
 */
export const dynamic = "force-dynamic";
export const revalidate = 0;

const startedAt = Date.now();

const HEADERS = {
 "Content-Type": "text/plain; charset=utf-8",
 "Cache-Control": "no-store, no-cache, must-revalidate, max-age=0",
 "X-Robots-Tag": "noindex",
 "Access-Control-Allow-Origin": "*",
} as const;

export async function GET(req: Request) {
 if (!new URL(req.url).searchParams.has("json")) {
 return new Response("OK", { status: 200, headers: HEADERS });
 }
 return new Response(
 JSON.stringify({ status: "OK", uptimeMs: Date.now() - startedAt, timestamp: new Date().toISOString() }),
 { status: 200, headers: { ...HEADERS, "Content-Type": "application/json; charset=utf-8" } },
);
}

export async function HEAD() {
 return new Response(null, { status: 200, headers: HEADERS });
}

@@ src/components/screens/ChartsPanel.tsx typescript
"use client";

import { useCallback, useEffect, useMemo, useState } from "react";
import { Button, Card, Field, SectionTitle } from "@components/ui";
import { BarChart, DonutChart, LineChart } from "@components/Charts";
import JalaliDateInput from "@components/JalaliDateInput";
import { JALALI_MONTHS, toPersianDigits, todayJalali } from "@lib/jalali";
import { Alert } from "@components/ui";
import { DEFAULT_PRODUCTS, type ProductConfig } from "@lib/defaults";

type Row = {
 period: string;
 repName: string;
 pharmacies: number;

```

```

doctors: number;
orders: number;
units: number;
bonus: number;
trips: number;
};
type ByRep = { period: string; repName: string; items: Record<string, number>; bonuses: Record<string, number> };

const MONTH_LABELS = JALALI_MONTHS.map((m) => m.slice(0, 4));

export default function ChartsPanel({ scope = "rep" }: { scope?: "rep" | "admin" }) {
 const [rows, setRows] = useState<Row[]>([]);
 const [byRep, setByRep] = useState<ByRep[]>([]);
 const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
 const [metric, setMetric] = useState<"orders" | "units" | "pharmacies" | "doctors" | "trips">("units");
 const [rep, setRep] = useState("");
 const [from, setFrom] = useState("");
 const [to, setTo] = useState("");
 // ۰۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰ ۰۰ ۰۰۰۰ ۰۰۰۰ «۰۰۰۰۰ ۰۰۰۰» ۰۰۰۰۰ ۰۰۰۰۰۰
 const [applied, setApplied] = useState<{ from: string; to: string; rep: string }>({ from: "", to: "", rep: "" });
 const [msg, setMsg] = useState("");

 const load = useCallback(async () => {
 const [r1, r2] = await Promise.all([
 fetch("/api/reports", { cache: "no-store" }),
 fetch("/api/settings?key=products", { cache: "no-store" }),
]);
 if (r1.ok) {
 const d = await r1.json();
 setRows(d.rows ?? []);
 setByRep(d.productByRep ?? []);
 }
 if (r2.ok) {
 const d = await r2.json();
 const v = d.value as ProductConfig[] | null;
 if (Array.isArray(v) && v.length) setProducts(v.filter((p) => p.enabled !== false));
 }
 }, []);

 useEffect(() => {
 load();
 }, [load]);

 const reps = useMemo(() => [...new Set(rows.map((r) => r.repName))].sort(), [rows]);
 const years = useMemo(() => [...new Set(rows.map((r) => r.period.slice(0, 4))].sort(), [rows]);

 const inRange = useCallback(
 (period: string) => {
 if (applied.from && period < applied.from.slice(0, 7)) return false;
 if (applied.to && period > applied.to.slice(0, 7)) return false;
 return true;
 },
 [applied],
);

 const scoped = useMemo(
 () => rows.filter((r) => (!applied.rep || r.repName === applied.rep) && inRange(r.period)),
 [rows, applied.rep, inRange],
);

 const apply = () => {
 if (from && to && from > to) {
 setMsg("۰۰ «۰۰ ۰۰۰۰» ۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰ «۰۰ ۰۰۰۰» ۰۰۰۰");
 return;
 }
 setApplied({ from, to, rep });
 setMsg(
 from || to || rep
 ? `۰ ۰۰۰۰ ۰۰۰۰ ۰۰ ${from ? `— ۰۰ ${toPersianDigits(from)}` : ""} ${to ? `۰ ۰۰ ${toPersianDigits(to)}` : ""} ${rep ? `| ${rep}` : ""}` : ""`
 : "۰ ۰۰۰۰ ۰۰ ۰۰۰۰۰۰",
);
 load();
 };

 const reset = () => {
 setFrom("");
 setTo("");
 setRep("");
 setApplied({ from: "", to: "", rep: "" });
 setMsg("۰۰۰۰ ۰۰ ۰۰ — ۰۰ ۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰");
 };

 const quick = (months: number) => {
 const t = todayJalali();

```

```

const [jy, jm] = t.split("/").map(Number);
let y = jy;
let m = jm - months + 1;
while (m <= 0) {
 m += 12;
 y -= 1;
}
const f = `${y}/${String(m).padStart(2, "0")}/01`;
setFrom(f);
setTo(t);
setApplied({ from: f, to: t, rep });
setMsg(␣ `${toPersianDigits(months)} ␣␣␣ ␣␣␣␣`);
};

const metricLabel: Record<string, string> = {
 units: "␣␣␣␣␣ ␣␣␣␣␣ ␣␣␣␣␣␣␣␣",
 orders: "␣␣␣␣␣ ␣␣␣␣␣",
 pharmacies: "␣␣␣␣␣␣␣␣ ␣␣␣␣␣␣␣",
 doctors: "␣␣␣␣ ␣␣␣␣␣␣␣",
 trips: "␣␣␣ ␣␣␣␣␣",
};

// ␣␣␣␣␣␣ ␣␣␣ ␣␣ ␣␣␣: ␣␣ ␣␣␣ ␣␣ ␣␣␣ ␣␣␣␣ ␣␣␣␣␣␣␣
const yearSeries = useMemo(
 () =>
 years.map((y) => ({
 name: toPersianDigits(y),
 values: Array.from({ length: 12 }, (_, m) => {
 const per = `${y}/${String(m + 1).padStart(2, "0")}`;
 return scoped.filter((r) => r.period === per).reduce((a, r) => a + r[metric], 0);
 }),
 })),
 [years, scoped, metric],
);

// ␣␣␣ ␣␣ ␣␣␣ (␣␣␣␣ ␣␣␣)
const monthly = useMemo(() => {
 const m = new Map<string, number>();
 for (const r of scoped) m.set(r.period, (m.get(r.period) ?? 0) + r[metric]);
 return [...m.entries()]
 .sort((a, b) => a[0].localeCompare(b[0]))
 .map(([label, value]) => ({ label: toPersianDigits(label), value }));
}, [scoped, metric]);

// ␣␣␣ ␣␣ ␣␣␣␣
const productShare = useMemo(() => {
 const filtered = byRep.filter((r) => (!applied.rep || r.repName === applied.rep) && inRange(r.period));
 return products
 .map((p) => ({
 label: p.label,
 value: filtered.reduce((a, r) => a + (r.items[p.key] ?? 0), 0),
 }))
 .filter((d) => d.value > 0);
}, [byRep, products, applied.rep, inRange]);

// ␣␣␣␣␣␣ ␣␣␣␣␣␣␣␣␣ (␣␣␣ ␣␣␣␣)
const repCompare = useMemo(() => {
 const m = new Map<string, number>();
 for (const r of rows.filter((x) => inRange(x.period))) m.set(r.repName, (m.get(r.repName) ?? 0) + r[metric]);
 return [...m.entries()].map(([label, value]) => ({ label, value })).sort((a, b) => b.value - a.value);
}, [rows, metric, inRange]);

const totals = scoped.reduce(
 (a, r) => ({
 pharmacies: a.pharmacies + r.pharmacies,
 doctors: a.doctors + r.doctors,
 orders: a.orders + r.orders,
 units: a.units + r.units,
 bonus: a.bonus + r.bonus,
 trips: a.trips + r.trips,
 }),
 { pharmacies: 0, doctors: 0, orders: 0, units: 0, bonus: 0, trips: 0 },
);

return (
 <div className="space-y-4">
 <SectionTitle icon="␣">␣␣␣␣␣␣␣ ␣ ␣␣␣␣␣ ␣␣␣␣␣␣</SectionTitle>

 <Card>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
 <Field label="␣␣␣␣ ␣␣␣␣␣">
 <select
 value={metric}
 onChange={(e) => setMetric(e.target.value as typeof metric)}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >

```

```

 >
 {Object.entries(metricLabel).map(([k, v]) => (
 <option key={k} value={k}>
 {v}
 </option>
))}
 </select>
</Field>
{scope === "admin" ? (
<Field label="□□□□□□□□">
 <select
 value={rep}
 onChange={(e) => setRep(e.target.value)}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="">□□□ □□□□□□□□□□</option>
 {reps.map((r) => (
 <option key={r} value={r}>
 {r}
 </option>
))}
 </select>
</Field>
) : null}
<Field label="□□ □□□□□□">
 <JalaliDateInput value={from} onChange={setFrom} placeholder="□□□□/□□/□□" />
</Field>
<Field label="□□ □□□□□□">
 <JalaliDateInput value={to} onChange={setTo} placeholder="□□□□/□□/□□" />
</Field>
</div>

 <div className="mt-3 flex flex-wrap items-center gap-2">
<Button onClick={apply}>□ □□□□□ □□□□ □ □□□□□</Button>
 <Button variant="ghost" onClick={reset}>
 □□□ □□□□ □□□□
 </Button>

 <Button variant="soft" onClick={() => quick(1)}>
 □□□ □□□
 </Button>
 <Button variant="soft" onClick={() => quick(3)}>
 □ □□□ □□□□
 </Button>
 <Button variant="soft" onClick={() => quick(6)}>
 □ □□□ □□□□
 </Button>
 <Button variant="soft" onClick={() => quick(12)}>
 □□ □□□ □□□□
 </Button>
</div>
{msg ? (
 <div className="mt-2">
<Alert kind={msg.startsWith("□□") ? "error" : "success"}>{msg}</Alert>
 </div>
) : null}

 <div className="mt-3 grid grid-cols-3 gap-2 sm:grid-cols-6">
 {
 ["□ □□□□□□□□□", totals.pharmacies],
 ["□ □□□□", totals.doctors],
 ["□ □□□□□", totals.orders],
 ["□ □□□□□", totals.units],
 ["□ □□□□□", totals.bonus],
 ["□□ □□□□□", totals.trips],
].map(([l, v]) => (
 <div key={String(l)} className="rounded-xl bg-slate-50 p-2 text-center ring-1 ring-slate-200">
 <div className="text-base font-black text-teal-700">{toPersianDigits(v as number)}</div>
 <div className="text-[10px] font-bold text-slate-500">{l as string}</div>
 </div>
))}
 </div>
</Card>

<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">
 □ □□□□□□ □□□ □□ □□□ — {metricLabel[metric]}
 </h3>
 <LineChart labels={MONTH_LABELS} series={yearSeries} />
</Card>

 <div className="grid grid-cols-1 gap-3 lg:grid-cols-2">
 <Card>
<h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□□ □□□ □□ □□□</h3>
 <BarChart data={monthly} />

```

```

 </Card>
 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□ □□ □□□□ □□ □□□□</h3>
 <DonutChart data={productShare} />
 </Card>
 </div>

 {scope === "admin" ? (
 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□□□□ □□□□□□□□ — {metricLabel[metric]}</h3>
 <BarChart data={repCompare} />
 </Card>
) : null}
 </div>
);
}

@@ src/components/screens/HomeScreen.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import dynamic from "next/dynamic";
import LocationPicker, { type LatLng } from "@/components/LocationPicker";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "@/components/ui";
import NavButton from "@/components/NavButton";
import { toPersianDigits } from "@/lib/jalali";
import { useLive } from "@/lib/useLive";
import { useConfirm } from "@/components/Confirm";

const MapBox = dynamic(() => import("@/components/MapBox"), { ssr: false });

type Home = {
 id: number;
 userId: number;
 repName: string;
 title: string;
 address: string;
 lat: number;
 lng: number;
};

export default function HomeScreen({ isAdmin = false }: { isAdmin?: boolean }) {
 const [rows, setRows] = useState<Home[]>([]);
 const [title, setTitle] = useState("□□□□");
 const [address, setAddress] = useState("");
 const [loc, setLoc] = useState<LatLng>({ lat: null, lng: null, accuracy: null });
 const [msg, setMsg] = useState<{ kind: "success" | "error"; text: string } | null>(null);
 const [repFilter, setRepFilter] = useState("");
 const confirm = useConfirm();

 const load = useCallback(async () => {
 const res = await fetch("/api/homes", { cache: "no-store" });
 if (res.ok) setRows((await res.json()).rows ?? []);
 }, []);

 useLive(load, 25000);

 const reps = [...new Set(rows.map((r) => r.repName))].sort();
 const visible = rows.filter((r) => !repFilter || r.repName === repFilter);
 const grouped = (() => {
 const m = new Map<string, Home[]>();
 for (const r of visible) {
 if (!m.has(r.repName)) m.set(r.repName, []);
 m.get(r.repName)!.push(r);
 }
 return [...m.entries()].sort((a, b) => a[0].localeCompare(b[0]));
 })();

 const save = async () => {
 if (!loc.lat || !loc.lng) {
 setMsg({ kind: "error", text: "□□□□□ □□□□□□ □□□□ □□ □□ □□□□ □□□□ □□□□" });
 return;
 }
 if (!(await confirm({ title: "□□□ □□□□□□ □□□□", message: `«${title}» □□ □□□□□□ □□□□□□□□□□ □□ □□□□`, confirmText: "□□ □" }))) {
 return;
 }
 const res = await fetch("/api/homes", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ title, address, ...loc }),
 });
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setMsg({ kind: "success", text: "□ □□□□□□ □□□□ □□□ □□ □ □□ □□□□ □□□□ □□□□ □□□□□□" });
 load();
 }
 };

```

```

} else setMsg({ kind: "error", text: d.error ?? "□□□ □□ □□□" });
};

return (
 <div className="space-y-4">
 <SectionTitle icon="□">{isAdmin ? "□□□□□□ □□□□ □□□□□□□□": "□□□ □□□□□□ □□□□"}</SectionTitle>
 {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}

 {!isAdmin ? (
 <Card>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2">
 <Field label="□□□ □□□□ / □□□□□" required>
 <Input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="□□□□□ □□□□ □□□□" />
 </Field>
 <Field label="□□□□□">
 <TextArea value={address} onChange={(e) => setAddress(e.target.value)} />
 </Field>
 </div>
 <div className="mt-3">
 <LocationPicker value={loc} onChange={setLoc} label={title} suggestQuery={address || title} />
 </div>
 <div className="mt-3 flex justify-end">
 <Button onClick={save}>□ □□□ □□□□□□ □□□□</Button>
 </div>
 </Card>
) : null}

 <Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <h3 className="text-sm font-bold text-slate-700">
 {isAdmin ? "□□□□□ □□□□□□□ — □□ □□□□□ □□□□□□□": "□□□□ □□□□□□□ □□□"}
 </h3>
 {isAdmin && reps.length > 1 ? (
 <select
 value={repFilter}
 onChange={(e) => setRepFilter(e.target.value)}
 className="rounded-xl border border-slate-300 px-3 py-2 text-xs"
 >
 <option value="">□□□ □□□□□□□□□□ ({toPersianDigits(reps.length)})</option>
 {reps.map((r) => (
 <option key={r} value={r}>
 {r}
 </option>
))}
 </select>
) : null}
 </div>

 {visible.length === 0 ? (
 <p className="text-sm text-slate-400">□□□□□ □□□ □□□□ □□□</p>
) : (
 <>
 <MapBox
 height={300}
 points={visible.map((r) => ({
 lat: r.lat,
 lng: r.lng,
 label: `□ ${r.title} — ${r.repName}`,
 color: "#7c3aed",
 })))}
 />
 <div className="mt-3 space-y-2">
 {grouped.map(([repName, list]) => (
 <div key={repName}>
 <div className="mb-1 flex items-center gap-2 rounded-lg bg-slate-100 px-2 py-1.5">
 {repName}
 <Badge tone="slate">{toPersianDigits(list.length)} □□□□</Badge>
 </div>
 {list.map((r) => (
 <div key={r.id} className="mb-1 flex flex-wrap items-center gap-2 rounded-lg bg-slate-50 px-3 py-2 t
ext-xs">
 □ {r.title}
 {r.address}

 <NavButton lat={r.lat} lng={r.lng} label={` ${r.title} — ${r.repName}`} />

 </div>
))}
 </div>
 </div>
 </>
)}
 </Card>
 </div>

```

```

 });
}

@@ src/components/screens/LeaveScreen.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import JalaliDateInput from "@components/JalaliDateInput";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "@components/ui";
import { toPersianDigits, todayJalali } from "@lib/jalali";
import { useSession } from "@components/SessionProvider";
import Combobox from "@components/Combobox";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { JALALI_MONTHS } from "@lib/jalali";
import { downloadFile } from "@lib/download";

type Leave = {
 id: number;
 repName: string;
 kind: string;
 fromDate: string;
 toDate: string;
 days: number;
 fromTime: string;
 toTime: string;
 hours: number;
 reason: string;
 supervisorStatus: string;
 supervisorNote: string;
 supervisorName: string;
 managerStatus: string;
 managerNote: string;
 managerName: string;
};

const KINDS = ["□□□□□□", "□□□□□□", "□□□□□□□□", "□□□□□□□□ □□□□", "□□□□ □□□□", "□□□□□□□□"];

function StatusBadge({ s }: { s: string }) {
 if (s === "approved") return <Badge tone="green">□□□□□ □□</Badge>;
 if (s === "rejected") return <Badge tone="amber">□□ □□</Badge>;
 return <Badge tone="slate">□□ □□□□□□</Badge>;
}

export default function LeaveScreen() {
 const { me } = useSession();
 const role = me?.role ?? "rep";
 const [showForm, setShowForm] = useState(true);
 const [rows, setRows] = useState<Leave[]>([]);
 const [kind, setKind] = useState("□□□□□□");
 const [fromDate, setFrom] = useState(todayJalali());
 const [toDate, setTo] = useState(todayJalali());
 const [days, setDays] = useState("1");
 const [fromTime, setFromTime] = useState("08:00");
 const [toTime, setToTime] = useState("12:00");
 const [reason, setReason] = useState("");
 const [msg, setMsg] = useState<{ kind: "success" | "error"; text: string } | null>(null);
 const [kinds, setKinds] = useState<string[]>(KINDS);
 const [period, setPeriod] = useState("");
 const confirm = useConfirm();

 const loadKinds = useCallback(async () => {
 const res = await fetch("/api/options?category=leaveKind", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 const vals = (d.rows ?? []).map((r: { value: string }) => r.value);
 setKinds([...new Set([...KINDS, ...vals])]);
 }, []);

 const load = useCallback(async () => {
 const res = await fetch("/api/leaves", { cache: "no-store" });
 if (res.ok) setRows((await res.json()).rows ?? []);
 }, []);

 useEffect(() => {
 loadKinds();
 }, [loadKinds]);

 // □□□□□□□□□□ □□□□□□□□ □□□□□□ □□□□□□□□
 useLive(load, 20000);

 const submit = async () => {
 const summary = isHourly
 ? `□□□□□ □□□□□ ${toPersianDigits(fromDate)} □□ □□□□ ${toPersianDigits(fromTime)} □□ □${toPersianDigits(toTime)} ${toPersianDigits(hoursCalc)} □□□□`
 };

```

```

: `toPersianDigits(fromDate)} و ${toPersianDigits(toDate)} (${toPersianDigits(days)} روز)`;
if (!(await confirm({ title: "تایید درخواست مرخصی", message: `${summary}\nلطفاً تایید کنید`, confirmText: "تایید درخواست" })))
))
 return;
const res = await fetch("/api/leaves", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({
 kind,
 fromDate,
 toDate,
 days: isHourly ? 0 : Number(days) || 1,
 fromTime: isHourly ? fromTime : "",
 toTime: isHourly ? toTime : "",
 hours: isHourly ? hoursCalc : 0,
 reason,
 })),
});
const d = await res.json().catch(() => ({}));
if (res.ok) {
 setMsg({ kind: "success", text: "درخواست مرخصی شما با موفقیت ثبت شد" });
 setReason("");
 load();
} else setMsg({ kind: "error", text: d.error ?? "خطای سیستمی رخ داده است" });
});

const decide = async (id: number, status: "approved" | "rejected") => {
 const row = rows.find((r) => r.id === id);
 if (
 !(await confirm({
 title: status === "approved" ? "تایید درخواست" : "رد درخواست",
 message: `درخواست مرخصی «${row?.repName ?? ""}» ${status === "approved" ? "تایید" : "رد"} شد`,
 confirmText: status === "approved" ? "تایید" : "رد",
 danger: status === "rejected",
 })))
)
 return;
const note = window.prompt(status === "approved" ? "توضیح مرخصی (اختیاری):" : "توضیح رد درخواست (اختیاری):");
const res = await fetch("/api/leaves", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ id, status, note }),
});
if (res.ok) load();
else setMsg({ kind: "error", text: "خطای سیستمی رخ داده است" });
});

const canDecide = role === "admin" || role === "supervisor";

const periods = [...new Set(rows.map((r) => r.fromDate.slice(0, 7)))]
 .sort()
 .reverse();
const monthName = (p: string) =>
 `${JALALI_MONTHS[Number(p.slice(5, 7)) - 1]} ?? ""`
 `${toPersianDigits(p.slice(0, 4))}`;

const scoped = rows.filter((r) => !period || r.fromDate.startsWith(period));

const summary = (() => {
 const m = new Map<string, { period: string; rep: string; count: number; days: number; hours: number }>();
 for (const r of scoped) {
 const per = r.fromDate.slice(0, 7);
 const k = `${per}|${r.repName}`;
 const cur = m.get(k) ?? { period: per, rep: r.repName, count: 0, days: 0, hours: 0 };
 cur.count++;
 cur.days += Number(r.days || 0);
 cur.hours += Number(r.hours || 0);
 m.set(k, cur);
 }
 return [...m.values()]
 .sort((a, b) =>
 a.period === b.period ? a.rep.localeCompare(b.rep) : b.period.localeCompare(a.period),
);
})();

const totals = scoped.reduce(
 (a, r) => {
 const ok = r.managerStatus === "approved";
 return {
 days: a.days + (ok ? Number(r.days || 0) : 0),
 hours: a.hours + (ok ? Number(r.hours || 0) : 0),
 pending: a.pending + (r.managerStatus === "pending" ? 1 : 0),
 };
 },
 { days: 0, hours: 0, pending: 0 },
);

const isHourly = kind.includes("ساعتی");
const hoursCalc = (() => {
 const [h1, m1] = fromTime.split(":").map(Number);

```



```

const [h2, m2] = toTime.split(":").map(Number);
const diff = (h2 * 60 + m2 - (h1 * 60 + m1)) / 60;
return diff > 0 ? Math.round(diff * 100) / 100 : 0;
})();

return (
 <div className="space-y-4">
 <SectionTitle icon="">⬅️ ⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️ (⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️: ⬅️⬅️⬅️⬅️⬅️ + ⬅️⬅️⬅️⬅️)</SectionTitle>
 {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}

 <div className="flex flex-wrap items-center gap-2">
 <Button variant={showForm ? "primary" : "ghost"} onClick={() => setShowForm(true)}>
 ⬅️ ⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️
 </Button>
 <Button variant={!showForm ? "primary" : "ghost"} onClick={() => setShowForm(false)}>
 ⬅️ ⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️⬅️⬅️⬅️⬅️
 </Button>
 </div>

 {showForm ? (
 <Card>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
 <Field label="⬅️⬅️ ⬅️⬅️⬅️⬅️" hint="⬅️⬅️⬅️⬅️ + ⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️">
 <Combobox value={kind} onChange={setKind} options={kinds} category="leaveKind" onAdded={loadKinds} />
 </Field>
 <Field label="⬅️⬅️ ⬅️⬅️⬅️⬅️" required>
 <JalaliDateInput value={fromDate} onChange={setFrom} />
 </Field>
 <Field label="⬅️⬅️ ⬅️⬅️⬅️⬅️" required>
 <JalaliDateInput value={toDate} onChange={setTo} />
 </Field>
 {isHourly ? (
 <>
 <Field label="⬅️⬅️⬅️ ⬅️⬅️⬅️" required>
 <input
 type="time"
 value={fromTime}
 onChange={(e) => setFromTime(e.target.value)}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 />
 </Field>
 <Field label="⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️" required>
 <input
 type="time"
 value={toTime}
 onChange={(e) => setToTime(e.target.value)}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 />
 </Field>
 <Field label="⬅️⬅️⬅️ (⬅️⬅️⬅️⬅️)">
 <div className="rounded-xl bg-slate-100 px-3 py-2.5 text-sm font-bold text-teal-700">
 {toPersianDigits(hoursCalc)} ⬅️⬅️⬅️
 </div>
 </Field>
 </>
) : (
 <Field label="⬅️⬅️⬅️⬅️ ⬅️⬅️">
 <Input inputMode="numeric" value={days} onChange={(e) => setDays(e.target.value.replace(/\D/g, ""))} />
 </Field>
)
 </div>
 <div className="sm:col-span-2 lg:col-span-4">
 <Field label="⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️">
 <TextArea value={reason} onChange={(e) => setReason(e.target.value)} />
 </Field>
 </div>
 <div>
 <div className="mt-3 flex justify-end">
 <Button onClick={submit}>⬅️ ⬅️⬅️⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️⬅️</Button>
 </div>
 </div>
 </Card>
) : null}

 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">⬅️ ⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️⬅️⬅️ ⬅️⬅️ ⬅️⬅️⬅️</h3>
 <div className="mb-3 flex flex-wrap gap-2">
 <select
 value={period}
 onChange={(e) => setPeriod(e.target.value)}
 className="rounded-xl border border-slate-300 px-3 py-2 text-xs"
 >
 <option value="">⬅️⬅️ ⬅️⬅️⬅️⬅️⬅️⬅️</option>
 {periods.map((p) => (
 <option key={p} value={p}>
 {monthName(p)}
 </option>
))}
 </select>
 </div>
 </Card>
 </div>
);

```

```

)))
 </select>
 {["approvedDays", "approvedHours", "pending"].map((k) => {
 const map: Record<string, [string, string | number, string]> = {
approvedDays: ["۰۰۰۰ ۰۰۰۰۰۰۰۰", toPersianDigits(totals.days), "bg-emerald-50 text-emerald-800"],
approvedHours: ["۰۰۰۰ ۰۰۰۰۰۰۰۰", toPersianDigits(totals.hours), "bg-sky-50 text-sky-800"],
pending: ["۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰", toPersianDigits(totals.pending), "bg-amber-50 text-amber-800"],
 };
 const [l, v, c] = map[k];
 return (
 <div key={k} className={`rounded-xl px-4 py-2 text-center ring-1 ring-slate-200 ${c}`}>
 <div className="text-lg font-black">{v}</div>
 <div className="text-[10px] font-bold">{l}</div>
 </div>
);
 })}
</div>
<div className="scroll-x">
 <table className="w-full min-w-[520px] text-right text-xs">
 <thead>
 <tr className="bg-slate-100 text-slate-600">
 <th className="px-2 py-2">۰۰۰ ۰ ۰۰۰</th>
 <th className="px-2 py-2">۰۰۰</th>
 <th className="px-2 py-2">۰۰۰۰ ۰۰۰۰۰۰۰</th>
 <th className="px-2 py-2">۰۰۰ ۰۰۰</th>
 <th className="px-2 py-2">۰۰۰ ۰۰۰۰</th>
 </tr>
 </thead>
 <tbody>
 {summary.map((s) => (
 <tr key={` ${s.period}-${s.rep}`} className="border-b border-slate-100">
 <td className="px-2 py-2 font-bold text-teal-700">{monthName(s.period)}</td>
 <td className="px-2 py-2">{s.rep}</td>
 <td className="px-2 py-2">{toPersianDigits(s.count)}</td>
 <td className="px-2 py-2">{toPersianDigits(s.days)}</td>
 <td className="px-2 py-2">{toPersianDigits(Math.round(s.hours * 100) / 100)}</td>
 </tr>
)))
 {summary.length === 0 ? (
 <tr>
 <td colSpan={5} className="py-4 text-center text-slate-400">
۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰
 </td>
 </tr>
) : null}
 </tbody>
 </table>
</div>
</Card>

<Card>
 <div className="mb-2 flex items-center justify-between">
<h3 className="text-sm font-bold text-slate-700">۰۰۰۰ ۰۰۰۰۰۰۰۰۰۰</h3>
 <button
 onClick={async () => setMsg({ kind: "success", text: await downloadFile("/api/export?type=leaves", "leaves.xls") })}
 className="rounded-lg bg-emerald-600 px-3 py-1.5 text-xs font-bold text-white"
 >
۰۰ ۰۰۰۰۰ ۰۰۰۰
 </button>
 </div>
 <div className="scroll-x">
 <table className="w-full min-w-[820px] text-right text-xs sm:text-sm">
 <thead>
 <tr className="bg-slate-100 text-slate-600">
 <th className="px-2 py-2">۰۰۰۰</th>
 <th className="px-2 py-2">۰۰۰ ۰۰۰۰۰۰۰</th>
 <th className="px-2 py-2">۰۰۰</th>
 <th className="px-2 py-2">۰۰</th>
 <th className="px-2 py-2">۰۰</th>
 <th className="px-2 py-2">۰۰۰ / ۰۰۰۰</th>
 <th className="px-2 py-2">۰۰۰</th>
 <th className="px-2 py-2">۰۰۰۰ ۰۰۰۰۰۰</th>
 <th className="px-2 py-2">۰۰۰۰ ۰۰۰۰</th>
 {canDecide ? <th className="px-2 py-2">۰۰۰۰</th> : null}
 </tr>
 </thead>
 <tbody>
 {scoped.map((r, i) => (
 <tr key={r.id} className="border-b border-slate-100">
 <td className="px-2 py-2">{toPersianDigits(i + 1)}</td>
 <td className="px-2 py-2 font-bold text-slate-700">{r.repName}</td>
 <td className="px-2 py-2">{r.kind}</td>
 <td className="px-2 py-2">{toPersianDigits(r.fromDate)}</td>
 <td className="px-2 py-2">{toPersianDigits(r.toDate)}</td>
 </tr>
))}
 </tbody>
 </table>
 </div>

```

```

 <td className="px-2 py-2">
 {r.hours > 0 ? (

 {toPersianDigits(r.hours)} □□□□
 <div className="text-[10px] text-slate-400">
 {toPersianDigits(r.fromTime)} □□ {toPersianDigits(r.toTime)}
 </div>

) : (
 `${toPersianDigits(r.days)} □□□`
)
 </td>
 <td className="max-w-[180px] truncate px-2 py-2 text-slate-500">{r.reason}</td>
 <td className="px-2 py-2">
 <StatusBadge s={r.supervisorStatus} />
 {r.supervisorName ? (
 <div className="text-[10px] text-slate-400">{r.supervisorName}</div>
) : null}
 </td>
 <td className="px-2 py-2">
 <StatusBadge s={r.managerStatus} />
 {r.managerName ? <div className="text-[10px] text-slate-400">{r.managerName}</div> : null}
 </td>
 {canDecide ? (
 <td className="px-2 py-2">
 <div className="flex gap-1">
 <button
 onClick={() => decide(r.id, "approved")}
 className="rounded-lg bg-emerald-100 px-2 py-1 text-[11px] font-bold text-emerald-700"
 >
 □□□□□
 </button>
 <button
 onClick={() => decide(r.id, "rejected")}
 className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
 >
 □□
 </button>
 </div>
 </td>
) : null}
 </tr>
 </tbody>
</table>
</div>
</Card>
</div>
);
}

```

```

@@ src/components/screens/MapExplorer.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useMemo, useState } from "react";
import nextDynamic from "next/dynamic";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import Combobox from "@components/Combobox";
import NavButton from "@components/NavButton";
import { fetchRows } from "@lib/useLive";
import { IRAN_CENTER, PROVINCE_GEO, TEHRAN_REGIONS, resolveArea } from "@lib/geo";
import { toPersianDigits } from "@lib/jalali";
import type { MapArea, MapPoint } from "@components/MapBox";

const MapBox = nextDynamic(() => import("@components/MapBox"), {
 ssr: false,
 loading: () => (
 <div className="flex h-[520px] items-center justify-center rounded-2xl bg-slate-100 text-sm text-slate-400">
 □□ □□ □□□□□□ □□□□...
 </div>
),
});

type Place = {
 id: number;
 kind: "pharmacy" | "doctor" | "home";
 name: string;
 repName: string;

```

```

 province: string;
 city: string;
 region: string;
 phone: string;
 address: string;
 lat: number;
 lng: number;
 extra: string;
 };

const KIND_META: Record<Place["kind"], { label: string; icon: string; color: string }> = {
 pharmacy: { label: "□□□□□□□□", icon: "□", color: "#0f766e" },
 doctor: { label: "□□□□", icon: "□", color: "#0369a1" },
 home: { label: "□□□□ □□□□□□", icon: "□", color: "#7c3aed" },
};

/**
 * □□□□ □□□□: □□□□□ □□□□□□□□□□□□ □□□□□□ □ □□□□□ □□□ □□□□ □□□□□
 * □□ □□□□□ □□□□□ / □□□□ / □□□□□□ / □□□□□□□□ □ □□□□□□□ □□□□
 */
export default function MapExplorer({ isAdmin = false }: { isAdmin?: boolean }) {
 const [places, setPlaces] = useState<Place[]>([]);
 const [loading, setLoading] = useState(true);

 const [kinds, setKinds] = useState<Record<Place["kind"], boolean>>({
 pharmacy: true,
 doctor: true,
 home: true,
 }));
 const [province, setProvince] = useState("");
 const [city, setCity] = useState("");
 const [region, setRegion] = useState("");
 const [rep, setRep] = useState("");
 const [q, setQ] = useState("");
 const [selected, setSelected] = useState<Place | null>(null);

 /* ----- □□□□□□□□ □□□□□□□□ ----- */
 const load = useCallback(async () => {
 setLoading(true);
 const [ph, dr, hm] = await Promise.all([
 fetchRows<Record<string, unknown>>("/api/records/pharmacies?limit=1000"),
 fetchRows<Record<string, unknown>>("/api/records/doctors?limit=1000"),
 fetchRows<Record<string, unknown>>("/api/homes"),
]);

 const out: Place[] = [];
 const push = (kind: Place["kind"], r: Record<string, unknown>, name: string, extra = "") => {
 const lat = Number(r.lat);
 const lng = Number(r.lng);
 if (!Number.isFinite(lat) || !Number.isFinite(lng) || (!lat && !lng)) return;
 out.push({
 id: Number(r.id),
 kind,
 name,
 repName: String(r.repName ?? ""),
 province: String(r.province ?? ""),
 city: String(r.city ?? ""),
 region: String(r.region ?? ""),
 phone: String(r.managerPhone ?? r.phone ?? ""),
 address: String(r.address ?? ""),
 lat,
 lng,
 extra,
 });
 };

 for (const r of ph) push("pharmacy", r, String(r.name ?? ""), String(r.managerName ?? ""));
 for (const r of dr) push("doctor", r, String(r.name ?? ""), String(r.specialty ?? ""));
 for (const r of hm) push("home", r, String(r.title ?? "□□□□"), String(r.repName ?? ""));

 setPlaces(out);
 setLoading(false);
 }, []);

 useEffect(() => {
 load();
 }, [load]);

 /* ----- □□□□□ □□□□□□□□ ----- */
 const provinces = useMemo(
 () => [...new Set([...Object.keys(PROVINCE_GEO), ...places.map((p) => p.province)])].filter(Boolean).sort(),
 [places],
);
 const cities = useMemo(
 () =>

```

```

[...new Set(places.filter((p) => !province || p.province === province).map((p) => p.city))]
 .filter(Boolean)
 .sort(),
 [places, province],
);
const regions = useMemo(() => {
 const fromData = places.filter((p) => (!city || p.city === city)).map((p) => p.region);
const base = city === "□□□□□" ? Object.values(TEHRAN_REGIONS).map((r) => r.name): [];
 return [...new Set([...base, ...fromData])].filter(Boolean).sort();
}, [places, city]);
const reps = useMemo(() => [...new Set(places.map((p) => p.repName))].filter(Boolean).sort(), [places]);

/* ----- □□□□□ □□□□□□□□ ----- */
const filtered = useMemo(() => {
 const term = q.trim().toLowerCase();
 return places.filter((p) => {
 if (!kinds[p.kind]) return false;
 if (province && p.province !== province) return false;
 if (city && p.city !== city) return false;
 if (region && p.region !== region) return false;
 if (rep && p.repName !== rep) return false;
 if (term && !`${p.name} ${p.repName} ${p.city} ${p.address} ${p.extra}`.toLowerCase().includes(term))
 return false;
 return true;
 });
}, [places, kinds, province, city, region, rep, q]);

/** □□□□□□□□□□ □□ □□□□ □□□ □□ □□□□□□ □□□□□□□□ */
const area: MapArea = useMemo(() => {
 const g = resolveArea(province, city, region);
 return { lat: g.lat, lng: g.lng, zoom: g.zoom, radiusKm: g.radiusKm, name: g.name };
}, [province, city, region]);

const mapPoints: MapPoint[] = useMemo(
 () =>
 (selected ? [selected] : filtered).map((p) => ({
 lat: p.lat,
 lng: p.lng,
 label: `${KIND_META[p.kind].icon} ${p.name}${p.repName ? ` — ${p.repName}`: ""}`,
 color: KIND_META[p.kind].color,
 })),
 [filtered, selected],
);

const counts = useMemo(
 () => ({
 pharmacy: filtered.filter((p) => p.kind === "pharmacy").length,
 doctor: filtered.filter((p) => p.kind === "doctor").length,
 home: filtered.filter((p) => p.kind === "home").length,
 }),
 [filtered],
);

const reset = () => {
 setProvince("");
 setCity("");
 setRegion("");
 setRep("");
 setQ("");
 setSelected(null);
};

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="□□">□□□□ □□□□ — □□□□□□□□□□□□ □□□□□□ □ □□□□□</SectionTitle>
 <div className="flex flex-wrap gap-2">
 <Badge tone="green">□ {toPersianDigits(counts.pharmacy)}</Badge>
 <Badge tone="slate">□ {toPersianDigits(counts.doctor)}</Badge>
 <Badge tone="amber">□ {toPersianDigits(counts.home)}</Badge>
 <Button variant="ghost" onClick={load}>
 □ □□□□□□□□□
 </Button>
 </div>
 </div>

 <Card>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-4">
 <Field label="□□□□□">
 <Combobox
 value={province}
 onChange={(v) => {
 setProvince(v);
 setCity("");
 setRegion("");
 }}
 >

```

```

 setSelected(null);
 }}
 options={provinces}
 selectOnly
 placeholder="کشور را انتخاب کنید"
 />
</Field>
<Field label="شهر">
 <Combobox
 value={city}
 onChange={(v) => {
 setCity(v);
 setRegion("");
 setSelected(null);
 }}
 options={cities}
 selectOnly
 placeholder={province ? "شهر را از لیست زیر انتخاب کنید" : "شهر را انتخاب کنید"}
 />
</Field>
<Field label="استان">
 <Combobox
 value={region}
 onChange={(v) => {
 setRegion(v);
 setSelected(null);
 }}
 options={regions}
 selectOnly
 placeholder="استان را انتخاب کنید"
 />
</Field>
<Field label={isAdmin ? "نوع کاربر" : "نوع کاربر"}>
 {isAdmin ? (
 <select
 value={rep}
 onChange={(e) => {
 setRep(e.target.value);
 setSelected(null);
 }}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="">نوع کاربر را انتخاب کنید</option>
 {reps.map((r) => (
 <option key={r} value={r}>
 {r}
 </option>
))}
 </select>
) : (
 <Input value={q} onChange={(e) => setQ(e.target.value)} placeholder="نوع کاربر را بنویسید" />
)}
</Field>
</div>

<div className="mt-3 flex flex-wrap items-center gap-2">
 {(Object.keys(KIND_META) as Place["kind"][]).map((k) => (
 <button
 key={k}
 onClick={() => setKinds({ ...kinds, [k]: !kinds[k] })}
 className={`rounded-xl px-3 py-1.5 text-xs font-bold ring-1 transition ${
 kinds[k] ? "bg-teal-50 text-teal-700 ring-teal-300" : "bg-slate-100 text-slate-400 ring-slate-200"
 }`}
 >
 {KIND_META[k].icon} {KIND_META[k].label} {kinds[k] ? "☑" : ""}
 </button>
))}
 {isAdmin ? (
 <Input value={q} onChange={(e) => setQ(e.target.value)} placeholder="نوع کاربر را بنویسید" className="max-w-[200px]"
) : null}
 <div className="flex-1" />
</div>
<Badge tone="slate">وضعیت: {area.name ?? IRAN_CENTER.name}</Badge>
<Button variant="ghost" onClick={reset}>
 ریست
</Button>
</div>
</Card>

{loading ? <Alert kind="info">در حال بارگذاری...</Alert> : null}
{!loading && filtered.length === 0 ? (
 <Alert kind="info">
 هیچ نتیجه‌ای یافت نشد. لطفاً دوباره جستجو کنید.
 </Alert>
) : null}

```

```

<div className="grid grid-cols-1 gap-3 lg:grid-cols-3">
 <Card className="lg:col-span-2">
 <MapBox
 height={520}
 points={mapPoints}
 area={area}
 follow={!selected}
 showIranProvinces
 selectedProvince={province}
 onProvinceSelect={(name) => {
 setProvince(name);
 setCity("");
 setRegion("");
 setSelected(null);
 }}
 />
 {selected ? (
 <div className="mt-3 rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
 <div className="flex flex-wrap items-center gap-2">

 {KIND_META[selected.kind].icon} {selected.name}

 {selected.extra ? <Badge tone="slate">{selected.extra}</Badge> : null}
 {selected.repName ? <Badge tone="green">{selected.repName}</Badge> : null}
 <div className="mr-auto flex items-center gap-2">
 <NavButton lat={selected.lat} lng={selected.lng} label={selected.name} />
 <button onClick={() => setSelected(null)} className="text-[11px] font-bold text-slate-500">
 □□□□
 </button>
 </div>
 </div>
 {selected.address ? <p className="mt-1 text-[11px] text-slate-600">{selected.address}</p> : null}
 <p className="text-[10px] text-slate-400" dir="ltr">
 {selected.lat.toFixed(5)}, {selected.lng.toFixed(5)}
 </p>
) : null}
 </Card>

 <Card className="lg:col-span-1">
 <h3 className="mb-2 text-sm font-bold text-slate-700">
 □□□□ □□□□ ({toPersianDigits(filtered.length)})
 </h3>
 <div className="max-h-[520px] space-y-1 overflow-y-auto">
 {filtered.slice(0, 300).map((p) => (
 <button
 key={` ${p.kind}-${p.id} `}
 onClick={() => setSelected(selected?.id === p.id && selected.kind === p.kind ? null : p)}
 className={`w-full rounded-xl px-3 py-2 text-right ring-1 transition ${
 selected?.id === p.id && selected.kind === p.kind
 ? "bg-teal-50 ring-teal-300"
 : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
 }`}
 >
 <div className="flex flex-wrap items-center gap-1.5">

 {KIND_META[p.kind].icon} {p.name}

 {p.extra ? {p.extra} : null}
 </div>
 <div className="mt-0.5 flex flex-wrap gap-2 text-[10px] text-slate-500">
 {p.city ? {p.city} : null}
 {p.region ? {p.region} : null}
 {p.repName ? □ {p.repName} : null}
 {p.phone ? {toPersianDigits(p.phone)} : null}
 </div>
 </button>
))}
 {filtered.length > 300 ? (
 <p className="py-2 text-center text-[11px] text-slate-400">
 {toPersianDigits(filtered.length - 300)} □□□□ □□□□ — □□□□ □□ □□□□□□ □□□□
 </p>
) : null}
 </div>
 </Card>
</div>
</div>
);
}

```

```

@@ src/components/screens/NotificationScreen.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useState } from "react";

```

```

import { Alert, Badge, Button, Card, Field, Input, SectionTitle, TextArea } from "@components/ui";
import { useSession } from "@components/SessionProvider";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";
import { useConfirm } from "@components/Confirm";

type N = {
 id: number;
 title: string;
 body: string;
 fromName: string;
 kind: string;
 readAt: string | null;
 createdAt: string;
};

type U = { id: number; fullName: string; role: string };

export default function NotificationScreen() {
 const { me } = useSession();
 const [rows, setRows] = useState<N[]>([]);
 const [users, setUsers] = useState<U[]>([]);
 const [title, setTitle] = useState("");
 const [body, setBody] = useState("");
 const [to, setTo] = useState("0");
 const [msg, setMsg] = useState("");
 const confirm = useConfirm();

 const load = useCallback(async () => {
 const res = await fetch("/api/notifications", { cache: "no-store" });
 if (res.ok) setRows((await res.json()).rows ?? []);
 }, []);

 useLive(load, 15000);

 useEffect(() => {
 if (me && me.role !== "rep") {
 fetch("/api/users", { cache: "no-store" })
 .then((r) => (r.ok ? r.json() : null))
 .then((d) => setUsers(d?.rows ?? []))
 .catch(() => undefined);
 }
 }, [me]);

 const send = async () => {
 if (!title.trim()) return;
 if (
 !(await confirm({
 title: "ارسال پیام",
 message: isRep ? "آیا می‌خواهید پیامی را برای کاربر ارسال کنید؟" : "آیا می‌خواهید پیامی را برای همه کاربران ارسال کنید؟",
 confirmText: "ارسال",
 })))
 return;
 const res = await fetch("/api/notifications", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ title, body, toUserId: Number(to) || 0 }),
 });
 if (res.ok) {
 setMsg("پیام ارسال شد");
 setTitle("");
 setBody("");
 load();
 } else setMsg("خطا در ارسال پیام");
 };

 const markAll = async () => {
 await fetch("/api/notifications", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ all: true }),
 });
 load();
 };

 const isRep = me?.role === "rep";

 return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <SectionTitle icon="📬">پیام‌ها</SectionTitle>
 <Button variant="ghost" onClick={markAll}>
 علامت‌گذاری همه
 </Button>
 </div>
 </div>
);
}

```



```

</div>
{msg ? <Alert kind="success">{msg}</Alert> : null}

<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">
 {isRep ? "تاییدیه‌ی شما تایید شد" : "تاییدیه‌ی شما تایید نشد"}
 </h3>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-3">
 <Field label="تاییدیه‌ی شما تایید شد">
 <Input value={title} onChange={(e) => setTitle(e.target.value)} />
 </Field>
 <Field label="تاییدیه‌ی شما تایید شد">
 <TextArea value={body} onChange={(e) => setBody(e.target.value)} />
 </Field>
 {!isRep ? (
 <Field label="تاییدیه‌ی شما تایید شد">
 <select
 value={to}
 onChange={(e) => setTo(e.target.value)}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="0">تاییدیه‌ی شما تایید شد</option>
 {users
 .filter((u) => u.role === "rep")
 .map((u) => (
 <option key={u.id} value={u.id}>
 {u.fullName}
 </option>
))}
 </select>
 </Field>
) : null}
 </div>
 <div className="mt-3 flex justify-end">
 <Button onClick={send}>تاییدیه‌ی شما تایید شد</Button>
 </div>
</Card>

<Card>
 <div className="space-y-2">
 {rows.length === 0 ? <p className="text-sm text-slate-400">تاییدیه‌ی شما تایید شد</p> : null}
 {rows.map((n) => (
 <div
 key={n.id}
 className={`rounded-xl px-3 py-2 text-sm ${n.readAt ? "bg-slate-50" : "bg-teal-50 ring-1 ring-teal-200"}`}
 >
 <div className="flex flex-wrap items-center gap-2">
 {n.title}
 {!n.readAt ? <Badge tone="green">تاییدیه‌ی شما تایید شد</Badge> : null}
 {tehranDateTime(n.createdAt)}
 </div>
 {n.body ? <p className="mt-1 whitespace-pre-wrap text-xs text-slate-600">{n.body}</p> : null}
 {n.fromName ? <p className="text-[10px] text-slate-400">تاییدیه‌ی شما تایید شد: {n.fromName}</p> : null}
 </div>
))}
 </div>
 <p className="mt-2 text-[11px] text-slate-400">تاییدیه‌ی شما تایید شد: {toPersianDigits(rows.length)}</p>
</Card>
</div>
);
}

```

```

@@ src/components/screens/OptionsScreen.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useState } from "react";
import { Alert, Button, Card, Input, SectionTitle } from "@components/ui";
import { OPTION_CATEGORIES } from "@lib/constants";
import { useConfirm } from "@components/Confirm";
import { toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";

```

```

type Opt = { id: number; category: string; value: string; parent: string; createdBy: string };

```

```

/** تاییدیه‌ی شما تایید شد — تاییدیه‌ی شما تایید شد (تاییدیه‌ی شما تایید شد) */
export default function OptionsScreen({ canDelete = false }: { canDelete?: boolean }) {
 const [rows, setRows] = useState<Opt[]>([]);
 const [draft, setDraft] = useState<Record<string, string>>({});
 const [msg, setMsg] = useState<{ kind: "success" | "error"; text: string } | null>(null);
 const [q, setQ] = useState("");
 const [editing, setEditing] = useState<{ id: number; value: string } | null>(null);
 const [parentSel, setParentSel] = useState<Record<string, string>>({});
 const confirm = useConfirm();

```

```

const load = useCallback(async () => {

```

```

const res = await fetch("/api/options", { cache: "no-store" });
if (res.ok) setRows((await res.json()).rows ?? []);
}, []);

useLive(load, 20000);

const add = async (category: string) => {
 const value = (draft[category] ?? "").trim();
 if (!value) return;
 if ((category === "city" || category === "region") && !(parentSel[category] ?? "")) {
 setMsg({ kind: "error", text: category === "city" ? "□□□□□ □□□□□ □□ □□□□□□ □□□□" : "□□□□□ □□□ □□ □□□□□□ □□□□" });
 return;
 }
 const label = OPTION_CATEGORIES.find((c) => c.key === category)?.label ?? category;
 if (!(await confirm({ title: "□□□□□□ □□□□□ □□□□", message: `«${value}» □□ □□□□ «${label}» □□□□□ □□□□`, confirmText:
"□□□□□□" })))
 return;
 const res = await fetch("/api/options", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ category, value, parent: parentSel[category] ?? "" }),
 });
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setDraft({ ...draft, [category]: "" });
 setMsg({
 kind: "success",
 text: d.duplicate ? "□□□□ □□□□□ □□ □□□ □□□□ □□□□" : "□ □□□□□□□ □□ □□□ □□□□□ □□□□ □□□□□□ □□□□",
 });
 load();
 } else setMsg({ kind: "error", text: d.error ?? "□□□ □□ □□□□□□" });
};

const remove = async (id: number, value: string) => {
 if (
 !(await confirm({
 title: "□□□ □□□□□",
 message: `«${value}» □□□ □□□□ □□□ □□□ □□□□□ □□□□□`,
 confirmText: "□□□",
 danger: true,
 })))
 return;
 const res = await fetch(`/api/options?id=${id}`, { method: "DELETE" });
 if (res.ok) {
 setMsg({ kind: "success", text: "□ □□□ □□" });
 load();
 } else setMsg({ kind: "error", text: "□□□ □□□□□□ □□□" });
};

const saveEdit = async () => {
 if (!editing) return;
 const value = editing.value.trim();
 if (!value) return;
 if (!(await confirm({ title: "□□□□□ □□□□□□", message: `□□□□□ □□ «${value}» □□□□□ □□□□`, confirmText: "□□□□□" })))
 return;
 const res = await fetch("/api/options", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ id: editing.id, value }),
 });
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setMsg({ kind: "success", text: "□□ □□□□□□ □□□□□ □□" });
 setEditing(null);
 load();
 } else setMsg({ kind: "error", text: d.error ?? "□□□□□□ □□□□□□" });
};

return (
 <div className="space-y-4">
 <SectionTitle icon="□">□□□□□□ □□□□□□ □□□□□ (□□□□□□□□□□ □□□□□□□□)</SectionTitle>
 {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}
 <Input placeholder="□ □□□□□ □□ □□□□□□..." value={q} onChange={(e) => setQ(e.target.value)} className="max-w-xs" /
 >

 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
 {OPTION_CATEGORIES.map((cat) => {
 const par = parentSel[cat.key] ?? "";
 const items = rows.filter(
 (r) =>
 r.category === cat.key &&
 (!q || r.value.toLowerCase().includes(q.toLowerCase())) &&
 (!par || r.parent === par),
);
 })}
 </div>

```

```

return (
 <Card key={cat.key}>
 <h3 className="mb-2 text-sm font-bold text-slate-800">
 {cat.label} {{toPersianDigits(items.length)}}
 </h3>
 {cat.key === "city" || cat.key === "region" ? (
 <select
 value={parentSel[cat.key] ?? ""}
 onChange={(e) => setParentSel({ ...parentSel, [cat.key]: e.target.value })}
 className="mb-2 w-full rounded-xl border border-slate-300 px-3 py-2 text-xs"
 >
 <option value="">
 {cat.key === "city" ? "— استانها و شهرستانها —": "— شهرستانها و بخشها —"}
 </option>
 {[...new Set(rows.filter((r) => r.category === (cat.key === "city" ? "province" : "city"))).map((r) =>
 r.value))]
 .sort()
 .map((p) => (
 <option key={p} value={p}>
 {p}
 </option>
))
 </select>
) : null}
 <div className="mb-3 flex gap-2">
 <Input
 value={draft[cat.key] ?? ""}
 onChange={(e) => setDraft({ ...draft, [cat.key]: e.target.value })}
 onKeyDown={(e) => {
 if (e.key === "Enter") add(cat.key);
 }}
 placeholder="شماره ثبت شرکت..."
 />
 <Button onClick={() => add(cat.key)}>ثبت شرکت</Button>
 </div>
 <ul className="max-h-60 space-y-1 overflow-y-auto">
 {items.map((it) =>
 editing?.id === it.id ? (
 <li key={it.id} className="flex items-center gap-1 rounded-lg bg-amber-50 px-2 py-1.5">
 <Input
 value={editing.value}
 onChange={(e) => setEditing({ ...editing, value: e.target.value })}
 className="px-2 py-1 text-xs"
 autoFocus
 />
 <button onClick={saveEdit} className="rounded-lg bg-emerald-600 px-2 py-1 text-[11px] font-bold text-white">
 تایید
 </button>
 <button
 onClick={() => setEditing(null)}
 className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
 >
 حذف
 </button>

) : (
 <li key={it.id} className="flex items-center justify-between gap-2 rounded-lg bg-slate-50 px-3 py-1.5 text-sm">

 {it.value}
 {it.parent ? {{it.parent}} : null}
 {it.createdBy ? {{it.createdBy}} : null}

 {canDelete ? (
 <button
 onClick={() => setEditing({ id: it.id, value: it.value })}
 className="rounded-lg bg-sky-100 px-2 py-0.5 text-[11px] font-bold text-sky-700"
 >
 ویرایش
 </button>
 <button
 onClick={() => remove(it.id, it.value)}
 className="rounded-lg bg-rose-100 px-2 py-0.5 text-[11px] font-bold text-rose-700"
 >
 حذف
 </button>
) : null}

),
)

 </Card>
)

```

```

))
 {items.length === 0 ? <li className="text-xs text-slate-400">□□□□ □□□□ : null}

 </Card>
);
}}
</div>
</div>
);
}

```

```

@@ src/components/screens/ReportScreen.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useMemo, useState } from "react";
import { Card, SectionTitle } from "@components/ui";
import TargetPanel from "@components/screens/TargetPanel";
import { JALALI_MONTHS, toPersianDigits } from "@lib/jalali";
import { DEFAULT_PRODUCTS, type ProductConfig } from "@lib/defaults";
import { downloadFile } from "@lib/download";

```

```

type Row = {
 period: string;
 repName: string;
 pharmacies: number;
 doctors: number;
 orders: number;
 units: number;
 bonus: number;
 trips: number;
};

```

```

export default function ReportScreen({ compact = false }: { compact?: boolean }) {
 const [rows, setRows] = useState<Row[]>([]);
 const [byRep, setByRep] = useState<
 { period: string; repName: string; items: Record<string, number>; bonuses: Record<string, number> }[]
 >([]);
 const [rep, setRep] = useState("");
 const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
 const [year, setYear] = useState("");
 const [month, setMonth] = useState("");

```

```

 const load = useCallback(async () => {
 const res = await fetch("/api/reports", { cache: "no-store" });
 if (res.ok) {
 const d = await res.json();
 setRows(d.rows ?? []);
 setByRep(d.productByRep ?? []);
 }
 }, []);

```

```

 useEffect(() => {
 load();
 fetch("/api/settings?key=products", { cache: "no-store" })
 .then((r) => (r.ok ? r.json() : null))
 .then((d) => {
 const v = d?.value as ProductConfig[] | null;
 if (Array.isArray(v) && v.length) setProducts(v.filter((p) => p.enabled !== false));
 })
 .catch(() => undefined);
 }, [load]);

```

```

 const years = useMemo(() => [...new Set(rows.map((r) => r.period.slice(0, 4)))]
 .sort()
 .reverse(), [rows]);

```

```

 const filtered = useMemo(
 () =>
 rows.filter(
 (r) => (!year || r.period.startsWith(year)) && (!month || r.period.slice(5, 7) === month.padStart(2, "0")),
),
 [rows, year, month],
);

```

```

 const sum = filtered.reduce(
 (a, r) => ({
 pharmacies: a.pharmacies + r.pharmacies,
 doctors: a.doctors + r.doctors,
 orders: a.orders + r.orders,
 units: a.units + r.units,
 bonus: a.bonus + r.bonus,
 trips: a.trips + r.trips,
 }),
 { pharmacies: 0, doctors: 0, orders: 0, units: 0, bonus: 0, trips: 0 },
);

```

```

 const reps = useMemo(() => [...new Set(byRep.map((r) => r.repName))].sort(), [byRep]);

```

```

const byRepFiltered = useMemo(
 () =>
 byRep.filter(
 (r) =>
 (!year || r.period.startsWith(year)) &&
 (!month || r.period.slice(5, 7) === month.padStart(2, "0")) &&
 (!rep || r.repName === rep),
),
 [byRep, year, month, rep],
);

const monthName = (p: string) => {
 const m = Number(p.slice(5, 7));
 return `${JALALI_MONTHS[m - 1]} ?? ""} ${toPersianDigits(p.slice(0, 4))}`;
};

return (
 <div className="space-y-4">
 {!compact ? <SectionTitle icon="📅">📅📅📅📅 📅📅📅📅 📅📅📅📅 📅📅📅📅 📅📅📅📅</SectionTitle> : null}

 <TargetPanel />

 <Card>
 <div className="mb-3 flex flex-wrap items-center gap-2">
 <select
 value={year}
 onChange={(e) => setYear(e.target.value)}
 className="rounded-xl border border-slate-300 px-3 py-2 text-sm"
 >
 <option value="">📅📅 📅📅📅📅</option>
 {years.map((y) => (
 <option key={y} value={y}>
 {toPersianDigits(y)}
 </option>
))}
 </select>
 <select
 value={month}
 onChange={(e) => setMonth(e.target.value)}
 className="rounded-xl border border-slate-300 px-3 py-2 text-sm"
 >
 <option value="">📅📅 📅📅📅📅</option>
 {JALALI_MONTHS.map((m, i) => (
 <option key={m} value={String(i + 1)}>
 {m}
 </option>
))}
 </select>
 <div className="flex-1" />
 <button
 onClick={() =>
 downloadFile(
 `/api/export?type=orders${year ? `&year=${year}` : ""}${month ? `&month=${month}` : ""}`,
 "orders.xls",
)
 }
 className="rounded-xl bg-emerald-600 px-3 py-2 text-xs font-bold text-white"
 >
 📅 📅📅 📅📅📅📅 📅📅 📅📅
 </button>
 </div>

 <div className="mb-3 grid grid-cols-3 gap-2 sm:grid-cols-6">
 {[
 ["📅📅📅📅📅📅", sum.pharmacies, "📅"],
 ["📅📅📅", sum.doctors, "📅"],
 ["📅📅📅📅", sum.orders, "📅"],
 ["📅📅📅📅 📅", sum.units, "📅"],
 ["📅📅📅📅 📅", sum.bonus, "📅"],
 ["📅📅 📅📅📅", sum.trips, "📅📅"],
]
 .map(([l, v, i]) => (
 <div key={String(l)} className="rounded-xl bg-slate-50 p-3 text-center ring-1 ring-slate-200">
 <div className="text-lg">{i as string}</div>
 <div className="text-lg font-black text-teal-700">{toPersianDigits(v as number)}</div>
 <div className="text-[10px] font-bold text-slate-500">{l as string}</div>
 </div>
))}
 </div>

 <div className="scroll-x">
 <table className="w-full min-w-[680px] text-right text-xs sm:text-sm">
 <thead>
 <tr className="bg-slate-100 text-slate-600">
 <th className="px-2 py-2">📅📅</th>
 </tr>
 </thead>
 </table>
 </div>
 </div>
);

```

```

<th className="px-2 py-2">□□□ □□□□□□□□</th>
<th className="px-2 py-2">□□□□□□□□</th>
<th className="px-2 py-2">□□□□</th>
<th className="px-2 py-2">□□□□□</th>
<th className="px-2 py-2">□□□□□ □□□□□</th>
<th className="px-2 py-2">□□□□□</th>
<th className="px-2 py-2">□□□ □□□□□</th>
</tr>
</thead>
<tbody>
 {filtered.map((r) => (
 <tr key={`${r.period}-${r.repName}`} className="border-b border-slate-100">
 <td className="px-2 py-2 font-bold text-teal-700">{monthName(r.period)}</td>
 <td className="px-2 py-2">{r.repName}</td>
 <td className="px-2 py-2">{toPersianDigits(r.pharmacies)}</td>
 <td className="px-2 py-2">{toPersianDigits(r.doctors)}</td>
 <td className="px-2 py-2">{toPersianDigits(r.orders)}</td>
 <td className="px-2 py-2">{toPersianDigits(r.units)}</td>
 <td className="px-2 py-2">{toPersianDigits(r.bonus)}</td>
 <td className="px-2 py-2">{toPersianDigits(r.trips)}</td>
 </tr>
))}
 {filtered.length === 0 ? (
 <tr>
 <td colSpan={8} className="py-6 text-center text-slate-400">
 □□□□□□□ □□□□ □□□ □□□□ □□□□ □□□□□
 </td>
 </tr>
) : null}
</tbody>
</table>
</div>
</Card>

{!compact ? (
 <Card>
 <div className="mb-2 flex flex-wrap items-center gap-2">
<h3 className="text-sm font-bold text-slate-700">□□□□ □□ □□□ — □□ □□□□□ □□□□ □□□ □ □□□□□□□</h3>
 <select
 value={rep}
 onChange={(e) => setRep(e.target.value)}
 className="rounded-xl border border-slate-300 px-3 py-2 text-xs"
 >
 <option value="">□□□ □□□□□□□□□□</option>
 {reps.map((r) => (
 <option key={r} value={r}>
 {r}
 </option>
))}
 </select>
 </div>

 <div className="scroll-x">
 <table className="w-full min-w-[760px] text-right text-xs">
 <thead>
 <tr className="bg-slate-100 text-slate-600">
<th className="px-2 py-2">□□□ □ □□□</th>
<th className="px-2 py-2">□□□ □□□□□□□□</th>
 {products.map((p) => (
 <th key={p.key} className="px-2 py-2 text-center">
 {p.label}
 <div className="text-[9px] font-normal text-slate-400">□□□□□ / □□□□□</div>
 </th>
))}
<th className="px-2 py-2 text-center">□□□</th>
 </tr>
 </thead>
 <tbody>
 {byRepFiltered.map((r) => {
 const sumUnits = products.reduce((a, p) => a + (r.items[p.key] ?? 0), 0);
 return (
 <tr key={`${r.period}-${r.repName}`} className="border-b border-slate-100">
 <td className="whitespace-nowrap px-2 py-2 font-bold text-teal-700">{monthName(r.period)}</td>
 <td className="whitespace-nowrap px-2 py-2 font-semibold text-slate-700">{r.repName}</td>
 {products.map((p) => (
 <td key={p.key} className="px-2 py-2 text-center">
 {toPersianDigits(r.items[p.key] ?? 0)}
 /
 {toPersianDigits(r.bonuses[p.key] ?? 0)}
 </td>
))}
 <td className="px-2 py-2 text-center font-black text-teal-700">{toPersianDigits(sumUnits)}</td>
 </tr>
);
 })}
 </tbody>
 </table>
 </div>
)
}

```

```

 {byRepFiltered.length === 0 ? (
 <tr>
 <td colSpan={products.length + 3} className="py-6 text-center text-slate-400">
 □□□□□□ □□ □□□ □□□ □□□ □□□ □□□
 </td>
 </tr>
) : (
 <tr className="bg-teal-50 font-black text-teal-800">
 <td className="px-2 py-2" colSpan={2}>
 □□□ □□ □□□□
 </td>
 {products.map((p) => (
 <td key={p.key} className="px-2 py-2 text-center">
 {toPersianDigits(byRepFiltered.reduce((a, r) => a + (r.items[p.key] ?? 0), 0))}
 /

 {toPersianDigits(byRepFiltered.reduce((a, r) => a + (r.bonuses[p.key] ?? 0), 0))}

 </td>
))}
 <td className="px-2 py-2 text-center">
 {toPersianDigits(
 byRepFiltered.reduce(
 (a, r) => a + products.reduce((x, p) => x + (r.items[p.key] ?? 0), 0),
 0,
),
)}
 </td>
 </tr>
)}
 </tbody>
 </table>
 </div>
 <p className="mt-2 text-[11px] text-slate-400">
 □□ □□ □□□□□ □□□ □□□ «□□□□□ □□□□□» □ □□□ □□□ «□□□□□ □□□□□» □□□.
 </p>
</Card>
) : null}
</div>
);
}

```

```

@@@ src/components/screens/TargetPanel.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useState } from "react";
import { Badge, Card, SectionTitle } from "@components/ui";
import { useLive } from "@lib/useLive";
import { JALALI_MONTHS, toPersianDigits, todayJalali } from "@lib/jalali";

```

```

type Progress = {
 productKey: string;
 productLabel: string;
 quantity: number;
 priceDistributor: number;
 pricePharmacy: number;
 sold: number;
 bonus: number;
 remaining: number;
 percent: number;
 soldValueDistributor: number;
 soldValuePharmacy: number;
};

```

```

type Totals = {
 quantity: number;
 sold: number;
 bonus: number;
 remaining: number;
 valueDistributor: number;
 valuePharmacy: number;
 soldValueDistributor: number;
 soldValuePharmacy: number;
};

```

```

const money = (n: number) => toPersianDigits(Math.round(n).toLocaleString("en-US"));
const currentPeriod = () => todayJalali().slice(0, 7);

```

```

/** □□□ □□□□□□ □□□□□ — □□ □□□□ □□□□ □□□□□□□ □ □□□□□□□ □□□□ □□□□ □□□□□□□ */
export default function TargetPanel({ userId, compact = false }: { userId?: number; compact?: boolean }) {
 const [rows, setRows] = useState<Progress[]>([]);
 const [totals, setTotals] = useState<Totals | null>(null);
 const [percent, setPercent] = useState(0);
 const [period, setPeriod] = useState(currentPeriod());
}

```

```

const load = useCallback(async () => {
 const q = new URLSearchParams({ period });
 if (userId) q.set("userId", String(userId));
 const res = await fetch(`/api/targets?${q}`, { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 setRows(d.progress ?? []);
 setTotals(d.totals ?? null);
 setPercent(d.percent ?? 0);
}, [period, userId]);

useLive(load, 30000);

const periods = (() => {
 const [y, m] = currentPeriod().split("/").map(Number);
 const out: string[] = [];
 for (let i = -5; i <= 1; i++) {
 let mm = m + i;
 let yy = y;
 while (mm <= 0) {
 mm += 12;
 yy -= 1;
 }
 while (mm > 12) {
 mm -= 12;
 yy += 1;
 }
 out.push(`${yy}/${String(mm).padStart(2, "0")}`);
 }
 return [...new Set(out)];
})();

const label = (p: string) => `${JALALI_MONTHS[Number(p.slice(5, 7)) - 1] ?? ""} ${toPersianDigits(p.slice(0, 4))}`;
const active = rows.filter((r) => r.quantity > 0);

if (active.length === 0 && !compact) {
 return (
 <Card>
 <SectionTitle icon="📅">📅📅📅📅📅📅📅📅📅📅📅📅</SectionTitle>
 <p className="py-4 text-center text-xs text-slate-400">
 📅📅 📅📅 📅📅📅📅 📅📅📅 📅📅 📅📅 📅📅📅📅📅📅 📅📅 «📅📅📅📅📅📅» 📅📅📅 📅📅.
 </p>
 </Card>
);
}
if (active.length === 0) return null;

return (
 <Card>
 <div className="mb-3 flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="📅">📅📅📅📅📅📅📅📅📅📅📅📅</SectionTitle>
 <div className="flex items-center gap-2">
 <select
 value={period}
 onChange={(e) => setPeriod(e.target.value)}
 className="rounded-xl border border-slate-300 px-2 py-1.5 text-xs"
 >
 {periods.map((p) => (
 <option key={p} value={p}>
 {label(p)}
 </option>
))}
 </select>
 <Badge tone={percent >= 100 ? "green" : percent >= 60 ? "amber" : "slate"}>
 📅📅 📅: {toPersianDigits(percent)}📅
 </Badge>
 </div>
 </div>

 {totals ? (
 <div className="mb-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
 {[
 ["📅📅📅📅", toPersianDigits(totals.quantity), "bg-slate-50 text-slate-700"],
 ["📅📅📅", toPersianDigits(totals.sold), "bg-teal-50 text-teal-700"],
 ["📅📅📅📅📅📅", toPersianDigits(totals.remaining), "bg-amber-50 text-amber-700"],
 ["📅📅📅📅", toPersianDigits(totals.bonus), "bg-emerald-50 text-emerald-700"],
]
 .map(([l, v, c]) => (
 <div key={l} className={`rounded-xl p-2.5 text-center ring-1 ring-slate-200 ${c}`}>
 <div className="text-lg font-black">{v}</div>
 <div className="text-[10px] font-bold opacity-70">{l}</div>
 </div>
))}
 </div>
) : null}
 </div>
 </div>

```



```

<div className="space-y-1.5">
 {active.map((r) => (
 <div key={r.productKey} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
 <div className="flex flex-wrap items-center gap-2 text-[11px]">
 {r.productLabel}

 □□□□ <b className="text-slate-700">{toPersianDigits(r.quantity)}

 □□□□ {toPersianDigits(r.sold)}

 □□□□□□□□ {toPersianDigits(r.remaining)}

 {toPersianDigits(r.percent)}□
 </div>
 <div className="mt-1 h-1.5 overflow-hidden rounded bg-slate-200">
 <div
 className={`h-full transition-all ${r.percent >= 100 ? "bg-emerald-500" : r.percent >= 60 ? "bg-teal-500" : "bg-amber-500"} }`
 style={{ width: `${Math.min(100, r.percent)}%` }}
 />
 </div>
 </div>
)))
</div>

{totals && (totals.soldValueDistributor > 0 || totals.valueDistributor > 0) ? (
 <div className="mt-3 grid grid-cols-1 gap-2 text-[11px] sm:grid-cols-2">
 <div className="rounded-xl bg-slate-50 p-2.5 ring-1 ring-slate-200">
 □□□□ □□□□ (□□□): <b className="text-slate-800">{money(totals.valueDistributor)} □□□□
 <div className="text-teal-700">
 □□□□ {money(totals.soldValueDistributor)} □□□□
 </div>
 </div>
 <div className="rounded-xl bg-slate-50 p-2.5 ring-1 ring-slate-200">
 □□□□ □□□□ (□□□□□□□□): <b className="text-slate-800">{money(totals.valuePharmacy)} □□□□
 <div className="text-teal-700">
 □□□□ {money(totals.soldValuePharmacy)} □□□□
 </div>
 </div>
 </div>
) : null}
</Card>
);
}

```

```

@@ src/app/admin/activity/page.tsx typescript
"use client";

```

```

import { useCallback, useEffect, useMemo, useState } from "react";
import { Badge, Button, Card, Input, SectionTitle } from "@components/ui";
import { BarChart } from "@components/Charts";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useLive } from "@lib/useLive";

```

```

type Log = { id: number; userId: number | null; userName: string; action: string; detail: string; createdAt: string };
type Rep = {
 id: number;
 fullName: string;
 role: string;
 active: boolean;
 lastSeenAt: string | null;
 pharmacies: number;
 doctors: number;
 orders: number;
 trips: number;
 activeTrip: number;
 lastAction: string | null;
 lastActionAt: string | null;
};

```

```

export default function ActivityPage() {
 const [logs, setLogs] = useState<Log[]>([]);
 const [reps, setReps] = useState<Rep[]>([]);
 const [q, setQ] = useState("");
 const [rep, setRep] = useState("");
 const [live, setLive] = useState(true);

 const load = useCallback(async () => {
 const res = await fetch("/api/activity", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 setLogs(d.logs ?? []);
 setReps(d.reps ?? []);
 }, []);

```

```

}, []);

useLive(load, 12000, live);

const filtered = useMemo(
 () =>
 logs.filter(
 (l) =>
 (!rep || l.userName === rep) &&
 (!q || `${l.userName} ${l.action} ${l.detail}`.toLowerCase().includes(q.toLowerCase())),
),
 [logs, rep, q],
);

const perRep = useMemo(() => {
 const m = new Map<string, number>();
 for (const l of logs) m.set(l.userName, (m.get(l.userName) ?? 0) + 1);
 return [...m.entries()].map(([label, value]) => ({ label, value })).sort((a, b) => b.value - a.value);
}, [logs]);

const perAction = useMemo(() => {
 const m = new Map<string, number>();
 for (const l of filtered) m.set(l.action, (m.get(l.action) ?? 0) + 1);
 return [...m.entries()].map(([label, value]) => ({ label, value })).sort((a, b) => b.value - a.value);
}, [filtered]);

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="📄">📄📄📄📄📄 📄📄📄📄📄📄📄📄📄</SectionTitle>
 <Button variant={live ? "success" : "ghost"} onClick={() => setLive((v) => !v)}>
 {live ? "📄 📄📄📄 (📄 📄 📄📄📄)" : "📄 📄📄📄📄"}
 </Button>
 </div>

 <div className="grid grid-cols-1 gap-3 lg:grid-cols-2">
 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">📄📄📄📄 📄📄📄📄📄 📄 📄📄📄📄📄</h3>
 <BarChart data={perRep} />
 </Card>
 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">📄📄 📄📄📄📄📄📄📄</h3>
 <BarChart data={perAction} />
 </Card>
 </div>

 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">📄📄📄📄 📄📄📄📄📄📄 📄 📄📄📄📄📄 📄📄📄📄</h3>
 <div className="grid grid-cols-1 gap-2 sm:grid-cols-2 lg:grid-cols-3">
 {reps.map((r) => (
 <button
 key={r.id}
 onClick={() => setRep(rep === r.fullName ? "" : r.fullName)}
 className={`rounded-xl p-3 text-right ring-1 transition ${
 rep === r.fullName ? "bg-teal-50 ring-teal-300" : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
 }`}
 >
 <div className="flex flex-wrap items-center gap-1">
 {r.fullName}
 {r.activeTrip > 0 ? <Badge tone="green">📄 📄 📄📄 📄📄📄</Badge> : null}
 {r.active ? <Badge tone="amber">📄📄📄📄</Badge> : null}
 </div>
 <div className="mt-1 grid grid-cols-4 gap-1 text-center text-[10px]">
 📄 {toPersianDigits(r.pharmacies)}
 📄 {toPersianDigits(r.doctors)}
 📄 {toPersianDigits(r.orders)}
 📄📄 {toPersianDigits(r.trips)}
 </div>
 <div className="mt-1 truncate text-[10px] text-slate-500">{r.lastAction ?? "📄📄📄 📄📄📄📄"}</div>
 <div className="text-[10px] text-slate-400">
 {r.lastActionAt ? tehranDateTime(r.lastActionAt) : "—"}
 </div>
 </button>
))}
 </div>
 </Card>

 </div>
)
);

/* ----- 📄📄📄📄 📄📄📄📄 📄📄📄📄📄📄 ----- */
{rep ? (
 <Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <h3 className="text-sm font-black text-teal-800">📄 📄📄📄📄📄 📄📄📄📄 «{rep}»</h3>
 <Button variant="ghost" onClick={() => setRep("")}>
 📄📄📄📄 📄 📄📄
 </Button>
 </div>
 </Card>
) : null}

```

```

</div>

<div className="mb-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
 {(() => {
 const r = reps.find((x) => x.fullName === rep);
 return [
 ["□ □□□□□□□", r?.pharmacies ?? 0],
 ["□ □□□□", r?.doctors ?? 0],
 ["□ □□□□□", r?.orders ?? 0],
 ["□□ □□ □□□□", r?.trips ?? 0],
].map(([l, v]) => (
 <div key={String(l)} className="rounded-xl bg-slate-50 p-3 text-center ring-1 ring-slate-200">
 <div className="text-lg font-black text-teal-700">{toPersianDigits(v as number)}</div>
 <div className="text-[10px] font-bold text-slate-500">{l as string}</div>
 </div>
));
 })()}
</div>

<div className="mb-3">
<h4 className="mb-1 text-xs font-bold text-slate-600">□□□□□ □□□ □□□□□</h4>
<div className="flex flex-wrap gap-1">
 {perAction.map((a) => (

 {a.label}: {toPersianDigits(a.value)}

))}
{perAction.length === 0 ? □□□□□□□ □□□ □□□□ : null}
</div>
</div>

<div className="max-h-72 space-y-1 overflow-y-auto">
 {filtered.map((l) => (
 <div key={l.id} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2 text-xs">
 {l.action}
 {l.detail ? {l.detail} : null}
 {tehranDateTime(l.createdAt)}
 </div>
))}
{filtered.length === 0 ? <p className="py-4 text-center text-slate-400">□□□□□□□ □□□□ □□□</p> : null}
</div>
</Card>
) : null}

<Card>
 <div className="mb-2 flex flex-wrap items-center gap-2">
<h3 className="text-sm font-bold text-slate-700">□□□□□ □□□□□ □□□ □□□□□□□□</h3>
<Input placeholder="□ □□□□□..." value={q} onChange={(e) => setQ(e.target.value)} className="max-w-[220px]" />
 {rep ? (
 <Button variant="ghost" onClick={() => setRep("")}>
 □□□ □□□□□ «{rep}»
 </Button>
) : null}
<Badge tone="slate">{toPersianDigits(filtered.length)} □□□□□□</Badge>
</div>
<div className="max-h-[480px] space-y-1 overflow-y-auto">
 {filtered.map((l) => (
 <div key={l.id} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2 text-xs">
 {l.userName}
 {l.action}
 {l.detail ? {l.detail} : null}
 {tehranDateTime(l.createdAt)}
 </div>
))}
{filtered.length === 0 ? <p className="py-6 text-center text-slate-400">□□□□□□□ □□□□ □□□</p> : null}
</div>
</Card>
</div>
);
}

@@ src/app/admin/backup/page.tsx typescript
"use client";

import { useCallback, useEffect, useRef, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@/components/ui";
import { tehranDateTime, toPersianDigits } from "@/lib/jalali";
import { useConfirm } from "@/components/Confirm";

type Cfg = { enabled: boolean; minutes: number; withFiles: boolean; folderName: string };

const DEFAULT_CFG: Cfg = { enabled: false, minutes: 30, withFiles: false, folderName: "namayandeelmi-javad" };
const IDB = "sek-backup";

```

```

// 000000 000000 000000000000 0000 000000 000
function idb(): Promise<IDBDatabase> {
 return new Promise((res, rej) => {
 const r = indexedDB.open(IDB, 1);
 r.onupgradeneeded = () => r.result.createObjectStore("h");
 r.onsuccess = () => res(r.result);
 r.onerror = () => rej(r.error);
 });
}

async function saveHandle(h: unknown) {
 const db = await idb();
 const tx = db.transaction("h", "readwrite");
 tx.objectStore("h").put(h, "dir");
}

async function loadHandle(): Promise<FileSystemDirectoryHandle | null> {
 try {
 const db = await idb();
 return await new Promise((res) => {
 const tx = db.transaction("h", "readonly");
 const rq = tx.objectStore("h").get("dir");
 rq.onsuccess = () => res((rq.result as FileSystemDirectoryHandle) ?? null);
 rq.onerror = () => res(null);
 });
 } catch {
 return null;
 }
}

export default function BackupPage() {
 const [cfg, setCfg] = useState<Cfg>(DEFAULT_CFG);
 const [msg, setMsg] = useState("");
 const [last, setLast] = useState("");
 const [dirOk, setDirOk] = useState(false);
 const [busy, setBusy] = useState(false);
 const [log, setLog] = useState<string[]>([]);
 const timer = useRef<ReturnType<typeof setInterval> | null>(null);
 const [dirName, setDirName] = useState("");
 const confirm = useConfirm();
 const supportsFs = typeof window !== "undefined" && "showDirectoryPicker" in window;

 const load = useCallback(async () => {
 const res = await fetch("/api/settings?key=backup", { cache: "no-store" });
 if (res.ok) {
 const d = await res.json();
 if (d.value) setCfg({ ...DEFAULT_CFG, ...(d.value as Cfg) });
 }
 setLast(localStorage.getItem("sek_backup_last") ?? "");
 const h = await loadHandle();
 setDirOk(!!h);
 setDirName(h?.name ?? localStorage.getItem("sek_backup_dir") ?? "");
 }, []);

 useEffect(() => {
 load();
 }, [load]);

 const saveCfg = async (next: Cfg) => {
 setCfg(next);
 await fetch("/api/settings", {
 method: "PUT",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ key: "backup", value: next }),
 });
 setMsg("0 0000000 0000000000000 000000 00");
 };

 const pickFolder = async () => {
 try {
 const w = window as unknown as {
 showDirectoryPicker: (o?: { mode?: string }) => Promise<FileSystemDirectoryHandle>;
 };
 const handle = await w.showDirectoryPicker({ mode: "readwrite" });
 await saveHandle(handle);
 setDirOk(true);
 setDirName(handle.name ?? "");
 localStorage.setItem("sek_backup_dir", handle.name ?? "");
 setMsg("0 0000 «${handle.name}» 000000 00. 000 000000000000 000000 00 0000 0000 000000 00000000 (0000 0000 00 Downloads)");
 } catch {
 setMsg("0000000 0000 000 00");
 }
 };

 const runBackup = useCallback(

```

[illegible]

```

 }, [cfg.enabled, cfg.minutes, runBackup]);

 const winScript = `@echo off
rem =====
rem 000000000000 000000 «000 00000000 00» 000 0000000
rem 0000 00 00 000 backup.bat 00 E:\porozheha\backup 00000 0000
rem 000 00 Task Scheduler 000000 00 000 000000 000000 0000 0000
rem =====
set URL=${typeof window !== "undefined" ? window.location.origin : "https://your-app.onrender.com"}/api/backup
set DEST=E:\porozheha\backup\${cfg.folderName}
set COOKIE=%-dp0cookie.txt
if not exist "%DEST%" mkdir "%DEST%"
for /f "tokens=1-4 delims=/ " %a in ('date /t') do set D=%d-%b-%c
set T=%time:~0,2%%time:~3,2%
curl -s -c "%COOKIE%" -X POST "%URL:/api/backup=/api/auth/login%" -H "Content-Type: application/json" -d "{\"username\\\": \"admin\\\", \"password\\\": \"PASSWORD\\\", \"mode\\\": \"admin\\\", \"remember\\\": true}" > nul
curl -s -b "%COOKIE%" "%URL%" -o "%DEST%\backup-%D%-%T%.json"
echo backup saved to %DEST%
`;

 return (
 <div className="space-y-4">
 <SectionTitle icon="📁">000000000000 00 00000000</SectionTitle>
 {msg ? <Alert kind={msg.startsWith("❌") ? "error" : "success"}>{msg}</Alert> : null}

 <Card>
 <div className="flex flex-wrap items-center gap-2">
 <Button
 onClick={async () => {
 if (
 !(await confirm({
 title: "000000000000",
 message: dirOk
 ? `0000 00000000 00 0000 «${dirName}» 000000 0000`
 : "000000 000000 0000 000 000 00 000 000000 0000 00 000000 000000",
 confirmText: "0000",
 }))
)
 return;
 runBackup(false);
 }}
 disabled={busy}
 >
 {busy ? "🔒 00 000 0000..." : "📁 00000000000000 00 0000"}
 </Button>
 <Button variant="ghost" onClick={() => runBackup(false, true)} disabled={busy}>
 00 000000 0000
 </Button>
 {supportsFs ? (
 <Button variant={dirOk ? "success" : "soft"} onClick={pickFolder}>
 {dirOk ? "📁 0000: ${dirName} || "000000000000" — 00000: "📁 000000 0000 0000"}
 </Button>
) : null}
 <Badge tone={last ? "green" : "amber"}>
 00000 0000000: {last ? tehranDateTime(last) : "0000"}
 </Badge>
 </div>

 {!dirOk ? (
 <Alert kind="error">
 00 0000 0000 0000 0000000 0000 000. 00 00000 00 0000 00 000000 000000 00000 0000000000000 00000 000000 0000
 000 000000. 0000 00000 0000000 000000 0000 «📁 000000 0000 0000» 00 00000.
 </Alert>
) : null}
 <Alert kind="info">
 00000000 0000 E:\porozheha\backup\{cfg.folderName} — 000 000000 00 000000 0000 00000000 0000 000000
 000000 000 00000 0000000 00 000 0000 0000 00 000000 0000: 0000 «📁 000000 0000 0000» 00 00000 0 0000
 E:\porozheha\backup\{cfg.folderName} 00 0000000 0000. 00 00 00 000 000000000000
 (0000 0 000000) 00000000 0000 0000 0000 000000 00000000. 000 000000 000 000 000000 00 000000 0000 00 0000
 000000 000000 0000000.
 </Alert>
 </Card>

 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">📁 00000000000000 000000</h3>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-3">
 <Field label="00 000 00000 0000000">
 <Input
 inputMode="numeric"
 value={String(cfg.minutes)}
 onChange={(e) => setCfg({ ...cfg, minutes: Number(e.target.value.replace(/\D/g, "")) || 1 }})
 />
 </Field>
 <Field label="0000 00000 0000">
 <Input value={cfg.folderName} onChange={(e) => setCfg({ ...cfg, folderName: e.target.value })} />
 </Field>
 </div>
 </Card>
 </div>
);

```

```

</Field>
<div className="flex flex-col justify-end gap-2">
 <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={cfg.withFiles}
 onChange={(e) => setCfg({ ...cfg, withFiles: e.target.checked })}
 />
 00000 00000000 0 0000000 (00000000)
 </label>
 <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={cfg.enabled}
 onChange={(e) => setCfg({ ...cfg, enabled: e.target.checked })}
 />
 00000000000000 0000000 0000 0000
 </label>
</div>
</div>
<div className="mt-3 flex flex-wrap gap-2">
 <Button
 onClick={async () => {
 if (
 !(await confirm({
 title: "000000 00000000 00000000000000",
 message: cfg.enabled
 ? `00000000000000 0000000 00 ${cfg.minutes} 000000 0000 0000`
 : `00000000000000 0000000 00000000 0000`,
 confirmText: "000000",
 }))
)
 return;
 saveCfg(cfg);
 }}
 >
 0 000000 00000000
 </Button>
 <Badge tone={cfg.enabled ? "green" : "slate"}>
 {cfg.enabled ? `00000 — 00 ${toPersianDigits(cfg.minutes)} 000000` : "00000000"}
 </Badge>
</div>
<p className="mt-2 text-[11px] text-slate-500">
 00000: 00000000000000 0000000 00000 00000 00000000 00 000 00000 00 000000 000 00000. 00000 000000 00000000 00000000 00
 00000000 00000000 000 00000000 00000.
</p>
</Card>

<Card>
<h3 className="mb-2 text-sm font-bold text-slate-700">0 00000000 0000000 0000000 (0000 00000 00 000 00000 0000000)</h3>
>
 <p className="mb-2 text-[11px] text-slate-500">
 00 000 00 0000 0 00 0000 backup.bat 000000 000000 0000 00000 00 00000 00 000000000 0 00
 Task Scheduler 0000000 000000000000 00000 (000000 00 00 000000).
 </p>
 <pre className="scroll-x max-h-56 overflow-auto rounded-xl bg-slate-900 p-3 text-[11px] leading-5 text-emerald-200" dir="ltr">
 {winScript}
 </pre>
 <div className="mt-2 flex gap-2">
 <Button
 variant="soft"
 onClick={() => {
 navigator.clipboard?.writeText(winScript);
 setMsg("0 00000000 0000 00");
 }}
 >
 0 000 00000000
 </Button>
 <Button
 variant="ghost"
 onClick={() => {
 const blob = new Blob([winScript], { type: "text/plain" });
 const a = document.createElement("a");
 a.href = URL.createObjectURL(blob);
 a.download = "backup.bat";
 a.click();
 }}
 >
 00 0000000 backup.bat
 </Button>
 </div>
</Card>

```

```

 {log.length ? (
 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□□ □□□□□□□□□□□ □□ □□□□</h3>
 <ul className="space-y-1 text-[11px] text-slate-600">
 {log.map((l, i) => (
 <li key={i} className="rounded-lg bg-slate-50 px-2 py-1">
 {l}

))}

 </Card>
) : null}
 </div>
);
}

@@ src/app/admin/columns/page.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Input, SectionTitle } from "@components/ui";
import {
 AVAILABLE_COLUMNS,
 AVAILABLE_FORM_FIELDS,
 DEFAULT_COLUMNS,
 DEFAULT_FORM_FIELDS,
 DEFAULT_PRODUCTS,
 type ColumnConfig,
 type FormFieldConfig,
 type ProductConfig,
} from "@lib/defaults";
import { useConfirm } from "@components/Confirm";
import { toPersianDigits } from "@lib/jalali";

const TABLES = [
 { key: "pharmacies", label: "□□□□ □□□□□□□□□□" },
 { key: "doctors", label: "□□□□ □□□□□□" },
 { key: "orders", label: "□□□□ □□□□□□" },
];

export default function ColumnsPage() {
 const [cols, setCols] = useState<Record<string, ColumnConfig[]>>>(DEFAULT_COLUMNS);
 const [formFields, setFormFields] = useState<Record<string, FormFieldConfig[]>>>(DEFAULT_FORM_FIELDS);
 const [products, setProducts] = useState<ProductConfig[]>>(DEFAULT_PRODUCTS);
 const [tab, setTab] = useState("pharmacies");
 const [msg, setMsg] = useState("");
 const [newProduct, setNewProduct] = useState("");
 const [addCol, setAddCol] = useState("");
 const [addField, setAddField] = useState("");
 const confirm = useConfirm();

 const load = useCallback(async () => {
 const res = await fetch("/api/settings", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 const v = d.values ?? {};
 setProducts((v.products as ProductConfig[]) ?? DEFAULT_PRODUCTS);

 /** □□□□□□□□ □□□□ □□□□□□ □□ □□□□□□ □□□□□□□□ □□□□□ □□□□□□□□ */
 const merge = (key: string, def: ColumnConfig[]) => {
 const stored = v[key] as ColumnConfig[] | undefined;
 if (!Array.isArray(stored) || stored.length === 0) return def;
 const have = new Set(stored.map((c) => c.key));
 const mandatory = def.filter((c) => c.key === "actions" && !have.has(c.key));
 return [...stored, ...mandatory];
 };
 setCols({
 pharmacies: merge("columns.pharmacies", DEFAULT_COLUMNS.pharmacies),
 doctors: merge("columns.doctors", DEFAULT_COLUMNS.doctors),
 orders: merge("columns.orders", DEFAULT_COLUMNS.orders),
 });

 const mergeFields = (key: string, def: FormFieldConfig[]) => {
 const stored = v[key] as FormFieldConfig[] | undefined;
 if (!Array.isArray(stored) || stored.length === 0) return def;
 const have = new Set(stored.map((f) => f.key));
 const mandatory = def.filter((f) => f.key === "location" && !have.has(f.key));
 return [...stored, ...mandatory];
 };
 setFormFields({
 pharmacies: mergeFields("fields.pharmacies", DEFAULT_FORM_FIELDS.pharmacies),
 doctors: mergeFields("fields.doctors", DEFAULT_FORM_FIELDS.doctors),
 orders: mergeFields("fields.orders", DEFAULT_FORM_FIELDS.orders),
 });
 }, [v]);
}

```



```

}, []);

useEffect(() => {
 load();
}, [load]);

const persist = async (key: string, value: unknown, success = "❏ □□□□□ □□ □ □□□□□□□□ □□□□□ □□□□□□") => {
 const res = await fetch("/api/settings", {
 method: "PUT",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ key, value }),
 });
 setMsg(res.ok ? success : "❏ □□□ □□ □□□□□ □□□□□□□□");
 return res.ok;
};

const save = async (key: string, value: unknown) => {
 if (!(await confirm({ title: "□□□□□ □□□□□□□□", message: "□□□□□□□ □□□□□ □ □□□□□□□□ □□ □□□ □□□□□ □□□□□ □□□□□", confirmText: "□□□□□" })))
 return;
 await persist(key, value);
};

const move = (list: ColumnConfig[], i: number, dir: -1 | 1) => {
 const next = [...list];
 const j = i + dir;
 if (j < 0 || j >= next.length) return next;
 [next[i], next[j]] = [next[j], next[i]];
 return next;
};

const moveP = (i: number, dir: -1 | 1) => {
 const next = [...products];
 const j = i + dir;
 if (j < 0 || j >= next.length) return;
 [next[i], next[j]] = [next[j], next[i]];
 setProducts(next);
};

const current = cols[tab] ?? [];
const currentFields = formFields[tab] ?? [];

return (
 <div className="space-y-4">
 <SectionTitle icon="❏">□□□□□□ □□□□□□□ □ □□□□□□</SectionTitle>
 {msg ? <Alert kind="success">{msg}</Alert> : null}

 <Card>
 <div className="mb-3 flex flex-wrap gap-1 rounded-xl bg-slate-100 p-1 text-xs font-bold">
 {TABLES.map((t) => (
 <button
 key={t.key}
 onClick={() => setTab(t.key)}
 className={`rounded-lg px-3 py-2 ${tab === t.key ? "bg-teal-600 text-white" : "text-slate-600"}`}
 >
 {t.label}
 </button>
))}
 </div>

 <p className="mb-2 text-[11px] text-slate-500">
 □□ □□□ □□□□ □□□□ □□□□ □□□□ □□ □□ □□□□□□ □□□□□ □□ □□□□□□ □□□□□□. □□□□□ □□□□ □□ □□□□ □□□□□ □□□.
 </p>

 <ul className="space-y-1">
 {current.map((c, i) => (
 <li key={c.key} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2">
 {toPersianDigits(i + 1)}
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={c.visible}
 onChange={(e) => {
 const next = [...current];
 next[i] = { ...c, visible: e.target.checked };
 setCols({ ...cols, [tab]: next });
 }}
 />
 <Input
 value={c.label}
 onChange={(e) => {
 const next = [...current];
 next[i] = { ...c, label: e.target.value };
 setCols({ ...cols, [tab]: next });
 }}
 />

))}

 </Card>
 </div>
)

```

```

 className="max-w-[180px] px-2 py-1 text-xs"
 />
 {c.key}
 <div className="mr-auto flex gap-1">
 <button
 onClick={() => setCols({ ...cols, [tab]: move(current, i, -1) })}
 className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
 >
 ↑
 </button>
 <button
 onClick={() => setCols({ ...cols, [tab]: move(current, i, 1) })}
 className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
 >
 ↓
 </button>
 <button
 onClick={async () => {
 if (
 !(await confirm({
 title: "تایید حذف",
 message: `حذف ستون «${c.label}» از جدول؟`,
 confirmText: "بله",
 danger: true,
 }))
)
 return;
 setCols({ ...cols, [tab]: current.filter((_, x) => x !== i) });
 }}
 className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
 >
 حذف
 </button>
 </div>

)}

<div className="mt-3 flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
 تعداد ستونها:
 <select
 value={addCol}
 onChange={(e) => setAddCol(e.target.value)}
 className="rounded-xl border border-slate-300 px-2 py-2 text-xs"
 >
 <option value="">افزودن ستون...</option>
 {(AVAILABLE_COLUMNS[tab] ?? []).filter((a) => !current.some((c) => c.key === a.key))
 .map((a) => (
 <option key={a.key} value={a.key}>
 {a.label}
 </option>
))}
 </select>
 <Button
 variant="soft"
 onClick={async () => {
 const found = (AVAILABLE_COLUMNS[tab] ?? []).find((a) => a.key === addCol);
 if (!found) {
 setMsg("ستون ناموجود است");
 return;
 }
 const next = [...current, { ...found, visible: true }];
 setCols({ ...cols, [tab]: next });
 setAddCol("");
 await persist(`columns.${tab}`, next, `تایید افزودن ستون «${found.label}»`);
 }}
 >
 افزودن ستون
 </Button>

 {toPersianDigits((AVAILABLE_COLUMNS[tab] ?? []).filter((a) => !current.some((c) => c.key === a.key)).length)}

</div>

<div className="mt-3 flex gap-2">
 <Button onClick={() => save(`columns.${tab}`, current)}>
 ذخیره
 </Button>
 <Button
 variant="ghost"
 onClick={async () => {
 if (!(await confirm({ title: "تایید حذف جدول", message: "آیا می‌خواهید این جدول را حذف کنید؟", confirmText: "بله" })))
 return;
 }}
 >
 حذف جدول
 </Button>
</div>

```

```

 setCols({ ...cols, [tab]: DEFAULT_COLUMNS[tab] });
 }}
 >
 ۰۰۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰۰۰
 </Button>
</div>
</Card>

{/* ۰۰۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰ ۰۰۰ — ۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰۰۰ ۰۰۰۰ */}
<Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <div>
 <h3 className="text-sm font-black text-slate-800">۰ ۰۰۰۰۰۰۰۰ ۰۰۰ ۰۰۰</h3>
 <p className="mt-1 text-[11px] text-slate-500">
 ۰۰۰ ۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰۰۰ ۰۰۰۰ ۰۰ «۰۰۰ ۰۰۰» ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰. ۰۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰ ۰۰
 ۰۰ ۰۰۰۰۰.
 </p>
 </div>
 <Badge tone="green">{toPersianDigits(currentFields.filter((f) => f.visible).length)} ۰۰۰۰ ۰۰۰۰</Badge>
 </div>

 <ul className="space-y-1">
 {currentFields.map((f, i) => (
 <li key={f.key} className="flex flex-wrap items-center gap-2 rounded-xl bg-indigo-50/60 px-3 py-2 ring-1 rin
g-indigo-100">
 {toPersianDigits(i + 1)}
 <input
 type="checkbox"
 className="size-4 accent-indigo-600"
 checked={f.visible}
 onChange={(e) => {
 const next = [...currentFields];
 next[i] = { ...f, visible: e.target.checked };
 setFormFields({ ...formFields, [tab]: next });
 }}
 </input>
 <Input
 value={f.label}
 onChange={(e) => {
 const next = [...currentFields];
 next[i] = { ...f, label: e.target.value };
 setFormFields({ ...formFields, [tab]: next });
 }}
 className="max-w-[220px] px-2 py-1 text-xs"
 </>
 {f.key}
 <div className="mr-auto flex gap-1">
 <button
 onClick={() => setFormFields({ ...formFields, [tab]: move(currentFields, i, -1) })}
 className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
 >
 ↑
 </button>
 <button
 onClick={() => setFormFields({ ...formFields, [tab]: move(currentFields, i, 1) })}
 className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200"
 >
 ↓
 </button>
 <button
 onClick={async () => {
 if (!(await confirm({
 title: "۰۰۰۰ ۰۰۰۰ ۰۰۰",
 message: `۰۰۰۰ «${f.label}» ۰۰ ۰۰۰ ۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰.`,
 confirmText: "۰۰۰",
 danger: true,
 }))) return;
 const next = currentFields.filter((_, x) => x !== i);
 setFormFields({ ...formFields, [tab]: next });
 await persist(`fields.${tab}`, next, `۰ ۰۰۰۰ «${f.label}» ۰۰۰ ۰۰`);
 }}
 className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
 >
 ۰۰۰
 </button>
 </div>

))}

 <div className="mt-3 flex flex-wrap items-center gap-2 rounded-xl bg-indigo-50 p-2 ring-1 ring-indigo-200">
 ۰۰۰۰۰۰ ۰۰۰۰ ۰۰ ۰۰۰
 <select
 value={addField}
 onChange={(e) => setAddField(e.target.value)}
 >

```

```

 className="rounded-xl border border-indigo-200 bg-white px-2 py-2 text-xs"
 >
<option value="">□□□□□□ □□□□...</option>
 {(AVAILABLE_FORM_FIELDS[tab] ?? [])
 .filter((a) => !currentFields.some((f) => f.key === a.key))
 .map((a) => (
 <option key={a.key} value={a.key}>{a.label}</option>
))
 }
 </select>
 <Button
 variant="soft"
 onClick={async () => {
 const found = (AVAILABLE_FORM_FIELDS[tab] ?? []).find((a) => a.key === addField);
 if (!found) {
 setMsg("□□□□□□ □□ □□□□ □□ □□ □□□□□□ □□□□");
 return;
 }
 const next = [...currentFields, { ...found, visible: true }];
 setFormFields({ ...formFields, [tab]: next });
 setAddField("");
 await persist(`fields.${tab}`, next, `□ □□□□ «${found.label}» □□ □□□ □□□ □□□□ □□`);
 }}
 >
 □ □□□□□□ □ □□□□□
 </Button>
</div>

<div className="mt-3 flex flex-wrap gap-2">
 <Button onClick={() => save(`fields.${tab}`, currentFields)}>□ □□□□□ □□□□□ □□□□□□□ □□□</Button>
 <Button
 variant="ghost"
 onClick={async () => {
 if (!(await confirm({ title: "□□□□□□□□□□ □□□", message: "□□□□ □□□□□□□□ □□ □□ □□□□ □□□□□□□□", confirmText: "□□□□□□□□□□" }))) return;
 const next = DEFAULT_FORM_FIELDS[tab];
 setFormFields({ ...formFields, [tab]: next });
 await persist(`fields.${tab}`, next, "□ □□□□□□□ □□ □□ □□□□ □□□□□□□□");
 }}
 >
 □□□□□□□□□ □□ □□□□□□
 </Button>
</div>
</Card>

<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□□□□□□ □ □□□□□ (□□□□□□ □□□□□□ □□□□/□□□□□□□□)</h3>
 <ul className="space-y-1">
 {products.map((p, i) => (
 <li key={p.key} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2">
 {toPersianDigits(i + 1)}
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={p.enabled}
 onChange={(e) => {
 const next = [...products];
 next[i] = { ...p, enabled: e.target.checked };
 setProducts(next);
 }}
 />
 <input
 value={p.label}
 onChange={(e) => {
 const next = [...products];
 next[i] = { ...p, label: e.target.value };
 setProducts(next);
 }}
 className="max-w-[150px] px-2 py-1 text-xs"
 />
 <input
 value={p.bonusLabel}
 onChange={(e) => {
 const next = [...products];
 next[i] = { ...p, bonusLabel: e.target.value };
 setProducts(next);
 }}
 className="max-w-[180px] px-2 py-1 text-xs"
 />
 <div className="mr-auto flex gap-1">
 <button onClick={() => moveP(i, -1)} className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200">
 ↑
 </button>
 <button onClick={() => moveP(i, 1)} className="rounded-lg bg-white px-2 py-1 text-xs ring-1 ring-slate-200">
 ↓
 </button>
 </div>

))


```

```

↓
</button>
<button
 onClick={async () => {
 if (
 !(await confirm({
 title: "□□□□ □□□□",
 message: `□□□□□ <{p.label}> □□□ □□□□`,
 confirmText: "□□□",
 danger: true,
)))
)
 return;
 setProducts(products.filter((_, x) => x !== i));
 }}
 className="rounded-lg bg-rose-100 px-2 py-1 text-xs font-bold text-rose-700"
>
□□□
</button>
</div>

)}}


```

```

<div className="mt-3 flex flex-wrap gap-2">
 <Input
 value={newProduct}
 onChange={(e) => setNewProduct(e.target.value)}
 placeholder="□□□ □□□□□ □□□□..."
 className="max-w-[220px]"
 />
 <Button
 variant="soft"
 onClick={() => {
 const label = newProduct.trim();
 if (!label) return;
 const key = `p_${Date.now()}`;
 setProducts([...products, { key, label, bonusLabel: `□□□□□ □□□□□ ${label}`, enabled: true }]);
 setNewProduct("");
 }}
 >
 □ □□□□□ □□□□
 </Button>
<Button onClick={() => save("products", products)}>□ □□□□□ □□□□□□</Button>
<Button variant="ghost" onClick={() => setProducts(DEFAULT_PRODUCTS)}>
 □□□□□□□□□ □□□□□□□□
</Button>
</div>
</Card>
</div>
);
}

```

```

@@ src/app/admin/homes/page.tsx typescript
import HomeScreen from "@components/screens/HomeScreen";
export default function Page() {
 return <HomeScreen isAdmin />;
}

```

```

@@ src/app/admin/leaves/page.tsx typescript
import LeaveScreen from "@components/screens/LeaveScreen";

export default function Page() {
 return <LeaveScreen />;
}

```

```

@@ src/app/admin/live/page.tsx typescript
"use client";

```

```

import { useCallback, useMemo, useState } from "react";
import dynamic from "next/dynamic";
import { Alert, Badge, Button, Card, SectionTitle } from "@components/ui";
import { useLive } from "@lib/useLive";
import { formatDistance } from "@lib/tracker";
import { tehranDateTime, tehranTime, toPersianDigits } from "@lib/jalali";
import NavButton from "@components/NavButton";

```

```

const MapBox = dynamic(() => import("@components/MapBox"), { ssr: false });

```

```

type Live = {
 userId: number;
 repName: string;
 lat: number;
 lng: number;
 accuracy: number | null;
}

```

```

speed: number | null;
battery: number | null;
gpsOn: boolean;
tripId: number | null;
recordedAt: string;
updatedAt: string;
ageMs: number;
live: boolean;
stale: boolean;
activeTrip: { id: number; distanceM: number } | null;
};

const ageLabel = (ms: number) => {
 const m = Math.floor(ms / 60000);
 if (m < 1) return "۰۰۰۰ ۰۰۰۰";
 if (m < 60) return `${toPersianDigits(m)} ۰۰۰۰ ۰۰۰`;
 const h = Math.floor(m / 60);
 if (h < 24) return `${toPersianDigits(h)} ۰۰۰۰ ۰۰۰`;
 return `${toPersianDigits(Math.floor(h / 24))} ۰۰۰ ۰۰۰`;
};

export default function LivePage() {
 const [rows, setRows] = useState<Live[]>([]);
 const [missing, setMissing] = useState<{ userId: number; repName: string }[]>([]);
 const [selected, setSelected] = useState<number | null>(null);
 const [autoRefresh, setAutoRefresh] = useState(true);

 const load = useCallback(async () => {
 const res = await fetch("/api/live", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 setRows(d.rows ?? []);
 setMissing(d.missing ?? []);
 }, []);

 const { refresh } = useLive(load, 12000, autoRefresh);

 const focused = useMemo(() => rows.find((r) => r.userId === selected) ?? null, [rows, selected]);
 const online = rows.filter((r) => r.live).length;
 const onTrip = rows.filter((r) => r.activeTrip).length;

 const mapPoints = (focused ? [focused] : rows).map((r) => ({
 lat: r.lat,
 lng: r.lng,
 label: `${r.repName} — ${r.live ? "۰ ۰۰۰۰" : "۰ ۰۰۰۰۰ ۰۰۰۰۰۰"} (${ageLabel(r.ageMs)})`,
 color: r.live ? (r.activeTrip ? "#16a34a" : "#0ea5e9") : r.stale ? "#f59e0b" : "#94a3b8",
 }));

 return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="۰">۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰۰۰۰</SectionTitle>
 <div className="flex flex-wrap gap-2">
 <Badge tone="green">۰ ۰۰۰۰۰: {toPersianDigits(online)}</Badge>
 <Badge tone="amber">۰ ۰۰ ۰۰۰ ۰۰۰۰: {toPersianDigits(onTrip)}</Badge>
 <Button variant={autoRefresh ? "success" : "ghost"} onClick={() => setAutoRefresh((v) => !v)}>
 {autoRefresh ? "۰ ۰۰۰ (۰۰ ۰۰ ۰۰۰۰)" : "۰ ۰۰۰۰"}
 </Button>
 <Button variant="ghost" onClick={refresh}>
 ۰ ۰۰۰۰۰۰۰
 </Button>
 </div>
 </div>

 <Alert kind="info">
 ۰۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰ ۰۰۰ ۰۰۰۰ GPS ۰۰۰۰۰ ۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰ ۰۰۰۰۰۰
 ۰۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰.
 </Alert>

 <div className="grid grid-cols-1 gap-3 lg:grid-cols-3">
 <Card className="lg:col-span-1">
 <div className="mb-2 flex items-center justify-between">
 <h3 className="text-sm font-bold text-slate-700">۰۰۰۰۰۰۰۰۰</h3>
 {focused ? (
 <button onClick={() => setSelected(null)} className="text-[11px] font-bold text-teal-700">
 ۰۰۰۰۰ ۰۰۰
 </button>
) : null}
 </div>
 </div>

 <div className="max-h-[460px] space-y-2 overflow-y-auto">
 {rows.map((r) => (
 <button
 key={r.userId}
 onClick={() => setSelected(selected === r.userId ? null : r.userId)}
 >

```

```

classList={w-full rounded-xl px-3 py-2 text-right ring-1 transition ${
 selected === r.userId ? "bg-teal-50 ring-teal-300" : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
}}`
}
>
<div className="flex flex-wrap items-center gap-1.5">
 {r.repName}
 <Badge tone={r.live ? "green" : r.stale ? "amber" : "slate"}>
 {r.live ? "🟢 📶📶📶📶" : r.stale ? "🟡 📶📶📶📶" : "🔴 📶📶📶📶"}
 </Badge>
 {r.activeTrip ? <Badge tone="green">📍 📶📶📶📶</Badge> : null}
 {!r.gpsOn ? <Badge tone="amber">GPS 📶📶📶📶</Badge> : null}
</div>
<div className="mt-1 flex flex-wrap gap-2 text-[10px] text-slate-500">
 {ageLabel(r.ageMs)}
 {r.accuracy ? 📏📏 {toPersianDigits(Math.round(r.accuracy))} : null}
 {r.speed ? 📏📏📏 {toPersianDigits(Math.round(r.speed * 3.6))} km/h : null}
 {r.battery !== null ? 🔋 {toPersianDigits(r.battery)} : null}
 {r.activeTrip ? 📏📏📏📏 {formatDistance(r.activeTrip.distanceM)} : null}
</div>
<div className="mt-1 text-[10px] text-slate-400">{tehranDateTime(r.updatedAt)}</div>
</button>
)}}
{rows.length === 0 ? (
<p className="py-6 text-center text-xs text-slate-400">📍📍📍📍 📶📶📶📶 📶📶📶📶 📶📶 📶📶</p>
) : null}

{missing.length ? (
 <div className="rounded-xl bg-slate-100 p-2 text-[11px] text-slate-500">
 📍📍📍 📶📶📶📶: {missing.map((m) => m.repName).join("📍 ")}
 </div>
) : null}
</div>
</Card>

<Card className="lg:col-span-2">
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <h3 className="text-sm font-bold text-slate-700">
 {focused ? `📍📍📍📍📍 ${focused.repName}` : "📍📍📍 📍📍 📍📍📍📍📍"}
 </h3>
 {focused ? (
 <div className="flex items-center gap-2">
 <NavButton lat={focused.lat} lng={focused.lng} label={focused.repName} />

 {focused.lat.toFixed(5)}, {focused.lng.toFixed(5)}

 </div>
) : null}
 </div>

 <MapBox height={430} points={mapPoints} follow={!!focused} />

 {focused ? (
 <div className="mt-3 grid grid-cols-2 gap-2 text-center text-[11px] sm:grid-cols-4">
 {[
 ["📍📍📍📍 📶📶📶📶📶📶", toPersianDigits(Math.round(focused.updatedAt))],
 ["📍📍📍📍 GPS", focused.gpsOn ? "📶📶📶📶" : "📶📶📶📶"],
 ["📍📍📍 📶📶📶📶", focused.accuracy ? `${toPersianDigits(Math.round(focused.accuracy))} 📏📏` : "—"],
 ["📍📍📍📍 📍📍", focused.activeTrip ? formatDistance(focused.activeTrip.distanceM) : "—"],
]}
 .map(([, v]) => (
 <div key={l} className="rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
 <div className="font-black text-slate-800">{v}</div>
 <div className="text-[10px] text-slate-500">{l}</div>
 </div>
)))
 </div>
) : null}
</Card>
</div>
</div>
);
}

@@ src/app/admin/map/page.tsx typescript
import MapExplorer from "@components/screens/MapExplorer";

export default function Page() {
 return <MapExplorer isAdmin />;
}

@@ src/app/admin/messengers/page.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";

```

```

import { PLATFORMS } from "@lib/constants";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";

type M = {
 id: number;
 platform: string;
 label: string;
 targetType: string;
 target: string;
 token: string;
 provider: string;
 apiUrl: string;
 enabled: boolean;
 lastStatus: string;
 lastOkAt: string | null;
 lastErrorAt: string | null;
};

type Log = { id: number; platform: string; target: string; ok: boolean; detail: string; createdAt: string };
type Proxy = { url: string; enabled: boolean; secret?: string };
type Chat = { id: string; title: string; type: string };
type Sms = { provider: string; apiKey: string; sender: string; pattern: string; enabled: boolean };

const WA_PROVIDERS = [
 { key: "whatsiplus", label: "☐☐☐☐☐☐☐☐☐☐ (whatsiplus.ir)", hint: "☐☐☐☐ API ☐☐ ☐☐☐ | ☐☐☐☐: ☐☐☐☐..." },
 { key: "ultrams", label: "UltraMsg", hint: "☐☐☐☐: instancelid:token | ☐☐☐☐: 989121111111" },
 { key: "cloudapi", label: "WhatsApp Cloud API (☐☐☐)", hint: "☐☐☐☐: phoneNumberId:accessToken" },
 { key: "custom", label: "☐☐☐☐☐☐ ☐☐☐☐☐☐", hint: "☐☐☐☐ ☐☐ {phone} ☐ {text} ☐ {token}" },
];

const emptyForm = (platform: string) => ({
 platform,
 label: "",
 targetType: platform === "whatsapp" ? "phone" : "group",
 target: "",
 token: "",
 provider: platform === "whatsapp" ? "whatsiplus" : "",
 apiUrl: "",
 enabled: true,
});

export default function MessengersPage() {
 const [rows, setRows] = useState<M[]>([]);
 const [logs, setLogs] = useState<Log[]>([]);
 const [proxy, setProxy] = useState<Proxy>({ url: "", enabled: true, secret: "" });
 const [sms, setSms] = useState<Sms>({ provider: "kavenegar", apiKey: "", sender: "", pattern: "", enabled: false });
 const [sysTokens, setSysTokens] = useState<Record<string, string>>({});
 const [tab, setTab] = useState("telegram");
 const [form, setForm] = useState(emptyForm("telegram"));
 const [chats, setChats] = useState<Chat[]>([]);
 const [chatBusy, setChatBusy] = useState(false);
 const [msg, setMsg] = useState("");
 const [showToken, setShowToken] = useState<Record<number, boolean>>({});
 const [testing, setTesting] = useState<number | null>(null);
 const [workerVersion, setWorkerVersion] = useState<"new" | "old" | "down" | "">("");
 const [autoSend, setAutoSend] = useState(true);
 const confirm = useConfirm();

 const meta = PLATFORMS.find((p) => p.key === tab)!;

 const load = useCallback(async () => {
 const res = await fetch("/api/messengers", { cache: "no-store" });
 if (!res.ok) return;
 const d = await res.json();
 setRows(d.rows ?? []);
 setLogs(d.logs ?? []);
 if (d.proxy) setProxy(d.proxy);
 if (d.sms) setSms(d.sms);
 if (typeof d.autoSend === "boolean") setAutoSend(d.autoSend);
 }, []);

 useLive(load, 30000);

 useEffect(() => {
 fetch("/api/messengers/updates", { cache: "no-store" })
 .then((r) => (r.ok ? r.json() : null))
 .then((d) => setSysTokens(d?.tokens ?? {}))
 .catch(() => undefined);
 }, []);

 useEffect(() => {
 setForm(emptyForm(tab));
 setChats([]);
 }, [tab]);
}

```



```

/* ----- 000000 ----- */

const add = async () => {
if (!form.target.trim()) return setMsg("0 00000 0000 000000 000");
 if (meta.needsToken && !form.token.trim() && !sysTokens[tab]) {
return setMsg("0 0000 000 0000000000 0000 0000 000");
 }
 if (
 !(await confirm({
 title: `000000 0000 ${meta.label}`,
 message: `0000 «${form.label || form.target}» 00000 0000`,
 confirmText: "000000",
 })))
)
 return;
 const res = await fetch("/api/messengers", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(form),
 });
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setForm(emptyForm(tab));
 setMsg("0 0000 0000 000 00");
 load();
 } else setMsg(`0 ${d.error ?? "000 00 000"}`);
};

const patch = async (body: Record<string, unknown>) => {
 await fetch("/api/messengers", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(body),
 });
 load();
};

const remove = async (m: M) => {
 if (
 !(await confirm({
 title: "000 0000",
 message: `0000 «${m.label || m.target}» 000 0000`,
 confirmText: "000",
 danger: true,
 })))
)
 return;
 await fetch(`/api/messengers?id=${m.id}`, { method: "DELETE" });
 load();
};

const test = async (id: number) => {
 setTesting(id);
 setMsg("0 00 000 00000 0000 00000000...");
 const res = await fetch("/api/messengers/test", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ id }),
 });
 const d = await res.json().catch(() => ({}));
 setTesting(null);
 setMsg(d.ok ? `0 00000 0000 (${d.via === "proxy" ? "00 0000 000000": "000000"})` : `0 ${d.detail ?? "000000"}`);
 load();
};

const testAll = async () => {
 setMsg("0 00000 0000 00000000 00 000 00000000 0000...");
 const res = await fetch("/api/messengers/test", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ all: true }),
 });
 const d = await res.json().catch(() => ({}));
 setMsg(
 d.summary
 ? `${d.sent}/${d.total} 0000 — ${d.summary}.slice(0, 400)`
 : `0 ${d.error ?? "00000 000000"}`,
);
 load();
};

const discover = async () => {
 setChatBusy(true);
 setChats([]);

```

```

setMsg("📄 📄📄📄📄 📄📄📄 📄📄📄📄📄...");
const res = await fetch("/api/messengers/updates", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ platform: tab, token: form.token }),
});
const d = await res.json().catch(() => ({}));
setChatBusy(false);
setChats(d.chats ?? []);
setMsg(d.ok ? `📄 ${d.chats ?? []}.length} 📄 📄📄📄 📄 — 📄📄 📄📄📄📄 📄📄📄📄: 📄 ${d.detail ?? "📄📄📄📄📄"}`);
};

const saveProxy = async (next: Proxy) => {
 setProxy(next);
 const res = await fetch("/api/settings", {
 method: "PUT",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ key: "messagingProxy", value: next }),
 });
 setMsg(res.ok ? "📄 📄📄📄📄📄 📄📄📄📄 📄📄📄 📄📄: 📄 📄📄 📄 📄📄📄📄");
};

const testProxy = async () => {
 setMsg("📄 📄📄📄📄 📄📄📄📄📄...");
 const res = await fetch("/api/messengers/test", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ pingProxy: true }),
 });
 const d = await res.json().catch(() => ({}));
 setWorkerVersion(d.version ?? "");
 setMsg(d.detail ?? (d.ok ? "📄📄📄📄📄 📄 📄📄📄📄 📄📄": "📄📄📄📄📄 📄📄📄 📄📄📄"));
};

const copyWorker = async () => {
 const code = await fetch("/cloudflare-worker.js").then((r) => r.text()).catch(() => "");
 if (!code) return setMsg("📄 📄📄📄📄 📄 📄📄📄 📄📄📄📄 📄📄");
 await navigator.clipboard.writeText(code).catch(() => window.open("/cloudflare-worker.js", "_blank"));
 setMsg("📄 📄 📄📄📄 📄📄 📄 — 📄📄 Cloudflare 📄📄📄📄📄 📄📄📄 📄 Deploy 📄📄📄📄");
};

const tabRows = rows.filter((r) => r.platform === tab);
const activeCount = rows.filter((r) => r.enabled).length;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="📄">📄📄📄📄 📄📄📄📄 📄📄📄📄📄📄 📄 📄📄📄📄📄📄📄📄📄📄</SectionTitle>
 <div className="flex gap-2">
 <Badge tone={activeCount ? "green" : "amber"}>{toPersianDigits(activeCount)} 📄📄📄 📄📄📄</Badge>
 <Button variant="soft" onClick={testAll}>
 📄 📄📄 📄📄
 </Button>
 </div>
 </div>
 {msg ? <Alert kind={msg.startsWith("📄") ? "error" : "success"}>{msg}</Alert> : null}

 /* ----- 📄📄📄📄 📄📄📄📄 ----- */
 <Card>
 <div className="flex flex-wrap items-center gap-3">
 📄 📄📄📄 📄📄📄📄 📄📄📄📄📄📄
 <button
 onClick={async () => {
 const next = !autoSend;
 if (
 !(await confirm({
 title: next ? "📄📄📄📄📄📄 📄📄📄 📄📄📄📄": "📄📄📄📄📄📄📄📄 📄📄📄 📄📄📄📄",
 message: next
 ? "📄📄 📄📄📄📄 📄📄📄📄📄📄 📄 📄 📄📄📄 📄📄📄📄 📄 📄📄 📄📄📄📄📄📄📄📄 📄📄📄 📄📄📄📄 📄📄📄"
 : "📄📄📄📄 📄📄📄📄 📄📄📄 📄📄📄 📄📄📄📄📄📄 📄📄📄 📄📄📄 📄📄📄 📄📄📄",
 confirmText: next ? "📄📄📄 📄": "📄📄📄📄 📄",
 })))
)
 return;
 setAutoSend(next);
 const res = await fetch("/api/settings", {
 method: "PUT",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ key: "autoSendOrders", value: { enabled: next } }),
 });
 setMsg(res.ok ? (next ? "📄 📄📄📄 📄📄📄📄 📄📄📄 📄📄: 📄 📄📄📄 📄📄📄📄 📄📄📄 📄📄": "📄 📄📄");
 }}
 className={`rounded-xl px-4 py-2 text-xs font-bold ${
 autoSend ? "bg-emerald-600 text-white" : "bg-slate-200 text-slate-700"
 }`
 </div>
 </Card>
 </div>
);

```

```

 >
 {autoSend ? "ارسال پیام — پیام شما در حال ارسال است" : "ارسال پیام — پیام شما در حال ارسال است"}
 </button>

 پیام شما در حال ارسال است. پیام شما در حال ارسال است. پیام شما در حال ارسال است.

</div>
</Card>

/* ----- لیست پلتفرمها ----- */
<div className="flex flex-wrap gap-1 rounded-2xl bg-slate-200 p-1">
 {PLATFORMS.map((p) => {
 const count = rows.filter((r) => r.platform === p.key).length;
 const on = rows.some((r) => r.platform === p.key && r.enabled);
 return (
 <button
 key={p.key}
 onClick={() => setTab(p.key)}
 className={`flex-1 rounded-xl px-3 py-2.5 text-xs font-bold transition ${
 tab === p.key ? "bg-white text-teal-700 shadow" : "text-slate-600 hover:bg-white/50"
 }`}
 >
 {p.icon} {p.label}
 {count ? (

 {toPersianDigits(count)}

) : null}
 </button>
);
 })}
</div>

/* ----- توضیحات پلتفرمها ----- */
<Card>
 <h3 className="mb-2 text-sm font-black text-slate-800">
 {meta.icon} توضیحات پلتفرمها {meta.label}
 </h3>
 <ol className="list-inside list-decimal space-y-1 text-[11px] leading-6 text-slate-600">
 {meta.guide.map((g, i) => (
 <li key={i}>{g}
))}

 {sysTokens[tab] ? (
 <p className="mt-2 rounded-lg bg-emerald-50 px-3 py-1.5 text-[11px] font-bold text-emerald-800">
 این پلتفرم برای ارسال پیام در واتساپ، تلگرام، و سایر شبکه‌های اجتماعی استفاده می‌شود.
 </p>
) : null}
</Card>

/* ----- فرم ارسال پیام ----- */
<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">فرم ارسال پیام {meta.label}</h3>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-3">
 <Field label="نام پلتفرم (برای ارسال پیام)">
 <Input
 value={form.label}
 onChange={(e) => setForm({ ...form, label: e.target.value })}
 placeholder="نام پلتفرم (برای ارسال پیام)"
 />
 </Field>

 {tab === "whatsapp" ? (
 <Field label="شماره تلفن (برای ارسال پیام)" hint={WA_PROVIDERS.find((p) => p.key === form.provider)?.hint}>
 <select
 value={form.provider}
 onChange={(e) => setForm({ ...form, provider: e.target.value })}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 {WA_PROVIDERS.map((p) => (
 <option key={p.key} value={p.key}>
 {p.label}
 </option>
))}
 </select>
 </Field>
) : (
 <Field label="گروه (برای ارسال پیام)">
 <select
 value={form.targetType}
 onChange={(e) => setForm({ ...form, targetType: e.target.value })}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="group">گروه</option>
 </select>
 </Field>
)}
 </div>

```

```

<option value="channel">کانال</option>
<option value="phone">شماره</option>
</select>
</Field>
)}

<Field label="شماره تماس" required hint={meta.targetHint}>
 <Input value={form.target} onChange={(e) => setForm({ ...form, target: e.target.value })} className="text-left" />
</Field>

<Field label="شماره / آیدی API" hint={meta.tokenHint}>
 <Input value={form.token} onChange={(e) => setForm({ ...form, token: e.target.value })} className="text-left" />
</Field>

{tab === "whatsapp" && form.provider === "custom" ? (
 <div className="sm:col-span-2">
 <Field label="آیدی API" hint="آیدی API: https://api.example.com/send?to={phone}&msg={text}&key={token}">
 <Input value={form.apiUrl} onChange={(e) => setForm({ ...form, apiUrl: e.target.value })} className="text-left" />
 </Field>
 </div>
) : null}
</div>

<div className="mt-3 flex flex-wrap items-center gap-2">
 {tab === "telegram" || tab === "bale" ? (
 <Button variant="soft" onClick={discover} disabled={chatBusy}>
 {chatBusy ? "در حال بارگذاری..." : "درخواست چت"}
 </Button>
) : null}
 <div className="flex-1" />
 <Button onClick={add}>افزودن</Button>
</div>

{chats.length ? (
 <div className="mt-3 rounded-xl bg-slate-50 p-2 ring-1 ring-slate-200">
 <div className="mb-1 text-[11px] font-bold text-slate-600">لیست چت ها</div>
 <div className="flex flex-wrap gap-1">
 {chats.map((c) => (
 <button
 key={c.id}
 onClick={() =>
 setForm({ ...form, target: c.id, label: c.title, targetType: c.type === "phone" ? "group" : "person" })
 }
 className="rounded-lg bg-white px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-teal-200 hover:r:bg-teal-50"
 >
 {c.type === "phone" ? "گروه" : "شخص"} {c.title}

 {c.id}

 </button>
))}
 </div>
 </div>
) : null}
</Card>

/* ----- لیست چت ها ----- */
<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">
 لیست چت ها {meta.label} ({toPersianDigits(tabRows.length)})
 </h3>
 {tabRows.length === 0 ? (
 <p className="py-6 text-center text-xs text-slate-400">لیست چت ها خالی است</p>
) : (
 <div className="space-y-2">
 {tabRows.map((m) => (
 <div key={m.id} className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
 <div className="flex flex-wrap items-center gap-2">
 {m.label} {m.id}
 <Badge tone={m.enabled ? "green" : "amber"}>{m.enabled ? "فعال" : "غیرفعال"}</Badge>

 {m.target}

 {m.provider ? (

 {WA_PROVIDERS.find((p) => p.key === m.provider)?.label ?? m.provider}

) : null}
 </div>
 <div className="mr-auto flex flex-wrap gap-1">

```

```

 <button
 onClick={() => test(m.id)}
 disabled={testing === m.id}
 className="rounded-lg bg-sky-100 px-2 py-1 text-[11px] font-bold text-sky-700 disabled:opacity-50"
 >
 {testing === m.id ? "...": "□ □□□ □□□□"}
 </button>
 <button
 onClick={() => patch({ id: m.id, enabled: !m.enabled })}
 className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
 >
 {m.enabled ? "□□□□□□□□ □□": "□□□□ □□"}
 </button>
 <button
 onClick={() => setShowToken((s) => ({ ...s, [m.id]: !s[m.id] }))}
 className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
 >
 {showToken[m.id] ? "□ □□□□": "□ □□□□"}
 </button>
 <button
 onClick={() => remove(m)}
 className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
 >
 □□□
 </button>
 </div>
</div>

 {showToken[m.id] ? (
 <div className="mt-2 grid grid-cols-1 gap-2 sm:grid-cols-2">
 <label className="block">
 □□□□
 <input
 value={m.token}
 onChange={(e) => setRows((p) => p.map((x) => (x.id === m.id ? { ...x, token: e.target.value } :
x))))}
 onBlur={() => patch({ id: m.id, token: m.token })}
 className="text-left font-mono text-xs"
 />
 </label>
 <label className="block">
 □□□□□ □□□□
 <input
 value={m.target}
 onChange={(e) => setRows((p) => p.map((x) => (x.id === m.id ? { ...x, target: e.target.value } :
x))))}
 onBlur={() => patch({ id: m.id, target: m.target })}
 className="text-left font-mono text-xs"
 />
 </label>
 </div>
) : null}

 {m.lastStatus ? (
 <div
 className={`mt-2 rounded-lg px-2 py-1 text-[10px] ${
 m.lastStatus.startsWith("□") ? "bg-emerald-50 text-emerald-700" : "bg-rose-50 text-rose-700"
 }`}
 >
 {m.lastStatus}
 </div>
 {m.lastOkAt ? □□□□□ □□□□: {tehranDateTime(m.lastOkAt)}
: null}

 </div>
) : null}
 </div>
)}
</div>
)}
</Card>

/* ----- □□□□□□ ----- */
<Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <h3 className="text-sm font-bold text-slate-700">□ □□□□□□ □□□□□ (□□□□ □□□□□□□□ □□□□ □□ □□□□□)</h3>
 <Badge tone={proxy.enabled ? "green" : "amber"}>{proxy.enabled ? "□□□□": "□□□□□□□□"}</Badge>
 </div>
 <Alert kind="info">
 □□□□ Render □□ □□□□□□/□□□□/□□□□□ □□□□□□ □□□□□□. □□ □□□□ □□□□ □□□□□□□□ □□□□□□□□ □□ □□□□ Cloudflare Worker □□□□
 □□□□□ □□□□□□□□. □□□□ □□□□□□ □□□□□□ □□□□ □□□□ □□□□□□□□ □□□□□□□□□□.
 </Alert>
 <div className="mt-3 grid grid-cols-1 gap-3 sm:grid-cols-3">
 <div className="sm:col-span-2">
 <Field label="□□□□ □□□□□□">
 <input value={proxy.url} onChange={(e) => setProxy({ ...proxy, url: e.target.value })} className="text-lef
t" />

```

```

 </Field>
 </div>
 <Field label="□□□□ □□□□□□□□">
 <Input value={proxy.secret ?? ""} onChange={(e) => setProxy({ ...proxy, secret: e.target.value })} />
 </Field>
 </div>
 {workerVersion === "old" ? (
 <Alert kind="error">
 □□□□ □□□□ □□□□ □□□□ □□ □ □□□ □□□□□□ □□ □□□□□□ □□□□□□. □□ □□□□ «□□□ □□ □□□□» □□□□ □□□□ □□ □□□□□□□□ □□□□.
 </Alert>
) : null}
 <div className="mt-3 flex flex-wrap gap-2">
 <Button onClick={() => saveProxy(proxy)}>□ □□□□□</Button>
 <Button variant={proxy.enabled ? "danger" : "success"} onClick={() => saveProxy({ ...proxy, enabled: !proxy.enabled })}>
 {proxy.enabled ? "□□□□□□□□ □□□□" : "□□□□□ □□□□□"}
 </Button>
 <Button variant="ghost" onClick={testProxy}>
 □ □□□ □□□□□
 </Button>
 <Button variant="soft" onClick={copyWorker}>
 □ □□□ □□ □□□□
 </Button>
 </div>
 </Card>

 /* ----- □□□□□ ----- */
 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□□□ □□□□□□ (□□□□□□ □□□□□□□□□□)</h3>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-4">
 <Field label="□□□□□□□□□□□□">
 <select
 value={sms.provider}
 onChange={(e) => setSms({ ...sms, provider: e.target.value })}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="kavenegar">□□□□□□□□□□</option>
 <option value="smsir">SMS.ir</option>
 <option value="melipayamak">□□□□□□□□□□</option>
 <option value="custom">□□□□□ □□□□□□□</option>
 </select>
 </Field>
 <Field label="□□□□□□□□□□ API">
 <Input value={sms.apiKey} onChange={(e) => setSms({ ...sms, apiKey: e.target.value })} className="text-left" />
 </Field>
 <Field label="□□□□□□ □□□□□□□□">
 <Input value={sms.sender} onChange={(e) => setSms({ ...sms, sender: e.target.value })} className="text-left" />
 </Field>
 <Field label="□□ □□□□">
 <Input value={sms.pattern} onChange={(e) => setSms({ ...sms, pattern: e.target.value })} className="text-left" />
 </Field>
 </div>
 <div className="mt-3 flex flex-wrap items-center gap-2">
 <label className="flex items-center gap-2 text-xs font-bold text-slate-600">
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={sms.enabled}
 onChange={(e) => setSms({ ...sms, enabled: e.target.checked })}
 />
 □□□□ □□□□
 </label>
 <Button
 onClick={async () => {
 const res = await fetch("/api/settings", {
 method: "PUT",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ key: "sms", value: sms }),
 });
 setMsg(res.ok ? "□ □□□□□□□□ □□□□□ □□□□□ □□" : "□ □□□□");
 }}
 >
 □ □□□□□
 </Button>
 </div>
 </Card>

 /* ----- □□□ ----- */
 <Card>
 <SectionTitle icon="□">□□□□□□ □□□□□□□□□□</SectionTitle>
 <div className="max-h-72 space-y-1 overflow-y-auto text-xs">
 {logs.map((l) => (

```

```

 <div key={l.id} className={`rounded-lg px-3 py-1.5 ${l.ok ? "bg-emerald-50" : "bg-rose-50"}`}>
 {l.ok ? "□" : "□"}{" "}>
 {PLATFORMS.find((p) => p.key === l.platform)?.label ?? l.platform} → {l.target}</sp
an>
 <div className="text-[10px] text-slate-600">{l.detail}</div>
 {tehranDateTime(l.createdAt)}
 </div>
)}
 {logs.length === 0 ? <p className="text-slate-400">□□□□ □□□□□□ □□□□ □□□□ □□□</p> : null}
 </div>
 </Card>
</div>
);
}

@@ src/app/admin/notifications/page.tsx typescript
import NotificationScreen from "@components/screens/NotificationScreen";

export default function Page() {
 return <NotificationScreen />;
}

@@ src/app/admin/options/page.tsx typescript
import OptionsScreen from "@components/screens/OptionsScreen";
export default function Page() {
 return <OptionsScreen canDelete />;
}

@@ src/app/admin/reports/page.tsx typescript
import ReportScreen from "@components/screens/ReportScreen";
export default function Page() {
 return <ReportScreen />;
}

@@ src/app/admin/sync/page.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { orderedEndpoints, pickFastestEndpoint } from "@lib/endpoints";
import { downloadFile } from "@lib/download";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";

type Peer = {
 id: number;
 peer: string;
 peerUrl: string;
 enabled: boolean;
 pullCursor: string | null;
 pushCursor: string | null;
 lastSyncAt: string | null;
 lastStatus: string;
 lastError: string;
 pulled: number;
 pushed: number;
 conflicts: number;
};

type Log = {
 id: number;
 peer: string;
 direction: string;
 tableName: string;
 applied: number;
 skipped: number;
 ok: boolean;
 detail: string;
 createdAt: string;
};

type Info = { node: string; url: string; peers: { peer: string; url: string }[]; hasSecret: boolean; tables: string[] };

export default function SyncPage() {
 const [info, setInfo] = useState<Info | null>(null);
 const [peers, setPeers] = useState<Peer[]>([]);
 const [logs, setLogs] = useState<Log[]>([]);
 const [msg, setMsg] = useState("");
 const [busy, setBusy] = useState(false);
 const [form, setForm] = useState({ peer: "", peerUrl: "" });
 const [speeds, setSpeeds] = useState<{ url: string; ms: number | null }[]>([]);
 const [srcInfo, setSrcInfo] = useState<{ count: number } | null>(null);
 const [dlBusy, setDlBusy] = useState(false);
 const confirm = useConfirm();

 const load = useCallback(async () => {

```

```

const res = await fetch("/api/sync/status", { cache: "no-store" });
if (!res.ok) return;
const d = await res.json();
setInfo(d.info);
setPeers(d.peers ?? []);
setLogs(d.logs ?? []);
}, []);

useLive(load, 20000);

const runSync = async () => {
if (!(await confirm({ title: "🔄 동기화 진행", message: "🔄 동기화 진행 중입니다. 동기화 완료 후 다시 시도해주세요", confirmText: "확인" })))
 return;
setBusy(true);
setMsg("🔄 동기화 진행 중입니다...");
const res = await fetch("/api/sync/run", { method: "POST" });
const d = await res.json().catch(() => ({}));
setBusy(false);
setMsg(d.summary ?? (res.ok ? "🔄 동기화 완료" : "🔄 동기화 실패"));
load();
};

const patch = async (body: Record<string, unknown>) => {
await fetch("/api/sync/status", {
method: "PATCH",
headers: { "Content-Type": "application/json" },
body: JSON.stringify(body),
});
load();
};

const addPeer = async () => {
if (!form.peer || !form.peerUrl.startsWith("http")) return setMsg("🚫 유효하지 않은 URL입니다");
const res = await fetch("/api/sync/status", {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify(form),
});
setMsg(res.ok ? "👤 새 서버 추가 완료" : "🚫 새 서버 추가 실패");
setForm({ peer: "", peerUrl: "" });
load();
};

const testSpeed = async () => {
setMsg("🚀 속도 테스트 진행 중입니다...");
const r = await pickFastestEndpoint(true);
setSpeeds(r);
const best = r.filter((x) => x.ms !== null).sort((a, b) => (a.ms ?? 9e9) - (b.ms ?? 9e9))[0];
setMsg(best ? `🏆 최속 서버: ${best.url} (${best.ms} ms)` : "🚫 속도 테스트 실패");
};

return (
<div className="space-y-4">
<div className="flex flex-wrap items-center justify-between gap-2">
<SectionTitle icon="🔄">🔄 동기화 상태</SectionTitle>
<div className="flex gap-2">
<Button variant="ghost" onClick={testSpeed}>
🚀 속도 테스트
</Button>
<Button onClick={runSync} disabled={busy}>
{busy ? "🔄 ...": "🔄 동기화 진행"}
</Button>
</div>
</div>
{msg ? <Alert kind={msg.startsWith("🚫") ? "error" : "success"}>{msg}</Alert> : null}

/* ----- 🔄 동기화 진행 중입니다. 동기화 완료 후 다시 시도해주세요 ----- */
<Card>
<h3 className="mb-2 text-sm font-black text-slate-800">🔄 동기화 진행 중입니다</h3>
<Alert kind="info">
🔄 동기화 진행 중입니다. 동기화 완료 후 다시 시도해주세요. 동기화 완료 후 다시 시도해주세요.
🔄 동기화 진행 중입니다. 동기화 완료 후 다시 시도해주세요. 동기화 완료 후 다시 시도해주세요.
🔄 동기화 진행 중입니다. 동기화 완료 후 다시 시도해주세요. 동기화 완료 후 다시 시도해주세요.
 git add -A && git commit && git push 동기화 완료. 동기화 완료 후 다시 시도해주세요.
</Alert>
<div className="mt-3 flex flex-wrap items-center gap-2">
<Button
onClick={async () => {
setDlBusy(true);
setMsg("📄 소스 코드 다운로드 진행 중입니다...");
const r = await downloadFile("/api/source", "namayandeeelmi-source.zip");
setDlBusy(false);
setMsg(r);
}}
disabled={dlBusy}
>

```



```
{dlBusy ? "❌ ❌ ❌ ❌ ❌ ❌..." : "📄 📄 📄 📄 📄 📄 (ZIP)"}
</Button>
<Button
 variant="ghost"
 onClick={async () => {
 const r = await fetch("/api/source?list=1", { cache: "no-store" });
 if (!r.ok) return setMsg("❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌");
 const d = await r.json();
 setSrcInfo({ count: d.count });
 setMsg(`📄 ${d.count} ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌`);
 }}
/>
❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌
</Button>
{srcInfo ? <Badge tone="green">{toPersianDigits(srcInfo.count)} ❌ ❌ ❌ </Badge> : null}
</div>
<p className="mt-2 text-[11px] text-slate-500">
❌ ❌ ❌: ❌ ❌ ❌ ❌ ❌ ❌ node_modules ❌ .next ❌ ❌ ❌ ❌
.env ❌.
</p>
</Card>

<Card>
<h3 className="mb-2 text-sm font-bold text-slate-700">❌ ❌ ❌ ❌ ❌</h3>
<div className="grid grid-cols-1 gap-2 text-xs sm:grid-cols-3">
 <div className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
❌ ❌ ❌ ❌: <b className="text-teal-700">{info?.node ?? "—"}
</div>
 <div className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
❌ ❌ ❌: <b dir="ltr">{info?.url || window.location.origin}
</div>
 <div className={`rounded-xl p-3 ring-1 ${info?.hasSecret ? "bg-emerald-50 ring-emerald-200" : "bg-rose-50 ring
-rose-200"}}`>
❌ ❌ ❌ ❌ ❌ ❌: {info?.hasSecret ? "❌ ❌ ❌ ❌ ❌ ❌": "❌ ❌ ❌ ❌ ❌ ❌ (SYNC_SECRET)"}
</div>
</div>
 {info?.tables?.length ? (
 <p className="mt-2 text-[11px] text-slate-500">
❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌: {info.tables.join("❌ ❌")}
</p>
) : null}
 {speeds.length ? (
 <div className="mt-2 flex flex-wrap gap-2">
 {speeds.map((s) => (
 <span
 key={s.url}
 className={`rounded-lg px-2 py-1 text-[11px] font-bold ring-1 ${
 s.ms === null ? "bg-rose-50 text-rose-700 ring-rose-200" : "bg-emerald-50 text-emerald-700 ring-emeral
d-200"
 }`}
 dir="ltr"
 >
 {s.url} — {s.ms === null ? "❌ ❌ ❌ ❌ ❌ ❌": `${toPersianDigits(s.ms)}ms`}

))}
 </div>
) : null}
 <p className="mt-2 text-[10px] text-slate-400" dir="ltr">
endpoints: {orderedEndpoints().join(", ")}
</p>
</Card>

<Card>
<h3 className="mb-2 text-sm font-bold text-slate-700">❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌</h3>
{peers.length === 0 ? (
 <Alert kind="info">
❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ <b dir="ltr">SYNC_PEERS ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌.
</Alert>
) : null}

<div className="space-y-2">
 {peers.map((p) => (
 <div key={p.id} className="rounded-xl bg-slate-50 p-3 ring-1 ring-slate-200">
 <div className="flex flex-wrap items-center gap-2">
 {p.peer}
 <Badge tone={p.enabled ? "green": "amber"}>{p.enabled ? "❌ ❌ ❌": "❌ ❌ ❌ ❌ ❌ ❌"}</Badge>

 {p.peerUrl}

 </div>
 <div className="mr-auto flex flex-wrap gap-1">
 <button
 onClick={() => patch({ id: p.id, enabled: !p.enabled })}
 className="rounded-lg bg-slate-200 px-2 py-1 text-[11px] font-bold text-slate-700"
 >
 {p.enabled ? "❌ ❌ ❌ ❌ ❌ ❌": "❌ ❌ ❌ ❌"}
 </button>
 </div>
 </div>
))}
</div>
```

```

 </button>
 <button
 onClick={async () => {
 if (
 !(await confirm({
 title: "□□□□□□□□□□ □□□□",
 message: "□□□□□□□□ □□ □□ □□ □□□□□□ □□ □□□□ □□□□□□ □ □□□□□ □□□□□□",
 confirmText: "□□□□",
 danger: true,
 }))
)
 return;
 patch({ id: p.id, resetCursor: true });
 }}
 className="rounded-lg bg-amber-100 px-2 py-1 text-[11px] font-bold text-amber-700"
 >
 □□□□□□□□□□ □□□□
 </button>
 </div>
 </div>

 <div className="mt-2 grid grid-cols-3 gap-2 text-center text-[11px]">
 <div className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
 □□□□□□□□□□ <b className="text-teal-700">{toPersianDigits(p.pulled)}
 </div>
 <div className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
 □□□□□□□□□□ <b className="text-sky-700">{toPersianDigits(p.pushed)}
 </div>
 <div className="rounded-lg bg-white py-1 ring-1 ring-slate-200">
 □□□□□□□□ <b className="text-slate-500">{toPersianDigits(p.conflicts)}
 </div>
 </div>

 <div className="mt-2 text-[11px]">

 {p.lastStatus || "□□□□ □□□□ □□□□"}

 {p.lastSyncAt ? {tehranDateTime(p.lastSyncAt)} : null}
 </div>
 {p.lastError ? <div className="mt-1 rounded-lg bg-rose-50 px-2 py-1 text-[10px] text-rose-700">{p.lastError}</div> : null}
 </div>
 </div>
 </div>
 </div>

 <div className="mt-3 grid grid-cols-1 gap-2 sm:grid-cols-3">
 <Field label="□□□ □□□□ □□□□">
 <Input value={form.peer} onChange={(e) => setForm({ ...form, peer: e.target.value })} placeholder="ndcohub" />
 </Field>
 <Field label="□□□□ □□□□ □□□□">
 <Input
 value={form.peerUrl}
 onChange={(e) => setForm({ ...form, peerUrl: e.target.value })}
 placeholder="https://ndcohub.ir"
 className="text-left"
 />
 </Field>
 <div className="flex items-end">
 <Button onClick={addPeer} className="w-full">
 □ □□□□□ □□□□
 </Button>
 </div>
 </div>
</Card>

<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□ □□□□□ □□□□□□□□□□</h3>
 <div className="max-h-72 space-y-1 overflow-y-auto text-[11px]">
 {logs.map((l) => (
 <div key={l.id} className={`rounded-lg px-2 py-1.5 ${l.ok ? "bg-slate-50" : "bg-rose-50"}`}>
 {l.ok ? "□" : "□"}{" "}>
 {l.peer} — {l.direction === "pull" ? "□□□□□□" : l.direction === "push" ? "□□□□□□" : "□□□□□□"}
 {l.tableName ? ` | ${l.tableName}` : ""}
 {l.applied || l.skipped ? ` | □□□□□ ${toPersianDigits(l.applied)} / □□ ${toPersianDigits(l.skipped)}` : ""}
 </div>
 {l.detail ? <div className="text-slate-500">{l.detail}</div> : null}
 {tehranDateTime(l.createdAt)}
 </div>
 </div>
 </div>
 <div>
 {logs.length === 0 ? <p className="text-slate-400">□□□□ □□□□□□□□□□ □□□□ □□□□ □□□□</p> : null}
 </div>
</Card>

<Card>

```

```

<h3 className="mb-2 text-sm font-bold text-slate-700">🔒 🔒🔒🔒🔒 🔒🔒🔒🔒</h3>
<ol className="list-inside list-decimal space-y-1 text-[11px] leading-6 text-slate-600">

 🔒🔒 🔒 🔒 🔒🔒🔒 🔒🔒🔒🔒 <code dir="ltr">SYNC_SECRET</code> 🔒 🔒 🔒 🔒🔒🔒 🔒🔒🔒 🔒🔒🔒.

 🔒🔒 Render: <code dir="ltr">NODE_NAME=render</code> 🔒{" "}
 <code dir="ltr">SYNC_PEERS=ndcohub|https://ndcohub.ir</code>

 🔒🔒 NdcoHub: <code dir="ltr">NODE_NAME=ndcohub</code> 🔒{" "}
 <code dir="ltr">SYNC_PEERS=render|https://namayandeelmi-javad.onrender.com</code>

 🔒🔒 🔒 🔒: <code dir="ltr">NEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandeelmi-javad.onrender.com
</code>

 🔒🔒🔒🔒🔒🔒 🔒🔒🔒🔒🔒: 🔒 🔒 cron-job.org 🔒🔒🔒🔒{" "}
 <code dir="ltr">/api/sync/run?key=SYNC_SECRET</code> 🔒 🔒 🔒 🔒🔒🔒🔒 🔒🔒🔒🔒🔒 🔒🔒.

</Card>
</div>
);
}

@@ src/app/admin/targets/page.tsx typescript
"use client";

import { useCallback, useEffect, useMemo, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import { useConfirm } from "@components/Confirm";
import { DEFAULT_PRODUCTS, type ProductConfig } from "@lib/defaults";
import { JALALI_MONTHS, toPersianDigits, todayJalali } from "@lib/jalali";

type User = { id: number; fullName: string; role: string; active: boolean };
type Progress = {
 productKey: string;
 productLabel: string;
 quantity: number;
 priceDistributor: number;
 pricePharmacy: number;
 sold: number;
 bonus: number;
 remaining: number;
 percent: number;
};

const money = (n: number) => toPersianDigits(Math.round(n).toLocaleString("en-US"));
const nowYear = () => Number(todayJalali().slice(0, 4));
const nowMonth = () => Number(todayJalali().slice(5, 7));

export default function TargetsPage() {
 const [users, setUsers] = useState<User[]>([]);
 const [products, setProducts] = useState<ProductConfig[]>(DEFAULT_PRODUCTS);
 const [userId, setUserId] = useState<number | null>(null);

 // 🔒 🔒 🔒 🔒
 const [year, setYear] = useState<number>(nowYear());
 const [month, setMonth] = useState<number>(nowMonth());
 const [years, setYears] = useState<number[]>([]);
 const [newYear, setNewYear] = useState("");

 const [rows, setRows] = useState<Progress[]>([]);
 const [msg, setMsg] = useState("");
 const [busy, setBusy] = useState(false);
 const confirm = useConfirm();

 const period = `${year}/${String(month).padStart(2, "0")}`;

 /* ----- 🔒🔒🔒🔒🔒 🔒🔒🔒🔒🔒 🔒🔒🔒🔒 🔒 🔒🔒🔒 ----- */
 const loadBase = useCallback(async () => {
 const res = await fetch("/api/users", { cache: "no-store" });
 if (res.ok) {
 const list: User[] = (await res.json()).rows ?? [];
 setUsers(list);
 setUserId((cur) => cur ?? list.find((u) => u.role === "rep")?.id ?? list[0]?.id ?? null);
 }
 const pr = await fetch("/api/settings?key=products", { cache: "no-store" });
 if (pr.ok) {
 const v = (await pr.json()).value as ProductConfig[] | null;
 if (Array.isArray(v) && v.length) setProducts(v.filter((p) => p.enabled !== false));
 }
 }, []);
 // 🔒🔒🔒🔒 🔒🔒🔒🔒🔒🔒 🔒 🔒🔒🔒🔒🔒 🔒🔒🔒

```

```

const yr = await fetch("/api/options?category=year", { cache: "no-store" });
const saved: number[] = yr.ok
 ? ((await yr.json()).rows ?? []).map((r: { value: string }) => Number(r.value)).filter(Boolean)
 : [];
const base = [nowYear() - 1, nowYear(), nowYear() + 1];
setYears([...new Set([...base, ...saved])].sort((a, b) => a - b));
}, []);

const loadTargets = useCallback(async () => {
 if (!userId) return;
 const res = await fetch(`/api/targets?userId=${userId}&period=${encodeURIComponent(period)}`, { cache: "no-store" });
;
 if (res.ok) setRows((await res.json()).progress ?? []);
}, [userId, period]);

useEffect(() => {
 loadBase();
}, [loadBase]);
useEffect(() => {
 loadTargets();
}, [loadTargets]);

/** ۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰ ۰۰۰۰ — ۰۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰۰ ۰۰۰۰۰۰ ۰۰۰۰۰۰ */
const setField = (key: string, field: "quantity" | "priceDistributor" | "pricePharmacy", value: number) =>
 setRows(p => p.map(r => (r.productKey === key ? { ...r, [field]: value } : r)));

const addYear = async () => {
 const y = Number(newYear());
 if (ly || y < 1300 || y > 1500) return setMsg("۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰ (۰۰۰۰۰ ۰۰۰۰)");
 if (!(await confirm({ title: "۰۰۰۰۰۰ ۰۰۰", message: `۰۰۰ ${toPersianDigits(y)} ۰۰۰۰۰ ۰۰۰۰`, confirmText: "۰۰۰۰۰۰" })))
 return;
 const res = await fetch("/api/options", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ category: "year", value: String(y) }),
 });
 if (res.ok) {
 setYears((p) => [...new Set([...p, y])].sort((a, b) => a - b));
 setYear(y);
 setNewYear("");
 setMsg("۰ ۰۰۰ ${toPersianDigits(y)} ۰۰۰۰۰ ۰۰");
 } else setMsg("۰ ۰۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰۰ ۰۰۰");
};

const save = async () => {
 if (!userId) return;
 const rep = users.find((u) => u.id === userId);
 if (
 !(await confirm({
 title: "۰۰۰۰۰۰ ۰۰۰۰۰",
 message: `۰۰۰۰۰ «${rep?.fullName}» ۰۰۰۰ ${JALALI_MONTHS[month - 1]} ${toPersianDigits(year)} ۰۰۰۰۰ ۰۰۰۰`,
 confirmText: "۰۰۰۰۰",
 })))
)
 return;
 setBusy(true);
 const res = await fetch("/api/targets", {
 method: "PUT",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({
 userId,
 period,
 items: rows.map((r) => ({
 productKey: r.productKey,
 quantity: r.quantity,
 priceDistributor: r.priceDistributor,
 pricePharmacy: r.pricePharmacy,
 })),
 }),
 });
 setBusy(false);
 setMsg(res.ok ? "۰ ۰۰۰۰۰ ۰۰۰ ۰۰۰۰۰۰۰ ۰۰۰۰۰ ۰۰ ۰ ۰۰۰۰۰۰۰۰ ۰۰ ۰۰۰ ۰۰۰۰۰ ۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰": "۰ ۰۰۰ ۰۰ ۰۰۰۰۰");
 loadTargets();
};

/** ۰۰۰ ۰۰۰۰۰ ۰۰۰۰ ۰۰۰ ۰۰۰۰ ۰۰۰۰ ۰۰۰۰۰۰۰ */
const copyPrev = async () => {
 if (!userId) return;
 let pm = month - 1;
 let py = year;
 if (pm === 0) {
 pm = 12;
 py -= 1;
 }
}

```

```
const prev = `${py}/${String(pm).padStart(2, "0")}`;
const res = await fetch(`/api/targets?userId=${userId}&period=${encodeURIComponent(prev)}`, { cache: "no-store" });
if (!res.ok) return setMsg("❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌");
const d = await res.json();
const prevRows: Progress[] = d.progress ?? [];
setRows((cur) => {
 cur.map((r) => {
 const p = prevRows.find((x) => x.productKey === r.productKey);
 return p ? { ...r, quantity: p.quantity, priceDistributor: p.priceDistributor, pricePharmacy: p.pricePharmacy } : r;
 }),
);
setMsg(`❌ ❌ ❌ ❌ ❌ ${JALALI_MONTHS[pm - 1]} ${toPersianDigits(py)} ❌ ❌ — ❌ ❌ ❌ ❌ «❌ ❌ ❌ ❌» ❌ ❌ ❌ ❌`);
};

/* ----- ❌ ❌ ❌ ❌ ❌ ❌ ❌ ❌ ----- */
const totals = useMemo(
 () => rows.reduce(
 (a, r) => ({
 quantity: a.quantity + r.quantity,
 sold: a.sold + r.sold,
 remaining: a.remaining + Math.max(0, r.quantity - r.sold),
 valueDistributor: a.valueDistributor + r.quantity * r.priceDistributor,
 valuePharmacy: a.valuePharmacy + r.quantity * r.pricePharmacy,
 soldValueDistributor: a.soldValueDistributor + r.sold * r.priceDistributor,
 soldValuePharmacy: a.soldValuePharmacy + r.sold * r.pricePharmacy,
 }),
 {
 quantity: 0,
 sold: 0,
 remaining: 0,
 valueDistributor: 0,
 valuePharmacy: 0,
 soldValueDistributor: 0,
 soldValuePharmacy: 0,
 },
), [rows],
);
const pct = totals.quantity > 0 ? Math.round((totals.sold / totals.quantity) * 100) : 0;
const rep = users.find((u) => u.id === userId);

return (

```

```

 <option key={m} value={i + 1}>
 {m}
 </option>
 </select>
 </Field>

<Field label="تعداد فروش را وارد کنید" hint="تعداد را وارد کنید">
 <div className="flex gap-1">
 <Input
 inputMode="numeric"
 value={newYear}
 onChange={(e) => setNewYear(e.target.value.replace(/\D/g, "").slice(0, 4))}
 placeholder="1406"
 className="text-center"
 />
 <Button variant="soft" onClick={addYear}>
 افزودن
 </Button>
 </div>
</Field>
</div>

<div className="mt-3 flex flex-wrap items-center gap-2">
 <Badge tone="slate">
 {rep?.fullName ?? "—"} | {JALALI_MONTHS[month - 1]} {toPersianDigits(year)}
 </Badge>
 <Button onClick={save} disabled={busy}>
 ذخیره
 </Button>
 <Button variant="ghost" onClick={copyPrev}>
 کپی
 </Button>
</div>

<div className="mt-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
 {[
 ["تعداد کل", toPersianDigits(totals.quantity), "bg-slate-50"],
 ["تعداد فروخته شده", toPersianDigits(totals.sold), "bg-teal-50"],
 ["تعداد باقی مانده", toPersianDigits(totals.remaining), "bg-amber-50"],
 ["درصد فروش", `${toPersianDigits(pct)}٪`, pct >= 100 ? "bg-emerald-50" : "bg-sky-50"],
]}.map(([l, v, c]) => (
 <div key={l} className={`rounded-xl p-3 text-center ring-1 ring-slate-200 ${c}`}>
 <div className="text-lg font-black text-teal-700">{v}</div>
 <div className="text-[10px] font-bold text-slate-500">{l}</div>
 </div>
))
</div>
</Card>

<Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">
 خلاصه عملکرد فروش
 </h3>
 <div className="scroll-x">
 <table className="w-full min-w-[980px] text-right text-xs">
 <thead>
 <tr className="bg-slate-100 text-slate-600">
 <th className="px-2 py-2">ردیف</th>
 <th className="px-2 py-2">نام محصول</th>
 <th className="px-2 py-2">تعداد کل</th>
 <th className="px-2 py-2">تعداد فروخته شده</th>
 <th className="px-2 py-2">تعداد باقی مانده</th>
 <th className="px-2 py-2 text-center">درصد فروش</th>
 <th className="px-2 py-2 text-center">وضعیت</th>
 </tr>
 </thead>
 <tbody>
 {rows.map((r) => {
 const valD = r.quantity * r.priceDistributor;
 const valP = r.quantity * r.pricePharmacy;
 const remaining = Math.max(0, r.quantity - r.sold);
 const percent = r.quantity > 0 ? Math.round((r.sold / r.quantity) * 100) : 0;
 return (
 <tr key={r.productKey} className="border-b border-slate-100">
 <td className="whitespace-nowrap px-2 py-2 font-bold text-slate-800">{r.productLabel}</td>
 <td className="px-2 py-2">
 <Input
 inputMode="numeric"
 value={r.quantity} || ""
 onChange={(e) => setField(r.productKey, "quantity", Number(e.target.value.replace(/\D/g, ""))) || 0}
 className="w-24 px-2 py-1 text-center text-xs"
 />
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>
</Card>

```

```

 placeholder="0"
 />
 </td>
 <td className="px-2 py-2">
 <Input
 inputMode="numeric"
 value={r.priceDistributor || ""}
 onChange={(e) =>
 setField(r.productKey, "priceDistributor", Number(e.target.value.replace(/\D/g, "")) || 0)
 }
 className="w-32 px-2 py-1 text-center text-xs"
 placeholder="0"
 />
 </td>
 <td className="px-2 py-2">
 <Input
 inputMode="numeric"
 value={r.pricePharmacy || ""}
 onChange={(e) =>
 setField(r.productKey, "pricePharmacy", Number(e.target.value.replace(/\D/g, "")) || 0)
 }
 className="w-32 px-2 py-1 text-center text-xs"
 placeholder="0"
 />
 </td>
 </tr>
 <tr>
 <td colspan="5">
 <div className="px-2 py-2 text-center font-black text-slate-700">
 {money(valD)}
 </td>
 </tr>
 <tr>
 <td colspan="5">
 <div className="px-2 py-2 text-center font-black">
 {money(valP)}
 </div>
 </td>
 </tr>
 <tr>
 <td colspan="5">
 <div className="px-2 py-2 text-center font-bold text-teal-700">{toPersianDigits(r.sold)}</td>
 </tr>
 <tr>
 <td colspan="5">
 <div className="px-2 py-2 text-center font-bold text-rose-700">{toPersianDigits(remaining)}</td>
 </tr>
 <tr>
 <td colspan="5">
 <div className="flex items-center justify-center gap-1">
 <div className="h-1.5 w-14 overflow-hidden rounded bg-slate-200">
 <div
 className={h-full ${percent >= 100 ? "bg-emerald-500" : "bg-teal-500"}}
 style={{ width: `${Math.min(100, percent)}%` }}
 />
 </div>
 </div>
 {toPersianDigits(percent)}%
 </div>
 </tr>
 </tr>
 </tr>
 </tr>
 </tbody>
</table>
</div>

<div className="mt-3 grid grid-cols-1 gap-2 sm:grid-cols-2">
 <div className="rounded-xl bg-slate-50 p-3 text-xs ring-1 ring-slate-200">
 <div>
 {money(totals.soldValueDistributor)}
 </div>
 <div>
 {money(totals.soldValuePharmacy)}
 </div>
 </div>
</div>
</Card>
</div>
);
}

@@ src/app/admin/trips/page.tsx typescript
"use client";

import { useCallback, useEffect, useMemo, useState } from "react";
import dynamic from "next/dynamic";
import { Alert, Badge, Button, Card, Input, SectionTitle } from "@components/ui";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";

```

```

import { formatDistance, formatDuration } from "@lib/tracker";
import { tehranDateTime, tehranTime, toPersianDigits } from "@lib/jalali";

const MapBox = dynamic(() => import("@/components/MapBox"), { ssr: false });

type Trip = {
 id: number;
 userId: number;
 repName: string;
 dateShamsi: string;
 status: string;
 startedAt: string;
 endedAt: string | null;
 points: number;
 distanceM?: number;
 stopSeconds?: number;
};

type Point = {
 id: number;
 lat: number;
 lng: number;
 kind: string;
 note: string;
 stopSeconds: number;
 gpsOn: boolean;
 speed: number | null;
 recordedAt: string;
};

type Place = { lat: number; lng: number; label: string; color: string };

export default function AdminTrips() {
 const [trips, setTrips] = useState<Trip[]>([]);
 const [sel, setSel] = useState<number | null>(null);
 const [points, setPoints] = useState<Point[]>([]);
 const [onlyActive, setOnlyActive] = useState(false);
 const [rep, setRep] = useState("");
 const [q, setQ] = useState("");
 const [places, setPlaces] = useState<Place[]>([]);
 const [dl, setDl] = useState("");

 /* ----- □□□□□□□□□□ □□□□□ □ □□□□ □□□□ □□ □□□□ ----- */
 useEffect(() => {
 (async () => {
 const get = async (u: string) => {
 const r = await fetch(u, { cache: "no-store" }).catch(() => null);
 return r?.ok ? ((await r.json()).rows ?? []) : [];
 };
 const [ph, dr, hm] = await Promise.all([
 get("/api/records/pharmacies?limit=500"),
 get("/api/records/doctors?limit=500"),
 get("/api/homes"),
]);
 const out: Place[] = [];
 for (const p of ph) if (p.lat && p.lng) out.push({ lat: p.lat, lng: p.lng, label: `□ ${p.name}`, color: "#0f766e" });
 })();
 for (const d of dr) if (d.lat && d.lng) out.push({ lat: d.lat, lng: d.lng, label: `□ ${d.name}`, color: "#0369a1" });
 for (const h of hm) if (h.lat && h.lng) out.push({ lat: h.lat, lng: h.lng, label: `□ ${h.title} (${h.repName})`, color: "#7c3aed" });
 setPlaces(out);
 })();
 }, []);

 const loadTrips = useCallback(async () => {
 const res = await fetch(`/api/trips${onlyActive ? "?active=1" : ""}`, { cache: "no-store" });
 if (res.ok) setTrips((await res.json()).rows ?? []);
 }, [onlyActive]);

 const loadPoints = useCallback(async (id: number) => {
 const res = await fetch(`/api/trips/points?tripId=${id}`, { cache: "no-store" });
 if (res.ok) setPoints((await res.json()).points ?? []);
 }, []);

 useLive(loadTrips, 15000);

 useEffect(() => {
 if (sel === null) return;
 loadPoints(sel);
 const t = setInterval(() => loadPoints(sel), 20000);
 return () => clearInterval(t);
 }, [sel, loadPoints]);

 /* ----- □□□□□□□□ □□ □□□□ □□□□□□ ----- */
 const reps = useMemo(() => [...new Set(trips.map((t) => t.repName))].sort(), [trips]);

```



```

const filtered = useMemo(
 () =>
 trips.filter(
 (t) => (!rep || t.repName === rep) && (!q || `${t.repName} ${t.dateShamsi}`.toLowerCase().includes(q.toLowerCase())
),
 [trips, rep, q],
);

const grouped = useMemo(() => {
 const m = new Map<string, Trip[]>();
 for (const t of filtered) {
 if (!m.has(t.repName)) m.set(t.repName, []);
 m.get(t.repName).push(t);
 }
 return [...m.entries()].sort((a, b) => a[0].localeCompare(b[0]));
}, [filtered]);

const selected = trips.find((t) => t.id === sel);
const stops = points.filter((p) => p.kind === "pause");
const totalStop = points.reduce((a, p) => a + (p.stopSeconds ?? 0), 0);

/**
 * حساب مسافت بین دو نقطه بر اساس مختصات جغرافیایی
 */
const distance = useMemo(() => {
 if (selected?.distanceM) return selected.distanceM;
 let d = 0;
 for (let i = 1; i < points.length; i++) {
 const a = points[i - 1];
 const b = points[i];
 const R = 6371000;
 const dLat = ((b.lat - a.lat) * Math.PI) / 180;
 const dLng = ((b.lng - a.lng) * Math.PI) / 180;
 const s =
 Math.sin(dLat / 2) ** 2 +
 Math.cos(a.lat * Math.PI / 180) * Math.cos(b.lat * Math.PI / 180) * Math.sin(dLng / 2) ** 2;
 const seg = 2 * R * Math.asin(Math.sqrt(s));
 if (seg > 5 && seg < 5000) d += seg;
 }
 return d;
}, [points, selected]);

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="📍">نقشه مسیر و توقف‌ها</SectionTitle>
 <div className="flex flex-wrap gap-2">
 <Button variant={onlyActive ? "primary" : "ghost"} onClick={() => setOnlyActive((v) => !v)}>
 {onlyActive ? "فیلتر توقف‌ها" : "نمایش همه توقف‌ها"}
 </Button>
 <Button variant="success" onClick={async () => setDl(await downloadFile("/api/export?type=trips", "trips.xls"))}>
 دانلود فایل
 </Button>
 </div>
 </div>
 {dl ? <Alert kind={dl.startsWith("error") ? "error" : "success"}>{dl}</Alert> : null}

 <Card>
 <div className="flex flex-wrap items-center gap-2">
 <select
 value={rep}
 onChange={(e) => {
 setRep(e.target.value);
 setSel(null);
 }}
 className="rounded-xl border border-slate-300 px-3 py-2 text-sm">
 <option value="">همه نمایندگان (toPersianDigits(reps.length))</option>
 {reps.map((r) => (
 <option key={r} value={r}>
 {r}
 </option>
))}
 </select>
 <Input placeholder="🔍 جستجو..." value={q} onChange={(e) => setQ(e.target.value)} className="max-w-[200px]" />
 <Badge tone="slate">{toPersianDigits(filtered.length)} نقطه</Badge>

 ۰ کیلومتر · ۰ کیلومتر · ۰ کیلومتر · ۰ کیلومتر

 </div>
 </Card>

 <div className="grid grid-cols-1 gap-3 lg:grid-cols-3">
 <Card className="lg:col-span-1">
 <div className="max-h-[520px] space-y-3 overflow-y-auto">
 {grouped.map(([repName, list]) => {

```

```

const totalDist = list.reduce((a, t) => a + (t.distanceM ?? 0), 0);
return (
 <div key={repName}>
 <div className="mb-1 flex flex-wrap items-center gap-2 rounded-lg bg-slate-100 px-2 py-1.5">
 {repName}
 <Badge tone="slate">{toPersianDigits(list.length)} □□□</Badge>
 {totalDist > 0 ? <Badge tone="green">{formatDistance(totalDist)}</Badge> : null}
 </div>
 {list.some((t) => t.status === "active") ? <Badge tone="green">□ □□□□</Badge> : null}
 <div className="space-y-1">
 {list.map((t) => (
 <button
 key={t.id}
 onClick={() => setSel(t.id)}
 className={w-full rounded-xl px-3 py-2 text-right text-xs ring-1 transition ${
 sel === t.id ? "bg-teal-50 ring-teal-300" : "bg-slate-50 ring-slate-200 hover:bg-slate-100"
 }}
 >
 <div className="flex items-center justify-between gap-1">
 {toPersianDigits(t.dateShamsi)}
 <Badge tone={t.status === "active" ? "green" : "slate"}>
 {t.status === "active" ? "□□ □□□ □□□□□" : "□□□□□ □□□□□"}
 </Badge>
 </div>
 <div className="mt-1 flex flex-wrap gap-2 text-[10px] text-slate-500">
 □□□□ {tehranTime(t.startedAt)}
 {t.endedAt ? □□□□□ {tehranTime(t.endedAt)} : null}
 {toPersianDigits(t.points ?? 0)} □□□□
 {t.distanceM ? {formatDistance(t.distanceM)} : null}
 {t.stopSeconds ? □□□□ {formatDuration(t.stopSeconds)} : null}
 </div>
 </button>
))}
 </div>
 </div>
);
}}}
</div>
{grouped.length === 0 ? <p className="py-6 text-center text-sm text-slate-400">□□□□ □□□□ □□□</p> : null}
</div>
</Card>

<Card className="lg:col-span-2">
 {selected ? (
 <>
 <div className="mb-2 flex flex-wrap items-center gap-2">
 {selected.repName}
 <Badge tone="slate">{toPersianDigits(selected.dateShamsi)}</Badge>
 <Badge tone="green">{formatDistance(distance)}</Badge>
 <Badge tone="amber">□□□□ {formatDuration(totalStop || selected.stopSeconds || 0)}</Badge>
 <Badge tone="slate">{toPersianDigits(points.length)} □□□□</Badge>
 </div>

 <MapBox
 height={380}
 path={points.map((p) => ({ lat: p.lat, lng: p.lng })))}
 points={[
 ...places,
 ...points
 .filter((p) => p.kind !== "move")
 .map((p) => ({
 lat: p.lat,
 lng: p.lng,
 label: `${
 p.kind === "start" ? "□□ □□□□" : p.kind === "end" ? "□ □□□□□" : "□ □□□□"
 } — ${tehranTime(p.recordedAt)} ${p.stopSeconds ? ` ${formatDuration(p.stopSeconds)}` : ""}`,
 color: p.kind === "pause" ? "#f59e0b" : p.kind === "end" ? "#e11d48" : "#16a34a",
 })
]}
 >
 <div className="mt-3 max-h-44 space-y-1 overflow-y-auto text-[11px]">
 {points
 .filter((p) => p.kind !== "move")
 .map((p) => (
 <div key={p.id} className="flex flex-wrap items-center gap-2 rounded-lg bg-slate-50 px-3 py-1.5">
 {p.kind === "start" ? "□□ □□□□ □□□□□" : p.kind === "end" ? "□ □□□□□ □□□" : "□ □□□□"}
 {tehranDateTime(p.recordedAt)}
 {p.stopSeconds ? <Badge tone="amber">{formatDuration(p.stopSeconds)}</Badge> : null}
 </div>
 {p.gpsOn ? <Badge tone="slate">GPS □□□□□</Badge> : null}
 {p.note ? {p.note} : null}
))}
 </div>
 </div>
 </>
) : null}
</div>
</>

```

```

) : (
 <>
 <p className="mb-2 text-center text-sm text-slate-400">
 □□ □□□ □□ □□□□□ □□□□□□ □□□□ — □□□□ □□□ □□□ □□□□□□□□□□□ □□□□□□ □ □□□□□ □□ □□□□ □□□□□□□□
 </p>
 <MapBox height={400} points={places} />
 </>
)}
 </Card>
</div>
</div>
);
}

@@ src/app/admin/users/page.tsx typescript
"use client";

import { useCallback, useEffect, useState } from "react";
import { Alert, Badge, Button, Card, Field, Input, SectionTitle } from "@components/ui";
import {
 ALL_PERMISSION_KEYS,
 PERMISSION_GROUPS,
 REP_DEFAULT_PERMISSIONS,
 SUPERVISOR_DEFAULT_PERMISSIONS,
} from "@lib/defaults";
import { tehranDateTime, toPersianDigits } from "@lib/jalali";
import { useConfirm } from "@components/Confirm";
import { useLive } from "@lib/useLive";
import { downloadFile } from "@lib/download";

type Role = { id: number; key: string; label: string; base: string; permissions: string[]; builtin: boolean };

type User = {
 id: number;
 username: string;
 fullName: string;
 phone: string;
 role: string;
 active: boolean;
 requirePhone: boolean;
 deviceId?: string;
 deviceInfo?: string;
 deviceBoundAt?: string | null;
 simMode?: string;
 permissions: string[];
 passwordPlain: string;
 lastSeenAt: string | null;
};

const blank = {
 username: "",
 password: "",
 fullName: "",
 phone: "",
 role: "rep",
 requirePhone: true,
 permissions: [...REP_DEFAULT_PERMISSIONS] as string[],
};

export default function UsersPage() {
 const [rows, setRows] = useState<User[]>([]);
 const [form, setForm] = useState<{ ...blank }>();
 const [msg, setMsg] = useState<{ kind: "error" | "success"; text: string } | null>(null);
 const [openUser, setOpenUser] = useState<number | null>(null);
 const [showAllPass, setShowAllPass] = useState(false);
 const [showPass, setShowPass] = useState<Record<number, boolean>>({});
 const [pwDraft, setPwDraft] = useState<Record<number, string>>({});
 const [busy, setBusy] = useState(false);
 const [roles, setRoles] = useState<Role[]>([]);
 const [roleForm, setRoleForm] = useState<{ label: "", key: "", base: "rep" }>();
 const [showRoles, setShowRoles] = useState(false);
 const confirm = useConfirm();

 const load = useCallback(async () => {
 const res = await fetch("/api/users", { cache: "no-store" });
 if (res.ok) {
 const d = await res.json();
 setRows(
 (d.rows ?? []).map((u: User) => ({ ...u, permissions: Array.isArray(u.permissions) ? u.permissions : [] })),
);
 } else setMsg({ kind: "error", text: "□□□□ □□ □□□□□□ □□□□□□□□ — □□□□□□ □□□□ □□□□" });
 }, []);

 const loadRoles = useCallback(async () => {
 const res = await fetch("/api/roles", { cache: "no-store" });
 }, []);

```

```

 if (res.ok) setRoles((await res.json()).rows ?? []);
 }, []);

useEffect(() => {
 loadRoles();
}, [loadRoles]);

useLive(load, 20000);

const create = async () => {
 if (!form.fullName.trim() || !form.username.trim() || !form.password.trim()) {
 setMsg({ kind: "error", text: "□□□□ □□ □□□□□ □ □□ □□□ □□□□□ □□□" });
 return;
 }
 if (
 !(await confirm({
 title: "□□□□ □□□□ □□□□",
 message: `□□□□ «${form.fullName}» □□ □□ «${roles.find((r) => r.key === form.role)?.label ?? form.role}» □□□□□`,
 confirmText: "□□□□□",
 })))
)
 return;
 setBusy(true);
 const res = await fetch("/api/users", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(form),
 });
 setBusy(false);
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setMsg({ kind: "success", text: "□ □□□□ □□□ □□□□ □□" });
 setForm({ ...blank });
 load();
 } else setMsg({ kind: "error", text: d.error ?? "□□□" });
};

const patch = async (body: Record<string, unknown>, silent = false) => {
 // □□□□□□□□ □□□□□□□□ □□□ □□□□ □□□□□□ □□□ □□□□□
 if (typeof body.id === "number") {
 setRows((prev) =>
 prev.map((u) => (u.id === body.id ? ({ ...u, ...(body as Partial<User>) } as User) : u)),
);
 }
 const res = await fetch("/api/users", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(body),
 });
 if (res.ok) {
 if (!silent) setMsg({ kind: "success", text: "□ □□□□ □□" });
 load();
 } else {
 setMsg({ kind: "error", text: "□□□ □□ □□□□□□□□□" });
 load();
 }
};

const remove = async (id: number, name: string) => {
 if (
 !(await confirm({
 title: "□□□ □□□□□",
 message: `□□□□□ «${name}» □□□□ □□□□□ □□ □□□□ □□ □□ □□□ □□□□□ □□□□.`,
 confirmText: "□□□",
 danger: true,
 })))
)
 return;
 const res = await fetch(`/api/users?id=${id}`, { method: "DELETE" });
 if (res.ok) load();
};

const togglePerm = (u: User, key: string) => {
 const has = u.permissions.includes(key);
 patch({ id: u.id, permissions: has ? u.permissions.filter((p) => p !== key) : [...u.permissions, key] }, true);
};

const setPassword = async (u: User) => {
 const pw = (pwDraft[u.id] ?? "").trim();
 if (pw.length < 4) {
 setMsg({ kind: "error", text: "□□□ □□□□ □□□□ □ □□□□□□□ □□□□" });
 return;
 }
 if (

```

```

 !(await confirm({
 title: "□□□□□ □□□ □□□□",
 message: `□□□□ □□□□ «${u.fullName}» □□ «${pw}» □□□□□ □□□□`,
 confirmText: "□□□□□ □□□",
 })))
)
 return;
 await patch({ id: u.id, password: pw });
 setPwDraft({ ...pwDraft, [u.id]: "" });
 setShowPass((s) => ({ ...s, [u.id]: true }));
};

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center justify-between gap-2">
 <SectionTitle icon="□">□□□□□□□□ □□ □□□□ □ □□□ □□□□□□</SectionTitle>
 <div className="flex gap-2">
 <Button variant={showAllPass ? "danger" : "ghost"} onClick={() => setShowAllPass((v) => !v)}>
 {showAllPass ? "□ □□□□□ □□□□ □□ □□□□□" : "□ □□□□□ □□□ □□□□□"}
 </Button>
 <Button
 variant="success"
 onClick={async () => setMsg({ kind: "success", text: await downloadFile("/api/export?type=users", "users.xls") }) }>
 >
 □□ □□□□□ □□□□
 </Button>
 </div>
 </div>
 {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}

 <Card>
 <h3 className="mb-2 text-sm font-bold text-slate-700">□□□□□ □□□□□ □□□□</h3>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-2 lg:grid-cols-5">
 <Field label="□□□□ □ □□□ □□□□□□□□ required">
 <Input value={form.fullName} onChange={(e) => setForm({ ...form, fullName: e.target.value })} />
 </Field>
 <Field label="□□□□ □□□□□□□□ required">
 <Input value={form.username} onChange={(e) => setForm({ ...form, username: e.target.value })} className="tex
t-left" />
 </Field>
 <Field label="□□□□ □□□□□" required">
 <Input value={form.password} onChange={(e) => setForm({ ...form, password: e.target.value })} className="tex
t-left" />
 </Field>
 <Field label="□□□□□□ □□□□□□ □□□□">
 <Input
 inputMode="numeric"
 value={form.phone}
 onChange={(e) => setForm({ ...form, phone: e.target.value.replace(/\D/g, "").slice(0, 11) })}
 placeholder="09xxxxxxxxx"
 />
 </Field>
 <Field label="□□□□">
 <div className="flex gap-1">
 <select
 value={form.role}
 onChange={(e) => {
 const role = e.target.value;
 const r = roles.find((x) => x.key === role);
 const perms = r?.permissions?.length
 ? r.permissions
 : r?.base === "admin"
 ? ALL_PERMISSION_KEYS
 : r?.base === "supervisor"
 ? SUPERVISOR_DEFAULT_PERMISSIONS
 : REP_DEFAULT_PERMISSIONS;
 setForm({ ...form, role, permissions: [...perms] });
 }}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 {roles.map((r) => (
 <option key={r.key} value={r.key}>
 {r.label}
 </option>
))}
 </select>
 <button
 type="button"
 onClick={() => setShowRoles((v) => !v)}
 title="□□□□□□□ □□□□□□□"
 className="shrink-0 rounded-xl bg-teal-50 px-3 text-sm font-bold text-teal-700 ring-1 ring-teal-200"
 >
 □
 </button>
 </div>
 </Field>
 </div>
 </Card>
 </div>
);

```

```

 </div>
 </Field>
</div>

<div className="mt-3 space-y-2 rounded-xl bg-slate-50 p-3">
 <div className="flex flex-wrap items-center gap-2">
□□□ □□□□□□ □□□□□ □□□□
 <button
 onClick={() => setForm({ ...form, permissions: [...ALL_PERMISSION_KEYS] })}
 className="rounded-lg bg-teal-600 px-2 py-1 text-[11px] font-bold text-white"
 >
 □□□
 </button>
 <button
 onClick={() => setForm({ ...form, permissions: [] })}
 className="rounded-lg bg-slate-300 px-2 py-1 text-[11px] font-bold text-slate-700"
 >
 □□□
 </button>
 </div>
<Badge tone="slate"><toPersianDigits(form.permissions.length)> □□□□</Badge>
</div>
{PERMISSION_GROUPS.map((g) => (
 <div key={g.key}>
 <div className="mb-1 text-[11px] font-black text-slate-500">
 {g.icon} {g.label}
 </div>
 <div className="flex flex-wrap gap-1">
 {g.items.map((it) => {
 const on = form.permissions.includes(it.key);
 return (
 <button
 key={it.key}
 onClick={() =>
 setForm({
 ...form,
 permissions: on
 ? form.permissions.filter((p) => p !== it.key)
 : [...form.permissions, it.key],
 })
 }
 className={`rounded-lg px-2 py-1 text-[11px] font-bold ${
 on ? "bg-teal-100 text-teal-700 ring-1 ring-teal-300" : "bg-white text-slate-400 ring-1 ring-sla
te-200"
 }`}
 >
 {on ? "□" : ""} {it.label}
 </button>
);
 })}
 </div>
 </div>
))}
</div>

<div className="mt-3 flex flex-wrap items-center gap-3">
 <label className="flex items-center gap-1 text-xs font-bold text-slate-600">
 <input
 type="checkbox"
 className="size-4 accent-teal-600"
 checked={form.requirePhone}
 onChange={(e) => setForm({ ...form, requirePhone: e.target.checked })}
 />
 □□□□ □□ □□□□□ □□□□□ □□□□□□ □□□□
 </label>
 <div className="flex-1" />
 <Button onClick={create} disabled={busy}>
 □ □□□□□ □□□□□
 </Button>
</div>
</Card>

{showRoles ? (
 <Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
<h3 className="text-sm font-bold text-slate-700">□ □□□□□□ □□□□□□□ □□□□□□□</h3>
 <Button variant="ghost" onClick={() => setShowRoles(false)}>
 □□□□
 </Button>
 </div>
 <div className="grid grid-cols-1 gap-3 sm:grid-cols-4">
<Field label="□□□□□ □□□" required>
 <Input
 value={roleForm.label}
 onChange={(e) => setRoleForm({ ...roleForm, label: e.target.value })}
 />
 </div>
 </Card>
) : null}

```

```

placeholder="□□□□□ □□□□□□ □□□□"
/>
</Field>
<Field label="□□□□□ □□□□□□□ (□□□□□□□)">
 <Input
 value={roleForm.key}
 onChange={(e) => setRoleForm({ ...roleForm, key: e.target.value })}
 placeholder="sales"
 className="text-left"
 />
</Field>
<Field label="□□□ □□□□ (□□□□□ □□□□□□)">
 <select
 value={roleForm.base}
 onChange={(e) => setRoleForm({ ...roleForm, base: e.target.value })}
 className="w-full rounded-xl border border-slate-300 px-3 py-2.5 text-sm"
 >
 <option value="rep">□□□□□□ □□□□□□□ (□□□□ □□ □□ □□□□□□)</option>
 <option value="supervisor">□□□□□□ □□□□□□ (□□□□ □□ □□ □□□□)</option>
 <option value="admin">□□□□□□ □□□□</option>
 </select>
</Field>
<div className="flex items-end">
 <Button
 onClick={async () => {
 if (!roleForm.label.trim()) return;
 if (
 !(await confirm({
 title: "□□□□□□ □□□ □□□□",
 message: `□□□ «${roleForm.label}» □□□□□ □□□□`,
 confirmText: "□□□□□",
 }))
)
 return;
 const res = await fetch("/api/roles", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(roleForm),
 });
 const d = await res.json().catch(() => ({}));
 if (res.ok) {
 setMsg({ kind: "success", text: "□ □□□ □□□□ □□□□□ □□" });
 setRoleForm({ label: "", key: "", base: "rep" });
 loadRoles();
 } else setMsg({ kind: "error", text: d.error ?? "□□□" });
 }}
 className="w-full"
 >
 □ □□□□□□ □□□
 </Button>
</div>
</div>

<ul className="mt-3 space-y-1">
 {roles.map((r) => (
 <li key={r.id} className="flex flex-wrap items-center gap-2 rounded-xl bg-slate-50 px-3 py-2 text-xs">
 <Input
 value={r.label}
 onChange={(e) => setRoles((p) => p.map((x) => (x.id === r.id ? { ...x, label: e.target.value } : x)))}
 className="max-w-[180px] px-2 py-1 text-xs"
 />

 {r.key}

 <select
 value={r.base}
 onChange={(e) => setRoles((p) => p.map((x) => (x.id === r.id ? { ...x, base: e.target.value } : x)))}
 className="rounded-lg border border-slate-200 px-2 py-1 text-[11px]"
 >
 <option value="rep">□□□□: □□□□□□□</option>
 <option value="supervisor">□□□□: □□□□□□□</option>
 <option value="admin">□□□□: □□□□</option>
 </select>
 {r.builtin ? <Badge tone="slate">□□□□ □□□□□</Badge> : null}

 <button
 onClick={async () => {
 if (!(await confirm({ title: "□□□□□ □□□", message: `□□□□□□□ «${r.label}» □□□□□ □□□□`, confirmText:
"□□□□□" })))
 return;
 const res = await fetch("/api/roles", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ id: r.id, label: r.label, base: r.base }),
 });
 }}
 >

```

```

 setMsg(
 res.ok
 ? { kind: "success", text: "❏ ❏❏❏❏ ❏❏" }
 : { kind: "error", text: "❏❏❏❏❏ ❏❏❏❏❏❏" },
);
 loadRoles();
 load();
 })
 className="rounded-lg bg-emerald-100 px-2 py-1 text-[11px] font-bold text-emerald-700"
 >
❏❏❏❏
 </button>
 {!r.builtin ? (
 <button
 onClick={async () => {
 if (
 !(await confirm({
 title: "❏❏❏ ❏❏❏",
 message: `❏❏❏ «${r.label}» ❏❏❏ ❏❏❏❏`,
 confirmText: "❏❏❏",
 danger: true,
)))
)
 return;
 const res = await fetch(`/api/roles?id=${r.id}`, { method: "DELETE" });
 const d = await res.json().catch(() => ({}));
 setMsg(
 res.ok ? { kind: "success", text: "❏ ❏❏❏ ❏❏" } : { kind: "error", text: d.error ?? "❏❏❏" },
);
 loadRoles();
 }}
 className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
 >
❏❏❏
 </button>
) : null}

)}}

</Card>
) : null}

{rows.length === 0 ? <Card>❏❏❏❏❏❏ ❏❏❏❏ ❏❏❏.</Card> : null}

{rows.map((u, i) => {
 const open = openUser === u.id;
 const visible = showAllPass || showPass[u.id];
 return (
 <Card key={u.id}>
 <div className="flex flex-wrap items-center gap-2">
 {toPersianDigits(i + 1)}
 {u.fullName}
 <Badge tone={u.role === "admin" ? "green" : u.role === "supervisor" ? "amber" : "slate"}>
 {roles.find((r) => r.key === u.role)?.label ?? u.role}
 </Badge>
 {!u.active ? <Badge tone="amber">❏❏❏❏❏❏</Badge> : null}

 ❏❏❏❏❏ ❏❏❏❏: {u.lastSeenAt ? tehranDateTime(u.lastSeenAt) : "—"}

 <div className="mr-auto flex flex-wrap gap-1">
 <button
 onClick={() => setOpenUser(open ? null : u.id)}
 className="rounded-lg bg-teal-50 px-2 py-1 text-[11px] font-bold text-teal-700 ring-1 ring-teal-200"
 >
❏❏ ❏❏❏ ❏❏❏❏❏❏ ({toPersianDigits(u.permissions.length)}) {open ? "▲" : "▼"}
 </button>
 <button
 onClick={() => patch({ id: u.id, active: !u.active })}
 className={`rounded-lg px-2 py-1 text-[11px] font-bold ${
 u.active ? "bg-amber-100 text-amber-700" : "bg-emerald-100 text-emerald-700"
 }`}
 >
 {u.active ? "❏❏❏❏❏❏❏ ❏❏" : "❏❏❏❏❏❏❏"}
 </button>
 <button
 onClick={() => remove(u.id, u.fullName)}
 className="rounded-lg bg-rose-100 px-2 py-1 text-[11px] font-bold text-rose-700"
 >
❏❏❏
 </button>
 </div>
 </div>
 </div>
 <div className="mt-3 grid grid-cols-1 gap-2 sm:grid-cols-2 lg:grid-cols-4">

```



```

<label className="block">
□□□ □□□□□
 <input
 value={u.username}
 onChange={(e) => setRows((p) => p.map((x) => (x.id === u.id ? { ...x, username: e.target.value } : x))
)}

 onBlur={() => patch({ id: u.id, username: u.username }, true)}
 className="text-left"
 />
</label>

<label className="block">
□□□ □□□□
 <div className="relative">
 <input
 type={visible ? "text" : "password"}
 readOnly
 value={u.passwordPlain || ""}
 placeholder="□□□ □□□□ — □□□ □□□□ □□□□□□□"
 dir="ltr"
 className="w-full rounded-xl border border-slate-300 bg-slate-50 py-2.5 pr-3 pl-12 text-left font-mo
no text-sm text-slate-800"
 onFocus={(e) => e.currentTarget.select()}
 />
 <button
 type="button"
 onClick={() => setShowPass((s) => ({ ...s, [u.id]: !s[u.id] })))}
 className="absolute left-2 top-1/2 -translate-y-1/2 rounded-lg bg-white px-2 py-1 text-xs ring-1 rin
g-slate-200"
 title={visible ? "□□□□□ □□□□ □□□" : "□□□□□ □□□"}
 >
 {visible ? "□" : "□"}
 </button>
</div>
 {u.passwordPlain ? (
 <button
 onClick={() => navigator.clipboard?.writeText(u.passwordPlain)}
 className="mt-1 text-[10px] font-bold text-teal-700"
 >
 □ □□□ □□□
 </button>
) : null}
</label>

<label className="block">
□□□□□ □□□ □□□□
 <div className="flex gap-1">
 <input
 type={visible ? "text" : "password"}
 value={pwDraft[u.id] ?? ""}
 onChange={(e) => setPwDraft({ ...pwDraft, [u.id]: e.target.value })}
 placeholder="□□□ □□□□"
 className="text-left"
 />
 <Button variant="soft" onClick={() => setPassword(u)}>
 □□□
 </Button>
</div>
</label>

<label className="block">
□□□□□ □□□□□ □□□□
 <input
 inputMode="numeric"
 value={u.phone}
 onChange={(e) =>
 setRows((p) =>
 p.map((x) => (x.id === u.id ? { ...x, phone: e.target.value.replace(/\D/g, "").slice(0, 11) } : x)
),
)
 }
 onBlur={() => patch({ id: u.id, phone: u.phone }, true)}
 />
</label>
</div>

<div className="mt-2 flex flex-wrap items-center gap-2">
□□□□□ □□□□□□□□
 <select
 value={u.simMode ?? "device"}
 onChange={(e) => patch({ id: u.id, simMode: e.target.value })}
 className="rounded-lg border border-slate-200 px-2 py-1 text-[11px] font-bold"
 title="□□□ □□□□□□□□ □□□□□ □□□□□□□□"
 >
 <option value="off">□□□□ □□□□□</option>

```

```

<option value="phone">□□□□□ □□□□□</option>
<option value="device">□□□ □□□□ (□□□□□□□□□□)</option>
<option value="otp">□□□□□ □□□□□□ (□□□□□□□□□□)</option>
</select>
<button
 onClick={() => patch({ id: u.id, requirePhone: !u.requirePhone })}
 className={`rounded-lg px-3 py-1 text-[11px] font-bold ${
 u.requirePhone ? "bg-emerald-100 text-emerald-700" : "bg-slate-200 text-slate-600"
 }`}
>
{u.requirePhone ? "□ □□□□□ □□□□□□" : "□ □□□□ □□□□□"}
</button>

□□□□ □□□□
{u.deviceId ? (

 □ {u.deviceInfo || "□□□□"}
 {u.deviceBoundAt ? ` — ${tehranDateTime(u.deviceBoundAt)}` : ""}

 <button
 onClick={async () => {
 if (
 !(await confirm({
 title: "□□□□□□□□ □□□□",
 message: `□□□□□ □□□□ «${u.fullName}» □□ □□□□ □□□□ □□□□ □□ □□ □□ □□□□□□□□ □□ □□ □□□□ □□□□ □
□□□ □□□.`,
 confirmText: "□□□□ □□",
 danger: true,
 }))
)
 return;
 patch({ id: u.id, releaseDevice: true });
 }}
 className="rounded-lg bg-amber-100 px-2 py-1 text-[11px] font-bold text-amber-700"
 >
 □□□□□□□□ □□□□
 </button>
</>
) : (

 □□□□ — □□□□□ □□□□□ □□□□ □□ □□ □□□□□□

)}

□□□
<select
 value={u.role}
 onChange={(e) => patch({ id: u.id, role: e.target.value })}
 className="rounded-lg border border-slate-200 px-2 py-1 text-[11px]"
>
 {roles.map((r) => (
 <option key={r.key} value={r.key}>
 {r.label}
 </option>
))}
</select>
</div>

{open ? (
 <div className="fade-in mt-3 space-y-3 rounded-2xl bg-slate-50 p-3 ring-1 ring-slate-200">
 <div className="flex flex-wrap gap-2">
 <Button onClick={() => patch({ id: u.id, permissions: ALL_PERMISSION_KEYS }, true)}>
 □ □□□□□□ □□□□
 </Button>
 <Button variant="soft" onClick={() => patch({ id: u.id, permissions: SUPERVISOR_DEFAULT_PERMISSIONS },
true)}>
 □□□□□□□ □□□□□□
 </Button>
 <Button variant="soft" onClick={() => patch({ id: u.id, permissions: REP_DEFAULT_PERMISSIONS }, true)}>
 >
 □□□□□□□ □□□□□□□
 </Button>
 <Button variant="danger" onClick={() => patch({ id: u.id, permissions: [] }, true)}>
 □□□ □□□
 </Button>
 </div>

 {PERMISSION_GROUPS.map((g) => {
 const groupKeys = g.items.map((it) => it.key);
 const allOn = groupKeys.every((k) => u.permissions.includes(k));
 return (
 <div key={g.key} className="rounded-xl bg-white p-3 ring-1 ring-slate-200">
 <div className="mb-2 flex items-center justify-between">


```



```

 db
 .select({ uid: trips.userId, c: count() })
 .from(trips)
 .where(eq(trips.status, "active"))
 .groupBy(trips.userId),
 db.select({ c: count() }).from(leaves).where(eq(leaves.managerStatus, "pending")),
]), "activity:load");

const toMap = (rows: { uid: number; c: number }[]) => {
 const m = new Map<number, number>();
 for (const r of rows) m.set(r.uid, Number(r.c));
 return m;
};
const mPh = toMap(phG);
const mDr = toMap(drG);
const mOr = toMap(orG);
const mTr = toMap(trG);
const mAc = toMap(activeG);

const lastByUser = new Map<number, (typeof logs)[number]>();
for (const l of logs) {
 if (l.userId !== null && !lastByUser.has(l.userId)) lastByUser.set(l.userId, l);
}

const reps = userRows.map((u) => {
 const last = lastByUser.get(u.id);
 return {
 ...u,
 pharmacies: mPh.get(u.id) ?? 0,
 doctors: mDr.get(u.id) ?? 0,
 orders: mOr.get(u.id) ?? 0,
 trips: mTr.get(u.id) ?? 0,
 activeTrip: mAc.get(u.id) ?? 0,
 lastAction: last ? `${last.action}${last.detail ? ` — ${last.detail}` : ''}` : null,
 lastActionAt: last ? last.createdAt : null,
 };
});

const byRep: Record<string, typeof logs> = {};
for (const l of logs) {
 const k = l.userName || "□□□□□□";
 (byRep[k] ?? []).push(l);
}

const sum = (m: Map<number, number>) => [...m.values()].reduce((a, b) => a + b, 0);
const counts = {
 pharmacies: sum(mPh),
 doctors: sum(mDr),
 orders: sum(mOr),
 trips: sum(mTr),
 activeTrips: sum(mAc),
 users: userRows.length,
 leaves: Number(pendingLeaves[0]?.c ?? 0),
};
void sql;

return Response.json({ logs, counts, reps, byRep });
}

```

```

@@ src/app/api/attachments/route.ts typescript
import { db } from "@db";
import { attachments } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { and, desc, eq, inArray } from "drizzle-orm";

export const dynamic = "force-dynamic";

const MAX_BYTES = 3 * 1024 * 1024; // 3MB

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "□□□□□□ □□□□□□" }, { status: 401 });
 const url = new URL(req.url);
 const ownerType = url.searchParams.get("ownerType") ?? "doctor";
 const ownerId = url.searchParams.get("ownerId");
 const isAdmin = user.role === "admin" || user.role === "supervisor";

 const conds = [eq(attachments.ownerType, ownerType)];
 if (ownerId) conds.push(eq(attachments.ownerId, Number(ownerId)));
 if (!isAdmin) conds.push(eq(attachments.userId, user.id));

 const rows = await db
 .select({
 id: attachments.id,
 ownerId: attachments.ownerId,

```

```

 fileName: attachments.fileName,
 mimeType: attachments.mimeType,
 size: attachments.size,
 repName: attachments.repName,
 createdAt: attachments.createdAt,
 })
 .from(attachments)
 .where(and(...conds))
 .orderBy(desc(attachments.id))
 .limit(300);
 return Response.json({ rows });
}

export async function POST(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "用户未登录" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const dataUrl = String(b.data ?? "");
 const match = dataUrl.match(/^data:([^\;]+);base64,(.+)$/);
 if (!match) return Response.json({ error: "数据格式错误" }, { status: 400 });
 const mimeType = match[1].slice(0, 100);
 const base64 = match[2];
 const size = Math.floor((base64.length * 3) / 4);
 if (size > MAX_BYTES) {
 return Response.json({ error: "文件大小超过限制" }, { status: 400 });
 }
 const [row] = await db
 .insert(attachments)
 .values({
 ownerType: String(b.ownerType ?? "doctor").slice(0, 30),
 ownerId: Number(b.ownerId) || null,
 userId: user.id,
 repName: user.fullName,
 fileName: String(b.fileName ?? "file").slice(0, 200),
 mimeType,
 size,
 data: base64,
 })
 .returning({
 id: attachments.id,
 fileName: attachments.fileName,
 mimeType: attachments.mimeType,
 size: attachments.size,
 ownerId: attachments.ownerId,
 });
 return Response.json({ row });
}

/** 更新附件信息 */
export async function PATCH(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "用户未登录" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const ids: number[] = Array.isArray(b.ids) ? b.ids.map(Number).filter(Number.isFinite) : [];
 const ownerId = Number(b.ownerId);
 if (ids.length || !Number.isFinite(ownerId)) return Response.json({ error: "参数错误" }, { status: 400 });
 await db
 .update(attachments)
 .set({ ownerId })
 .where(and(inArray(attachments.id, ids), eq(attachments.userId, user.id)));
 return Response.json({ ok: true });
}

export async function DELETE(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "用户未登录" }, { status: 401 });
 const id = Number(new URL(req.url).searchParams.get("id"));
 if (!Number.isFinite(id)) return Response.json({ error: "参数错误" }, { status: 400 });
 if (user.role === "admin") await db.delete(attachments).where(eq(attachments.id, id));
 else await db.delete(attachments).where(and(eq(attachments.id, id), eq(attachments.userId, user.id)));
 return Response.json({ ok: true });
}

@@ src/app/api/backup/route.ts typescript
import { db } from "@/db";
import {
 activityLogs,
 attachments,
 doctors,
 homes,
 leaves,
 messengers,
 notifications,
 options,
 orders,

```

```

 roles,
 pharmacies,
 settings,
 tripPoints,
 trips,
 users,
} from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { todayJalali } from "@lib/jalali";

export const dynamic = "force-dynamic";

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user || user.role !== "admin") return Response.json({ error: "دسترسی ندارید", { status: 401 }});
 const withFiles = new URL(req.url).searchParams.get("files") === "1";

 const [
 usersRows,
 pharmaciesRows,
 doctorsRows,
 ordersRows,
 optionsRows,
 homesRows,
 leavesRows,
 tripsRows,
 tripPointsRows,
 messengersRows,
 notificationsRows,
 settingsRows,
 activityRows,
 rolesRows,
] = await Promise.all([
 db.select().from(users),
 db.select().from(pharmacies),
 db.select().from(doctors),
 db.select().from(orders),
 db.select().from(options),
 db.select().from(homes),
 db.select().from(leaves),
 db.select().from(trips),
 db.select().from(tripPoints),
 db.select().from(messengers),
 db.select().from(notifications),
 db.select().from(settings),
 db.select().from(activityLogs),
 db.select().from(roles),
]);

 const files = withFiles
 ? await db.select().from(attachments)
 : await db
 .select({
 id: attachments.id,
 ownerType: attachments.ownerType,
 ownerId: attachments.ownerId,
 fileName: attachments.fileName,
 mimeType: attachments.mimeType,
 size: attachments.size,
 })
 .from(attachments);

 const payload = {
 app: "sabt-etelaat-kol",
 version: 1,
 createdAt: new Date().toISOString(),
 dateShamsi: todayJalali(),
 includesFiles: withFiles,
 counts: {
 users: usersRows.length,
 pharmacies: pharmaciesRows.length,
 doctors: doctorsRows.length,
 orders: ordersRows.length,
 trips: tripsRows.length,
 leaves: leavesRows.length,
 attachments: files.length,
 },
 data: {
 users: usersRows,
 pharmacies: pharmaciesRows,
 doctors: doctorsRows,
 orders: ordersRows,
 options: optionsRows,
 homes: homesRows,
 leaves: leavesRows,

```

[illegible]

```
const w: SQL | undefined = isAdmin ? undefined : eq(doctors.userId, user.id);
const data = (await db.select().from(doctors).where(w).orderBy(desc(doctors.id))).filter((r) =>
 inPeriod(r.dateShamsi),
);
const fileCount = new Map<number, number>();
try {
 const atts = await db
 .select({ ownerId: attachments.ownerId })
 .from(attachments)
 .where(eq(attachments.ownerType, "doctor"));
 for (const a of atts) if (a.ownerId) fileCount.set(a.ownerId, (fileCount.get(a.ownerId) ?? 0) + 1);
} catch {
 /* ignore */
}
rows = [
 ["🏠", "🏠 🏠🏠🏠🏠", "🏠🏠🏠 🏠", "🏠🏠🏠", "🏠", "🏠🏠🏠", "🏠 🏠🏠", "🏠🏠", "🏠🏠🏠 🏠🏠🏠 🏠🏠", "🏠 🏠🏠",
 "🏠🏠🏠 🏠", "🏠🏠 🏠", "🏠🏠 🏠🏠🏠🏠 🏠", "🏠🏠🏠", "🏠🏠🏠 🏠", "🏠🏠🏠 🏠", "🏠🏠 🏠", "🏠🏠🏠 🏠"],
 ...data.map((r, i) => [
 i + 1,
 r.repName,
 r.dateShamsi,
 r.province,
 r.city,
 r.region,
 r.name,
 r.specialty,
 r.phone,
 r.secretaryName,
 r.secretaryPhone,
 r.address,
 r.otherAddresses,
 r.isPercent ? "🏠": "🏠",
 r.percentValue,
 fileCount.get(r.id) ?? 0,
 mapLink(r.lat, r.lng),
 r.sent ? "🏠": "🏠",
]),
];
} else if (type === "orders") {
 const w: SQL | undefined = isAdmin ? undefined : eq(orders.userId, user.id);
 const data = (await db.select().from(orders).where(w).orderBy(desc(orders.id))).filter((r) =>
 inPeriod(r.dateShamsi),
);
 rows = [
 [
 "🏠",
 "🏠 🏠🏠🏠🏠",
 "🏠🏠🏠 🏠🏠",
 "🏠 🏠🏠🏠🏠",
 "🏠 🏠🏠🏠🏠",
 "🏠🏠🏠 🏠",
 "🏠🏠🏠 🏠",
 "🏠",
 ...PRODUCTS.flatMap((p) => [p.label, p.bonusLabel]),
 "🏠 🏠",
 "🏠 🏠🏠🏠",
 "🏠🏠",
 "🏠🏠 🏠",
 "🏠🏠 🏠",
],
 ...data.map((r, i) => [
 i + 1,
 r.repName,
 r.dateShamsi,
 r.pharmacyName,
 r.managerName,
 r.managerPhone,
 r.address,
 ...PRODUCTS.flatMap((p) => [Number(r.items?.[p.key] ?? 0), Number(r.items?.[bonusKeyOf(p.key)] ?? 0)]),
 r.distributor,
 r.visitor,
 r.notes,
 mapLink(r.lat, r.lng),
 r.sendStatus || (r.sent ? "🏠🏠 🏠": "🏠🏠 🏠"),
]),
];
} else if (type === "trips") {
 const w: SQL | undefined = isAdmin ? undefined : eq(trips.userId, user.id);
 const data = await db.select().from(trips).where(w).orderBy(desc(trips.id));
 rows = [
 ["🏠", "🏠 🏠🏠🏠", "🏠", "🏠", "🏠", "🏠", "🏠 (🏠)", "🏠 (🏠)",
 ...data.map((r, i) => [
 i + 1,
 r.repName,
 r.dateShamsi,
 r.status === "active" ? "🏠 🏠 🏠": "🏠 🏠",
]),
];
}
```



```
tehranDateTime(r.startedAt),
r.endedAt ? tehraneDateime(r.endedAt) : "",
((r.distanceM ?? 0) / 1000).toFixed(2),
Math.round((r.stopSeconds ?? 0) / 60),
]),
];
} else if (type === "leaves") {
const w: SQL | undefined = isAdmin ? undefined : eq(leaves.userId, user.id);
const data = await db.select().from(leaves).where(w).orderBy(desc(leaves.id));
const fa = (s: string) => (s === "approved" ? "مجاز شده" : s === "rejected" ? "رد شده" : "در حال بررسی");
rows = [
["مجاز شده", "لیست مجازات‌ها", "تاریخ", "نام پرسنل", "مدت مجازت", "دلیل مجازت", "وضعیت", "کارمند (پشتیبان)", "مدیر", "محل کار"],
...data.map((r, i) => [
i + 1,
r.repName,
r.kind,
r.fromDate,
r.toDate,
r.days,
r.fromTime,
r.toTime,
r.hours,
r.reason,
fa(r.supervisorStatus),
fa(r.managerStatus),
]),
]);
} else if (type === "homes") {
if (!isAdmin) return Response.json({ error: "شما دسترسی ندارید" }, { status: 401 });
const data = await db.select().from(homes).orderBy(desc(homes.id));
rows = [
["مکان", "آدرس", "نام مکان", "توضیحات", "وضعیت", "مسئول"],
...data.map((r, i) => [i + 1, r.repName, r.title, r.address, mapLink(r.lat, r.lng)]),
]);
} else if (type === "users") {
if (user.role !== "admin") return Response.json({ error: "شما دسترسی ندارید" }, { status: 401 });
const data = await db.select().from(users).orderBy(desc(users.id));
rows = [
["نام", "نام خانوادگی", "نام کاربری", "کلمه عبور", "ایمیل", "تلفن", "وضعیت", "نوع کاربر"],
...data.map((r, i) => [
i + 1,
r.fullName,
r.username,
r.passwordPlain,
r.phone,
r.role === "admin" ? "ادمین" : r.role === "supervisor" ? "سرپرست" : "کارمند",
r.active ? "فعال" : "غیرفعال",
]),
]);
} else {
return Response.json({ error: "نوع درخواست نامعتبر است" }, { status: 400 });
}

return xlsResponse(`${name}-${Date.now()}`, name, rows);
}
```

```

 });
 const data = (await res.json()) as { display_name: string; lat: string; lon: string; type?: string }[];
 return data.map((d) => ({
 label: d.display_name,
 lat: Number(d.lat),
 lng: Number(d.lon),
 type: d.type ?? "",
 }));
}

async function photon(q: string): Promise<Hit[]> {
 // lat/lon 00000 000000 00000 000000000000 000000 00000 00000
 const url =
 `https://photon.komoot.io/api/?limit=10&lang=default&lat=32.4279&lon=53.6880` +
 `&q=${encodeURIComponent(q)}`;
 const res = await fetchWithRetry(url, {}, { retries: 1, timeoutMs: 10000, label: "geocode2" });
 const data = (await res.json()) as {
 features: { properties: Record<string, string>; geometry: { coordinates: [number, number] } }[];
 };
 return (data.features ?? []).
 .filter((f) => {
 const c = (f.properties?.countrycode ?? "").toUpperCase();
 return !c || c === "IR"; // 00000000 00 000000 000000
 })
 .map((f) => {
 const p = f.properties ?? {};
 const label = [p.name, p.street, p.district, p.city, p.county, p.state].filter(Boolean).join(" ");
 return {
 label: label || p.name || "",
 lat: f.geometry.coordinates[1],
 lng: f.geometry.coordinates[0],
 type: p.osm_value ?? "",
 };
 })
 .filter((h) => h.label);
}

async function reverse(lat: number, lng: number): Promise<Hit[]> {
 const url = `https://nominatim.openstreetmap.org/reverse?format=jsonv2&accept-language=fa&lat=${lat}&lon=${lng}`;
 const res = await fetchWithRetry(
 url,
 { headers: { "User-Agent": UA, "Accept-Language": "fa" } },
 { retries: 2, timeoutMs: 12000, label: "revgeo" },
);
 const d = (await res.json()) as { display_name?: string };
 return d?.display_name ? [{ label: d.display_name, lat, lng, type: "reverse" }] : [];
}

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "0000000 00000000" }, { status: 401 });

 const url = new URL(req.url);
 const q = (url.searchParams.get("q") ?? "").trim();

 // 0000: Number(null) 000000 0000 0000 00 0000 0000 00000000 00000000 000000 0000
 // 000000 00 0000000 0000 0000000000 «0000000 000000 00 0000000 0000» 0000 000000.
 const latRaw = url.searchParams.get("lat");
 const lngRaw = url.searchParams.get("lng");
 const lat = latRaw === null ? NaN : Number(latRaw);
 const lng = lngRaw === null ? NaN : Number(lngRaw);

 try {
 if (Number.isFinite(lat) && Number.isFinite(lng) && (lat !== 0 || lng !== 0)) {
 return Response.json({ results: await reverse(lat, lng) }, { headers: { "Cache-Control": "no-store" } });
 }
 if (q.length < 2) return Response.json({ results: [] });

 // 0000 000000 0000000 0000000 00000 0000: «35.7219, 51.4089»
 const coord = q.match(/^(?!\d{1,2}\.?\d+)\s*[,()]\s*(-?\d{1,3}\.?\d+)\s*$/);
 if (coord) {
 const la = Number(coord[1]);
 const ln = Number(coord[2]);
 return Response.json({
 results: [{ label: "0000000 00000: ${la}, ${ln}", lat: la, lng: ln, type: "coords" }],
 });
 }
 }

 /**
 * 0000 000000 0000 00 00 0000000 000000000 00 000000 000000 000000 00000.
 * Photon 0000 0000 0000 000000000 0000 00000000 Nominatim 00000 /search
 * 00000000 0000 0 00000 000000000000000000 000000 00 00 00000000.
 */
 const providers: { name: string; run: () => Promise<Hit[]> }[] = [
 { name: "photon", run: () => photon(q) },
];
}

```

```

 { name: "nominatim", run: () => nominatim(q) },
];

 let results: Hit[] = [];
 const notes: string[] = [];
 for (const p of providers) {
 try {
 const r = await p.run();
 if (r.length) {
 results = r;
 notes.push(`${p.name}: ${r.length}`);
 break;
 }
 notes.push(`${p.name}: 0`);
 } catch (e) {
 notes.push(`${p.name}: ${(e instanceof Error ? e.message : String(e)).slice(0, 60)}`);
 }
 }

 return Response.json(
 { results, provider: notes.join(" | ") },
 { headers: { "Cache-Control": "no-store" } },
);
} catch (err) {
 return Response.json(
 { results: [], error: err instanceof Error ? err.message : "?????? ??????? ????",
 { status: 200 },
);
}
}

@@ src/app/api/health/route.ts typescript
import { checkDatabase, dbInfo, hasDatabaseUrl } from "@/db";
import { ensureSeed } from "@/lib/bootstrap";

export const dynamic = "force-dynamic";
export const revalidate = 0;

/**
 * Health check — ?? Retry ????? ? ????? ??????.
 * ??????? /ping ??? ????? ????? ????????? ?? ?? ????? ??????.
 */
export async function GET() {
 if (!hasDatabaseUrl) {
 return Response.json(
 { ok: false, db: false, reason: "DATABASE_URL ????? ????? ??" },
 { status: 500, headers: { "Cache-Control": "no-store" } },
);
 }

 const health = await checkDatabase();
 if (!health.ok) {
 return Response.json(
 { ok: false, db: false, latencyMs: health.latencyMs, reason: health.detail, pooler: dbInfo.usedPooler },
 { status: 503, headers: { "Cache-Control": "no-store" } },
);
 }

 // ????? ????????? / ????? ????? ?? ??????????? health ?? ????? ?????????
 void ensureSeed();

 return Response.json(
 {
 ok: true,
 db: true,
 latencyMs: health.latencyMs,
 pooler: dbInfo.usedPooler,
 connectTimeoutSec: dbInfo.connectTimeoutSec,
 },
 { headers: { "Cache-Control": "no-store" } },
);
}

@@ src/app/api/homes/route.ts typescript
import { db } from "@/db";
import { activityLogs, homes } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { desc, eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

export async function GET() {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "?????? ?????????", { status: 401 }});
 const isAdmin = user.role === "admin" || user.role === "supervisor";

```

```

const rows = isAdmin
 ? await db.select().from(homes).orderBy(desc(homes.id))
 : await db.select().from(homes).where(eq(homes.userId, user.id)).orderBy(desc(homes.id));
return Response.json({ rows });
}

```

```

export async function POST(req: Request) {
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "کاربر پیدا نشد", { status: 401 }});
 const b = await req.json().catch(() => ({}));
 const lat = Number(b.lat);
 const lng = Number(b.lng);
 if (!Number.isFinite(lat) || !Number.isFinite(lng)) {
 return Response.json({ error: "مختصات باید عدد باشند", { status: 400 }});
 }
 const values = {
 userId: user.id,
 repName: user.fullName,
 title: String(b.title ?? "").trim().slice(0, 160) || "",
 address: String(b.address ?? "").trim().slice(0, 1000),
 lat,
 lng,
 accuracy: Number(b.accuracy) || null,
 updatedAt: new Date(),
 };
 const existing = await db.select().from(homes).where(eq(homes.userId, user.id)).limit(1);
 let row;
 if (existing.length) {
 [row] = await db.update(homes).set(values).where(eq(homes.id, existing[0].id)).returning();
 } else {
 [row] = await db.insert(homes).values(values).returning();
 }
 await db
 .insert(activityLogs)
 .values({ userId: user.id, userName: user.fullName, action: "ثبت مکان", detail: values.title })
 .catch(() => undefined);
 return Response.json({ row });
}

```

@@ src/app/api/leaves/route.ts typescript

```

import { db } from "@/db";
import { activityLogs, leaves } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { isValidJalali } from "@/lib/jalali";
import { notify } from "@/lib/notify";
import { desc, eq } from "drizzle-orm";

```

```

export const dynamic = "force-dynamic";

```

```

export async function GET() {
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "کاربر پیدا نشد", { status: 401 }});
 const isAdmin = user.role === "admin" || user.role === "supervisor";
 const rows = isAdmin
 ? await db.select().from(leaves).orderBy(desc(leaves.id)).limit(500)
 : await db.select().from(leaves).where(eq(leaves.userId, user.id)).orderBy(desc(leaves.id)).limit(200);
 return Response.json({ rows, role: user.role });
}

```

```

export async function POST(req: Request) {
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "کاربر پیدا نشد", { status: 401 }});
 const b = await req.json().catch(() => ({}));
 const fromDate = String(b.fromDate ?? "").trim();
 const toDate = String(b.toDate ?? "").trim();
 if (!isValidJalali(fromDate) || !isValidJalali(toDate)) {
 return Response.json({ error: "تاریخ‌ها باید معتبر باشند", { status: 400 }});
 }
 const [row] = await db
 .insert(leaves)
 .values({
 userId: user.id,
 repName: user.fullName,
 kind: String(b.kind ?? "").slice(0, 30),
 fromDate,
 toDate,
 days: Math.max(0, Number(b.days) || 0),
 fromTime: String(b.fromTime ?? "").slice(0, 5),
 toTime: String(b.toTime ?? "").slice(0, 5),
 hours: Math.max(0, Number(b.hours) || 0),
 reason: String(b.reason ?? ""),
 })
 .returning();
 await db
 .insert(activityLogs)

```

```

 .values({
 userId: user.id,
 userName: user.fullName,
 action: "□□□□□□ □□□□",
 detail: `${fromDate} □□ ${toDate}`,
 })
 .catch(() => undefined);
 for (const role of ["admin", "supervisor"] as const) {
 await notify({
 toRole: role,
 fromName: user.fullName,
 kind: "leave",
 title: `□ □□□□□□□ □□□□ □□□□ □□ ${user.fullName}`,
 body: `□□ ${fromDate} □□ ${toDate} — ${row.kind}${row.reason ? `| ${row.reason}` : ""}`,
 link: "/admin/leaves",
 });
 }
}
return Response.json({ row });
}

export async function PATCH(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "□□□□□□ □□□□□□" }, { status: 401 });
 if (user.role !== "admin" && user.role !== "supervisor") {
 return Response.json({ error: "□□□ □□□□□□ □□ □□□□ □□□□□□□□ □□□□ □□□" }, { status: 403 });
 }
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 const status = b.status === "approved" ? "approved" : b.status === "rejected" ? "rejected" : null;
 if (!Number.isFinite(id) || !status) return Response.json({ error: "□□□□□□ □□□□□□□□" }, { status: 400 });
 const note = String(b.note ?? "");

 const patch =
 user.role === "supervisor"
 ? { supervisorStatus: status, supervisorNote: note, supervisorName: user.fullName }
 : { managerStatus: status, managerNote: note, managerName: user.fullName };

 const target = (await db.select().from(leaves).where(eq(leaves.id, id)).limit(1))[0];
 await db.update(leaves).set(patch).where(eq(leaves.id, id));
 if (target) {
 await notify({
 toUserId: target.userId,
 fromName: user.fullName,
 kind: "leave",
 title: `${status === "approved" ? "□": "□"} □□□□□ □□□ □□□□ ${user.role === "supervisor" ? "□□□□□□": "□□□□"} ${status === "approved" ? "□□□□□□": "□□□□"} □□`,
 body: `${target.fromDate} □□ ${target.toDate}${note ? ` — ${note}` : ""}`,
 link: "/panel/leaves",
 });
 }
}
await db
 .insert(activityLogs)
 .values({
 userId: user.id,
 userName: user.fullName,
 action: user.role === "supervisor" ? "□□□□□□/□□ □□□□□□": "□□□□□□/□□ □□□□",
 detail: `□□□□□ #${id} → ${status === "approved" ? "□□□□□□": "□□"}`,
 })
 .catch(() => undefined);
return Response.json({ ok: true });
}

```

```

@@ src/app/api/live/route.ts typescript
import { db, dbRetrySafe } from "@db";
import { liveLocations, trips, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { desc, eq, sql } from "drizzle-orm";

```

```
export const dynamic = "force-dynamic";
```

```

/**
 * □□□□□□ □□□□ □□□□□□□□□□.
 *
 * POST → □□□□□□□□ □□□□□□ □□□ □□ □□□□□ □□□□□□ □□□ □□□□ □□□ □□□□ □□□□
 * GET → □□□□ □□□□□ □□□□□□ □□□ □□□□□□□□□□ □□ □□□□□□□□
 */

```

```

export async function POST(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "□□□□□□ □□□□□□□□" }, { status: 401 });
 const b = await req.json().catch(() => ({}));

 const lat = Number(b.lat);
 const lng = Number(b.lng);
 if (!Number.isFinite(lat) || !Number.isFinite(lng)) {
 return Response.json({ error: "□□□□□□ □□□□□□□□" }, { status: 400 });
 }
}

```

```

}

const values = {
 userId: user.id,
 repName: user.fullName,
 lat,
 lng,
 accuracy: Number(b.accuracy) || null,
 speed: Number(b.speed) || null,
 heading: Number(b.heading) || null,
 battery: Number.isFinite(Number(b.battery)) ? Math.round(Number(b.battery)) : null,
 gpsOn: b.gpsOn !== false,
 online: true,
 tripId: Number(b.tripId) || null,
 recordedAt: b.recordedAt ? new Date(String(b.recordedAt)) : new Date(),
 updatedAt: new Date(),
};

await dbRetrySafe(
 () =>
 db
 .insert(liveLocations)
 .values(values)
 .onConflictDoUpdate({ target: liveLocations.userId, set: values }),
 undefined,
 "live:upsert",
);

return Response.json({ ok: true }, { headers: { "Cache-Control": "no-store" } });
}

export async function GET() {
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "用户未登录", { status: 401 }});
 const isManager = user.role === "admin" || user.role === "supervisor";

 const rows = await dbRetrySafe(
 () =>
 db
 .select({
 userId: liveLocations.userId,
 repName: liveLocations.repName,
 lat: liveLocations.lat,
 lng: liveLocations.lng,
 accuracy: liveLocations.accuracy,
 speed: liveLocations.speed,
 battery: liveLocations.battery,
 gpsOn: liveLocations.gpsOn,
 tripId: liveLocations.tripId,
 recordedAt: liveLocations.recordedAt,
 updatedAt: liveLocations.updatedAt,
 })
 .from(liveLocations)
 .orderBy(desc(liveLocations.updatedAt)),
 [],
 "live:list",
);

 const scoped = isManager ? rows : rows.filter((r) => r.userId === user.id);

 // 获取行程列表
 const active = await dbRetrySafe(
 () =>
 db
 .select({ userId: trips.userId, id: trips.id, distanceM: trips.distanceM })
 .from(trips)
 .where(eq(trips.status, "active")),
 [],
 "live:trips",
);

 const allUsers = isManager
 ? await dbRetrySafe(
 () => db.select({ id: users.id, fullName: users.fullName, role: users.role, active: users.active }).from(users),
 [],
 "live:users",
)
 : [];

 const now = Date.now();
 const enriched = scoped.map((r) => {
 const ageMs = now - new Date(r.updatedAt).getTime();
 return {
 ...r,
 ageMs,
 };
 });
}

```

```

/**
 * live: ageMs < 3 * 60 * 1000,
 * stale: ageMs >= 3 * 60 * 1000 && ageMs < 60 * 60 * 1000,
 * activeTrip: active.find((t) => t.userId === r.userId) ?? null,
 */;
});
});

//
const missing = allUsers
 .filter((u) => u.role === "rep" && u.active && !scoped.some((r) => r.userId === u.id))
 .map((u) => ({ userId: u.id, repName: u.fullName }));

void sql;
return Response.json({ rows: enriched, missing }, { headers: { "Cache-Control": "no-store" } });
}

@@ src/app/api/messengers/route.ts typescript
import { db } from "@/db";
import { messageLogs, messengers, settings } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { getProxy } from "@/lib/messaging";
import { getSmsConfig } from "@/lib/otp";
import { desc, eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

async function adminGuard() {
 const user = await getSessionUser();
 if (!user) return null;
 if (user.role === "admin" || user.permissions.includes("messengers")) return user;
 return null;
}

export async function GET() {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "Unauthorized", status: 401 });
 const rows = await db.select().from(messengers).orderBy(desc(messengers.id));
 const logs = await db.select().from(messageLogs).orderBy(desc(messageLogs.id)).limit(50);
 const proxy = await getProxy();
 const sms = await getSmsConfig();
 let autoSend = true;
 try {
 const st = await db.select().from(settings).where(eq(settings.key, "autoSendOrders")).limit(1);
 const v = st[0]?.value as { enabled?: boolean } | undefined;
 if (v && typeof v.enabled === "boolean") autoSend = v.enabled;
 } catch {
 /* ignore */
 }
 return Response.json({ rows, logs, proxy, sms, autoSend });
}

export async function POST(req: Request) {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "Unauthorized", status: 401 });
 const b = await req.json().catch(() => ({}));
 const platform = String(b.platform ?? "");
 if (!["telegram", "bale", "eitaa", "whatsapp"].includes(platform)) {
 return Response.json({ error: "Invalid platform", status: 400 });
 }
 const [row] = await db
 .insert(messengers)
 .values({
 platform,
 label: String(b.label ?? "").trim().slice(0, 160),
 targetType: ["group", "channel", "phone"].includes(b.targetType) ? b.targetType : "phone",
 target: String(b.target ?? "").trim(),
 token: String(b.token ?? "").trim(),
 provider: String(b.provider ?? "").trim().slice(0, 30),
 apiUrl: String(b.apiUrl ?? "").trim(),
 enabled: b.enabled !== false,
 })
 .returning();
 return Response.json({ row });
}

export async function PATCH(req: Request) {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "Unauthorized", status: 401 });
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 if (!Number.isFinite(id)) return Response.json({ error: "Invalid id", status: 400 });
 const patch: Record<string, unknown> = {};
 if (typeof b.label === "string") patch.label = b.label.trim();
 if (typeof b.target === "string") patch.target = b.target.trim();
 if (typeof b.targetType === "string" && ["group", "channel", "phone"].includes(b.targetType)) {

```

```

 patch.targetType = b.targetType;
 }
 if (typeof b.token === "string") patch.token = b.token.trim();
 if (typeof b.provider === "string") patch.provider = b.provider.trim().slice(0, 30);
 if (typeof b.apiUrl === "string") patch.apiUrl = b.apiUrl.trim();
 if (typeof b.enabled === "boolean") patch.enabled = b.enabled;
 if (Object.keys(patch).length === 0) return Response.json({ error: "消息内容不能为空", { status: 400 }});
 await db.update(messengers).set(patch).where(eq(messengers.id, id));
 return Response.json({ ok: true });
}

export async function DELETE(req: Request) {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "只有管理员可以删除消息", { status: 401 }});
 const id = Number(new URL(req.url).searchParams.get("id"));
 if (!Number.isFinite(id)) return Response.json({ error: "消息ID必须为数字", { status: 400 }});
 await db.delete(messengers).where(eq(messengers.id, id));
 return Response.json({ ok: true });
}

@@ src/app/api/notifications/route.ts typescript
import { db } from "@/db";
import { notifications, users } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { and, desc, eq, inArray, isNull, or, sql, SQL } from "drizzle-orm";

export const dynamic = "force-dynamic";

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "用户未登录", { status: 401 }});
 const onlyUnread = new URL(req.url).searchParams.get("unread") === "1";

 const scope: SQL | undefined =
 user.role === "rep"
 ? or(eq(notifications.toUserId, user.id), eq(notifications.toRole, "rep"))
 : or(
 eq(notifications.toUserId, user.id),
 eq(notifications.toRole, user.role),
 and(isNull(notifications.toUserId), eq(notifications.toRole, "")),
);

 const where = onlyUnread ? and(scope, isNull(notifications.readAt)) : scope;
 const rows = await db.select().from(notifications).where(where).orderBy(desc(notifications.id)).limit(100);
 const unread = rows.filter((r) => !r.readAt).length;
 return Response.json({ rows, unread });
}

/** 发送消息 */
export async function POST(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "用户未登录", { status: 401 }});
 const b = await req.json().catch(() => ({}));
 const title = String(b.title ?? "").trim().slice(0, 200);
 if (!title) return Response.json({ error: "消息标题不能为空", { status: 400 }});
 const body = String(b.body ?? "").slice(0, 2000);

 if (user.role === "rep") {
 await db.insert(notifications).values({
 toRole: "admin",
 fromName: user.fullName,
 kind: "message",
 title,
 body,
 });
 return Response.json({ ok: true });
 }

 const targetId = Number(b.toUserId);
 if (Number.isFinite(targetId) && targetId > 0) {
 await db
 .insert(notifications)
 .values({ toUserId: targetId, fromName: user.fullName, kind: "message", title, body });
 } else {
 const reps = await db.select({ id: users.id }).from(users).where(eq(users.role, "rep"));
 if (reps.length) {
 await db
 .insert(notifications)
 .values(reps.map((r) => ({ toUserId: r.id, fromName: user.fullName, kind: "message", title, body })));
 }
 }
 return Response.json({ ok: true });
}

/** 消息历史记录 */

```



```

export async function PATCH(req: Request) {
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "用户不存在" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const ids: number[] = Array.isArray(b.ids) ? b.ids.map(Number).filter(Number.isFinite) : [];
 if (b.all === true) {
 const scope: SQL | undefined =
 user.role === "rep"
 ? or(eq(notifications.toUserId, user.id), eq(notifications.toRole, "rep"))
 : or(
 eq(notifications.toUserId, user.id),
 eq(notifications.toRole, user.role),
 and(isNull(notifications.toUserId), eq(notifications.toRole, "")),
);
 await db
 .update(notifications)
 .set({ readAt: new Date() })
 .where(and(scope, isNull(notifications.readAt)));
 return Response.json({ ok: true });
 }
 if (ids.length) return Response.json({ error: "ids 不能为空" }, { status: 400 });
 await db.update(notifications).set({ readAt: new Date() }).where(inArray(notifications.id, ids));
 void sql;
 return Response.json({ ok: true });
}

@@ src/app/api/options/route.ts typescript
import { db } from "@/db";
import { activityLogs, options } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { ensureSeed } from "@/lib/bootstrap";
import { OPTION_CATEGORIES } from "@/lib/constants";
import { and, asc, eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

const VALID = OPTION_CATEGORIES.map((c) => c.key);

export async function GET(req: Request) {
 await ensureSeed();
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "用户不存在" }, { status: 401 });
 const url = new URL(req.url);
 const category = url.searchParams.get("category");
 const parent = url.searchParams.get("parent");
 const where =
 category && parent !== null
 ? and(eq(options.category, category), eq(options.parent, parent))
 : category
 ? eq(options.category, category)
 : undefined;
 const rows = await db
 .select()
 .from(options)
 .where(where)
 .orderBy(asc(options.category), asc(options.value))
 .limit(5000);
 return Response.json({ rows });
}

/** 用户添加选项 (用户添加选项 options) 用户添加选项 用户添加选项 用户添加选项.*/
export async function POST(req: Request) {
 const user = await getSessionUser();
 if(!user) return Response.json({ error: "用户不存在" }, { status: 401 });
 const canAdd = user.role === "admin" || user.permissions.includes("options");
 if(!canAdd) return Response.json({ error: "用户没有权限添加选项" }, { status: 403 });

 const body = await req.json().catch(() => ({}));
 const category = String(body.category ?? "").trim();
 const value = String(body.value ?? "").trim().slice(0, 200);
 const parent = String(body.parent ?? "").trim().slice(0, 200);
 if (!VALID.includes(category)) return Response.json({ error: "category 无效" }, { status: 400 });
 if (value.length < 2) return Response.json({ error: "value 不能为空" }, { status: 400 });
 if ((category === "city" || category === "region") && !parent) {
 return Response.json(
 { error: category === "city" ? "city 必须有 parent" : "region 必须有 parent" },
 { status: 400 },
);
 }

 const exists = await db
 .select()
 .from(options)
 .where(and(eq(options.category, category), eq(options.value, value), eq(options.parent, parent)))
 .limit(1);

```

```

 if (exists.length) return Response.json({ row: exists[0], duplicate: true });

 const [row] = await db
 .insert(options)
 .values({ category, value, parent, createdBy: user.fullName })
 .returning();
 await db
 .insert(activityLogs)
 .values({ userId: user.id, userName: user.fullName, action: "□□□□□□ □□□□□ □□□□□", detail: `${category}: ${value}` })
 .catch(() => undefined);
 return Response.json({ row });
 }

export async function PATCH(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "□□□□□□ □□□□□□□□", { status: 401 }});
 const canEdit = user.role === "admin" || user.permissions.includes("optionsDelete");
 if (!canEdit) return Response.json({ error: "□□□ □□□□□ □□□□□□ □□□□□□", { status: 403 }});
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 const value = String(b.value ?? "").trim().slice(0, 200);
 if (!Number.isFinite(id) || !value) return Response.json({ error: "□□□□□ □□□□□□□□", { status: 400 }});
 await db.update(options).set({ value }).where(eq(options.id, id));
 await db
 .insert(activityLogs)
 .values({ userId: user.id, userName: user.fullName, action: "□□□□□□ □□□□□ □□□□□", detail: value })
 .catch(() => undefined);
 return Response.json({ ok: true });
}

export async function DELETE(req: Request) {
 const user = await getSessionUser();
 if (!user || (user.role !== "admin" && !user.permissions.includes("optionsDelete"))) {
 return Response.json({ error: "□□□ □□□□□ □□□ □□□□□□", { status: 401 }});
 }
 const id = Number(new URL(req.url).searchParams.get("id"));
 if (!Number.isFinite(id)) return Response.json({ error: "□□□□□ □□□□□□□□", { status: 400 }});
 await db.delete(options).where(eq(options.id, id));
 return Response.json({ ok: true });
}

@@ src/app/api/ping/route.ts typescript
/**
 * Keep-Alive endpoint — □□□□ □□□□□ □□□□ □ □□□□ □□□□ □□ □□□□□□□□.
 *
 * □□□□: □□□□□□□□ □□ □□□□□□□□ □□□□□ □□□ □□□□ □□□□ □□□□ □□ □□ □□ □□□□□
 * □□□□□□□□ □□□□□ □□□□□□ □ □□□□□ □□□ □□□□ □□ □□ □□□□□ □□□ □□□□□□□□.
 *
 * □□ UptimeRobot □□ □□□□□□□□ □□ □□□□ HTTP(s) □□□□□□□□:
 * URL : https://<your-app>.onrender.com/ping
 * Interval: □□ □ □□□□□□
 *
 * □□□□□□□□ □□□□□□: /ping □ /api/ping
 */

export const dynamic = "force-dynamic";
export const revalidate = 0;

const startedAt = Date.now();

const headers = {
 "Content-Type": "text/plain; charset=utf-8",
 "Cache-Control": "no-store, no-cache, must-revalidate, max-age=0",
 "X-Robots-Tag": "noindex",
 "Access-Control-Allow-Origin": "*",
} as const;

export async function GET(req: Request) {
 // □□□□ □□□□ □□□□ □□□□□□□□□□ □□□□□□□□□□□□: □□□ □□□ OK
 const wantsJson = new URL(req.url).searchParams.has("json");
 if (!wantsJson) return new Response("OK", { status: 200, headers });

 return new Response(
 JSON.stringify({
 status: "OK",
 uptimeMs: Date.now() - startedAt,
 timestamp: new Date().toISOString(),
 }),
 { status: 200, headers: { ...headers, "Content-Type": "application/json; charset=utf-8" } },
);
}

export async function HEAD() {
 return new Response(null, { status: 200, headers });
}

```

```

@@ src/app/api/reports/route.ts typescript
import { db, dbRetry } from "@/db";
import { doctors, orders, pharmacies, trips } from "@/db/schema";
import { getSessionUser } from "@/lib/auth";
import { bonusKeyOf } from "@/lib/defaults";
import { getProducts } from "@/lib/settings-server";
import { eq, SQL } from "drizzle-orm";

export const dynamic = "force-dynamic";

export type MonthlyRow = {
 period: string; // 1405/03
 repName: string;
 pharmacies: number;
 doctors: number;
 orders: number;
 units: number;
 bonus: number;
 trips: number;
};

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "Unauthorized", status: 401 });
 const url = new URL(req.url);
 const isAdmin = user.role === "admin" || user.role === "supervisor";
 const repFilter = url.searchParams.get("userId");
 const scopeId = isAdmin ? (repFilter ? Number(repFilter) : null) : user.id;

 const wP: SQL | undefined = scopeId ? eq(pharmacies.userId, scopeId) : undefined;
 const wD: SQL | undefined = scopeId ? eq(doctors.userId, scopeId) : undefined;
 const wO: SQL | undefined = scopeId ? eq(orders.userId, scopeId) : undefined;
 const wT: SQL | undefined = scopeId ? eq(trips.userId, scopeId) : undefined;

 const PRODUCTS = await getProducts();
 const [ph, dr, or, tr] = await dbRetry(() => Promise.all([
 db
 .select({ d: pharmacies.dateShamsi, rep: pharmacies.repName })
 .from(pharmacies)
 .where(wP)
 .limit(5000),
 db.select({ d: doctors.dateShamsi, rep: doctors.repName }).from(doctors).where(wD).limit(5000),
 db
 .select({ d: orders.dateShamsi, rep: orders.repName, items: orders.items })
 .from(orders)
 .where(wO)
 .limit(5000),
 db.select({ d: trips.dateShamsi, rep: trips.repName }).from(trips).where(wT).limit(5000),
]), "reports:load");

 const map = new Map<string, MonthlyRow>();
 const key = (d: string, rep: string) => `${d.slice(0, 7)}|${rep}`;
 const get = (d: string, rep: string) => {
 const k = key(d, rep);
 let row = map.get(k);
 if (!row) {
 row = {
 period: d.slice(0, 7),
 repName: rep,
 pharmacies: 0,
 doctors: 0,
 orders: 0,
 units: 0,
 bonus: 0,
 trips: 0,
 };
 map.set(k, row);
 }
 return row;
 };

 for (const r of ph) get(r.d, r.rep).pharmacies++;
 for (const r of dr) get(r.d, r.rep).doctors++;
 for (const r of tr) get(r.d, r.rep).trips++;
 for (const r of or) {
 const row = get(r.d, r.rep);
 row.orders++;
 for (const p of PRODUCTS) {
 row.units += Number(r.items?.[p.key] ?? 0);
 row.bonus += Number(r.items?.[bonusKeyOf(p.key)] ?? 0);
 }
 }

 const rows = [...map.values()].sort((a, b) =>

```

```

 a.period === b.period ? a.repName.localeCompare(b.repName) : b.period.localeCompare(a.period),
);
};

// 商品の総数と、各商品ごとの賞金総額を計算
const productTotals: Record<string, Record<string, number>> = {};
const productByRep: {
 period: string;
 repName: string;
 items: Record<string, number>;
 bonuses: Record<string, number>;
}[] = [];
const idx = new Map<string, number>();

for (const r of or) {
 const per = r.d.slice(0, 7);
 productTotals[per] ??= {};
 const k = `${per}|${r.rep}`;
 let i = idx.get(k);
 if (i === undefined) {
 i = productByRep.length;
 idx.set(k, i);
 productByRep.push({ period: per, repName: r.rep, items: {}, bonuses: {} });
 }
 for (const p of PRODUCTS) {
 const q = Number(r.items?.[p.key] ?? 0);
 const b = Number(r.items?.[bonusKeyOf(p.key)] ?? 0);
 productTotals[per][p.key] = (productTotals[per][p.key] ?? 0) + q;
 productByRep[i].items[p.key] = (productByRep[i].items[p.key] ?? 0) + q;
 productByRep[i].bonuses[p.key] = (productByRep[i].bonuses[p.key] ?? 0) + b;
 }
}
productByRep.sort((a, b) =>
 a.period === b.period ? a.repName.localeCompare(b.repName) : b.period.localeCompare(a.period),
);

return Response.json({ rows, productTotals, productByRep });
}

@@ src/app/api/roles/route.ts typescript
import { db } from "@db";
import { roles, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { asc, eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

async function guard(write = false) {
 const user = await getSessionUser();
 if (!user) return null;
 if (!write) return user;
 if (user.role === "admin" || user.permissions.includes("users")) return user;
 return null;
}

export async function GET() {
 await ensureSeed();
 const user = await guard(false);
 if (!user) return Response.json({ error: "権限がありません" }, { status: 401 });
 const rows = await db.select().from(roles).orderBy(asc(roles.id));
 return Response.json({ rows });
}

export async function POST(req: Request) {
 const user = await guard(true);
 if (!user) return Response.json({ error: "権限がありません" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const label = String(b.label ?? "").trim();
 if (!label) return Response.json({ error: "ラベルは必須です" }, { status: 400 });
 const key =
 String(b.key ?? "").trim().toLowerCase().replace(/[^a-z0-9_-]/g, "") || `role_${Date.now().toString(36)}`;
 const base = ["admin", "supervisor", "rep"].includes(b.base) ? b.base : "rep";
 const dup = await db.select().from(roles).where(eq(roles.key, key)).limit(1);
 if (dup.length) return Response.json({ error: "このラベルは既に存在します" }, { status: 400 });
 const [row] = await db.insert(roles).values({ key, label, base, permissions: Array.isArray(b.permissions) ? b.permissions : [] }).returning();
 return Response.json({ row });
}

export async function PATCH(req: Request) {

```

```

 const user = await guard(true);
 if (!user) return Response.json({ error: "用户不存在", { status: 401 }});
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 if (!Number.isFinite(id)) return Response.json({ error: "id 格式不正确", { status: 400 }});
 const patch: Record<string, unknown> = {};
 if (typeof b.label === "string" && b.label.trim()) patch.label = b.label.trim();
 if (typeof b.base === "string" && ["admin", "supervisor", "rep"].includes(b.base)) patch.base = b.base;
 if (Array.isArray(b.permissions)) patch.permissions = b.permissions;
 if (Object.keys(patch).length === 0) return Response.json({ error: "更新内容不能为空", { status: 400 }});
 await db.update(roles).set(patch).where(eq(roles.id, id));
 return Response.json({ ok: true });
}

export async function DELETE(req: Request) {
 const user = await guard(true);
 if (!user) return Response.json({ error: "用户不存在", { status: 401 }});
 const id = Number(new URL(req.url).searchParams.get("id"));
 if (!Number.isFinite(id)) return Response.json({ error: "id 格式不正确", { status: 400 }});
 const rows = await db.select().from(roles).where(eq(roles.id, id)).limit(1);
 const r = rows[0];
 if (!r) return Response.json({ error: "角色不存在", { status: 404 }});
 if (r.builtin) return Response.json({ error: "内置角色不能被删除", { status: 400 }});
 const inUse = await db.select({ id: users.id }).from(users).where(eq(users.role, r.key)).limit(1);
 if (inUse.length) return Response.json({ error: "该角色正在被使用", { status: 400 }});
 await db.delete(roles).where(eq(roles.id, id));
 return Response.json({ ok: true });
}

@@ src/app/api/settings/route.ts typescript
import { db } from "@db";
import { settings } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { DEFAULT_COLUMNS, DEFAULT_FORM_FIELDS, DEFAULT_PRODUCTS } from "@lib/defaults";
import { eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

const FALLBACK: Record<string, unknown> = {
 products: DEFAULT_PRODUCTS,
 "columns.pharmacies": DEFAULT_COLUMNS.pharmacies,
 "columns.doctors": DEFAULT_COLUMNS.doctors,
 "columns.orders": DEFAULT_COLUMNS.orders,
 "fields.pharmacies": DEFAULT_FORM_FIELDS.pharmacies,
 "fields.doctors": DEFAULT_FORM_FIELDS.doctors,
 "fields.orders": DEFAULT_FORM_FIELDS.orders,
};

export async function GET(req: Request) {
 await ensureSeed();
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "用户不存在", { status: 401 }});
 const key = new URL(req.url).searchParams.get("key");
 const rows = await db.select().from(settings);
 const map: Record<string, unknown> = { ...FALLBACK };
 for (const r of rows) map[r.key] = r.value;
 if (user.role !== "admin") delete map["sms"];
 if (key) return Response.json({ key, value: map[key] ?? null });
 return Response.json({ values: map });
}

export async function PUT(req: Request) {
 const user = await getSessionUser();
 const b0 = user ? null : null;
 void b0;
 if (!user) return Response.json({ error: "用户不存在", { status: 401 }});
 const allowed =
 user.role === "admin" ||
 user.permissions.includes("columns") ||
 user.permissions.includes("messengers") ||
 user.permissions.includes("users");
 if (!allowed) return Response.json({ error: "权限不足", { status: 401 }});
 const b = await req.json().catch(() => ({}));
 const key = String(b.key ?? "");
 if (!key || b.value === undefined) return Response.json({ error: "请求格式不正确", { status: 400 }});
 const existing = await db.select().from(settings).where(eq(settings.key, key)).limit(1);
 if (existing.length) {
 await db.update(settings).set({ value: b.value, updatedAt: new Date() }).where(eq(settings.key, key));
 } else {
 await db.insert(settings).values({ key, value: b.value });
 }
 return Response.json({ ok: true });
}

```

```

@@ src/app/api/source/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { createZip, type ZipEntry } from "@lib/zip";

export const dynamic = "force-dynamic";
export const maxDuration = 60;

/**
 * 0000000 0000 0000 00000000 0000 ZIP (000 0000).
 *
 * 0000000: 0000000 000000 0000000 00 0000 00 000000 000 00 00
 * GitHub / GitLab 0000 0000 0 000 00 00 0000 000000 000.
 *
 * GET /api/source → 000 000000000 000000
 * GET /api/source?list=1 → 000 000000 00000000 (JSON)
 *
 * 0000 000: 0000000000 00000000000 0000000 «000000000» 0 000 00 0000 0000
 * 000000000 00000000. 000 00000000 0000 import 000000 000000 0000000 Turbopack
 * 0000 0000000 00 000000 00000000 000 0000 000 0 000000
 * «Encountered unexpected file in NFT list» 0000000.
 */

/** 0000000000 00 0000 00 000000 0000 0000000000 */
const SKIP_DIRS = new Set([
 "node_modules",
 ".next",
 ".git",
 ".turbo",
 "dist",
 "build",
 ".vercel",
 ".cache",
 "coverage",
]);

/** 0000000000 00 00 000000 0000000 00 0000 000 000000000 */
const SKIP_FILES = new Set([".env", ".env.local", ".env.production", "package-lock.json"]);

const MAX_FILE_BYTES = 3 * 1024 * 1024;

type FsApi = {
 readFile: (p: string) => Promise<Buffer>;
 readdir: (p: string) => Promise<string[]>;
 stat: (p: string) => Promise<{ isDirectory: () => boolean; size: number }>;
 join: (...p: string[]) => string;
 root: string;
};

/** 0000000000 00000000000 0000000 00 0000 0000 */
async function loadFs(): Promise<FsApi> {
 const fsp = await import("node:fs/promises");
 const nodePath = await import("node:path");
 return {
 readFile: (p: string) => fsp.readFile(p) as unknown as Promise<Buffer>,
 readdir: (p: string) => fsp.readdir(p),
 stat: (p: string) => fsp.stat(p),
 join: (...p: string[]) => nodePath.join(...p),
 root: process.cwd(),
 };
}

async function walk(fs: FsApi, dir: string, base = ""): Promise<string[]> {
 const out: string[] = [];
 let items: string[] = [];
 try {
 items = await fs.readdir(dir);
 } catch {
 return out;
 }
 for (const name of items) {
 if (name === ".git" || name === ".next") continue;
 const rel = base ? `${base}/${name}` : name;
 const full = fs.join(dir, name);
 let st;
 try {
 st = await fs.stat(full);
 } catch {
 continue;
 }
 if (st.isDirectory()) {
 if (SKIP_DIRS.has(name)) continue;
 out.push(...(await walk(fs, full, rel)));
 } else {
 if (SKIP_FILES.has(name)) continue;
 if (st.size > MAX_FILE_BYTES) continue;
 }
 }
}

```

```

 out.push(rel);
 }
}
return out;
}

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user || user.role !== "admin") {
 return Response.json({ error: "🚫🚫🚫🚫🚫🚫 — 🚫🚫 🚫🚫🚫", { status: 401 }});
 }

 const fs = await loadFs();
 const files = (await walk(fs, fs.root)).sort();

 if (new URL(req.url).searchParams.get("list") === "1") {
 return Response.json({ count: files.length, files }, { headers: { "Cache-Control": "no-store" } });
 }

 const entries: ZipEntry[] = [];
 for (const rel of files) {
 try {
 entries.push({ path: `namayandeelmi-javad/${rel}`, content: await fs.readFile(fs.join(fs.root, rel)) });
 } catch {
 /* 🚫🚫🚫🚫🚫🚫 🚫🚫 — 🚫🚫 🚫🚫🚫🚫🚫 */
 }
 }

 entries.push({
 path: "namayandeelmi-javad/HOW-TO-UPDATE.md",
 content: `# 🚫🚫🚫🚫🚫🚫🚫🚫🚫🚫🚫 🚫🚫🚫`
 });

 🚫🚫 🚫🚫🚫🚫 🚫🚫🚫 🚫🚫🚫 🚫🚫🚫 🚫 🚫🚫🚫🚫🚫 🚫🚫🚫 🚫🚫🚫🚫🚫 🚫🚫 (${entries.length} 🚫🚫🚫).

 ## 🚫🚫🚫🚫

 1. 🚫🚫🚫🚫🚫🚫 🚫🚫 ZIP 🚫🚫 🚫🚫 🚫🚫🚫🚫 🚫🚫🚫 🚫🚫 🚫🚫🚫🚫 🚫🚫 🚫🚫🚫
 (🚫🚫🚫🚫🚫🚫 🚫🚫🚫🚫 🚫🚫🚫🚫🚫 🚫🚫🚫 — 🚫🚫🚫🚫🚫🚫 node_modules 🚫 .next 🚫🚫 🚫🚫 🚫🚫🚫🚫).

 2. 🚫🚫 🚫🚫🚫 🚫🚫🚫 🚫🚫🚫🚫:

 \`\`\`bash
 npm install
 npm run build # 🚫🚫🚫 🚫🚫🚫🚫🚫 🚫🚫 🚫🚫🚫 🚫🚫🚫
 git add -A
 git status
 git commit -m "🚫🚫🚫🚫🚫🚫🚫🚫 🚫🚫🚫 🚫🚫🚫🚫🚫"
 git push origin main
 \`\`\`

 3. 🚫🚫 🚫🚫🚫 GitLab 🚫🚫 🚫🚫🚫🚫:

 \`\`\`bash
 git remote add gitlab https://gitlab.com/USER/REPO.git # 🚫🚫 🚫🚫 🚫🚫
 git push gitlab main
 \`\`\`

 ## 🚫🚫🚫🚫🚫🚫 🚫🚫🚫🚫

 🚫🚫🚫 \.env\ 🚫🚫 🚫🚫 🚫🚫🚫🚫 🚫🚫🚫 (🚫🚫 🚫🚫🚫🚫 🚫🚫🚫🚫).
 🚫🚫🚫🚫 🚫🚫🚫🚫🚫 🚫🚫🚫 🚫🚫🚫🚫 🚫🚫 \.env.ndcohub.example\ 🚫🚫🚫 🚫🚫🚫.

 🚫🚫🚫🚫 🚫🚫🚫: ${new Date().toISOString()}
 `,
 });

 const zip = createZip(entries);
 const stamp = new Date().toISOString().slice(0, 16).replace(/[:T]/g, "-");

 return new Response(new Uint8Array(zip), {
 headers: {
 "Content-Type": "application/zip",
 "Content-Length": String(zip.length),
 "Content-Disposition": `attachment; filename="namayandeelmi-source-${stamp}.zip"`,
 "Cache-Control": "no-store",
 },
 });
}

@@ src/app/api/targets/route.ts typescript
import { db, dbRetry, dbRetrySafe } from "@db";
import { orders, targets, users } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { bonusKeyOf } from "@lib/defaults";

```

```

import { getProducts } from "@lib/settings-server";
import { todayJalali } from "@lib/jalali";
import { and, eq, SQL } from "drizzle-orm";

export const dynamic = "force-dynamic";

export type TargetProgress = {
 productKey: string;
 productLabel: string;
 quantity: number;
 priceDistributor: number;
 pricePharmacy: number;
 /** 00000 00000000000 00 0000 */
 sold: number;
 /** 00000 00000 0000000000 */
 bonus: number;
 remaining: number;
 percent: number;
 valueDistributor: number;
 valuePharmacy: number;
 soldValueDistributor: number;
 soldValuePharmacy: number;
};

const currentPeriod = () => todayJalali().slice(0, 7);

/**
 * GET /api/targets?userId=&period=
 * 000000 00000 00 0000: 00000 0000 0000 0000000000 0000000000 0 0000 000000
 */
export async function GET(req: Request) {
 await ensureSeed();
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "000000 00000000" }, { status: 401 });

 const url = new URL(req.url);
 const isManager = user.role === "admin" || user.role === "supervisor";
 const scopeId = isManager ? Number(url.searchParams.get("userId")) || user.id : user.id;
 const period = url.searchParams.get("period") ?? currentPeriod();
 const allPeriods = url.searchParams.get("all") === "1";

 const products = await getProducts();

 // 0000000000 0000000000 (0000 0000 + 00000 000000)
 const rows = await dbRetrySafe(
 () => db.select().from(targets).where(eq(targets.userId, scopeId)),
 [],
 "targets:list",
);
 const forPeriod = rows.filter((t) => allPeriods || t.period === period || t.period === "");

 // 0000000000 0000 0000
 const orderRows = await dbRetrySafe(
 () => db.select({ d: orders.dateShamsi, items: orders.items }).from(orders).where(eq(orders.userId, scopeId)),
 [],
 "targets:orders",
);
 const scoped = allPeriods ? orderRows : orderRows.filter((o) => o.d.startsWith(period));

 const sold: Record<string, number> = {};
 const bonusMap: Record<string, number> = {};
 for (const o of scoped) {
 for (const p of products) {
 sold[p.key] = (sold[p.key] ?? 0) + Number(o.items?.[p.key] ?? 0);
 bonusMap[p.key] = (bonusMap[p.key] ?? 0) + Number(o.items?.[bonusKeyOf(p.key)] ?? 0);
 }
 }

 const progress: TargetProgress[] = products.map((p) => {
 const t = forPeriod.find((x) => x.productKey === p.key);
 const quantity = t?.quantity ?? 0;
 const pd = t?.priceDistributor ?? 0;
 const pp = t?.pricePharmacy ?? 0;
 const s = sold[p.key] ?? 0;
 return {
 productKey: p.key,
 productLabel: p.label,
 quantity,
 priceDistributor: pd,
 pricePharmacy: pp,
 sold: s,
 bonus: bonusMap[p.key] ?? 0,
 remaining: Math.max(0, quantity - s),
 percent: quantity > 0 ? Math.min(999, Math.round((s / quantity) * 100)) : 0,
 valueDistributor: quantity * pd,
 };
 });
}

```



```

 valuePharmacy: quantity * pp,
 soldValueDistributor: s * pd,
 soldValuePharmacy: s * pp,
 };
});

const totals = progress.reduce(
 (a, r) => ({
 quantity: a.quantity + r.quantity,
 sold: a.sold + r.sold,
 bonus: a.bonus + r.bonus,
 remaining: a.remaining + r.remaining,
 valueDistributor: a.valueDistributor + r.valueDistributor,
 valuePharmacy: a.valuePharmacy + r.valuePharmacy,
 soldValueDistributor: a.soldValueDistributor + r.soldValueDistributor,
 soldValuePharmacy: a.soldValuePharmacy + r.soldValuePharmacy,
 }),
 {
 quantity: 0,
 sold: 0,
 bonus: 0,
 remaining: 0,
 valueDistributor: 0,
 valuePharmacy: 0,
 soldValueDistributor: 0,
 soldValuePharmacy: 0,
 },
);

return Response.json({
 period,
 userId: scopeId,
 progress,
 totals,
 percent: totals.quantity > 0 ? Math.round((totals.sold / totals.quantity) * 100) : 0,
});
}

/**
 * PUT /api/targets — 创建/更新目标
 * { userId, period, items: [{ productKey, quantity, priceDistributor, pricePharmacy }] }
 */
export async function PUT(req: Request) {
 const user = await getSessionUser();
 if (!user || (user.role !== "admin" && !user.permissions.includes("users"))) {
 return Response.json({ error: "权限不足" }, { status: 401 });
 }
 const b = await req.json().catch(() => ({}));
 const userId = Number(b.userId);
 const period = String(b.period ?? "").slice(0, 7);
 if (!Number.isFinite(userId)) return Response.json({ error: "无效的 ID" }, { status: 400 });

 const target = (await dbRetry(() => db.select().from(users).where(eq(users.id, userId)).limit(1, "targets:user")))[0];
 if (!target) return Response.json({ error: "目标不存在" }, { status: 404 });

 const products = await getProducts();
 const items = Array.isArray(b.items) ? b.items : [];

 for (const it of items) {
 const key = String(it.productKey ?? "");
 const p = products.find((x) => x.key === key);
 if (!p) continue;
 const values = {
 userId,
 repName: target.fullName,
 period,
 productKey: key,
 productLabel: p.label,
 quantity: Math.max(0, Number(it.quantity) || 0),
 priceDistributor: Math.max(0, Number(it.priceDistributor) || 0),
 pricePharmacy: Math.max(0, Number(it.pricePharmacy) || 0),
 updatedAt: new Date(),
 };
 const existing = await dbRetrySafe(
 () =>
 db
 .select()
 .from(targets)
 .where(and(eq(targets.userId, userId), eq(targets.period, period), eq(targets.productKey, key)))
 .limit(1),
 [],
 "targets:find",
);
 if (existing.length) {
 await dbRetry(() => db.update(targets).set(values).where(eq(targets.id, existing[0].id)), "targets:update");
 }
 }
}

```



```

 tripId: row.id,
 lat: Number(b.lat),
 lng: Number(b.lng),
 accuracy: Number(b.accuracy) || null,
 kind: "start",
 note: "□□□□ □□□□",
 });
}
await db.insert(activityLogs).values({
 userId: user.id,
 userName: user.fullName,
 action: "□□□□ □□□□",
 detail: `□□□ #${row.id}`,
});
return Response.json({ trip: row });
}

export async function PATCH(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "□□□□□□ □□□□□□" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 if (!Number.isFinite(id)) return Response.json({ error: "□□□□□ □□□□□□" }, { status: 400 });
 const rows = await db.select().from(trips).where(eq(trips.id, id)).limit(1);
 const trip = rows[0];
 if (!trip || (user.role !== "admin" && trip.userId !== user.id)) {
 return Response.json({ error: "□□□ □□□□ □□□" }, { status: 404 });
 }
 if (b.action === "end") {
 if (Number.isFinite(Number(b.lat)) && Number.isFinite(Number(b.lng))) {
 await db.insert(tripPoints).values({
 tripId: id,
 lat: Number(b.lat),
 lng: Number(b.lng),
 kind: "end",
 note: "□□□□□ □□□□□",
 });
 }
 await db.update(trips).set({ status: "ended", endedAt: new Date() }).where(eq(trips.id, id));
 await db.insert(activityLogs).values({
 userId: user.id,
 userName: user.fullName,
 action: "□□□□□ □□□□□",
 detail: `□□□ #${id}`,
 });
 }
 return Response.json({ ok: true });
}

```

@@ src/app/api/users/route.ts typescript

```

import { db } from "@/db";
import { users } from "@/db/schema";
import { getSessionUser, hashPassword } from "@/lib/auth";
import { ALL_PERMISSION_KEYS } from "@/lib/defaults";
import { asc, eq } from "drizzle-orm";

```

```
export const dynamic = "force-dynamic";
```

```

async function adminGuard() {
 const user = await getSessionUser();
 if (!user) return null;
 if (user.role === "admin" || user.permissions.includes("users")) return user;
 return null;
}

```

```

export async function GET() {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "□□□□□□ □□□□□□" }, { status: 401 });
 const rows = await db
 .select({
 id: users.id,
 username: users.username,
 fullName: users.fullName,
 phone: users.phone,
 role: users.role,
 active: users.active,
 permissions: users.permissions,
 passwordPlain: users.passwordPlain,
 requirePhone: users.requirePhone,
 deviceId: users.deviceId,
 deviceInfo: users.deviceInfo,
 deviceBoundAt: users.deviceBoundAt,
 simMode: users.simMode,
 lastSeenAt: users.lastSeenAt,
 createdAt: users.createdAt,

```

```

 })
 .from(users)
 .orderBy(asc(users.id));
 return Response.json({ rows });
}

export async function POST(req: Request) {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "权限不足 无法操作" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const username = String(b.username ?? "").trim().toLowerCase();
 const password = String(b.password ?? "").trim();
 const fullName = String(b.fullName ?? "").trim();
 if (!username || !password || !fullName) {
 return Response.json({ error: "用户名 密码 或 姓名 不能为空" }, { status: 400 });
 }
 const dup = await db.select().from(users).where(eq(users.username, username)).limit(1);
 if (dup.length) return Response.json({ error: "用户名 密码 或 姓名 已存在" }, { status: 400 });
 const [row] = await db
 .insert(users)
 .values({
 username,
 passwordHash: hashPassword(password),
 passwordPlain: password,
 fullName,
 phone: String(b.phone ?? "").trim(),
 role: String(b.role ?? "rep").slice(0, 40),
 active: b.active !== false,
 requirePhone: b.requirePhone !== false,
 simMode: ["off", "phone", "device", "otp"].includes(b.simMode) ? b.simMode : "device",
 permissions: Array.isArray(b.permissions) && b.permissions.length ? b.permissions : ALL_PERMISSION_KEYS,
 })
 .returning();
 return Response.json({ row: { ...row, passwordHash: undefined } });
}

export async function PATCH(req: Request) {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "权限不足 无法操作" }, { status: 401 });
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 if (!Number.isFinite(id)) return Response.json({ error: "ID 无效" }, { status: 400 });
 const patch: Record<string, unknown> = {};
 if (typeof b.fullName === "string" && b.fullName.trim()) patch.fullName = b.fullName.trim();
 if (typeof b.phone === "string") patch.phone = b.phone.trim();
 if (typeof b.username === "string" && b.username.trim()) patch.username = b.username.trim().toLowerCase();
 if (typeof b.role === "string" && b.role.trim()) patch.role = b.role.trim().slice(0, 40);
 if (typeof b.active === "boolean") patch.active = b.active;
 if (typeof b.requirePhone === "boolean") patch.requirePhone = b.requirePhone;
 if (typeof b.simMode === "string" && ["off", "phone", "device", "otp"].includes(b.simMode)) {
 patch.simMode = b.simMode;
 }
 // 设备信息
 if (b.releaseDevice === true) {
 patch.deviceId = "";
 patch.deviceInfo = "";
 patch.deviceBoundAt = null;
 }
 if (Array.isArray(b.permissions)) patch.permissions = b.permissions;
 if (typeof b.password === "string" && b.password.length >= 4) {
 patch.passwordHash = hashPassword(b.password);
 patch.passwordPlain = b.password;
 }
 if (Object.keys(patch).length === 0) return Response.json({ error: "无有效数据" }, { status: 400 });
 await db.update(users).set(patch).where(eq(users.id, id));
 return Response.json({ ok: true });
}

export async function DELETE(req: Request) {
 const admin = await adminGuard();
 if (!admin) return Response.json({ error: "权限不足 无法操作" }, { status: 401 });
 const id = Number(new URL(req.url).searchParams.get("id"));
 if (!Number.isFinite(id)) return Response.json({ error: "ID 无效" }, { status: 400 });
 if (id === admin.id) return Response.json({ error: "不能删除自己" }, { status: 400 });
 await db.delete(users).where(eq(users.id, id));
 return Response.json({ ok: true });
}

@@ src/app/icons/[file]/route.ts typescript
import { pngResponse, renderIcon } from "@lib/pwa-image";

export const dynamic = "force-static";
export const revalidate = false;

/**

```

```

* 00000000 00000000: 000 0000 00000000 00 public/icons 000000 000000
* (000000 000000 00000000 00 0000 00 000000 0000)0 0000 0000 000000 PNG 00
* 00 0000 00 000 0000 00000000 00 0000 0000 000 00 0000.
*/
export async function GET(_req: Request, ctx: { params: Promise<{ file: string }> }) {
 const { file } = await ctx.params;
 const m = /^(icon|maskable)-(\d{2,4})\.png$/.exec(file);
 if (!m) return new Response("not found", { status: 404 });
 const size = Math.min(1024, Math.max(16, Number(m[2])));
 const bytes = await renderIcon(size, m[1] === "maskable");
 if (!bytes) return new Response("not available", { status: 404 });
 return pngResponse(bytes);
}

@@ src/app/panel/doctors/page.tsx typescript
import RecordScreen from "@components/RecordScreen";

export default function Page() {
 return <RecordScreen type="doctors" />;
}

@@ src/app/panel/home/page.tsx typescript
import HomeScreen from "@components/screens/HomeScreen";
export default function Page() {
 return <HomeScreen />;
}

@@ src/app/panel/leaves/page.tsx typescript
import LeaveScreen from "@components/screens/LeaveScreen";

export default function Page() {
 return <LeaveScreen />;
}

@@ src/app/panel/map/page.tsx typescript
import MapExplorer from "@components/screens/MapExplorer";

export default function Page() {
 return <MapExplorer />;
}

@@ src/app/panel/notifications/page.tsx typescript
import NotificationScreen from "@components/screens/NotificationScreen";

export default function Page() {
 return <NotificationScreen />;
}

@@ src/app/panel/options/page.tsx typescript
import OptionsScreen from "@components/screens/OptionsScreen";
export default function Page() {
 return <OptionsScreen />;
}

@@ src/app/panel/orders/page.tsx typescript
import RecordScreen from "@components/RecordScreen";

export default function Page() {
 return <RecordScreen type="orders" />;
}

@@ src/app/panel/pharmacies/page.tsx typescript
import RecordScreen from "@components/RecordScreen";

export default function Page() {
 return <RecordScreen type="pharmacies" />;
}

@@ src/app/panel/reports/page.tsx typescript
import ReportScreen from "@components/screens/ReportScreen";
export default function Page() {
 return <ReportScreen />;
}

@@ src/app/panel/trip/page.tsx typescript
"use client";

import { useCallback, useEffect, useRef, useState } from "react";
import dynamic from "next/dynamic";
import { Alert, Badge, Button, Card, SectionTitle } from "@components/ui";
import { useConfirm } from "@components/Confirm";
import { tehranTime, toPersianDigits, todayJalali } from "@lib/jalali";
import {
 bufferedCount,
 createTracker,

```

```

 flushPoints,
 formatDistance,
 formatDuration,
 pushLive,
 type TrackPoint,
 type TrackerState,
 } from "@lib/tracker";

const MapBox = dynamic(() => import("@components/MapBox"), { ssr: false });

const LS_TRIP = "sek_trip_id";
const LS_STATE = "sek_trip_state";

type Place = { lat: number; lng: number; label: string; color: string };

export default function TripPage() {
 const [tripId, setTripId] = useState<number | null>(null);
 const [active, setActive] = useState(false);
 const [st, setSt] = useState<TrackerState>({
 lastPoint: null,
 distanceM: 0,
 stopSeconds: 0,
 gpsOn: true,
 pending: 0,
 points: [],
 });
 const [startedAt, setStartedAt] = useState("");
 const [msg, setMsg] = useState<{ kind: "info" | "error" | "success"; text: string } | null>(null);
 const [places, setPlaces] = useState<Place[]>([]);
 const trackerRef = useRef<ReturnType<typeof createTracker> | null>(null);
 const confirm = useConfirm();

 /* ----- 00000000 000000000000 000000 0 0000 0000 0000 0000 0000 ----- */
 const loadPlaces = useCallback(async () => {
 const out: Place[] = [];
 const get = async (url: string) => {
 const r = await fetch(url, { cache: "no-store" }).catch(() => null);
 return r?.ok ? ((await r.json()).rows ?? []) : [];
 };
 const [ph, dr, hm] = await Promise.all([
 get("/api/records/pharmacies?limit=300"),
 get("/api/records/doctors?limit=300"),
 get("/api/homes"),
]);
 for (const p of ph) if (p.lat && p.lng) out.push({ lat: p.lat, lng: p.lng, label: `□ ${p.name}`, color: "#0f766e" });
 };
 for (const d of dr) if (d.lat && d.lng) out.push({ lat: d.lat, lng: d.lng, label: `□ ${d.name}`, color: "#0369a1" });
 for (const h of hm) if (h.lat && h.lng) out.push({ lat: h.lat, lng: h.lng, label: `□ ${h.title}`, color: "#7c3aed" });
 setPlaces(out);
 }, []);

 /* ----- 00000000 000 00 000 0000 00 00 0000 000000 ----- */
 useEffect(() => {
 loadPlaces();
 const id = Number(localStorage.getItem(LS_TRIP));
 if (id) {
 setTripId(id);
 setActive(true);
 try {
 const saved = JSON.parse(localStorage.getItem(LS_STATE) ?? "{}");
 if (saved.startedAt) setStartedAt(saved.startedAt);
 } catch {
 /* ignore */
 }
 }
 bufferedCount().then((n) => setSt((s) => ({ ...s, pending: n })));
 });

 // 00000 0000 000000000000 000000 000000 00000000
 const onOnline = () => void flushPoints();
 window.addEventListener("online", onOnline);
 return () => window.removeEventListener("online", onOnline);
}, [loadPlaces]);

/* ----- 00000000000 000000 0000 000 0000 000 ----- */
useEffect(() => {
 if (!active || !tripId) return;
 const tracker = createTracker({ tripId, onUpdate: setSt });
 trackerRef.current = tracker;
 const ok = tracker.start();
 if (!ok) setMsg({ kind: "error", text: "00000000000 000 000000 00 000000 0000" });
 return () => {
 void tracker.stop("0000 0000");
 trackerRef.current = null;
 };
}, [active, tripId]);

```

```

 });
}, [active, tripId]);

const currentPosition = () =>
 new Promise<{ lat: number; lng: number; accuracy?: number } | null>((resolve) => {
 if (!navigator.geolocation) return resolve(null);
 navigator.geolocation.getCurrentPosition(
 (p) => resolve({ lat: p.coords.latitude, lng: p.coords.longitude, accuracy: p.coords.accuracy }),
 () => resolve(null),
 { enableHighAccuracy: true, timeout: 15000, maximumAge: 30000 },
);
 });

const startTrip = async () => {
 if (!(await confirm({ title: "شروع مسافرت", message: "آیا می‌خواهید مسافرت خود را شروع کنید؟", confirmText: "بله" }))) return;
 setMsg({ kind: "info", text: "مسافرت شما با موفقیت شروع شد." });
 const pos = await currentPosition();

 const res = await fetch("/api/trips", {
 method: "POST",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify(pos ?? {}),
 }).catch(() => null);

 // اگر پاسخ موفق باشد، ID مسافرت را از پاسخ دریافت می‌کنیم
 let id: number | null = null;
 if (res?.ok) {
 const d = await res.json();
 id = d.trip?.id ?? null;
 }
 if (!id) {
 setMsg({
 kind: "info",
 text: "خطای سرور: ID مسافرت دریافت نشد. لطفاً دوباره تلاش کنید.",
 });
 id = -Date.now(); // ID موقت برای ذخیره در LocalStorage
 } else {
 setMsg({ kind: "success", text: "مسافرت شما با موفقیت ثبت شد." });
 }

 localStorage.setItem(LS_TRIP, String(id));
 localStorage.setItem(LS_STATE, JSON.stringify({ startedAt: tehranTime(new Date()) }));
 setStartedAt(tehranTime(new Date()));
 setTripId(id);
 setActive(true);
 if (pos) void pushLive({ ...pos, gpsOn: true, tripId: id });
};

const markPause = async () => {
 if (!(await confirm({ title: "توقف مسافرت", message: "آیا می‌خواهید مسافرت خود را متوقف کنید؟", confirmText: "بله" }))) return;
 await trackerRef.current?.markPause(`${tehranTime(new Date())}`);
 setMsg({ kind: "info", text: `مسافرت شما در ${tehranTime(new Date())} متوقف شد.` });
};

const endTrip = async () => {
 if (
 !(await confirm({
 title: "پایان مسافرت",
 message: `مسافت مسافرت شما: ${formatDistance(st.distanceM)} متر. مدت زمان مسافرت: ${formatDuration(st.stopSeconds)} دقیقه. آیا می‌خواهید مسافرت خود را به پایان برسانید؟`,
 confirmText: "بله",
 danger: true,
 })))
)
 return;

 await trackerRef.current?.stop("پایان مسافرت");
 const pos = await currentPosition();
 if (tripId && tripId > 0) {
 await fetch("/api/trips", {
 method: "PATCH",
 headers: { "Content-Type": "application/json" },
 body: JSON.stringify({ id: tripId, action: "end", ...(pos ?? {}) }),
 }).catch(() => null);
 }
 localStorage.removeItem(LS_TRIP);
 localStorage.removeItem(LS_STATE);
 setActive(false);
 setTripId(null);
 setSt({ lastPoint: null, distanceM: 0, stopSeconds: 0, gpsOn: true, pending: 0, points: [] });
 setMsg({ kind: "success", text: "مسافرت شما به پایان رسید." });
};

const pauses = st.points.filter((p: TrackPoint) => p.kind === "pause");
const path = st.points.map((p) => ({ lat: p.lat, lng: p.lng }));

```

```

return (
 <div className="space-y-4">
 <SectionTitle icon="📍">📍📍📍📍 📍📍📍📍 — {toPersianDigits(todayJalali())}</SectionTitle>
 {msg ? <Alert kind={msg.kind}>{msg.text}</Alert> : null}

 <Card>
 <div className="flex flex-wrap items-center gap-2">
 {!active ? (
 <Button onClick={startTrip} className="flex-1 sm:flex-none">
 📍📍📍📍 📍📍📍📍
 </Button>
) : (
 <Button variant="soft" onClick={markPause}>
 📍📍📍 📍📍📍
 </Button>
 <Button variant="danger" onClick={endTrip}>
 📍📍📍📍 📍📍📍📍
 </Button>
)}
 </div>
 <div className="flex-1" />
 <Badge tone={active ? "green" : "slate"}>{active ? "📍📍📍📍 📍📍📍📍" : "📍📍📍📍📍📍"}</Badge>
 <Badge tone={st.gpsOn ? "green" : "amber"}>{st.gpsOn ? "GPS 📍📍📍📍" : "GPS 📍📍📍📍 — 📍📍📍📍 📍📍📍📍"}</Badge>
 {st.pending > 0 ? <Badge tone="amber">{toPersianDigits(st.pending)} 📍📍📍 📍📍📍</Badge> : null}
 </div>

 {active ? (
 <div className="mt-3 grid grid-cols-2 gap-2 sm:grid-cols-4">
 {[
 ["📍📍📍📍 📍📍📍📍", formatDistance(st.distanceM), "bg-teal-50 text-teal-700"],
 ["📍📍📍 📍📍📍📍📍", formatDuration(st.stopSeconds), "bg-amber-50 text-amber-700"],
 ["📍📍📍📍 📍📍📍📍📍", toPersianDigits(st.points.length), "bg-slate-50 text-slate-700"],
 ["📍📍📍📍 📍📍📍", startedAt || "—", "bg-sky-50 text-sky-700"],
].map(([l, v, c]) => (
 <div key={l} className={`rounded-xl p-2.5 text-center ring-1 ring-slate-200 ${c}`}>
 <div className="text-sm font-black">{v}</div>
 <div className="text-[10px] font-bold opacity-70">{l}</div>
 </div>
))}
 </div>
) : null}

 {!st.gpsOn && active ? (
 <p className="mt-2 rounded-lg bg-amber-50 px-3 py-2 text-[11px] text-amber-800">
 📍📍📍📍 GPS 📍📍📍 📍📍 📍📍 📍📍📍📍📍📍📍 📍📍📍📍 📍📍 — 📍📍📍📍 📍📍📍📍 📍📍📍 📍📍📍📍📍 📍📍📍📍 📍📍📍 📍📍📍📍📍.
 </p>
) : null}
 </Card>

 <Card>
 <div className="mb-2 flex flex-wrap items-center justify-between gap-2">
 <h3 className="text-sm font-bold text-slate-700">
 {active ? "📍📍📍 📍📍📍 📍📍" : "📍📍📍📍📍📍📍📍 📍📍📍📍 📍📍📍 📍📍📍 📍📍📍"}
 </h3>

 📍📍📍📍📍📍 · 📍📍📍📍 · 📍📍📍📍 · 📍📍📍📍 · 📍📍📍📍

 </div>
 <MapBox
 height={380}
 follow={active}
 path={path}
 points={[
 ...places,
 ...st.points
].filter((p) => p.kind !== "move")
 .map((p) => ({
 lat: p.lat,
 lng: p.lng,
 label: p.note || (p.kind === "start" ? "📍📍📍 📍📍📍" : "📍📍📍📍"),
 color: p.kind === "pause" ? "#f59e0b" : p.kind === "end" ? "#e11d48" : "#16a34a",
 })),
]}
 />
 {pauses.length ? (
 <ul className="mt-3 space-y-1 text-xs text-slate-600">
 {pauses.map((p, i) => (
 <li key={i} className="rounded-lg bg-amber-50 px-3 py-1.5">
 📍 {p.note} — {tehranTime(p.recordedAt)}

))}

) : null}
)

```



```

</Card>

{!active ? (
 <Alert kind="info">
 00 000 «0000 00000»0 0000 000 000 000000. 000 0000000 000 000 00 GPS 0000 00000 00000000 000 0000 000000
 000000 0 000000 00000000 0000000 000000 00000000 — 0000 000 000000.
 </Alert>
) : null}
</div>
);
}

@@ src/app/screenshots/[file]/route.ts typescript
import { pngResponse, renderScreenshot } from "@lib/pwa-image";

export const dynamic = "force-static";
export const revalidate = false;

/** 00000000 0000000000000000 — 0000 00 0000 00 0000 SVG */
export async function GET(_req: Request, ctx: { params: Promise<{ file: string }> }) {
 const { file } = await ctx.params;
 if (!/^[\w-]+\.(png$/.test(file)) return new Response("not found", { status: 404 });
 const bytes = await renderScreenshot(file.replace(/\.png$/, ""));
 if (!bytes) return new Response("not available", { status: 404 });
 return pngResponse(bytes);
}

@@ src/app/admin/records/[type]/page.tsx typescript
import { notFound } from "next/navigation";
import RecordScreen, { type RecordType } from "@components/RecordScreen";

const TYPES = ["pharmacies", "doctors", "orders"] as const;

/** 000000000 00 00 0000 00 0000 0000 0 0000000 0000000 000 0000 */
export function generateStaticParams() {
 return TYPES.map((type) => ({ type }));
}

export const dynamicParams = false;

export default async function AdminRecordsPage({ params }: { params: Promise<{ type: string }> }) {
 const { type } = await params;
 if (!TYPES.includes(type as RecordType)) notFound();
 return <RecordScreen type={type as RecordType} isAdmin />;
}

@@ src/app/api/attachments/[id]/route.ts typescript
import { db } from "@db";
import { attachments } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

/** 000000/0000000 0000. 0000 0000000 0000 0000 0000 0000000 */
export async function GET(req: Request, ctx: { params: Promise<{ id: string }> }) {
 const user = await getSessionUser();
 const { id } = await ctx.params;
 const numId = Number(id);
 if (!Number.isFinite(numId)) return new Response("not found", { status: 404 });

 const rows = await db.select().from(attachments).where(eq(attachments.id, numId)).limit(1);
 const f = rows[0];
 if (!f) return new Response("not found", { status: 404 });

 const isAdmin = user?.role === "admin" || user?.role === "supervisor";
 if (!user || (!isAdmin && f.userId !== user.id)) return new Response("forbidden", { status: 403 });

 const download = new URL(req.url).searchParams.get("download") === "1";
 const buffer = Buffer.from(f.data, "base64");
 const bytes = new Uint8Array(buffer);
 return new Response(bytes, {
 headers: {
 "Content-Type": f.mimeType,
 "Content-Length": String(bytes.length),
 "Content-Disposition": `${download ? "attachment" : "inline"}; filename="${encodeURIComponent(f.fileName)}"`,
 "Cache-Control": "private, max-age=600",
 },
 });
}

@@ src/app/api/auth/check-username/route.ts typescript
import { db } from "@db";
import { roles, users } from "@db/schema";
import { ensureSeed } from "@lib/bootstrap";

```

```

import { eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

/** 注册接口 注册成功返回 201 注册失败返回 400 注册失败原因返回 400 注册失败原因返回 400 */
export async function POST(req: Request) {
 await ensureSeed();
 const b = await req.json().catch(() => ({}));
 const username = String(b.username ?? "").trim().toLowerCase();
 if (!username) return Response.json({ exists: false });
 const rows = await db
 .select({
 role: users.role,
 requirePhone: users.requirePhone,
 phone: users.phone,
 deviceId: users.deviceId,
 })
 .from(users)
 .where(eq(users.username, username))
 .limit(1);
 const u = rows[0];
 if (!u) return Response.json({ exists: false });
 let base: string = u.role;
 if (!["admin", "supervisor", "rep"].includes(u.role)) {
 const r = (await db.select().from(roles).where(eq(roles.key, u.role)).limit(1))[0];
 base = r?.base ?? "rep";
 }
 const digits = String(u.phone ?? "").replace(/\D/g, "");
 // 手机号位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数
 const phoneMask = digits.length >= 8 ? `${digits.slice(0, 4)}***${digits.slice(-4)}` : "";
 return Response.json({
 exists: true,
 kind: base === "rep" ? "rep" : "admin",
 requirePhone: u.requirePhone,
 hasPhone: digits.length >= 10,
 phoneMask,
 deviceBound: Boolean(u.deviceId),
 });
}

@@ src/app/api/auth/forgot/route.ts typescript
import { db } from "@/db";
import { users } from "@/db/schema";
import { ensureSeed } from "@/lib/bootstrap";
import { notify } from "@/lib/notify";
import { eq } from "drizzle-orm";

export const dynamic = "force-dynamic";

/** 忘记密码接口 忘记密码成功返回 201 忘记密码失败返回 400 忘记密码失败原因返回 400 */
export async function POST(req: Request) {
 await ensureSeed();
 const b = await req.json().catch(() => ({}));
 const username = String(b.username ?? "").trim().toLowerCase();
 const phone = String(b.phone ?? "").replace(/\D/g, "");
 if (!username) return Response.json({ error: "用户名不能为空" }, { status: 400 });

 const rows = await db.select().from(users).where(eq(users.username, username)).limit(1);
 const u = rows[0];
 // 手机号位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数 11 位数
 const done = Response.json({
 ok: true,
 message:
 "密码重置成功 请按照短信提示操作 重置密码 重置密码 重置密码 重置密码 重置密码 重置密码 重置密码 重置密码 重置密码 重置密码 重置密码",
 });
 if (!u) return done;

 for (const role of ["admin", "supervisor"] as const) {
 await notify({
 toRole: role,
 fromName: u.fullName,
 kind: "password",
 title: `密码重置成功 请按照短信提示操作 — ${u.fullName}`,
 body: `密码重置成功: ${u.username} 重置密码 重置密码: ${phone || "—"} 重置密码 重置密码: ${u.phone || "—"} 重置密码 重置密码`,
 link: "/admin/users",
 });
 }
 return done;
}

@@ src/app/api/auth/login/route.ts typescript
import { db, dbRetry } from "@/db";
import { activityLogs, roles, users } from "@/db/schema";

```

```

import { SESSION_COOKIE, createToken, verifyPassword } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { eq } from "drizzle-orm";
import { cookies } from "next/headers";

export const dynamic = "force-dynamic";

function normalizePhone(p: string) {
 return String(p ?? "")
 .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
 .replace(/D/g, "")
 .replace(/^0098/, "0")
 .replace(/^98/, "0");
}

export async function POST(req: Request) {
 await ensureSeed();
 const body = await req.json().catch(() => ({}));
 const username = String(body.username ?? "").trim().toLowerCase();
 const password = String(body.password ?? "");
 const phone = normalizePhone(body.phone ?? "");
 const mode = body.mode === "admin" ? "admin" : "rep"; // 00 0000000000 00 0000 0000
 const remember = body.remember === true;
 const deviceOnline = body.simActive !== false;

 if (!username || !password) {
 return Response.json({ error: "0000 0000000 0 000 0000 000000 000", }, { status: 400 });
 }

 let rows: (typeof users.$inferSelect)[] = [];
 try {
 rows = await dbRetry(() => db.select().from(users).where(eq(users.username, username)).limit(1), "login:user");
 } catch (err) {
 console.error("login db error", err);
 return Response.json(
 { error: "0 000000 00 000000 0000 000000 000. 000000 DATABASE_URL 00 000000 0000.", },
 { status: 500 },
);
 }

 const user = rows[0];
 if (!user || !verifyPassword(password, user.passwordHash)) {
 return Response.json({ error: "0000 000000 00 000 0000 000000 000", }, { status: 401 });
 }
 if (!user.active) return Response.json({ error: "00000 0000000 00000000 000", }, { status: 403 });

 // 0 000000 0000 0000 0000 0 00000000
 let baseRole: string = user.role;
 if (!["admin", "supervisor", "rep"].includes(user.role)) {
 try {
 const r = (await dbRetry(() => db.select().from(roles).where(eq(roles.key, user.role)).limit(1), "login:role"))[0];
 baseRole = r?.base ?? "rep";
 } catch {
 baseRole = "rep";
 }
 }
 const isManagerAccount = baseRole === "admin" || baseRole === "supervisor";
 if (mode === "admin" && !isManagerAccount) {
 return Response.json(
 { error: "0 000 0000 0000000 00000000 000. 00000 00 00 «0000 00000000 0000» 0000 0000.", },
 { status: 403 },
);
 }
 if (mode === "rep" && isManagerAccount) {
 return Response.json(
 { error: "0 000 0000 0000/0000000 000. 00000 00 00 «0000 0000» 0000 0000.", },
 { status: 403 },
);
 }

 /* =====
 * 000000 0000000000 — 00000 0000 00 0000 0000 00 000000 0000000 0000000:
 * off → 0000 000 000000
 * phone → 000 000000 00000000 0000 00 000000 00000000 000 0000
 * device → 00000 00 0000000 0000 00 00 0000 000 0000000 (00000000)
 * otp → 000000000000: 00000 0000 000000000 00 00 0000000
 * ===== */
 const simMode = isManagerAccount ? "off" : (user.simMode ?? "device");
 const deviceId = String(body.deviceId ?? "").replace(/[^a-zA-Z0-9]/g, "").slice(0, 80);

 if (simMode !== "off" && user.requirePhone) {
 if (!deviceOnline) {
 return Response.json(
 { error: "00 000000 00 0000 0000 0000. 000000000 00 00000000 00 0000 0000.", simError: true },
);
 }
 }

```

```

 { status: 403 },
);
}
if (!/^0\d{10}$/.test(phone)) {
 return Response.json(
 { error: "☎️ 手机号码 00000 0000 0000 00 00 0000 0000 0000", simError: true },
 { status: 400 },
);
}
const registered = normalizePhone(user.phone);
if (registered && phone !== registered) {
 return Response.json(
 {
 error: "☎️ 手机号码 00000000: 00000 0000000 0000 0000 0000 000 «${registered.slice(0, 4)}***${registered.slice(-4)}» 0000 0000 «${phone.slice(0, 4)}***${phone.slice(-4)}» 0000 0000.",
 simError: true,
 reason: "phone-mismatch",
 },
 { status: 403 },
);
}
if (body.simActiveOnDevice === false) {
 return Response.json(
 { error: "☎️ 000000000 000000 0000 0000 00000000 000.", simError: true, reason: "no-sim" },
 { status: 403 },
);
}
}

if ((simMode === "device" || simMode === "otp") && !isManagerAccount) {
 if (!deviceId) {
 return Response.json(
 { error: "☎️ 00000 0000000 0000000 000. 00000 0000000 00 0000 0000 00 00 0000 0000000 0000 0000.", simError: true },
 { status: 400 },
);
 }
 if (user.deviceId && user.deviceId !== deviceId) {
 return Response.json(
 {
 error:
 "☎️ 000 0000 00 0000 000000 000 0000 000. 000 00000000 000 0000 0000 0000 0000 00 «00000 0000000» 000000 000000 00 0000 0000 00 0000 00000000 0000 00 0000 000.",
 needOtp: true,
 simError: true,
 reason: "device-mismatch",
 },
 { status: 403 },
);
 }
}
if (!user.deviceId) {
 if (simMode === "otp") {
 return Response.json(
 {
 error: "☎️ 0000 00000000 00 0000 000000000 00 0000 000000 00 00000 0000000 0000 000.",
 needOtp: true,
 reason: "first-device",
 },
 { status: 403 },
);
 }
}
// 0000 device: 00000 0000 00000000 0000000 000 0000000
await db
 .update(users)
 .set({
 deviceId,
 deviceInfo: String(body.deviceInfo ?? "").slice(0, 200),
 deviceBoundAt: new Date(),
 })
 .where(eq(users.id, user.id));
await db
 .insert(activityLogs)
 .values({
 userId: user.id,
 userName: user.fullName,
 action: "☎️☎️ ☎️☎️☎️",
 detail: `${String(body.deviceInfo ?? "").slice(0, 120)} | ${phone}`,
 })
 .catch(() => undefined);
}
}

const token = createToken(user.id);
// 0000 000000 00 0000000 00 0000000 00000000 (00 000 0000000 0000000) 00 000000 0000 0000.
// 0000 «0000 00 0000 000000» 00000000 00000 0000000 0000 0 00 0000 00000000 0000 00000000.
{

```

```

const store = await cookies();
store.set(SESSION_COOKIE, token, {
 httpOnly: true,
 sameSite: "lax",
 secure: process.env.NODE_ENV === "production",
 path: "/",
 ...(remember ? { maxAge: 60 * 60 * 24 * 30 } : {}),
});
}

await db.update(users).set({ lastSeenAt: new Date() }).where(eq(users.id, user.id));
await db
 .insert(activityLogs)
 .values({ userId: user.id, userName: user.fullName, action: "登录成功", detail: phone || "手机号" })
 .catch(() => undefined);

return Response.json({
 token,
 user: {
 id: user.id,
 username: user.username,
 fullName: user.fullName,
 role: baseRole,
 roleKey: user.role,
 roleLabel: baseRole === "admin" ? "系统管理员" : baseRole === "supervisor" ? "系统监督员" : "系统普通用户",
 phone: user.phone,
 requirePhone: user.requirePhone,
 permissions: user.permissions,
 },
});
}

```

```

@@ src/app/api/auth/logout/route.ts typescript
import { SESSION_COOKIE } from "@lib/auth";
import { cookies } from "next/headers";

```

```
export const dynamic = "force-dynamic";
```

```

export async function POST() {
 const store = await cookies();
 store.delete(SESSION_COOKIE);
 return Response.json({ ok: true });
}

```

```

@@ src/app/api/auth/me/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";

```

```
export const dynamic = "force-dynamic";
```

```

export async function GET() {
 await ensureSeed();
 const user = await getSessionUser();
 if (!user) return Response.json({ user: null }, { status: 401 });
 return Response.json({ user });
}

```

```

@@ src/app/api/auth/otp/route.ts typescript
import { db } from "@db";
import { activityLogs, users } from "@db/schema";
import { SESSION_COOKIE, createToken, verifyPassword } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { getSmsConfig, issueOtp, verifyOtp } from "@lib/otp";
import { notify } from "@lib/notify";
import { eq } from "drizzle-orm";
import { cookies } from "next/headers";

```

```
export const dynamic = "force-dynamic";
```

```

function normalizePhone(p: string) {
 return String(p ?? "")
 .replace(/[\u06F0-\u06F9]/g, (c) => String(c.charCodeAt(0) - 0x06f0))
 .replace(/D/g, "")
 .replace(/^0098/, "0")
 .replace(/^98/, "0");
}

```

```

/**
 * 登录成功
 * action=request → 系统普通用户 系统监督员 系统管理员
 * action=verify → 系统普通用户 系统监督员 系统管理员
 */

```

```

export async function POST(req: Request) {
 await ensureSeed();
 const b = await req.json().catch(() => ({}));
}

```

```

const action = String(b.action ?? "request");
const username = String(b.username ?? "").trim().toLowerCase();
const password = String(b.password ?? "");
const deviceId = String(b.deviceId ?? "").replace(/[^a-zA-Z0-9]/g, "").slice(0, 80);

const rows = await db.select().from(users).where(eq(users.username, username)).limit(1);
const user = rows[0];
if (!user || !verifyPassword(password, user.passwordHash)) {
 return Response.json({ error: "用户名或密码错误", { status: 401 }});
}
if (!user.active) return Response.json({ error: "用户未激活", { status: 403 }});

const registered = normalizePhone(user.phone);
if (!registered) {
 return Response.json(
 { error: "手机号格式不正确", { status: 400 } },
);
}

if (action === "request") {
 const r = await issueOtp(user.id, registered, deviceId);
 const cfg = await getSmsConfig();

 if (!r.sent) {
 // 发送短信验证码
 await notify({
 toRole: "admin",
 fromName: user.fullName,
 kind: "otp",
 title: `验证码 ${r.code}`,
 body: `验证码: ${r.code}，有效期: ${r.validity}分钟。`,
 link: "/admin/users",
 });
 }

 return Response.json({
 ok: true,
 sent: r.sent,
 channel: r.channel,
 masked: `${registered.slice(0, 4)}***${registered.slice(-4)}`,
 message: r.sent
 ? `验证码 ${r.code} 已发送。`
 : `验证码 ${r.code} 未发送。`;
 });
}

if (action === "verify") {
 const v = await verifyOtp(user.id, registered, String(b.code ?? ""));
 if (!v.ok) return Response.json({ error: v.error }, { status: 400 });
}

// 更新设备信息
await db
 .update(users)
 .set({
 deviceId,
 deviceInfo: String(b.deviceInfo ?? "").slice(0, 200),
 deviceBoundAt: new Date(),
 lastSeenAt: new Date(),
 })
 .where(eq(users.id, user.id));

await db
 .insert(activityLogs)
 .values({
 userId: user.id,
 userName: user.fullName,
 action: "登录成功",
 detail: `${registered} | ${String(b.deviceInfo ?? "").slice(0, 100)}`,
 })
 .catch(() => undefined);

const token = createToken(user.id);
{
 const store = await cookies();
 store.set(SESSION_COOKIE, token, {
 httpOnly: true,
 sameSite: "lax",
 secure: process.env.NODE_ENV === "production",
 path: "/",
 ...(b.remember !== false ? { maxAge: 60 * 60 * 24 * 30 } : {}),
 });
}

```

```

 }

 return Response.json({
 ok: true,
 token,
 user: {
 id: user.id,
 username: user.username,
 fullName: user.fullName,
 role: user.role,
 roleKey: user.role,
 phone: user.phone,
 requirePhone: user.requirePhone,
 permissions: user.permissions,
 },
 });
 }
}

return Response.json({ error: "权限不足", { status: 400 }});
}

@@ src/app/api/backup/restore/route.ts typescript
import { db } from "@db";
import {
 activityLogs,
 attachments,
 doctors,
 homes,
 leaves,
 messageLogs,
 messengers,
 notifications,
 options,
 orders,
 pharmacies,
 roles,
 settings,
 tripPoints,
 trips,
 users,
} from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { sql } from "drizzle-orm";

export const dynamic = "force-dynamic";
export const maxDuration = 60;

type Row = Record<string, unknown>;

/** 清理数据库数据，将日期字段转换为 Date 类型 */
function clean(rows: unknown, dateFields: string[] = []): Row[] {
 if (!Array.isArray(rows)) return [];
 return rows.map((r) => {
 const o = { ...(r as Row) };
 for (const f of dateFields) {
 if (o[f]) o[f] = new Date(String(o[f]));
 else if (o[f] === null) o[f] = null;
 }
 return o;
 });
}

async function resetSeq(table: string) {
 try {
 await db.execute(
 sql.raw(
 `SELECT setval(pg_get_serial_sequence('${table}', 'id'), COALESCE((SELECT MAX(id) FROM ${table}), 1), true)`
),
);
 } catch {
 /* ignore */
 }
}

export async function POST(req: Request) {
 await ensureSeed();
 const user = await getSessionUser();
 if (!user || user.role !== "admin") return Response.json({ error: "权限不足", { status: 401 }});

 const body = await req.json().catch(() => null);
 const payload = body?.data ? body : body?.backup;
 const data = (payload?.data ?? {}) as Record<string, unknown>;
 if (!data || typeof data !== "object" || !Array.isArray(data.users)) {
 return Response.json({ error: "数据格式错误", { status: 400 }});
 }
}

```

```

}

const mode = body?.mode === "merge" ? "merge" : "replace";
const report: Record<string, number> = {};

try {
 if (mode === "replace") {
 // 删除所有数据
 for (const t of [
 "attachments",
 "trip_points",
 "trips",
 "orders",
 "doctors",
 "pharmacies",
 "homes",
 "leaves",
 "notifications",
 "message_logs",
 "messengers",
 "activity_logs",
 "options",
 "settings",
 "roles",
 "users",
]) {
 await db.execute(sql.raw(`DELETE FROM ${t}`));
 }
 }

 const put = async (
 // eslint-disable-next-line @typescript-eslint/no-explicit-any
 table: any,
 key: string,
 tableName: string,
 dates: string[] = [],
) => {
 const rows = clean(data[key], dates);
 if (!rows.length) return;
 for (let i = 0; i < rows.length; i += 200) {
 const chunk = rows.slice(i, i + 200);
 await db.insert(table).values(chunk).onConflictDoNothing();
 }
 report[key] = rows.length;
 await resetSeq(tableName);
 };

 await put(users, "users", "users", ["createdAt", "lastSeenAt", "deviceBoundAt"]);
 await put(roles, "roles", "roles", ["createdAt"]);
 await put(settings, "settings", "settings", ["updatedAt"]);
 await put(options, "options", "options", ["createdAt"]);
 await put(pharmacies, "pharmacies", "pharmacies", ["createdAt"]);
 await put(doctors, "doctors", "doctors", ["createdAt"]);
 await put(orders, "orders", "orders", ["createdAt"]);
 await put(homes, "homes", "homes", ["updatedAt"]);
 await put(leaves, "leaves", "leaves", ["createdAt"]);
 await put(trips, "trips", "trips", ["startedAt", "endedAt"]);
 await put(tripPoints, "tripPoints", "trip_points", ["recordedAt"]);
 await put(messengers, "messengers", "messengers", ["createdAt"]);
 await put(messageLogs, "messageLogs", "message_logs", ["createdAt"]);
 await put(notifications, "notifications", "notifications", ["createdAt", "readAt"]);
 await put(activityLogs, "activityLogs", "activity_logs", ["createdAt"]);

 // 插入附件
 const atts = clean(data.attachments, ["createdAt"]).filter((a) => typeof a.data === "string" && a.data);
 if (atts.length) {
 for (let i = 0; i < atts.length; i += 50) {
 await db
 .insert(attachments)
 .values(atts.slice(i, i + 50) as (typeof attachments.$inferInsert)[])
 .onConflictDoNothing();
 }
 report.attachments = atts.length;
 await resetSeq("attachments");
 }

 await db
 .insert(activityLogs)
 .values({
 userId: user.id,
 userName: user.fullName,
 action: "用户活动",
 detail: `${mode === "replace" ? "数据重置" : "数据更新"}: ${JSON.stringify(report)}.slice(0, 400)`,
 })
 .catch(() => undefined);

```





```

/* ----- 0) 0000 00000 → 0000 00000 000000 ----- */
if (!phone || !/^0\d{10}$/.test(phone)) {
 return Response.json(
 {
 error:
 "00000 00000000 000 0000 000000 000 (0000000000 000000 0000000 00000 00 000 00000000 00000 00000000). 00000 00 00 0
00000 0000 0000.",
 code: "SIM_NUMBER_UNAVAILABLE",
 needOtp: true,
 simCarrier,
 },
 { status: 422 },
);
}

/* ----- 0) 000000 000000 00 000 00000 ----- */
const rows = await dbRetry(() => db.select().from(users).where(eq(users.phone, phone)).limit(1, "mobile:findUser"));
let found: typeof users.$inferSelect | undefined = rows[0];

if (!found) {
 // 000000 0000: 000000000000 00 00 0000 0000000 000000 000000000
 const all = await dbRetry(() => db.select().from(users), "mobile:allUsers");
 found = all.find((u) => normalizePhone(u.phone) === phone);
}

if (!found) {
 return Response.json(
 { error: "0000 000000 000000 00 000000 0000 0000 0000. 00 00000 00000 0000000.", code: "PHONE_NOT_REGISTERED" },
 { status: 404 },
);
}
const user = found;
if (!user.active) {
 return Response.json({ error: "00000 00000000 00000000 0000", code: "USER_INACTIVE" }, { status: 403 });
}

/* ----- 0) 000 0000 ----- */
let baseRole: string = user.role;
if (!["admin", "supervisor", "rep"].includes(user.role)) {
 const r = (await dbRetry(() => db.select().from(roles).where(eq(roles.key, user.role)).limit(1, "mobile:role")))[0];
 baseRole = r?.base ?? "rep";
}
const isManager = baseRole === "admin" || baseRole === "supervisor";

/* ----- 0) 000 000000 000000 00000 000 000000 0000 0000 ----- */
const isPhoneVerificationRequired = user.requirePhone && !isManager;
const simMode = isManager ? "off" : (user.simMode ?? "device");

if (!isPhoneVerificationRequired || simMode === "off") {
 return Response.json(
 {
 error: "00000 000 000000 00000 00 0000 0000000 0 000 00000 000000 0000 0000.",
 code: "PASSWORD_LOGIN_REQUIRED",
 loginUrl: "/login",
 },
 { status: 403 },
);
}

/* ----- 0) 000 000000 ----- */
if (user.deviceId && user.deviceId !== deviceId) {
 return Response.json(
 {
 error:
 "0000 00000 00 00000 000000 0000 0000 0000. 0000 00000 0000 00 0000 0000000000 00 00 00000000 000000 0000 00 00 00000 00000000 0
00000 00 00000 0000.",
 code: "DEVICE_MISMATCH",
 needOtp: true,
 },
 { status: 403 },
);
}

/* ----- 0) 000000 000000 0000000000 0000 00000 0000 ----- */
if (user.simFingerprint && user.simFingerprint !== simFp) {
 return Response.json(
 {
 error: "0000000000 0000 00000 000000 00000 0000. 00000 00000000 0000000 0000000 00000 0000.",
 code: "SIM_CHANGED",
 needOtp: true,
 },
 { status: 403 },
);
}

```

```

/* ----- 0) 0000 0000000000: 00000 00 000000 ----- */
if (simMode === "otp" && !user.deviceId) {
 const r = await issueOtp(user.id, phone, deviceId);
 if (!r.sent) {
 await notify({
 toRole: "admin",
 fromName: user.fullName,
 kind: "otp",
 title: `0 00 000000 0000000000 0000 ${user.fullName}`,
 body: `00: ${r.code}\n000000: ${phone}\n00000000: 0 000000`,
 link: "/admin/users",
 });
 }
}
return Response.json(
 {
 error: "0000 000000 0000 000 000 000000 00 000000 00000000 0000 000.",
 code: "OTP_REQUIRED",
 needOtp: true,
 masked: maskPhone(phone),
 sent: r.sent,
 },
 { status: 403 },
);
}

/* ----- 0) 0000 0000: 000 0000000 + 000 000 000000000 ----- */
await dbRetry(
 () =>
 db
 .update(users)
 .set({
 deviceId,
 deviceInfo,
 deviceBoundAt: user.deviceBoundAt ?? new Date(),
 simFingerprint: simFp || user.simFingerprint,
 simCarrier: simCarrier || user.simCarrier,
 simVerifiedAt: new Date(),
 lastSeenAt: new Date(),
 })
 .where(eq(users.id, user.id)),
 "mobile:bind",
);

await db
 .insert(activityLogs)
 .values({
 userId: user.id,
 userName: user.fullName,
 action: "0000 00 00 0000000",
 detail: `${maskPhone(phone)} | ${simCarrier || "00000000 0000000"} | ${deviceInfo}`,
 })
 .catch(() => undefined);

const token = createToken(user.id);
if (b.remember !== false) {
 const store = await cookies();
 store.set(SESSION_COOKIE, token, {
 httpOnly: true,
 sameSite: "lax",
 path: "/",
 maxAge: 60 * 60 * 24 * 30,
 });
}

return Response.json({
 ok: true,
 token,
 user: {
 id: user.id,
 username: user.username,
 fullName: user.fullName,
 role: baseRole,
 roleKey: user.role,
 phone: user.phone,
 requirePhone: user.requirePhone,
 permissions: user.permissions,
 },
});
}

```

@@ src/app/api/mobile/nonce/route.ts typescript  
 import { issueNonce } from "@lib/mobile-auth";  
 import { ensureSeed } from "@lib/bootstrap";

```

export const dynamic = "force-dynamic";

/**
 * 生成 nonce 并返回 nonce 和 expiresAt, ttlMs, serverTime
 * 返回的 nonce 是 32 位的十六进制字符串
 *
 * POST /api/mobile/nonce { deviceId }
 * → { nonce, expiresAt, ttlMs, serverTime }
 */
export async function POST(req: Request) {
 void ensureSeed();
 const b = await req.json().catch(() => ({}));
 const deviceId = String(b.deviceId ?? "").replace(/^[^a-zA-Z0-9_-]/g, "").slice(0, 80);
 if (!deviceId) {
 return Response.json({ error: "deviceId 不能为空" }, { status: 400 });
 }
 const { nonce, expiresAt, ttlMs } = await issueNonce(deviceId);
 return Response.json(
 { nonce, expiresAt: expiresAt.toISOString(), ttlMs, serverTime: Date.now() },
 { headers: { "Cache-Control": "no-store" } }
);
}

@@ src/app/api/records/[type]/route.ts typescript
import { db, dbRetry } from "@db";
import { activityLogs, attachments, doctors, orders, pharmacies, settings } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { and, desc, eq, inArray, SQL } from "drizzle-orm";

export const dynamic = "force-dynamic";

type Ctx = { params: Promise<{ type: string }> };

const num = (v: unknown): number | null => {
 const n = Number(v);
 return Number.isFinite(n) && n !== 0 ? n : null;
};

const str = (v: unknown, max = 0) => {
 const t = String(v ?? "").trim();
 return max > 0 ? t.slice(0, max) : t; // max=0 时返回原字符串
};

export async function GET(req: Request, ctx: Ctx) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "未登录" }, { status: 401 });
 const { type } = await ctx.params;
 const url = new URL(req.url);
 const userIdFilter = url.searchParams.get("userId");
 const isAdmin = user.role === "admin" || user.role === "supervisor";
 const limit = Math.min(Number(url.searchParams.get("limit")) || 300, 1000);

 try {
 if (type === "pharmacies") {
 const w: SQL | undefined = !isAdmin
 ? eq(pharmacies.userId, user.id)
 : userIdFilter
 ? eq(pharmacies.userId, Number(userIdFilter))
 : undefined;
 const rows = await dbRetry(
 () => db.select().from(pharmacies).where(w).orderBy(desc(pharmacies.id)).limit(limit),
 "pharmacies:list",
);
 return Response.json({ rows });
 }
 if (type === "doctors") {
 const w: SQL | undefined = !isAdmin
 ? eq(doctors.userId, user.id)
 : userIdFilter
 ? eq(doctors.userId, Number(userIdFilter))
 : undefined;
 const rows = await dbRetry(
 () => db.select().from(doctors).where(w).orderBy(desc(doctors.id)).limit(limit),
 "doctors:list",
);
 return Response.json({ rows });
 }
 if (type === "orders") {
 const w: SQL | undefined = !isAdmin
 ? eq(orders.userId, user.id)
 : userIdFilter
 ? eq(orders.userId, Number(userIdFilter))
 : undefined;
 const rows = await dbRetry(
 () => db.select().from(orders).where(w).orderBy(desc(orders.id)).limit(limit),
 "orders:list",
);
 }
 }
}

```

[illegible]

```

 ? body.fileIds.map(Number).filter((n: number) => Number.isFinite(n))
 : [];
 if (fileIds.length) {
 await db
 .update(attachments)
 .set({ ownerId: row.id })
 .where(and(inArray(attachments.id, fileIds), eq(attachments.userId, user.id)));
 }
 await log(user.id, user.fullName, "□□□ □□□□", str(body.name));
 return Response.json({ row });
}
if (type === "orders") {
 if (!base.dateShamsi || !str(body.pharmacyName)) {
 return Response.json({ error: "□□□□ □ □□ □□□□□□□□ □□□□□ □□□", { status: 400 } });
 }
 const items: Record<string, number> = {};
 for (const [k, v] of Object.entries((body.items ?? {}) as Record<string, unknown>)) {
 const n = Number(v);
 if (Number.isFinite(n) && n !== 0) items[k] = n;
 }
 const [row] = await db
 .insert(orders)
 .values({
 ...base,
 pharmacyName: str(body.pharmacyName),
 managerName: str(body.managerName, 160),
 managerPhone: str(body.managerPhone, 20),
 address: str(body.address),
 locationLabel: base.locationLabel || str(body.pharmacyName),
 items,
 distributor: str(body.distributor, 160),
 visitor: str(body.visitor, 160),
 notes: str(body.notes),
 })
 .returning();
 await log(user.id, user.fullName, "□□□ □□□□□", str(body.pharmacyName));

 // □□□□□ □□□□□□ □□ □□□□□□□□□□□□□□ □□□□ (□□ □□□□□□□□ — □□□ □□ □□□□ □□□□□□□□)
 const autoSend = await getAutoSend();
 if (autoSend) {
 void (async () => {
 try {
 const { dispatchOrder } = await import("@/lib/messaging");
 const status = await dispatchOrder({
 id: row.id,
 repName: row.repName,
 dateShamsi: row.dateShamsi,
 pharmacyName: row.pharmacyName,
 managerName: row.managerName,
 managerPhone: row.managerPhone,
 address: row.address,
 lat: row.lat,
 lng: row.lng,
 items: row.items ?? {},
 distributor: row.distributor,
 visitor: row.visitor,
 notes: row.notes,
 });
 await db.update(orders).set({ sendStatus: status, sent: true }).where(eq(orders.id, row.id));
 const { notify } = await import("@/lib/notify");
 await notify({
 toRole: "admin",
 fromName: row.repName,
 kind: "order",
 title: `□ □□□□□ □□□□ □□ ${row.repName}`,
 body: `${row.pharmacyName} — ${status}`,
 link: "/admin/records/orders",
 });
 } catch (err) {
 console.error("[order:autosend]", err instanceof Error ? err.message : err);
 }
 })();
 }

 return Response.json({ row, autoSend });
}
} catch {
 return Response.json({ error: "□□□ □□ □□□ □□□□□□□□", { status: 500 } });
}
return Response.json({ error: "□□□ □□□□□□□□", { status: 400 } });
}

/** □□□ □□□□□ □□□□□□ □□□□□ □□ □□□□□□□□□□□□ □□□□ □□□□ (□□□□□□□□: □□□) */
async function getAutoSend(): Promise<boolean> {
 try {

```

```

const rows = await db.select().from(settings).where(eq(settings.key, "autoSendOrders")).limit(1);
const v = rows[0]?.value as { enabled?: boolean } | boolean | undefined;
if (typeof v === "boolean") return v;
if (v && typeof v === "object" && typeof v.enabled === "boolean") return v.enabled;
} catch {
 /* ignore */
}
return true;
}

async function log(userId: number, userName: string, action: string, detail: string) {
 try {
 await db.insert(activityLogs).values({ userId, userName, action, detail });
 } catch {
 /* ignore */
 }
}

/**
 * 00000000 000000 — 00000000 000 000000 000000 0000 0000 */
export async function PATCH(req: Request, ctx: Ctx) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "00000000 00000000" }, { status: 401 });
 const { type } = await ctx.params;
 const body = await req.json().catch(() => ({}));
 const id = Number(body.id);
 if (!Number.isFinite(id)) return Response.json({ error: "000000 00000000" }, { status: 400 });

 const isAdmin = user.role === "admin" || user.role === "supervisor";
 const patch: Record<string, unknown> = {};
 const setIf = (key: string, val: unknown) => {
 if (val !== undefined) patch[key] = val;
 };

 try {
 if (type === "pharmacies") {
 setIf("dateShamsi", body.dateShamsi ? str(body.dateShamsi, 12) : undefined);
 setIf("province", body.province !== undefined ? str(body.province, 100) : undefined);
 setIf("city", body.city !== undefined ? str(body.city, 100) : undefined);
 setIf("region", body.region !== undefined ? str(body.region, 100) : undefined);
 setIf("name", body.name !== undefined ? str(body.name) : undefined);
 setIf("landline", body.landline !== undefined ? str(body.landline, 20) : undefined);
 setIf("managerName", body.managerName !== undefined ? str(body.managerName, 160) : undefined);
 setIf("managerPhone", body.managerPhone !== undefined ? str(body.managerPhone, 20) : undefined);
 setIf("address", body.address !== undefined ? str(body.address) : undefined);
 setIf("isPercent", typeof body.isPercent === "boolean" ? body.isPercent : undefined);
 setIf("percentValue", body.percentValue !== undefined ? str(body.percentValue) : undefined);
 setIf("lat", body.lat !== undefined ? num(body.lat) : undefined);
 setIf("lng", body.lng !== undefined ? num(body.lng) : undefined);
 if (Object.keys(patch).length === 0) return Response.json({ error: "00000000 000000 0000" }, { status: 400 });
 const w = isAdmin ? eq(pharmacies.id, id) : and(eq(pharmacies.id, id), eq(pharmacies.userId, user.id));
 await dbRetry(() => db.update(pharmacies).set(patch).where(w), "pharmacies:update");
 } else if (type === "doctors") {
 setIf("dateShamsi", body.dateShamsi ? str(body.dateShamsi, 12) : undefined);
 setIf("province", body.province !== undefined ? str(body.province, 100) : undefined);
 setIf("city", body.city !== undefined ? str(body.city, 100) : undefined);
 setIf("region", body.region !== undefined ? str(body.region, 100) : undefined);
 setIf("name", body.name !== undefined ? str(body.name) : undefined);
 setIf("specialty", body.specialty !== undefined ? str(body.specialty, 160) : undefined);
 setIf("phone", body.phone !== undefined ? str(body.phone, 20) : undefined);
 setIf("secretaryName", body.secretaryName !== undefined ? str(body.secretaryName, 160) : undefined);
 setIf("secretaryPhone", body.secretaryPhone !== undefined ? str(body.secretaryPhone, 20) : undefined);
 setIf("address", body.address !== undefined ? str(body.address) : undefined);
 setIf("otherAddresses", body.otherAddresses !== undefined ? str(body.otherAddresses) : undefined);
 setIf("isPercent", typeof body.isPercent === "boolean" ? body.isPercent : undefined);
 setIf("percentValue", body.percentValue !== undefined ? str(body.percentValue) : undefined);
 setIf("lat", body.lat !== undefined ? num(body.lat) : undefined);
 setIf("lng", body.lng !== undefined ? num(body.lng) : undefined);
 if (Object.keys(patch).length === 0) return Response.json({ error: "00000000 000000 0000" }, { status: 400 });
 const w = isAdmin ? eq(doctors.id, id) : and(eq(doctors.id, id), eq(doctors.userId, user.id));
 await dbRetry(() => db.update(doctors).set(patch).where(w), "doctors:update");
 } else if (type === "orders") {
 setIf("dateShamsi", body.dateShamsi ? str(body.dateShamsi, 12) : undefined);
 setIf("pharmacyName", body.pharmacyName !== undefined ? str(body.pharmacyName) : undefined);
 setIf("managerName", body.managerName !== undefined ? str(body.managerName, 160) : undefined);
 setIf("managerPhone", body.managerPhone !== undefined ? str(body.managerPhone, 20) : undefined);
 setIf("address", body.address !== undefined ? str(body.address) : undefined);
 setIf("distributor", body.distributor !== undefined ? str(body.distributor, 160) : undefined);
 setIf("visitor", body.visitor !== undefined ? str(body.visitor, 160) : undefined);
 setIf("notes", body.notes !== undefined ? str(body.notes) : undefined);
 setIf("lat", body.lat !== undefined ? num(body.lat) : undefined);
 setIf("lng", body.lng !== undefined ? num(body.lng) : undefined);
 if (body.items && typeof body.items === "object") {
 const items: Record<string, number> = {};
 for (const [k, v] of Object.entries(body.items as Record<string, unknown>)) {
 const n = Number(v);

```

```

 if (Number.isFinite(n) && n !== 0) items[k] = n;
 }
 patch.items = items;
}
if (Object.keys(patch).length === 0) return Response.json({ error: "□□□□□□ □□□□□ □□□" }, { status: 400 });
const w = isAdmin ? eq(orders.id, id) : and(eq(orders.id, id), eq(orders.userId, user.id));
await dbRetry(() => db.update(orders).set(patch).where(w), "orders:update");
} else {
 return Response.json({ error: "□□□ □□□□□□□□" }, { status: 400 });
}
} catch {
 return Response.json({ error: "□□□ □□ □□□□□□" }, { status: 500 });
}

await log(user.id, user.fullName, "□□□□□□ □□□□□", `${type} #${id}`);
return Response.json({ ok: true });
}

/** □□□ □□□□□□ */
export async function DELETE(req: Request, ctx: Ctx) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "□□□□□□ □□□□□□□□" }, { status: 401 });
 const { type } = await ctx.params;
 const id = Number(new URL(req.url).searchParams.get("id"));
 if (!Number.isFinite(id)) return Response.json({ error: "□□□□□ □□□□□□□□" }, { status: 400 });

 const isAdmin = user.role === "admin" || user.role === "supervisor";
 const perm = type === "pharmacies" ? "pharmacy.delete" : type === "doctors" ? "doctor.delete" : "order.delete";
 if (!isAdmin && !user.permissions.includes(perm)) {
 return Response.json({ error: "□□□ □□□□□ □□□ □□□□□□. □□ □□□□ □□□□ □□□□□□." }, { status: 403 });
 }

 try {
 if (type === "pharmacies") {
 const w = isAdmin ? eq(pharmacies.id, id) : and(eq(pharmacies.id, id), eq(pharmacies.userId, user.id));
 await dbRetry(() => db.delete(pharmacies).where(w), "pharmacies:delete");
 } else if (type === "doctors") {
 const w = isAdmin ? eq(doctors.id, id) : and(eq(doctors.id, id), eq(doctors.userId, user.id));
 await dbRetry(() => db.delete(doctors).where(w), "doctors:delete");
 } else if (type === "orders") {
 const w = isAdmin ? eq(orders.id, id) : and(eq(orders.id, id), eq(orders.userId, user.id));
 await dbRetry(() => db.delete(orders).where(w), "orders:delete");
 } else {
 return Response.json({ error: "□□□ □□□□□□□□" }, { status: 400 });
 }
 } catch {
 return Response.json({ error: "□□□ □□ □□□" }, { status: 500 });
 }

 await log(user.id, user.fullName, "□□□ □□□□□□", `${type} #${id}`);
 return Response.json({ ok: true });
}

@@ src/app/api/records/lookup/route.ts typescript
import { db, dbRetrySafe } from "@db";
import { doctors, orders, pharmacies } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { bonusKeyOf } from "@lib/defaults";
import { getProducts } from "@lib/settings-server";
import { desc } from "drizzle-orm";

export const dynamic = "force-dynamic";

/** □□□□□□□□□□ □□□ □□□□ □□□□□□: □□□ □□□□□□ □□□□□□□□□□ «□□□□□□□□□» □ «□□□□» □ □□□□□□□□□□ □□□□□ */
export function normalizeName(input: string) {
 return String(input ?? "").
 .replace(/[\u200c\u200f\u200e]/g, "")
 .replace(/[\[\]]/g, "□")
 .replace(/□/g, "□")
 .replace(/[\{\}\}]/g, "□")
 .replace(/□/g, "□")
 .replace(/^(□□□□□□□□□□|□□□□□|□□□□□□□□□□|□□□□)\s*/g, "")
 .replace(/\s+/g, "")
 .trim()
 .toLowerCase();
}

function normPhone(p: string) {
 return String(p ?? "").replace(/\D/g, "");
}

/**
 * GET /api/records/lookup?type=pharmacies|doctors|orders&name=...&phone=...
 *
 * □□ □□□□□□□:

```



```

* D) 000000 000000 0000 000000 000 00000000/0000 0000
* D) 000000 000000 000 00000 00 000 000000 0000 000000000
*/
export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "0000000 00000000" }, { status: 401 });

 const url = new URL(req.url);
 const type = url.searchParams.get("type") ?? "pharmacies";
 const rawName = url.searchParams.get("name") ?? "";
 const phone = normPhone(url.searchParams.get("phone") ?? "");
 const key = normalizeName(rawName);

 if (key.length < 2 && !phone) return Response.json({ duplicates: [], suggestion: null });

 /* ----- 000000 0000000:0000 ----- */
 if (type === "doctors") {
 const rows = await dbRetrySafe(
 () => db.select().from(doctors).orderBy(desc(doctors.id)).limit(3000),
 [],
 "lookup:doctors",
);
 const dup = rows.filter(
 (r) => (key.length >= 2 && normalizeName(r.name) === key) || (!!phone && normPhone(r.phone) === phone),
);
 return Response.json({
 duplicates: dup.slice(0, 5).map((r) => ({
 id: r.id,
 name: r.name,
 specialty: r.specialty,
 phone: r.phone,
 city: r.city,
 address: r.address,
 repName: r.repName,
 dateShamsi: r.dateShamsi,
 mine: r.userId === user.id,
 })),
 count: dup.length,
 });
 }

 /* ----- 000000000:000000 + 000000 000000 ----- */
 const phRows = await dbRetrySafe(
 () => db.select().from(pharmacies).orderBy(desc(pharmacies.id)).limit(3000),
 [],
 "lookup:pharmacies",
);
 const dup = phRows.filter(
 (r) => (key.length >= 2 && normalizeName(r.name) === key) || (!!phone && normPhone(r.managerPhone) === phone),
);

 const duplicates = dup.slice(0, 5).map((r) => ({
 id: r.id,
 name: r.name,
 province: r.province,
 city: r.city,
 region: r.region,
 landline: r.landline,
 managerName: r.managerName,
 managerPhone: r.managerPhone,
 address: r.address,
 lat: r.lat,
 lng: r.lng,
 accuracy: r.accuracy,
 isPercent: r.isPercent,
 percentValue: r.percentValue,
 repName: r.repName,
 dateShamsi: r.dateShamsi,
 mine: r.userId === user.id,
 })));

 if (type !== "orders") return Response.json({ duplicates, count: dup.length });

 /* ----- 000000 000000 0000 000000000 ----- */
 const orderRows = await dbRetrySafe(
 () => db.select().from(orders).orderBy(desc(orders.id)).limit(3000),
 [],
 "lookup:orders",
);
 const history = orderRows.filter((o) => normalizeName(o.pharmacyName) === key);

 const products = await getProducts();
 const itemTotals: Record<string, { label: string; qty: number; bonus: number }> = {};
 for (const o of history) {
 for (const p of products) {

```

```

 const q = Number(o.items?.[p.key] ?? 0);
 const bn = Number(o.items?.[bonusKeyOf(p.key)] ?? 0);
 if (!q && !bn) continue;
 itemTotals[p.key] ??= { label: p.label, qty: 0, bonus: 0 };
 itemTotals[p.key].qty += q;
 itemTotals[p.key].bonus += bn;
 }
}

const last = history[0] ?? null;
const base = duplicates[0] ?? null;

// 00000000 00 00 0000 000000 00 000000 (000000 00 000000 000000)
const suggestion =
 last || base
 ? {
 source: last ? "order" : "pharmacy",
 pharmacyName: last?.pharmacyName ?? base?.name ?? rawName,
 managerName: last?.managerName || base?.managerName || "",
 managerPhone: last?.managerPhone || base?.managerPhone || "",
 address: last?.address || base?.address || "",
 lat: last?.lat ?? base?.lat ?? null,
 lng: last?.lng ?? base?.lng ?? null,
 accuracy: last?.accuracy ?? base?.accuracy ?? null,
 distributor: last?.distributor ?? "",
 visitor: last?.visitor ?? "",
 }
 : null;

return Response.json({
 duplicates,
 count: dup.length,
 history: {
 orderCount: history.length,
 lastDate: last?.dateShamsi ?? "",
 lastRep: last?.repName ?? "",
 totalUnits: Object.values(itemTotals).reduce((a, x) => a + x.qty, 0),
 totalBonus: Object.values(itemTotals).reduce((a, x) => a + x.bonus, 0),
 items: Object.entries(itemTotals).map(([k, v]) => ({ key: k, ...v })),
 recent: history.slice(0, 5).map(o => ({
 id: o.id,
 dateShamsi: o.dateShamsi,
 repName: o.repName,
 units: products.reduce((a, p) => a + Number(o.items?.[p.key] ?? 0), 0),
 })),
 },
 suggestion,
});
}

@@ src/app/api/sync/pull/route.ts typescript
import { collectChanges, verifySync } from "@lib/sync";
import { ensureSeed } from "@lib/bootstrap";
import { nodeName } from "@lib/sync-config";

export const dynamic = "force-dynamic";

/**
 * 0000 00000000 00 00 00000000
 * POST /api/sync/pull { since, excludeOrigin } (00 000000 HMAC)
 */
export async function POST(req: Request) {
 await ensureSeed();
 const raw = await req.text();
 const ts = Number(req.headers.get("x-sync-timestamp") ?? 0);
 const sig = req.headers.get("x-sync-signature") ?? "";

 if (!verifySync(raw, ts, sig)) {
 return Response.json({ error: "000000 000000000000 000000 000000" }, { status: 401 });
 }

 let body: { since?: string | null; excludeOrigin?: string } = {};
 try {
 body = JSON.parse(raw);
 } catch {
 /* ignore */
 }

 const since = body.since ? new Date(body.since) : null;
 const changes = await collectChanges(Number.isNaN(since?.getTime()) ? null : since, body.excludeOrigin);
 const count = Object.values(changes).reduce((a, r) => a + r.length, 0);

 return Response.json(
 { node: nodeName(), cursor: new Date().toISOString(), count, changes },
 { headers: { "Cache-Control": "no-store" } },
);
}

```

```

 });
}

@@ src/app/api/sync/push/route.ts typescript
import { applyChanges, verifySync, type ChangeSet } from "@lib/sync";
import { ensureSeed } from "@lib/bootstrap";
import { nodeName } from "@lib/sync-config";

export const dynamic = "force-dynamic";

/**
 * 同步数据到数据库并验证签名。
 * POST /api/sync/push {node, changes} (需要 HMAC)
 */
export async function POST(req: Request) {
 await ensureSeed();
 const raw = await req.text();
 const ts = Number(req.headers.get("x-sync-timestamp")) ?? 0;
 const sig = req.headers.get("x-sync-signature") ?? "";

 if (!verifySync(raw, ts, sig)) {
 return Response.json({ error: "验证失败: 签名或时间戳不匹配", status: 401 });
 }

 let body: { node?: string; changes?: ChangeSet } = {};
 try {
 body = JSON.parse(raw);
 } catch {
 return Response.json({ error: "无效的 JSON 数据", status: 400 });
 }

 const from = String(body.node ?? req.headers.get("x-sync-node") ?? "peer");
 const res = await applyChanges(body.changes ?? {}, from);

 return Response.json(
 {
 node: nodeName(),
 applied: res.applied,
 skipped: res.skipped,
 tables: res.perTable,
 headers: { "Cache-Control": "no-store" },
 },
);
}

@@ src/app/api/sync/run/route.ts typescript
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { syncAll } from "@lib/sync";
import { syncSecret } from "@lib/sync-config";

export const dynamic = "force-dynamic";
export const maxDuration = 60;

/**
 * 同步数据并验证权限。
 * - 需要 x-sync-key
 * - 需要 Cron 任务
 */
async function authorize(req: Request) {
 const key = req.headers.get("x-sync-key") ?? new URL(req.url).searchParams.get("key");
 if (key && syncSecret() && key === syncSecret()) return true;
 const user = await getSessionUser();
 return Boolean(user && (user.role === "admin" || user.permissions.includes("users")));
}

export async function POST(req: Request) {
 await ensureSeed();
 if (!(await authorize(req))) return Response.json({ error: "权限不足", status: 401 });
 const reports = await syncAll();
 const ok = reports.every((r) => r.ok);
 return Response.json(
 {
 ok,
 count: reports.length,
 reports,
 summary:
 reports.length === 0
 ? "所有数据同步成功"
 : reports.map((r) => `${r.peer}: ${r.ok ? "成功" : "失败"} / ${r.pulled} / ${r.pushed}`)
 .join(" | "),
 headers: { "Cache-Control": "no-store" },
 },
);
}

export async function GET(req: Request) {
 return POST(req);
}

```

```

@@ src/app/api/sync/status/route.ts typescript
import { db, dbRetrySafe } from "@db";
import { syncLogs, syncState } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { ensureSeed } from "@lib/bootstrap";
import { ensurePeers, syncInfo } from "@lib/sync";
import { desc } from "drizzle-orm";

export const dynamic = "force-dynamic";

export async function GET() {
 await ensureSeed();
 const user = await getSessionUser();
 if (!user || (user.role !== "admin" && !user.permissions.includes("users"))) {
 return Response.json({ error: "权限不足", status: 401 });
 }
 const peers = await ensurePeers();
 const logs = await dbRetrySafe(
 () => db.select().from(syncLogs).orderBy(desc(syncLogs.id)).limit(40),
 [],
 "sync:logs",
);
 return Response.json({ info: syncInfo(), peers, logs }, { headers: { "Cache-Control": "no-store" } });
}

/** 更新同步状态 */
export async function PATCH(req: Request) {
 const user = await getSessionUser();
 if (!user || user.role !== "admin") return Response.json({ error: "权限不足", status: 401 });
 const b = await req.json().catch(() => ({}));
 const id = Number(b.id);
 if (!Number.isFinite(id)) return Response.json({ error: "ID 无效", status: 400 });

 const patch: Record<string, unknown> = {};
 if (typeof b.enabled === "boolean") patch.enabled = b.enabled;
 if (typeof b.peerUrl === "string") patch.peerUrl = b.peerUrl.trim();
 if (b.resetCursor === true) {
 patch.pullCursor = null;
 patch.pushCursor = null;
 }
 if (Object.keys(patch).length === 0) return Response.json({ error: "无有效数据", status: 400 });

 const { eq } = await import("drizzle-orm");
 await db.update(syncState).set(patch).where(eq(syncState.id, id));
 return Response.json({ ok: true });
}

/** 添加同步状态 */
export async function POST(req: Request) {
 const user = await getSessionUser();
 if (!user || user.role !== "admin") return Response.json({ error: "权限不足", status: 401 });
 const b = await req.json().catch(() => ({}));
 const peer = String(b.peer ?? "").trim().slice(0, 80);
 const peerUrl = String(b.peerUrl ?? "").trim();
 if (!peer || !peerUrl.startsWith("http")) {
 return Response.json({ error: "peer 或 peerUrl 无效", status: 400 });
 }
 await db.insert(syncState).values({ peer, peerUrl }).onConflictDoNothing();
 return Response.json({ ok: true });
}

@@ src/app/api/trips/points/route.ts typescript
import { db } from "@db";
import { liveLocations, tripPoints, trips } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { asc, eq } from "drizzle-orm";

/** 计算两点距离 */
function haversine(a: { lat: number; lng: number }, b: { lat: number; lng: number }) {
 const R = 6371000;
 const dLat = ((b.lat - a.lat) * Math.PI) / 180;
 const dLng = ((b.lng - a.lng) * Math.PI) / 180;
 const s =
 Math.sin(dLat / 2) ** 2 +
 Math.cos((a.lat * Math.PI) / 180) * Math.cos((b.lat * Math.PI) / 180) * Math.sin(dLng / 2) ** 2;
 const d = 2 * R * Math.asin(Math.sqrt(s));
 // 判断 GPS 点是否重合
 return d < 5 || d > 5000 ? 0 : d;
}

export const dynamic = "force-dynamic";

export async function GET(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "未登录", status: 401 });

```

```

const tripId = Number(new URL(req.url).searchParams.get("tripId"));
if (!Number.isFinite(tripId)) return Response.json({ error: "Invalid tripId", status: 400 });
const t = (await db.select().from(trips).where(eq(trips.id, tripId)).limit(1))[0];
if (!t || (user.role !== "admin" && t.userId !== user.id)) {
 return Response.json({ error: "Invalid tripId", status: 404 });
}
const points = await db
 .select()
 .from(tripPoints)
 .where(eq(tripPoints.tripId, tripId))
 .orderBy(asc(tripPoints.recordedAt))
 .limit(5000);
return Response.json({ trip: t, points });
}

export async function POST(req: Request) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "Unauthorized", status: 401 });
 const b = await req.json().catch(() => ({}));
 const tripId = Number(b.tripId);
 if (!Number.isFinite(tripId)) return Response.json({ error: "Invalid tripId", status: 400 });
 const t = (await db.select().from(trips).where(eq(trips.id, tripId)).limit(1))[0];
 if (!t || t.userId !== user.id) return Response.json({ error: "Invalid tripId", status: 404 });

 const raw = Array.isArray(b.points) ? b.points : [];
 const values = raw
 .map((p: Record<string, unknown>) => ({
 tripId,
 lat: Number(p.lat),
 lng: Number(p.lng),
 accuracy: Number(p.accuracy) || null,
 speed: Number(p.speed) || null,
 stopSeconds: Math.max(0, Math.round(Number(p.stopSeconds) || 0)),
 gpsOn: p.gpsOn !== false,
 kind: ["start", "move", "pause", "end"].includes(String(p.kind)) ? String(p.kind) : "move",
 note: String(p.note ?? "").slice(0, 200),
 recordedAt: p.recordedAt ? new Date(String(p.recordedAt)) : new Date(),
 synced: true,
 })))
 .filter((p: { lat: number; lng: number }) => Number.isFinite(p.lat) && Number.isFinite(p.lng))
 .slice(0, 500);

 if (values.length) await db.insert(tripPoints).values(values);

 // Calculate distance and stop seconds
 try {
 const all = await db
 .select({ lat: tripPoints.lat, lng: tripPoints.lng, stopSeconds: tripPoints.stopSeconds })
 .from(tripPoints)
 .where(eq(tripPoints.tripId, tripId))
 .orderBy(asc(tripPoints.recordedAt));

 let dist = 0;
 for (let i = 1; i < all.length; i++) dist += haversine(all[i - 1], all[i]);
 const stops = all.reduce((a, x) => a + (x.stopSeconds ?? 0), 0);

 await db.update(trips).set({ distanceM: Math.round(dist), stopSeconds: stops }).where(eq(trips.id, tripId));

 // Create live location
 const last = values[values.length - 1];
 if (last) {
 const lv = {
 userId: user.id,
 repName: user.fullName,
 lat: last.lat,
 lng: last.lng,
 accuracy: last.accuracy,
 speed: last.speed,
 gpsOn: last.gpsOn,
 online: true,
 tripId,
 recordedAt: last.recordedAt,
 updatedAt: new Date(),
 };
 await db.insert(liveLocations).values(lv).onConflictDoUpdate({ target: liveLocations.userId, set: lv });
 }
 } catch {
 /* Handle error */
 }

 return Response.json({ ok: true, inserted: values.length });
}

```

```

@@ src/app/api/records/[type]/send/route.ts typescript
import { db } from "@/db";

```

```
import { activityLogs, doctors, orders, pharmacies } from "@db/schema";
import { getSessionUser } from "@lib/auth";
import { dispatchOrder } from "@lib/messaging";
import { notify } from "@lib/notify";
import { eq, inArray, and } from "drizzle-orm";

export const dynamic = "force-dynamic";

type Ctx = { params: Promise<{ type: string }> };

export async function POST(req: Request, ctx: Ctx) {
 const user = await getSessionUser();
 if (!user) return Response.json({ error: "❌ ❌ ❌ ❌ ❌ ", status: 401 });
 const { type } = await ctx.params;
 const body = await req.json().catch(() => ({}));
 const ids: number[] = Array.isArray(body.ids)
 ? body.ids.map((v: unknown) => Number(v)).filter((n: number) => Number.isFinite(n))
 : [];
 if (ids.length === 0) return Response.json({ error: "❌ ❌ ❌ ❌ ❌ ❌ ❌ ", status: 400 });

 let sendStatusText = "";
 try {
 if (type === "pharmacies") {
 await db
 .update(pharmacies)
 .set({ sent: true })
 .where(and(inArray(pharmacies.id, ids), eq(pharmacies.userId, user.id)));
 } else if (type === "doctors") {
 await db
 .update(doctors)
 .set({ sent: true })
 .where(and(inArray(doctors.id, ids), eq(doctors.userId, user.id)));
 } else if (type === "orders") {
 const rows = await db
 .select()
 .from(orders)
 .where(and(inArray(orders.id, ids), eq(orders.userId, user.id)));
 let lastStatus = "";
 for (const row of rows) {
 const status = await dispatchOrder({
 id: row.id,
 repName: row.repName,
 dateShamsi: row.dateShamsi,
 pharmacyName: row.pharmacyName,
 managerName: row.managerName,
 managerPhone: row.managerPhone,
 address: row.address,
 lat: row.lat,
 lng: row.lng,
 items: row.items ?? {},
 distributor: row.distributor,
 visitor: row.visitor,
 notes: row.notes,
 });
 lastStatus = status;
 await db.update(orders).set({ sent: true, sendStatus: status }).where(eq(orders.id, row.id));
 }
 sendStatusText = lastStatus;
 } else {
 return Response.json({ error: "❌ ❌ ❌ ❌ ❌ ", status: 400 });
 }
 await db.insert(activityLogs).values({
 userId: user.id,
 userName: user.fullName,
 action: "📝 📝 📝 📝 📝",
 detail: `${type} - ${ids.length} 📝 📝 📝`,
 });
 const typeLabel = type === "pharmacies" ? "🏪 🏪 🏪 🏪 🏪" : type === "doctors" ? "⚕️ ⚕️ ⚕️ ⚕️ ⚕️";
 await notify({
 toRole: "admin",
 fromName: user.fullName,
 kind: "record",
 title: `📝 ${ids.length} 📝 📝 📝 ${typeLabel} 📝 📝 📝 ${user.fullName}`,
 body: sendStatusText || `📝 📝 📝 📝 📝 ${typeLabel} 📝 📝 📝 📝 📝 📝 📝 📝 \`,` ,
 link: `/admin/records/${type}` ,
 });
 await notify({
 toRole: "supervisor",
 fromName: user.fullName,
 kind: "record",
 title: `📝 ${ids.length} 📝 📝 📝 ${typeLabel} 📝 📝 📝 ${user.fullName}`,
 link: `/admin/records/${type}` ,
 });
 return Response.json({ ok: true, count: ids.length, status: sendStatusText });
 } catch {}
}
```

```

 return Response.json({ error: "0000 00 000000 00000000" }, { status: 500 });
 }
}

@@ src/app/api/tiles/[z]/[x]/[y]/route.ts typescript
import { fetchWithRetry } from "@lib/retry";

export const dynamic = "force-dynamic";

/**
 * 0000000 00000 OpenStreetMap. 00000000/Service Worker 0000 00 00000 00 00000000
 * 00 000000 0000000000000 00 000000 00000 0000 00000000 00 000000 00000 000000000.
 */
export async function GET(_req: Request, ctx: { params: Promise<{ z: string; x: string; y: string }> }) {
 const { z, x, y } = await ctx.params;
 if (!/^\\d+$/.test(z) || !/^\\d+$/.test(x) || !/^\\d+$/.test(y)) return new Response("bad tile", { status: 400 });
 const zi = Number(z);
 if (zi < 0 || zi > 18) return new Response("bad zoom", { status: 400 });

 try {
 const sub = ["a", "b", "c"][(Number(x) + Number(y)) % 3];
 const res = await fetchWithRetry(`https://${sub}.tile.openstreetmap.org/${z}/${x}/${y}.png`, {
 headers: { "User-Agent": "SabtEtelaatKol/1.0 (offline map cache)" },
 }, { retries: 2, timeoutMs: 12000, label: "tile" });
 if (!res.ok) return new Response("tile unavailable", { status: res.status });
 const bytes = new Uint8Array(await res.arrayBuffer());
 return new Response(bytes, {
 headers: {
 "Content-Type": "image/png",
 "Cache-Control": "public, max-age=2592000, stale-while-revalidate=86400",
 "Access-Control-Allow-Origin": "",
 },
 });
 } catch {
 // 000000 00000/0000000000 0000000 0000 00000 000000 00000000
 return new Response(null, { status: 204, headers: { "Cache-Control": "no-store" } });
 }
}

@@ src/app/api/map/tiles/[z]/[x]/[y]/route.ts typescript
import { fetchWithRetry } from "@lib/retry";

export const dynamic = "force-dynamic";
export const revalidate = 86400;

/**
 * 0000000 0000 0000 00000 00 00000 403 00 00000000 000000.
 *
 * 0000000 0000 0000000000 00 tile.openstreetmap.org 0000 0000000000 00000000 00
 * 000000 000 0000000 00 0000 Route 0000000 0 00000 00 User-Agent 00000 00000 00
 * 00000000 0000000. 0000 00 00000 000 000000 000000 00000 00000000 00000000.
 */

const SOURCES = [
 (z: number, x: number, y: number) => `https://tile.openstreetmap.org/${z}/${x}/${y}.png`,
 (z: number, x: number, y: number) => `https://a.tile.openstreetmap.fr/hot/${z}/${x}/${y}.png`,
 (z: number, x: number, y: number) => `https://basemaps.cartocdn.com/light_all/${z}/${x}/${y}.png`,
];

const TILE_HEADERS = {
 "User-Agent": "NdcoHub-SabtEtelaat/1.0 (https://ndcohub.ir; map tile proxy)",
 Accept: "image/avif,image/webp,image/png,image/*,*/*;q=0.8",
 Referer: "https://ndcohub.ir/",
};

/** 00000 SVG 00000 00 00000 000 0000 0000000 — 00000 000000000 403/00000 000000 00000 000000000 */
function fallbackTile(z: number, x: number, y: number) {
 const svg = `<svg xmlns="http://www.w3.org/2000/svg" width="256" height="256">
 <rect width="256" height="256" fill="#eef2f1"/>
 <path d="M0 64h256M0 128h256M0 192h256M64 0v256M128 0v256M192 0v256" stroke="#d9e3e0" stroke-width="1"/>
 <path d="M0 180Q80 100 150 155T256 75" fill="none" stroke="#c2d6d0" stroke-width="5"/>
 <text x="128" y="224" text-anchor="middle" fill="#94a3b8" font-size="10" font-family="sans-serif">${z}/${x}/${y}</te
xt>
</svg>`;
 return new Response(svg, {
 headers: {
 "Content-Type": "image/svg+xml",
 "Cache-Control": "public, max-age=300",
 "Access-Control-Allow-Origin": "",
 "X-Map-Fallback": "1",
 },
 });
}

export async function GET(

```

```

_req: Request,
ctx: { params: Promise<{ z: string; x: string; y: string }> },
) {
 const { z: rz, x: rx, y: ry } = await ctx.params;
 const z = Number(rz);
 const x = Number(rx);
 const y = Number(ry.replace(/\.png$/i, ""));

 if (
 !Number.isInteger(z) ||
 !Number.isInteger(x) ||
 !Number.isInteger(y) ||
 z < 1 ||
 z > 19 ||
 x < 0 ||
 y < 0 ||
 x >= 2 ** z ||
 y >= 2 ** z
) {
 return new Response("invalid tile", { status: 400 });
 }

 for (let i = 0; i < SOURCES.length; i++) {
 try {
 const res = await fetchWithRetry(
 SOURCES[i](z, x, y),
 { headers: TILE_HEADERS },
 { retries: 2, baseDelayMs: 250, maxDelayMs: 1200, timeoutMs: 9000, label: `tile-${i}` },
);
 if (!res.ok) continue;
 const type = res.headers.get("content-type") ?? "";
 if (!type.startsWith("image/")) continue;
 const bytes = new Uint8Array(await res.arrayBuffer());
 if (bytes.length < 50) continue;
 return new Response(bytes, {
 headers: {
 "Content-Type": type,
 "Content-Length": String(bytes.length),
 "Cache-Control": "public, max-age=604800, stale-while-revalidate=2592000",
 "Access-Control-Allow-Origin": "*",
 "X-Map-Source": String(i + 1),
 },
 });
 } catch {
 //
 }
 }

 return fallbackTile(z, x, y);
}

```

```

@@ .env.ndcohub.example text
=====
00000000 00000 0000 00000 — NdcoHub.ir (000000000)
000 00000 00 000 00000 0 00 000 .env 0000 00000 00000
=====

--- 00000000 000 00000 (000000 00 Render) ---
DATABASE_URL=postgres://USER:PASS@HOST:5432/DBNAME?sslmode=require

--- 00000 000 000 ---
NODE_NAME=ndcohub
PUBLIC_BASE_URL=https://ndcohub.ir

--- 000000000000 00 00000 00000 ---
00000 0000 00 00 00000 00000000 000000 00000
SYNC_SECRET=00-00000-00000000-00000-0-000000-0000-00-00000
SYNC_PEERS=render|https://namayandee1mi-javad.onrender.com
SYNC_INTERVAL_MINUTES=5

--- 00000000000 00 00000000 000 000000 00000000 00000000 ---
NEXT_PUBLIC_ENDPOINTS=https://ndcohub.ir,https://namayandee1mi-javad.onrender.com

--- 000000 ---
APP_SECRET=00-00000-00000000-00000
MOBILE_APP_SECRET=00000-000000-00-00-00-0000000-00000000

--- 00000000 ---
ADMIN_PASSWORD=admin1234
ADMIN_PHONE=091200000000
BALE_BOT_TOKEN=
SMS_API_KEY=

@@ .gitlab-ci.yml yml
=====

```



```

GitLab CI — ۰۰۰۰۰ ۰۰۰۰۰۰ GitHub ۰۰۰۰۰ ۰۰۰۰۰ ۰۰۰۰۰۰ NdcoHub
=====
stages: [build, mirror, deploy, sync]

default:
 image: node:20
 cache:
 key: "$CI_COMMIT_REF_SLUG"
 paths: [.npm/, node_modules/]

variables:
 NEXT_PUBLIC_ENDPOINTS: "https://ndcohub.ir,https://namayandeeelmi-javad.onrender.com"
 GIT_DEPTH: "0"

build:
 stage: build
 script:
 - npm ci --include=dev --cache .npm --prefer-offline
 - npx next typegen
 - npx tsc --noEmit
 - npm run build
 artifacts:
 expire_in: 1 day
 paths: [.next/, public/]

mirror-to-github:
 stage: mirror
 needs: [build]
 rules:
 - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "master"
 script:
 - |
 if [-z "$GITHUB_REPO_URL"]; then echo "GITHUB_REPO_URL ۰۰۰۰۰۰ ۰۰۰۰۰ — ۰۰ ۰۰"; exit 0; fi
 AUTH_URL=$(echo "$GITHUB_REPO_URL" | sed "s#https://#https://x-access-token:${GITHUB_TOKEN}@#")
 git remote add github "$AUTH_URL" || git remote set-url github "$AUTH_URL"
 git push github HEAD:$CI_COMMIT_BRANCH --force
 git push github --tags --force

deploy:
 stage: deploy
 needs: [build]
 rules:
 - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "master"
 script:
 - '[-n "$NDCOHUB_DEPLOY_HOOK"] && curl -fsSL -X POST "$NDCOHUB_DEPLOY_HOOK" && echo "۰ NdcoHub" || echo "hook ۰۰۰۰
۰ ۰۰۰۰"'
 - '[-n "$RENDER_DEPLOY_HOOK"] && curl -fsSL -X POST "$RENDER_DEPLOY_HOOK" && echo "۰ Render" || true'

sync-after-deploy:
 stage: sync
 needs: [deploy]
 rules:
 - if: $CI_COMMIT_BRANCH == "main" || $CI_COMMIT_BRANCH == "master"
 script:
 - |
 if [-z "$SYNC_SECRET"]; then echo "SYNC_SECRET ۰۰۰۰۰۰ ۰۰۰۰۰"; exit 0; fi
 sleep 90
 curl -fsSL -X POST "https://ndcohub.ir/api/sync/run" -H "x-sync-key: $SYNC_SECRET" || true
 curl -fsSL -X POST "https://namayandeeelmi-javad.onrender.com/api/sync/run" -H "x-sync-key: $SYNC_SECRET" || true

@@ drizzle.config.json json
{
 "dialect": "postgresql",
 "schema": "./src/db/schema.ts",
 "dbCredentials": {
 "url": "postgresql://postgres:postgres@127.0.0.1:5432/app_db"
 }
}

@@ eslint.config.mjs javascript
import { defineConfig, globalIgnores } from "eslint/config";
import nextCoreWebVitals from "eslint-config-next/core-web-vitals";

export default defineConfig([
 // Keep the starter on the flat config export that actually runs under the pinned ESLint/Next toolchain.
 ...nextCoreWebVitals,
 globalIgnores([".next/**", "out/**", "build/**", "next-env.d.ts"]),
]);

@@ postcss.config.mjs javascript
const postcssConfig = {
 plugins: {
 "@tailwindcss/postcss": {},
 },
};

```

```
export default postcssConfig;
```

```
@@ render.yaml yaml
```

```
services:
```

```
- type: web
 name: namayandeelmi
 runtime: node
 plan: free
 region: frankfurt
 branch: main
```

```
0 00000000 000: 0000 000 000 Render 000 npm start 00 00000 0000000 0 0000
```

```
"Could not find a production build in the '.next' directory" 0000000.
```

```
buildCommand: npm install --include=dev && npm run build
```

```
startCommand: npm run start
```

```
healthCheckPath: /api/health
```

```
autoDeploy: true
```

```
envVars:
```

```
- key: NODE_VERSION
 value: "20.18.0"
- key: NODE_ENV
 value: production
- key: NPM_CONFIG_PRODUCTION
 value: "false"
```

```
000000 DATABASE_URL 00 00 00000000 Neon 000 0 000000 (00 00 00000000 Render) 000 00000
```

```
000000: postgresql://user:pass@ep-xxx.eu-central-1.aws.neon.tech/neondb?sslmode=require
```

```
- key: DATABASE_URL
 sync: false
- key: APP_SECRET
 generateValue: true
- key: ADMIN_PASSWORD
 sync: false
- key: ADMIN_PHONE
 sync: false
```

```
---- 000000000000 00 000000 ----
```

```
- key: NODE_NAME
 value: render
- key: SYNC_SECRET
 sync: false
- key: SYNC_PEERS
 value: ndcohub|https://ndcohub.ir
- key: SYNC_INTERVAL_MINUTES
 value: "5"
- key: NEXT_PUBLIC_ENDPOINTS
 value: https://ndcohub.ir,https://namayandeelmi-javad.onrender.com
- key: PUBLIC_BASE_URL
 value: https://namayandeelmi-javad.onrender.com
- key: MOBILE_APP_SECRET
 sync: false
```

```
@@ tsconfig.json json
```

```
{
 "compilerOptions": {
 "target": "ES2017",
 "lib": [
 "dom",
 "dom.iterable",
 "esnext"
],
 "allowJs": false,
 "skipLibCheck": true,
 "strict": true,
 "noEmit": true,
 "esModuleInterop": true,
 "module": "esnext",
 "moduleResolution": "bundler",
 "resolveJsonModule": true,
 "isolatedModules": true,
 "jsx": "react-jsx",
 "incremental": true,
 "baseUrl": ".",
 "paths": {
 "@/*": [
 "./src/*"
]
 },
 "plugins": [
 {
 "name": "next"
 }
]
 },
 "include": [
 "next-env.d.ts",
 "**/*.ts",
]
}
```

```

 "**/*.tsx",
 ".next/types/**/*.ts",
 ".next/dev/types/**/*.ts"
],
 "exclude": [
 "node_modules",
 "mobile"
]
}

```

@@ public/manifest.webmanifest text

```

{
 "id": "/?source=pwa",
 "name": "📄 📄📄📄📄 📄 — 📄📄📄📄 📄📄📄📄📄 📄📄",
 "short_name": "📄📄 📄📄📄📄",
 "description": "📄📄📄📄 📄📄 📄📄📄📄📄 📄📄📄📄📄 📄📄📄 📄📄📄📄 📄 📄📄 📄📄📄📄📄 📄📄📄 📄 📄📄📄📄📄 📄📄 📄📄📄📄📄 📄📄 📄📄📄📄📄 📄📄 📄📄📄📄📄 📄📄 📄📄📄📄📄",
 "start_url": "/?source=pwa",
 "scope": "/",
 "display": "standalone",
 "display_override": [
 "window-controls-overlay",
 "standalone",
 "minimal-ui",
 "browser"
],
 "orientation": "any",
 "background_color": "#f1f5f9",
 "theme_color": "#0f766e",
 "dir": "rtl",
 "lang": "fa-IR",
 "categories": [
 "business",
 "productivity",
 "medical",
 "utilities"
],
 "prefer_related_applications": false,
 "icons": [
 {
 "src": "/icons/icon-96.png",
 "sizes": "96x96",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-128.png",
 "sizes": "128x128",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-144.png",
 "sizes": "144x144",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-152.png",
 "sizes": "152x152",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-180.png",
 "sizes": "180x180",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-192.png",
 "sizes": "192x192",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-256.png",
 "sizes": "256x256",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-384.png",
 "sizes": "384x384",
 "type": "image/png",

```

```

 "purpose": "any"
 },
 {
 "src": "/icons/icon-512.png",
 "sizes": "512x512",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/icon-1024.png",
 "sizes": "1024x1024",
 "type": "image/png",
 "purpose": "any"
 },
 {
 "src": "/icons/maskable-192.png",
 "sizes": "192x192",
 "type": "image/png",
 "purpose": "maskable"
 },
 {
 "src": "/icons/maskable-512.png",
 "sizes": "512x512",
 "type": "image/png",
 "purpose": "maskable"
 }
],
"screenshots": [
 {
 "src": "/screenshots/desktop-1280x720.png",
 "sizes": "1280x720",
 "type": "image/png",
 "form_factor": "wide",
 "label": "모니터용 데스크톱 화면 (1280x720)"
 },
 {
 "src": "/screenshots/brand-1280x720.png",
 "sizes": "1280x720",
 "type": "image/png",
 "form_factor": "wide",
 "label": "브랜드 로고 (1280x720)"
 },
 {
 "src": "/screenshots/mobile-720x1280.png",
 "sizes": "720x1280",
 "type": "image/png",
 "form_factor": "narrow",
 "label": "스마트폰용 모바일 화면 (720x1280)"
 }
],
"shortcuts": [
 {
 "name": "약국 찾기",
 "short_name": "약국",
 "url": "/panel/pharmacies",
 "icons": [
 {
 "src": "/icons/icon-192.png",
 "sizes": "192x192",
 "type": "image/png"
 }
]
 },
 {
 "name": "의사 찾기",
 "short_name": "의사",
 "url": "/panel/doctors",
 "icons": [
 {
 "src": "/icons/icon-192.png",
 "sizes": "192x192",
 "type": "image/png"
 }
]
 },
 {
 "name": "주문하기",
 "short_name": "주문",
 "url": "/panel/orders",
 "icons": [
 {
 "src": "/icons/icon-192.png",
 "sizes": "192x192",
 "type": "image/png"
 }
]
 }
]

```

```

]
 },
 {
 "name": "□□□□ □ □□□□□",
 "short_name": "□□□□",
 "url": "/panel/trip",
 "icons": [
 {
 "src": "/icons/icon-192.png",
 "sizes": "192x192",
 "type": "image/png"
 }
]
 }
],
"launch_handler": {
 "client_mode": "navigate-existing"
},
"edge_side_panel": {
 "preferred_width": 420
},
"handle_links": "preferred"
}

```

@@ scripts/build-sw.mjs javascript

#!/usr/bin/env node

/\*\*

\* □□□□□ □□□□□ □□□□□□□□□□ □□ □□ build.

\*

\* □□□ □□□□□ □□□□ □□□□□□ □□ □□ React □□□□ □□□□ HTML □□□□ □□□□□□□□

\* □□□□□□□□□□□□ □□□ □□□□□□□□□ □□□. □□□ □□□ HTML □□ □□□□ □□□ □□□□□□□□

\* □□□□□□□ (□□□) □□□□□□□□ □□□□□□□□ □□□□□□□□ □ □□□□ □□□□ □□□□□□□□.

\* □□□ □□□□□□□□ □□□ □□□□□□□□ JS/CSS □□□□□□□□ build □□ □□ □□□□□□□□□□□□ □□□ □□□□□□□□.

\*/

import { readdirSync, readFileSync, statSync, writeFileSync, existsSync } from "node:fs";

import path from "node:path";

const ROOT = process.cwd();

const NEXT\_STATIC = path.join(ROOT, ".next", "static");

const SW\_SRC = path.join(ROOT, "public", "sw-template.js");

const SW\_OUT = path.join(ROOT, "public", "sw.js");

function walk(dir, base) {

const out = [];

if (!existsSync(dir)) return out;

for (const name of readdirSync(dir)) {

const full = path.join(dir, name);

const st = statSync(full);

if (st.isDirectory()) out.push(...walk(full, `\${base}/\${name}`));

else out.push({ url: `\${base}/\${name}`, size: st.size });

}

return out;

}

const PAGES = [

"/",

"/login",

"/offline",

"/install",

"/diagnostics",

"/panel",

"/panel/pharmacies",

"/panel/doctors",

"/panel/orders",

"/panel/trip",

"/panel/map",

"/panel/home",

"/panel/leaves",

"/panel/notifications",

"/panel/options",

"/panel/reports",

"/admin",

"/admin/activity",

"/admin/records/pharmacies",

"/admin/records/doctors",

"/admin/records/orders",

"/admin/map",

"/admin/live",

"/admin/trips",

"/admin/homes",

"/admin/leaves",

"/admin/notifications",

"/admin/reports",

"/admin/options",

"/admin/columns",

```

"/admin/users",
"/admin/messengers",
"/admin/backup",
];

const EXTRAS = [
 "/manifest.webmanifest",
 "/logo.svg",
 "/icons/icon-192.png",
 "/icons/icon-512.png",
 "/apple-touch-icon.png",
 "/data/iran-provinces.geojson",
];

function main() {
 if (!existsSync(SW_SRC)) {
 console.warn("⚠️ 📁 sw-template.js 📁📁 sw.js 📁📁📁 📁📁📁📁");
 return;
 }
 const assets = walk(NEXT_STATIC, "/_next/static")
 .filter((a) => /\.?(js|css|woff2?)$/i.test(a.url))
 // 📁📁📁📁📁 📁📁📁📁 📁📁 (📁📁📁 📁 📁📁📁📁 📁) 📁 📁📁 📁📁 📁 📁📁📁📁📁📁
 .filter((a) => a.size < 700 * 1024)
 .map((a) => a.url)
 .sort();

 const total = walk(NEXT_STATIC, "/_next/static").reduce((s, a) => s + a.size, 0);
 const build = `${Date.now().toString(36)}-${assets.length}`;

 // 📁📁📁 RSC 📁📁📁📁 📁 📁📁📁📁 📁📁📁📁 📁 📁📁📁📁 📁📁📁📁 📁📁 📁📁
 const rscPages = PAGES.map((p) => `${p}${p.includes("?") ? "&" : "?"}_rsc=offline`);

 let sw = readFileSync(SW_SRC, "utf8");
 sw = sw
 .replace("__BUILD_ID__", build)
 .replace("__PAGES__", JSON.stringify(PAGES, null, 2))
 .replace("__RSC_PAGES__", JSON.stringify(rscPages, null, 2))
 .replace("__ASSETS__", JSON.stringify(assets, null, 2))
 .replace("__EXTRAS__", JSON.stringify(EXTRAS, null, 2));

 writeFileSync(SW_OUT, sw);
 console.log(
 `📁 📁📁📁📁📁📁📁 📁📁📁📁 📁: ${PAGES.length} 📁📁📁 + ${assets.length} 📁📁📁 📁📁📁📁📁 ( ${Math.round(total / 1024)} 📁📁📁📁📁📁 )`
);
};

main();

@@ scripts/generate-assets.mjs javascript
#!/usr/bin/env node
/**
 * 📁📁📁📁 📁📁📁📁📁 📁📁📁📁📁📁📁📁📁 PWA 📁📁 📁📁 📁📁📁📁 SVG.
 *
 * 📁📁 📁📁📁 📁📁📁 📁📁📁📁📁📁 PNG 📁📁📁📁 📁📁📁 📁📁📁📁 📁📁 📁📁📁/📁📁📁 📁📁 📁📁📁📁📁📁
 * 📁 📁 📁📁📁📁 PWABuilder 📁📁📁 «Fix the links to your icons» 📁📁📁📁📁.
 * 📁📁📁 📁📁📁📁📁 📁📁 📁📁 📁📁 build 📁📁📁 📁📁📁📁📁 📁 📁📁 📁📁📁📁📁 📁📁 📁📁📁 📁📁📁📁📁📁
 * 📁📁📁📁📁📁 📁📁📁📁 📁📁📁📁 📁 📁📁📁 📁📁📁📁.
 */
import { existsSync, mkdirSync, writeFileSync } from "node:fs";
import path from "node:path";

const ROOT = process.cwd();
const PUB = path.join(ROOT, "public");
const ICONS = path.join(PUB, "icons");
const SHOTS = path.join(PUB, "screenshots");

const LOGO_SVG = `<svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 512 512" width="512" height="512">
 <defs>
 <linearGradient id="pill" x1="0%" y1="1" x2="1" y2="0">
 <stop offset="0%" stop-color="#F9A03F"/>
 <stop offset="35%" stop-color="#F7714B"/>
 <stop offset="70%" stop-color="#EC3B75"/>
 <stop offset="100%" stop-color="#C81E63"/>
 </linearGradient>
 <linearGradient id="glossTop" x1="0" y1="0" x2="0" y2="1">
 <stop offset="0%" stop-color="ffffff" stop-opacity=".45"/>
 <stop offset="55%" stop-color="ffffff" stop-opacity="0"/>
 </linearGradient>
 <filter id="soft" x="-30%" y="-30%" width="160%" height="160%">
 <feDropShadow dx="0" dy="10" stdDeviation="16" flood-color="#8b1141" flood-opacity=".28"/>
 </filter>
 </defs>
 <rect transform="translate(256 256) rotate(-45)" filter="url(#soft)">
 <rect x="-168" y="-84" width="336" height="168" rx="84" ry="84" fill="url(#pill)"/>
 </rect>
 <rect x="0" y="0" width="512" height="512" fill="url(#glossTop)"/>
</svg>`;

writeFileSync(path.join(PUB, "logo.svg"), LOGO_SVG);

```

```

<line x1="0" y1="-84" x2="0" y2="84" stroke="ffffff" stroke-opacity=".35" stroke-width="6"/>
<rect x="-160" y="-76" width="320" height="80" rx="40" ry="40" fill="url(#glossTop)"/>
<circle cx="104" cy="-30" r="18" fill="ffffff" fill-opacity=".92"/>
<circle cx="-96" cy="34" r="10" fill="ffffff" fill-opacity=".28"/>
</g>
</svg>`;

const F = "Tahoma, DejaVu Sans, sans-serif";
const card = (x, y, w, h, r = 18, fill = "ffffff") =>
 `<rect x="${x}" y="${y}" width="${w}" height="${h}" rx="${r}" fill="${fill}" stroke="#e2e8f0"/>`;
const tile = (x, y, w, icon, num, label, color) =>
 `<g>${card(x, y, w, 96)}<text x="${x + w / 2}" y="${y + 34}" font-size="22" text-anchor="middle" font-family="${F}">${icon}</text>` +
 `<text x="${x + w / 2}" y="${y + 64}" font-size="24" font-weight="bold" fill="${color}" text-anchor="middle" font-family="${F}">${num}</text>` +
 `<text x="${x + w / 2}" y="${y + 84}" font-size="13" fill="#64748b" text-anchor="middle" font-family="${F}">${label}</text></g>`;
const bars = (x, y, w, h, vals, color) => {
 const bw = w / vals.length - 10;
 return vals
 .map((v, i) => {
 const bh = Math.max(6, (v / Math.max(...vals)) * h);
 return `<rect x="${x + i * (bw + 10)}" y="${y + h - bh}" width="${bw}" height="${bh}" rx="5" fill="${color}" opacity="${0.55 + (i / vals.length) * 0.45}"/>`;
 })
 .join("");
};

const header = (w, title) =>
 `<rect width="${w}" height="72" fill="#0f766e"/>` +
 `<text x="${w - 24}" y="34" font-size="19" font-weight="bold" fill="ffffff" text-anchor="end" font-family="${F}">〇〇〇〇〇〇〇〇〇〇〇〇〇〇〇</text>` +
 `<text x="${w - 24}" y="56" font-size="13" fill="#c7f0ea" text-anchor="end" font-family="${F}">${title}</text>` +
 `<circle cx="42" cy="36" r="15" fill="ffffff" opacity=".18"/>`;

const WIDE = `<svg xmlns="http://www.w3.org/2000/svg" width="1280" height="720">
<rect width="1280" height="720" fill="#f1f5f9"/>
${header(1280, "〇〇〇〇〇〇〇〇 〇〇〇〇〇〇 — 〇〇〇〇〇 〇 〇〇〇〇〇〇 〇〇〇〇〇〇〇〇)}
${tile(40, 100, 190, "\u{1F3E5}", "〇〇〇", "〇〇〇〇〇〇〇〇〇〇〇〇", "#0f766e")}
${tile(248, 100, 190, "\u{1FA7A}", "〇〇", "〇〇〇〇〇〇", "#0369a1")}
${tile(456, 100, 190, "\u{1F9FE}", "〇〇〇", "〇〇〇〇〇〇〇", "#4f46e5")}
${tile(664, 100, 190, "\u{1F4CD}", "〇", "〇〇〇〇〇 〇〇〇〇", "#059669")}
${tile(872, 100, 190, "\u{1F4DD}", "〇", "〇〇〇〇〇", "#b45309")}
${tile(1080, 100, 160, "\u{1F464}", "〇〇", "〇〇〇〇〇〇", "#334155")}
${card(40, 220, 760, 300)}
<text x="770" y="252" font-size="16" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="${F}">〇〇〇〇〇〇 〇〇〇〇
〇〇 〇〇 〇〇〇</text>
${bars(80, 280, 660, 200, [40, 62, 55, 88, 72, 96, 110, 85, 120, 100, 135, 148], "#0d9488")}
${card(820, 220, 420, 300)}
<text x="1210" y="252" font-size="16" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="${F}">〇〇〇 〇〇〇〇〇〇〇〇
</text>
<g transform="translate(1030,390)">
 <circle r="78" fill="none" stroke="#0d9488" stroke-width="26" stroke-dasharray="180 490"/>
 <circle r="78" fill="none" stroke="#0ea5e9" stroke-width="26" stroke-dasharray="120 490" stroke-dashoffset="-180"/>
 <circle r="78" fill="none" stroke="#6366f1" stroke-width="26" stroke-dasharray="95 490" stroke-dashoffset="-300"/>
 <circle r="78" fill="none" stroke="#f59e0b" stroke-width="26" stroke-dasharray="95 490" stroke-dashoffset="-395"/>
 <text y="8" font-size="22" font-weight="bold" fill="#0f766e" text-anchor="middle" font-family="${F}">〇〇〇</text>
</g>
${card(40, 540, 1200, 150)}
<text x="1210" y="572" font-size="16" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="${F}">〇〇〇〇〇〇 〇〇〇〇〇〇
〇〇 〇〇〇〇〇〇〇〇〇</text>
${[0, 1, 2]
 .map(
 (i) =>
 `<rect x="60" y="${590 + i * 32}" width="1160" height="26" rx="8" fill="#f8faff"/>` +
 `<text x="1200" y="${608 + i * 32}" font-size="13" fill="#334155" text-anchor="end" font-family="${F}">${["〇〇〇 〇〇〇
〇〇 — 〇〇〇 〇〇〇〇〇 〇〇〇〇〇〇〇〇 〇〇〇", "〇〇〇〇 〇〇〇〇〇 — 〇〇〇〇 〇〇〇〇〇", "〇〇〇 〇〇〇〇〇 — 〇〇〇 〇〇〇〇 〇〇〇〇〇〇 〇〇〇"][i]}</text>`,
)
 .join("")}
</svg>`;

const NARROW = `<svg xmlns="http://www.w3.org/2000/svg" width="720" height="1280">
<rect width="720" height="1280" fill="#f1f5f9"/>
${header(720, "〇〇〇 〇〇〇〇〇〇〇〇 〇〇〇〇〇)}
${card(24, 92, 672, 96)}
<text x="672" y="130" font-size="20" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="${F}">〇〇〇〇 〇〇〇〇〇〇〇〇
〇〇〇〇</text>
<text x="672" y="162" font-size="14" fill="#64748b" text-anchor="end" font-family="${F}">〇〇〇〇〇 〇〇〇〇/〇〇/〇〇 — 〇〇〇〇〇〇 〇〇〇〇〇
〇〇〇</text>
<rect x="24" y="208" width="672" height="92" rx="18" fill="#0f766e"/>
<text x="668" y="248" font-size="19" font-weight="bold" fill="ffffff" text-anchor="end" font-family="${F}">〇〇〇 〇〇〇〇〇〇〇〇
〇〇〇〇〇〇〇〇〇</text>
<text x="668" y="276" font-size="13" fill="#c7f0ea" text-anchor="end" font-family="${F}">〇〇〇 〇〇〇〇〇 〇〇〇〇〇〇〇〇</text>
<rect x="24" y="312" width="672" height="92" rx="18" fill="#0369a1"/>
<text x="668" y="352" font-size="19" font-weight="bold" fill="ffffff" text-anchor="end" font-family="${F}">〇〇〇 〇〇〇〇〇〇〇〇
〇〇〇〇</text>

```

```
<text x="668" y="380" font-size="13" fill="#cfe9fb" text-anchor="end" font-family="{F}">□□ □□□□ □□□□□□</text>
<rect x="24" y="416" width="672" height="92" rx="18" fill="#4f46e5"/>
<text x="668" y="456" font-size="19" font-weight="bold" fill="#ffffff" text-anchor="end" font-family="{F}">□□□ □□□□□□
□□□□□□□□</text>
<text x="668" y="484" font-size="13" fill="#d8d6fb" text-anchor="end" font-family="{F}">□□□ □□□□□ □□□□□□□□□□</text>
$([["□□□□", "□□□□", "□□□□□", "□□□□□□", "□□□□□□", "□□□□□□"]
 .map((t, i) => {
 const x = 24 + (i % 3) * 228;
 const y = 528 + Math.floor(i / 3) * 116;
 return `${card(x, y, 208, 100)}<text x="${x + 104}" y="${y + 60}" font-size="16" font-weight="bold" fill="#334155" t
ext-anchor="middle" font-family="{F}">${t}</text>`;
 })
 .join(""));
${card(24, 772, 672, 300)}
<text x="668" y="806" font-size="16" font-weight="bold" fill="#0f172a" text-anchor="end" font-family="{F}">□□□□ □□□□ □□
□ □□ □□□</text>
${bars(56, 830, 610, 210, [30, 48, 42, 70, 62, 88, 96, 78, 110, 96, 124, 140], "#0ea5e9")}
${card(24, 1092, 672, 92)}
<text x="668" y="1128" font-size="15" font-weight="bold" fill="#0f766e" text-anchor="end" font-family="{F}">□□□□□□□□ □□
□□□□□ □□□□ □□□ □ □□□□□□□□</text>
<text x="668" y="1156" font-size="13" fill="#64748b" text-anchor="end" font-family="{F}">□□□ □□□□□□ + □□□ □□□□□□ □□□□ G
PS</text>
<rect x="0" y="1208" width="720" height="72" fill="#0f766e"/>
<text x="360" y="1252" font-size="15" font-weight="bold" fill="#ffffff" text-anchor="middle" font-family="{F}">□□□ □□□□
□□□ □□ — □□□□ □□□□□□□□ □□□□□□ □ □□</text>
</svg>`;

const BRAND = `<svg xmlns="http://www.w3.org/2000/svg" width="1280" height="720">
 <defs><linearGradient id="bg" x1="0" y1="0" x2="1" y2="1">
 <stop offset="0%" stop-color="#0f766e"/><stop offset="100%" stop-color="#134e4a"/></linearGradient></defs>
 <rect width="1280" height="720" fill="url(#bg)"/>
 <text x="640" y="560" font-size="46" font-weight="bold" fill="#ffffff" text-anchor="middle" font-family="{F}">□□□ □□□
□□□□ □□</text>
 <text x="640" y="612" font-size="22" fill="#a7f3d0" text-anchor="middle" font-family="{F}">□□□□□□ □□□□□□□□□□ □□□□ — □□
□□□□□□□ □□□□□ □□□□□□□ □ □□□□</text>
</svg>`;

export const ICON_SIZES = [96, 128, 144, 152, 180, 192, 256, 384, 512, 1024];

const plate = (size) =>
 Buffer.from(
 `<svg xmlns="http://www.w3.org/2000/svg" width="${size}" height="${size}">
 <defs><linearGradient id="g" x1="0" y1="0" x2="1" y2="1">
 <stop offset="0%" stop-color="#ffffff"/><stop offset="100%" stop-color="#eef2f6"/>
 </linearGradient></defs>
 <rect width="${size}" height="${size}" fill="url(#g)"/>
 </svg>`,
);

/** □□□□□ □□□□ □□ □□□□□□ □□□□ */
export async function makeIcon(sharp, size) {
 return sharp(Buffer.from(LOGO_SVG), { density: 500 })
 .resize(size, size, { fit: "contain", background: { r: 0, g: 0, b: 0, alpha: 0 } })
 .png({ compressionLevel: 9 })
 .toBuffer();
}

/** □□□□□ maskable □□ □□□□□ □□□ □ □□□□□□□□ □□□□ */
export async function makeMaskable(sharp, size) {
 const inner = await sharp(Buffer.from(LOGO_SVG), { density: 500 })
 .resize(Math.round(size * 0.62), Math.round(size * 0.62))
 .png()
 .toBuffer();
 return sharp(plate(size)).composite([{ input: inner, gravity: "center" }]).png({ compressionLevel: 9 }).toBuffer();
}

export async function makeScreenshot(sharp, key) {
 if (key === "mobile-720x1280") return sharp(Buffer.from(NARROW)).png().toBuffer();
 if (key === "brand-1280x720") {
 const logo = await sharp(Buffer.from(LOGO_SVG), { density: 500 }).resize(300, 300).png().toBuffer();
 return sharp(Buffer.from(BRAND)).composite([{ input: logo, top: 140, left: 490 }]).png().toBuffer();
 }
 return sharp(Buffer.from(WIDE)).png().toBuffer();
}

export { LOGO_SVG };

async function main() {
 let sharp;
 try {
 sharp = (await import("sharp")).default;
 } catch {
 console.warn("□□ sharp □□ □□□□ □□□□ □□□□□□□□ □□□□ □□□□ □□□□□□□□ □□□□ □□□□□□□□.");
 return;
 }
}
```



```

mkdirSync(ICONS, { recursive: true });
mkdirSync(SHOTS, { recursive: true });
writeFileSync(path.join(PUB, "logo.svg"), LOGO_SVG);
writeFileSync(path.join(PUB, "icon.svg"), LOGO_SVG);
// 000000 SVG (0000) 0000 PNG 000000 00000000 00 00 0000 0000 PNG0
// 00000000 00000000 00000000 0000 000000 00 00 0000 000000.
mkdirSync(path.join(PUB, "src-svg"), { recursive: true });
writeFileSync(path.join(PUB, "src-svg", "desktop-1280x720.svg"), WIDE);
writeFileSync(path.join(PUB, "src-svg", "mobile-720x1280.svg"), NARROW);
writeFileSync(path.join(PUB, "src-svg", "brand-1280x720.svg"), BRAND);

for (const s of ICON_SIZES) {
 writeFileSync(path.join(ICONS, `icon-${s}.png`), await makeIcon(sharp, s));
}
for (const s of [192, 512]) {
 writeFileSync(path.join(ICONS, `maskable-${s}.png`), await makeMaskable(sharp, s));
}
writeFileSync(path.join(PUB, "apple-touch-icon.png"), await makeMaskable(sharp, 180));
writeFileSync(path.join(PUB, "favicon-32.png"), await makeIcon(sharp, 32));
writeFileSync(path.join(PUB, "favicon-16.png"), await makeIcon(sharp, 16));

for (const k of ["desktop-1280x720", "mobile-720x1280", "brand-1280x720"]) {
 writeFileSync(path.join(SHOTS, `${k}.png`), await makeScreenshot(sharp, k));
}

console.log(
 `0 0000 000000 PWA 0000 00: ${ICON_SIZES.length} 00000 + 0 maskable + 0 000000000000 (${existsSync(ICONS) ? "ok" : "er
r"})`,
);
}

if (import.meta.url === `file://${process.argv[1]}`) {
 main().catch((e) => {
 console.error("0000 00 0000 000000:", e?.message ?? e);
 process.exit(0); // build 00 000000 0000 00000000 0000000000 00000000 000000
 });
}

@@ scripts/simplify-geojson.mjs javascript
#!/usr/bin/env node
/** 0000000000 0000000 0000000000 00 0000000000 Ramer-Douglas-Peucker */
import { readFileSync, writeFileSync, statSync } from "node:fs";

const input = process.argv[2] || "public/data/iran-provinces.geojson";
const output = process.argv[3] || "public/data/iran-provinces-lite.geojson";
const tolerance = Number(process.argv[4] || 0.012);

const distanceToSegment = (p, a, b) => {
 const dx = b[0] - a[0];
 const dy = b[1] - a[1];
 if (!dx && !dy) return Math.hypot(p[0] - a[0], p[1] - a[1]);
 const t = Math.max(0, Math.min(1, ((p[0] - a[0]) * dx + (p[1] - a[1]) * dy) / (dx * dx + dy * dy)));
 return Math.hypot(p[0] - (a[0] + t * dx), p[1] - (a[1] + t * dy));
};

function simplify(points, eps) {
 if (points.length <= 4) return points;
 const wasClosed = points[0][0] === points.at(-1)[0] && points[0][1] === points.at(-1)[1];
 const line = wasClosed ? points.slice(0, -1) : points;
 // 0000 0000 000000 00 00000000 0000 0000 00 0000 0000 000 00000000 00 0000 0000 0000
 let split = 0;
 if (wasClosed) {
 let max = -1;
 for (let i = 1; i < line.length; i++) {
 const d = Math.hypot(line[i][0] - line[0][0], line[i][1] - line[0][1]);
 if (d > max) { max = d; split = i; }
 }
 const a = rdp(line.slice(0, split + 1), eps);
 const b = rdp([...line.slice(split), line[0]], eps);
 const result = [...a.slice(0, -1), ...b];
 return result.length >= 4 ? result : points;
 }
 return rdp(line, eps);
}

function rdp(points, eps) {
 if (points.length <= 2) return points;
 let max = 0;
 let idx = 0;
 const first = points[0];
 const last = points.at(-1);
 for (let i = 1; i < points.length - 1; i++) {
 const d = distanceToSegment(points[i], first, last);
 if (d > max) { max = d; idx = i; }
 }

```

```

 }
 if (max > eps) {
 const left = rdp(points.slice(0, idx + 1), eps);
 const right = rdp(points.slice(idx), eps);
 return [...left.slice(0, -1), ...right];
 }
 return [first, last];
 }
}

const geo = JSON.parse(readFileSync(input, "utf8"));
for (const feature of geo.features) {
 const g = feature.geometry;
 if (g.type === "Polygon") g.coordinates = g.coordinates.map((ring) => simplify(ring, tolerance));
 if (g.type === "MultiPolygon") {
 g.coordinates = g.coordinates.map((poly) => poly.map((ring) => simplify(ring, tolerance)));
 }
 const raw = String(feature.properties?.province_name || feature.properties?.name || "");
 const name = raw.replace(/^[0-9]{4}\s+/, "");
 feature.properties.name_fa = name === "□□□□□□□□ □□□□ □□□□" ? "□□□□□□□□ □□□□□□□□" : name;
}
writeFileSync(output, JSON.stringify(geo));
console.log(`□ ${geo.features.length} □□□□: ${statSync(input).size} → ${statSync(output).size} bytes`);

@@ scripts/start.mjs javascript
#!/usr/bin/env node
/**
 * Self-healing production starter.
 *
 * □□□□ □□□□□□□□ (□□□□ Render □□□□ Build Command □□□□ □□□□) □□□ `npm start` □□
 * □□□□ □□□□□□□□ □ □□ □□□□□ □□□□□□□□ `next` □□□□ □□□□□ □ □□□□
 * "Could not find a production build in the 'next' directory" □□□□□□□□□□.
 * □□□ □□□□□□□□ □□□ □□□□ □□□□□ □□□□□ □□□□□ □□□□□ `next build` □□ □□□□ □□□□□ □
 * □□□ □□□□ □□ □□□ □□□□□ □□ □□□□□ □□□□□ □□□□ □□□□□ □□□□□□□□.
 */
import { spawn } from "node:child_process";
import { existsSync } from "node:fs";
import path from "node:path";
import process from "node:process";

const root = process.cwd();
const buildIdFile = path.join(root, ".next", "BUILD_ID");
const port = process.env.PORT || "3000";
const host = process.env.HOST || "0.0.0.0";
const nextBin = path.join(root, "node_modules", "next", "dist", "bin", "next");

function run(args, label) {
 return new Promise((resolve, reject) => {
 const child = spawn(process.execPath, [nextBin, ...args], {
 stdio: "inherit",
 env: process.env,
 });
 child.on("exit", (code, signal) => {
 if (signal) return reject(new Error(`${label} terminated by ${signal}`));
 if (code !== 0) return reject(new Error(`${label} exited with code ${code}`));
 resolve();
 });
 child.on("error", reject);
 });
}

async function main() {
 if (!existsSync(buildIdFile)) {
 console.log("□□ production build not found – running `next build` first...");
 await run(["build"], "next build");
 console.log("□ build finished, starting server...");
 }
 console.log(`□ starting Next.js on ${host}:${port}`);
 await run(["start", "-H", host, "-p", String(port)], "next start");
}

main().catch((err) => {
 console.error("□ start failed:", err.message);
 process.exit(1);
});

@@ public/data/iran-provinces-lite.geojson text
{"type":"FeatureCollection","features":[{"type":"Feature","properties":{"province_name":"□□□□□ □□□□□□ □□□□□","name_fa":"□□□□□□ □□□□□"},
"geometry":{"type":"Polygon","coordinates":[[[58.0196353,36.5830045],[57.9382428,36.5841636],[57.7773976,
36.6302737],[57.4898883,36.7691637],[57.03292,36.8713972],[56.7212399,36.7988951],[56.6512373,36.7677781],[56.5320652,36.
6675034],[56.5017855,36.6644417],[56.4465028,36.6941673],[56.3595572,36.6827762],[56.3651105,36.7530568],[56.2645516,36.
8380601],[56.2214921,36.9008362],[56.0145378,36.8344501],[55.9842616,36.9000082],[55.9791784,36.9962453],[56.0145379,37.
0069571],[56.0400978,37.1002915],[55.9942616,37.1191819],[56.0628744,37.1608486],[56.0087049,37.2344567],[55.9514823,37.
2044565],[55.9087027,37.2055724],[55.908423,37.3323405],[56.013985,37.366409],[56.0012054,37.4425221],[56.0584315,37.51
91926],[56.0612082,37.5722467],[56.118711,37.6347476],[56.2156603,37.8594822],[56.2803861,37.8672577],[56.3103894,37.910
3136],[56.2889756,38.0846583],[56.335248,38.0772496],[56.352079,38.1171643],[56.319101,38.176137],[56.3327792,38.1975537

```

], [56.442778, 38.26], [56.546667, 38.266111], [56.613889, 38.240556], [56.756389, 38.285], [56.8404712, 38.2298997], [56.9919985, 38.2143439], [57.0122329, 38.1871898], [57.025, 38.201944], [57.076111, 38.199444], [57.183675, 38.2841776], [57.216285, 38.2804549], [57.243889, 38.270833], [57.288333, 38.170556], [57.371111, 38.093056], [57.356944, 37.985833], [57.5121923, 37.919263], [57.5796241, 37.9380422], [57.7203676, 37.9218564], [57.7541404, 37.8974949], [57.7958352, 37.899553], [57.8254738, 37.8632465], [57.8915765, 37.8707624], [58.0278554, 37.8059025], [58.2054052, 37.7743691], [58.2287711, 37.7296962], [58.2270467, 37.6840365], [58.3112048, 37.66222], [58.3709134, 37.5846745], [58.3549337, 37.4083545], [58.3160405, 37.3558537], [58.3274269, 37.32585], [58.4229856, 37.2563997], [58.3896527, 37.2119583], [58.3243732, 37.1994596], [58.3049269, 37.1680631], [58.2429806, 37.1811205], [58.1590148, 37.0751305], [58.2440936, 37.046954], [58.2496468, 37.0200059], [58.2927063, 37.002787], [58.3010527, 36.9644639], [58.2588132, 36.9108352], [58.1343611, 36.8647246], [58.0988083, 36.8688929], [58.1107479, 36.7727824], [58.0879746, 36.722503], [58.1032478, 36.63139], [58.0824144, 36.5999959], [58.0196353, 36.5830045]]]]], {"type": "Feature", "properties": {"province\_name": "□□□□□ □□□□□ □□□□□", "name\_fa": "□□□□□□ □□□□□", "geometry": {"type": "Polygon", "coordinates": [[[60.533076, 34.0002495], [60.3344603, 33.9838216], [60.1830615, 33.8668734], [60.1441686, 33.856312], [60.0880562, 33.9277046], [59.9912849, 34.004841], [59.610536, 34.0154875], [59.5360906, 34.1040964], [59.347193, 34.1079869], [59.2594074, 34.1499297], [59.162462, 34.1190981], [59.0441236, 34.1238187], [58.9796708, 34.0946565], [58.8638325, 34.1143781], [58.7857734, 34.0818761], [58.7057745, 34.099937], [58.6363217, 34.0216001], [58.4652041, 34.1274391], [58.3707267, 34.2893055], [58.3201997, 34.3146603], [58.2640871, 34.3213297], [58.1890811, 34.2416029], [58.0710226, 34.2349424], [58.0351832, 34.2391031], [57.9160113, 34.32161], [57.7660067, 34.3471594], [57.6404487, 34.5205058], [57.5566901, 34.598], [57.7021148, 34.774395], [57.7865737, 34.822289], [57.7115619, 34.9199611], [57.5729435, 35.0001039], [57.552944, 35.0591252], [57.5157178, 35.0713477], [57.3176536, 35.0960721], [57.1526455, 35.0755155], [57.0006939, 34.9980118], [56.8798552, 34.9799613], [56.8337425, 34.954405], [56.820686, 34.9374544], [56.8515221, 34.9191232], [56.7901295, 34.8374511], [56.7398501, 34.7827327], [56.6723442, 34.7543938], [56.4384468, 34.682448], [56.2279736, 34.7104172], [56.4884468, 35.0285743], [56.6265119, 35.0649624], [56.6909577, 35.0624638], [56.8437427, 35.1977425], [56.9370816, 35.221075], [57.0002296, 35.2152486], [57.0520871, 35.3149691], [56.9912513, 35.441914], [57.0001414, 35.4874687], [57.0476409, 35.5355282], [57.0348643, 35.5641375], [56.7217953, 35.8599842], [56.7312388, 36.0899851], [56.800408, 36.2386072], [56.7331824, 36.313053], [56.6984597, 36.3883299], [56.8167982, 36.4619424], [56.8640245, 36.5208313], [56.8615248, 36.5680528], [56.7692958, 36.6158298], [56.7584628, 36.7038922], [56.6020678, 36.669163], [56.5351185, 36.6783365], [56.6587373, 36.7727843], [56.7122399, 36.7988951], [57.0295866, 36.8716763], [57.4898883, 36.7691637], [57.5887801, 36.7341645], [57.7521249, 36.6405581], [57.9360229, 36.5847219], [58.0824144, 36.5999959], [58.1057475, 36.6455556], [58.0879746, 36.722503], [58.1107479, 36.7727824], [58.0988083, 36.8688929], [58.1343611, 36.8647246], [58.2588132, 36.9108352], [58.3010527, 36.9644639], [58.2927063, 37.002787], [58.2496468, 37.0200059], [58.2440936, 37.046954], [58.1590148, 37.0751305], [58.2429806, 37.1811205], [58.3049269, 37.1680631], [58.3243732, 37.1994596], [58.382706, 37.2077922], [58.4204854, 37.24390105], [58.3168736, 37.3516793], [58.3549337, 37.4083545], [58.3709134, 37.5846745], [58.3112048, 37.66222], [58.3596512, 37.6611066], [58.3884341, 37.6269846], [58.474167, 37.640556], [58.55, 37.705833], [58.712778, 37.648611], [58.804444, 37.666944], [58.822778, 37.703611], [58.876389, 37.669167], [59.060278, 37.63], [59.2292497, 37.5120499], [59.3534794, 37.5284162], [59.392131, 37.4791374], [59.3814207, 37.4497464], [59.4113447, 37.4357739], [59.377877, 37.4075897], [59.3968977, 37.3173029], [59.4334955, 37.3054859], [59.4834259, 37.248484], [59.491833, 37.2300429], [59.4747378, 37.2147753], [59.5517307, 37.1775677], [59.5525408, 37.136204], [59.6433988, 37.1192367], [59.6534908, 37.1422099], [59.6789985, 37.1412196], [59.7474053, 37.1077803], [59.7718478, 37.1253495], [59.8995292, 37.0514459], [59.9814679, 37.0471137], [60.0430687, 37.0207635], [60.0890045, 36.9717244], [60.1042036, 36.9244661], [60.266905, 36.7572095], [60.3455423, 36.6293939], [60.6752205, 36.6222116], [61.1461942, 36.646493], [61.1860031, 36.5643208], [61.1822189, 36.525769], [61.1568767, 36.5074018], [61.1757576, 36.4641629], [61.166353, 36.4135861], [61.1425, 36.392778], [61.1604472, 36.3305686], [61.1491692, 36.3174795], [61.2193283, 36.1923075], [61.2251307, 36.1405553], [61.2106077, 36.1277446], [61.2309759, 36.1175896], [61.164722, 36.035278], [61.17, 35.995278], [61.1171678, 35.9621307], [61.1836284, 35.9430414], [61.245, 35.897222], [61.261944, 35.813333], [61.246389, 35.7175], [61.2166545, 35.675801], [61.2737591, 35.6094121], [61.2825792, 35.5379033], [61.22207, 35.442909], [61.227161, 35.421655], [61.187459, 35.399592], [61.200429, 35.373588], [61.17575, 35.343314], [61.190296, 35.292865], [61.157867, 35.275352], [61.108955, 35.282812], [61.091775, 35.263737], [61.09836, 35.2185], [61.117854, 35.209644], [61.08717, 35.164807], [61.13257, 35.135873], [61.141486, 35.104899], [61.126077, 35.095384], [61.145771, 35.080611], [61.117421, 35.073975], [61.125996, 35.058858], [61.1117, 35.060622], [61.099702, 35.024205], [61.121625, 35.003515], [61.062264, 34.917265], [61.073857, 34.908623], [61.055943, 34.904862], [61.080891, 34.876279], [61.075061, 34.844264], [61.002802, 34.7092165], [61.000798, 34.642976], [60.984492, 34.619758], [60.922214, 34.623018], [60.897272, 34.56726], [60.84865, 34.542077], [60.739083, 34.518086], [60.809274, 34.471653], [60.849631, 34.368448], [60.918373, 34.304756], [60.6720322, 34.3027538], [60.66949, 34.272504], [60.585931, 34.21266], [60.5374215, 34.150697], [60.51788, 34.0724107], [60.533076, 34.0002495]]]]], {"type": "Feature", "properties": {"province\_name": "□□□□□ □□□□□ □□□□□", "name\_fa": "□□□□□□ □□□□□", "geometry": {"type": "Polygon", "coordinates": [[[59.9397084, 30.7612473], [59.8774826, 30.7098487], [59.4635747, 30.5167946], [59.1938393, 30.8168007], [59.0785612, 30.9079166], [59.0585552, 31.008478], [58.780493, 31.1554155], [58.3899192, 31.4765441], [58.0782371, 31.6587712], [57.5334915, 31.8234911], [57.1026374, 31.8996052], [56.9395766, 31.9540503], [56.7734587, 31.9557208], [56.6048378, 31.922106], [56.5967812, 31.9996114], [56.6534506, 32.0873839], [56.5437256, 32.2315637], [56.3623284, 32.4057319], [56.221487, 32.5937882], [55.9720338, 32.7396312], [55.8764718, 32.7532385], [55.8075795, 32.7874056], [55.6861878, 32.931025], [55.6672981, 32.9987977], [55.5517337, 32.515794], [55.386452, 33.3649236], [55.497844, 33.5290919], [55.4786782, 34.2518854], [55.9195327, 34.2457751], [56.2279736, 34.7104172], [56.4323336, 34.6818977], [56.6115081, 34.7324514], [56.7398501, 34.7827327], [56.8495757, 34.9149554], [56.8442988, 34.9352351], [56.8198526, 34.9391233], [56.8645754, 34.9744045], [57.0006939, 34.9980118], [57.1526455, 35.0755155], [57.3176536, 35.0960721], [57.5462775, 35.0638518], [57.5729435, 35.0001039], [57.7043352, 34.9260685], [57.7860073, 34.8238449], [57.7021148, 34.774395], [57.5566901, 34.598], [57.6404487, 34.5205058], [57.7660067, 34.3471594], [57.9160113, 34.32161], [58.0351832, 34.2391031], [58.0710226, 34.2349424], [58.1890811, 34.2416029], [58.2640871, 34.3213297], [58.3201997, 34.3146603], [58.3707267, 34.2893055], [58.4652041, 34.1274391], [58.6363217, 34.0216001], [58.7057745, 34.099937], [58.7857734, 34.0818761], [58.8638325, 34.1143781], [58.9796708, 34.0946565], [59.0441236, 34.1238187], [59.162462, 34.1190981], [59.2594074, 34.1499297], [59.347193, 34.1079869], [59.5360906, 34.1040964], [59.610536, 34.0154875], [59.9912849, 34.004841], [60.0880562, 33.9277046], [60.1441686, 33.856312], [60.1830615, 33.8668734], [60.3344603, 33.9838216], [60.533076, 34.0002495], [60.5822135, 33.8054437], [60.5468937, 33.7314095], [60.5514636, 33.6400296], [60.5812773, 33.597023], [60.7034182, 33.54552], [60.9027952, 33.5399665], [60.9459035, 33.5134944], [60.9287078, 33.4972033], [60.8824864, 33.5052436], [60.8509231, 33.4820012], [60.8664836, 33.421487], [60.6340548, 33.2281412], [60.5833775, 33.1299045], [60.6294139, 33.0212334], [60.7095814, 32.6958555], [60.814771, 32.4812948], [60.882921, 32.197736], [60.8159668, 32.0265393], [60.832519, 31.979859], [60.8134835, 31.7087013], [60.8569712, 31.4632045], [60.8355847, 31.4140384], [60.7430788, 31.3290335], [60.6583536, 31.2859724], [60.3877834, 31.2245871], [60.2780581, 31.1518107], [60.2486117, 31.1098626], [60.299171, 31.0226379], [60.2008322, 30.9401374], [59.9774885, 30.9198562], [59.9519285, 30.8823533], [59.9397084, 30.7612473]]]]], {"type": "Feature", "properties": {"province\_name": "□□□□ □□□□□ □ □□□□□□□", "name\_fa": "□□□□□□ □ □□□□□□□", "geometry": {"type": "Polygon", "coordinates": [[[61.5497327, 24.8465103], [61.3546759, 24.8662768], [60.5765756, 25.0727683], [60.4193982, 25.0794386], [59.5677381, 25.1954933], [59.2187761, 25.1994033], [59.2104926, 25.5578014], [59.2357725, 25.6367001], [59.2113266, 25.6914167], [59.218833, 25.7405878], [59.0835491, 25.8508662], [59.072156, 25.8944817], [59.9771108, 25.937814], [59.0190967, 25.9733706], [59.0804892, 25.996152], [59.0827163, 26.0303127], [59.0218772, 26.1308698], [59.0288239, 26.166987], [59.9829846, 26.2172627], [59.9760453, 26.28671], [58.9760453, 26.3355961], [58.8796535, 26.353383], [58.8843736, 26.4236558], [58.8438142, 26.4831065], [58.9224331, 26.5150435], [59.0003551, 26.5003229], [59.0060451, 26.5281025], [59.0374382, 26.5475481], [58.9996588, 26.596439], [59.0177119, 26.6561592], [58.9904926, 26.75922], [58.9057608, 26.7878299], [58.8507615, 26.8311605], [58.8643748, 26.9086636], [58.9129876, 26.9999296], [58.8485422, 27.1017277], [58.8310364, 27.2536735], [58.9010426, 27.5764568], [58.8904901, 27.8981235], [58.9324362, 27.9731322], [58.9338231, 28.0298011], [59.0145546, 28.1070656], [59.1483843, 28.1273763], [59.1614392, 28.2507186], [59.1261603, 28.2984927], [59.1225478, 28.3632191], [59.0619893, 28.4162781], [59.2305063, 28.4764732], [59.536348, 28.4731397], [59.5652411, 28.5098105], [59.546075, 28.6236971], [59.5627415, 28.6903676], [59.5505158, 28.8159292], [59.4969032, 28.96093], [59.5055168, 28.9998199], [59.5707959, 29.0820391], [59.4944042, 29.2445422], [59.4635717, 29.3742691], [59.4335736, 29.8717785], [59.4494066, 29.9934475], [59.4249612, 30.2526259], [59.4635747, 30.5167946], [59.8774826, 30.7098487], [59.9422086, 30.7656877], [59.9519285, 30.8823533], [59.9774885, 30.9198562], [60.1655526

,30.9315265],[60.2219452,30.9512486],[60.299171,31.0226379],[60.2486117,31.1098626],[60.2755585,31.1498609],[60.3877834,31.2245871],[60.6583536,31.2859724],[60.7430788,31.3290335],[60.8372511,31.4162593],[60.856057,31.4865192],[61.7066673,31.375896],[61.7783849,31.3022694],[61.7716308,31.2062588],[61.8045461,31.1657792],[61.8390347,31.036451],[61.8238446,31.0008681],[61.8347019,30.9740985],[61.785528,30.9332587],[61.8016237,30.8325765],[60.874444,29.855556],[61.1625238,29.5333387],[61.308947,29.395202],[61.360123,29.378458],[61.372156,29.321058],[61.352909,29.275983],[61.430218,29.199095],[61.412681,29.175421],[61.420823,29.135652],[61.474371,29.123403],[61.471443,29.090872],[61.507259,29.061686],[61.478861,29.023639],[61.533361,29.010222],[61.6455716,28.8029274],[61.910018,28.564377],[62.043906,28.510134],[62.389486,28.432248],[62.488771,28.373405],[62.512278,28.322855],[62.597191,28.24842],[62.7929605,28.2789044],[62.7610215,28.0259231],[62.8289545,27.7644629],[62.8484229,27.4629391],[62.791328,27.338783],[62.824803,27.299191],[62.80882,27.264235],[62.775572,27.252896],[62.820731,27.218458],[62.873027,27.228859],[62.936974,27.20144],[63.040552,27.23729],[63.198716,27.259436],[63.224699,27.225621],[63.281777,27.213732],[63.3332704,27.1184003],[63.3071253,27.1296461],[63.2510944,27.0921506],[63.246334,26.849903],[63.185979,26.836726],[63.208945,26.77333],[63.191485,26.770481],[63.19692,26.647126],[63.108061,26.652392],[63.048857,26.627541],[62.989971,26.649943],[62.824409,26.637955],[62.7947,26.651003],[62.7535368,26.6450164],[62.738103,26.613237],[62.651571,26.610976],[62.604098,26.576273],[62.445165,26.566895],[62.4211762,26.5331578],[62.370225,26.539378],[62.303304,26.508287],[62.306943,26.462772],[62.284882,26.439747],[62.257767,26.446225],[62.259498,26.408339],[62.284695,26.380344],[62.250133,26.346581],[62.271944,26.328003],[62.228244,26.329538],[62.215011,26.267929],[62.125388,26.273986],[62.098288,26.285776],[62.085181,26.318861],[62.033364,26.334491],[62.021768,26.302815],[61.961143,26.260217],[61.896325,26.261399],[61.832838,26.059377],[61.828691,26.031244],[61.846411,26.018672],[61.818359,25.967194],[61.831706,25.911402],[61.85684,25.914788],[61.890722,25.883199],[61.84219,25.877674],[61.820408,25.849211],[61.851539,25.816431],[61.836235,25.798511],[61.840067,25.74484],[61.798234,25.717887],[61.736329,25.74172],[61.672325,25.647835],[61.6505818,25.3058125],[61.5755479,25.0453876],[61.5497327,24.8465103]]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000\u0000\u0000","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[51.9649046,34.8652387],[51.7332272,35.0016298],[51.5993289,35.0177316],[51.5815524,35.0488522],[51.48877,35.1069061],[51.1451407,35.1805155],[51.0123591,35.1380151],[50.8692974,35.1596891],[50.8490093,35.1730286],[50.890687,35.2274604],[50.9615226,35.417188],[50.9920724,35.4371339],[50.8956036,35.4850921],[50.7676246,35.4869185],[50.6259494,35.5474747],[50.353419,35.5706023],[50.3722676,35.6189298],[50.3970214,35.6383536],[50.7015885,35.6741458],[50.7935226,35.7036144],[50.9976279,35.70036],[51.0298331,35.7422542],[51.0882389,35.7222773],[51.0941937,35.7577693],[51.1082599,35.7517637],[51.0951403,35.8122012],[51.2432017,35.9399772],[51.4130707,35.8931068],[51.4065431,35.977209],[51.4568225,36.0608238],[51.4592965,36.1400995],[51.760449,36.0066529],[51.8782311,35.8599847],[52.0726856,35.7633173],[52.2374138,35.7696997],[52.2921399,35.8155375],[52.3782555,35.8485917],[52.4357581,35.9122089],[52.5679864,35.943313],[52.7049383,35.9280361],[52.9368923,35.8338723],[53.0174514,35.8535946],[53.0471743,35.8199818],[53.1532932,35.8113644],[53.1527365,35.7919297],[52.999675,35.6813629],[52.8771663,35.6260885],[52.801054,35.5494151],[52.6977153,35.5519216],[52.555207,35.4119141],[52.3829791,35.3380212],[52.2576937,35.4405264],[52.0351831,35.4935832],[51.9857369,35.575526],[51.9296242,35.5585843],[51.9124013,35.4144115],[51.9439291,35.3613021],[51.8729552,35.2783007],[51.954901,35.1507968],[52.0346303,35.0905171],[52.0446302,35.0496826],[52.0015707,34.9185647],[51.9649046,34.8652387]]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000\u0000\u0000","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[50.4676139,34.1904934],[50.4284443,34.211324],[50.4245543,34.2774425],[50.3601083,34.3488291],[50.3317721,34.4068837],[50.3351052,34.4421688],[50.1464877,34.4752237],[50.1570406,34.5091175],[50.0862046,34.5835621],[50.1081511,34.6238331],[50.1220405,34.7555112],[50.1689866,34.8216255],[50.2534357,34.8141193],[50.3412177,34.7599505],[50.4284433,34.7541222],[50.451223,34.8269037],[50.4892791,34.8602302],[50.705123,34.8682868],[50.6923434,34.9518961],[50.6498404,34.9996803],[50.6537302,35.0182898],[50.8490093,35.1730286],[50.9165201,35.1432915],[51.0123591,35.1380151],[51.1034746,35.178017],[51.1704203,35.1782967],[51.48877,35.1069061],[51.5815524,35.0488522],[51.5993289,35.0177316],[51.7332272,35.0016298],[51.9649046,34.8652387],[51.9143454,34.6969028],[51.8403823,34.5680611],[51.8341678,34.5059559],[51.3259864,34.4410599],[51.1273654,34.4327213],[51.0115269,34.3721675],[51.0140271,34.287443],[50.9948608,34.234664],[50.9081887,34.1518826],[50.8848555,34.1913332],[50.815125,34.2047764],[50.7962399,34.2360537],[50.727624,34.2152142],[50.6509551,34.2557736],[50.5956723,34.2168837],[50.4676139,34.1904934]]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000\u0000\u0000","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[50.2948234,35.5610801],[50.2295442,35.541361],[50.1631517,35.6105336],[50.2437107,35.6477508],[50.250403,35.806938],[50.2970462,35.8224764],[50.3731618,35.8983085],[50.4259411,36.0016496],[50.5473363,36.0183135],[50.5776122,36.0824867],[50.6781745,36.1152612],[50.4320636,36.2452451],[50.490664,36.3022661],[50.4348301,36.3288809],[50.4959428,36.3422144],[50.7498434,36.3180525],[50.9506879,36.2622126],[51.2015319,36.1624881],[51.2679243,36.1841587],[51.315984,36.1544326],[51.4592965,36.1400995],[51.4568225,36.0608238],[51.4065431,35.977209],[51.4130707,35.8931068],[51.2432017,35.9399772],[51.0951403,35.8122012],[51.1082599,35.7517637],[51.0941937,35.7577693],[51.0882389,35.7222773],[51.0298331,35.7422542],[50.9976279,35.70036],[50.8353366,35.7086742],[50.7015885,35.6741458],[50.3970214,35.6383536],[50.353419,35.5706023],[50.2948234,35.5610801]]]]},{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000\u0000\u0000","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[46.9310485,36.7552725],[46.8732689,36.7866601],[46.8988245,36.8947151],[46.8427122,36.927501],[46.8307648,36.9675048],[46.8627115,36.9997297],[46.7893754,37.0541661],[46.7063165,37.0722294],[46.6421472,37.0411149],[46.4607487,37.0202777],[46.4960282,37.0919479],[46.4476919,37.1013887],[46.4124124,37.0563881],[46.2862944,36.9716711],[46.1987926,36.9444408],[46.1774022,36.9552699],[46.174566,36.9955512],[46.0046132,37.1505605],[45.7523794,37.1822271],[45.7808896,37.228471],[45.5946336,37.4257989],[45.3563683,37.7386848],[45.292854,37.8604891],[45.2801507,37.9420312],[45.3749078,38.0226729],[45.3484723,38.1245645],[45.30281,38.1583172],[45.1280592,38.1915139],[45.0820642,38.3325346],[45.2104025,38.4300311],[45.2698514,38.4417052],[45.339577,38.4864231],[45.2929078,38.5339291],[45.2334572,38.66504],[45.2756837,38.7319877],[45.2654031,38.8550443],[45.329295,38.8922705],[45.3926311,38.9714417],[45.4529047,38.9929965],[45.5415028,38.9648285],[45.580174,38.9789688],[45.6145737,38.9416943],[45.6924464,38.9514945],[45.736821,38.9275348],[45.7979968,38.9249278],[45.8152735,38.9052549],[45.8493614,38.9095575],[45.9087076,38.8739169],[45.9445223,38.8965928],[46.0120046,38.8718894],[46.0654204,38.8874386],[46.1476006,38.8418111],[46.1841363,38.8414876],[46.3461404,38.9168198],[46.4210438,38.8830219],[46.4534998,38.8990073],[46.522597,38.8889067],[46.5339832,38.867172],[46.6549773,38.9540911],[46.6969383,39.0190302],[46.7536928,39.0259147],[46.7683432,39.0704193],[46.8342366,39.1144444],[46.8390194,39.1374974],[46.9255441,39.1647287],[46.9624954,39.1403575],[47.0191103,39.1670014],[47.0539374,39.1939118],[47.0580694,39.2398185],[47.0983303,39.2767338],[47.315114,39.3691936],[47.363399,39.4217256],[47.439111,39.3372671],[47.4307925,39.3186376],[47.5049349,39.2954356],[47.4810658,39.2725645],[47.518569,39.2197857],[47.5194023,39.1508961],[47.4974563,39.1283969],[47.5274594,39.0806154],[47.5052366,39.0353314],[47.5230132,39.0142262],[47.50218,39.0000537],[47.5630161,39.9775567],[47.5266267,38.9586645],[47.5363466,38.9500526],[47.5116235,38.9158902],[47.5980199,38.8189437],[47.4696776,38.7914396],[47.4313449,38.7564406],[47.4357886,38.7003296],[47.5257943,38.6697677],[47.564954,38.6422788],[47.5688509,38.6117084],[47.4985682,38.5811588],[47.4641225,38.4780964],[47.345784,38.3569797],[47.2907813,38.3319841],[47.2871719,38.302263],[47.3985677,38.2378125],[47.4688466,38.2494817],[47.4827366,38.1819793],[47.6171887,38.2183641],[47.7246942,38.2122555],[47.7874733,38.2514228],[47.8521997,38.1466985],[47.8622001,38.0066999],[47.9007032,37.9908649],[47.9377595,37.9466916],[47.887757,37.8978047],[47.8930368,37.8683574],[47.977486,37.8774863],[47.999403,37.8949567],[48.1511045,37.8103041],[48.1352716,37.7522446],[48.0666557,37.7294704],[48.0255429,37.654462],[48.0216532,37.6122422],[48.079156,37.5783566],[48.0530467,37.5333525],[48.0608231,37.5147348],[48.1552721,37.4783524],[48.1494391,37.416124],[48.2255553,37.3661248],[48.2441653,37.3269628],[48.2600016,37.329455],[48.338375,37.2483443],[48.1502733,37.2211248],[48.0997139,37.1558395],[48.0411012,37.1494499],[47.9993331,37.112217],[47.9724852,37.1552812],[47.8719263,37.1747277],[47.7788675,37.1322247],[47.7660877,37.0763991],[47.663029,37.0994478],[47.59886,37.0375081],[47.5241339,37.0672274],[47.4924644,37.0000012],[47.3027333,36.8972287],[47.1982853,36.7994371],[47.1741153,36.791667],[47.1405026,36.8322183],[47.1024463,36.8324973],[47.0218869,36.7908292],[47.0029977,36.7466648],[46.9310485,36.7552725]]]]},{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000\u0000\u0000","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[54.4816862,29.6409672],[54.2427864,29.5962233],[54.1030585,29.7928965],[54.0980585,29.9384533],[53.9055514,30.106788],[53

.8644386,30.2026288],[53.9255511,30.3340175],[53.8986047,30.4456825],[53.9308275,30.5401367],[53.8794384,30.630138],[53.734987,30.6890253],[53.4585844,30.7373638],[53.3774721,30.8254161],[52.9693979,31.0145934],[52.9118953,31.1212617],[52.8335564,31.1790434],[52.8505028,31.2143124],[52.9457847,31.240988],[52.9782876,31.3226504],[52.9507846,31.3898795],[52.8010566,31.4948743],[52.9977306,31.4962647],[53.1355152,31.5423805],[53.2035743,31.5901582],[53.1402385,31.6804336],[53.0530131,31.7459903],[53.009677,31.8546066],[53.0424564,31.9898882],[53.058846,32.3321159],[53.051066,32.4146177],[53.0107865,32.5090657],[53.1866274,32.6176809],[53.4624737,32.6951803],[53.842492,32.6860155],[53.941384,32.7988012],[54.2241771,32.8988003],[54.283623,32.9601897],[54.3075126,33.0301923],[54.3483488,33.0529635],[54.5500228,33.0907497],[54.7447535,33.0657485],[54.790313,33.1179718],[54.8972616,33.1801987],[54.9917104,33.1257457],[55.2225541,33.324088],[55.386452,33.3649236],[55.5117337,33.2515794],[55.6672981,32.9987977],[55.6861878,32.931025],[55.8075795,32.7874056],[55.8764718,32.7532385],[55.9720338,32.7396312],[56.221487,32.5937882],[56.3623284,32.4057319],[56.5912282,32.1740559],[56.6545639,32.0812801],[56.5967812,31.9996114],[56.6065043,31.8682121],[56.5792811,31.7115449],[56.538725,31.7076627],[56.2556554,31.5534935],[56.1605022,31.5526083],[56.1211953,31.5906712],[56.0350885,31.5704307],[56.0128652,31.4648737],[55.9639761,31.4307118],[55.9267531,31.3673728],[55.7400756,31.2745965],[55.6009042,31.2437592],[55.399507,31.1518177],[54.9433733,31.0276458],[54.7116966,31.0659818],[54.5333558,31.0143137],[54.5594686,30.5826324],[54.4800197,30.4784661],[54.3366818,30.3851253],[54.547802,30.1781876],[54.541132,29.8917816],[54.5619684,29.6270564],[54.4816862,29.6409672]]}],{"type":"Feature","properties":{"province.name":"\u0000\u0000 \u0000\u0000\u0000","name.fa":"\u0000\u0000\u0000","geometry":{"type":"Polygon","coordinates":[[[59.2187761,25.1994033],[59.0569858,25.2037298],[58.031375,25.3618275],[57.6780601,25.4491451],[57.1858621,25.6190402],[57.1125809,25.685279],[57.0835873,25.7454101],[57.0712282,25.8426445],[57.0020189,25.9007986],[56.9624421,25.9743274],[56.9514165,26.1048542],[56.8713617,26.2448854],[56.8160077,26.4318077],[56.8296427,26.5188522],[56.8591859,26.5687237],[56.8452011,26.6470807],[56.8084558,26.7407944],[56.6707157,26.9030693],[56.5868216,26.7419718],[56.5462498,26.7026737],[56.2417336,26.5902874],[56.1322549,26.5037413],[55.8933406,26.4122357],[55.4229986,26.0700522],[55.2766237,25.8867867],[55.2661847,25.7944904],[55.2077397,25.7132112],[55.1285553,25.6699815],[55.0008802,25.6543418],[54.8669759,25.7069237],[54.6504956,25.7147905],[54.5300633,25.6925179],[54.3932537,25.7216287],[54.2936202,25.8178637],[54.2696951,25.9814492],[54.2235447,26.0576023],[53.9642391,26.2946269],[53.849842,26.3222684],[53.1282214,26.6061774],[52.482398,26.989407],[52.7379214,27.2743065],[52.8165666,27.3237282],[52.8852276,27.2928411],[52.9457867,27.3292368],[53.0496784,27.2750621],[53.1307937,27.2720136],[53.3535802,27.1911741],[53.4096961,27.1139499],[53.496088,27.1011746],[53.5419239,27.1578432],[53.6322058,27.0983966],[53.7933232,27.045617],[53.9333276,27.1236778],[54.0349994,27.230064],[54.015833,27.274235],[53.9963867,27.2731199],[53.9708304,27.3070144],[53.9686071,27.3500664],[54.1094482,27.408124],[54.3530677,27.4586864],[54.6289134,27.4678509],[55.1256022,27.430071],[55.2567201,27.4956291],[55.437004,27.642573],[55.4714501,27.7186889],[55.5672887,27.7814665],[55.5800652,27.8086906],[55.5006199,27.9234117],[55.386725,27.998693],[55.4078383,28.4184168],[55.4511744,28.5523166],[55.3508892,28.6603707],[55.2539408,28.7217641],[55.247831,28.7470381],[55.5189535,28.6937067],[55.8156322,28.8195388],[56.0511988,28.880375],[56.0603652,28.7298187],[56.0364752,28.3762016],[56.0557361,28.2946235],[56.1042574,28.2348024],[56.255095,28.1611992],[56.4445487,28.1306398],[56.6520588,28.1431368],[56.7490074,28.2653651],[56.992627,28.241195],[57.0081829,27.9739722],[57.2040265,27.8947957],[57.416533,27.890628],[57.4251464,27.8100748],[57.4751451,27.7925702],[57.3256941,27.7064581],[57.2230512,27.5768406],[57.2290414,27.5600461],[57.2608172,27.5655853],[57.3015782,27.5980461],[57.3587125,27.574488],[57.3090081,27.4685003],[57.3251405,27.437571],[57.316254,27.3606232],[57.3868129,27.2661754],[57.5440368,27.1845091],[57.5982091,27.1011722],[57.748769,27.0001759],[57.7984922,26.9431154],[57.9918356,26.9156125],[58.1918457,26.7272228],[58.4354681,26.7322748],[58.6451978,26.586596],[58.8438142,26.4831065],[58.8843736,26.4236558],[58.8796535,26.353383],[58.9432658,26.3355961],[58.9760453,26.28671],[58.9829846,26.2172627],[59.0288239,26.166987],[59.0218772,26.1308698],[59.0827163,26.0303127],[59.0804892,25.996152],[59.0190967,25.9733706],[59.9977108,25.937814],[59.072156,25.8944817],[59.0835491,25.8508662],[59.218833,25.7405878],[59.2113266,25.6914167],[59.2357725,25.6367001],[59.2104926,25.5578014],[59.2187761,25.1994033]]]},"type":"Feature","properties":{"province.name":"\u0000\u0000 \u0000\u0000\u0000","name.fa":"\u0000\u0000\u0000"},"geometry":{"type":"Polygon","coordinates":[[[49.5801433,29.6674426],[49.4643006,29.7895783],[48.8544653,29.7477043],[48.826035,29.8117841],[48.7164401,29.8544406],[48.6150246,29.9323151],[48.4629249,29.9910509],[48.438333,30.0725],[48.3827499,30.1231946],[48.4159566,30.1705652],[48.408056,30.211389],[48.2727326,30.3360874],[48.2027331,30.3205769],[48.182222,30.399444],[48.1511605,30.4351201],[48.0244151,30.4773465],[48.0321895,30.9953741],[47.6873876,31.0029569],[47.682971,31.394609],[47.865855,31.783613],[47.852568,31.818223],[47.805337,31.842578],[47.80986,31.881373],[47.731711,31.939258],[47.705397,31.994321],[47.712418,32.018895],[47.690236,32.013243],[47.664111,32.048542],[47.8106025,32.116073],[47.9850235,32.2638935],[47.9619751,32.2943871],[47.9717102,32.3274573],[48.0356255,32.3553508],[48.0160638,32.5233567],[47.9805985,32.5363511],[47.9425773,32.5170105],[47.9558342,32.5357321],[47.9328002,32.5349499],[47.944046,32.561572],[47.9006195,32.5683061],[47.9328576,32.5824091],[47.9152184,32.6027823],[47.9883271,32.6114121],[47.9546661,32.6353339],[47.9691466,32.6582799],[48.0881156,32.6893842],[48.0639953,32.7087511],[48.0860481,32.7196815],[48.0599975,32.7403727],[48.0682412,32.7888861],[48.0490266,32.8057852],[48.1288223,32.8704693],[48.1186829,32.8990325],[48.1327477,32.9171759],[48.1796875,32.9009407],[48.2745972,32.9963338],[48.389679,32.9654538],[48.5045663,32.9616899],[48.5471992,32.9836302],[48.632473,32.9747246],[48.7093164,32.936459],[48.718581,32.8960203],[48.7686882,32.922986],[48.7648849,32.9683579],[48.8307382,32.9485216],[48.8538629,32.9108387],[48.8998756,32.8849407],[49.0061035,32.915383],[49.0138666,32.8994067],[49.0553438,32.9051092],[49.1370774,32.8698313],[49.1719742,32.8839216],[49.2179907,32.8521658],[49.3692588,32.8435754],[49.4283178,32.7793093],[49.5043142,32.7382883],[49.5157779,32.7248254],[49.5000772,32.7030932],[49.5868493,32.6092272],[49.9078903,32.320275],[49.9368208,32.1860741],[50.1182176,32.0164292],[50.2396272,31.9401716],[50.2903518,31.8704122],[50.2831,31.8500772],[50.4470368,31.6452259],[50.4575423,31.6141558],[50.4269791,31.5934585],[50.3923798,31.6013915],[50.3584136,31.5550185],[50.3751105,31.5412341],[50.2540909,31.4483382],[50.2479222,31.4182174],[50.2998317,31.3584786],[50.2467612,31.306489],[50.1527063,31.2518317],[50.099861,31.2575942],[49.95467,31.2114047],[49.9899942,31.1256725],[49.9475175,31.0748626],[49.97143,31.0129259],[49.8862115,30.9642596],[49.9236754,30.9277065],[49.9511414,30.9293713],[49.9555053,30.877364],[50.0333405,30.8713077],[50.0697479,30.8299317],[50.2104225,30.9322635],[50.2434539,30.8325063],[50.2315138,30.7905283],[50.2541769,30.7555763],[50.3354417,30.7087156],[50.392841,30.7313077],[50.4237277,30.6651337],[50.5339013,30.5996434],[50.5495662,30.5630552],[50.5060119,30.5305816],[50.3864555,30.524084],[50.4526215,30.4322936],[50.5207594,30.3904245],[50.520584,30.1655169],[50.4165763,30.1821897],[50.2000071,30.2949624],[50.1390342,30.1759444],[49.5801433,29.6674426]]]},"type":"Feature","properties":{"province.name":"\u0000\u0000 \u0000\u0000\u0000","name.fa":"\u0000\u0000\u0000"},"geometry":{"type":"Polygon","coordinates":[[[58.8438142,26.4831065],[58.6451978,26.586596],[58.4354681,26.7322748],[58.1918457,26.7792228],[57.9918356,26.9156125],[57.7984922,26.9431154],[57.748769,27.0001759],[57.5982091,27.1011722],[57.5440368,27.1845091],[57.3868129,27.2661754],[57.316254,27.3606232],[57.3251405,27.437571],[57.3090081,27.4685003],[57.3587125,27.574488],[57.3015782,27.5980461],[57.2608172,27.5655853],[57.2290414,27.5600461],[57.2230512,27.5768406],[57.3256941,27.7064581],[57.4751451,27.7925702],[57.4251464,27.8100748],[57.416533,27.890628],[57.2040265,27.8947957],[57.0081829,27.9739722],[56.992627,28.241195],[56.7490074,28.2653651],[56.6520588,28.1431368],[56.4445487,28.1306398],[56.255095,28.1611992],[56.1042574,28.2348024],[56.0557361,28.2946235],[56.0364752,28.3762016],[56.0548061,28.8793005],[55.8156322,28.8195388],[55.5189535,28.6937067],[55.2547831,28.7470381],[55.2772774,29.033435],[55.2622777,29.0945466],[55.1653258,29.2523276],[54.983932,29.3948318],[54.8161445,29.5937251],[54.5614116,29.6264973],[54.541132,29.8917816],[54.5450221,30.1892939],[54.3366818,30.3809655],[54.4800197,30.4784661],[54.5594686,30.5826324],[54.5333558,31.0143137],[54.7116966,31.0659818],[54.9464299,31.0281963],[55.2106096,31.0912565],[55.494789,31.185976],[55.6009042,31.2437592],[55.7400756,31.2745965],[55.9384197,31.3768158],[55.9639761,31.4307118],[56.0128652,31.4648737],[56.0350885,31.5704307],[56.1211953,31.5906712],[56.1605022,31.5526083],[56.2556554,31.5534935],[56.538725,31.7076627],[56.5790044,31.7112741],[56.6048378,31.922106],[56.8851306,31.960163],[57.3393144,31.848772],[57.5334915,31.8234911],[58.0671235,31.6640438],[58.3899192,31.4765441],[58.780493,31.1554151],[59.0680015,31.000833],[59.0785612,30.9079166],[59.1938393,30.8168007],[59.4682103,30.4995767],[59.4249612,30.2526259],[59.4494066,29.9934475],[59.4335736,29.8717785],[59.4635717,29.3742691],[59.4944042,29.2445422],[59.5707959,29.0820391],[59.5055168,28.9998199],[59.4969032,28.96093],[59.5505155,28.8159292],[59.5627415,28.6903676],[59.546075,28.6236971],[59.5660743,28.5184221

], [59.536348, 28.4731397], [59.2305063, 28.4764732], [59.0619893, 28.4162781], [59.1225478, 28.3632191], [59.1261603, 28.2984927], [59.1614392, 28.2507186], [59.1483843, 28.1273763], [59.0145546, 28.1070656], [58.9338231, 28.0298011], [58.9324362, 27.9731322], [58.8904901, 27.8981235], [58.9010426, 27.5764568], [58.8310364, 27.2536735], [58.8485422, 27.1017277], [58.9129876, 26.9999296], [58.8643748, 26.9086636], [58.8507615, 26.8311605], [58.9057608, 26.7878299], [58.9904926, 26.75922], [59.0177119, 26.6561592], [58.9996588, 26.596439], [59.0374382, 26.5475481], [59.0060451, 26.5281025], [59.0003551, 26.5003229], [58.9224331, 26.5150435], [58.8438142, 26.4831065]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000 \u0000\u0000","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[47.664111, 32.048542], [47.62147, 32.103803], [47.565708, 32.112076], [47.529201, 32.141752], [47.527892, 32.170546], [47.565278, 32.193232], [47.56696, 32.225089], [47.499849, 32.277875], [47.519494, 32.304516], [47.46314, 32.314316], [47.435256, 32.351867], [47.465348, 32.391316], [47.379567, 32.473212], [47.283461, 32.49637], [47.247204, 32.463729], [47.168401, 32.457281], [46.887238, 32.631954], [46.720615, 32.78596], [46.622615, 32.822218], [46.447421, 32.933275], [46.266369, 32.972447], [46.159405, 32.955131], [46.096511, 32.98168], [46.099162, 33.019813], [46.129911, 33.020975], [46.164625, 33.069428], [46.04037, 33.094539], [46.0531735, 33.1282554], [46.125454, 33.11708], [46.212346, 33.197072], [46.180584, 33.206344], [46.189827, 33.26397], [46.134, 33.292185], [46.080962, 33.351755], [46.077601, 33.389648], [46.007252, 33.50781], [45.939491, 33.481894], [45.863003, 33.493148], [45.961002, 33.56133], [45.905254, 33.606284], [45.907062, 33.638097], [45.814716, 33.628415], [45.753165, 33.592917], [45.77167, 33.636936], [45.69252, 33.69097], [45.7701754, 33.7040816], [45.8510102, 33.7949231], [45.8596235, 33.8432538], [45.9215692, 33.8379732], [45.9693482, 33.8588075], [45.8537902, 33.9265889], [45.8446235, 33.9490921], [45.8673995, 33.9890883], [45.9193491, 33.9688151], [45.9832378, 33.9724168], [46.0549102, 34.0377041], [46.2782541, 33.9507633], [46.2985308, 33.9954801], [46.62077, 33.8313134], [46.724106, 33.7421437], [46.7627183, 33.7713108], [46.8630042, 33.7332522], [46.9202271, 33.7685308], [46.9746759, 33.7418645], [46.9971753, 33.8115904], [47.0521778, 33.8332553], [47.0927342, 33.8243639], [47.1063476, 33.8404771], [47.1374601, 33.8146418], [47.2952454, 33.788536], [47.3866369, 33.8613118], [47.3180243, 33.9604875], [47.3913602, 33.9304835], [47.4213599, 33.9524271], [47.4855323, 33.9390951], [47.5069223, 33.90715], [47.5138657, 33.8282556], [47.4574732, 33.8052004], [47.4113605, 33.7371443], [47.3038552, 33.7238107], [47.2580222, 33.7460268], [47.2419094, 33.7257514], [47.1788504, 33.7227], [47.1557938, 33.7029751], [47.0680114, 33.709917], [47.0602357, 33.6646442], [47.0224562, 33.6282538], [47.038013, 33.5846409], [46.9921768, 33.5863019], [46.9424506, 33.6204718], [46.8846714, 33.6201917], [46.8457816, 33.6104778], [46.8643915, 33.5776981], [46.8393918, 33.5615833], [46.8638386, 33.5199195], [47.0235664, 33.4240774], [47.0102334, 33.3840821], [47.100236, 33.3493591], [47.1063496, 33.3324128], [47.151352, 33.3385251], [47.1799621, 33.3018637], [47.2019079, 33.3024113], [47.1955179, 33.2826838], [47.2252443, 33.27324], [47.2324649, 33.2479689], [47.3996957, 33.1976884], [47.5922039, 33.0451818], [47.8097147, 32.9718482], [47.9344431, 32.8999041], [47.9952793, 32.8418461], [48.0100024, 32.7793454], [47.9394436, 32.7623972], [47.9102776, 32.7212864], [47.9693907, 32.6587288], [47.9546661, 32.6353339], [47.9892081, 32.6128352], [47.9152184, 32.6027823], [47.9328576, 32.5824091], [47.9005089, 32.5676159], [47.944046, 32.561572], [47.9328002, 32.5349491], [47.9558342, 32.5357321], [47.9425773, 32.5170105], [47.9805985, 32.5363511], [48.0156975, 32.5242384], [48.0354539, 32.3539695], [47.9760476, 32.3335637], [47.9617768, 32.2954034], [47.9850235, 32.2638935], [47.8388139, 32.1374699], [47.664111, 32.048542]]]]],{"type":"Feature","properties":{"province\_name":"","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[49.7317497, 32.8082425], [49.6600773, 32.7793454], [49.5906318, 32.789073], [49.5775754, 32.7401792], [49.5155333, 32.7251812], [49.4283178, 32.7793093], [49.3671645, 32.8445515], [49.2179907, 32.8521658], [49.1719742, 32.8839216], [49.1370774, 32.8698313], [49.0553438, 32.9051092], [49.0138666, 32.8994067], [49.0061035, 32.915383], [48.8998756, 32.8849407], [48.8538629, 32.9108387], [48.8307382, 32.9485216], [48.7648849, 32.9683579], [48.7686882, 32.922986], [48.718581, 32.8960203], [48.7093164, 32.936459], [48.632473, 32.9747246], [48.5471992, 32.9836302], [48.5045663, 32.9616899], [48.389679, 32.9654538], [48.2796974, 32.9971281], [48.1796875, 32.9009407], [48.1280898, 32.9126331], [48.1288223, 32.8704693], [48.0488091, 32.8054172], [48.0682412, 32.7888861], [48.0599975, 32.7403727], [48.0860481, 32.7196815], [48.0639953, 32.7087511], [48.0897255, 32.6917658], [48.064911, 32.6745416], [48.0020447, 32.6745484], [47.9821206, 32.6534494], [47.9102776, 32.7212864], [47.9394436, 32.7623972], [47.996946, 32.7676794], [48.0125025, 32.7904605], [47.9941696, 32.8435165], [47.9344431, 32.8999041], [47.7533252, 32.9982401], [47.6905427, 33.0024024], [47.5922039, 33.0451818], [47.3996957, 33.1976884], [47.2324649, 33.2479689], [47.2254443, 33.27324], [47.1955179, 33.2826838], [47.2019079, 33.3024113], [47.1799621, 33.3018604], [47.151352, 33.3385251], [47.1063496, 33.3324128], [47.100236, 33.3493591], [47.0102334, 33.3840821], [47.0260664, 33.4201862], [47.0113434, 33.435471], [46.8616151, 33.5221313], [46.8396684, 33.5599222], [46.8643915, 33.5776981], [46.8457816, 33.6104778], [46.9410639, 33.6207516], [46.9921768, 33.5863019], [47.0374558, 33.584361], [47.0224562, 33.6282538], [47.0602357, 33.6646442], [47.0680114, 33.709917], [47.1557938, 33.7029751], [47.1788504, 33.7227], [47.2419094, 33.7257514], [47.2580222, 33.7460268], [47.3038552, 33.7238107], [47.4080276, 33.7352035], [47.4574732, 33.8052004], [47.5110891, 33.8249246], [47.5069223, 33.90715], [47.4799755, 33.942146], [47.4985851, 33.9757608], [47.4452492, 34.0310376], [47.4266392, 34.0843725], [47.4869185, 34.1132645], [47.488308, 34.1829876], [47.5649772, 34.1821484], [47.6944284, 34.2760476], [47.6791485, 34.3560528], [47.7002613, 34.3782714], [47.8280466, 34.3238259], [47.8483233, 34.2785479], [47.9002729, 34.2385487], [47.9386057, 34.2457689], [48.053891, 34.2127187], [48.1772298, 34.086044], [48.2405656, 34.0491006], [48.3044582, 34.0566015], [48.304735, 34.0213183], [48.3523379, 34.0068765], [48.4830761, 34.0102073], [48.5194655, 34.0507621], [48.5269684, 34.1071563], [48.5603017, 34.0665938], [48.5858581, 34.0635431], [48.6172508, 34.0960456], [48.6576542, 34.0760602], [48.7094765, 34.0918759], [48.7236428, 34.1232677], [48.765589, 34.072434], [48.8417048, 34.0363209], [48.8517047, 34.0027073], [48.9042073, 34.0063179], [48.9161503, 34.0285404], [48.948377, 34.0027074], [49.0111558, 34.0221501], [49.0775724, 34.01471], [49.0417125, 33.9763146], [49.0089329, 33.9818748], [48.968653, 33.9596517], [48.9561538, 33.9224262], [48.9642103, 33.9049325], [49.0122663, 33.9060429], [48.9936567, 33.8904809], [49.0061566, 33.8671466], [48.9758771, 33.8396507], [48.9939334, 33.8299286], [48.9822672, 33.7982618], [49.0117136, 33.7677095], [49.0211569, 33.696317], [49.0703263, 33.6888153], [49.075606, 33.7179829], [49.1417187, 33.7215939], [49.1950546, 33.6863149], [49.2156111, 33.6129748], [49.2842269, 33.6040916], [49.29645, 33.6502049], [49.3514492, 33.6529765], [49.3820087, 33.6946475], [49.3606191, 33.7382587], [49.410065, 33.7504818], [49.4311784, 33.736318], [49.4358981, 33.6996489], [49.5283801, 33.6879626], [49.6189621, 33.7424207], [49.633129, 33.6968687], [49.5972963, 33.6729819], [49.6034062, 33.6410334], [49.6361858, 33.625199], [49.6514656, 33.5863075], [49.7123016, 33.5685319], [49.7709142, 33.5176957], [49.7964708, 33.4465905], [49.9345325, 33.3849186], [50.018148, 33.3685323], [49.9987051, 33.3035274], [49.9375892, 33.2471437], [49.9470359, 33.2299169], [49.8895336, 33.2196333], [49.8714771, 33.191301], [49.8906435, 33.1446311], [49.8778636, 33.1296341], [49.9242565, 33.0896361], [49.9148102, 33.0657454], [49.8636975, 33.0690771], [49.837308, 33.0499084], [49.722026, 33.0843538], [49.6689667, 33.0501881], [49.6550771, 33.0074084], [49.7078563, 32.982407], [49.6373006, 32.9746328], [49.6823002, 32.8990767], [49.7486925, 32.8557365], [49.7317497, 32.8082425]]]]],{"type":"Feature","properties":{"province\_name":"","name\_fa":"","geometry":{"type":"Polygon","coordinates":[[[49.9478688, 33.3841916], [49.8067507, 33.4379776], [49.7623043, 33.5263085], [49.7123016, 33.5685319], [49.6514656, 33.5863075], [49.6361858, 33.625199], [49.5956295, 33.656597], [49.633129, 33.6968687], [49.6189621, 33.7424207], [49.5283801, 33.6879626], [49.4358981, 33.6996489], [49.4311784, 33.736318], [49.410065, 33.7504818], [49.3606191, 33.7382587], [49.3820087, 33.6946475], [49.3514492, 33.6529765], [49.29645, 33.6502049], [49.2842269, 33.6040916], [49.2156111, 33.6129748], [49.1950546, 33.6863149], [49.1417187, 33.7215939], [49.075606, 33.7179829], [49.0703263, 33.6888153], [49.017136, 33.767705], [48.9822672, 33.7982618], [48.9939334, 33.8299286], [48.9758771, 33.8396507], [49.0061566, 33.8671466], [48.9936567, 33.8904809], [49.0122663, 33.9060429], [48.9594871, 33.9079835], [48.9619867, 33.9499301], [49.0089329, 33.9818748], [49.0417125, 33.9763146], [49.1753304, 34.115983], [49.1383843, 34.1193784], [49.093938, 34.1627197], [49.1233842, 34.1910418], [49.1217173, 34.2488219], [49.0208784, 34.3096594], [49.007265, 34.3843837], [48.9500427, 34.388553], [49.003655, 34.4238294], [49.0036549, 34.4657827], [49.0547675, 34.5052174], [49.0011545, 34.5974473], [49.060047, 34.6349478], [49.0811569, 34.5846704], [49.1347728, 34.6135541], [49.1031031, 34.7535606], [49.1492156, 34.7971742], [49.1681052, 34.8660652], [49.2117179, 34.8421758], [49.3036666, 34.9119035], [49.3897823, 34.9174518], [49.4142284, 35.0016269], [49.2889433, 35.0060648], [49.259773, 35.0421831], [49.3064462, 35.0849574], [49.4111719, 35.0341191], [49.4453414, 35.0871766], [49.4661739, 35.1791241], [49.426171, 35.2221821], [49.3703351, 35.234962], [49.3747816, 35.2632994], [49.3506119, 35.2813548], [49.4070043, 35.3638547], [49.395338, 35.3863542], [49.4308941, 35.4188541], [49.5508994, 35.4035785], [49.6228485, 35.4488555], [49.6822942, 35.4471963], [49.6936838, 35.4810829], [49.6697942, 35.5183046], [49.6875739, 35.5341379], [49.728561, 35.5133096], [49.8803583, 35.5469146], [49.9723072, 35.5060874], [50.0431466, 35.5055284], [50.1942646, 35.5244097], [50.353419, 35.5706023], [50.6259494, 35.5474747], [50.7676246, 35.4869185], [50.8956036, 35.4850921], [50.9920724, 35.4371339], [50.9615226, 35.417188], [50.890687, 35.2274604], [50.857074, 35.18

16245],[50.654287,35.0193994],[50.6498404,34.9996803],[50.6923434,34.9518966],[50.705123,34.8682868],[50.4892791,34.8602302],[50.451223,34.8269037],[50.4340001,34.7574514],[50.4092738,34.7519027],[50.3412177,34.7599505],[50.2514887,34.8146787],[50.1673198,34.8213458],[50.1220405,34.7555112],[50.1081511,34.6238313],[50.0862046,34.5835621],[50.1570406,34.5091175],[50.1481542,34.4732836],[50.3351052,34.4421688],[50.3317721,34.4068837],[50.4245543,34.2774425],[50.4420545,34.2010531],[50.5170602,34.1899339],[50.6456749,34.2560534],[50.727624,34.2152142],[50.7962399,34.2360537],[50.815125,34.2047774],[50.8848555,34.1913332],[50.8973554,34.1571626],[51.0526483,34.0387274],[51.0200362,33.9818351],[51.0487506,33.9366011],[51.0320842,33.8529864],[51.049584,33.8202096],[51.0093045,33.7729798],[50.8926328,33.7071527],[50.7040117,33.7271481],[50.5956733,33.6621504],[50.5742836,33.6307614],[50.4537253,33.6346521],[50.3370535,33.615757],[50.2731613,33.5824265],[50.2328815,33.6027028],[50.1917691,33.5310397],[49.9837016,33.4924176],[49.9720351,33.4676986],[49.9903684,33.4199214],[49.959812,33.4157504],[49.9478688,33.3874196]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000 \u0000\u0000\u0000\u0000 \u0000 \u0000\u0000 \u0000\u0000","name\_fa":"\u0000\u0000\u0000\u0000 \u0000 \u0000\u0000\u0000\u0000"},"geometry":{"type":"Polygon","coordinates":[[[51.06952,31.1878167],[51.0309765,31.1818113],[50.9694423,31.2265568],[50.963202,31.2604226],[50.9245891,31.2984821],[50.8923662,31.3087554],[50.8570867,31.2904206],[50.7892982,31.3143162],[50.7198524,31.3832062],[50.6395699,31.42765],[50.5748441,31.4087636],[50.5003954,31.4526482],[50.476229,31.4276498],[50.4009467,31.4151461],[50.3306676,31.4448749],[50.2932415,31.4815706],[50.3756484,31.5418299],[50.3584136,31.5550185],[50.3923798,31.6013915],[50.4269791,31.5934585],[50.4575423,31.6141558],[50.4464036,31.646511],[50.2831,31.8500772],[50.2903518,31.8704122],[50.2396202,31.9401711],[50.1182176,32.0164292],[49.9368208,32.1860741],[49.9078903,32.320275],[49.5868493,32.6092272],[49.5015438,32.7165475],[49.5775754,32.7401792],[49.5906318,32.789073],[49.6660073,32.7793485],[49.7298032,32.8090731],[49.767306,32.7640711],[49.9823168,32.6346172],[50.0445391,32.6274027],[50.1537111,32.6637915],[50.2559398,32.5643506],[50.3220524,32.5368379],[50.4912268,32.5912856],[50.5176162,32.631566],[50.4581705,32.6607308],[50.5270627,32.7179593],[50.5498848,32.7138973],[50.548673,32.6799061],[50.5622097,32.6883087],[50.565016,32.6723313],[50.6518463,32.7105145],[50.8523551,32.7121267],[50.887358,32.6962892],[50.9170809,32.664903],[50.9318041,32.5985097],[50.99653,32.5546268],[51.0079166,32.5240621],[50.9684705,32.4335058],[50.973194,32.3882322],[51.1443119,32.2876719],[51.2337608,32.1743356],[51.3457126,32.1396181],[51.3857121,32.0915555],[51.3768256,32.0557275],[51.4143253,32.0323871],[51.4343282,31.9721088],[51.4132154,31.9523803],[51.4282151,31.9268282],[51.4096022,31.858497],[51.2968203,31.8229413],[51.2854305,31.8021027],[51.3690463,31.7126555],[51.3054338,31.6618246],[51.2965441,31.5157101],[51.2559598,31.4607111],[51.2884877,31.4451559],[51.3140441,31.3934804],[51.2932112,31.365702],[51.3371005,31.2965324],[51.3298774,31.1959862],[51.2237637,31.2237608],[51.1804259,31.2098696],[51.179041,31.1748696],[51.1529267,31.1498667],[51.06952,31.1878167]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000 \u0000\u0000\u0000\u0000\u0000","name\_fa":"\u0000\u0000\u0000\u0000\u0000"},"geometry":{"type":"Polygon","coordinates":[[[45.619252,33.69097],[45.642104,33.733966],[45.643764,33.772],[45.603272,33.803023],[45.511585,33.945574],[45.431545,33.945556],[45.407661,33.977255],[45.466848,34.009824],[45.473516,34.075608],[45.581594,34.14613],[45.554874,34.195116],[45.581097,34.23956],[45.583566,34.302663],[45.538075,34.352107],[45.496178,34.336867],[45.439814,34.458932],[45.521255,34.480982],[45.51199,34.551635],[45.5314626,34.6018982],[45.594687,34.5498888],[45.6834717,34.5595769],[45.7432041,34.5380347],[45.702625,34.693772],[45.653259,34.72311],[45.69876,34.77002],[45.689538,34.815161],[45.73689,34.846417],[45.753623,34.828685],[45.792074,34.853026],[45.788431,34.912296],[45.86504,34.896328],[45.897312,34.959992],[45.876525,35.031513],[45.930795,35.045347],[45.921987,35.095489],[45.998198,35.061935],[46.015202,35.071155],[46.059426,35.037953],[46.08028,35.05592],[46.071919,35.083131],[46.116562,35.074065],[46.191291,35.110888],[46.155059,35.128999],[46.154758,35.159001],[46.201247,35.201229],[46.177792,35.228359],[46.108403,35.241129],[46.14894,35.279894],[46.3471386,35.1705126],[46.4465912,35.0538401],[46.5010369,35.0349514],[46.5562231,34.9219012],[46.6399327,34.8693974],[46.7435483,34.8399508],[46.735492,34.7874429],[46.7829948,34.7393902],[46.894943,34.7643916],[47.0132854,34.7460585],[47.0885643,34.7577246],[47.0671746,34.7924511],[47.1252335,34.8035574],[47.0721745,34.8510576],[47.0810636,34.8907797],[47.1732896,34.9032836],[47.159679,34.9610643],[47.1732895,34.9724488],[47.2141256,34.9371676],[47.2432948,34.9532884],[47.2819082,34.9277316],[47.3266309,34.9385662],[47.3082975,35.0063403],[47.3549665,35.0180041],[47.3608,35.052733],[47.399689,35.0616181],[47.728592,35.0621781],[47.8030414,35.0149562],[47.8052646,34.980515],[47.8399878,34.9388838],[47.9272133,34.9299524],[47.9177704,34.8955089],[47.863601,34.8580062],[47.7971938,34.8511636],[47.8652683,34.7224487],[47.8555451,34.6535598],[47.8730448,34.6293872],[47.9547173,34.6227191],[47.9994402,34.5866082],[48.0344432,34.5885573],[48.0497196,34.620499],[48.1013849,34.5296699],[48.090556,34.4935566],[48.1050029,34.4674413],[48.0858332,34.4182685],[48.0444438,34.3896611],[47.989713,34.4013069],[47.9936287,34.364661],[47.9524951,34.3454946],[47.8866593,34.2552092],[47.8483233,34.2785479],[47.8280466,34.3238259],[47.6952616,34.3754918],[47.6788719,34.3510532],[47.6944284,34.2760476],[47.5649772,34.1821484],[47.488308,34.1829876],[47.4869185,34.1132645],[47.4319159,34.0963137],[47.423583,34.0646507],[47.4980321,33.9771508],[47.4927519,33.9485369],[47.4213599,33.9524271],[47.3913602,33.9304835],[47.3149679,33.9571477],[47.3866369,33.8613118],[47.343301,33.8160326],[47.3116348,33.8163123],[47.304412,33.791867],[47.1374601,33.8146418],[47.1063476,33.8404771],[47.0927342,33.8243639],[47.0521778,33.8332553],[46.9971753,33.8115904],[46.9746759,33.7418645],[46.9202271,33.7685308],[46.8596674,33.733532],[46.7627183,33.7713108],[46.7252193,33.7418638],[46.62077,33.8313134],[46.3557568,33.9704863],[46.2985308,33.9954801],[46.2782541,33.9507633],[46.0549102,34.0377041],[45.9832378,33.9724168],[45.9193491,33.9688151],[45.8673995,33.9890883],[45.8446235,33.9490921],[45.8537902,33.9265889],[45.9693482,33.8588075],[45.9215692,33.8379732],[45.8596235,33.8432538],[45.8510102,33.7949231],[45.7701754,33.7050816],[45.69252,33.69097]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000 \u0000\u0000\u0000","name\_fa":"\u0000\u0000\u0000"},"geometry":{"type":"Polygon","coordinates":[[[48.4722371,34.0063171],[48.3791841,34.0013163],[48.304735,34.0213183],[48.3044582,34.0566015],[48.2405656,34.0491006],[48.1869531,34.0796536],[48.053891,34.2127187],[47.9386057,34.2457689],[47.9002729,34.2385487],[47.8866593,34.2552092],[47.9524951,34.3454946],[47.9936287,34.364661],[47.989713,34.4013069],[48.0444438,34.3896611],[48.0858332,34.4182685],[48.1050029,34.4674413],[48.090556,34.4935566],[48.1013894,34.5296699],[48.0466665,34.6229991],[48.0344432,34.5885573],[47.9994402,34.5866082],[47.9547173,34.6227191],[47.865545,34.6332761],[47.8652683,34.7224487],[47.7971938,34.8511636],[47.8586012,34.8557778],[47.9086037,34.8877324],[47.9227703,34.9518918],[47.9527696,34.9705112],[47.9999871,34.9612717],[48.040832,34.9860633],[48.074448,34.9616167],[48.1438907,34.9707822],[48.1758366,34.9988461],[48.1922262,35.0927395],[48.1525034,35.1180134],[48.1813929,35.1610646],[48.1506136,35.2098726],[48.0958339,35.2282944],[48.0897204,35.2694025],[48.0266608,35.3707958],[47.9447149,35.389679],[47.8660987,35.5582986],[47.8924882,35.5855279],[47.9741575,35.5741315],[48.0491627,35.6391369],[48.2111141,35.6874692],[48.2516703,35.7202482],[48.2961198,35.6805276],[48.3916789,35.6430234],[48.4836278,35.6452504],[48.4961278,35.598864],[48.5427971,35.5849702],[48.6639126,35.5988642],[48.7164155,35.5521872],[48.7530849,35.5702471],[48.7766977,35.6141376],[48.770031,35.6785813],[48.7328049,35.6916443],[48.7289148,35.7119193],[48.7797506,35.7335824],[48.8839629,35.6994157],[49.1272706,35.5688593],[49.1342137,35.5224694],[49.1608804,35.4933001],[49.428394,35.4266363],[49.3506119,35.2813548],[49.3747816,35.2632994],[49.3703351,35.234962],[49.426171,35.2221821],[49.4661739,35.1791221],[49.4628408,35.1382925],[49.4111719,35.0341191],[49.3239462,35.0832977],[49.2608868,35.0516185],[49.2828334,35.0077337],[49.4142284,35.0016269],[49.3897823,34.9174518],[49.3036666,34.9119035],[49.2117179,34.8421758],[49.1681052,34.8660652],[49.1492156,34.7971742],[49.1031031,34.7535606],[49.1347728,34.6135541],[49.0811569,34.5846704],[49.060047,34.6349478],[49.0011545,34.5974473],[49.0547675,34.5052174],[49.0036549,34.4657827],[49.003655,34.4238294],[48.9500427,34.388553],[49.007265,34.3843837],[49.0208784,34.3096594],[49.1217173,34.2488219],[49.1233842,34.1910418],[49.093938,34.1627197],[49.1333634,34.1193784],[49.1736636,34.1163187],[49.1672741,34.0935465],[49.0811585,34.0152091],[49.0111558,34.0221501],[49.948377,34.0027074],[48.9161503,34.0285404],[48.9042073,34.0063179],[48.8611478,34.0010465],[48.8417048,34.0363209],[48.765589,34.072434],[48.7308659,34.1210474],[48.7133664,34.1221575],[48.7094765,34.0918759],[48.6576542,34.0760602],[48.6172508,34.0960456],[48.5858581,34.0635431],[48.5603017,34.0665938],[48.5269684,34.1071563],[48.5194655,34.0507621],[48.4722371,34.0063171]]]],{"type":"Feature","properties":{"province\_name":"\u0000\u0000\u0000 \u0000\u0000\u0000","name\_fa":"\u0000\u0000\u0000"},"geometry":{"type":"Polygon","coordinates":[[[49.5427042,35.4027091],[49.391171,35.4252475],[49.1608804,35.4933001],[49.1342137,35.5224694],[49.1339371,35.5627462],[49.0475413,35.6171944],[48.8839229,35.6994157],[48.7797506,35.7335824],[48.7363237,35.7329575],[48.7247478,35.7494213],[48.7844737,35.8841459],[48.9033685,35.9824777],[48.9342018,35.9305373],[49.053205,35.8783147],[49.0758768,35.8899781],[49.0717101,35.9238678],[49.1517126,35.9299788],[49.2089383,35.9822079],[49.3019973,35.9758182],[49.3619997,36

.0108135],[49.4128357,36.07387],[49.4108922,36.1366516],[49.4336719,36.1930405],[49.3911688,36.2680397],[49.4245017,36.3272128],[49.3708889,36.3385999],[49.3369993,36.3894352],[49.2775532,36.3883271],[49.1911575,36.4286022],[49.109765,36.4336014],[49.0605954,36.4958249],[48.9719792,36.5533242],[49.1083657,36.7780687],[49.1891487,36.7607539],[49.2806052,36.8155578],[49.484227,36.6080555],[49.615622,36.606939],[49.7876613,36.5593829],[49.9008466,36.5825708],[49.9359834,36.5724961],[49.938971,36.6026379],[50.0581539,36.6040616],[50.1714824,36.6688887],[50.2320418,36.6786142],[50.4462162,36.6105516],[50.5920582,36.5268717],[50.6798406,36.4999976],[50.8387356,36.4005536],[50.8565154,36.3455479],[50.8479127,36.2940161],[50.6534512,36.33471],[50.4959428,36.3422142],[50.4134978,36.3090408],[50.4320636,36.2452451],[50.6781745,36.1152612],[50.5776122,36.0824867],[50.5473363,36.0183135],[50.4259411,36.0016496],[50.3731618,35.8983085],[50.2970462,35.8224764],[50.250403,35.806938],[50.2437107,35.6477508],[50.1634282,35.6141399],[50.2289876,35.5405225],[50.0042535,35.5035807],[49.8803583,35.5469146],[49.7928561,35.5133096],[49.6875739,35.5341379],[49.6697942,35.5183046],[49.6936838,35.4810829],[49.6842407,35.4488555],[49.6228485,35.4488555],[49.5427042,35.4027091]]]}},{ "type": "Feature", "properties": { "province\_name": "四川省 四川省", "name\_fa": "四川省", "geometry": { "type": "Polygon", "coordinates": [[[49.878335,36.5781833],[49.7876613,36.5593829],[49.615622,36.606939],[49.4750603,36.6130538],[49.2375491,36.8500039],[49.2144926,36.9008328],[49.1703228,36.9189003],[49.0567071,37.0544501],[48.9308649,37.1516745],[48.8953085,37.2255624],[48.9047549,37.3144585],[48.7152994,37.4741889],[48.6800198,37.5928009],[48.6252935,37.6330795],[48.5958469,37.6905837],[48.639463,37.7216943],[48.6541827,37.7789116],[48.5919597,37.8030771],[48.5708464,37.9019768],[48.6205722,38.0133698],[48.6330717,38.1130875],[48.6830743,38.1522617],[48.6666845,38.1786488],[48.6900177,38.2222562],[48.6716842,38.2897628],[48.6391812,38.2908765],[48.5739016,38.3617065],[48.5818264,38.4603383],[48.6018782,38.4173601],[48.6622715,38.3929551],[48.7383031,38.4111207],[48.7913236,38.4517477],[49.2123986,38.4083871],[49.0663083,37.966461],[49.1204966,37.7506298],[49.1957131,37.6751317],[49.4286418,37.5739205],[49.7869314,37.5271119],[49.8572953,37.5508405],[49.9567752,37.5521229],[50.033611,37.5008104],[50.2325693,37.4655106],[50.292419,37.4359754],[50.4388082,37.1715669],[50.6550013,37.0454031],[50.5201082,36.8808404],[50.432606,36.8566679],[50.4323293,36.7300045],[50.3564903,36.6394415],[50.2012056,36.6780559],[50.0581539,36.6040616],[49.938971,36.6026379],[49.9359834,36.5724961],[49.878335,36.5781833]]]}},{ "type": "Feature", "properties": { "province\_name": "四川省 四川省", "name\_fa": "四川省", "geometry": { "type": "Polygon", "coordinates": [[[48.7250284,35.5548745],[48.6986389,35.5530258],[48.6639126,35.5593829],[48.5255742,35.5883061],[48.495018,35.5999729],[48.4836278,35.6452504],[48.3916789,35.6430234],[48.2961198,35.6805276],[48.2491702,35.7235838],[48.192501,35.7291459],[48.1733244,35.7883087],[48.0511056,35.8074728],[47.9999924,35.933032],[47.9097105,35.8891465],[47.8677643,35.9069207],[47.8622074,35.9413596],[47.9822125,35.9710839],[47.9030436,36.0022039],[47.9549895,36.049977],[47.994992,36.1397035],[47.9469321,36.2219334],[47.9563786,36.2813709],[47.9413786,36.3102638],[47.7822034,36.3719379],[47.6596985,36.3805406],[47.5283028,36.4249865],[47.4032943,36.4130417],[47.3552376,36.4752668],[47.3294013,36.6108249],[47.2913448,36.6863798],[47.2130085,36.7336091],[47.1741153,36.791667],[47.3027336,36.8972278],[47.4924644,37.0000012],[47.5241339,37.0672274],[47.59886,37.0375081],[47.663029,37.0994478],[47.7660877,37.0763991],[47.7788675,37.1322247],[47.8508135,37.171956],[47.9483186,37.1652881],[47.9833218,37.1477849],[47.9993331,37.112217],[48.0411012,37.1494499],[48.0997139,37.1558395],[48.1502733,37.2211248],[48.3922305,37.2516826],[48.4408434,37.2116773],[48.4741762,37.2102915],[48.5091796,37.1383458],[48.5464026,37.1352862],[48.5775155,37.1030587],[48.6558547,37.1291758],[48.7375235,37.1855641],[48.7900263,37.1391743],[48.8153061,37.1541761],[48.8697522,37.1464009],[48.9222549,37.1766737],[48.9319782,37.1502887],[49.0567071,37.0544501],[49.1703228,36.9189003],[49.2144926,36.9008328],[49.2314356,36.8575033],[49.2806052,36.8155578],[49.1891487,36.7607539],[49.1083657,36.7780687],[49.0664862,36.7191856],[49.061816,36.6801731],[49.0186487,36.6511105],[48.9755927,36.5461009],[49.0605954,36.4958249],[49.109765,36.4336014],[49.1911575,36.4286022],[49.2775532,36.3883271],[49.3369993,36.3894352],[49.3708889,36.3385999],[49.4245017,36.3272128],[49.3911688,36.2680397],[49.4336719,36.1930405],[49.3978359,36.0463661],[49.3019973,35.9758182],[49.2089383,35.9822079],[49.1517126,35.9299788],[49.0717101,35.9238678],[49.0758768,35.8899781],[49.0486539,35.8780353],[48.9342018,35.9305373],[48.9033685,35.9824777],[48.7872534,35.8872014],[48.7244712,35.7513594],[48.7328049,35.6916443],[48.770031,35.6785813],[48.7750314,35.6013619],[48.7250284,35.5548745]]]}},{ "type": "Feature", "properties": { "province\_name": "四川省 四川省", "name\_fa": "四川省", "geometry": { "type": "Polygon", "coordinates": [[[51.9685142,35.1399642],[51.8723985,35.2810784],[51.9439291,35.3613021],[51.9124013,35.4144115],[51.923511,35.5458079],[51.9579572,35.5780235],[51.9935134,35.5685834],[52.0351831,35.4935832],[52.2567937,35.4405264],[52.3790892,35.3374621],[52.5363172,35.4016339],[52.6674356,35.5358092],[52.7132718,35.5552487],[52.801054,35.5494151],[52.8991128,35.6419201],[52.995785,35.6788656],[53.1527365,35.7919297],[53.1399566,35.8183142],[53.0205078,35.8355399],[53.0207845,35.8841507],[53.0649541,35.9149853],[53.2205156,35.9269277],[53.2246855,35.9680443],[53.3705239,36.0602662],[53.6791502,36.0983219],[53.8874946,36.219994],[53.9538838,36.305829],[53.8799913,36.3672156],[53.9391606,36.4374972],[54.0124998,36.4455501],[54.155285,36.4997206],[54.508911,36.5147261],[54.5708602,36.5449978],[54.6494727,36.6180556],[54.7008655,36.6325007],[54.7780913,36.6002781],[54.8653169,36.6294477],[55.0100454,36.767221],[55.4008978,36.8169571],[55.5000701,36.9011157],[55.5603495,36.9252866],[55.6214658,37.0991758],[55.705915,37.1919577],[55.7164715,37.2664082],[55.8223071,37.322245],[55.908423,37.3323405],[55.9087027,37.2055724],[55.9514823,37.2044565],[56.0087049,37.2344567],[56.0628744,37.1608486],[55.9942616,37.1191819],[56.0400978,37.1002915],[56.0145379,37.0069571],[55.9791784,36.9962453],[55.9842616,36.9000082],[56.015651,36.8336127],[56.2214921,36.9008362],[56.2645516,36.8380601],[56.3651105,36.7530568],[56.3612239,36.6816685],[56.4465028,36.6941673],[56.4978956,36.6647208],[56.5937346,36.668611],[56.7584628,36.7038922],[56.7692958,36.6158298],[56.8420784,36.5897217],[56.8654112,36.5566687],[56.8640245,36.5208313],[56.8167982,36.4619424],[56.6984597,36.3883299],[56.7331824,36.313053],[56.800408,36.2386072],[56.7312388,36.0899851],[56.7217953,35.8599842],[56.915412,36.65927568],[57.0451409,35.5469161],[56.909714,35.4505255],[57.0501403,35.3071863],[57.0002296,35.2152486],[56.8556893,35.2024685],[56.6909577,35.0624638],[56.5367828,35.0477427],[56.4537237,34.9999609],[56.1639868,34.6263434],[55.9161995,34.2443852],[52.8863373,34.2488261],[52.6732697,34.3666098],[52.4999285,34.4191138],[52.3943698,34.4293925],[52.1551893,34.4013356],[52.0146278,34.4305022],[51.8335099,34.5071703],[51.8403823,34.5680611],[51.8885126,34.635511],[51.9590713,34.8580124],[52.0404635,35.0152423],[52.0357403,35.0884844],[51.9685142,35.1399642]]]}},{ "type": "Feature", "properties": { "province\_name": "四川省 四川省", "name\_fa": "四川省", "geometry": { "type": "Polygon", "coordinates": [[[51.7854549,30.701524],[51.5084986,30.8784775],[51.3776605,30.9965304],[51.3421007,31.0626407],[51.3271009,31.1212606],[51.3371005,31.2965324],[51.2934877,31.3640409],[51.3140441,31.3798755],[51.3037675,31.4209882],[51.259598,31.4607111],[51.2965441,31.5157101],[51.3054338,31.6618246],[51.3690463,31.7126555],[51.2854305,31.8021027],[51.2968203,31.8229413],[51.4121055,31.8618287],[51.429325,31.9998914],[51.3784923,32.0526666],[51.3857121,32.0915555],[51.3457126,32.1396181],[51.2337608,32.1743356],[51.1443119,32.2876719],[50.973194,32.3882322],[50.9684705,32.4335058],[51.0079166,32.5240621],[50.99653,32.5546268],[50.9318041,32.5985097],[50.9170809,32.664903],[50.887358,32.6962892],[50.8523551,32.7121267],[50.6518463,32.7105145],[50.565016,32.6723313],[50.5622097,32.6883087],[50.548673,32.6799061],[50.5498848,32.7138973],[50.5270627,32.7179593],[50.4581705,32.6607308],[50.5176162,32.631566],[50.4912268,32.5912856],[50.3153859,32.5371178],[50.2559398,32.5643506],[50.153711,32.6637915],[50.0445391,32.6274077],[49.9823168,32.6346172],[49.767306,32.7640711],[49.7317498,32.806572],[49.7486925,32.8557365],[49.6823002,32.8990767],[49.6375772,32.9774138],[49.7078563,32.982407],[49.6536869,33.0101894],[49.6689667,33.0501881],[49.722026,33.0843538],[49.837308,33.0499084],[49.9214799,33.0721379],[49.9187001,33.1004708],[49.8767539,33.132151],[49.8906435,33.1446311],[49.8714771,33.191301],[49.8895336,33.2196333],[49.9470359,33.2299169],[49.9375892,33.2471437],[50.0000919,33.3060283],[50.018148,33.3685323],[49.9478688,33.3874196],[49.959812,33.4157504],[49.9903684,33.4199214],[49.9720351,33.4676986],[49.9837016,33.4924176],[50.1917691,33.5310397],[50.2353814,33.6035334],[50.2731613,33.5824265],[50.3370535,33.615757],[50.4537253,33.6346521],[50.5631739,33.6282607],[50.7006785,33.7263176],[50.8870763,33.7057624],[51.0093045,33.7729798],[51.050974,33.8274312],[51.0320842,33.8529864],[51.0487506,33.9366011],[51.0200362,33.9818351],[51.0526483,34.0387274],[50.9081887,34.1518826],[50.9948608,34.234664],[51.0140271,34.287443],[51.0115269,34.3721675],[51.1773681,34.4443897],[51.3259864,34.4410599],[51.8007816,34.510984],[52.139633,34.4027254],[52.3943698,34.4293925],[52.5635444,34.4049456],[52.7485521,34.3332726],[52.8921704,34.2485553],[55.4786782,34.2518854],[55.497844,33.5290919],[55.3858954,33.3643638],[55.2225541,33.324088],[54.9917104,33.1257457],[54.8972616,33.1801987],[54.790313,33.1179718],[54.7447535,33.0657485],[54.5500228,33.0907497],[54.3483488,33.0529635],[54.3075126,33.0301923],[54.283623,32.9601897],[54.



```
2241771,32.8988003],[53.941384,32.7988012],[53.842492,32.6860155],[53.4624737,32.6951803],[53.1866274,32.6176809],[53.0152331,32.5154586],[53.058846,32.3321159],[53.0424564,31.9898882],[53.009677,31.8546066],[53.0530131,31.7459903],[53.1980177,31.6134876],[53.1944078,31.5732112],[52.9938406,31.4954342],[52.754944,31.4962646],[52.3935389,31.5482125],[52.2474209,31.6615455],[52.1304691,31.6565526],[52.0463004,31.59571],[51.9365721,31.4134866],[51.9679618,31.1943167],[51.8976831,30.9893098],[51.8354575,30.8934789],[51.8585137,30.8004145],[51.8326809,30.7404135],[51.7854549,30.7015241]]]]],{"type":"Feature","properties":{"province name":"\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1","name fa":"\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1","geometry":{"type":"Polygon","coordinates":[[[50.6031831,29.9437282],[50.5737369,29.9245595],[50.5342908,29.9762298],[50.5392903,30.0731804],[50.5770698,30.1253917],[50.5138933,30.1660606],[50.5207594,30.3904234],[50.4526215,30.4322936],[50.3864593,30.5235268],[50.5060119,30.5305816],[50.5495662,30.5630552],[50.5339013,30.5996434],[50.4237277,30.6651337],[50.392841,30.7131077],[50.3354417,30.7087156],[50.2541769,30.7555763],[50.2315138,30.7905283],[50.2434539,30.8325063],[50.2104225,30.9322635],[50.0697479,30.8299317],[50.0333405,30.8713077],[49.9555053,30.877364],[49.9511414,30.9293713],[49.9236754,30.9277065],[49.8865088,30.9625502],[49.97143,30.0129259],[49.9475175,31.0748626],[49.9899942,31.1256725],[49.95467,31.2114047],[50.099861,31.2575942],[50.1527063,31.2518317],[50.2467612,31.306489],[50.2998317,31.3584786],[50.2479222,31.4182174],[50.2540909,31.4483382],[50.2932415,31.4815706],[50.3795569,31.4173669],[50.476229,31.4276498],[50.5003954,31.4526482],[50.5748441,31.4087636],[50.6395699,31.42765],[50.7198524,31.3832062],[50.7892982,31.3143162],[50.8570867,31.2904206],[50.8923662,31.3087554],[50.9425891,31.2984821],[50.963202,31.2604226],[50.9694423,31.2265568],[51.0406997,31.1787511],[51.0784777,62,31.1887642],[51.1529267,31.1498667],[51.179041,31.1748696],[51.1804259,31.2098696],[51.2237631,31.2237608],[51.3298774,31.1959862],[51.3298776,31.1020875],[51.3776605,30.9965304],[51.5084986,30.8784775],[51.8340676,30.6659746],[51.8882369,30.5981899],[51.8335144,30.519299],[51.8374012,30.4537399],[51.7429526,30.3715171],[51.6193375,30.4437414],[51.5746115,30.4223512],[51.3743279,30.4759652],[51.3496013,30.562633],[51.212632,30.6531948],[51.1773730,30.6498557],[51.1365375,30.5984689],[51.1429277,30.5640225],[51.2195967,30.4542988],[51.294879,30.3845735],[51.2384865,30.3362448],[51.2690428,30.2795718],[51.1797276,30.2662363],[51.1423714,30.2156827],[51.1065353,30.1876232],[51.0662558,30.2003987],[51.050423,30.1806761],[50.9559742,30.1559554],[50.9248614,30.1151223],[50.9031953,30.03484],[50.8465261,29.9656759],[50.686795,29.9959588],[50.6031831,29.9437282]]]]],{"type":"Feature","properties":{"province name":"\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1","name fa":"\u25a1\u25a1\u25a1\u25a1\u25a1\u25a1","geometry":{"type":"Polygon","coordinates":[[[46.8977236,34.7643915],[46.7677187,34.7432789],[46.7332716,34.7941102],[46.7345834,34.8399508],[46.6399327,34.8693974],[46.5562231,34.9219012],[46.4996471,35.0363406],[46.46465912,35.0538401],[46.3471386,35.1705126],[46.2171325,35.2377326],[46.050838,35.3879811],[45.985425,35.496023],[45.992784,35.519464],[46.017991,35.520698],[45.972104,35.581178],[46.029957,35.576616],[46.013005,35.627783],[45.992997,35.634401],[46.009149,35.678707],[46.042465,35.706728],[46.165081,35.688964],[46.19956,35.72232],[46.277041,35.728866],[46.305853,35.761978],[46.333321,35.763349],[46.340244,35.819902],[46.294615,35.835867],[46.212274,35.794952],[46.183814,35.812492],[46.167906,35.799993],[46.137288,35.843527],[46.078562,35.85843],[46.024647,35.834468],[45.964013,35.843593],[45.954344,35.822803],[45.896036,35.837649],[45.874868,35.811362],[45.80038,35.827039],[45.763701,35.799014],[45.665625,35.926916],[45.5579743,35.9951027],[45.5545969,36.07414],[45.6746055,36.107753],[45.6412687,36.1388683],[45.6796053,36.1724786],[45.6832145,36.2247025],[45.732843,36.2388664],[45.8301661,36.2191437],[45.9471146,36.2391462],[45.9846175,36.2810865],[45.9540609,36.3933188],[45.9923967,36.4249824],[46.1454585,36.4633196],[46.1996276,36.4544294],[46.2801875,36.388041],[46.3221337,36.3774833],[46.4168591,36.3858163],[46.4596381,36.4163721],[46.46743725,36.3960954],[46.8029876,36.4238767],[46.8971601,36.383043],[46.9216069,36.3227659],[47.0179989,36.3191536],[47.080502,36.2508172],[47.1605078,36.2433122],[47.2985693,36.2477635],[47.3463491,36.2777567],[47.392185,36.3344337],[47.3657952
```

8471], [45.7509927, 37.1836129], [46.0046132, 37.1505605], [46.175456, 36.9955512], [46.1774022, 36.9552699], [46.1987926, 36.944408], [46.2862944, 36.9716711], [46.4124124, 37.0563881], [46.4476919, 37.1013887], [46.4960282, 37.0919479], [46.4607487, 37.020277], [46.6421472, 37.0411149], [46.7093696, 37.0724994], [46.8019014, 37.0492971], [46.8627115, 36.9997297], [46.8318781, 36.976102], [46.8427122, 36.927501], [46.8988245, 36.8947151], [46.8779925, 36.7799973], [46.9979975, 36.7452781], [47.0368872, 36.8013815], [47.1405026, 36.8322183], [47.2130085, 36.7336091], [47.2913448, 36.6863798], [47.3294013, 36.6108249], [47.3713513, 36.4388763], [47.3657952, 36.3869358], [47.3880185, 36.3222085], [47.312736, 36.2513763], [47.1899545, 36.2416544], [47.080502, 36.2508172], [47.0179989, 36.3191536], [46.9216069, 36.3227659], [46.8971601, 36.383043], [46.7888209, 36.4269304], [46.7560413, 36.4055363], [46.7296517, 36.4155441], [46.6743725, 36.3960954], [46.4596381, 36.4163721], [46.4168591, 36.3858163], [46.3221337, 36.3774833], [46.2801875, 36.388041], [46.1996276, 36.4544294], [46.0976794, 36.4563747], [45.9923967, 36.4249824], [45.9540609, 36.3933188], [45.9846175, 36.2810865], [45.9551718, 36.2435882], [45.8301661, 36.2191437], [45.698771, 36.2352622], [45.6743245, 36.2138633], [45.6796053, 36.1724786], [45.6412687, 36.1388683], [45.6746055, 36.107753], [45.5545969, 36.07414], [45.5579743, 35.9951027], [45.5184017, 35.9957851]]], {"type": "Feature", "properties": {"province\_name": "四川省 甘孜州", "name\_fa": "甘孜州"}, "geometry": {"type": "Polygon", "coordinates": [[[ [53.9922201, 27.1889532], [53.9333276, 27.1236778], [53.7960999, 27.0458956], [53.6322058, 27.0983966], [53.5419239, 27.1578432], [53.496088, 27.1011746], [53.4096961, 27.1139499], [53.3535802, 27.1911741], [53.1307937, 27.2720136], [53.0496784, 27.2750621], [52.9438433, 27.3295156], [52.7546663, 27.5028527], [52.6238284, 27.5634067], [52.5632693, 27.6245148], [52.5016001, 27.7864703], [52.3879852, 27.9931387], [52.2852003, 28.0442452], [52.1707517, 28.130082], [52.1254724, 28.1995309], [52.1229722, 28.2623041], [52.0540799, 28.4284232], [51.9604644, 28.8212057], [51.8674055, 28.9798252], [51.79407, 29.0445478], [51.8151794, 29.068163], [51.8093463, 29.0995487], [51.6740614, 29.2342696], [51.587116, 29.3914949], [51.5085004, 29.4528891], [51.4276615, 29.475107], [51.364606, 29.4634389], [51.3209933, 29.4976138], [51.2701572, 29.6081709], [51.2996034, 29.6784425], [51.2782135, 29.7006648], [51.1726518, 29.7142832], [50.9998675, 29.7828922], [50.8154136, 29.7753968], [50.7037418, 29.7915055], [50.6448493, 29.822064], [50.6751287, 29.8628971], [50.7137415, 29.873171], [50.6456824, 29.9354023], [50.6031831, 29.9437282], [50.6159594, 29.9601188], [50.686795, 29.9959588], [50.8465261, 29.9656759], [50.9031953, 30.03484], [50.9248614, 30.1151223], [50.9559742, 30.1559554], [51.050423, 30.1806761], [51.0662558, 30.2003987], [51.1065353, 30.1876232], [51.1423714, 30.2156827], [51.1779276, 30.2062363], [51.2690428, 30.2795718], [51.2384865, 30.3362448], [51.294879, 30.3845735], [51.2195967, 30.4542988], [51.1429277, 30.5640225], [51.1365375, 30.5984689], [51.1773736, 30.6498557], [51.2112632, 30.6531948], [51.3496013, 30.562633], [51.3665478, 30.4862435], [51.3993274, 30.4637379], [51.4471068, 30.4679069], [51.5746115, 30.4223552], [51.619375, 30.4437414], [51.7429526, 30.3715171], [51.8262914, 30.4365226], [51.8429576, 30.4656874], [51.8335144, 30.519299], [51.8882369, 30.5981899], [51.7851782, 30.7006937], [51.836809, 30.7404135], [51.8585137, 30.8004145], [51.8354575, 30.8934789], [51.8976831, 30.9893098], [51.9679618, 31.1943167], [51.9365721, 31.4134866], [52.0463004, 31.59571], [52.1304691, 31.6565526], [52.2474209, 31.6615455], [52.3935389, 31.5482125], [52.5596566, 31.5123797], [52.8010566, 31.4948743], [52.8485593, 31.4746488], [52.9346715, 31.4048746], [52.968841, 31.3618213], [52.9716178, 31.2770885], [52.9457847, 31.240988], [52.8688358, 31.2237639], [52.831613, 31.1865358], [52.9118953, 31.1212617], [52.9693979, 31.0145934], [53.3774721, 30.8254161], [53.4710875, 30.7315245], [53.7838764, 30.6751358], [53.854992, 30.6479168], [53.9108279, 30.5918013], [53.9308275, 30.5401367], [53.8986047, 30.4456825], [53.9255511, 30.3340175], [53.8641586, 30.1937419], [53.9399976, 30.0615178], [54.0980585, 29.9384533], [54.1030585, 29.7928965], [54.2427864, 29.5962233], [54.4764096, 29.6412243], [54.8161445, 29.5937251], [54.983932, 29.3948318], [55.1642158, 29.253716], [55.2622777, 29.0945466], [55.2806106, 28.9951025], [55.2439278, 28.842599], [55.2519975, 28.7239896], [55.3508892, 28.6603707], [55.4528411, 28.5439838], [55.4078383, 28.4184168], [55.386725, 27.998693], [55.5006199, 27.9234117], [55.5772885, 27.7945249], [55.4806168, 27.7292443], [55.4103378, 27.6195123], [55.3314457, 27.5717377], [55.2567201, 27.4956291], [55.1392152, 27.4342357], [54.6289134, 27.4678505], [54.3530677, 27.4586846], [54.1577842, 27.4214541], [53.9686071, 27.3500664], [53.9708304, 27.3070144], [54.0349994, 27.2350637], [53.9922201, 27.1889532]]]]], {"type": "Feature", "properties": {"province\_name": "四川省 凉山州", "name\_fa": "凉山州"}, "geometry": {"type": "Polygon", "coordinates": [[[ [52.482398, 26.989407], [51.2988492, 27.699037], [50.1313982, 29.0842133], [49.5801433, 29.6674426], [50.1390342, 30.1759444], [50.2000071, 30.2949624], [50.4165763, 30.1821897], [50.5717901, 30.137902], [50.5748464, 30.1162229], [50.5392903, 30.0731804], [50.5342908, 29.9762298], [50.5737369, 29.9245595], [50.6031831, 29.9437282], [50.642349, 29.9373417], [50.7137415, 29.873171], [50.6751287, 29.8628971], [50.6445727, 29.8287296], [50.6656856, 29.8056752], [50.8373599, 29.7734487], [50.9998675, 29.7828922], [51.1726518, 29.7142832], [51.2873801, 29.6948386], [51.2965467, 29.6584482], [51.2701572, 29.6081709], [51.3209933, 29.4976138], [51.364606, 29.4634389], [51.4421081, 29.4739979], [51.5515564, 29.425388], [51.6740614, 29.2342696], [51.8065698, 29.1037131], [51.8151794, 29.068163], [51.79407, 29.0445478], [51.8674055, 28.9798252], [51.9604644, 28.8212057], [52.0540799, 28.4284232], [52.1229722, 28.2623041], [52.1254724, 28.1995309], [52.1582518, 28.1456406], [52.2652003, 28.0442452], [52.3879852, 27.9931387], [52.5016001, 27.7864703], [52.5427129, 27.6631241], [52.6027152, 27.5806265], [52.7546663, 27.5028527], [52.9438433, 27.3297854], [52.8852276, 27.2928411], [52.8165666, 27.3237282], [52.7379214, 27.2743065], [52.482398, 26.989407]]]]], {"type": "Feature", "properties": {"province\_name": "四川省 攀枝花市", "name\_fa": "攀枝花市"}, "geometry": {"type": "Polygon", "coordinates": [[[ [48.5872376, 37.1094519], [48.5091796, 37.1383458], [48.4780662, 37.2069532], [48.4208402, 37.2219532], [48.4169537, 37.2458431], [48.3350043, 37.2394544], [48.3352812, 37.2664028], [48.1494391, 37.416124], [48.1552721, 37.4783524], [48.0608231, 37.5147348], [48.0530467, 37.533525], [48.079156, 37.5783566], [48.0216532, 37.6122422], [48.0255429, 37.654462], [48.0666557, 37.7294704], [48.1352716, 37.7522446], [48.1511045, 37.8103041], [47.999403, 37.8949567], [47.977486, 37.8758633], [47.8930368, 37.8683574], [47.887753, 37.8978047], [47.9377595, 37.9466916], [47.9097032, 37.9908649], [47.8622001, 38.0066999], [47.8521997, 38.1466985], [47.8747737, 38.2514228], [47.7246942, 38.2122555], [47.6171887, 38.2183641], [47.4827366, 38.1819793], [47.4688466, 38.2494817], [47.3985677, 38.2378125], [47.2871719, 38.302263], [47.2907813, 38.3319841], [47.345784, 38.3569797], [47.4641225, 38.4780964], [47.4985682, 38.5811588], [47.5688509, 38.6117084], [47.564954, 38.6422788], [47.5257943, 38.6697777], [47.4357886, 38.7003296], [47.4313449, 38.7564406], [47.4696776, 38.7914396], [47.5980199, 38.8189437], [47.5116235, 38.9158902], [47.5363466, 38.9500526], [47.5266267, 38.9586645], [47.5630161, 38.9775567], [47.50218, 39.0000537], [47.5230132, 39.0142262], [47.5052366, 39.0353314], [47.5274594, 39.0806154], [47.4974563, 39.1283969], [47.45194023, 39.1508961], [47.518569, 39.2197857], [47.4810658, 39.2725645], [47.5049349, 39.2954356], [47.4307925, 39.3186376], [47.4391129, 39.3372671], [47.363399, 39.4217256], [47.3775701, 39.4549047], [47.5629369, 39.5137751], [47.752969, 39.606023], [47.8073216, 39.6591904], [47.9174231, 39.6614998], [47.9821222, 39.7090003], [48.0320584, 39.6925275], [48.3686221, 39.3798361], [48.2653165, 39.3334445], [48.211001, 39.3291082], [48.1297734, 39.2713758], [48.1502833, 39.1982545], [48.2184151, 39.1627439], [48.2349786, 39.1296652], [48.3077928, 39.1085272], [48.3332227, 39.0211657], [48.2707532, 38.9673018], [48.0882345, 38.9469508], [48.0692648, 38.9109368], [48.01027, 38.907306], [48.0207028, 38.8378898], [48.1071082, 38.7885576], [48.1073103, 38.7698813], [48.2491558, 38.7260959], [48.252657, 38.6619076], [48.2949419, 38.6473191], [48.3102234, 38.6076539], [48.361489, 38.5948538], [48.4592932, 38.6076878], [48.464684, 38.5613214], [48.5055078, 38.5470001], [48.5133037, 38.518379], [48.579433, 38.4668173], [48.5739016, 38.3617065], [48.6391812, 38.2908765], [48.6716842, 38.2897628], [48.6900177, 38.2222562], [48.6666845, 38.1786488], [48.6830743, 38.1522617], [48.6330717, 38.1130875], [48.6205722, 38.0133698], [48.5708464, 37.9019768], [48.5919597, 37.8030771], [48.6541827, 37.7789116], [48.639463, 37.7216943], [48.5958469, 37.6905837], [48.6252935, 37.6330795], [48.6800198, 37.5928009], [48.7152994, 37.4741889], [48.9105878, 37.3036261], [48.8953085, 37.2255624], [48.9222549, 37.1766737], [48.8697522, 37.1464009], [48.8153061, 37.1541761], [48.7900263, 37.1391743], [48.7375235, 37.1855641], [48.658547, 37.1291758], [48.5872376, 37.1094519]]]]], {"type": "Feature", "properties": {"province\_name": "四川省 甘孜州", "name\_fa": "甘孜州"}, "geometry": {"type": "Polygon", "coordinates": [[[ [52.2165806, 35.7655351], [52.0726856, 35.7633173], [52.0062932, 35.7866467], [51.8782311, 35.8599847], [51.760449, 36.0066529], [51.4448757, 36.1494318], [51.315984, 36.1544326], [51.2679243, 36.1841587], [51.2015319, 36.1624881], [50.8479127, 36.2940161], [50.8565154, 36.3455479], [50.8345689, 36.4047155], [50.6798406, 36.4999976], [50.3598236, 36.6416659], [50.432323, 36.7300045], [50.432606, 36.8566679], [50.5201082, 36.8808404], [50.6550013, 37.0454031], [50.9556187, 36.8932945], [51.2168547, 36.8156359], [51.487796, 36.7748336], [51.6236711, 36.7197484], [51.9164495, 36.6626774], [53.2905669, 36.9308012], [53.8526691, 36.9844424], [53.8647148, 36.6794543], [53.9647204, 36.675285], [54.1172253, 36.5977745], [54.1255585, 36.4911097], [54.0124998, 36.4455501], [53.9391606, 36.4374972], [53.8813813, 36.3777735], [53.9538838, 36.3099913], [53.9277742, 36.2574933], [53.6791502, 36.0983219], [53.3705239, 36.0602662], [53.2246855, 35.9680443], [53.2232956, 35.9283157], [53.0649541, 35.9149853], [52.9874517, 35.83554], [52.9207825, 35.8377573], [52.7049383, 35.9280361], [52.5679864, 35.943313], [52.4357581, 35.9122089], [52.3782555, 35.8485917], [52.2165806, 35.7655351]]]]], {"type": "Feature", "properties": {"province\_name": "四川省 甘孜州", "name\_fa": "甘孜州"}}

```
[[{"type": "Polygon", "coordinates": [[[54.3327929, 36.510556], [54.1261152, 36.4916591], [54.1280586, 36.5702788], [54.1044488, 36.6158313], [53.9647204, 36.675285], [53.8647148, 36.6794543], [53.8526691, 36.9844424], [53.897152, 37.0186755], [53.8623745, 37.203783], [53.8777413, 37.2469311], [53.8607569, 37.32222], [53.9277235, 37.3472619], [54.2446779, 37.3199856], [54.2851929, 37.351769], [54.3976597, 37.3606995], [54.5867621, 37.4600312], [54.6736676, 37.437055], [54.7925151, 37.5254889], [54.8152475, 37.6062298], [54.7827548, 37.6343399], [54.8402131, 37.745694], [55.0233227, 37.8429634], [55.0410295, 37.8796158], [55.1276555, 37.9491996], [55.1695862, 37.9606129], [55.2015097, 37.9473068], [55.2271298, 37.9792886], [55.384722, 38.041944], [55.442778, 38.086111], [55.756111, 38.125278], [55.9625, 38.078333], [56.164444, 38.095], [56.216111, 38.072222], [56.2889756, 38.0846583], [56.3103894, 37.9103136], [56.2803861, 37.8672577], [56.2156603, 37.8594822], [56.118711, 37.6347476], [56.0612082, 37.5722467], [56.0584315, 37.5191926], [56.0012054, 37.4425221], [56.0164852, 37.3702954], [55.8086256, 37.3178633], [55.7200814, 37.2711319], [55.705915, 37.1919577], [55.6214658, 37.0991758], [55.5534062, 36.918346], [55.5000701, 36.9011157], [55.4008978, 36.8169571], [55.0014356, 36.7641687], [54.9122631, 36.6644427], [54.8478172, 36.6222252], [54.7678114, 36.6002781], [54.7008655, 36.6325007], [54.6447528, 36.6161104], [54.5175242, 36.5172211], [54.3327929, 36.510556]]]]]]}
```

```
@@ public/data/world-countries.geojson text
{"type": "FeatureCollection", "attribution": "Natural Earth public domain", "features": [{"type": "Feature", "properties": {"name": "中国", "bbox": [-180, -18.28799, 180, -16.020882], "geometry": {"type": "MultiPolygon", "coordinates": [[[[[180, -16.067133], [180, -16.555217], [179.364143, -16.801354], [178.725059, -17.012042], [178.596839, -16.63915], [179.096609, -16.433984], [179.413509, -16.379054], [180, -16.067133]]], [[178.12557, -17.50481], [178.3736, -17.33992], [178.71806, -17.62846], [178.55271, -18.15059], [177.93266, -18.28799], [177.38146, -18.16432], [177.28504, -17.72465], [177.67087, -17.38114], [178.12557, -17.50481], [[[-17.979332, -16.020882], [-179.917369, -16.501783], [-180, -16.555217], [-180, -16.067133], [-179.79332, -16.020882]]]]}], {"type": "Feature", "properties": {"name": "中国", "bbox": [29.339998, -11.720938, 40.31659, -0.95], "geometry": {"type": "Polygon", "coordinates": [[33.903711, -0.95], [34.07262, -1.05982], [37.69869, -3.09699], [37.7669, -3.67712], [39.20222, -4.67677], [38.74054, -5.09895], [38.79977, -6.47566], [39.44, -6.84], [39.47, -7.1], [39.19469, -7.7039], [39.25203, -8.00781], [39.18652, -8.48551], [39.53574, -9.11237], [39.9496, -10.0984], [40.316586, -10.317098], [40.31659, -10.3171], [39.521, -10.89688], [38.427557, -11.285202], [37.82764, -11.26879], [37.47129, -11.56876], [36.775151, -11.594537], [36.514082, -11.720938], [35.312398, -11.439146], [34.559989, -11.52002], [34.28, -10.16], [33.940838, -9.693674], [33.73972, -9.41715], [32.759375, -9.230599], [32.191865, -8.930359], [31.556348, -8.762049], [31.157751, -8.594579], [30.74001, -8.340006], [30.740015, -8.340007], [30.199997, -7.079981], [29.620032, -6.520015], [29.419993, -5.939999], [29.519987, -5.419979], [29.339998, -4.499983], [29.753512, -4.452389], [30.11632, -4.09012], [30.50554, -3.56858], [30.75224, -3.35931], [30.74301, -3.03431], [30.52766, -2.80762], [30.469674, -2.413855], [30.46967, -2.41383], [30.758309, -2.28725], [30.816135, -1.698914], [30.419105, -1.134659], [30.76986, -1.01455], [31.86617, -1.02736], [33.903711, -0.95]]]]}], {"type": "Feature", "properties": {"name": "中国", "bbox": [-17.063423, 20.999752, -8.665124, 27.656426], "geometry": {"type": "Polygon", "coordinates": [[[-8.66559, 27.656426], [-8.665124, 27.589479], [-8.6844, 27.395744], [-8.687294, 25.881056], [-11.969419, 25.933533], [-11.937224, 23.374594], [-12.874222, 23.284832], [-13.118754, 22.77122], [-12.929102, 21.327071], [-16.845194, 21.333323], [-17.063423, 20.999752], [-17.020428, 21.42231], [-17.002962, 21.420734], [-14.750955, 21.5006], [-14.630833, 21.86094], [-14.221168, 22.310163], [-13.89111, 23.691009], [-12.500963, 24.770116], [-12.030759, 26.030866], [-11.71822, 26.104092], [-11.392555, 26.883424], [-10.551263, 26.990808], [-10.189424, 26.860945], [-9.735343, 26.860945], [-9.413037, 27.088476], [-8.794884, 27.120696], [-8.817828, 27.656426], [-8.66559, 27.656426]]]]}], {"type": "Feature", "properties": {"name": "中国", "bbox": [-140.99778, 41.675105, -52.648099, 83.23324], "geometry": {"type": "MultiPolygon", "coordinates": [[[[[-122.84, 49], [-122.97421, 49.002538], [-124.91024, 49.98456], [-125.62461, 50.41656], [-127.43561, 50.83061], [-127.99276, 51.71583], [-127.85032, 52.32961], [-129.12979, 52.75538], [-129.30523, 53.56159], [-130.51497, 54.28757], [-130.536109, 54.802754], [-130.53611, 54.80278], [-129.98, 55.285], [-130.00778, 55.91583], [-131.70781, 56.55212], [-132.73042, 57.69289], [-133.35556, 58.41028], [-134.27111, 58.86111], [-134.94559, 59.27056], [-135.47583, 59.78778], [-136.47972, 59.46389], [-137.4525, 58.905], [-138.34089, 59.56211], [-139.039, 60], [-140.013, 60.27682], [-140.99778, 60.30639], [-140.9925, 66.00003], [-140.986, 69.712], [-140.985988, 69.711998], [-139.12052, 69.47102], [-137.54636, 68.99002], [-136.50358, 68.89804], [-135.62576, 69.31512], [-134.41464, 69.62743], [-132.92925, 69.50534], [-131.43136, 69.94451], [-129.79471, 70.19369], [-129.10773, 69.77927], [-128.36156, 70.01286], [-128.13817, 70.48384], [-127.44712, 70.37721], [-125.75632, 69.48058], [-124.42483, 70.1584], [-124.28968, 69.39969], [-123.06108, 69.56372], [-122.6835, 69.85553], [-121.47226, 69.79778], [-119.94288, 69.37786], [-117.60268, 69.01128], [-116.22643, 68.84151], [-115.2469, 68.90591], [-113.89794, 68.3989], [-115.30489, 67.90261], [-113.49272, 67.68815], [-110.798, 67.80612], [-109.94619, 67.98104], [-108.8802, 67.38144], [-107.79239, 67.8736], [-108.81299, 68.31164], [-108.16721, 68.65392], [-106.95, 68.7], [-106.15, 68.8], [-105.34282, 68.56122], [-104.33791, 68.018], [-103.22115, 68.09775], [-101.45433, 67.64689], [-99.90195, 67.80566], [-98.4432, 67.78165], [-98.5586, 68.40394], [-97.66948, 68.57864], [-96.11991, 68.23939], [-96.12588, 67.29338], [-95.48943, 68.0907], [-94.685, 68.06383], [-94.23282, 69.06903], [-95.30408, 69.68571], [-96.47131, 70.08976], [-96.39115, 71.19482], [-95.2088, 71.92053], [-93.88997, 71.76015], [-92.87818, 71.31869], [-91.51964, 70.19129], [-92.40692, 69.69997], [-90.5471, 69.49766], [-90.55151, 68.47499], [-89.21515, 69.25873], [-88.01966, 68.61508], [-88.31749, 67.87338], [-87.35017, 67.19872], [-86.30607, 67.92146], [-85.57664, 68.78456], [-85.52197, 69.88211], [-84.10081, 69.80539], [-82.62258, 69.65826], [-81.28043, 69.16202], [-81.22022, 68.66567], [-81.96436, 68.13253], [-81.25928, 67.59716], [-81.38653, 67.11078], [-83.34456, 66.41154], [-84.73542, 66.2573], [-85.76943, 66.55833], [-86.0676, 66.05625], [-87.03143, 65.21297], [-87.32324, 64.77563], [-88.48296, 64.09897], [-89.91444, 64.03273], [-90.70398, 63.61017], [-90.77004, 62.96021], [-91.93342, 62.83508], [-93.15698, 62.02469], [-94.24153, 60.89865], [-94.62931, 60.11021], [-94.6846, 58.94882], [-93.21502, 58.78212], [-92.76462, 57.84571], [-92.29703, 57.08709], [-90.89769, 57.28468], [-89.03953, 56.85172], [-88.03978, 56.47162], [-87.32421, 55.99914], [-86.07121, 55.72383], [-85.01181, 55.3026], [-83.36055, 55.24489], [-82.27285, 55.14832], [-82.4362, 54.28227], [-82.12502, 53.27703], [-81.40075, 52.15788], [-79.91289, 51.20842], [-79.14301, 51.53393], [-78.60191, 52.56208], [-79.12421, 54.14145], [-79.82958, 54.66772], [-78.22874, 55.13645], [-77.0956, 55.83741], [-76.54137, 56.53423], [-76.62319, 57.20263], [-77.30226, 58.05209], [-78.51688, 58.80458], [-77.33676, 59.85261], [-77.77272, 60.75788], [-78.10687, 62.31964], [-77.41067, 62.55053], [-75.69621, 62.2784], [-74.76682, 62.18111], [-73.83988, 62.4438], [-72.90853, 62.10507], [-71.67708, 61.52535], [-71.37369, 61.13717], [-69.59042, 61.06141], [-69.62033, 60.22125], [-69.2879, 58.95736], [-68.37455, 58.80106], [-67.64976, 58.21206], [-66.20178, 58.76731], [-65.24517, 59.87071], [-64.58352, 60.33558], [-63.80475, 59.4426], [-62.50236, 58.16708], [-61.39655, 56.96745], [-61.79866, 56.33945], [-60.46853, 55.77548], [-59.56962, 55.20407], [-57.97508, 54.94549], [-57.3332, 54.6265], [-56.93689, 53.78032], [-56.15811, 53.64749], [-55.75632, 53.27036], [-55.68338, 52.14664], [-56.40916, 51.7707], [-57.12691, 51.41972], [-58.77482, 51.0643], [-60.03309, 50.24277], [-61.72366, 50.08046], [-63.86251, 50.29099], [-65.36331, 50.2982], [-66.39905, 50.22897], [-67.23631, 49.51156], [-68.51114, 49.06836], [-69.95362, 47.74488], [-71.10458, 46.82171], [-70.25522, 46.98606], [-68.65, 48.3], [-66.55243, 49.1331], [-65.05626, 49.23278], [-64.17099, 48.74248], [-65.11545, 48.07085], [-64.79854, 46.99297], [-64.47219, 46.23849], [-63.17329, 45.73902], [-61.52072, 45.88377], [-60.51815, 47.00793], [-60.44486, 46.28264], [-59.80287, 45.9204], [-61.03988, 45.26525], [-63.25471, 44.67014], [-64.24656, 44.26553], [-65.36406, 43.54523], [-66.1234, 43.61867], [-66.16173, 44.46512], [-64.42549, 45.29204], [-66.02605, 45.25931], [-67.13741, 45.13753], [-67.79134, 45.70281], [-67.79046, 47.06636], [-68.23444, 47.35486], [-68.905, 47.185], [-69.237216, 47.447781], [-69.9999, 47.469307], [-70.305, 45.915], [-70.66, 45.46], [-71.08482, 45.30524], [-71.405, 45.255], [-71.50506, 45.0082], [-73.34783, 45.00738], [-74.867, 45.00048], [-75.31821, 44.81645], [-76.375, 44.09631], [-76.5, 44.018459], [-76.820034, 43.628784], [-77.737885, 43.629056], [-78.72028, 43.625089], [-79.171674, 43.466339], [-79.01, 43.27], [-78.92, 42.965], [-78.939362, 42.863611], [-80.247448, 42.3662], [-81.277747, 42.209026], [-82.439278, 41.675105], [-82.690089, 41.675105], [-83.02981, 41.832796], [-83.142, 41.975681], [-83.12, 42.08], [-82.9, 42.43], [-82.43, 42.98], [-82.137642, 43.571088], [-82.337763, 44.44], [-82.550925, 45.347117], [-83.592851, 45.816894], [-83.469551, 45.994686], [-83.616131, 46.116927], [-83.890765, 46.116927], [-84.091851, 46.275419], [-84.14212, 46.512226], [-84.3367, 46.40877], [-84.6049, 46.4396], [-84.543749, 46.538684], [-84.779238, 46.637102], [-84.87608, 46.900083], [-85.652363, 47.220219], [-86.461991, 47.553338], [-87.439793, 47.94], [-88.378114, 48.302918], [-89.272917, 48.019808], [-89.6, 48.01], [-90.83, 48.27], [-91.64, 48.14], [-92.61, 48.45], [-93.63087, 48.60926], [-94.32914, 48.67074], [-94.64, 48.84], [-94.81758, 49.38905], [-95.15609, 49.38425], [-95.15907, 49], [-97.22872, 49.0007], [-100.65, 49], [-104.04826, 48.99986], [-107.05, 49], [-110.05, 49], [-113.49], [-116.04818, 49], [-117.03121, 49], [-120, 49], [-122.84, 49]]]]], [[[-83.99367, 62.4528], [-83.25048, 62.91409], [-81.87699, 62.9
```

[+8.8925, 62.7108], [-81.9885, 62.7108], [-83.06857, 62.15922], [-83.77462, 62.18231], [-83.99367, 62.4528]], [[[-79.775833, 72.802902], [-80.876099, 73.333183], [-80.833885, 73.693184], [-80.353058, 73.75972], [-78.064438, 73.651932], [-76.34, 73.102685], [-76.251404, 72.826385], [-77.314438, 72.855545], [-78.39167, 72.876656], [-79.486252, 72.742203], [-79.775833, 72.802902], [[[-80.315395, 62.08565], [-79.92399, 62.3856], [-79.52002, 62.36371], [-79.26582, 62.158675], [-79.65752, 61.63308], [-80.09956, 61.7181], [-80.36215, 62.01649], [-80.315395, 62.085565]]], [[[-93.612756, 74.979997], [-94.156909, 74.592347], [-95.608681, 74.666864], [-96.820932, 74.927623], [-96.288587, 75.377828], [-94.85082, 75.647218], [-93.977747, 75.29649], [-93.612756, 74.979997]], [[[-93.840003, 77.519997], [-94.295608, 77.491343], [-96.169654, 77.555111], [-96.436304, 77.834629], [-94.422577, 77.820005], [-93.720656, 77.634331], [-93.840003, 77.519997]], [[[-96.754399, 78.765813], [-95.559278, 78.418315], [-95.830295, 78.056941], [-97.309843, 77.850597], [-98.124289, 78.082857], [-98.552868, 78.458105], [-98.631984, 78.87193], [-97.337321, 78.331984], [-96.754399, 78.765813]], [[[-88.15035, 74.392307], [-89.764722, 74.515555], [-92.422441, 74.837758], [-92.768285, 75.38682], [-92.889906, 75.882655], [-93.893824, 76.319244], [-95.962457, 76.441381], [-97.121379, 76.751078], [-96.745123, 77.161389], [-94.684086, 77.097878], [-93.573921, 76.776296], [-91.605023, 76.778518], [-90.741846, 76.449597], [-90.969661, 76.074013], [-89.822238, 75.847774], [-89.187083, 75.610166], [-87.838276, 75.566189], [-86.379192, 75.482421], [-84.789625, 75.699204], [-82.753445, 75.784315], [-81.128531, 75.713983], [-80.057511, 75.336849], [-79.833933, 74.923127], [-80.457771, 74.657304], [-81.948843, 74.442459], [-83.228894, 74.564028], [-86.097452, 74.410032], [-88.15035, 74.392307]], [[[-111.264443, 78.152956], [-109.854452, 77.996325], [-110.186938, 77.697015], [-112.205119, 77.409229], [-113.534279, 77.732207], [-112.724587, 78.05105], [-111.264443, 78.152956]], [[[-110.963661, 78.804441], [-109.663146, 78.601973], [-110.881314, 78.40692], [-112.542091, 78.407902], [-112.525891, 78.550555], [-111.50001, 78.849994], [-110.963661, 78.804441]], [[[-55.600218, 51.317075], [-56.134036, 50.68701], [-56.795882, 49.812309], [-56.143105, 50.150117], [-55.471492, 49.935815], [-55.822401, 49.587129], [-54.935143, 49.313011], [-54.473775, 49.566991], [-53.476549, 49.249139], [-53.786014, 48.516781], [-53.086134, 48.687804], [-52.958648, 48.157164], [-52.648099, 47.535548], [-53.069158, 46.655499], [-53.521456, 46.618292], [-54.178936, 46.807066], [-53.961869, 47.625207], [-54.240482, 47.752279], [-55.400773, 46.884994], [-55.997481, 46.91972], [-55.291219, 47.389562], [-56.250799, 47.632545], [-57.325229, 47.572807], [-59.266015, 47.603348], [-59.419494, 47.899454], [-58.796586, 48.251525], [-59.231625, 48.523188], [-58.391805, 49.125581], [-57.35869, 50.718274], [-56.73865, 51.287438], [-55.870977, 51.632094], [-55.406974, 51.588723], [-55.600218, 51.317075]], [[[-83.882626, 65.109618], [-82.787577, 64.766693], [-81.642014, 64.455136], [-81.55344, 63.979609], [-80.817361, 64.057486], [-80.103451, 63.725981], [-80.99102, 63.411246], [-82.547178, 63.651722], [-83.108798, 64.101876], [-84.100417, 63.569712], [-85.523405, 63.052379], [-85.866769, 63.637253], [-87.221983, 63.541238], [-86.35276, 64.035833], [-86.224886, 64.822917], [-85.883848, 65.738778], [-85.161308, 65.657285], [-84.975764, 65.217518], [-84.464012, 65.371772], [-83.882626, 65.109618]], [[[-78.770639, 72.352173], [-77.824624, 72.749617], [-75.605845, 72.243678], [-74.28616, 71.767144], [-74.099141, 71.33084], [-72.242226, 71.556925], [-71.200015, 70.920013], [-68.786054, 70.526024], [-67.91497, 70.121948], [-66.969033, 69.186087], [-68.805123, 68.720198], [-66.449866, 68.067163], [-64.862314, 67.847539], [-63.424934, 66.928473], [-61.851981, 66.862121], [-62.163177, 66.160251], [-63.918444, 64.998669], [-65.14886, 65.426033], [-66.721219, 66.388041], [-68.015016, 66.262726], [-68.141287, 65.689789], [-67.089646, 65.108455], [-65.73208, 64.648406], [-65.320168, 64.382737], [-64.669406, 63.392927], [-65.013804, 62.674185], [-66.275045, 62.945099], [-68.783186, 63.74567], [-67.369681, 62.883966], [-66.328297, 62.280075], [-66.165568, 61.930897], [-68.877367, 62.330149], [-71.023437, 62.910708], [-72.235379, 63.397836], [-71.886278, 63.679989], [-73.378306, 64.193963], [-74.834419, 64.679076], [-74.818503, 64.389093], [-77.70998, 64.229542], [-78.555949, 64.572906], [-77.897281, 65.309192], [-76.018274, 65.326969], [-73.959795, 65.454765], [-74.293883, 65.811771], [-73.944912, 6

6.34,78.18],[-87.96192,78.37181],[-87.15198,78.75867],[-85.37868,78.9969],[-85.09495,79.34543],[-86.50734,79.73624],[-86.93179,80.25145],[-84.19844,80.20836],[-83.408696,80.1],[-81.84823,80.46442],[-84.1,80.58],[-87.59895,80.51627],[-89.36663,80.85569],[-90.2,81.26],[-91.36786,81.5531],[-91.58702,81.89429]]],[[[-75.21597,67.44425],[-75.86588,67.14886],[-76.98687,67.09873],[-77.2364,67.58809],[-76.81166,68.14856],[-75.89521,68.28721],[-75.1145,68.01036],[-75.10333,67.58202],[-75.21597,67.44425]]],[[-96.257401,69.49003],[-95.647681,69.10769],[-96.269521,68.75704],[-97.617401,69.06003],[-98.431801,68.9507],[-99.797401,69.40003],[-98.917401,69.71003],[-98.218261,70.14354],[-97.157401,69.86003],[-96.557401,69.68003],[-96.257401,69.49003]]],[[-64.51912,49.87304],[-64.17322,49.95718],[-62.85829,49.70641],[-61.835585,49.28855],[-61.806305,49.10506],[-62.29318,49.08717],[-63.58926,49.40069],[-64.51912,49.87304]]],[[-64.01486,47.03601],[-63.6645,46.55001],[-62.9393,46.41587],[-62.01208,46.44314],[-62.50391,46.03339],[-62.8743,45.96818],[-64.1428,46.39265],[-64.39261,46.72747],[-64.01486,47.03601]]]],"type":"Feature","properties":{"name":"□□□□□□□□□□□□□□□□□□□□","bbox":[-171.79111,18.91619,-66.96466,71.357764],"geometry":{"type":"MultiPolygon","coordinates":[[[-122.84,49],[-120,49],[-117.03121,49],[-116.04818,49],[-113,49],[-110.05,49],[-107.05,49],[-104.04826,48.99986],[-100.65,49],[-97.22872,49.0007],[-95.15907,49],[-95.15609,49.38425],[-94.81758,49.38905],[-94.64,48.84],[-94.32914,48.67074],[-93.63087,48.60926],[-92.61,48.45],[-91.64,48.14],[-90.83,48.27],[-89.6,48.01],[-89.272917,48.019808],[-88.378114,48.302918],[-87.439793,47.94],[-86.461991,47.553338],[-85.652363,47.220219],[-84.87608,46.900083],[-84.779238,46.637102],[-84.543749,46.538684],[-84.6049,46.4396],[-84.3367,46.40877],[-84.14212,46.512226],[-84.091851,46.275419],[-83.890765,46.116927],[-83.616131,46.116927],[-83.469551,45.994686],[-83.592851,45.816894],[-82.550925,45.347517],[-82.337763,44.44],[-82.137642,43.571088],[-82.43,42.98],[-82.9,42.43],[-83.12,42.08],[-83.142,41.975681],[-83.02981,41.832796],[-82.690089,41.675105],[-82.439278,41.675105],[-81.277747,42.209026],[-80.247448,42.3662],[-78.939362,42.863611],[-78.92,42.965],[-79.01,43.27],[-79.171674,43.466339],[-78.72028,43.625089],[-77.73885,43.629056],[-76.820034,43.628784],[-76.5,44.018459],[-76.375,44.09631],[-75.31821,44.81645],[-74.867,45.00048],[-73.34783,45.00738],[-71.50506,45.0082],[-71.405,45.255],[-71.08482,45.30524],[-70.66,45.46],[-70.305,45.915],[-69.99997,46.69307],[-69.237216,47.447781],[-68.905,47.185],[-68.23444,47.35486],[-67.79046,47.06636],[-67.79134,45.70281],[-67.13741,45.13753],[-66.96466,44.8097],[-68.03252,44.3252],[-69.06,43.98],[-70.11617,43.68405],[-70.645476,43.090238],[-70.81489,42.8653],[-70.825,42.335],[-70.495,41.805],[-70.08,41.78],[-70.185,42.145],[-69.88497,41.92283],[-69.96503,41.63717],[-70.64,41.475],[-71.12039,41.49445],[-71.86,41.32],[-72.295,41.27],[-72.87643,41.22065],[-73.71,40.931102],[-72.24126,41.11948],[-71.945,40.93],[-73.345,40.63],[-73.982,40.628],[-73.952325,40.75075],[-74.25671,40.47351],[-73.96244,40.42763],[-74.17838,39.70926],[-74.90604,38.93954],[-74.98041,39.1964],[-75.20002,39.24845],[-75.52805,39.4985],[-75.32,38.96],[-75.071835,38.782032],[-75.05673,38.40412],[-75.37747,38.01551],[-75.94023,37.21689],[-76.03127,37.2566],[-75.72205,37.93705],[-76.23287,38.319215],[-76.35,39.15],[-76.542725,38.717615],[-76.32933,38.08326],[-76.989998,38.239992],[-76.30162,37.917945],[-76.25874,36.9664],[-75.9718,36.89726],[-75.86804,36.55125],[-75.72749,35.55074],[-76.36318,34.80854],[-77.397635,34.51201],[-78.05496,33.92547],[-78.55435,33.86133],[-79.06067,33.49395],[-79.20357,33.15839],[-80.301325,32.509355],[-80.86498,32.0333],[-81.33629,31.44049],[-81.49042,30.72999],[-81.31371,30.03552],[-80.98,29.18],[-80.535585,28.47213],[-80.53,28.04],[-80.056539,26.88],[-80.088015,26.205765],[-80.13156,25.816775],[-80.38103,25.20616],[-80.68,25.08],[-81.17213,25.20126],[-81.33,25.64],[-81.71,25.87],[-82.24,26.73],[-82.70515,27.49504],[-82.85526,27.88624],[-82.65,28.55],[-82.93,29.11],[-83.70959,29.93656],[-84.1,30.09],[-85.10882,29.63615],[-85.28784,29.68612],[-85.7731,30.15261],[-86.4,30.41],[-87.53036,30.27433],[-88.41782,30.3849],[-89.18049,30.31598],[-89.593831,30.159994],[-89.413735,29.89419],[-89.43,29.48864],[-89.21767,29.29108],[-89.40823,29.15961],[-89.77928,29.30714],[-90.15463,29.11743],[-90.880225,29.148535],[-91.626785,29.677],[-92.49906,29.5523],[-93.22637,29.78375],[-93.84842,29.71363],[-94.69,29.48],[-95.60026,28.73863],[-96.59404,28.30748],[-97.14,27.83],[-97.37,27.38],[-97.38,26.69],[-97.33,26.21],[-97.14,25.87],[-97.53,25.84],[-98.24,26.06],[-99.02,26.37],[-99.3,26.84],[-99.52,27.54],[-100.11,28.11],[-100.45584,28.69612],[-100.9576,29.38071],[-101.6624,29.7793],[-102.48,29.76],[-103.11,28.97],[-103.94,29.27],[-104.45697,29.57196],[-104.70575,30.12173],[-105.03737,30.64402],[-105.63159,31.08383],[-106.1429,31.39995],[-106.50759,31.75452],[-108.24,31.754854],[-108.24194,31.34222],[-109.035,31.34194],[-111.02361,31.33472],[-113.30498,32.03914],[-114.815,32.52528],[-114.72139,32.72083],[-115.99135,32.61239],[-117.12776,32.53534],[-117.295938,33.046225],[-117.944,33.621236],[-118.410602,33.740909],[-118.519895,34.027782],[-119.081,34.078],[-119.438841,34.348477],[-120.36778,34.44711],[-120.62286,34.60855],[-120.74433,35.15686],[-121.71457,36.16153],[-122.54747,37.55176],[-122.51201,37.78339],[-122.95319,38.11371],[-123.7272,38.95166],[-123.86517,39.76699],[-124.39807,40.3132],[-124.17886,41.14202],[-124.2137,41.99964],[-124.53284,42.76599],[-124.14214,43.70838],[-124.020535,44.615895],[-123.89893,45.52341],[-124.079635,46.86475],[-124.39567,47.72017],[-124.68721,48.184433],[-124.566101,48.379715],[-123.12,48.04],[-122.58736,47.096],[-122.34,47.36],[-122.5,48.18],[-122.84,49]]],[[-155.22452,19.99302],[-155.06226,19.8591],[-154.80741,19.50871],[-154.83147,19.45328],[-155.22217,19.23972],[-155.54211,19.08348],[-155.68817,18.91619],[-155.93665,19.05939],[-155.90806,19.33888],[-156.07347,19.70294],[-156.02368,19.81422],[-155.85008,19.97729],[-155.91907,20.17395],[-155.86108,20.26721],[-155.78505,20.2487],[-155.40214,20.07975]]],[[-155.99566,20.76404],[-156.07926,20.64397],[-156.41445,20.57241],[-156.58673,20.783],[-156.70167,20.8643],[-156.71055,20.92676],[-156.61258,21.01249],[-156.25711,20.91745],[-155.99566,20.76404]]],[[-156.75824,21.17684],[-156.78933,21.06873],[-157.32521,21.09777],[-157.25027,21.21958],[-156.75824,21.17684]]],[[-158.0252,21.71696],[-157.94161,21.65272],[-157.65283,21.32217],[-157.70703,21.26442],[-157.7786,21.27729],[-158.12667,21.31244],[-158.2538,21.53919],[-158.29625,21.57912],[-158.0252,21.71696]]],[[-159.36569,22.21494],[-159.34512,21.982],[-159.46372,21.88299],[-159.80051,22.06533],[-159.74877,22.1382],[-159.5962,22.23618],[-159.36569,22.21494]]],[[-166.467792,60.38417],[-165.67443,60.293607],[-165.579164,59.909987],[-166.19277,59.754441],[-166.848337,59.941406],[-167.455277,60.213069],[-166.467792,60.38417]]],[[-165.228729,57.968968],[-152.564791,57.901427],[-152.141147,57.591059],[-153.006314,57.115842],[-154.00509,56.734677],[-154.516403,56.992749],[-154.670993,57.461196],[-153.76278,57.816575],[-153.228729,57.968968]]],[[-140.985988,69.711998],[-140.986,69.712],[-140.9925,66.00003],[-140.99778,60.30639],[-140.013,60.27682],[-139.039,60],[-138.34089,59.56211],[-137.4525,58.9057],[-136.47972,59.46389],[-135.47583,59.78778],[-134.945,59.27056],[-134.27111,58.86111],[-133.35556,58.41028],[-132.7304,57.69289],[-131.70781,56.55212],[-130.00778,55.91583],[-129.98,55.285],[-130.53611,54.80278],[-130.536109,54.802754],[-130.53611,54.802753],[-131.085818,55.178906],[-131.967211,55.497776],[-132.250011,56.369996],[-133.539181,57.178887],[-134.078063,58.123068],[-135.03821,58.187715],[-136.628062,58.212209],[-137.800006,58.499995],[-139.867787,59.537762],[-140.825274,59.727517],[-142.574444,60.084447],[-143.958881,59.99918],[-145.925557,60.45861],[-147.114374,60.884656],[-148.224306,60.672989],[-148.018066,59.978329],[-148.570823,59.914173],[-149.727858,59.705658],[-150.608243,59.368211],[-151.716393,59.155821],[-151.859433,59.744984],[-151.409719,60.725803],[-150.346941,61.033588],[-150.621111,61.284425],[-151.895839,60.727198],[-152.57833,60.061657],[-154.019172,59.350279],[-153.287511,58.864728],[-154.232492,58.146374],[-155.307491,57.727795],[-156.308335,57.422774],[-156.556097,56.979985],[-158.117217,56.463608],[-158.433321,55.994154],[-159.603327,55.566686],[-160.28972,55.643581],[-161.223048,55.364735],[-162.237766,55.024187],[-163.069447,54.689737],[-164.785569,54.404173],[-164.942226,54.572225],[-163.84834,55.039431],[-162.870001,55.348043],[-161.804175,55.894986],[-160.563605,56.008055],[-160.07056,56.418055],[-158.684443,57.016675],[-158.461097,57.216921],[-157.72277,57.570001],[-157.550274,58.328326],[-157.041675,58.918885],[-158.194731,58.615802],[-158.517218,58.787781],[-159.058606,58.424186],[-159.711667,58.93139],[-159.981289,58.752549],[-160.355271,59.071123],[-161.355003,58.670838],[-161.968894,58.671665],[-162.054987,59.266925],[-161.874171,59.633621],[-162.518059,59.989724],[-163.818341,59.798056],[-164.662218,60.267484],[-165.346388,60.507496],[-165.350832,61.073895],[-166.121379,61.500019],[-165.734452,62.074997],[-164.919179,62.633076],[-164.562508,63.146378],[-163.753332,63.219449],[-163.067224,63.059459],[-162.260555,63.541936],[-161.53445,63.455817],[-160.772507,63.766108],[-160.958335,64.222799],[-161.518068,64.402788],[-160.777778,64.788604],[-161.391926,64.777235],[-162.453056,64.559445],[-162.757786,64.338605],[-163.546394,64.55916],[-164.96083,64.446945],[-166.425288,64.686672],[-166.845004,65.088896],[-168.11056,65.669997],[-166.705271,66.088318],[-164.47471,66.57666],[-163.652512,66.57666],[-163.788602,66.072087],[-161.677774,66.11612],[-162.489715,66.735565],[-163.71917,67.116395],[-164.430991,67.616338],[-165.390287,68.042772],[-166.764441,68.358877],[-166.204707,68.883031],[-164.430811,68.915535],[-163.168614,69.371115],[-162.930566,69.858062],[-161.908897,70.33333],[-160.934797,70.44769],[-159.039176,70.891642],[-158.119723,70.824721],[-156.580825,71.35776

```
[[-155.06779,71.147776],[[-154.344165,70.696409],[[-153.900006,70.889989],[[-152.210006,70.829992],[[-152.270002,70.600006],[-150.739992,70.430017],[[-149.720003,70.53001],[[-147.613362,70.214035],[[-145.68999,70.12001],[[-144.920011,69.989992],[[-143.589446,70.152514],[[-142.07251,69.851938],[[-140.985988,69.711998],[[-140.985988,69.711998]]],[[[-171.731657,63.782515],[[-171.114434,63.592191],[[-170.491112,63.694975],[[-169.682505,63.431116],[[-168.689439,63.297506],[[-168.771941,63.188598],[[-169.52944,62.976931],[[-170.290556,63.194438],[[-170.671386,63.375822],[[-171.553063,63.317789],[[-171.791111,63.405846],[[-171.731657,63.782515]]]]]]],{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[46.464646,40.662325,87.359975,38.852515],"geometry":{"type":"Polygon","coordinates":[[[87.35997,49.214981],[86.598776,48.549182],[85.768233,48.455751],[85.720484,47.452969],[85.16429,47.000956],[83.180484,47.330031],[82.458926,45.53965],[81.947071,45.317027],[79.966106,44.917517],[80.866206,43.180362],[80.18015,42.920068],[80.25999,42.349999],[79.643645,42.496683],[79.142177,42.856092],[77.658392,42.906686],[76.000354,42.988022],[75.636965,42.8779],[74.212866,43.298339],[73.645304,43.091272],[73.489758,42.500894],[71.844638,42.845395],[71.186281,42.704293],[70.962315,42.266154],[70.388965,42.081308],[69.070027,41.384244],[68.632483,40.668681],[68.259896,40.662325],[67.985856,41.135991],[66.714047,41.168444],[66.510649,41.987644],[66.023392,41.994646],[66.098012,42.99766],[64.900824,43.728081],[63.185787,43.650075],[62.0133,43.504477],[61.05832,44.405817],[60.239972,44.784037],[58.689989,45.500014],[58.503127,45.586804],[55.928917,44.995858],[55.968191,41.308642],[55.455251,41.259859],[54.755345,42.043971],[54.079418,42.324109],[52.944293,42.116034],[52.50246,41.783316],[52.446339,42.027151],[52.692112,42.443895],[52.501426,42.792298],[51.342427,43.132975],[50.891292,44.031034],[50.339129,44.284016],[50.305643,44.609836],[51.278503,44.514854],[51.316899,45.245998],[52.16739,45.408391],[53.040876,45.259047],[53.220866,46.234644],[53.042737,46.853006],[52.042023,46.804637],[51.191945,47.048705],[50.034083,46.60899],[49.10116,46.39933],[48.59325,46.56104],[48.694734,47.075628],[48.057525,47.74377],[47.31524,47.71585],[46.466446,48.394152],[47.043672,49.152039],[46.751596,49.356006],[47.54948,50.454698],[48.577841,49.87476],[48.702382,50.605128],[50.766648,51.692762],[52.328724,51.718652],[54.532878,51.02624],[55.71694,50.62171],[56.77798,51.04355],[58.36332,51.06364],[59.642282,50.545442],[59.932807,50.842194],[61.337424,50.79907],[61.588003,51.272659],[59.967534,51.96042],[60.927269,52.447548],[60.739993,52.719986],[61.699986,52.979996],[60.978066,53.664993],[61.4366,54.00625],[65.178534,54.354228],[65.66687,54.60125],[68.1691,54.970392],[69.068167,55.38525],[70.865267,55.169734],[71.180131,54.133285],[72.22415,54.376655],[73.508516,54.035617],[73.425679,53.48981],[74.38482,53.54685],[76.8911,54.490524],[76.525179,54.177003],[77.800916,53.404415],[80.03556,50.864751],[80.56844,75.51.388336],[81.945986,50.812196],[83.383004,51.069183],[83.935115,50.889246],[84.416377,50.3114],[85.11556,50.117303],[85.541279,49.692595],[86.829357,49.826675],[87.35997,49.214981]]]]]]],{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":bbox:"[55.928917,37.144994,73.055417,45.586804],"geometry":{"type":"Polygon","coordinates":[[[55.968191,41.308642],[55.928917,44.995858],[58.503127,45.586804],[58.689989,45.500014],[60.239972,44.784037],[61.05832,44.405817],[62.0133,43.504477],[63.185787,43.650075],[64.900824,43.728081],[66.098012,42.99766],[66.023392,41.994646],[66.510649,41.987644],[66.714047,41.168444],[67.985856,41.135991],[68.259896,40.662325],[68.632483,40.668681],[69.070027,41.384244],[70.388965,42.081308],[70.962315,42.266154],[71.186281,42.704293],[71.844638,42.845395],[72.22415,42.266154],[73.425679,42.500894],[74.38482,42.845395],[76.8911,42.845395],[77.800916,42.845395],[80.03556,42.845395],[80.56844,42.845395],[81.945986,42.845395],[83.383004,42.845395],[83.935115,42.845395],[84.416377,42.845395],[85.11556,42.845395],[85.541279,42.845395],[86.829357,42.845395],[87.35997,42.845395]]]]]]],{"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":bbox:"[141.00021,-10.652476,156.019965,-2.500002],"geometry":{"type":"MultiPolygon","coordinates":[[[141.00021,-2.600151],[142.735247,-3.289153],[144.583971,-3.861418],[145.27318,-4.373738],[145.829786,-4.876498],[145.981922,-5.465609],[147.648072,-6.083659],[147.891108,-6.614015],[146.970905,-6.721657],[147.191874,-7.388024],[148.084636,-8.044108],[148.734105,-9.104664],[149.306835,-9.071436],[149.266631,-9.514406],[150.038728,-9.684318],[149.738798,-9.872937],[150.801628,-10.239687],[150.690575,-10.582713],[150.028393,-10.652476],[149.78231,-10.39326
```

6906, -5.379878], [119.653606, -4.459417], [119.498835, -3.494412], [119.078344, -3.487022], [118.767769, -2.801999], [119.180974, -2.147104], [119.323394, -1.353147], [119.825999, 0.154254], [120.035702, 0.566477], [120.885779, 1.309223], [121.666817, 1.013944], [122.927567, 0.875192]], [[120.295014, -10.25865], [118.967808, -9.557969], [119.90031, -9.36134], [120.425756, -9.665921], [120.775502, -9.969675], [120.715609, -10.239581], [120.295014, -10.25865]], [[121.341669, -8.53674], [122.007365, -8.46062], [122.903537, -8.094234], [122.756983, -8.649808], [121.254491, -8.933666], [119.924391, -8.810418], [119.920929, -8.444859], [120.715092, -8.236965], [121.341669, -8.53674]], [[118.260616, -8.362383], [118.87846, -8.280683], [119.126507, -8.705825], [117.970402, -8.906639], [117.277731, -9.040895], [116.740141, -9.032937], [117.083737, -8.457158], [117.632024, -8.449303], [117.900018, -8.095681], [118.260616, -8.362383]], [[108.486846, -6.421985], [108.623479, -6.777674], [110.539227, -6.877358], [110.759576, -6.465186], [112.614811, -6.946036], [112.978768, -7.594213], [114.478935, -7.776528], [115.705527, -8.370807], [114.564511, -8.751817], [113.464734, -8.348947], [112.559672, -8.376181], [111.522061, -8.302129], [110.58615, -8.122605], [109.427667, -7.740664], [108.693655, -7.6416], [108.277763, -7.766657], [106.454102, -7.3549], [106.280624, -6.9249], [105.365486, -6.851416], [106.051646, -5.895919], [107.265009, -5.954985], [108.072091, -6.345762], [108.486846, -6.421985]], [[104.369991, -1.084843], [104.53949, -1.782372], [104.887893, -2.340425], [105.622111, -2.428844], [106.108593, -3.061777], [105.857446, -4.305525], [105.817655, -5.852356], [104.710384, -5.873285], [103.868213, -5.037315], [102.584261, -4.220259], [102.156173, -3.614146], [101.399113, -2.799777], [100.902503, -2.050262], [100.141981, -0.650348], [99.26374, 0.183142], [98.970011, 1.042882], [98.601351, 1.823507], [97.699598, 2.453184], [97.176942, 3.308791], [96.424017, 3.86886], [95.380876, 4.970782], [95.293026, 5.479821], [95.936863, 5.439513], [97.484882, 5.246321], [98.369169, 4.26837], [99.142559, 3.59035], [99.693998, 3.174329], [100.641434, 2.099381], [101.658012, 2.083697], [102.498271, 1.3987], [103.07684, 0.561361], [103.838396, 0.104542], [103.437645, -0.711946], [104.010789, -1.059212], [104.369991, -1.084843]]], {"type": "Feature", "properties": {"name": "多边形", "bbox": [-73.415436, -55.25, 53.628349, -21.83231], "geometry": {"type": "MultiPolygon", "coordinates": [[[-68.63401, -52.63637], [-68.25, -53.1], [-67.75, -53.85], [-66.45, -54.45], [-65.05, -54.7], [-65.5, -55.2], [-66.45, -55.25], [-66.95992, -54.89681], [-67.56244, -54.87001], [-68.63335, -54.8695], [-68.63401, -52.63637], [[-57.625133, -30.216295], [-57.874937, -31.016556], [-58.14244, -32.044504], [-58.132648, -33.040567], [-58.349611, -33.263189], [-58.427074, -33.909454], [-58.495442, -34.43149], [-57.22583, -35.288027], [-57.362359, -35.97739], [-56.737487, -36.413126], [-56.788285, -36.901572], [-57.749157, -38.183871], [-59.231857, -38.72022], [-61.237445, -38.928425], [-62.335957, -38.827707], [-62.125763, -39.424105], [-62.330531, -40.172586], [-62.145994, -40.676897], [-62.745803, -41.028761], [-63.770495, -41.166789], [-64.732609, -40.802677], [-65.118035, -41.064315], [-64.978561, -42.058001], [-64.303408, -42.359016], [-63.755948, -42.043687], [-63.458059, -42.563138], [-64.378804, -42.873558], [-65.181804, -43.495381], [-65.328823, -44.501366], [-65.565269, -45.036786], [-66.509966, -45.039628], [-67.293794, -45.551896], [-67.580546, -46.301773], [-66.597066, -47.033925], [-65.641027, -47.236135], [-65.985088, -48.133289], [-67.166179, -48.697337], [-67.816088, -49.869669], [-68.728745, -50.264218], [-69.138539, -50.73251], [-68.815561, -51.771104], [-68.149995, -52.349983], [-68.571545, -52.299444], [-69.498362, -52.142761], [-71.914804, -52.009222], [-72.329404, -51.425956], [-72.309974, -50.67701], [-72.975747, -50.74145], [-73.328051, -50.378785], [-73.415436, -49.318436], [-72.648247, -48.878618], [-72.331161, -48.244238], [-72.447355, -47.738533], [-71.917258, -46.884838], [-71.552009, -45.560733], [-71.659316, -44.973689], [-71.222779, -44.784243], [-71.329801, -44.407522], [-71.793623, -44.207172], [-71.464056, -43.787611], [-71.915424, -43.408565], [-72.148898, -42.254888], [-71.746804, -42.051386], [-71.915734, -40.832339], [-71.680761, -39.808164], [-71.413517, -38.916022], [-70.814664, -38.552995], [-71.118625, -37.576827], [-71.121881, -36.658124], [-70.364769, -36.005089], [-70.388049, -35.169688], [-69.817309, -34.193571], [-69.814777, -33.273886], [-70.074399, -33.09121], [-70.535069, -31.36501], [-69.919008, -30.336339], [-70.01355, -29.367923], [-69.65613, -28.459141], [-69.001235, -27.521214], [-68.295542, -26.89934], [-68.5948, -26.506909], [-68.386001, -26.185016], [-68.417653, -24.518555], [-67.328443, -24.025303], [-66.985234, -22.986349], [-67.106674, -22.735925], [-66.273339, -21.83231], [-64.964892, -22.075862], [-64.377021, -22.798091], [-63.986838, -21.993444], [-62.846468, -22.034985], [-62.685057, -22.249029], [-60.846565, -22.880713], [-60.028966, -24.032796], [-58.807128, -24.771459], [-57.777217, -25.16234], [-57.63366, -25.603657], [-58.618174, -27.123719], [-57.60976, -27.395899], [-56.486702, -27.548499], [-55.695846, -27.387837], [-54.788795, -26.621786], [-54.625291, -25.739255], [-54.13005, -25.547639], [-53.628349, -26.124865], [-53.648735, -26.923473], [-54.490725, -27.474757], [-55.162286, -27.881915], [-56.2909, -28.852761], [-57.625133, -30.216295]]]]}, {"type": "Feature", "properties": {"name": "多边形", "bbox": [-75.644395, -55.61183, -66.95992, -17.580012], "geometry": {"type": "MultiPolygon", "coordinates": [[[-68.63401, -52.63637], [-68.63335, -54.8695], [-67.56244, -54.87001], [-66.95992, -54.89681], [-67.29103, -55.30124], [-68.14863, -55.61183], [-68.639991, -55.580018], [-69.2321, -55.49906], [-69.95809, -55.19843], [-71.00568, -55.05383], [-72.2639, -54.49514], [-73.2852, -53.95752], [-74.66253, -52.83749], [-73.8381, -53.04743], [-72.43418, -53.7154], [-71.10773, -54.07433], [-70.59178, -53.61583], [-70.26748, -52.93123], [-69.34565, -52.5183], [-68.63401, -52.63637]], [[-69.590424, -17.580012], [-69.100247, -18.260123], [-68.966818, -18.981683], [-68.442225, -19.505068], [-68.757167, -20.372658], [-68.219913, -21.494347], [-67.82818, -22.872919], [-67.106674, -22.735925], [-66.985234, -22.986349], [-67.328443, -24.025303], [-68.417653, -24.518555], [-68.386001, -26.185016], [-68.5948, -26.506909], [-68.295542, -26.89934], [-69.001235, -27.521214], [-69.65613, -28.459141], [-70.01355, -29.367923], [-69.919008, -30.336339], [-70.535069, -31.36501], [-70.074399, -33.09121], [-69.814777, -33.273886], [-69.817309, -34.193571], [-70.388049, -35.169688], [-70.364769, -36.005089], [-71.121881, -36.658124], [-71.118625, -37.576827], [-70.814664, -38.552995], [-71.413517, -38.916022], [-71.680761, -39.808164], [-71.915734, -40.832339], [-71.746804, -42.051386], [-72.148898, -42.254888], [-71.915424, -43.408565], [-71.464056, -43.787611], [-71.793623, -44.207172], [-71.329801, -44.407522], [-71.222779, -44.784243], [-71.659316, -44.973689], [-71.552009, -45.560733], [-71.917258, -46.884838], [-72.447355, -47.738533], [-72.331161, -48.244238], [-72.648247, -48.878618], [-73.415436, -49.318436], [-73.328051, -50.378785], [-72.975747, -50.74145], [-72.309974, -50.67701], [-72.329404, -51.425956], [-71.914804, -52.009022], [-69.498362, -52.142761], [-68.571545, -52.299444], [-69.461284, -52.291951], [-69.94278, -52.437931], [-70.845102, -52.899201], [-71.006332, -53.833252], [-71.429795, -53.856455], [-72.557943, -53.53141], [-73.702757, -52.835069], [-73.702757, -52.83507], [-74.946763, -52.262754], [-75.260026, -51.629355], [-74.976632, -51.043396], [-75.479754, -50.378372], [-75.608015, -48.673773], [-75.18277, -47.711919], [-74.126581, -46.939253], [-75.644395, -46.647643], [-74.692154, -45.763976], [-74.351709, -44.103044], [-73.240356, -44.454961], [-72.717804, -42.383356], [-73.3889, -42.17532], [-73.701336, -43.365776], [-74.331943, -43.224958], [-74.017957, -41.794813], [-73.677099, -39.942213], [-73.217593, -39.258689], [-73.505559, -38.282883], [-73.588061, -37.156285], [-73.166717, -37.12378], [-72.553137, -35.50884], [-71.861732, -33.909093], [-71.43845, -32.418899], [-71.668721, -30.920645], [-71.370083, -30.095682], [-71.489894, -28.861442], [-70.905124, -27.64038], [-70.724954, -25.705924], [-70.403966, -23.628997], [-70.091246, -21.393319], [-70.16442, -19.756468], [-70.372572, -18.347975], [-69.858444, -18.092694], [-69.504024, -17.580012]]]]}, {"type": "Feature", "properties": {"name": "多边形", "bbox": [12.182337, -12.257227, 31.174149, 5.256088], "geometry": {"type": "Polygon", "coordinates": [[29.339998, -4.499983], [29.519987, -5.419979], [29.419993, -5.939999], [29.620032, -6.520015], [30.199997, -7.079981], [30.740015, -8.340007], [30.74001, -8.340006], [30.346086, -8.238257], [29.002912, -8.407032], [28.734867, -8.526559], [28.449871, -9.164918], [28.673682, -9.605925], [28.49607, -10.789884], [28.372253, -11.793647], [28.642417, -11.971569], [29.341548, -12.360744], [29.616001, -12.178895], [29.699614, -13.257227], [28.934286, -13.248958], [28.523562, -12.698604], [28.155109, -12.272481], [27.388799, -12.132747], [27.16442, -11.608748], [26.553088, -11.92444], [25.75231, -11.784965], [25.418118, -11.330936], [24.78317, -11.238694], [24.314516, -11.262826], [24.257155, -10.951993], [23.912215, -10.926826], [23.456791, -10.867863], [22.837345, -11.017622], [22.402798, -10.993075], [22.155268, -11.084801], [22.208753, -9.894796], [21.875182, -9.523708], [21.801801, -8.908707], [21.949131, -8.305901], [21.746456, -7.920085], [21.728111, -7.290872], [20.514748, -7.299606], [20.601823, -6.939318], [20.091622, -6.94309], [20.037723, -7.116361], [19.417502, -7.155429], [19.166613, -7.738184], [19.016752, -7.988246], [18.464176, -7.847014], [18.134222, -7.987678], [17.47297, -8.068551], [17.089996, -7.545689], [16.860191, -7.222298], [16.57318, -6.622645], [16.326528, -5.87747], [13.375597, -5.864241], [13.024869, -5.984389], [12.735171, -5.965682], [12.322432, -6.100092], [12.182337, -5.789931], [12.436688, -5.684304], [12.468004, -5.248362], [12.631612, -4.991271], [12.995517, -4.781103], [13.25824, -4.882957], [13.600235, -4.500138], [14.144956, -4.510009], [14.209035, -4.793092], [14.582604, -4.970239], [15.170992, -4.343507], [15.75354, -3.855165], [16.00629, -3.535133], [15.972803, -2.712392], [16.407092, -1.740927], [16.865307, -1.225816], [17.523716, -0.74383], [17.638645, -0.424832], [17.663553, -0.058084], [17.82654, 0.288923], [17.774192, 0.855659], [17.898835, 1.741832], [18.094276, 2.365722], [18.393792, 2.900443], [18.453665, 3.504386], [18.542982, 4.201785], [18.932312, 4.709506], [19.467784, 5.031528], [20.290679, 4.691678], [20.927591, 4.322786], [21.659123, 4.224342], [22.405124, 4.02916], [22.704124, 4.633051], [22.84148, 4.710126], [23.297214, 4.609693], [24.410531, 5.108784], [24.805029,

4.897247],[25.128833,4.927245],[25.278798,5.170408],[25.650455,5.256088],[26.402761,5.150875],[27.044065,5.127853],[27.374226,5.233944],[27.979977,4.408413],[28.428994,4.287155],[28.696678,4.455077],[29.159078,4.389267],[29.715995,4.600805],[29.9535,4.173699],[30.833852,3.509172],[30.83386,3.509166],[30.773347,2.339883],[31.174149,2.204465],[30.85267,1.849396],[30.468508,1.583805],[30.086154,1.062313],[29.875779,0.59738],[29.819503,-0.20531],[29.587838,-0.587406],[29.579466,-1.341313],[29.291887,-1.620056],[29.254835,-2.21511],[29.117479,-2.292211],[29.024926,-2.839258],[29.276384,-3.293907],[29.339998,-4.499833]]],{"type":"Feature","properties":{"name":"□□□□□□","bbox":[40.98105,-1.683255,1.13387,12.02464]},"geometry":{"type":"Polygon","coordinates":[[[41.58513,-1.68325],[40.993,-0.85829],[40.98105,2.78452],[41.855083,3.918912],[42.12861,4.23413],[42.76967,4.25259],[43.66087,4.95755],[44.9636,5.00162],[47.78942,8.003],[48.486736,8.837626],[48.93813,9.451749],[48.938233,9.9735],[48.938491,10.982327],[48.942005,11.394266],[48.948205,11.410617],[48.948205,11.410617],[49.26776,11.43033],[49.72862,11.5789],[50.25878,11.67957],[50.73202,12.0219],[51.1112,12.02464],[51.13387,11.74815],[51.04153,11.16651],[51.04531,10.6409],[50.83418,10.27972],[50.55239,9.19874],[50.07092,8.08173],[49.4527,6.80466],[48.59455,5.33911],[47.74079,4.2194],[46.56476,2.85529],[45.56399,2.04576],[44.06815,1.05283],[43.13597,0.2922],[42.04157,-0.91916],[41.81095,-1.44647],[41.58513,-1.68325]]]],{"type":"Feature","properties":{"name":"□□□□","bbox":[33.893569,-4.6777,41.855083,5.506]},"geometry":{"type":"Polygon","coordinates":[[[39.20222,-4.67677],[37.7669,-3.67712],[37.69869,-3.09699],[34.07262,-1.05982],[33.903711,-0.95],[33.893569,0.109814],[34.18,0.515],[34.6721,1.17694],[35.03599,1.90584],[34.59607,3.05374],[34.47913,3.5556],[34.005,4.249885],[34.620196,4.847123],[35.298007,5.506],[35.817448,5.338232],[35.817448,4.776966],[36.159079,4.447864],[36.855093,4.447864],[38.120915,3.598605],[38.43697,3.58851],[38.67114,3.61607],[38.89251,3.50074],[39.559384,3.42206],[39.85494,3.83879],[40.76848,4.25702],[41.1718,3.91909],[41.855083,3.918912],[40.98105,2.78452],[40.993,-0.85829],[41.58513,-1.68325],[40.88477,-2.08255],[40.63785,-2.49979],[40.26304,-2.57309],[40.12119,-3.27768],[39.80006,-3.68116],[39.60489,-4.34653],[39.20222,-4.67677]]]],{"type":"Feature","properties":{"name":"□□□□□","bbox":21.93681,8.229188,38.41009,22},"geometry":{"type":"Polygon","coordinates":[[[24.567369,8.229188],[23.805813,8.666319],[23.459013,8.954286],[23.394779,9.265068],[23.55725,9.681218],[23.554304,10.089255],[22.977544,10.714463],[22.864165,11.142395],[22.87622,11.38461],[22.50869,11.67936],[22.49762,12.26024],[22.28801,12.64605],[21.93681,12.58818],[22.03759,12.95546],[22.29658,13.37232],[22.18329,13.78648],[22.51202,14.09318],[22.30351,14.32682],[22.56795,14.94429],[23.02459,15.68072],[23.88689,15.61084],[23.83766,19.58047],[23.85,20],[25,20.00304],[25,22],[29.02,22],[32.9,22],[36.86623,22],[37.18872,21.01885],[36.96941,20.83744],[37.1147,19.80796],[37.48179,18.61409],[37.86276,18.36786],[37.841009,17.998307],[37.904,17.42754],[37.16747,17.26314],[36.85253,16.95655],[36.75389,16.29186],[36.32322,14.82249],[36.42951,14.42211],[36.27022,13.56333],[35.86363,12.57828],[35.26049,12.08286],[34.83163,11.31896],[34.73115,10.91017],[34.25745,10.63009],[33.96162,9.58358],[33.97498,8.68456],[33.963393,9.464285],[33.824963,9.484061],[33.842131,9.981915],[33.721959,10.325262],[33.206938,10.720112],[33.086766,11.441141],[33.206938,12.17938],[32.743419,12.248008],[32.67475,12.024832],[32.073892,11.97333],[32.314235,11.681484],[32.400072,11.080626],[31.850716,10.531271],[31.352862,9.810241],[30.837841,9.707237],[29.996639,10.290927],[29.618957,10.084919],[29.515953,9.793074],[29.000932,9.604232],[28.966597,9.398224],[27.97089,9.398224],[27.833551,9.604232],[27.112521,9.638567],[26.752006,9.466893],[26.477328,9.55273],[25.962307,10.136421],[25.790633,10.411099],[25.069604,10.27376],[24.794926,9.810241],[24.537415,8.917538],[24.194068,8.728696],[23.88698,8.61973],[24.567369,8.229188]]]],{"type":"Feature","properties":{"name":"□□□□","bbox":13.540394,7.412925,23.88689,23.40972},"geometry":{"type":"Polygon","coordinates":[[[23.83766,19.58047],[23.88689,15.61084],[23.02459,15.68072],[22.56795,14.94429],[22.30351,14.32682],[22.51202,14.09318],[22.18329,13.78648],[22.29658,13.37232],[22.03759,12.95546],[21.93681,12.58818],[22.8801,12.64605],[22.49762,12.26024],[22.50869,11.67936],[22.87622,11.38461],[22.864165,11.142395],[22.231129,10.971889],[21.723822,10.567056],[21.000868,9.475985],[20.059685,9.012706],[19.094008,9.074847],[18.81201,9.982915],[18.911022,8.630895],[18.389555,8.281304],[17.96493,7.890914],[16.705988,7.508328],[16.456185,7.734774],[16.290562,7.754307],[16.106232,7.497088],[15.27946,7.421925],[15.436092,7.692812],[15.120866,8.38215],[14.979996,8.796104],[14.544467,8.965861],[13.954218,9.549495],[14.171466,10.021378],[14.627201,9.920919],[14.909354,9.992129],[15.467873,9.982337],[14.923565,10.89132],[14.960152,11.555774],[14.89336,12.21905],[14.495787,12.859396],[14.595781,13.330427],[13.954477,13.353449],[13.95669,13.996691],[13.540394,14.367134],[13.97217,15.68437],[15.247731,16.627306],[15.300441,17.92795],[15.685741,19.95718],[15.903247,20.387619],[15.487148,20.730415],[15.47106,21.04845],[15.096888,21.308519],[14.8513,22.86295],[15.86085,23.40972],[19.84926,21.49509],[23.83766,19.58047]]]],{"type":"Feature","properties":{"name":"□□□□□□","bbox":13.540394,7.412925,23.88689,23.40972},"geometry":{"type":"Polygon","coordinates":[[[-71.712361,19.714456],[-71.624873,19.169838],[-71.701303,18.785417],[-71.945112,18.6169],[-71.687738,18.31666],[-71.708305,18.044997],[-72.372476,18.214961],[-72.844411,18.145611],[-73.445555,18.217906],[-73.922433,18.030993],[-74.458034,18.34255],[-74.369925,18.664908],[-73.449541,18.526053],[-72.694937,18.445799],[-72.334882,18.668422],[-72.79165,19.101625],[-72.784105,19.483591],[-73.415022,19.639551],[-73.189791,19.915684],[-72.579673,19.871501],[-71.712361,19.714456]]]],{"type":"Feature","properties":{"name":"□□□□□□","bbox":13.540394,7.412925,23.88689,23.40972},"geometry":{"type":"Polygon","coordinates":[[[-71.708305,18.044997],[-71.687738,18.31666],[-71.945112,18.6169],[-71.701303,18.785417],[-71.624873,19.169838],[-71.712361,19.714456],[-71.587304,19.884911],[-70.806706,19.880286],[-70.214365,19.622885],[-69.950815,19.648],[-69.76925,19.293267],[-69.22126,19.313214],[-69.254346,19.015196],[-68.809442,18.979074],[-68.317943,18.612198],[-68.689316,18.205142],[-69.16494,18.422648],[-69.623988,18.380713],[-69.952934,18.428307],[-70.133233,18.245915],[-70.517137,18.184291],[-70.669298,18.426886],[-70.99995,18.283329],[-71.40021,17.598564],[-71.657662,17.757573],[-71.708305,18.044997]]]]},{"type":"Feature","properties":{"name":"□□□□□□","bbox":180.41151416,180.812504},"geometry":{"type":"MultiPolygon","coordinates":[[[[178.7253,71.0988],[180,71.515714],[180,70.832199],[178.903425,70.78114],[178.7253,71.0988]]],[[49.10116,46.39933],[48.64541,45.80629],[47.67591,45.64149],[46.68201,44.6092],[47.59094,43.66016],[47.49252,42.98658],[48.58437,41.80888],[48.584353,41.808869],[47.987283,41.405819],[47.815666,41.15416],[47.373315,41.219732],[46.686071,41.827137],[46.404951,41.860675],[45.7764,42.09244],[45.470279,42.502781],[44.537623,42.711993],[43.93121,42.55496],[43.75599,42.74083],[42.3944,43.2703],[40.92219,43.38215],[40.076965,43.553104],[39.955009,43.434998],[38.68,44.28],[37.53912,44.65721],[36.67546,45.24469],[37.40317,45.40451],[38.23295,46.24087],[37.67372,46.63657],[39.14767,47.04475],[39.1212,47.26336],[38.223538,47.10219],[38.255112,47.5464],[38.77057,47.82562],[39.738278,47.898937],[39.89562,48.23241],[39.67465,48.78382],[40.080789,49.30743],[40.06904,49.60105],[38.594988,49.926462],[38.010631,49.915662],[37.39346,50.383953],[36.626168,50.225591],[35.356116,50.577197],[35.37791,50.77394],[35.022183,51.207572],[34.224816,51.255993],[34.141978,51.566413],[34.391731,51.768882],[33.7527,52.335075],[32.715761,52.238465],[32.412058,52.288695],[32.15944,52.06125],[31.785992,52.101678],[31.78597,52.10168],[31.540018,52.742052],[31.305201,53.073996],[31.49764,53.16743],[32.304519,53.132726],[32.693643,53.351421],[32.405599,53.618045],[31.731273,53.794029],[31.791424,53.974639],[31.384472,54.157056],[30.757534,54.811771],[30.971836,55.081548],[30.873909,55.550976],[29.896294,55.789463],[29.371572,55.670091],[29.229513,55.918344],[28.176709,56.16913],[27.855282,56.759326],[27.770016,57.244258],[27.288185,57.474528],[27.716686,57.791899],[27.42015,58.72457],[28.131699,59.300825],[27.98112,59.47537],[27.981127,59.475373],[29.1177,60.02805],[28.070002,60.503519],[28.07,60.50352],[30.211107,61.780028],[31.139991,62.357693],[31.516092,62.867687],[30.035872,63.552814],[30.444685,64.204453],[29.54443,64.948672],[30.21765,65.80598],[29.054589,66.944286],[29.977426,67.698297],[28.445944,68.364613],[28.59193,69.064777],[29.39955,69.15692],[31.101402,69.558101],[31.10108,69.55811],[32.13272,69.90595],[33.77547,69.30142],[36.51396,69.06342],[40.29234,67.9324],[41.05987,67.45713],[41.12595,66.79158],[40.01583,66.26618],[38.38295,65.99953],[33.91871,66.75961],[33.18444,66.63253],[34.81477,65.90015],[34.878574,65.436213],[34.94391,64.41437],[36.23129,64.10945],[37.01273,63.84983],[37.14197,64.33471],[36.539579,64.76446],[37.17604,65.14322],[39.59345,64.52079],[40.4356,64.76446],[39.7626,65.49682],[42.09309,66.47623],[43.01604,66.41858],[43.94975,66.06908],[44.53226,66.75634],[43.69839,67.35245],[44.18795,67.95051],[43.45282,68.57079],[46.25,68.25],[46.82134,67.68997],[45.55517,67.56652],[45.56202,67.01005],[46.34915,66.66767],[47.89416,66.88455],[48.13876,67.52238],[50.22766,67.99867],[53.71743,68.85738],[54.47171,68.80815],[53.48582,68.20131],[54.72628,68.09702],[55.44268,68.43866],[57.31702,68.46628],[58.802,68.88082],[59.94142,68.27844],[61.07784,68.94069],[60.03,69.52],[60.55,69.85],[63.504,69.54739],[64.888115,69.234835],[68.51216,68.09233],[69.18068,68.61563],[68.16444,69.14436],[68.13522,69.35649],[66.93008,69.45461],[67.25976,69.92873],[66.72492,70.70889],[66.69466,71.02897],[68.54006,71.9345],[69.19636,72.



84336], [69.94, 73.04], [72.58754, 72.77629], [72.79603, 72.22006], [71.84811, 71.40898], [72.47011, 71.09019], [72.79188, 70.39114], [72.5647, 69.02085], [73.66787, 68.4079], [73.2387, 67.7404], [71.28, 66.32], [72.42301, 66.17267], [72.82077, 66.53267], [73.92099, 66.78946], [74.18651, 67.28429], [75.052, 67.76047], [74.46926, 68.32899], [74.93584, 68.98918], [73.84236, 69.07146], [73.60187, 69.62763], [74.3998, 70.63175], [73.1011, 71.44717], [74.89082, 72.12119], [74.65926, 72.83227], [75.15801, 72.85497], [75.68351, 72.30056], [75.28898, 71.33556], [76.35911, 71.15287], [75.90313, 71.87401], [77.57665, 72.26717], [79.65202, 72.32011], [81.5, 71.75], [80.61071, 72.52855], [80.51109, 73.6482], [82.25, 73.85], [84.65526, 73.80591], [86.8223, 73.93688], [86.00956, 74.45967], [87.1668, 75.11643], [88.31571, 75.14393], [90.26, 75.64], [92.90058, 75.77333], [93.23421, 76.0472], [95.86, 76.14], [96.67821, 75.91548], [98.92254, 76.44689], [100.75967, 76.43028], [101.03532, 76.86189], [101.99084, 77.28754], [104.3516, 77.69792], [106.06664, 77.37389], [104.705, 77.1274], [106.97013, 76.97419], [107.24, 76.48], [108.1538, 76.72335], [111.07726, 76.71], [113.33151, 76.22224], [114.13417, 75.84764], [113.88539, 75.32779], [112.77918, 75.03186], [110.15125, 74.47673], [109.4, 74.18], [110.64, 74.04], [112.11919, 73.78774], [113.01954, 73.97693], [113.52958, 73.33505], [113.96881, 73.59488], [115.56782, 73.75285], [118.77633, 73.58772], [119.02, 73.12], [123.20066, 72.97122], [123.25777, 73.73503], [125.38, 73.56], [126.97644, 73.56549], [128.59126, 73.03871], [129.05157, 72.39872], [128.46, 71.98], [129.71599, 71.19304], [131.28858, 70.78699], [132.2535, 71.8363], [133.85766, 71.38642], [135.56193, 71.65525], [137.49755, 71.34763], [138.23409, 71.62803], [139.86983, 71.48783], [139.14791, 72.41619], [140.46817, 72.84941], [149.5, 72.2], [150.35118, 71.60643], [152.9689, 70.84222], [157.00688, 71.03141], [158.99779, 70.86672], [159.83031, 70.45324], [159.70866, 69.72198], [160.94053, 69.43728], [162.27907, 69.64204], [164.05248, 69.66823], [165.94037, 69.47199], [167.83567, 69.58269], [169.57763, 68.6938], [170.81688, 69.01363], [170.0082, 69.65276], [170.45345, 70.09703], [173.64391, 69.81743], [175.72403, 69.87725], [178.6, 69.4], [180.68, 963636], [180.64, 979709], [179.99281, 64.97433], [178.7072, 64.53493], [177.41128, 64.60821], [178.313, 64.07593], [178.90825, 63.25197], [179.37034, 62.98262], [179.48636, 62.56894], [179.22825, 62.3041], [177.3643, 62.5219], [174.56929, 61.76915], [173.68013, 61.65261], [172.15, 60.95], [170.6985, 60.33618], [170.33085, 59.88177], [168.90046, 60.57355], [166.29498, 59.78855], [165.84, 60.16], [164.87674, 59.7316], [163.53929, 59.86871], [163.21711, 59.21101], [162.01733, 58.24328], [162.05297, 57.83912], [163.19191, 57.61503], [163.05794, 56.15924], [162.12958, 56.12219], [161.70146, 55.28568], [162.11749, 54.85514], [160.36877, 54.34433], [160.02173, 53.20257], [158.53094, 52.95868], [158.23118, 51.94269], [156.78979, 51.01105], [156.42, 51.7], [155.9918, 53.15895], [155.43366, 55.38103], [155.91442, 56.76792], [156.75815, 57.3647], [156.81035, 57.83204], [158.36433, 58.05575], [160.15064, 59.31477], [161.87204, 60.343], [163.66969, 61.1409], [164.47355, 62.55061], [163.25842, 62.46627], [162.65791, 61.6425], [160.12148, 60.54423], [159.30232, 61.77396], [156.72068, 61.43442], [154.21806, 59.75818], [155.04375, 59.14495], [152.81185, 58.88385], [151.26573, 58.78089], [151.33815, 59.50396], [149.78371, 59.65573], [148.54481, 59.16448], [145.48722, 59.33637], [142.19782, 59.03998], [138.95848, 57.08805], [135.12619, 54.72959], [136.70171, 54.60355], [137.19342, 53.97732], [138.1647, 53.75501], [138.80463, 54.25455], [139.90151, 54.18968], [141.34531, 53.08957], [141.37923, 52.23877], [140.59742, 51.23967], [140.51308, 50.04553], [140.06193, 48.44671], [138.55472, 46.99965], [138.21971, 46.30795], [136.86232, 45.1435], [135.51535, 43.989], [134.86939, 43.3982], [133.53687, 42.81147], [132.90627, 42.79849], [132.27807, 43.28456], [130.93587, 42.55274], [130.780005, 42.22001], [130.780004, 42.22008], [130.78, 42.22], [130.77992, 42.22001], [130.64, 42.395], [130.64, 42.395024], [130.633866, 42.903015], [131.144688, 42.92999], [131.28855, 44.11152], [131.02519, 44.96796], [131.883454, 45.321162], [133.09712, 45.14409], [133.769644, 46.116927], [134.11235, 47.21248], [134.50081, 47.57845], [135.026311, 48.47823], [133.373596, 48.183442], [132.50669, 47.78896], [130.98726, 47.79013], [130.582293, 48.729687], [129.397818, 49.4406], [127.6574, 49.76027], [127.287456, 50.739797], [126.939157, 51.353894], [126.564399, 51.784255], [125.946349, 52.792799], [125.068211, 53.161045], [123.57147, 53.4588], [122.245748, 53.431726], [121.00308, 53.251401], [120.177089, 52.753886], [120.725789, 52.516226], [120.7382, 51.96411], [120.18208, 51.64355], [119.27939, 50.58292], [119.288461, 50.142883], [117.879244, 49.510983], [116.678801, 49.888531], [115.485695, 49.805177], [114.96211, 50.140247], [114.362456, 50.248303], [112.89774, 49.543565], [111.581231, 49.37968], [110.662011, 49.130128], [109.402449, 49.292961], [108.475167, 49.282548], [107.868176, 49.793705], [106.888804, 50.274296], [105.886591, 50.406019], [104.62158, 50.27532], [103.676545, 50.089966], [102.25589, 50.51056], [102.06521, 51.25991], [100.88948, 51.516856], [99.981732, 51.634006], [98.861491, 52.047366], [97.82574, 51.010995], [98.231762, 50.422401], [97.25976, 49.72605], [95.81402, 49.97746], [94.815949, 50.013433], [94.147566, 50.480537], [93.10421, 50.49529], [92.234712, 50.802171], [90.713667, 50.331812], [88.805567, 49.470521], [87.751264, 49.297198], [87.35997, 49.214981], [86.829357, 49.826675], [85.54127, 49.692859], [85.11556, 50.117303], [84.416377, 50.3114], [83.935115, 50.889246], [83.383004, 51.069183], [81.945986, 50.812196], [80.568447, 51.388336], [80.03556, 50.864751], [77.800916, 53.404415], [76.525179, 54.177003], [76.8911, 54.490524], [74.38482, 53.54685], [73.425679, 53.48981], [73.508516, 54.035617], [72.22415, 54.376655], [71.180137, 54.133285], [70.865267, 55.169734], [69.068167, 55.38525], [68.1691, 54.970392], [65.66687, 54.60125], [65.178534, 54.354228], [61.4366, 54.00625], [60.978066, 53.66499], [61.699986, 52.979996], [60.739993, 52.719986], [60.927269, 52.447548], [59.967534, 51.96042], [61.588003, 51.272659], [61.337424, 50.79907], [59.932807, 50.842194], [59.642282, 50.545442], [58.36332, 51.06364], [56.77798, 51.04355], [55.71694, 50.62171], [54.532878, 51.02624], [52.328724, 51.718652], [50.766648, 51.692762], [48.702382, 50.605128], [48.577841, 49.87476], [47.54948, 50.454698], [46.751596, 49.356006], [47.043672, 49.152039], [46.466446, 48.394152], [47.31524, 47.71585], [48.05725, 47.74377], [48.694734, 47.075628], [48.59325, 46.56104], [49.10116, 46.39933], [93.77766, 81.0246], [95.940895, 81.2504], [97.88385, 80.746975], [100.186655, 79.780135], [99.93976, 78.88094], [97.75794, 78.7562], [97.97259, 79.044745], [93.31288, 79.4265], [92.5454, 80.14379], [91.18107, 80.34146], [93.77766, 81.0246], [102.837815, 79.28129], [105.37243, 78.71334], [105.07547, 78.30689], [99.43814, 77.921], [101.2649, 79.23399], [102.08635, 79.34641], [102.837815, 79.28129], [138.831075, 76.13676], [141.471615, 76.09289], [145.086285, 75.562625], [144.3, 74.82], [140.61381, 74.84768], [138.95544, 74.61148], [136.97439, 75.26167], [137.51176, 75.94917], [138.831075, 76.13676], [148.22223, 75.345845], [150.73167, 75.08406], [149.575925, 74.68892], [147.977465, 74.778355], [146.11919, 75.17298], [146.358485, 75.49682], [148.22223, 75.345845], [139.86312, 73.36983], [140.81171, 73.76506], [142.06207, 73.85758], [143.48283, 73.47525], [143.60385, 73.21244], [142.08763, 73.20544], [140.038155, 73.31692], [139.86312, 73.36983], [44.846958, 80.58981], [46.799139, 80.771918], [48.318477, 80.78401], [48.522806, 80.514569], [49.09719, 80.753986], [50.039768, 80.918885], [51.522933, 80.699726], [51.136187, 80.54728], [49.793685, 80.415428], [48.894411, 80.339567], [48.754937, 80.175468], [47.586119, 80.010181], [46.502826, 80.247247], [47.072455, 80.559424], [44.846958, 80.58981], [22.731099, 54.327537], [20.892245, 54.312525], [19.66064, 54.426084], [19.888481, 54.86616], [21.268449, 55.190482], [22.315724, 55.015299], [22.757764, 54.856574], [22.651052, 54.582741], [22.731099, 54.327537], [53.50829, 73.749814], [55.902459, 74.627486], [55.631933, 75.081412], [57.868644, 75.60939], [61.170044, 76.251883], [64.498368, 76.439055], [66.210977, 76.809782], [68.15706, 76.939697], [68.852211, 76.544811], [68.180573, 76.233642], [64.637326, 75.737755], [61.583508, 75.260885], [58.477082, 74.309056], [56.986786, 73.333044], [55.419336, 72.371268], [55.622838, 71.540595], [57.535693, 70.720464], [56.944979, 70.632743], [53.677375, 70.762658], [53.412017, 71.206662], [51.601895, 71.474759], [51.455754, 72.014881], [52.478275, 72.229442], [52.444169, 72.774731], [54.427614, 73.627548], [53.50829, 73.749814], [142.914616, 53.704578], [143.260848, 52.74076], [143.235268, 51.75666], [143.648007, 50.7476], [144.654148, 48.976391], [143.173928, 49.306551], [142.558668, 47.861575], [143.533492, 46.836728], [143.505277, 46.137908], [142.747701, 46.740765], [142.09203, 45.966755], [141.906925, 46.805929], [142.018443, 47.780133], [141.904445, 48.859189], [142.1358, 49.615163], [142.179983, 50.952342], [141.594076, 51.935435], [141.682546, 53.301966], [142.606934, 53.762145], [142.209749, 54.225476], [142.654786, 54.365881], [142.914616, 53.704578], [174.92825, 67.20589], [175.01425, 66.58435], [174.33983, 66.33556], [174.57182, 67.06219], [171.85731, 66.91308], [169.89958, 65.97724], [170.89107, 65.54139], [172.53025, 65.43791], [172.555, 64.46079], [172.95533, 64.25269], [173.89184, 64.2826], [174.65392, 64.63125], [175.98353, 64.92288], [176.29716, 65.35667], [177.22266, 65.52024], [178.35993, 65.39052], [178.90332, 65.74044], [178.68611, 66.11211], [179.88377, 65.87456], [179.43268, 65.40411], [180.64, 979709], [180.68, 963636], [177.55, 68.2], [174.92825, 67.20589], [178.69378, 70.89302], [180.70, 832199], [180.71, 515714], [179.871875, 51.55762], [179.02433, 71.55553], [177.577945, 71.26948], [177.663575, 71.13277], [178.69378, 70.89302], [33.435988, 45.971917], [33.699462, 46.219573], [34.410402, 46.005162], [34.732017, 45.96566], [34.861792, 45.768182], [35.012659, 45.737725], [35.020788, 45.651219], [35.510009, 45.409993], [36.529998, 45.46999], [36.334713, 45.113216], [35.239999, 44.939996], [33.882511, 44.361479], [33.326421, 44.564877], [33.546924, 45.034771], [32.454174, 45.327466], [32.630804, 45.519186], [33.588162, 45.851569], [33.435988, 45.971917]]], "type": "Feature", "properties": {"name": "□□□□□□", "bbox": [-78.98, 23.71, -77.27, 0.04], "geometry": "MultiPolygon", "coordinates": [[[[[-78.98, 26.79], [-78.51, 26.87], [-77.85, 26.84], [-77.82, 26.58], [-78.91, 26.42], [-78.98, 26.79]]], [[[-77.79, 27.04], [-77.26, 59], [-77.17255, 25.87918], [-77.35641, 26.00735], [-77.34, 26.53], [-77.78802, 26.92516], [-77.79, 27.04]]], [[[-78.19087, 25.2103], [-77.89, 25.17], [-77.54, 24.34], [-77.84336], [69.94, 73.04], [72.58754, 72.77629], [72.79603, 72.22006], [71.84811, 71.40898], [72.47011, 71.09019], [72.79188, 70.39114], [72.5647, 69.02085], [73.66787, 68.4079], [73.2387, 67.7404], [71.28, 66.32], [72.42301, 66.17267], [72.82077, 66.53267], [73.92099, 66.78946], [74.18651, 67.28429], [75.052, 67.76047], [74.46926, 68.32899], [74.93584, 68.98918], [73.84236, 69.07146], [73.60187, 69.62763], [74.3998, 70.63175], [73.1011, 71.44717], [74.89082, 72.12119], [74.65926, 72.83227], [75.15801, 72.85497], [75.68351, 72.30056], [75.28898, 71.33556], [76.35911, 71.15287], [75.90313, 71.87401], [77.57665, 72.26717], [79.65202, 72.32011], [81.5, 71.75], [80.61071, 72.52855], [80.51109, 73.6482], [82.25, 73.85], [84.65526, 73.80591], [86.8223, 73.93688], [86.00956, 74.45967], [87.1668, 75.11643], [88.31571, 75.14393], [90.26, 75.64], [92.90058, 75.77333], [93.23421, 76.0472], [95.86, 76.14], [96.67821, 75.91548], [98.92254, 76.44689], [100.75967, 76.43028], [101.03532, 76.86189], [101.99084, 77.28754], [104.3516, 77.69792], [106.06664, 77.37389], [104.705, 77.1274], [106.97013, 76.97419], [107.24, 76.48], [108.1538, 76.72335], [111.07726, 76.71], [113.33151, 76.22224], [114.13417, 75.84764], [113.88539, 75.32779], [112.77918, 75.03186], [110.15125, 74.47673], [109.4, 74.18], [110.64, 74.04], [112.11919, 73.78774], [113.01954, 73.97693], [113.52958, 73.33505], [113.96881, 73.59488], [115.56782, 73.75285], [118.77633, 73.58772], [119.02, 73.12], [123.20066, 72.97122], [123.25777, 73.73503], [125.38, 73.56], [126.97644, 73.56549], [128.59126, 73.03871], [129.05157, 72.39872], [128.46, 71.98], [129.71599, 71.19304], [131.28858, 70.78699], [132.2535, 71.8363], [133.85766, 71.38642], [135.56193, 71.65525], [137.49755, 71.34763], [138.23409, 71.62803], [139.86983, 71.48783], [139.14791, 72.41619], [140.46817, 72.84941], [149.5, 72.2], [150.35118, 71.60643], [152.9689, 70.84222], [157.00688, 71.03141], [158.99779, 70.86672], [159.83031, 70.45324], [159.70866, 69.72198], [160.94053, 69.43728], [162.27907, 69.64204], [164.05248, 69.66823], [165.94037, 69.47199], [167.83567, 69.58269], [169.57763, 68.6938], [170.81688, 69.01363], [170.0082, 69.65276], [170.45345, 70.09703], [173.64391, 69.81743], [175.72403, 69.87725], [178.6, 69.4], [180.68, 963636], [180.64, 979709], [179.99281, 64.97433], [178.7072, 64.53493], [177.41128, 64.60821], [178.313, 64.07593], [178.90825, 63.25197], [179.37034, 62.98262], [179.48636, 62.56894], [179.22825, 62.3041], [177.3643, 62.5219], [174.56929, 61.76915], [173.68013, 61.65261], [172.15, 60.95], [170.6985, 60.33618],

```

{"type": "Feature", "properties": {"name": "MultiPolygon", "coordinates": [[[15.14282, 79.67431], [15.52255, 80.01608], [16.99085, 80.05086], [18.25183, 79.70175], [21.54383, 78.95611], [19.02737, 78.5626], [18.47172, 77.82669], [17.59441, 77.63796], [17.1182, 76.80941], [15.91315, 76.77045], [13.76259, 77.38035], [14.66956, 77.73565], [13.1706, 78.02493], [11.22231, 78.8693], [10.44453, 79.65239], [13.17077, 80.01046], [13.71852, 79.66039], [15.14282, 79.67431]]], [[31.101042, 69.558101], [29.39955, 69.15692], [28.59193, 69.064777], [29.015573, 69.766491], [27.732292, 70.164139], [26.179622, 69.825299], [25.689213, 69.092114], [24.735679, 68.649557], [23.66205, 68.891247], [22.356238, 68.841741], [21.244936, 69.370443], [20.645593, 69.106247], [20.025269, 69.065139], [19.87586, 68.407194], [17.93868, 68.567391], [17.729182, 68.010552], [16.768879, 68.013937], [16.108712, 67.302456], [15.108411, 66.193867], [13.55569, 64.787028], [13.919905, 64.445421], [13.571916, 64.049114], [12.579935, 64.066219], [11.930569, 63.128318], [11.992064, 61.800362], [12.631147, 61.293572], [22.300366, 60.117933], [11.468272, 59.432393], [11.027369, 58.856149], [10.356557, 59.469807], [8.382, 58.313288], [7.048748, 58.078884], [5.665835, 58.588155], [5.308234, 59.663232], [4.992078, 61.970998], [5.9129, 62.614473], [8.553411, 63.454008], [10.527709, 64.486038], [12.358347, 65.879726], [14.761146, 67.810642], [16.435927, 68.563205], [19.184028, 69.817444], [21.378416, 70.255169], [23.023742, 70.202072], [24.546543, 71.030497], [26.37005, 70.986262], [28.165547, 71.185474], [31.293418, 70.453788], [30.005435, 70.186259], [31.101042, 69.558101], [27.407506, 80.056406], [25.924651, 79.517834], [23.024466, 79.490012], [20.075188, 79.566823], [19.897266, 79.842362], [18.462264, 79.85988], [17.368015, 80.318896], [20.455992, 80.598156], [21.907945, 80.357679], [22.919253, 80.657144], [25.447625, 80.40734], [27.407506, 80.056406], [24.72412, 77.85385], [22.49032, 77.44493], [20.72601, 77.67704], [21.41611, 77.93504], [20.8119, 78.25463], [22.88426, 78.45494], [23.28134, 78.07954], [24.72412, 77.85385]]], [{"name": "MultiPolygon", "bbox": [-73.297, 60.03676, -12.0855, 83.64513], "geometry": {"type": "Polygon", "coordinates": [[[-46.76379, 82.62796], [-43.40644, 83.22516], [-39.89753, 83.18018], [-38.62214, 83.54905], [-35.0878, 83.64513], [-27.10046, 83.51966], [-20.84539, 82.72669], [-22.69182, 82.34165], [-26.51753, 82.29765], [-31.9, 82.2], [-31.39646, 82.02154], [-27.85666, 82.13178], [-24.84448, 81.78697], [-22.90328, 82.09317], [-22.07175, 81.73449], [-23.16961, 81.15271], [-20.62363, 81.52462], [-15.76818, 81.91245], [-12.77018, 81.71885], [-12.20855, 81.29154], [-16.28533, 80.58004], [-16.85, 80.35], [-20.04624, 80.17708], [-17.73035, 80.12912], [-18.9, 79.4], [-19.70499, 78.75128], [-19.67353, 77.63859], [-18.47285, 76.98565], [-20.03503, 76.94434], [-21.67944, 76.62795], [-19.83407, 76.09808], [-19.59896, 75.24838], [-20.66818, 75.15585], [-19.37281, 74.29561], [-21.59422, 74.22382], [-20.43454, 73.81713], [-20.76234, 73.46436], [-22.17221, 73.30955], [-23.56593, 73.30663], [-22.31311, 72.62928], [-22.29954, 72.18409], [-24.27834, 72.59788], [-24.79296, 72.3302], [-23.44296, 72.08016], [-22.13281, 71.46898], [-21.75356, 70.66369], [-23.53603, 70.471], [-24.30702, 70.85649], [-25.54341, 71.43094], [-25.20135, 70.75226], [-26.36276, 70.22646], [-23.7242, 70.18401], [-22.34902, 70.12946], [-25.02927, 69.2588], [-27.74737, 68.47046], [-30.67371, 68.12503], [-31.77665, 68.12078], [-32.81105, 67.73547], [-34.20196, 66.67974], [-36.35284, 65.9789], [-37.04378, 65.93768], [-38.37505, 65.69213], [-39.81222, 65.45848], [-40.66899, 64.83997], [-40.68281, 64.13902], [-41.1887, 63.48246], [-42.81938, 62.68233], [-42.41666, 61.90093], [-42.86619, 61.07404], [-43.3784, 60.09772], [-44.7875, 60.03676], [-46.26364, 60.85328], [-48.26294, 60.85843], [-49.23308, 61.40681], [-49.9003, 62.38336], [-51.63325, 63.62691], [-52.14014, 64.27842], [-52.27659, 65.1767], [-53.66166, 66.09957], [-53.30161, 66.8365], [-53.96911, 67.18899], [-52.9804, 68.35759], [-51.47536, 68.72958], [-51.08041, 69.14781], [-50.87122, 69.9291], [-52.013585, 69.574925], [-52.55792, 69.42616], [-53.45629, 69.283625], [-54.68336, 69.61003], [-54.75001, 70.28932], [-54.35884, 70.821315], [-53.431315, 70.835755], [-51.39014, 70.56978], [-53.10937, 71.20485], [-54.00422, 71.54719], [-55.71.406537], [-55.83468, 71.65444], [-54.71819, 72.58625], [-55.32634, 72.95861], [-56.12003, 73.64977], [-57.32363, 74.71026], [-58.59679, 75.09861], [-58.58516, 75.51727], [-61.26861, 76.10238], [-63.39165, 76.1752], [-66.06427, 76.13486], [-68.50438, 76.06141], [-69.66485, 76.37975], [-71.40257, 77.00857], [-68.77671, 77
```

9.029857,18.001511],[-89.150909,17.955468],[-89.14308,17.808319],[-90.067934,17.819326],[-91.00152,17.817595],[-91.00226  
9,17.254658],[-91.453921,17.252177],[-91.08167,16.918477],[-90.711822,16.687483],[-90.600847,16.470778],[-90.438867,16.4  
1011],[-90.464473,16.069562],[-91.74796,16.066565],[-92.229249,15.251447],[-92.087216,15.064585],[-92.20323,14.830103],[-  
-92.22775,14.538829],[-93.359464,15.61543],[-93.875169,15.940164],[-94.691656,16.200975],[-95.250227,16.128318],[-96.053  
382,15.752088],[-96.557434,15.653515],[-97.263592,15.917065],[-98.01303,16.107312],[-98.947676,16.566043],[-99.697397,16  
.706164],[-100.829499,17.171071],[-101.666089,17.649026],[-101.918528,17.91609],[-102.478132,17.975751],[-103.50099,18.2  
92295],[-103.917527,18.748572],[-104.99201,19.316134],[-105.493038,19.946767],[-105.731396,20.434102],[-105.397773,20.53  
1719],[-105.500661,20.816895],[-105.270752,21.076285],[-105.265817,21.422104],[-105.603161,21.871146],[-105.693414,22.26  
908],[-106.028716,22.773752],[-106.90998,23.767774],[-107.915449,24.548915],[-108.401905,25.172314],[-109.260199,25.5806  
09],[-109.444089,25.824884],[-109.291644,26.442934],[-109.801458,26.676176],[-110.391732,27.162115],[-110.641019,27.8598  
76],[-111.178919,27.941241],[-111.759607,28.467953],[-112.228235,28.954409],[-112.271824,29.266844],[-112.809594,30.0211  
14],[-113.163811,30.786881],[-113.148669,31.170966],[-113.871881,31.567608],[-114.205737,31.524045],[-114.776451,31.7995  
32],[-114.9367,31.393485],[-114.771232,30.913617],[-114.673899,30.162681],[-114.330974,29.750432],[-113.588875,29.061611  
],[-113.424053,28.826174],[-113.271969,28.754783],[-113.140039,28.411289],[-112.962298,28.42519],[-112.761587,27.780217]  
[-112.457911,27.525814],[-112.244952,27.171727],[-111.616489,26.662817],[-111.284675,25.73259],[-110.987819,25.294606]  
[-110.710007,24.826004],[-110.655049,24.298595],[-110.172856,24.265548],[-109.771847,23.811183],[-109.409104,23.364672]  
[-109.433392,23.185588],[-109.854219,22.818272],[-110.031392,22.823078],[-110.295071,23.430973],[-110.949501,24.000964]  
[-111.670568,24.484423],[-112.182036,24.738413],[-112.148989,25.470125],[-112.300711,26.012004],[-112.777297,26.32196]  
[-113.464671,26.768186],[-113.59673,26.63946],[-113.848937,26.900064],[-114.465747,27.14209],[-115.055142,27.722727],[-11  
4.982253,27.7982],[-114.570366,27.741485],[-114.199329,28.115093],[-114.162018,28.566112],[-114.931842,29.279479],[-115.  
518654,29.556362],[-115.887365,30.180794],[-116.25835,30.836464],[-116.721526,31.635744],[-117.12776,32.53534]]}}],{"typ  
e":"Feature","properties":{"name":"□□□□□□","bbox":[-58.427074,34.952647,-53.209589,-30.109686],"geometry":{"type":"Po  
lygon","coordinates":[[[-57.625133,-30.216295],[-56.976026,-30.109686],[-55.973245,-30.883076],[-55.60151,-30.853879],[-  
54.572452,-31.494511],[-53.787952,-32.047243],[-53.209589,-32.727666],[-53.650544,-33.202004],[-53.373662,-33.768378],[-  
53.806426,-34.396815],[-54.935866,-34.952647],[-55.67409,-34.752659],[-56.215297,-34.859836],[-57.139685,-34.430456],[-5  
7.817861,-34.462547],[-58.427074,-33.909454],[-58.349611,-33.263189],[-58.132648,-33.040567],[-58.14244,-32.044504],[-57  
.874937,-31.016556],[-57.625133,-30.216295]]]},"type":"Feature","properties":{"name":"□□□□□","bbox":[-73.987235,-33.7  
68378,-34.729993,5.244486],"geometry":{"type":"Polygon","coordinates":[[[-53.373662,-33.768378],[-53.650544,-33.202004]  
[-53.209589,-32.727666],[-53.787952,-32.047243],[-54.572452,-31.494511],[-55.60151,-30.853879],[-55.973245,-30.883076]  
[-56.976026,-30.109686],[-57.625133,-30.216295],[-56.2909,-28.852761],[-55.162286,-27.881915],[-54.490725,-27.474757],[-5  
3.648735,-26.923473],[-53.628349,-26.124865],[-54.13005,-25.547639],[-54.625291,-25.739255],[-54.428946,-25.162185],[-54  
.293476,-24.5708],[-54.29296,-24.021014],[-54.652834,-23.839578],[-55.027902,-24.001274],[-55.400747,-23.956935],[-55.51  
7639,-23.571998],[-55.610683,-22.655619],[-55.797958,-22.356937],[-56.473317,-22.0863],[-56.88151,-22.282154],[-57.937156  
,-22.090176],[-57.870674,-20.732688],[-58.166392,-20.176701],[-57.853802,-19.969995],[-57.949997,-19.400004],[-57.676009  
,-18.96184],[-57.498371,-18.174188],[-57.734558,-17.552468],[-58.280804,-17.27171],[-58.388058,-16.877109],[-58.24122,-1  
6.299573],[-60.15839,-16.258284],[-60.542966,-15.09391],[-60.251149,-15.077219],[-60.264326,-14.645979],[-60.459198,-14.  
354007],[-60.503304,-13.775955],[-61.084121,-13.479384],[-61.713204,-13.489202],[-62.127081,-13.198781],[-62.80306,-13.0  
00653],[-63.196499,-12.627033],[-64.316353,-12.461978],[-65.402281,-11.56627],[-65.321899,-10.895872],[-65.444837,-10.51  
1451],[-65.338435,-9.761988],[-66.646908,-9.931331],[-67.173801,-10.306812],[-68.048192,-10.712059],[-68.271254,-11.0145  
21],[-68.786158,-11.03638],[-69.529678,-10.951734],[-70.093752,-11.123972],[-70.548686,-11.009147],[-70.481894,-9.490118  
],[-71.302412,-10.079436],[-72.184891,-10.053598],[-72.563033,-9.520194],[-73.226713,-9.462213],[-73.015383,-9.032833],[-  
73.571059,-8.424447],[-73.987235,-7.52383],[-73.723401,-7.340999],[-73.724487,-6.918595],[-73.120027,-6.629931],[-73.21  
9711,-6.089189],[-72.964507,-5.741251],[-72.891928,-5.274561],[-71.748406,-4.593983],[-70.928843,-4.401591],[-70.794769  
,-4.251265],[-69.893635,-4.298187],[-69.444102,-1.556287],[-69.420486,-1.122619],[-69.577065,-0.549992],[-70.020656,-0.18  
5156],[-70.015566,0.541414],[-69.452396,0.706159],[-69.252434,0.602651],[-69.218638,0.985677],[-69.804597,1.089081],[-69  
816973,1.714805],[-67.868565,1.692455],[-67.53781,2.037163],[-67.259998,1.719999],[-67.065048,1.130112],[-66.876326,1.2  
53361],[-66.325765,0.724452],[-65.548267,0.789254],[-65.354713,1.095282],[-64.611012,1.328731],[-64.199306,1.492855],[-6  
4.083085,1.916369],[-63.368788,2.2009],[-63.422867,2.411068],[-64.269999,2.497006],[-64.408828,3.126786],[-64.368494,3.7  
9721],[-64.816064,4.056445],[-64.628659,4.148481],[-63.888343,4.02053],[-63.093198,3.770571],[-62.804533,4.006965],[-62.  
08543,4.162124],[-60.966893,4.536468],[-60.601179,4.918098],[-60.733574,5.200277],[-60.213683,5.244486],[-59.980959,5.01  
4061],[-60.111002,4.574967],[-59.767406,4.423503],[-59.53804,3.958803],[-59.815413,3.606499],[-59.974525,2.755233],[-59.  
718546,2.24963],[-59.646044,1.786894],[-59.030862,1.371698],[-58.540013,1.268088],[-58.429477,1.463942],[-58.11345,1.507  
195],[-57.660971,1.682585],[-57.335823,1.948538],[-56.782704,1.863711],[-56.539386,1.899523],[-55.995698,1.817667],[-55.  
9056,2.021996],[-56.073342,2.220795],[-55.973322,2.510364],[-55.569755,2.421506],[-55.097587,2.523748],[-54.524754,2.311  
849],[-54.088063,2.105557],[-53.778521,2.376703],[-53.554839,2.334897],[-53.418465,2.053389],[-52.939657,2.124858],[-52.  
556425,2.504705],[-52.249338,3.241094],[-51.657797,4.156232],[-51.317146,4.203491],[-51.069771,3.650398],[-50.508875,1.9  
01564],[-49.974076,1.736483],[-49.947101,1.04619],[-50.699251,0.222984],[-50.388211,-0.078445],[-48.620567,-0.235489],[-  
48.584497,-1.237805],[-47.824956,-0.581618],[-46.566584,-0.941028],[-44.905703,-1.55174],[-44.417619,-2.13775],[-44.5815  
89,-2.691308],[-43.418791,-2.38311],[-41.472657,-2.912018],[-39.978665,-2.873054],[-38.500383,-3.700652],[-37.223252,-4.  
820946],[-36.452937,-5.109404],[-35.597796,-5.149504],[-35.235389,-5.464937],[-34.89603,-6.738193],[-34.729993,-7.343221  
],[-35.128212,-8.996401],[-35.636967,-9.649282],[-37.046519,-11.040721],[-37.683612,-12.171195],[-38.423877,-13.038119]  
[-38.673887,-13.057652],[-38.953276,-13.79337],[-38.882298,-15.667054],[-39.161092,-17.208407],[-39.267339,-17.867746],[-  
39.583521,-18.262296],[-39.760823,-19.599113],[-40.774741,-20.904512],[-40.944756,-21.937317],[-41.754164,-22.370676],[-  
41.988284,-22.97007],[-43.074704,-22.967693],[-44.647812,-23.351959],[-45.352136,-23.796842],[-46.472093,-24.088969],[-  
47.648972,-24.885199],[-48.495458,-25.877025],[-48.641005,-26.623698],[-48.474736,-27.175912],[-48.66152,-28.186135],[-4  
8.888457,-28.674115],[-49.587329,-29.224469],[-50.696874,-30.984465],[-51.576226,-31.777698],[-52.256081,-32.24537],[-52  
.7121,-33.196578],[-53.373662,-33.768378]]]},"type":"Feature","properties":{"name":"□□□□□","bbox":[-69.590424,-22.87  
2919,-57.498371,-9.761988],"geometry":{"type":"Polygon","coordinates":[[[-69.529678,-10.951734],[-68.786158,-11.03638],[-  
68.271254,-11.014521],[-68.048192,-10.712059],[-67.173801,-10.306812],[-66.646908,-9.931331],[-65.338435,-9.761988],[-6  
5.444837,-10.511451],[-65.321899,-10.895872],[-65.402281,-11.56627],[-64.316353,-12.461978],[-63.196499,-12.627033],[-62  
.80306,-13.000653],[-62.127081,-13.198781],[-61.713204,-13.489202],[-61.084121,-13.479384],[-60.503304,-13.775955],[-60.  
459198,-14.354007],[-60.264326,-14.645979],[-60.251149,-15.077219],[-60.542966,-15.09391],[-60.15839,-16.258284],[-58.24  
122,-16.299573],[-58.388058,-16.877109],[-58.280804,-17.27171],[-57.734558,-17.552468],[-57.498371,-18.174188],[-57.6760  
09,-18.96184],[-57.949997,-19.400004],[-57.853802,-19.969995],[-58.166392,-20.176701],[-58.183471,-19.868399],[-59.11504  
2,-19.356906],[-60.043565,-19.342747],[-61.786326,-19.633737],[-62.265961,-20.513735],[-62.291179,-21.051635],[-62.68505  
7,-22.249029],[-62.846468,-22.034985],[-63.986838,-21.993644],[-64.377021,-22.798091],[-64.964892,-22.075862],[-66.27333  
9,-21.83231],[-67.106674,-22.735925],[-67.82818,-22.872919],[-68.219913,-21.494347],[-68.757167,-20.372658],[-68.442225,  
-19.405068],[-68.966818,-18.981683],[-69.100247,-18.260125],[-69.590424,-17.580012],[-68.959635,-16.500698],[-69.389764,  
-15.660129],[-69.160347,-15.323974],[-69.339535,-14.953195],[-68.948887,-14.453639],[-68.929224,-13.602684],[-68.88008,-  
12.899729],[-68.66508,-12.5613],[-69.529678,-10.951734]]]},"type":"Feature","properties":{"name":"□□□","bbox":[-81.41  
0943,-18.347975,-68.66508,-0.057205],"geometry":{"type":"Polygon","coordinates":[[[-69.893635,-4.298187],[-70.794769,-4.  
251265],[-70.928843,-4.401591],[-71.748406,-4.593983],[-72.891928,-5.274561],[-72.964507,-5.741251],[-73.219711,-6.08918  
9],[-73.120027,-6.629931],[-73.724487,-6.918595],[-73.723401,-7.340999],[-73.987235,-7.52383],[-73.571059,-8.424447],[-7  
3.015383,-9.032833],[-73.226713,-9.462213],[-72.563033,-9.520194],[-72.184891,-10.053598],[-71.302412,-10.079436],[-70.4  
81894,-9.490118],[-70.548686,-11.009147],[-70.093752,-11.123972],[-69.529678,-10.951734],[-68.66508,-12.5613],[-68.88008

, -12.899729], [-68.929224, -13.602684], [-68.948887, -14.453639], [-69.339535, -14.953195], [-69.160347, -15.323974], [-69.389764, -15.660129], [-68.959635, -16.500698], [-69.590424, -17.580012], [-69.858444, -18.092694], [-70.372572, -18.347975], [-71.37525, -17.773799], [-71.462041, -17.363488], [-73.44453, -16.359363], [-75.237883, -15.265683], [-76.009205, -14.649286], [-76.423469, -13.823187], [-76.259242, -13.535039], [-77.106192, -12.222716], [-78.092153, -10.377712], [-79.036953, -8.386568], [-79.44592, -7.930833], [-79.760578, -7.194341], [-80.537482, -6.541668], [-81.249996, -6.136834], [-80.926347, -5.690557], [-81.410943, -4.736765], [-81.09967, -4.036394], [-80.302561, -3.404856], [-80.184015, -3.821162], [-80.469295, -4.059287], [-80.442242, -4.425724], [-80.028908, -4.346091], [-79.624979, -4.454198], [-79.205289, -4.959129], [-78.639897, -4.547784], [-78.450684, -3.873097], [-77.837905, -3.003021], [-76.635394, -2.608678], [-75.544996, -1.56161], [-75.233723, -0.911417], [-75.373223, -0.152032], [-75.106625, -0.057205], [-74.441601, -0.53082], [-74.122395, -1.002833], [-73.659504, -1.260491], [-73.070392, -2.308954], [-72.325787, -2.434218], [-71.774761, -2.16979], [-71.413646, -2.342802], [-70.813476, -2.256865], [-70.047709, -2.725156], [-70.692682, -3.742872], [-70.0394044, -3.766591], [-69.893635, -4.298187]]]]], {"type": "Feature", "properties": {"name": "□□□□□□", "bbox": [-78.990935, -4.298187, 8187, -66.876326, 12.437303], "geometry": {"type": "Polygon", "coordinates": [[[-66.876326, 1.253361], [-67.065048, 1.130112], [-67.259998, 1.719999], [-67.53781, 2.037163], [-67.868565, 1.692455], [-69.816973, 1.714805], [-69.804597, 1.089081], [-69.218638, 0.985677], [-69.252434, 0.602651], [-69.452396, 0.706159], [-70.015566, 0.541414], [-70.020656, -0.185156], [-69.577065, -0.549992], [-69.420486, -1.122619], [-69.444102, -1.556287], [-69.893635, -4.298187], [-70.394044, -3.766591], [-70.692682, -3.742872], [-70.0394044, -3.766591], [-70.813476, -2.256865], [-71.413646, -2.342802], [-71.774761, -2.16979], [-72.325787, -2.434218], [-73.070392, -2.308954], [-73.659504, -1.260491], [-74.122395, -1.002833], [-74.441601, -0.53082], [-75.106625, -0.057205], [-75.373223, -0.152032], [-75.801466, 0.084801], [-76.292314, 0.416047], [-76.57638, 0.256936], [-77.424984, 0.395687], [-77.668613, 0.825893], [-77.855061, 0.809925], [-78.855259, 1.380924], [-78.990935, 1.69137], [-78.617831, 1.766404], [-78.662118, 2.267355], [-78.42761, 2.629556], [-77.931543, 2.696606], [-77.510431, 3.325017], [-77.12769, 3.849636], [-77.496272, 4.087606], [-77.307601, 4.667984], [-77.533221, 5.582812], [-77.318815, 5.845354], [-77.476661, 6.691116], [-77.881571, 7.223771], [-77.753414, 7.70984], [-77.431108, 7.638061], [-77.242566, 7.935278], [-77.474723, 8.524286], [-77.353361, 8.670505], [-76.836674, 8.638749], [-76.086384, 9.336821], [-75.6746, 9.443248], [-75.664704, 9.774003], [-75.480426, 10.61899], [-74.906895, 11.083045], [-74.27675, 11.102036], [-74.179223, 11.310473], [-73.414764, 11.227015], [-72.627835, 11.731972], [-72.238195, 11.95555], [-71.75409, 12.437303], [-71.399822, 12.376041], [-71.137461, 12.112982], [-71.331584, 11.776284], [-71.973922, 11.608672], [-72.227575, 11.108702], [-72.614658, 10.821975], [-72.905286, 10.450344], [-73.027604, 9.73677], [-73.304952, 9.152], [-72.78873, 9.085027], [-72.660495, 8.625288], [-72.439862, 8.405275], [-72.360901, 8.002638], [-72.479679, 7.632506], [-72.444487, 7.423785], [-72.198352, 7.340431], [-71.960176, 6.991615], [-70.674234, 7.087785], [-70.093313, 6.960376], [-69.38948, 6.099861], [-68.985319, 6.206805], [-68.265052, 6.153268], [-67.695087, 6.267318], [-67.34144, 6.095468], [-67.521532, 5.55687], [-67.744697, 5.221129], [-67.823012, 4.503937], [-67.621836, 3.839482], [-67.337564, 3.542342], [-67.303173, 3.318454], [-67.809938, 2.820655], [-67.447092, 2.600281], [-67.181294, 2.250638], [-66.876326, 1.253361]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□", "bbox": [-82.965783, 7.220541, -77.242566, 9.61161], "geometry": {"type": "Polygon", "coordinates": [[[-77.353361, 8.670505], [-77.474723, 8.524286], [-77.242566, 7.935278], [-77.431108, 7.638061], [-77.753414, 7.70984], [-77.881571, 7.223771], [-78.214936, 7.12255], [-78.429161, 8.052041], [-78.182096, 8.319182], [-78.435465, 8.387705], [-78.622121, 8.718124], [-79.120307, 8.996092], [-79.557877, 8.932375], [-79.760578, 8.584515], [-80.164481, 8.333316], [-80.382659, 8.298409], [-80.480689, 8.090308], [-80.00369, 7.547524], [-80.276671, 7.419754], [-80.422158, 7.271572], [-80.886401, 7.220541], [-81.059543, 7.817921], [-81.189716, 7.647906], [-81.519515, 7.70661], [-81.721311, 8.108963], [-82.131441, 8.175391], [-82.390934, 8.292362], [-82.820081, 8.290864], [-82.850958, 8.073823], [-82.965783, 8.225028], [-82.913176, 8.423517], [-82.829771, 8.626295], [-82.868657, 8.807266], [-82.719183, 8.925709], [-82.927155, 9.07433], [-82.932891, 9.476812], [-82.546196, 9.566135], [-82.187123, 9.207449], [-82.207586, 8.995575], [-81.808567, 8.950617], [-81.714154, 9.031955], [-81.439287, 8.786234], [-80.947302, 8.858504], [-80.521901, 9.11072], [-79.9146, 9.312765], [-79.573303, 9.61161], [-79.021192, 9.552931], [-79.05845, 9.454565], [-78.500888, 9.420459], [-78.055928, 9.24773], [-77.729514, 8.946844], [-77.353361, 8.670505]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [-85.941725, 8.225028, -82.546196, 11.217119], "geometry": {"type": "Polygon", "coordinates": [[[-82.546196, 9.566135], [-82.932891, 9.476812], [-82.927155, 9.07433], [-82.719183, 8.925709], [-82.868657, 8.807266], [-82.829771, 8.626295], [-82.913176, 8.423517], [-82.965783, 8.225028], [-83.508437, 8.446927], [-83.711474, 8.656836], [-83.596313, 8.830443], [-83.632642, 9.051386], [-83.909886, 9.290803], [-84.303402, 9.487354], [-84.647644, 9.615537], [-84.713351, 9.908052], [-84.97566, 10.086723], [-84.911375, 9.795992], [-85.110923, 9.55704], [-85.339488, 9.834542], [-85.660787, 9.933347], [-85.797445, 10.134886], [-85.791709, 10.439337], [-85.659314, 10.754331], [-85.941725, 10.895278], [-85.71254, 11.088445], [-85.561852, 11.217119], [-84.903003, 10.952303], [-84.673069, 11.082267], [-84.355931, 10.999226], [-84.190179, 10.79345], [-83.895054, 10.726839], [-83.655612, 10.938764], [-83.40232, 10.395438], [-83.015677, 9.992982], [-82.546196, 9.566135]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [-87.668493, 10.726839, -83.147219, 15.016267], "geometry": {"type": "Polygon", "coordinates": [[[-83.655612, 10.938764], [-83.895054, 10.726839], [-84.190179, 10.79345], [-84.355931, 10.999226], [-84.673069, 11.082267], [-84.903003, 10.952303], [-85.561852, 11.217119], [-85.71254, 11.088445], [-86.058488, 11.403439], [-86.52585, 11.806877], [-86.745992, 12.143962], [-87.167516, 12.458258], [-87.668493, 12.90991], [-87.557467, 13.064552], [-87.392386, 12.914018], [-87.316654, 12.984686], [-87.005769, 13.025794], [-86.880557, 13.254204], [-86.733822, 13.263093], [-86.755087, 13.754845], [-86.520708, 13.778487], [-86.312142, 13.771356], [-86.096264, 14.038187], [-85.801295, 13.836055], [-85.698665, 13.960078], [-85.514413, 14.079012], [-85.165365, 14.35437], [-85.148751, 14.560197], [-85.052787, 14.551541], [-84.820037, 14.819587], [-84.649582, 14.666805], [-84.449336, 14.621614], [-84.228342, 14.748764], [-83.975721, 14.749436], [-83.628585, 14.880074], [-83.489989, 15.016267], [-83.147219, 14.995829], [-83.233234, 14.899866], [-83.284162, 14.676624], [-83.182126, 14.310703], [-83.4125, 13.970078], [-83.519832, 13.567699], [-83.552207, 13.127054], [-83.498515, 12.869292], [-83.473323, 12.419087], [-83.626104, 12.32085], [-83.719613, 11.893124], [-83.650858, 11.629032], [-83.85547, 11.373311], [-83.808936, 11.103044], [-83.655612, 10.938764]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [-89.353326, 12.984686, -83.147219, 16.005406], "geometry": {"type": "Polygon", "coordinates": [[[-83.147219, 14.995829], [-83.489989, 15.016267], [-83.628585, 14.880074], [-83.975721, 14.749436], [-84.228342, 14.748764], [-84.449336, 14.621614], [-84.649582, 14.666805], [-84.820037, 14.819587], [-84.924501, 14.790493], [-85.052787, 14.551541], [-85.148751, 14.560197], [-85.165365, 14.35437], [-85.514413, 14.079012], [-85.698665, 13.960078], [-85.801295, 13.836055], [-86.096264, 14.038187], [-86.312142, 13.771356], [-86.520708, 13.778487], [-86.755087, 13.754845], [-86.733822, 13.263093], [-86.880557, 13.254204], [-87.005769, 13.025794], [-87.316654, 12.984686], [-87.489409, 13.297535], [-87.793111, 13.38448], [-87.723503, 13.78505], [-87.859515, 13.893312], [-88.065343, 13.964626], [-88.503998, 13.845486], [-88.541231, 13.980155], [-88.843073, 14.140507], [-89.058512, 14.340029], [-89.353326, 14.424133], [-89.145535, 14.678019], [-89.22522, 14.874286], [-89.154811, 15.066419], [-88.68068, 15.346247], [-88.225023, 15.727722], [-88.12153, 15.688655], [-87.901813, 15.864458], [-87.61568, 15.878799], [-87.522921, 15.797279], [-87.367762, 15.84694], [-86.903191, 15.756713], [-86.440946, 15.782835], [-86.119234, 15.893449], [-86.001954, 16.005406], [-85.683317, 15.953652], [-85.444004, 15.885749], [-85.182444, 15.909158], [-84.983722, 15.995923], [-84.52698, 15.857224], [-84.368256, 15.835158], [-84.063055, 15.648244], [-83.773977, 15.424072], [-83.410381, 15.270903], [-83.147219, 14.995829]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [-90.095555, 13.149017, -87.723503, 14.424133], "geometry": {"type": "Polygon", "coordinates": [[[-89.353326, 14.424133], [-89.058512, 14.340029], [-88.843073, 14.140507], [-88.541231, 13.980155], [-88.503998, 13.845486], [-88.065343, 13.964626], [-87.859515, 13.893312], [-87.723503, 13.78505], [-87.793111, 13.38448], [-87.904112, 13.149017], [-88.483302, 13.163951], [-88.843228, 13.259734], [-89.256743, 13.458533], [-89.812394, 13.520622], [-90.095555, 13.735338], [-90.064678, 13.88197], [-89.721934, 14.134228], [-89.534219, 14.244816], [-89.587343, 14.362586], [-89.353326, 14.424133]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [-92.229249, 13.735338, -88.225023, 17.819326], "geometry": {"type": "Polygon", "coordinates": [[[-92.22775, 14.538829], [-92.20323, 14.830103], [-92.087216, 15.064585], [-92.229249, 15.251447], [-91.74796, 16.066565], [-90.464473, 16.069562], [-90.438867, 16.41011], [-90.600847, 16.470778], [-90.711822, 16.687483], [-91.08167, 16.918477], [-91.453921, 17.252177], [-91.002269, 17.254658], [-91.00152, 17.817595], [-90.067934, 17.819326], [-89.14308, 17.808319], [-89.150806, 17.015577], [-89.229122, 15.886938], [-88.930613, 15.887273], [-88.604586, 15.70638], [-88.518364, 15.855389], [-88.225023, 15.727722], [-88.68068, 15.346247], [-89.154811, 15.066419], [-89.22522, 14.874286], [-89.145535, 14.678019], [-89.353326, 14.424133], [-89.587343, 14.362586], [-89.534219, 14.244816], [-89.721934, 14.134228], [-90.064678, 13.88197], [-90.095555, 13.735338], [-90.608624, 13.909771], [-91.23241, 13.927832], [-91.689747, 14.126218], [-92.22775, 14.538829]]]]], {"type": "Featu

re","properties":{"name":"□□□□","bbox":[-89.229122,15.886938,-88.106813,18.499982],"geometry":{"type":"Polygon","coordinates":[[[-89.14308,17.808319],[-89.150909,17.955468],[-89.029857,18.001511],[-88.848344,17.883198],[-88.490123,18.486831],[-88.300031,18.499982],[-88.296336,18.353273],[-88.106813,18.348674],[-88.123479,18.076675],[-88.285355,17.644143],[-88.197867,17.489475],[-88.302641,17.131694],[-88.239518,17.036066],[-88.355428,16.530774],[-88.551825,16.265467],[-88.732434,16.233635],[-88.930613,15.887273],[-89.229122,15.886938],[-89.150806,17.015577],[-89.14308,17.808319]]]},"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[-73.304952,0.724452,-59.758285,12.162307],"geometry":{"type":"Polygon","coordinates":[[[-60.733574,5.200277],[-60.601179,4.918098],[-60.966893,4.536468],[-62.08543,4.162124],[-62.804533,4.006965],[-63.093198,3.770571],[-63.888343,4.02053],[-64.628659,4.148481],[-64.816064,4.056445],[-64.368494,3.79721],[-64.408828,3.126786],[-64.269999,2.497006],[-63.422867,2.411068],[-63.368788,2.2009],[-64.083085,1.916369],[-64.199306,1.492855],[-64.611012,1.328731],[-65.354713,1.095282],[-65.548267,0.789254],[-66.325765,0.724452],[-66.876326,1.253361],[-67.181294,2.250638],[-67.447092,2.600281],[-67.809938,2.820655],[-67.303173,3.318454],[-67.337564,3.542342],[-67.621836,3.839482],[-67.823012,4.503937],[-67.744697,5.221129],[-67.521532,5.55687],[-67.34144,6.095468],[-67.695087,6.267318],[-68.265052,6.153268],[-68.985319,6.206805],[-69.38948,6.099861],[-70.093313,6.960376],[-70.674234,7.087785],[-71.960176,6.991615],[-72.198352,7.340431],[-72.444487,7.423785],[-72.479679,7.632506],[-72.360901,8.002638],[-72.439862,8.405275],[-72.660495,8.625288],[-72.78873,9.085027],[-73.304952,9.152],[-73.027604,9.73677],[-72.905286,10.450344],[-72.614658,10.821975],[-72.227575,11.108702],[-71.973922,11.608672],[-71.331584,11.776284],[-71.360006,11.539994],[-71.94705,11.423282],[-71.620868,10.96946],[-71.633064,10.446494],[-72.074174,9.865651],[-71.695644,9.072263],[-71.264559,9.137195],[-71.039999,9.859993],[-71.350084,10.211935],[-71.400623,10.968969],[-70.155299,11.375482],[-70.293843,11.846822],[-69.943245,12.162307],[-69.5843,11.459611],[-68.882999,11.443385],[-68.233271,10.885744],[-68.194127,10.554653],[-67.296249,10.545868],[-66.227864,10.648627],[-65.655238,10.200799],[-64.890452,10.077215],[-64.329479,10.389599],[-64.338007,10.641418],[-63.079322,10.701724],[-61.880946,10.715625],[-62.730119,10.420269],[-62.388512,9.948204],[-61.588767,9.873067],[-60.830597,9.38134],[-60.671252,8.580174],[-60.150096,8.602757],[-59.758285,8.367035],[-60.550588,7.779603],[-60.637973,7.415],[-60.295688,7.043911],[-60.543999,6.856584],[-61.159336,6.696077],[-61.139415,6.234297],[-61.410303,5.959068],[-60.733574,5.200277]]]},"type":"Feature","properties":{"name":"□□□□□□","bbox":[-61.410303,1.268088,-56.539386,8.367035],"geometry":{"type":"Polygon","coordinates":[[[-56.539386,1.899523],[-56.782704,1.863711],[-57.335823,1.948538],[-57.660971,1.682585],[-58.11345,1.507195],[-58.429477,1.463942],[-58.540013,1.268088],[-59.038082,1.317698],[-59.646044,1.786894],[-59.718546,2.24963],[-59.974525,2.755233],[-59.815413,3.606499],[-59.53804,3.958803],[-59.767406,4.423503],[-60.111002,4.574967],[-59.980959,5.014061],[-60.213683,5.244486],[-60.733574,5.200277],[-61.410303,5.959068],[-61.139415,6.234297],[-61.159336,6.696077],[-60.543999,6.856584],[-60.295688,7.043911],[-60.637973,7.415],[-60.550588,7.779603],[-59.758285,8.367035],[-59.101684,7.999202],[-58.482962,7.347691],[-58.454876,6.832787],[-58.078103,6.809094],[-57.542219,6.321268],[-57.147436,5.97315],[-57.307246,5.073567],[-57.914289,4.812626],[-57.86021,4.576801],[-58.044694,4.060864],[-57.601569,3.334655],[-57.281433,3.333492],[-57.150098,2.768927],[-56.539386,1.899523]]]},"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[-58.044694,1.817667,-53.958045,6.025291],"geometry":{"type":"Polygon","coordinates":[[[-54.524754,2.311849],[-55.097587,2.523748],[-55.569755,2.421506],[-55.973322,2.510364],[-56.073342,2.220795],[-55.9056,2.021996],[-55.995698,1.817667],[-56.539386,1.899523],[-57.150098,2.768927],[-57.281433,3.333492],[-57.601569,3.334655],[-58.044694,4.060864],[-57.86021,4.576801],[-57.914289,4.812626],[-57.307246,5.073567],[-57.147436,5.97315],[-55.949318,5.959068],[-55.84178,5.953125],[-55.03325,6.025291],[-53.958045,5.756548],[-54.478633,4.896756],[-54.399542,4.212611],[-54.006931,3.620038],[-54.181726,3.18978],[-54.269705,2.732392],[-54.524754,2.311849]]]},"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[-54.524754,2.053389,9.560016,51.148506],"geometry":{"type":"MultiPolygon","coordinates":[[[-51.657797,4.156232],[-52.249338,3.241094],[-52.556425,2.504705],[-52.939657,2.124858],[-53.418465,2.053389],[-53.554839,2.334897],[-53.778521,2.376703],[-54.088063,2.105557],[-54.524754,2.311849],[-54.269705,2.732392],[-54.181726,3.18978],[-54.006931,3.620038],[-54.399542,4.212611],[-54.478633,4.896756],[-53.958045,5.756548],[-53.618453,5.646529],[-52.882141,5.409851],[-51.823343,4.565768],[-51.657797,4.156232]]],[[-6.18632,49.463803],[6.65823,49.201958],[8.099279,49.017784],[7.593676,48.333019],[7.466759,47.620582],[7.192202,47.449766],[6.736571,47.541801],[6.768714,47.287708],[6.037389,46.725779],[6.022609,46.27299],[6.5001,46.429673],[6.843593,45.991147],[6.802355,45.70858],[7.096652,45.33099],[6.749955,45.028518],[7.007562,44.254767],[7.549596,44.127901],[7.435185,43.693845],[6.529245,43.128892],[4.556963,43.399651],[3.100411,43.075201],[2.985999,42.473015],[1.826793,42.343385],[0.701591,42.795734],[0.338047,42.579546],[-1.502771,43.034014],[-1.901351,43.422802],[-1.384225,44.02261],[-1.193798,46.014918],[-2.225724,47.064363],[-2.963276,47.570327],[-4.491555,47.954954],[-4.59235,48.68416],[-3.295814,48.901692],[-1.616511,48.644421],[-1.933494,49.776342],[-0.989469,49.347376],[1.338761,50.127173],[1.639001,50.946606],[2.513573,51.148506],[2.658422,50.796848],[3.123252,50.780363],[3.588184,50.378992],[4.286023,49.907497],[4.799222,49.985373],[5.674052,49.529484],[5.897759,49.442667],[6.18632,49.463803]]],[[8.746009,42.628122],[9.390001,43.009985],[9.560016,42.152492],[9.229752,41.380007],[8.775723,41.583612],[8.544213,42.256517],[8.746009,42.628122]]]]]},"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[-80.967765,-4.959129,-75.233723,1.380924],"geometry":{"type":"Polygon","coordinates":[[[-75.373223,-0.152032],[-75.233723,-0.911417],[-75.544996,-1.56161],[-76.635394,-2.608678],[-77.837905,-3.003021],[-78.450684,-3.873097],[-78.639897,-4.547784],[-79.205289,-4.959129],[-79.624979,-4.454198],[-80.028908,-4.366091],[-80.442242,-4.425724],[-80.469295,-4.059287],[-80.184015,-3.821162],[-80.302561,-3.404856],[-79.770293,-2.657512],[-79.986559,-2.220794],[-80.368784,-2.685159],[-80.967765,-2.246943],[-80.764806,-1.965048],[-80.933659,-1.057455],[-80.58337,-0.906663],[-80.399325,-0.283703],[-80.020898,0.36034],[-80.09061,0.768429],[-79.542762,0.982938],[-78.855259,1.380924],[-77.855061,0.809925],[-77.668613,0.825893],[-77.424984,0.395687],[-76.57638,0.256938],[-76.292314,0.416047],[-75.801466,0.084801],[-75.373223,-0.152032]]]},"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[-67.242428,17.946553,-65.591004,18.520601],"geometry":{"type":"Polygon","coordinates":[[[-66.282434,18.514762],[-65.771303,18.426679],[-65.591004,18.228035],[-65.847164,17.975906],[-66.559934,17.981823],[-67.184162,17.946553],[-67.242428,18.37446],[-67.100679,18.520601],[-66.282434,18.514762]]]},"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[-78.337719,17.701116,-76.199659,18.524218],"geometry":{"type":"Polygon","coordinates":[[[-77.569601,18.490525],[-76.896619,18.400867],[-76.365359,18.160701],[-76.199659,17.886867],[-76.902561,17.868238],[-77.206341,17.701116],[-77.766023,17.861597],[-78.337719,18.225968],[-78.217727,18.454533],[-77.797365,18.524218],[-77.569601,18.490525]]]},"type":"Feature","properties":{"name":"□□□□□","bbox":[-84.974911,19.855481,-74.178025,23.188611],"geometry":{"type":"Polygon","coordinates":[[[-82.268151,23.188611],[-81.404457,23.112711],[-80.618769,23.10598],[-79.679524,22.765303],[-79.281486,22.399202],[-78.347434,22.512166],[-77.993296,22.277194],[-77.146422,21.657851],[-76.523825,21.20682],[-76.19462,21.220565],[-75.598222,21.016624],[-75.67106,20.735091],[-74.933896,20.693905],[-74.178025,20.284628],[-74.296648,20.050379],[-74.961595,19.923435],[-75.63468,19.873774],[-76.323656,19.952891],[-77.755481,19.855481],[-77.085108,20.413354],[-77.492655,20.673105],[-78.137292,20.739949],[-78.482827,21.028613],[-78.719867,21.598114],[-79.285,21.559175],[-80.217475,21.827324],[-80.517535,22.037079],[-81.820943,22.192057],[-82.169992,22.387109],[-81.795002,22.636965],[-82.775898,22.68815],[-83.494459,22.168518],[-83.9088,22.154565],[-84.052151,21.910575],[-84.54703,21.801228],[-84.974911,21.896028],[-84.447062,22.20495],[-84.230357,22.565755],[-83.77824,22.788118],[-83.267548,22.983042],[-82.510436,23.078747],[-82.268151,23.188611]]]},"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[-25.264226,-22.271612,32.849861,-15.507787],"geometry":{"type":"Polygon","coordinates":[[[31.191409,-22.25151],[30.659865,-22.151567],[30.322883,-22.271612],[29.839037,-22.102216],[29.432188,-22.091313],[28.794656,-21.639454],[28.02137,-21.485975],[-27.727228,-20.851802],[27.724747,-20.499059],[27.296505,-20.39152],[26.164791,-19.293086],[25.850391,-18.714413],[25.649163,-18.536026],[25.264226,-17.73654],[26.381935,-17.846042],[26.706773,-17.961229],[27.044427,-17.938026],[27.598243,-17.290831],[28.467906,-16.4684],[28.825869,-16.389749],[28.947463,-16.043051],[29.516834,-15.644678],[30.274256,-15.507787],[30.338955,-15.880839],[31.173064,-15.860944],[31.636498,-16.07199],[31.852041,-16.319417],[32.328239,-16.392074],[32.847639,-16.713398],[32.849861,-17.979057],[32.654886,-18.67209],[32.611994,-19.419383],[32.772708,-19.715592],[32.659743,-20.30429],[32.508693,-20.395292],[32.244988,-21.116489],[31.191409,-22.25151]]]},"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[-19.895458,-26.828543,29.432188,-17.661816],"geometry":{"type":"Polygon","coordinates":[[[29.432188,-22.091313],[28.017236,-22.827754],[27.11941,-23.574323],[26

.786407, -24.240691], [26.485753, -24.616327], [25.941652, -24.696373], [25.765849, -25.174845], [25.664666, -25.486816], [25.025171, -25.71967], [24.211267, -25.670216], [23.73357, -25.390129], [23.312097, -25.26869], [22.824271, -25.500459], [22.579532, -25.979448], [22.105969, -26.280256], [21.605896, -26.726534], [20.889609, -26.828543], [20.66647, -26.477453], [20.758609, -25.868136], [20.165726, -24.917962], [19.895768, -24.76779], [19.895458, -21.849157], [20.881134, -21.814327], [20.910641, -18.252219], [21.65504, -18.219146], [23.196858, -17.869038], [23.579006, -18.281261], [24.217365, -17.889347], [24.520705, -17.887125], [25.084443, -17.661816], [25.264226, -17.73654], [25.649163, -18.536026], [25.850391, -18.714413], [26.164791, -19.293086], [27.296505, -20.39152], [27.724747, -20.499059], [27.727228, -20.851802], [28.02137, -21.485975], [28.794656, -21.639454], [29.432188, -22.091313]]], {"type": "Feature", "properties": {"name": "□□□□□□□", "bbox": [11.734199, -29.045462, 25.084443, -16.941343], "geometry": {"type": "Polygon", "coordinates": [[[19.895768, -24.76779], [19.894734, -28.461105], [19.002127, -28.972443], [18.464899, -29.045462], [17.836152, -28.856378], [17.387497, -28.783514], [17.218929, -28.355943], [16.824017, -28.082162], [16.344977, -28.576705], [15.601818, -27.821247], [15.210472, -27.090956], [14.989711, -26.117372], [14.743214, -25.39292], [14.408144, -23.853014], [14.385717, -22.656653], [14.257714, -22.111208], [13.868642, -21.699037], [13.352498, -20.872834], [12.826845, -19.673166], [12.608564, -19.045349], [11.794919, -18.069129], [11.734199, -17.301889], [12.215461, -17.111668], [12.814081, -16.941343], [13.462362, -16.971212], [14.058501, -17.423381], [14.209707, -17.353101], [18.263309, -17.309951], [18.956187, -17.789095], [21.377176, -17.930636], [23.215048, -17.523116], [24.033862, -17.295843], [24.682349, -17.353411], [25.07695, -17.578823], [25.084443, -17.661816], [24.520705, -17.887125], [24.217365, -17.889347], [23.579006, -18.281261], [23.196858, -17.869038], [21.65504, -18.219146], [20.910641, -18.252219], [20.881134, -21.814327], [19.895458, -21.849157], [19.895768, -24.76779]]]]}, {"type": "Feature", "properties": {"name": "□□□□□□", "bbox": [-17.625043, 12.33209, -11.467899, 16.598264], "geometry": {"type": "Polygon", "coordinates": [[[16.713729, 13.594959], [17.126107, 14.373516], [17.625043, 14.729541], [17.185173, 14.919477], [16.700706, 15.621527], [16.463098, 16.135036], [16.12069, 16.455663], [15.623666, 16.369337], [15.135737, 16.587282], [14.577348, 16.598264], [14.099521, 16.304302], [13.435738, 16.039383], [12.830658, 15.303692], [12.17075, 14.616834], [12.124887, 13.994727], [11.927716, 13.422075], [11.553398, 13.141214], [11.467899, 12.754519], [11.513943, 12.442988], [11.658301, 12.386583], [12.203565, 12.465648], [12.278599, 12.35444], [12.499051, 12.33209], [13.217818, 12.57874], [13.700476, 12.586183], [15.548477, 12.62817], [15.816574, 12.515567], [16.147717, 12.547762], [16.677452, 12.384852], [16.841525, 13.151394], [15.931296, 13.130284], [15.691001, 13.270353], [15.511813, 13.27857], [15.441163, 13.509512], [14.712197, 13.298207], [14.277702, 13.280585], [13.844963, 13.505042], [14.046992, 13.794068], [14.376714, 13.62568], [14.687031, 13.630357], [15.081735, 13.876492], [15.39877, 13.860369], [15.624596, 13.623587], [16.713729, 13.594959]]]]}, {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [-12.17075, 10.096361, 4.27021, 24.974574], "geometry": {"type": "Polygon", "coordinates": [[[11.513943, 12.442988], [11.467899, 12.754519], [11.553398, 13.141214], [11.927716, 13.422075], [12.124887, 13.994727], [12.17075, 14.616834], [11.834208, 14.799097], [11.666078, 15.388208], [11.349095, 15.411256], [10.650791, 15.132746], [10.086846, 15.330486], [9.700255, 15.264107], [9.550238, 15.486497], [5.537744, 15.50169], [5.315277, 16.201854], [5.488523, 16.325102], [5.971129, 20.640833], [6.453787, 24.956591], [4.923337, 24.974574], [3.550055, 22.792666], [1.823228, 20.610809], [2.060991, 20.142233], [2.683588, 19.85623], [3.146661, 19.693579], [3.158133, 19.057364], [4.267419, 19.155265], [4.27021, 16.852227], [3.723422, 16.184284], [3.638259, 15.56812], [2.749993, 15.409525], [1.385528, 15.323561], [1.015783, 14.968182], [0.374892, 14.928908], [0.266257, 14.924309], [0.515854, 15.116158], [1.066363, 14.973815], [2.01035, 14.559008], [2.191825, 14.246418], [2.967694, 13.79815], [3.103707, 13.541267], [3.522803, 13.37662], [4.006391, 13.472485], [4.280405, 13.228444], [4.427166, 12.542646], [5.220942, 11.713859], [5.197843, 11.375146], [5.470565, 10.95127], [5.404342, 10.370737], [5.816926, 10.222555], [6.050452, 10.096361], [6.205223, 10.524061], [6.493965, 10.411303], [6.666461, 10.430811], [6.850507, 10.138994], [7.622759, 10.147236], [7.89959, 10.297382], [8.029944, 10.206535], [8.335377, 10.494812], [8.282357, 10.792597], [8.407311, 10.909257], [8.620321, 10.810891], [8.581305, 11.136246], [8.376305, 11.393646], [8.786099, 11.812561], [8.905265, 12.088358], [9.127474, 12.30806], [9.327616, 12.334286], [9.567912, 12.194243], [9.890993, 12.060479], [10.165214, 11.844084], [10.593224, 11.923975], [10.87083, 12.177887], [11.036556, 12.211245], [11.297574, 12.077971], [11.456169, 12.076834], [11.513943, 12.442988]]]]}, {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [-17.063423, 14.616834, -4.923337, 27.395744], "geometry": {"type": "Polygon", "coordinates": [[[17.063423, 20.999752], [16.845194, 21.333323], [12.929102, 21.327071], [13.118754, 22.77122], [12.874222, 23.284832], [11.937224, 23.374594], [11.969419, 25.933353], [8.687294, 25.881056], [8.6844, 27.395744], [4.923337, 24.974574], [6.453787, 24.956591], [5.971129, 20.640833], [5.488523, 16.325102], [5.315277, 16.201854], [5.537744, 15.50169], [9.550238, 15.486497], [9.700255, 15.264107], [10.086846, 15.330486], [10.650791, 15.132746], [11.349095, 15.411256], [11.666078, 15.388208], [11.834208, 14.799097], [12.17075, 14.616834], [12.830658, 15.303692], [13.435738, 16.039383], [14.099521, 16.304302], [14.577348, 16.598264], [15.135737, 16.587282], [15.623666, 16.369337], [16.12069, 16.455663], [16.463098, 16.135036], [16.549708, 16.673892], [16.270552, 17.166963], [16.146347, 18.108482], [16.256883, 19.096716], [16.377651, 19.593817], [16.277838, 20.092521], [16.536324, 20.567866], [17.063423, 20.999752]]]]}, {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [0.772336, 6.142158, 3.797112, 12.235636], "geometry": {"type": "Polygon", "coordinates": [[[2.691702, 6.258817], [1.865241, 6.142158], [1.618951, 6.832038], [1.664478, 9.12859], [1.463043, 9.334624], [1.425061, 9.825395], [1.077795, 10.175607], [0.772336, 10.478008], [0.899563, 10.997339], [1.24347, 11.110511], [1.447178, 11.547719], [1.935986, 11.64115], [2.154474, 11.94015], [2.490164, 12.233052], [2.848643, 12.235636], [3.61118, 11.660167], [3.572216, 11.327939], [3.797112, 10.734746], [3.60007, 10.332186], [3.705438, 10.06321], [3.220352, 9.444153], [2.912308, 9.137608], [2.723793, 8.506845], [2.749063, 7.870734], [2.691702, 6.258817]]]]}, {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [0.295646, 11.660167, 15.903247, 23.471668], "geometry": {"type": "Polygon", "coordinates": [[[14.8513, 22.86295], [15.096888, 21.308519], [15.47106, 21.04845], [15.487148, 20.730415], [15.903247, 20.387619], [15.685741, 19.95718], [15.300441, 17.92795], [15.247731, 16.627306], [13.97217, 15.68437], [13.540394, 14.367134], [13.956699, 13.996691], [13.954477, 13.353449], [14.595781, 13.330427], [14.495787, 12.859396], [14.213531, 12.802035], [14.181336, 12.483657], [13.995353, 12.461565], [13.318702, 13.556356], [13.083987, 13.596147], [12.302071, 13.037189], [11.527803, 13.32898], [10.989593, 13.387323], [10.701032, 13.246918], [10.114814, 13.277252], [9.524928, 12.851102], [9.014933, 12.826659], [7.804671, 13.343527], [7.330747, 13.098038], [6.820442, 13.115091], [6.445426, 13.492768], [5.443058, 13.865924], [4.368344, 13.747482], [4.107946, 13.531216], [3.967283, 12.956109], [3.680634, 12.552903], [3.61118, 11.660167], [2.848643, 12.235636], [2.490164, 12.233052], [2.154474, 11.94015], [2.177108, 12.625018], [1.024103, 12.851826], [0.993046, 13.33575], [0.429928, 13.988733], [0.295646, 14.444235], [0.374892, 14.928908], [1.015783, 14.968182], [1.385528, 15.323561], [2.749993, 15.409525], [3.638259, 15.56812], [3.723422, 16.184284], [4.27021, 16.852227], [4.267419, 19.155265], [5.677566, 19.601207], [8.572893, 21.565661], [11.999506, 23.471668], [13.581425, 23.040506], [14.143871, 22.491289], [14.8513, 22.86295]]]]}, {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [2.691702, 6.258817, 15.903247, 23.471668], "geometry": {"type": "Polygon", "coordinates": [[[2.691702, 6.258817], [2.749063, 7.870734], [2.723793, 8.506845], [2.912308, 9.137608], [3.220352, 9.444153], [3.705438, 10.06321], [3.60007, 10.332186], [3.797112, 10.734746], [3.572216, 11.327939], [3.61118, 11.660167], [3.680634, 12.552903], [3.967283, 12.956109], [4.107946, 13.531216], [4.368344, 13.747482], [5.443058, 13.865924], [6.445426, 13.492768], [6.820442, 13.115091], [7.330747, 13.098038], [7.804671, 13.343527], [9.014933, 12.826659], [9.524928, 12.851102], [10.114814, 13.277252], [10.701032, 13.246918], [10.989593, 13.387323], [11.527803, 13.32898], [12.302071, 13.037189], [13.083987, 13.596147], [13.318702, 13.556356], [13.995353, 12.461565], [14.181336, 12.483657], [14.577178, 12.085361], [14.468192, 11.904752], [14.415379, 11.572369], [13.57295, 10.798566], [13.30867, 10.160362], [13.1676, 9.640626], [12.955468, 9.417772], [12.753672, 8.717763], [12.218872, 8.305824], [12.063946, 7.799808], [11.839309, 7.397042], [11.745774, 6.981383], [11.058786, 6.444427], [10.497375, 6.055358], [10.118277, 7.03877], [9.522706, 6.453482], [9.233163, 6.444491], [8.757533, 5.479666], [8.500288, 4.771983], [7.462108, 4.412108], [7.082596, 4.464689], [6.698072, 4.240594], [5.898173, 4.262453], [5.362805, 4.887971], [5.033574, 5.611802], [4.325607, 6.270651], [3.57418, 6.2583], [2.691702, 6.258817]]]]}, {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [8.488816, 1.727673, 16.012852, 12.859396], "geometry": {"type": "Polygon", "coordinates": [[[14.495787, 12.859396], [14.89336, 12.21905], [14.960152, 11.555574], [14.923565, 10.891325], [15.467873, 9.982337], [14.909354, 9.992129], [14.627201, 9.920919], [14.171466, 10.021378], [13.954218, 9.549495], [14.544467, 8.965861], [14.97996, 8.796104], [15.120866, 8.38215], [15.436092, 7.692812], [15.27946, 7.421925], [14.776545, 6.408498], [14.53656, 6.226959], [14.459407, 5.451761], [14.558936, 5.030598], [14.478372, 4.732605], [14.950593, 4.210389], [15.03622, 3.851367], [15.405396, 3.33501], [15.862732, 3.013537], [15.907381, 2.557389], [16.012852, 2.26764], [15.940919, 1.727673], [15.146342, 1.964015], [14.337813, 2.227875], [13.075822, 2.267097], [12.951334, 2.321616], [12.35938, 2.192812], [11.751665, 2.326758], [11.276449, 2.261051], [9.64915

8, 2.283866], [9.795196, 3.073404], [9.404367, 3.734527], [8.948116, 3.904129], [8.744924, 4.352215], [8.488816, 4.495617], [8.500288, 4.771983], [8.757533, 5.479666], [9.233163, 6.444491], [9.522706, 6.453482], [10.118277, 7.03877], [10.497375, 7.055358], [11.058788, 6.644427], [11.745774, 6.981383], [11.839309, 7.397042], [12.063946, 7.799808], [12.218872, 8.305824], [12.753672, 8.717763], [12.955468, 9.417772], [13.1676, 9.640626], [13.308676, 10.160362], [13.57295, 10.798566], [14.415379, 11.572369], [14.468192, 11.904752], [14.577178, 12.085361], [14.181336, 12.483657], [14.213531, 12.802035], [14.495787, 12.859396]]]]], {"type": "Feature", "properties": {"name": "□□□□", "bbox": [-0.049785, 5.928837, 1.865241, 11.018682], "geometry": {"type": "Polygon", "coordinates": [[[0.899563, 10.997339], [0.772336, 10.470808], [1.077795, 10.175607], [1.425061, 9.825395], [1.463043, 9.334624], [1.664478, 9.12859], [1.618951, 6.832038], [1.865241, 6.142158], [1.060122, 5.928837], [0.836931, 6.279979], [0.570384, 6.914359], [0.490957, 7.411744], [0.712029, 8.312465], [0.461192, 8.677223], [0.365901, 9.465004], [0.36758, 10.191213], [-0.049785, 10.706918], [0.023803, 11.018682], [0.899563, 10.997339]]]]], {"type": "Feature", "properties": {"name": "□□□", "bbox": [-3.24437, 4.710462, 1.060122, 11.098341], "geometry": {"type": "Polygon", "coordinates": [[[0.023803, 11.018682], [-0.049785, 10.706918], [0.36758, 10.191213], [0.365901, 9.465004], [0.461192, 8.677223], [0.712029, 8.312465], [0.490957, 7.411744], [0.570384, 6.914359], [0.836931, 6.279979], [1.060122, 5.928837], [-0.049785, 5.928837], [-1.063625, 5.000548], [-1.964707, 4.710462], [-2.856125, 4.994476], [-2.810701, 5.389051], [-3.24437, 4.710462], [-2.983585, 7.379705], [-2.56219, 8.219628], [-2.827496, 9.642461], [-2.963896, 10.395335], [-2.940409, 10.96269], [-1.203358, 11.009819], [-0.761576, 10.93693], [-0.438702, 11.098341], [0.023803, 11.018682]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [-8.60288, 4.338288, -2.56219, 10.524061], "geometry": {"type": "Polygon", "coordinates": [[[-8.029944, 10.206535], [-7.89959, 10.297382], [-7.622759, 10.147236], [-6.850507, 10.138994], [-6.666461, 10.430811], [-6.493965, 10.411303], [-6.205223, 10.524061], [-6.050452, 10.096361], [-5.816926, 10.222555], [-5.404342, 10.370737], [-4.954653, 10.152714], [-4.779884, 9.821985], [-4.330247, 9.610835], [-3.980449, 9.862344], [-3.511899, 9.900326], [-2.827496, 9.642461], [-2.56219, 8.219628], [-2.983585, 7.379705], [-3.24437, 6.250472], [-2.810701, 5.389051], [-2.856125, 4.994476], [-3.311084, 4.984296], [-4.00882, 5.179813], [-4.649917, 5.168264], [-5.834496, 4.993701], [-6.528769, 4.705088], [-7.518941, 4.338288], [-7.712159, 4.364566], [-7.635368, 5.188159], [-7.539715, 5.313345], [-7.570153, 5.707352], [-7.993693, 6.12619], [-8.311348, 6.193033], [-8.60288, 6.467564], [-8.385452, 6.911801], [-8.485446, 7.395208], [-8.439298, 7.686043], [-8.280703, 7.68718], [-8.221792, 8.123329], [-8.299049, 8.316444], [-8.203499, 8.455453], [-7.8321, 8.575704], [-8.079114, 9.376224], [-8.309616, 9.789532], [-8.229337, 10.12902], [-8.029944, 10.206535]]]]], {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [-15.130311, 7.309037, -7.8321, 12.586183], "geometry": {"type": "Polygon", "coordinates": [[[-13.700476, 12.586183], [-13.217818, 12.57874], [-12.499051, 12.33209], [-12.278599, 12.35444], [-12.203565, 12.465648], [-11.658301, 12.386583], [-11.513943, 12.442988], [-11.456169, 12.076834], [-11.297574, 12.077971], [-11.036556, 12.211245], [-10.87083, 12.177887], [-10.593224, 11.923975], [-10.165214, 11.844084], [-9.890993, 12.060479], [-9.567912, 12.194243], [-9.327616, 12.334286], [-9.127474, 12.30806], [-8.905265, 12.088358], [-8.786099, 11.812561], [-8.376305, 11.939646], [-8.581305, 11.136246], [-8.620321, 10.810891], [-8.407311, 10.909257], [-8.282357, 10.792597], [-8.335377, 10.494812], [-8.029944, 10.206535], [-8.229337, 10.12902], [-8.309616, 9.789532], [-8.079114, 9.376224], [-7.8321, 8.575704], [-8.203499, 8.455453], [-8.299049, 8.316444], [-8.203499, 8.316444], [-8.221792, 8.123329], [-8.280703, 7.68718], [-8.439298, 7.686043], [-8.722124, 7.711674], [-8.926065, 7.309037], [-9.020876, 7.313921], [-9.403348, 7.526905], [-9.33728, 7.928534], [-9.755342, 8.541055], [-10.016567, 8.428504], [-10.230094, 8.406206], [-10.505477, 8.348896], [-10.494315, 8.715541], [-10.65477, 8.977178], [-10.622395, 9.26791], [-10.839152, 9.688246], [-11.117481, 10.045873], [-11.912777, 10.046984], [-12.150338, 9.858572], [-12.425929, 9.835834], [-12.596719, 9.620188], [-12.711958, 9.342712], [-13.24655, 8.903049], [-13.685154, 9.494744], [-14.074045, 9.886167], [-14.330076, 10.01572], [-14.579699, 10.214467], [-14.693232, 10.656301], [-14.839554, 10.876572], [-15.130311, 11.040412], [-14.685687, 11.527824], [-14.382192, 11.509272], [-14.121406, 11.67717], [-13.9008, 11.678719], [-13.743161, 11.811269], [-13.828272, 12.142644], [-13.718744, 12.247186], [-13.700476, 12.586183]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [-16.677452, 11.040412, -13.700476, 12.62817], "geometry": {"type": "Polygon", "coordinates": [[[-16.677452, 12.384852], [-16.147717, 12.547762], [-15.816574, 12.515567], [-15.548477, 12.62817], [-13.700476, 12.586183], [-13.718744, 12.247186], [-13.828272, 12.142644], [-13.743161, 11.811269], [-13.9008, 11.678719], [-14.121406, 11.67717], [-14.382192, 11.509272], [-14.685687, 11.527824], [-15.130311, 11.040412], [-15.66418, 11.458474], [-16.085214, 11.524594], [-16.314787, 11.806515], [-16.308947, 11.958702], [-16.613838, 12.170911], [-16.677452, 12.384852]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [-11.438779, 4.355755, -7.539715, 8.541055], "geometry": {"type": "Polygon", "coordinates": [[[-8.439298, 7.686043], [-8.485446, 7.395208], [-8.385452, 6.911801], [-8.60288, 6.467564], [-8.311348, 6.193033], [-7.993693, 6.12619], [-7.570153, 5.707352], [-7.539715, 5.313345], [-7.635368, 5.188159], [-7.712159, 4.364566], [-7.974107, 4.355755], [-9.004794, 4.832419], [-9.91342, 5.593561], [-10.765384, 6.140711], [-11.438779, 6.785917], [-11.199802, 7.105846], [-11.146704, 7.396706], [-10.695595, 7.939464], [-10.230094, 8.406206], [-10.016567, 8.428504], [-9.755342, 8.541055], [-9.33728, 7.928534], [-9.403348, 7.526905], [-9.208786, 7.313921], [-8.926065, 7.309037], [-8.722124, 7.711674], [-8.439298, 7.686043]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [-13.24655, 6.785917, -10.230094, 10.046984], "geometry": {"type": "Polygon", "coordinates": [[[-13.24655, 8.903049], [-12.711958, 9.342712], [-12.596719, 9.620188], [-12.425929, 9.835834], [-12.150338, 9.858572], [-11.912777, 10.046984], [-11.117481, 10.045873], [-10.839152, 9.688246], [-10.622395, 9.26791], [-10.65477, 8.977178], [-10.494315, 8.715541], [-10.505477, 8.348896], [-10.230094, 8.406206], [-10.016567, 8.428504], [-11.146704, 7.396706], [-11.199802, 7.105846], [-11.438779, 6.785917], [-11.708195, 6.860098], [-12.428099, 7.262942], [-12.949049, 7.798646], [-13.124025, 8.163946], [-13.24655, 8.903049]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□□□", "bbox": [-5.470565, 9.610835, 2.177108, 15.116158], "geometry": {"type": "Polygon", "coordinates": [[[-5.440342, 10.370737], [-5.470565, 10.95127], [-5.197843, 11.375146], [-5.220942, 11.713859], [-4.427166, 12.542646], [-4.280405, 13.228444], [-4.006391, 13.472485], [-3.522803, 13.337662], [-3.103707, 13.541267], [-2.967694, 13.79815], [-2.191825, 14.246418], [-2.001035, 14.559008], [-1.066363, 14.973815], [-0.515854, 15.116158], [-0.266257, 14.924309], [0.374892, 14.928908], [0.295646, 14.444235], [0.429928, 13.98873], [0.993046, 13.33575], [1.024103, 12.851262], [2.177108, 12.625018], [2.154474, 11.94015], [1.935986, 11.64115], [1.447178, 11.547719], [1.24347, 11.110511], [0.899563, 10.997339], [0.023803, 11.018682], [-0.438702, 11.098341], [-0.761576, 10.93693], [-1.203358, 11.009819], [-2.940409, 10.96269], [-2.963896, 10.395335], [-2.827496, 9.642461], [-3.511899, 9.900326], [-3.980449, 9.862344], [-4.330247, 9.610835], [-4.779884, 9.821985], [-4.954653, 10.152714], [-5.404342, 10.370737]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□□□□□", "bbox": [14.459407, 2.26764, 27.374226, 11.142395], "geometry": {"type": "Polygon", "coordinates": [[27.374226, 5.233944], [27.044065, 5.127853], [26.402761, 5.150875], [25.650455, 5.256088], [25.278798, 5.170408], [25.128833, 4.927245], [24.805029, 4.897247], [24.410531, 5.108784], [23.297214, 4.609693], [22.84148, 4.710126], [22.704124, 4.633051], [22.405124, 4.02916], [21.659123, 4.244342], [20.927591, 4.322786], [20.290679, 4.691678], [19.467784, 5.031528], [18.932312, 4.709506], [18.542982, 4.201785], [18.453065, 3.504386], [17.8099, 3.560196], [17.133042, 3.728197], [16.537058, 3.198255], [16.012852, 2.26764], [15.907381, 2.557389], [15.862732, 3.013537], [15.405396, 3.335301], [15.03622, 3.851367], [14.950953, 4.210389], [14.478372, 4.732605], [14.558936, 5.030598], [14.459407, 5.451761], [14.536566, 6.226959], [14.776545, 6.408498], [15.27946, 7.421925], [16.106232, 7.497088], [16.290562, 7.754307], [16.456185, 7.73474], [16.705988, 7.508328], [17.96493, 7.890914], [18.389555, 8.281304], [18.911022, 8.630895], [18.81201, 8.982915], [19.094008, 9.074847], [20.059685, 9.012706], [21.000868, 9.475985], [21.723822, 10.567056], [22.231129, 10.971889], [22.864165, 11.142395], [22.977544, 10.714463], [23.554304, 10.089255], [23.55725, 9.681218], [23.394779, 9.265068], [23.459013, 8.954286], [23.805813, 8.666319], [24.567369, 8.229188], [25.114932, 7.825104], [25.124131, 7.500085], [25.796648, 6.979316], [26.213418, 6.546603], [26.465909, 5.946717], [27.213409, 5.550953], [27.374226, 5.233944]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [14.459407, 0.993773, -5.037987, 18.453065], "geometry": {"type": "Polygon", "coordinates": [[18.453065, 3.504386], [18.393792, 2.900443], [18.094276, 2.365722], [17.898835, 1.741832], [17.774192, 0.855659], [17.82654, 0.288923], [17.663553, -0.058084], [17.638645, -0.424832], [17.523716, -0.74383], [16.865307, -1.225816], [16.407092, -1.740927], [15.972803, -2.712392], [16.000629, -3.535133], [15.75354, -3.855165], [15.170992, -4.343507], [14.582604, -4.970239], [14.209035, -4.793092], [14.144956, -4.510099], [13.600235, -4.500138], [13.25824, -4.882957], [12.995517, -4.781103], [12.62076, -4.438023], [12.318608, -4.60623], [11.914963, -5.037987], [11.093773, -3.978827], [11.855122, -3.426871], [11.478039, -2.765619], [11.820964, -2.514161], [12.495703, -2.391688], [12.575284, -1.948511], [13.109619, -2.42874], [13.992407, -2.478005], [14.29921, -1.998276], [14.425456, -1.333407], [14.316418, -0.552627], [13.843321, 0.038758], [14.276266, 1.19693], [14.026669, 1.395677], [13.282631, 1.314184], [13.003114, 1.830896], [13.075822, 2.267097], [14.337813, 2.227875], [15.146342, 1.964015], [15.940919, 1.727673], [16.012852, 2.26764], [16.537058, 3.198255], [17.133042, 3.728197], [17.8099, 4.691678], [18.453065, 3.504386]]]]], {"type": "Feature", "properties": {"name": "□□□□□", "bbox": [8.797996, -3.97



```

827,14.425456,2.326758],"geometry":{"type":"Polygon","coordinates": [[[11.276449,2.261051],[11.751665,2.326758],[12.3593
8,2.192812],[12.951334,2.321616],[13.075822,2.267097],[13.003114,1.830896],[13.282631,3.141841],[14.026669,1.395677],[14
.276266,1.19693],[13.843321,0.038758],[14.316418,-0.552627],[14.425456,-1.333407],[14.29921,-1.998276],[13.992407,-2.470
805],[13.109619,-2.42874],[12.575284,-1.948511],[12.495703,-2.391688],[11.820964,-2.514161],[11.478039,-2.765619],[11.85
5122,-3.426871],[11.093773,-3.978827],[10.066135,-2.969483],[9.405245,-2.144313],[8.797996,-1.111301],[8.830087,-0.77907
4],[9.04842,-0.459351],[9.291351,0.268666],[9.492889,1.01012],[9.830284,1.067894],[11.285079,1.057662],[11.276449,2.2610
51]]]},"type":"Feature","properties":{"name":"\u2500\u2500\u2500\u2500\u2500\u2500","bbox":["30.50554,34.59339],[36.61175,34.201789],[36.06646,33.824912],[35.821101,33.277426]]]},"type":"MultiPolygon","
coordinates": [[[12.995517,-4.781103],[12.631612,-4.991271],[12.468004,-5.248362],[12.436688,-5.684304],[12.182337,-5.78
9931],[11.914963,-5.037987],[12.318608,-4.60623],[12.62076,-4.438023],[12.995517,-4.781103],[[12.322432,-6.100092],[1
2.735171,-5.965682],[13.024869,-5.984389],[13.375597,-5.864241],[16.326528,-5.87747],[16.57318,-6.626465],[16.860191,-7.
222298],[17.089996,-7.545689],[17.47297,-8.068551],[18.134222,-7.987678],[18.464176,-7.847014],[19.016752,-7.988246],[19
.166613,-7.738184],[19.417502,-7.155429],[20.037723,-7.116361],[20.091622,-6.94309],[20.601823,-6.939318],[20.514748,-7.
299606],[21.728111,-7.290872],[21.746456,-7.920085],[21.949131,-8.305901],[21.801801,-8.908707],[21.875182,-9.523708],[2
2.208753,-9.894796],[22.155268,-11.084801],[22.402798,-10.993075],[22.837345,-11.017622],[23.456791,-10.867863],[23.9122
15,-10.926826],[24.017894,-11.237298],[23.904154,-11.722822],[24.079905,-12.191297],[23.930922,-12.565848],[24.016137,-1
2.911046],[21.933886,-12.988437],[21.887843,-16.08031],[22.562478,-16.898451],[23.215048,-17.523116],[21.377176,-17.9306
36],[18.956187,-17.789095],[18.263309,-17.309951],[14.209707,-17.353101],[14.058501,-17.423381],[13.462362,-16.971212],[
12.814081,-16.941343],[12.215461,-17.111668],[11.734199,-17.301889],[11.640096,-16.673142],[11.778537,-15.793816],[12.12
3581,-14.878316],[12.175619,-14.49144],[12.500095,-13.5477],[12.738479,-13.137906],[13.312914,-12.48363],[13.633721,-12
.038645],[13.738728,-11.297863],[13.686379,-10.731076],[13.387328,-10.373578],[13.120988,-9.766897],[12.87537,-9.166934],
[12.929061,-8.959091],[13.236433,-8.562629],[12.93304,-7.596539],[12.728298,-6.927122],[12.227347,-6.294448],[12.322432
,-6.100092]]]},"type":"Feature","properties":{"name":"\u2500\u2500\u2500\u2500\u2500\u2500","bbox":["29.024926,-4.499983,30.75224,-2.348487],"geom
etry":{"type":"Polygon","coordinates": [[[30.469674,-2.413855],[30.52766,-2.80762],[30.74301,-3.03431],[30.75224,-3.35931
],[30.50554,-3.56858],[30.11632,-4.09012],[29.753512,-4.452389],[29.339998,-4.499983],[29.276384,-3.293907],[29.024926,-
2.839258],[29.632176,-2.917858],[29.938359,-2.348487],[30.469674,-2.413855]]]},"type":"Feature","properties":{"name":"
\u2500\u2500\u2500\u2500\u2500","bbox":["34.265433,29.501326,35.836397,33.277426],"geometry":{"type":"Polygon","coordinates": [[[35.719918,32.70
9192],[35.545665,32.393992],[35.18393,32.532511],[34.974641,31.866582],[35.225892,31.754341],[34.970507,31.616778],[34.9
27408,31.353435],[35.397561,31.489086],[35.420918,31.100066],[34.922603,29.501326],[34.823243,29.761081],[34.26544,31.21
936],[34.265433,31.219357],[34.265433,31.219361],[34.556372,31.548824],[34.488107,31.605539],[34.752587,32.072926],[34.9
55417,32.827376],[35.098457,33.080539],[35.126053,33.0909],[35.460709,33.08904],[35.552793,33.264275],[35.821101,33.2774
26],[35.836397,32.868123],[35.700798,32.716014],[35.719918,32.709192]]]},"type":"Feature","properties":{"name":"\u2500\u2500\u2500
"},"bbox":["33.126053,33.08904,36.61175,34.64914],"geometry":{"type":"Polygon","coordinates": [[[35.821101,33.277426],[35.
552797,33.264275],[35.460709,33.08904],[35.126053,33.0909],[35.482207,33.90545],[35.979592,34.610058],[35.998403,34.6449
14],[36.448194,34.593395],[36.61175,34.201789],[36.06646,33.824912],[35.821101,33.277426]]]},"type":"Feature","propert
ies":{"name":"\u2500\u2500\u2500\u2500\u2500\u2500","bbox":["43.254187,-25.601434,50.476537,-12.040557],"geometry":{"type":"Polygon","coordinates
": [[[49.543519,-12.469833],[49.808981,-12.895285],[50.956511,-13.555761],[50.217431,-14.758789],[50.476537,-15.226512],[5
0.377111,-15.706069],[50.200275,-16.000263],[49.806006,-15.414253],[49.672607,-15.710204],[49.863344,-16.451037],[49.774
564,-16.875042],[49.498612,-17.106036],[49.435619,-17.953064],[49.041792,-19.118781],[48.548541,-20.496888],[47.930749,-
22.391501],[47.547723,-23.781959],[47.095761,-24.94163],[46.282478,-25.178463],[45.409508,-25.601434],[44.833574,-25.346
101],[44.03972,-24.988345],[43.763768,-24.406677],[43.697778,-23.574116],[43.345654,-22.776904],[43.254187,-22.057413],[
43.433298,-21.336475],[43.893683,-21.163
```



```

94.94651,-12.040557],[49.543519,-12.469833]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox":[34.927408,31.3
53435,35.545665,32.532511],"geometry":{"type":"Polygon","coordinates":[[[[35.397561,31.489086],[34.927408,31.353435],[34.
970507,31.617778],[35.225892,31.754341],[34.974641,31.866582],[35.18393,32.532511],[35.545665,32.393992],[35.545252,31.7
82505],[35.397561,31.489086]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox":[-16.841525,13.130284,-13.84496
3,13.876492],"geometry":{"type":"Polygon","coordinates":[[[[-16.713729,13.594959],[-15.624596,13.623587],[-15.39877,13.86
0369],[-15.081735,13.876492],[-14.687031,13.630357],[-14.376714,13.62568],[-14.046992,13.794068],[-13.844963,13.505042],
[-14.277702,13.280585],[-14.712197,13.298207],[-15.141163,13.509512],[-15.511813,13.27857],[-15.691003,13.270353],[-15.9
31296,13.130284],[-16.841525,13.151394],[-16.713729,13.594959]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox"
[7.524482,30.307556,11.488787,37.349994],"geometry":{"type":"Polygon","coordinates":[[[[9.48214,30.307556],[9.055603,32.
102692],[8.439103,32.506285],[8.430473,32.748337],[7.612642,33.344115],[7.524482,34.097376],[8.140981,34.655146],[8.3763
68,35.479876],[8.217824,36.433177],[8.420964,36.946427],[9.509994,37.349994],[10.210002,37.230002],[10.18065,36.724038],
[11.028867,37.092103],[11.100026,36.899996],[10.600005,36.41],[10.593287,35.947444],[10.939519,35.698984],[10.807847,34.
833507],[10.149593,34.330773],[10.339659,33.785742],[10.856836,33.76874],[11.108501,33.293343],[11.488787,33.136996],[11.
432253,32.368903],[10.94479,32.08185],[10.636901,31.761421],[9.950225,31.37607],[10.056575,30.961831],[9.970017,30.539
325],[9.48214,30.307556]]]]],{"type":"Feature","properties":{"name":"□□□□□□□□"},"bbox":[-8.6844,19.057364,11.999506,37.11
8381],"geometry":{"type":"Polygon","coordinates":[[[[-8.6844,27.395744],[-8.665124,27.589479],[-8.66559,27.656426],[-8.67
4116,28.841289],[-7.7059228,29.579228],[-6.060632,29.7317],[-5.242129,30.000443],[-4.859646,30.501188],[-3.690441,30.8969
52],[-3.647498,31.637294],[-3.06898,31.724498],[-2.616605,32.094346],[-1.307899,32.262889],[-1.124551,32.651522],[-1.388
049,32.864015],[-1.733455,33.919713],[-1.792986,34.527919],[-2.169914,35.168396],[-1.208603,35.714849],[-0.127454,35.888
662],[0.503877,36.301273],[1.466919,36.605647],[3.161699,36.783905],[4.815758,36.865037],[5.32012,36.716519],[6.26182,37.
110655],[7.330385,37.118381],[7.737078,36.885708],[8.420964,36.946427],[8.217824,36.433177],[8.376368,35.479876],[8.140
981,34.655146],[7.524482,34.097376],[7.612642,33.344115],[8.430473,32.748337],[8.439103,32.506285],[9.055603,32.102692],
[9.48214,30.307556],[9.805634,29.424638],[9.859998,28.95999],[9.683885,28.144174],[9.756128,27.688259],[9.629056,27.1409
53],[9.716286,26.512206],[9.319411,26.094325],[9.910693,25.365455],[9.948261,24.936954],[10.303847,24.379313],[10.771364
,24.562532],[11.560669,24.097909],[11.999506,23.471668],[8.572893,21.565661],[5.677566,19.601207],[4.267419,19.155265],[3.
3158133,19.057364],[3.146661,19.693579],[2.638588,19.85623],[2.060991,20.142233],[1.823228,20.610809],[-1.550055,22.792
666],[-4.923337,24.974574],[-6.8844,27.395744]]]]],{"type":"Feature","properties":{"name":"□□□□□"},"bbox":[34.927408,31.3
97495,39.195468,33.378686],"geometry":{"type":"Polygon","coordinates":[[[[35.545665,32.393992],[35.719918,32.709192],[36.
834062,32.312938],[38.792341,33.378686],[39.195468,32.161009],[39.004886,32.010217],[37.002166,31.508413],[37.998849,30.
5085],[37.66812,30.338665],[37.503582,30.003776],[36.744058,29.865283],[36.501214,29.505254],[36.068941,29.197495],[34.9
56037,29.356555],[34.922603,29.501326],[35.420918,31.100066],[35.397561,31.489086],[35.545252,31.782505],[35.545665,32.3
93992]]]]],{"type":"Feature","properties":{"name":"□□□□□□□□□□□□□□□□"},"bbox":[51.579519,22.496948,56.396847,26.055464],
"geometry":{"type":"Polygon","coordinates":[[[[51.579519,24.245497],[51.757441,24.294073],[51.794389,24.019826],[52.57708
1,24.177439],[53.404007,24.151317],[54.008001,24.121758],[54.693024,24.797892],[55.439025,25.439145],[56.070821,26.05546
4],[56.261042,25.714606],[56.396847,24.924732],[55.886233,24.920831],[55.804119,24.269604],[55.981214,24.130543],[55.528
632,23.933604],[55.525841,23.524869],[55.234489,23.110933],[55.208341,22.70833],[55.006803,22.496948],[52.000733,23.0011
54],[51.617708,24.014219],[51.579519,24.245497]]]]],{"type":"Feature","properties":{"name":"□□□□□"},"bbox":[51.579519,22.496948,56.396847,26.055464],
56331,51.6067,26.114582],"geometry":{"type":"Polygon","coordinates":[[[[50.810108,24.754743],[50.743911,25.482424],[51.01
3352,26.006992],[51.286462,26.114582],[51.589079,25.801113],[51.6067,25.21567],[51.389608,24.627386],[51.112415,24.55633
1],[50.810108,24.754743]]]]],{"type":"Feature","properties":{"name":"□□□□□"},"bbox":[46.568713,28.526063,48.416094,30.059
07],"geometry":{"type":"Polygon","coordinates":[[[[47.974519,29.975819],[48.183189,29.534477],[48.093943,29.306299],[48.4
16094,28.552004],[47.708851,28.526063],[47.459822,29.002519],[46.568713,29.099025],[47.302622,30.05907],[47.974
```

], [101.270026, 21.201652], [101.80312, 21.174367], [101.652018, 22.318199], [102.170436, 22.464753], [102.754896, 21.675137], [103.203861, 20.766562], [104.435, 20.758733], [104.822574, 19.886642], [104.183388, 19.624668], [103.896532, 19.265181], [105.094598, 18.666975], [105.925762, 17.485315], [106.556008, 16.604284], [107.312706, 15.908538], [107.564525, 15.202173], [107.382727, 14.202441]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [92.303234, 9.93296, 101.180005, 28.335945], "geometry": {"type": "Polygon", "coordinates": [[ [100.115988, 20.41785], [99.543309, 20.186598], [98.959676, 19.752981], [98.253724, 19.708203], [97.797783, 18.62708], [97.375896, 18.445438], [97.859123, 17.567946], [98.493761, 16.837836], [98.903348, 16.177824], [98.537376, 15.308497], [98.192074, 15.123703], [98.430819, 14.622028], [99.097755, 13.827503], [99.212012, 13.269294], [99.196354, 12.804748], [99.587286, 11.892763], [99.038121, 10.960546], [98.553551, 9.93296], [98.457174, 10.675266], [98.764546, 11.441292], [98.428339, 12.032987], [98.509574, 13.122378], [98.103604, 13.64046], [97.777732, 14.837286], [97.597072, 16.100568], [97.16454, 16.928734], [96.505769, 16.427241], [95.369352, 15.71439], [94.808405, 15.803454], [94.188804, 16.037936], [94.533486, 17.27724], [94.324817, 18.213514], [93.540988, 19.366493], [93.663255, 19.726962], [93.078278, 19.855145], [92.368554, 20.670883], [92.303234, 21.475485], [92.652257, 21.324048], [92.672721, 22.041239], [93.166128, 22.27846], [93.060294, 22.703111], [93.286327, 23.043658], [93.325188, 24.078556], [94.106742, 23.850741], [94.552658, 24.675238], [94.603249, 25.162495], [95.155153, 26.001307], [95.124768, 26.573572], [96.419366, 27.264589], [97.133999, 27.083774], [97.051989, 27.699059], [97.402561, 27.882536], [97.327114, 28.261583], [97.91198, 28.335945], [98.246231, 27.747221], [98.68269, 27.508812], [98.712094, 26.743536], [98.671838, 25.918703], [97.724609, 25.083637], [97.60472, 23.897405], [98.660262, 24.063286], [98.898749, 23.142722], [99.531992, 22.949039], [99.240899, 22.118314], [99.98348, 21.742937], [100.416538, 21.558839], [101.150033, 21.849984], [101.180005, 21.436573], [100.329101, 20.786122], [100.115988, 20.41785]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [102.170436, 8.59976, 109.33527, 23.352063], "geometry": {"type": "Polygon", "coordinates": [[ [104.334335, 10.486544], [105.199915, 10.88931], [106.24967, 10.961812], [105.810524, 11.567615], [107.491403, 12.337206], [107.614548, 13.535531], [107.382727, 14.202441], [107.564525, 15.202173], [107.312706, 15.908538], [106.556008, 16.604284], [105.925762, 17.485315], [105.094598, 18.666975], [103.896532, 19.265181], [104.183388, 19.624668], [104.822574, 19.886642], [104.435, 20.758733], [103.203861, 20.766562], [102.754896, 21.675137], [102.170436, 22.464753], [102.706992, 22.708795], [103.504515, 22.703757], [104.476858, 22.81915], [105.329209, 23.352063], [105.811247, 22.976892], [106.725403, 22.794268], [106.567273, 22.218205], [107.04342, 21.811899], [108.05018, 21.55238], [106.715068, 20.696851], [105.881682, 19.75205], [105.662006, 19.058165], [106.426817, 18.004121], [107.361954, 16.697457], [108.269495, 16.079742], [108.877107, 15.276691], [109.33527, 14.266028], [109.200136, 11.666859], [108.36613, 11.008321], [107.220929, 10.364484], [106.405113, 9.53084], [105.158264, 8.59976], [104.795185, 9.241038], [105.076202, 9.918491], [104.334335, 10.486544]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [124.265625, 37.669071, 130.780007, 42.985387], "geometry": {"type": "MultiPolygon", "coordinates": [[ [ [130.780004, 42.220008], [130.780005, 42.22001], [130.780007, 42.220007], [130.780004, 42.220008]]], [ [ [130.64, 42.395024], [130.64, 42.395], [130.77992, 42.22001], [130.400031, 42.280004], [129.965949, 41.941368], [129.667362, 41.601104], [129.705189, 40.882828], [129.188115, 40.661808], [129.0104, 40.485436], [128.633368, 40.189847], [127.967414, 40.025413], [127.533436, 39.75685], [127.50212, 39.32393], [127.385434, 39.213472], [127.783343, 39.050898], [128.349716, 38.612243], [128.205746, 38.370397], [127.780035, 38.304536], [127.073309, 38.256115], [126.68372, 37.804773], [126.237339, 37.840378], [126.174759, 37.749686], [125.689104, 37.94001], [125.568439, 37.752089], [125.27533, 37.669071], [125.240087, 37.857224], [124.981033, 37.948821], [124.712161, 38.108346], [124.985994, 38.548474], [125.221949, 38.665857], [125.132859, 38.848559], [125.38659, 39.387958], [125.321116, 39.551385], [124.737482, 39.660344], [124.265625, 39.928493], [125.079942, 40.569824], [126.182045, 41.107336], [126.869083, 41.816569], [127.343783, 41.503152], [128.208433, 41.466772], [128.052215, 41.994285], [129.596669, 42.424982], [129.994267, 42.985387], [130.64, 42.395024]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [126.174759, 37.749686], "geometry": {"type": "Polygon", "coordinates": [[ [126.174759, 37.749686], [126.237339, 37.840378], [126.68372, 37.804773], [127.073309, 38.256115], [127.780035, 38.304536], [128.205746, 38.370397], [128.349716, 38.612243], [129.21292, 37.432392], [129.46045, 36.784189], [129.468304, 35.632141], [129.091377, 35.082484], [128.18585, 34.890377], [127.386519, 34.475674], [126.485748, 34.390046], [126.37392, 34.93456], [126.559231, 35.684541], [126.117398, 36.725485], [126.860143, 36.893924], [126.174759, 37.749686]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [87.751264, 41.59741, 119.772824, 52.047366], "geometry": {"type": "Polygon", "coordinates": [[ [87.751264, 49.297198], [88.805567, 49.470521], [90.713667, 50.331812], [92.234712, 50.802171], [93.10421, 50.49529], [94.147566, 50.480537], [94.815949, 50.013433], [95.81402, 49.97746], [97.25976, 49.72605], [98.231762, 50.422401], [97.82574, 51.010995], [98.861491, 52.047366], [99.981732, 51.634006], [100.88948, 51.516856], [102.06521, 51.25991], [102.25589, 50.51056], [103.676545, 50.89966], [104.62158, 50.27532], [105.886591, 50.406019], [106.888804, 50.274296], [107.868176, 49.793705], [108.475167, 49.282548], [109.402449, 49.292961], [110.662011, 49.130128], [111.581231, 49.377968], [112.89774, 49.543565], [114.362456, 50.248303], [114.96211, 50.140247], [115.485695, 49.805177], [116.678801, 49.888531], [116.191802, 49.134598], [115.485282, 48.135383], [115.742837, 47.726545], [116.308953, 47.85341], [117.295507, 47.697709], [118.064143, 48.06673], [118.866574, 47.74706], [119.772824, 47.048059], [119.66327, 46.69268], [118.874326, 46.805412], [117.421701, 46.672733], [116.717868, 46.388202], [115.985096, 45.727235], [114.460332, 45.339817], [113.463907, 44.808893], [112.436062, 45.011646], [111.873306, 45.102079], [111.348377, 44.457442], [111.66737, 44.073176], [111.829588, 43.743118], [111.129682, 43.406834], [110.412103, 42.871234], [109.243596, 42.519446], [107.744773, 42.481516], [106.129316, 42.134328], [104.964994, 41.59741], [104.522282, 41.908347], [103.312278, 41.907468], [101.83304, 42.514873], [100.845866, 42.663804], [99.515817, 42.524691], [97.451757, 42.74889], [96.349396, 42.725635], [95.762455, 43.319449], [95.306875, 44.241331], [94.688929, 44.352332], [93.480734, 44.975472], [92.133891, 45.115076], [90.94554, 45.06073], [90.585768, 45.719716], [90.970809, 46.888146], [90.280826, 47.693549], [88.854298, 48.069082], [88.013832, 48.599463], [87.751264, 49.297198]]]]], {"type": "Feature", "properties": {"name": "□□□□", "bbox": [68.176645, 7.965535, 97.402561, 35.49401], "geometry": {"type": "Polygon", "coordinates": [[ [97.327114, 28.261583], [97.402561, 27.882536], [97.051989, 27.699059], [97.133999, 27.083774], [96.419366, 27.264589], [95.124768, 26.573572], [95.155153, 26.001307], [94.603249, 25.162495], [94.552658, 24.675238], [94.106742, 23.850741], [93.325188, 24.078556], [93.286327, 23.043658], [93.060294, 22.703111], [93.166128, 22.27846], [92.672721, 22.041239], [92.146035, 23.627499], [91.869928, 23.624346], [91.706475, 22.985264], [91.158963, 23.503527], [91.46773, 24.072639], [91.915093, 24.130414], [92.376202, 24.976693], [91.799596, 25.107442], [90.872211, 25.132601], [89.920693, 25.26975], [89.832481, 25.965082], [89.355094, 26.014407], [88.563049, 26.446526], [88.209789, 25.768066], [88.931554, 25.238692], [88.306373, 24.866079], [88.084422, 24.501657], [88.69994, 24.233715], [88.52977, 23.631142], [88.876312, 22.879146], [89.031961, 22.055708], [88.888766, 21.690588], [88.208497, 21.703172], [86.975704, 21.495562], [87.033169, 20.743308], [86.499351, 20.151638], [85.060266, 19.478579], [83.941006, 18.30201], [83.189217, 17.671221], [82.192792, 17.016636], [82.191242, 16.556664], [81.692719, 16.310219], [80.791999, 15.951972], [80.324896, 15.899185], [80.025069, 15.136415], [80.233274, 13.835771], [80.286294, 13.006261], [79.862547, 12.056215], [79.857999, 10.357275], [79.340512, 10.308854], [78.885345, 9.546136], [79.18972, 9.216544], [78.277941, 8.933047], [77.941165, 8.252959], [77.539898, 7.965535], [76.592979, 8.899276], [76.130061, 10.29963], [75.746467, 11.308251], [75.396101, 11.781245], [74.864816, 12.741936], [74.616717, 13.992583], [74.443859, 14.617222], [73.534199, 15.990652], [73.119909, 17.92857], [72.820909, 19.208234], [72.824475, 20.419503], [72.630533, 21.356009], [71.175272, 20.757441], [70.470459, 20.877331], [69.16413, 22.089298], [69.644928, 22.450775], [69.349597, 22.84318], [68.176645, 23.691965], [68.842599, 24.359134], [71.04324, 24.356524], [70.844699, 25.215102], [70.282873, 25.722229], [70.168927, 26.491872], [69.514393, 26.940966], [70.616496, 27.989196], [71.777666, 27.91318], [72.823752, 28.961592], [73.450638, 29.976413], [74.42138, 30.97981], [74.405929, 31.692639], [75.258642, 32.271105], [74.451559, 32.7649], [74.104294, 33.441473], [73.749948, 34.317699], [74.240203, 34.748887], [75.757061, 34.504923], [76.871722, 34.653544], [77.837451, 35.49401], [78.912269, 34.321936], [78.811086, 33.506198], [79.208892, 32.994395], [79.176129, 32.48378], [78.458446, 32.618164], [78.738894, 31.515906], [79.712367, 30.882715], [81.111256, 30.183481], [80.476721, 29.729865], [80.088425, 28.79447], [81.057203, 28.416095], [81.999887, 27.925479], [83.304249, 27.364506], [84.675018, 27.234901], [85.251779, 26.726188], [86.024393, 26.630985], [87.227472, 26.397898], [88.060238, 26.414615], [88.174804, 26.810405], [88.043133, 27.445819], [88.120441, 27.876542], [88.730326, 28.086865], [88.814248, 27.299316], [88.835643, 27.098966], [89.744528, 26.719403], [90.373275, 26.875724], [91.217513, 26.808648], [92.033484, 26.83831], [92.103712, 27.452614], [91.696657, 27.771742], [92.503119, 27.896876], [93.413348, 28.640629], [94.56599, 29.277438], [95.404802, 29.031717], [96.117679, 29.452802], [96.586591, 28.83098], [96.248833, 28.411031], [97.327114, 28.261583]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [88.084422, 20.670883, 92.672721, 26.446526], "geometry": {"type": "Polygon", "coordinates": [[ [92.672721, 22.041239], [92.652257, 21.324048], [92.303234, 21.475485], [92.368554, 20.670883], [92.082886, 21.192195], [92

.025215,21.70157],[91.834891,22.182936],[91.417087,22.765019],[90.496006,22.805017],[90.586957,22.392794],[90.272971,21.836368],[89.847467,22.039146],[89.70205,21.857116],[89.418863,21.966179],[89.031961,22.055708],[88.876312,22.879146],[88.52977,23.631142],[88.69994,24.233715],[88.084422,24.501657],[88.306373,24.866079],[88.931554,25.238692],[88.209789,25.768066],[88.563049,26.446526],[89.355094,26.014407],[89.832481,25.965082],[89.920693,25.26975],[90.872211,25.132601],[91.799596,25.147432],[92.376202,24.976693],[91.915093,24.130414],[91.46773,24.072639],[91.158963,23.503527],[91.706475,22.985264],[91.869928,23.624346],[92.146035,23.627499],[92.672721,22.041239]]]]],{"type":"Feature","properties":{"name":"","bbox":"[88.814248,26.719403,92.103712,28.296439]","geometry":{"type":"Polygon","coordinates":[[[91.696657,27.771742],[92.103712,27.452614],[92.033484,26.83831],[91.217513,26.808648],[90.373275,26.875724],[89.744528,26.719403],[88.835643,27.098966],[88.814248,27.299316],[89.47581,28.042759],[90.015829,28.296439],[90.730514,28.064954],[91.258854,28.040614],[91.696657,27.771742]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[80.088425,26.397898,88.174804,30.422771]","geometry":{"type":"Polygon","coordinates":[[[88.120441,27.876542],[88.043133,27.445819],[88.174804,26.810405],[88.060238,26.414615],[87.227472,26.397898],[86.024393,26.630985],[85.251779,26.726198],[84.675018,27.234901],[83.304249,27.364506],[81.999987,27.925479],[81.057203,28.416095],[80.088425,28.79447],[80.476721,29.729865],[81.111256,30.183481],[81.525804,30.422717],[82.327513,30.115268],[83.337115,29.463732],[83.898993,29.320226],[84.23458,28.839894],[85.011638,28.642774],[85.82332,28.203576],[86.954517,27.974262],[88.120441,27.876542]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[60.874248,23.691965,77.837451,37.133031]","geometry":{"type":"Polygon","coordinates":[[[77.837451,35.494011],[76.871722,34.653544],[75.757061,34.504923],[74.240203,34.748887],[73.749948,34.317699],[74.104294,33.441473],[74.451559,32.76491],[75.258642,32.271105],[74.405929,31.692639],[74.42138,30.979815],[73.450638,29.976413],[72.823752,28.961592],[71.777666,27.91318],[70.616496,27.989196],[69.514393,26.940966],[70.168927,26.491872],[70.282873,25.722229],[70.844699,25.215102],[71.04324,24.356524],[68.842599,24.359134],[68.176645,23.691965],[67.443667,23.944844],[67.145442,24.663611],[66.372828,25.425141],[64.530408,25.237039],[62.905701,25.218409],[61.497363,25.078237],[61.874187,26.239975],[63.316632,26.756532],[63.233898,27.217047],[62.755426,27.378923],[62.72783,28.259645],[61.771868,28.699334],[61.369309,29.303276],[60.874248,29.829239],[62.549857,29.318572],[63.550261,29.468331],[64.148002,29.340819],[64.350419,29.560031],[65.046862,29.472181],[66.346473,29.887943],[66.381458,30.738899],[66.938891,31.304911],[67.683394,31.303154],[67.792689,31.58293],[68.556932,31.71331],[68.926677,31.620189],[69.317764,31.901412],[69.262522,32.501944],[69.687147,33.105499],[70.323594,33.358533],[69.930543,34.02012],[70.881803,33.988856],[71.156773,34.348911],[71.115019,34.731261],[71.613076,35.453203],[71.498768,35.650563],[71.262348,36.074388],[71.846292,36.509942],[72.920025,36.720007],[74.067552,36.836176],[74.575893,37.020841],[75.158028,37.133031],[75.896897,36.666806],[76.192848,35.898403],[77.837451,35.494011]]]],{"type":"Feature","properties":{"name":"","bbox":"[60.52843,29.318572,75.158028,38.486282]","geometry":{"type":"Polygon","coordinates":[[[66.518607,37.362784],[67.075782,37.356144],[67.83,37.144994],[68.135562,37.023115],[68.859446,37.344336],[69.196273,37.151144],[69.518785,37.608997],[70.116578,37.588223],[70.270574,37.735165],[70.376304,38.138396],[70.806821,38.486282],[71.348131,38.258905],[71.239404,37.953265],[71.541918,37.905774],[71.448693,37.065645],[71.844638,36.738171],[72.193041,36.948288],[72.63689,37.047558],[73.260056,37.495257],[73.948696,37.421566],[74.980002,37.41999],[75.158028,37.133031],[74.575893,37.020841],[74.067552,36.836176],[72.920025,36.720007],[71.846292,36.509942],[71.262348,36.074388],[71.498768,35.650563],[71.613076,35.153203],[71.115019,34.733122],[71.156773,34.348911],[70.881803,33.988856],[69.930543,34.02012],[70.323594,33.358533],[69.687147,33.105499],[69.262522,32.501944],[69.317764,31.901412],[68.926677,31.620189],[68.556932,31.71331],[67.792689,31.58293],[67.683394,31.303154],[66.938891,31.304911],[66.381458,30.738899],[66.346473,29.887943],[65.046862,29.472181],[64.350419,29.560031],[64.148002,29.340819],[63.550261,29.468331],[62.549857,29.318572],[60.874248,29.829239],[61.781222,30.73585],[61.699314,31.379506],[60.941945,31.548075],[60.863655,32.18292],[60.536078,32.981269],[60.9637,33.528832],[60.52843,33.676446],[60.803193,34.404102],[61.210817,35.650072],[62.230651,35.270664],[62.984662,35.404041],[63.193538,35.857166],[63.982896,36.007957],[64.546479,36.312073],[64.746105,37.111818],[65.588948,37.305217],[65.745631,37.661164],[66.217385,37.39379],[66.518607,37.362784]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[67.83,37.144994],[68.392033,38.157025],[68.176025,38.901553],[67.44222,39.140144],[67.701429,39.580478],[68.536416,39.533453],[69.011633,40.086158],[69.329495,40.727824],[70.666622,40.960213],[70.45816,40.496495],[70.601407,40.218527],[71.014198,40.244366],[70.648019,39.935754],[69.55961,40.103211],[69.464887,39.526683],[70.549162,39.604198],[71.784694,39.279463],[73.675379,39.431237],[73.928852,38.505815],[74.257514,38.606507],[74.864816,38.378846],[74.829986,37.990007],[74.980002,37.41999],[73.948696,37.421566],[73.260056,37.495257],[72.63689,37.047558],[72.193041,36.948288],[71.844638,36.738171],[71.448693,37.065645],[71.541918,37.905774],[71.239404,37.953265],[71.348131,38.258905],[70.806821,38.486282],[70.376304,38.138396],[70.270574,37.735165],[70.116578,37.588223],[69.518785,37.608997],[69.196273,37.151144],[68.859446,37.344336],[68.135562,37.023115],[67.83,37.144994]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[69.464887,39.279463,80.25999,40.086158]","geometry":{"type":"Polygon","coordinates":[[[71.844638,42.845395],[73.489758,42.500894],[73.645304,43.091272],[74.212866,43.298339],[75.636965,42.8779],[76.000354,42.988022],[77.658392,42.960686],[79.142177,42.856092],[79.643645,42.496683],[80.25999,42.349999],[80.11943,42.123941],[78.543661,41.582243],[78.187197,41.185316],[76.904484,41.066486],[76.526368,40.427946],[75.467828,40.562072],[74.776862,40.366425],[73.82244,39.893973],[73.960013,39.660008],[73.675379,39.431237],[71.784694,39.279463],[70.549162,39.604198],[69.464887,39.526683],[69.55961,40.103211],[70.648019,39.935754],[71.014198,40.244366],[71.774875,40.145844],[73.055417,40.866033],[71.870115,41.3929],[71.157859,41.143587],[70.420022,41.519998],[71.259248,42.167711],[70.962315,42.266154]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[52.50246,35.270664,66.54615,42.751551]","geometry":{"type":"Polygon","coordinates":[[[52.50246,41.783316],[52.944293,42.116034],[54.079418,42.324109],[54.755345,42.043971],[55.455251,41.259859],[55.968191,41.308642],[57.096391,41.32231],[56.932215,41.826026],[57.78653,42.170553],[58.629011,42.751551],[59.976422,42.213082],[60.083341,41.425146],[60.465953,41.220327],[61.547179,41.26637],[61.882714,41.084857],[62.37426,40.053886],[63.528015,39.363257],[64.170223,38.892407],[65.215999,38.402699],[66.54615,37.974685],[66.518607,37.362784],[66.217385,37.39379],[65.745631,37.661164],[65.588948,37.305217],[64.746105,37.111818],[64.546479,36.312073],[63.982896,36.007957],[63.193538,35.857166],[62.984662,35.404041],[62.230651,35.270664],[61.210817,35.650072],[61.123071,36.491597],[60.377638,36.527383],[59.234762,37.412988],[58.436154,37.522309],[58.436154,37.522309],[59.234762,37.412988],[60.377638,36.527383],[61.123071,36.491597],[61.210817,35.650072],[60.803193,34.404102],[60.52843,33.676446],[60.9637,33.528832],[60.536078,32.981269],[60.863655,32.18292],[60.941945,31.548075],[61.699314,31.379506],[61.781222,30.73585],[60.874248,29.829239],[61.369309,29.303276],[61.771868,28.699334],[62.72783,28.259645],[62.755426,27.378923],[63.233898,27.217047],[63.316632,26.756532],[61.874187,26.239975],[61.497363,25.078237],[59.616134,25.380157],[58.525761,25.609962],[57.397251,25.739902],[56.970766,26.966106],[56.492139,27.143305],[55.72371,26.964633],[54.71509,26.480658],[53.493097,26.812369],[52.483598,27.580849],[51.520763,27.86569],[50.852948,28.814521],[50.115009,30.147773],[49.57685,29.985715],[48.941333,30.31709],[48.567971,29.926778]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[35.700798,32.312938,42.349591,37.229873]","geometry":{"type":"Polygon","coordinates":[[[48.567971,29.926778],[48.014568,30.452457],[48.004698,30.985137],[47.685286,30.984853],[47.849204,31.709176],[47.334661,32.469155],[46.109362,33.017287],[45.416691,33.967798],[45.64846,34.748138],[46.151788,35.093259],[46.07634,35.677383],[45.420618,35.977546],[44.772677,37.170437],[44.77267,37.17045],[44.225756,37.971584],[44.421403,38.281281],[44.109225,39.428136],[44.79399,39.713003],[44.952688,39.335765],[45.457722,38.874139],[46.143623,38.741201],[46.50572,38.770605],[47.685079,39.508364],[48.060095,39.582235],[48.355529,39.288765],[48.010744,38.794015],[48.634375,38.270378],[48.883249,38.320245],[49.199612,37.582874],[50.147771,37.374567],[50.842354,36.872814],[52.264025,36.700422],[53.82579,36.965031],[53.921598,37.198918],[54.800304,37.392421],[55.511578,37.964117],[56.180375,37.935127],[56.619366,38.121394],[57.330434,38.029229],[58.436154,37.522309],[59.234762,37.412988],[60.377638,36.527383],[61.123071,36.491597],[61.210817,35.650072],[60.803193,34.404102],[60.52843,33.676446],[60.9637,33.528832],[60.536078,32.981269],[60.863655,32.18292],[60.941945,31.548075],[61.699314,31.379506],[61.781222,30.73585],[60.874248,29.829239],[61.369309,29.303276],[61.771868,28.699334],[62.72783,28.259645],[62.755426,27.378923],[63.233898,27.217047],[63.316632,26.756532],[61.874187,26.239975],[61.497363,25.078237],[59.616134,25.380157],[58.525761,25.609962],[57.397251,25.739902],[56.970766,26.966106],[56.492139,27.143305],[55.72371,26.964633],[54.71509,26.480658],[53.493097,26.812369],[52.483598,27.580849],[51.520763,27.86569],[50.852948,28.814521],[50.115009,30.147773],[49.57685,29.985715],[48.941333,30.31709],[48.567971,29.926778]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[35.700798,32.312938,42.349591,37.229873]","geometry":{"type":"Polygon","coordinates":[[[48.567971,29.926778],[48.014568,30.452457],[48.004698,30.985137],[47.685286,30.984853],[47.849204,31.709176],[47.334661,32.469155],[46.109362,33.017287],[45.416691,33.967798],[45.64846,34.748138],[46.151788,35.093259],[46.07634,35.677383],[45.420618,35.977546],[44.772677,37.170437],[44.77267,37.17045],[44.225756,37.971584],[44.421403,38.281281],[44.109225,39.428136],[44.79399,39.713003],[44.952688,39.335765],[45.457722,38.874139],[46.143623,38.741201],[46.50572,38.770605],[47.685079,39.508364],[48.060095,39.582235],[48.355529,39.288765],[48.010744,38.794015],[48.634375,38.270378],[48.883249,38.320245],[49.199612,37.582874],[50.147771,37.374567],[50.842354,36.872814],[52.264025,36.700422],[53.82579,36.965031],[53.921598,37.198918],[54.800304,37.392421],[55.511578,37.964117],[56.180375,37.935127],[56.619366,38.121394],[57.330434,38.029229],[58.436154,37.522309],[59.234762,37.412988],[60.377638,36.527383],[61.123071,36.491597],[61.210817,35.650072],[60.803193,34.404102],[60.52843,33.676446],[60.9637,33.528832],[60.536078,32.981269],[60.863655,32.18292],[60.941945,31.548075],[61.699314,31.379506],[61.781222,30.73585],[60.874248,29.829239],[61.369309,29.303276],[61.771868,28.699334],[62.72783,28.259645],[62.755426,27.378923],[63.233898,27.217047],[63.316632,26.756532],[61.874187,26.239975],[61.497363,25.078237],[59.616134,25.380157],[58.525761,25.609962],[57.397251,25.739902],[56.970766,26.966106],[56.492139,27.143305],[55.72371,26.964633],[54.71509,26.480658],[53.493097,26.812369],[52.483598,27.580849],[51.520763,27.86569],[50.852948,28.814521],[50.115009,30.147773],[49.57685,29.985715],[48.941333,30.31709],[48.567971,29.926778]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[35.700798,32.312938,42.349591,37.229873]","geometry":{"type":"Polygon","coordinates":[[[48.567971,29.926778],[48.014568,30.452457],[48.004698,30.985137],[47.685286,30.984853],[47.849204,31.709176],[47.334661,32.469155],[46.109362,33.017287],[45.416691,33.967798],[45.64846,34.748138],[46.151788,35.093259],[46.07634,35.677383],[45.420618,35.977546],[44.772677,37.170437],[44.77267,37.17045],[44.225756,37.971584],[44.421403,38.281281],[44.109225,39.428136],[44.79399,39.713003],[44.952688,39.335765],[45.457722,38.874139],[46.143623,38.741201],[46.50572,38.770605],[47.685079,39.508364],[48.060095,39.582235],[48.355529,39.288765],[48.010744,38.794015],[48.634375,38.270378],[48.883249,38.320245],[49.199612,37.582874],[50.147771,37.374567],[50.842354,36.872814],[52.264025,36.700422],[53.82579,36.965031],[53.921598,37.198918],[54.800304,37.392421],[55.511578,37.964117],[56.180375,37.935127],[56.619366,38.121394],[57.330434,38.029229],[58.436154,37.522309],[59.234762,37.412988],[60.377638,36.527383],[61.123071,36.491597],[61.210817,35.650072],[60.803193,34.404102],[60.52843,33.676446],[60.9637,33.528832],[60.536078,32.981269],[60.863655,32.18292],[60.941945,31.548075],[61.699314,31.379506],[61.781222,30.73585],[60.874248,29.829239],[61.369309,29.303276],[61.771868,28.699334],[62.72783,28.259645],[62.755426,27.378923],[63.233898,27.217047],[63.316632,26.756532],[61.874187,26.239975],[61.497363,25.078237],[59.616134,25.380157],[58.525761,25.609962],[57.397251,25.739902],[56.970766,26.966106],[56.492139,27.143305],[55.72371,26.964633],[54.71509,26.480658],[53.493097,26.812369],[52.483598,27.580849],[51.520763,27.86569],[50.852948,28.814521],[50.115009,30.147773],[49.57685,29.985715],[48.941333,30.31709],[48.567971,29.926778]]]]}],{"type":"Feature","properties":{"name":"","bbox":"[35.700798,32.312938,42.349591,37.229873]","geometry":{"type":"Polygon","coordinates":[[[48.567971,29.926778],[48.014568,30.452457],[48.004698,30.985137],[47.685286,30.984853],[47.849204,31.709176],[47.334661,32.469155],[46.109362,33.017287],[45.416691,33.967798],[45.64846,34.748138],[46.151788,35.093259],[46.07634,35.677383],[45.420618,35.977546],[44.772677,37.170437],[44.77267,37.17045],[44.225756,37.971584],[44.421403,38.

coordinates":[[[35.719918,32.709192],[35.700798,32.716014],[35.836397,32.868123],[35.821101,33.277426],[36.06646,33.824912],[36.61175,34.201789],[36.448194,34.593935],[35.998403,34.644914],[35.905023,35.410009],[36.149763,35.821535],[36.41755,36.040617],[36.685389,36.259699],[36.739494,36.81752],[37.066761,36.623036],[38.167727,36.90121],[38.699891,36.712927],[39.52258,36.716054],[40.673259,37.091276],[41.212089,37.074352],[42.349591,37.229873],[41.837064,36.605854],[41.289707,36.358815],[41.383965,35.628317],[41.006159,34.419372],[38.792341,33.378686],[36.834062,32.319387],[35.719918,32.709192]]]],{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[43.582746,38.741201,46.50572,41.248129],"geometry":{"type":"Polygon","coordinates":[[[46.50572,38.770605],[46.143623,38.741201],[45.735379,39.319719],[45.739978,39.473999],[45.298145,39.471751],[45.001987,39.740004],[44.79399,39.713003],[44.400009,40.005],[43.656436,40.253564],[43.752658,40.740201],[43.582746,41.092143],[44.97248,41.248129],[45.179496,40.985354],[45.560351,40.81229],[45.359175,40.561504],[45.891907,40.218476],[45.610012,39.899994],[46.034534,39.628021],[46.834399,39.464155],[46.50572,38.770605]]]}},{"type":"Feature","properties":{"name":"□□□□□","bbox":[11.027369,55.361737,23.903379,69.106247],"geometry":{"type":"Polygon","coordinates":[[[11.027369,58.856149],[11.468272,59.432393],[12.300366,60.117933],[12.631147,61.293572],[11.992064,61.800362],[11.930569,63.128318],[12.579935,64.066219],[13.571916,64.049114],[13.919905,64.445421],[13.55569,64.787028],[15.108411,66.193867],[16.108712,67.302456],[16.768879,68.013937],[17.729182,68.010552],[17.993868,68.567391],[19.87856,68.407194],[20.025269,69.065139],[20.645593,69.106247],[21.978535,68.616846],[23.539473,67.936009],[23.56588,66.396051],[23.903379,66.006927],[22.183173,65.72341],[21.213517,65.026005],[21.369631,64.413588],[19.778876,63.609554],[17.847779,62.7494],[17.119555,61.341166],[17.831346,60.636583],[18.787722,60.081914],[17.869225,58.953766],[16.829185,58.719827],[16.44771,57.041118],[15.879786,56.104302],[14.666681,56.200885],[14.100721,55.407781],[12.942911,55.361737],[12.625101,56.30708],[11.787942,57.441817],[11.027369,58.856149]]]}},{"type":"Feature","properties":{"name":"□□□□□□□","bbox":[23.199494,51.319503,32.693643,56.169133],"geometry":{"type":"Polygon","coordinates":[[[28.176709,56.169133],[29.229513,55.918344],[29.371572,55.670091],[29.896294,55.789463],[30.873909,55.550976],[30.971836,55.081548],[30.757534,54.811771],[31.384472,54.157056],[31.791424,53.974639],[31.731273,53.794029],[32.405599,53.618045],[32.693643,53.351421],[32.304519,53.132726],[31.49764,53.16743],[31.305201,53.073996],[31.540018,52.742055],[31.78597,52.10168],[31.785992,52.101678],[30.927549,52.042353],[30.619454,51.822806],[30.555117,51.319503],[30.157364,51.416138],[29.254938,51.368234],[28.992835,51.602044],[28.617613,51.427714],[28.241615,51.572227],[27.454066,51.592303],[26.337959,51.832289],[25.327788,51.910656],[24.553106,51.888461],[24.005078,51.617444],[23.527071,51.578454],[23.508002,52.023647],[23.199494,52.486977],[23.799199,52.691099],[23.804935,53.089731],[23.527536,53.470122],[23.484128,53.912498],[24.450684,53.905702],[25.536354,54.282423],[25.768433,54.846963],[26.588279,55.167176],[26.494331,55.615107],[27.10246,55.783314],[28.176709,56.169133]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[22.085608,45.293308,40.080789,52.330575],"geometry":{"type":"Polygon","coordinates":[[[31.785992,52.101678],[32.15944,52.06125],[32.412058,52.288695],[32.715761,52.238465],[33.7527,52.330575],[34.391731,51.768882],[34.141978,51.566413],[34.224816,51.255993],[35.022183,51.207572],[35.37791,50.77394],[35.356116,50.577197],[36.626168,50.225591],[37.39346,50.383953],[38.010631,49.915662],[38.594988,49.926462],[40.06904,49.60105],[40.080789,49.30743],[39.67465,48.78382],[39.89562,48.23241],[39.738278,47.898937],[38.77057,47.82562],[38.255122,47.5464],[38.223538,47.10219],[37.425137,47.022221],[36.759855,46.6987],[35.823685,46.645964],[34.962342,46.273197],[35.012659,45.737725],[34.861792,45.768182],[34.732017,45.965666],[34.410402,46.005162],[33.694962,46.219573],[33.435988,45.971917],[33.298567,46.080598],[31.74414,46.333348],[31.675307,46.706245],[30.748749,46.5831],[30.377609,46.03241],[29.603289,45.293308],[29.14972,45.464925],[28.679779,45.304031],[28.233554,45.488283],[28.485269,45.596907],[28.659987,45.939987],[28.933717,46.25883],[28.862972,46.437889],[29.072107,46.517678],[29.170654,46.379262],[29.759972,46.349988],[30.024659,46.423937],[29.83821,46.525326],[29.908852,46.674361],[29.559674,46.928583],[29.415135,47.346645],[29.050868,47.510227],[29.122698,47.849095],[28.670891,48.118149],[28.259547,48.155562],[27.522537,48.467119],[26.857824,48.368211],[26.619337,48.220726],[26.19745,48.220881],[25.945941,47.987149],[25.207743,47.891056],[24.866317,47.737526],[24.402056,47.981878],[23.760958,47.985598],[23.142236,48.096341],[22.710531,47.882194],[22.64082,48.15024],[22.085608,48.422264],[22.280842,48.825392],[22.558138,49.085738],[22.776419,49.027395],[22.51845,49.476774],[23.426508,50.308506],[23.922757,50.424881],[24.029986,50.705407],[23.527071,51.578454],[24.005078,51.617444],[24.553106,51.888461],[25.327788,51.910656],[26.337959,51.832289],[27.454066,51.592303],[28.241615,51.572227],[28.617613,51.427714],[28.992835,51.602044],[29.254938,51.368234],[30.157364,51.416138],[30.555117,51.319503],[30.619454,51.822806],[30.927549,52.042353],[31.785992,52.101678]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[14.074521,49.027395,24.029986,54.851536],"geometry":{"type":"Polygon","coordinates":[[[23.484128,53.912498],[23.527536,53.470122],[23.804935,53.089731],[23.799199,52.691099],[23.199494,52.486977],[23.508002,52.023647],[23.527071,51.578454],[24.029986,50.705407],[23.922757,50.424881],[23.426508,50.308506],[22.51845,49.476774],[22.776419,49.027395],[22.558138,49.085738],[21.607808,49.470107],[20.887955,49.328772],[20.415839,49.431453],[19.825023,49.217125],[19.320713,49.571574],[18.909575,49.435846],[18.853144,49.49623],[18.392914,49.988629],[17.649445,50.049038],[17.554567,50.362146],[16.868769,50.473974],[16.719476,50.215747],[16.176253,50.422607],[16.238627,50.697733],[15.490972,50.78473],[15.016996,51.106674],[14.607098,51.745188],[14.685026,52.089947],[14.4376,52.62485],[14.074521,52.981263],[14.353315,53.248171],[14.119686,53.757029],[14.8029,54.050706],[16.363477,54.513159],[17.622832,54.851536],[18.620859,54.682606],[18.696255,54.438719],[19.66064,54.426084],[20.892245,54.312525],[22.731099,54.327537],[23.243987,54.220567],[23.484128,53.912498]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[9.477967,46.431817,16.979667,49.039074],"geometry":{"type":"Polygon","coordinates":[[[16.979667,48.123497],[16.903754,47.714866],[16.340584,47.712902],[16.534268,47.496171],[16.202298,46.852386],[16.011664,46.683611],[15.137092,46.658703],[14.632472,46.431817],[13.806475,46.509306],[12.376485,46.767559],[12.115393,47.115393],[11.164828,46.941579],[11.048556,46.751359],[10.442701,46.893546],[9.932448,46.920728],[9.47997,47.10281],[9.632932,47.347601],[9.594226,47.525058],[9.896068,47.580197],[10.402084,47.302488],[10.544504,47.566399],[11.426414,47.523766],[12.141357,47.703083],[12.62076,47.672388],[12.932627,47.467646],[13.025851,47.637584],[12.884103,48.289146],[13.243357,48.416115],[13.595946,48.877172],[14.338898,48.555305],[14.901447,48.964402],[15.253416,49.039074],[16.029647,48.733899],[16.499283,48.785808],[16.960288,48.596982],[16.879983,48.470013],[16.979667,48.123497]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[16.202298,45.759481,22.710531,48.623854],"geometry":{"type":"Polygon","coordinates":[[[26.619337,48.220726],[26.619337,48.220726],[26.619337,48.220726]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[20.220192,43.688445,29.626543,48.220881],"geometry":{"type":"Polygon","coordinates":[[[28.233554,45.488283],[28.679779,45.304031],[29.149725,45.464925],[29.603289,45.293308],[29.626543,45.035391],[29.141612,44.82021],[28.837858,44.913874],[28.558081,43.707462],[27.970107,43.812468],[27.2424,44.175986],[26.065159,43.943494],[25.569272,43.688445],[24.100679,43.741051],[23.332302,43.897011],[22.944832,43.823785],[22.65715,44.234923],[22.474008,44.409228],[22.705726,44.578003],[22.459022,44.702517],[22.145088,44.478427],[21.562023,44.768947],[21.483256,45.18117],[20.874313,45.416375],[20.762175,45.734573],[20.220192,46.127469],[21.021952,46.316088],[21.626515,46.994238],[22.099768,47.672439],[22.710531,47.882194],[23.142236,48.096341],[23.760958,47.985598],[24.402056,47.981878],[24.866317,47.737526],[25.207743,47.891056],[25.945941,47.987149],[26.19745,48.220881],[26.619337,48.220726],[26.924176,48.123264],[27.233873,47.826771],[27.551166,47.405117],[28.12803,46.810476]]]}},

,[28.160018,46.371563],[28.054443,45.944586],[28.233554,45.488283]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [21.0558,53.905702,26.588279,56.372528],"geometry":{"type":"Polygon","coordinates": [[ [26.494331,55.615107],[26.588279,55.167176],[25.768433,54.846963],[25.536354,54.282423],[24.450684,53.905702],[23.484128,53.912498],[23.243987,54.20567],[22.731099,54.327537],[22.651052,54.582741],[22.757764,54.856574],[22.315724,55.015299],[21.268449,55.190482],[21.0558,56.031076],[22.201157,56.337802],[23.878264,56.273671],[24.860684,56.372528],[25.009934,56.164531],[25.533047,56.100297],[26.494331,55.615107]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [21.0558,55.615107,28.176709,57.970157],"geometry":{"type":"Polygon","coordinates": [[ [27.288185,57.474528],[27.770016,57.244258],[27.855282,56.759326],[28.176709,56.16913],[27.10246,55.783314],[26.494331,55.615107],[25.533047,56.100297],[25.009934,56.164531],[24.860684,56.372528],[23.878264,56.273671],[22.201157,56.337802],[21.0558,56.031076],[21.090424,56.783873],[21.581866,57.411871],[22.524341,57.753374],[23.318453,57.006236],[24.12073,57.025693],[24.312863,57.793424],[25.164594,57.970157],[25.60281,57.847529],[26.463532,57.476389],[27.288185,57.474528]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [23.33979,5,57.474528,28.131699,59.61109],"geometry":{"type":"Polygon","coordinates": [[ [27.981127,59.475373],[27.98112,59.47537],[28.131699,59.300825],[27.42015,58.72457],[27.716686,57.791899],[27.288185,57.474528],[26.463532,57.476389],[25.60281,57.847529],[25.164594,57.970157],[24.312863,57.793424],[24.428928,58.383413],[24.061198,58.257375],[23.42656,58.612753],[23.339795,59.18724],[24.604214,59.465854],[25.864189,59.61109],[26.949136,59.445803],[27.981114,59.475388],[27.981127,59.475373]]]]],{"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [23.33979,5,57.474528,28.131699,59.61109],"geometry":{"type":"Polygon","coordinates": [[ [14.119686,53.757029],[14.353315,53.248171],[14.074521,52.981263],[14.4376,52.62485],[14.685026,52.089947],[14.607098,51.745188],[15.016996,51.106674],[14.570718,51.002339],[14.307013,51.117268],[14.056228,50.926918],[13.338132,50.733234],[12.966837,50.484076],[12.240111,50.266338],[12.415191,49.969121],[12.521024,49.547415],[13.031329,49.307068],[13.595946,48.877172],[13.243357,48.416115],[12.884103,48.289146],[13.025851,47.637584],[12.932627,47.467646],[12.62076,47.672388],[12.141357,47.703083],[11.426414,47.523766],[10.544504,47.566399],[10.402084,47.302488],[9.896068,47.580197],[9.594226,47.525058],[8.522612,47.830828],[8.317301,47.61358],[7.466759,47.620582],[7.593676,48.333019],[8.099279,49.017784],[6.65823,49.201958],[6.18632,49.463803],[6.242751,49.902226],[6.043073,50.128052],[6.156658,50.803721],[5.988658,51.851616],[6.589397,51.852029],[6.84287,52.22844],[7.092053,53.144043],[6.90514,53.482162],[7.100425,53.693932],[7.936239,53.748296],[8.121706,53.527792],[8.800734,54.020786],[8.572118,54.395646],[8.526229,54.962744],[9.282049,54.830865],[9.921906,54.983104],[9.93958,54.596642],[10.950112,54.363607],[10.939467,54.008693],[11.956252,54.196486],[12.51844,54.470371],[13.647467,54.075511],[14.119686,53.757029]]]]],{"type":"Feature","properties":{"name":"□□□□□□□□□□"},"bbox": [22.380526,41.234486,28.558081,44.234923],"geometry":{"type":"Polygon","coordinates": [[ [22.65715,44.234923],[22.44832,43.823785],[23.332302,43.897011],[24.100679,43.741051],[25.569272,43.688445],[26.065159,43.943494],[27.2424,44.175986],[27.970107,43.812468],[28.558081,43.707462],[28.039995,43.293172],[27.673898,42.577892],[27.99672,42.007359],[27.135739,42.141485],[26.117042,41.826905],[26.106138,41.328899],[25.197201,41.234486],[24.492645,41.583896],[23.692074,41.309081],[22.952377,41.337994],[22.881374,41.999297],[22.380526,42.32026],[22.545012,42.461362],[22.436595,42.580321],[22.604801,42.898519],[22.986019,43.211161],[22.500157,43.642814],[22.410446,44.008063],[22.65715,44.234923]]]]}, {"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [20.150016,34.919988,26.604196,41.826905],"geometry":{"type":"MultiPolygon","coordinates": [[ [ [26.299003,35.29999],[26.164998,35.004995],[24.724982,34.919988],[24.735007,35.084991],[23.514978,35.279992],[23.69998,35.705004],[24.246665,35.368022],[25.025015,35.424996],[25.769208,35.354018],[25.745023,35.179998],[26.290003,35.29999]]], [[ [22.952377,41.337994],[23.692074,41.309081],[24.492645,41.583896],[25.197201,41.234486],[26.106138,41.328899],[26.117042,41.826905],[26.604196,41.562115],[26.294602,40.936261],[26.056942,40.824123],[25.447677,40.852545],[24.925848,40.947062],[23.714811,40.687129],[24.407999,40.124993],[23.899968,39.962006],[23.342999,39.960998],[22.813988,40.476005],[22.626299,40.256561],[22.849748,39.659311],[23.350027,39.190011],[22.973099,38.970903],[23.530016,38.510001],[24.025025,38.219993],[24.040011,37.655015],[23.115003,37.920011],[23.409972,37.409991],[22.774972,37.30501],[23.154225,36.422506],[22.490028,36.41],[21.670026,36.844986],[21.295011,37.644989],[21.120034,38.310323],[20.730032,38.769985],[20.217712,39.340235],[20.150016,39.624998],[20.615,40.110007],[20.674997,40.435],[20.99999,40.580004],[21.020004,40.842727],[21.674161,40.931275],[22.055378,41.149866],[22.597308,41.130487],[22.76177,41.3048],[22.952377,41.337994]]]]]]}, {"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [26.043351,35.821535,44.79399,42.141485],"geometry":{"type":"MultiPolygon","coordinates": [[ [ [44.772677,37.170437],[44.293452,37.001514],[43.942259,37.256228],[42.779126,37.385264],[42.349591,37.229873],[41.212089,37.074352],[40.673259,37.091276],[39.52258,36.716054],[38.699891,36.712927],[38.167727,36.90121],[37.066761,36.623036],[36.739494,36.81752],[36.685389,36.259699],[36.41755,36.040617],[36.149763,35.821535],[35.782085,36.274995],[36.160822,36.650606],[35.550936,36.565443],[34.714553,36.795532],[34.026895,36.21996],[32.509158,36.107564],[31.699595,36.644275],[30.621625,36.677865],[30.391096,36.262981],[29.699976,36.144357],[28.732903,36.676831],[27.641187,36.658822],[27.048768,37.653361],[26.318218,38.208133],[26.8047,38.98576],[26.170785,39.463612],[27.28002,40.420014],[28.819978,40.460011],[29.240004,41.219991],[31.145934,41.087622],[32.347979,41.736264],[33.513283,42.01896],[35.167704,42.040225],[36.913127,41.335358],[38.347665,40.948586],[39.512607,41.102763],[40.373433,41.013673],[41.554084,41.535656],[42.619549,41.583173],[43.582746,41.092143],[43.752658,40.740201],[43.656436,40.253564],[44.400009,40.005],[44.79399,71.3003],[44.109225,39.428136],[44.421403,38.281281],[44.225756,37.971584],[44.77267,37.17045],[44.772677,37.170437]]], [[ [26.117042,41.826905],[27.135739,42.141485],[27.99672,42.007359],[28.115525,41.622886],[28.988443,41.299934],[28.806438,41.054962],[27.619017,40.999823],[27.192377,40.690566],[26.358009,40.151994],[26.043351,40.617754],[26.056942,40.824123],[26.294602,40.936261],[26.604196,41.562115],[26.117042,41.826905]]]]]]], {"type":"Feature","properties":{"name":"□□□□□□□□"},"bbox": [19.304486,39.624998,21.02004,42.688247],"geometry":{"type":"Polygon","coordinates": [[ [21.02004,40.842727],[20.99999,40.580004],[20.674997,40.435],[20.615,40.110007],[20.150016,39.624998],[19.98,39.694993],[19.960002,39.915006],[19.406082,40.250773],[19.319059,40.72723],[19.40355,41.409566],[19.540027,41.719986],[19.371769,41.877548],[19.371768,41.877551],[19.304486,42.195745],[19.738051,42.688247],[19.801613,42.500093],[20.0707,42.58863],[20.283755,42.32026],[20.52295,42.21787],[20.590247,41.855409],[20.590247,41.855404],[20.463175,41.515089],[20.605182,41.086226],[21.020004,40.842727]]]]], {"type":"Feature","properties":{"name":"□□□□□□□"},"bbox": [13.656976,42.479991,19.390476,46.503751],"geometry":{"type":"Polygon","coordinates": [[ [16.564808,46.503751],[16.882515,46.380632],[17.630066,45.951769],[18.456062,45.759481],[18.829825,45.908872],[19.072769,45.521511],[19.390476,45.236516],[19.005485,44.860234],[18.553214,45.08159],[17.861783,45.06774],[17.002146,45.233777],[16.534939,45.211608],[16.318157,45.004127],[15.959367,45.233777],[15.750026,44.818712],[16.23966,44.351143],[16.456443,44.04124],[16.916156,43.667722],[17.297373,43.446341],[17.674922,43.028563],[18.56,42.65],[18.450017,42.479992],[18.450016,42.479991],[17.50997,42.849995],[16.930006,43.209998],[16.015385,43.507215],[15.174454,44.243191],[15.37625,44.317915],[14.920309,44.738484],[14.901602,45.07606],[14.258748,45.233777],[13.952255,44.802124],[13.656976,45.136935],[13.679403,45.484149],[13.71506,45.500324],[14.411968,45.466166],[14.595109,45.634941],[14.935244,45.471695],[15.327675,45.452316],[15.323954,45.731783],[15.67153,45.834154],[15.768733,46.238108],[16.564808,46.503751]]]]], {"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [6.022609,45.776948,10.442701,47.830828],"geometry":{"type":"Polygon","coordinates": [[ [9.594226,47.525058],[9.632932,47.347601],[9.47997,47.10281],[9.932448,46.920728],[10.442701,46.893546],[10.363378,46.483571],[9.922837,46.314899],[9.182882,46.440215],[8.966306,46.036932],[8.489952,46.005151],[8.31663,46.163642],[7.755992,45.82449],[7.273851,45.776948],[6.843593,45.991147],[6.5001,46.429673],[6.022609,46.27299],[6.037389,46.725779],[6.768714,47.287708],[6.736571,47.541801],[7.192202,47.449766],[7.466759,47.620582],[8.317301,47.61358],[8.522612,47.830828],[9.594226,47.525058]]]]], {"type":"Feature","properties":{"name":"□□□□□□□□□□"},"bbox": [5.674052,44.442667,6.242751,50.128052],"geometry":{"type":"Polygon","coordinates": [[ [6.043073,50.128052],[6.242751,49.902226],[6.18632,49.463803],[5.897759,49.442667],[5.674052,49.529484],[5.782417,50.090328],[6.043073,50.128052]]]]}, {"type":"Feature","properties":{"name":"□□□□□□"},"bbox": [2.513573,49.529484,6.156658,51.475024],"geometry":{"type":"Polygon","coordinates": [[ [6.156658,50.803721],[6.043073,50.128052],[5.782417,50.090328],[5.674052,49.529484],[4.799222,49.985373],[4.286023,49.907497],[3.588184,50.378992],[3.123252,50.780363],[2.658422,50.796848],[2.513573,51.148506],[3.314971,51.345781],[3.315011,51.345777],[3.314971,51.345755],[4.047071,51.267259],[4.973991,51.475024],[5.606976,51.037298],[6.156658,50.803721]]]]}, {"type":"Feature","properties":{"name":"□□□□□"},"bbox": [3.314971,50.803721,7.092053,53.510403],"geometry":{"type":"Pol

ygon","coordinates":[[[6.90514,53.482162],[7.092053,53.144043],[6.84287,52.22844],[6.589397,51.852029],[5.988658,51.851616],[6.156658,50.803721],[5.606976,51.037298],[4.973991,51.475024],[4.047071,51.267259],[3.314971,51.345755],[3.315011,51.345777],[3.830289,51.620545],[4.705997,53.091798],[6.074183,53.510403],[6.90514,53.482162]]]],{"type":"Feature","properties":{"name":"□□□□□□","bbox":[-9.526571,36.838269,-6.389088,42.280469],"geometry":{"type":"Polygon","coordinates":[[[-9.034818,41.880571],[-8.671946,42.134689],[-8.263857,42.280469],[-8.013175,41.790886],[-7.422513,41.792075],[-7.251309,41.918346],[-6.668606,41.883387],[-6.389088,41.381815],[-6.851127,41.11083],[6.86402,40.330872],[-7.026413,40.184524],[-7.066592,39.711892],[-7.498632,39.629571],[-7.098037,39.030073],[-7.374092,38.373059],[-7.029281,38.075764],[-7.166508,37.803894],[-7.537105,37.428904],[-7.453726,37.097788],[-7.855613,36.838269],[-8.382816,36.97888],[-8.898857,36.868809],[-8.746101,37.651346],[-8.839998,38.266243],[-9.287464,38.358486],[-9.526571,38.737429],[-9.446989,39.392066],[-9.048305,39.755093],[-8.977353,40.159306],[-8.768684,40.760639],[-8.790853,41.184334],[-8.990789,41.543459],[-9.034818,41.880571]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□","bbox":[-9.392884,35.94685,3.039484,43.748338],"geometry":{"type":"Polygon","coordinates":[[[-7.453726,37.097788],[-7.537105,37.428904],[-7.166508,37.803894],[-7.029281,38.075764],[-7.374092,38.373059],[-7.098037,39.030073],[-7.498632,39.629571],[-7.066592,39.711892],[-7.026413,40.184524],[-6.86402,40.330872],[-6.851127,41.11083],[-6.389088,41.381815],[-6.668606,41.883387],[-7.251309,41.918346],[-7.422513,41.792075],[-8.013175,41.790886],[-8.263857,42.280469],[-8.671946,42.134689],[-9.034818,41.880571],[-8.984433,42.592775],[-9.392884,43.026625],[-7.97819,43.748338],[-6.754492,43.567909],[-5.411886,43.57424],[-4.347843,43.403449],[-3.517532,43.455901],[-1.901351,43.422802],[-1.502771,43.034014],[0.338047,42.579546],[0.701591,42.795734],[1.826793,42.343385],[2.985999,42.473015],[3.039484,41.89212],[2.091842,41.226089],[0.810525,41.014732],[0.721331,40.678318],[0.106692,40.123934],[-0.278711,39.309978],[0.111291,38.738514],[-0.467124,38.292366],[-0.683389,37.642354],[-1.438382,37.443064],[-2.146453,36.674144],[-3.415781,36.6589],[-4.368901,36.677839],[-4.995219,36.324708],[-5.37716,35.94685],[-5.866432,36.029817],[-6.236694,36.367677],[-6.520191,36.942913],[-7.453726,37.097788]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□","bbox":[-9.97086,51.669301,-6.032985,55.131622],"geometry":{"type":"Polygon","coordinates":[[[-6.197885,53.867565],[-6.032985,53.153164],[-6.788857,52.260118],[-8.561617,51.669301],[-9.977086,51.820455],[-9.166283,52.864629],[-9.688525,53.881363],[-8.327987,54.664519],[-7.572168,55.131622],[-7.366031,54.595841],[-7.572168,54.059956],[-6.95373,54.073702],[-6.197885,53.867565]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":["164.029606,-22.399976,167.120011,-20.105646],"geometry":{"type":"Polygon","coordinates":[[[165.77999,-21.080005],[166.599991,-21.700019],[167.120011,-22.159991],[166.740035,-22.399976],[166.189732,-22.129708],[165.474375,-21.679607],[164.829815,-21.14982],[164.167995,-20.444747],[164.029606,-20.105646],[164.459967,-20.120012],[165.020036,-20.459991],[165.460009,-20.800022],[165.77999,-21.080005]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":["156.491358,-10.826367,162.398646,-6.599338],"geometry":{"type":"MultiPolygon","coordinates":[[[162.119025,-10.482719],[162.398646,-10.826367],[161.700032,-10.820011],[161.319797,-10.204751],[161.917383,-10.446701],[162.119025,-10.482719]]],[[161.679982,-9.599982],[161.529397,-9.784312],[160.788253,-8.917543],[160.579997,-8.832009],[160.920028,-8.320009],[161.280006,-9.120011],[161.679982,-9.599982]]],[[160.852229,-9.872937],[160.462588,-9.89521],[159.849447,-9.794027],[159.640003,-9.63998],[159.702945,-9.24295],[160.362956,-9.400304],[160.688518,-9.610162],[160.852229,-9.872937]]],[[159.640003,-8.020027],[159.875027,-8.33732],[159.917402,-8.53829],[159.133677,-8.114181],[158.586114,-7.754824],[158.21115,-7.421872],[158.359978,-7.320018],[158.820001,-7.560003],[159.640003,-8.020027]]],[[157.14,-7.021638],[157.538426,-7.34782],[157.33942,-7.404767],[156.90203,-7.176874],[156.491358,-6.765943],[156.542828,-6.599338],[157.14,-7.021638]]]]]}],{"type":"Feature","properties":{"name":"□□□□□□□□□","bbox":["166.509144,-46.641235,178.517094,-34.450662],"geometry":{"type":"MultiPolygon","coordinates":[[[176.885824,-40.065978],[176.508017,-40.604808],[176.01244,-41.289624],[175.239567,-41.688308],[175.067898,-41.425895],[174.650973,-41.281821],[175.22763,-40.459236],[174.900157,-39.908933],[173.824047,-39.508854],[173.852262,-39.146602],[174.574802,-38.797683],[174.743474,-38.027808],[174.697017,-37.381129],[174.292028,-36.711092],[174.319004,-36.534824],[173.840997,-36.121981],[173.054171,-35.237125],[172.636005,-34.529107],[173.007042,-34.450662],[173.551298,-35.006183],[174.32939,-35.265496],[174.612009,-36.156397],[175.336616,-37.209098],[175.357596,-36.526194],[175.808887,-36.798942],[175.95849,-37.555382],[176.763195,-37.881253],[177.438813,-37.961248],[178.010354,-37.579825],[178.517094,-37.695373],[178.274731,-38.582813],[177.97046,-39.166343],[177.206993,-39.145776],[176.939981,-39.449736],[177.032946,-39.879943],[176.885824,-40.065978]]],[[169.667815,-43.555326],[170.52492,-43.031688],[171.12509,-42.512754],[171.569714,-41.767424],[171.948709,-41.514417],[172.097227,-40.956104],[172.79858,-40.493962],[173.020375,-40.919052],[173.247234,-41.331999],[173.958405,-40.926701],[174.247587,-41.349155],[174.248517,-41.770008],[173.876447,-42.233184],[173.22274,-42.970038],[172.711246,-43.372288],[173.080113,-43.853344],[172.308584,-43.865694],[171.452925,-44.242519],[171.185138,-44.897104],[170.616697,-45.908929],[169.831422,-46.355775],[169.332331,-46.641235],[168.411354,-46.619945],[167.763745,-46.290197],[166.676886,-46.219917],[166.509144,-45.852705],[167.046424,-45.110941],[168.303763,-44.123973],[168.949409,-43.935819],[169.667815,-43.555326]]]]]}],{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":["113.338953,-43.634597,153.569469,-10.668186],"geometry":{"type":"MultiPolygon","coordinates":[[[147.689259,-40.88258],[148.289068,-40.875438],[148.359865,-42.062445],[148.017301,-42.407024],[147.914052,-43.211522],[147.564564,-42.937689],[146.870343,-43.634597],[146.663327,-43.580854],[146.048378,-43.549745],[145.43193,-42.693776],[145.29509,-42.03361],[144.718071,-41.162552],[144.743755,-40.703975],[145.397978,-40.792549],[146.364121,-41.137695],[146.908584,-41.000546],[147.689259,-40.88258]],[[126.148714,-32.215966],[125.088623,-32.728751],[124.221648,-32.959487],[124.028947,-33.483847],[123.659667,-33.890179],[122.811036,-33.914467],[122.183064,-34.003402],[121.299191,-33.821036],[120.580268,-33.930177],[119.893695,-33.976065],[119.298899,-34.509366],[119.007341,-34.464149],[118.505718,-34.746819],[118.024972,-35.064733],[117.295507,-35.025459],[116.625109,-35.025097],[115.564347,-34.386428],[115.026809,-34.196517],[115.048616,-33.623425],[115.545123,-33.487258],[115.714674,-33.259572],[115.679379,-32.900369],[115.801645,-32.205062],[115.689611,-31.612437],[115.160909,-30.601594],[114.997043,-30.030725],[115.040038,-29.461095],[114.641974,-28.810231],[114.616498,-28.516399],[114.173579,-28.118077],[114.048884,-27.334765],[113.477498,-26.543134],[113.338953,-26.116545],[113.778358,-26.549025],[113.440962,-25.621278],[113.936601,-25.911235],[114.232852,-26.298446],[114.216161,-25.786281],[113.721255,-24.998939],[113.625344,-24.683971],[113.393523,-24.384764],[113.502044,-23.80635],[113.706993,-23.560215],[113.843418,-23.059987],[113.736552,-22.475475],[114.149756,-21.755881],[114.225307,-22.517488],[114.647762,-21.82952],[115.460167,-21.405173],[115.947373,-21.068687],[116.711615,-20.701682],[117.166316,-20.623599],[117.441545,-20.746899],[118.229559,-20.374208],[118.836085,-20.263311],[118.987807,-20.044203],[119.252494,-19.952942],[119.805225,-19.976506],[120.85622,-19.683708],[121.399856,-19.239756],[121.655138,-18.705318],[122.241665,-18.197649],[122.286624,-17.798603],[122.312772,-17.254967],[123.012574,-16.4052],[123.433789,-17.268558],[123.859345,-17.069035],[123.503242,-16.596506],[123.817073,-16.111316],[124.258287,-16.327944],[124.379726,-15.56706],[124.926153,-15.0751],[125.167275,-14.680396],[125.670087,-14.51007],[125.685796,-14.230656],[126.125149,-14.347341],[126.142823,-14.095987],[126.582589,-13.952791],[127.065867,-13.817968],[127.804633,-14.276906],[128.35969,-14.86917],[128.985543,-14.875991],[129.621473,-14.969784],[129.4096,-14.42067],[129.888641,-13.618703],[130.339466,-13.357376],[130.183506,-13.10752],[130.617795,-12.536392],[131.223495,-12.183649],[131.735091,-12.302453],[132.575298,-12.114041],[132.557212,-11.603012],[131.824698,-11.273782],[132.357224,-11.128519],[133.019561,-11.376411],[133.550846,-11.786515],[134.393068,-12.042365],[134.678632,-11.941183],[135.298491,-12.248606],[135.882693,-11.962267],[136.258381,-12.049342],[136.492475,-11.857209],[136.95162,-12.351959],[136.685125,-12.887223],[136.305407,-13.29123],[135.961758,-13.324509],[136.077617,-13.724278],[135.783836,-14.223989],[135.428664,-14.715432],[135.500184,-14.997741],[136.295175,-15.550265],[137.06536,-15.870762],[137.580471,-16.215082],[138.303217,-16.807604],[138.585164,-16.806622],[139.108543,-17.062679],[139.260575,-17.371601],[140.215245,-17.710805],[140.875463,-17.369069],[141.07111,-16.832047],[141.274095,-16.38887],[141.398222,-15.840532],[141.702183,-15.044921],[141.56338,-14.561333],[141.63552,-14.270395],[141.519869,-13.698078],[141.65092,-12.944688],[141.842691,-12.741548],[141.68699,-12.407614],[141.928629,-11.877466],[142.118488,-11.328042],[142.143706,-11.042737],[142.51526,-10.668186],[142.79731,-11.157355],[142.866763,-11.784707],[143.115947,-11.90563],[143.158632,-12.325656],[143.522124,-12.834358],[143.597158,-13.400422],[143.561811,-13.763656],[143.922099,-14.548311],[144.563714,-14.171176],[144.894908,-14.594458],[145.374724,-14.984976],[145.271991,-15.428205],[145.48526,-16.285672],[145.637033,-16.784918],[145.888904,-16.9069

```

[[{"x": 146.160309, "y": -17.761655}, {"x": 146.063674, "y": -18.280073}, {"x": 146.387478, "y": -18.958274}, {"x": 147.471082, "y": -19.480723}, {"x": 148.177602, "y": -19.955939}, {"x": 148.848414, "y": -20.39121}, {"x": 148.717465, "y": -20.633469}, {"x": 149.289542, "y": -21.260511}, {"x": 149.678337, "y": -22.342512}, {"x": 150.077382, "y": -22.122784}, {"x": 150.482939, "y": -22.556142}, {"x": 150.727265, "y": -22.402405}, {"x": 150.899554, "y": 23.462237}, {"x": 151.609175, "y": 24.076256}, {"x": 152.07354, "y": -24.457887}, {"x": 152.855197, "y": -25.267501}, {"x": 153.136162, "y": -26.071173}, {"x": 153.161949, "y": -26.641319}, {"x": 153.092909, "y": -27.2603}, {"x": 153.569469, "y": -28.110067}, {"x": 153.512108, "y": -28.995077}, {"x": 153.339095, "y": -29.458202}, {"x": 153.069241, "y": -30.35024}, {"x": 153.089602, "y": -30.923642}, {"x": 152.891578, "y": -31.640446}, {"x": 152.450002, "y": -32.550003}, {"x": 151.709117, "y": -33.004132}, {"x": 151.343972, "y": -33.816023}, {"x": 151.010555, "y": -34.31036}, {"x": 150.714139, "y": -35.17346}, {"x": 150.32822, "y": -35.671879}, {"x": 150.075212, "y": -36.420266}, {"x": 149.946124, "y": -37.109052}, {"x": 149.997284, "y": -37.425261}, {"x": 149.423882, "y": -37.772681}, {"x": 148.304622, "y": -37.809061}, {"x": 147.381733, "y": -38.219217}, {"x": 146.922123, "y": -38.606532}, {"x": 146.317922, "y": -39.035757}, {"x": 145.489652, "y": -38.593768}, {"x": 144.876976, "y": -38.417448}, {"x": 145.032212, "y": -37.896188}, {"x": 144.485682, "y": -38.085324}, {"x": 143.609974, "y": -38.809465}, {"x": 142.745427, "y": -38.538268}, {"x": 142.17833, "y": -38.380043}, {"x": 141.606582, "y": -38.308514}, {"x": 140.638579, "y": -38.019333}, {"x": 139.992158, "y": -37.402936}, {"x": 139.806588, "y": -36.643603}, {"x": 139.574148, "y": -36.138362}, {"x": 139.082808, "y": -35.732754}, {"x": 138.120748, "y": -35.612296}, {"x": 138.449462, "y": -35.127261}, {"x": 138.207564, "y": -34.384723}, {"x": 137.71917, "y": -35.076825}, {"x": 136.829406, "y": -35.260535}, {"x": 137.352371, "y": -34.707339}, {"x": 137.503886, "y": -34.130268}, {"x": 137.890116, "y": -33.640479}, {"x": 137.810328, "y": -32.900077}, {"x": 136.996837, "y": -33.752771}, {"x": 136.372069, "y": -34.094766}, {"x": 135.989043, "y": -34.890118}, {"x": 135.208213, "y": -34.47867}, {"x": 135.239218, "y": -33.947953}, {"x": 134.613417, "y": -33.222778}, {"x": 134.085904, "y": -32.848072}, {"x": 134.273903, "y": -32.617234}, {"x": 132.990777, "y": -32.011224}, {"x": 132.288081, "y": -31.982647}, {"x": 131.326331, "y": -31.495803}, {"x": 129.535794, "y": -31.590423}, {"x": 128.240938, "y": -31.948489}, {"x": 127.102867, "y": -32.282267}, {"x": 126.148714, "y": -32.215966}], [{"x": 146.160309, "y": -17.761655, "order": 1}, {"x": 146.063674, "y": -18.280073, "order": 2}, {"x": 146.387478, "y": -18.958274, "order": 3}, {"x": 147.471082, "y": -19.480723, "order": 4}, {"x": 148.177602, "y": -19.955939, "order": 5}, {"x": 148.848414, "y": -20.39121, "order": 6}, {"x": 148.717465, "y": -20.633469, "order": 7}, {"x": 149.289542, "y": -21.260511, "order": 8}, {"x": 149.678337, "y": -22.342512, "order": 9}, {"x": 150.077382, "y": -22.122784, "order": 10}, {"x": 150.482939, "y": -22.556142, "order": 11}, {"x": 150.727265, "y": -22.402405, "order": 12}, {"x": 150.899554, "y": 23.462237, "order": 13}, {"x": 151.609175, "y": 24.076256, "order": 14}, {"x": 152.07354, "y": -24.457887, "order": 15}, {"x": 152.855197, "y": -25.267501, "order": 16}, {"x": 153.136162, "y": -26.071173, "order": 17}, {"x": 153.161949, "y": -26.641319, "order": 18}, {"x": 153.092909, "y": -27.2603, "order": 19}, {"x": 153.569469, "y": -28.110067, "order": 20}, {"x": 153.512108, "y": -28.995077, "order": 21}, {"x": 153.339095, "y": -29.458202, "order": 22}, {"x": 153.069241, "y": -30.35024, "order": 23}, {"x": 153.089602, "y": -30.923642, "order": 24}, {"x": 152.891578, "y": -31.640446, "order": 25}, {"x": 152.450002, "y": -32.550003, "order": 26}, {"x": 151.709117, "y": -33.004132, "order": 27}, {"x": 151.343972, "y": -33.816023, "order": 28}, {"x": 151.010555, "y": -34.31036, "order": 29}, {"x": 150.714139, "y": -35.17346, "order": 30}, {"x": 150.32822, "y": -35.671879, "order": 31}, {"x": 150.075212, "y": -36.420266, "order": 32}, {"x": 149.946124, "y": -37.109052, "order": 33}, {"x": 149.997284, "y": -37.425261, "order": 34}, {"x": 149.423882, "y": -37.772681, "order": 35}, {"x": 148.304622, "y": -37.809061, "order": 36}, {"x": 147.381733, "y": -38.219217, "order": 37}, {"x": 146.922123, "y": -38.606532, "order": 38}, {"x": 146.317922, "y": -39.035757, "order": 39}, {"x": 145.489652, "y": -38.593768, "order": 40}, {"x": 144.876976, "y": -38.417448, "order": 41}, {"x": 145.032212, "y": -37.896188, "order": 42}, {"x": 144.485682, "y": -38.085324, "order": 43}, {"x": 143.609974, "y": -38.809465, "order": 44}, {"x": 142.745427, "y": -38.538268, "order": 45}, {"x": 142.17833, "y": -38.380043, "order": 46}, {"x": 141.606582, "y": -38.308514, "order": 47}, {"x": 140.638579, "y": -38.019333, "order": 48}, {"x": 139.992158, "y": -37.402936, "order": 49}, {"x": 139.806588, "y": -36.643603, "order": 50}, {"x": 139.574148, "y": -36.138362, "order": 51}, {"x": 139.082808, "y": -35.732754, "order": 52}, {"x": 138.120748, "y": -35.612296, "order": 53}, {"x": 138.4
```



```

MultiPolygon", "coordinates": [[[[[9.921906, 54.983104], [9.282049, 54.830865], [8.526229, 54.962744], [8.120311, 55.517723], [8.089977, 56.540012], [8.256582, 56.809969], [8.543438, 57.110003], [9.424469, 57.172066], [9.775559, 57.447941], [10.580006, 57.730017], [10.546106, 57.215733], [10.25, 56.890016], [10.369993, 56.609982], [10.912182, 56.458621], [10.667804, 56.081383], [10.369993, 56.190007], [9.649985, 55.469999], [9.921906, 54.983104]]], [[112.370904, 56.111407], [112.690006, 55.609991], [112.089991, 54.800015], [11.043543, 55.364864], [10.903914, 55.779955], [12.370904, 56.111407]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□□"}, {"type": "MultiPolygon", "coordinates": [[[-6.1972168, 49.96, 1.681531, 58.635], "geometry": {"type": "MultiPolygon", "coordinates": [[[-6.197885, 53.867565], [-6.95373, 54.073702], [-7.572168, 54.05956], [-7.366031, 54.595841], [-7.572168, 55.131622], [-6.733847, 55.12786], [-5.661949, 54.554603], [-6.197885, 53.867565]]], [[[-3.093831, 53.404547], [-3.09208, 53.404441], [-2.945009, 53.985], [-3.614701, 54.600937], [-3.630005, 54.615013], [-4.844169, 54.790971], [-5.082527, 55.061601], [-4.719112, 55.508473], [-5.047981, 55.783986], [-5.586398, 55.311146], [-5.644999, 56.275015], [-6.149981, 56.78501], [-5.786825, 57.818848], [-5.009999, 58.630013], [-4.211495, 58.550845], [-3.005005, 58.635], [-4.073828, 57.553025], [-3.055002, 57.690019], [-1.959281, 57.6848], [-2.219988, 56.870017], [-3.119003, 55.973793], [-2.085009, 55.909998], [-2.005676, 55.804903], [-1.114991, 54.624986], [-0.430485, 54.464376], [0.184981, 53.325014], [0.469977, 52.929999], [1.681531, 52.73952], [1.559988, 52.099998], [1.050562, 51.806761], [1.449865, 51.289428], [0.550334, 50.765739], [-0.787517, 50.774989], [-2.489998, 50.500019], [-2.956274, 50.69688], [-3.617448, 50.228356], [-4.542508, 50.341837], [-5.245023, 49.96], [-5.776567, 50.159678], [-4.30999, 51.20001], [-3.414851, 51.426009], [-3.422719, 51.426848], [-4.984367, 51.593466], [-5.267296, 51.9914], [-4.222347, 52.301356], [-4.770013, 52.840005], [-4.579999, 53.495004], [-3.093831, 53.404547]]]]], {"type": "Feature", "properties": {"name": "□□□□□□", "bbox": [-24.326184, 63.496383, -13.609732, 66.526792], "geometry": {"type": "Polygon", "coordinates": [[[-14.508695, 66.455892], [-14.739637, 65.808748], [-13.609732, 65.126671], [-14.909834, 64.364082], [-17.794438, 63.678749], [-18.656246, 63.496383], [-19.972755, 63.643635], [-22.762972, 63.960179], [-21.778484, 64.402116], [-23.955044, 64.89113], [-22.184403, 65.084968], [-22.227423, 65.378594], [-24.326184, 65.11189], [-23.650515, 66.262519], [-22.134922, 66.410469], [-20.576284, 65.732112], [-19.056842, 66.276601], [-17.98624, 65.993853], [-16.167819, 66.526792], [-14.508695, 66.455892]]]]], {"type": "MultiPolygon", "coordinates": [[[[[46.404951, 41.860675], [46.686071, 41.827137], [47.373315, 41.219732], [47.815666, 41.151416], [47.987283, 41.405819], [48.584353, 41.808869], [49.110264, 41.282287], [49.618915, 40.572924], [50.08483, 40.526157], [50.392821, 40.256561], [49.569202, 40.176101], [49.395259, 39.399482], [49.223228, 39.049219], [48.856532, 38.815486], [48.883249, 38.320245], [48.643375, 38.270378], [48.010744, 38.794015], [48.355529, 39.288765], [48.060095, 39.582235], [47.685079, 39.508364], [46.50572, 38.770605], [46.483499, 39.464155], [46.034534, 39.628021], [45.610012, 39.899994], [45.891907, 40.218476], [45.359175, 40.561504], [45.560351, 40.81229], [45.179496, 40.985354], [44.97248, 41.248129], [45.217426, 41.411452], [45.962601, 41.123873], [46.501637, 41.064445], [46.637908, 41.181673], [46.145432, 41.722802], [46.404951, 41.860675]]], [[46.134623, 38.741201], [45.457722, 38.874139], [44.952688, 39.335765], [44.79399, 39.713003], [45.001987, 39.740004], [45.298145, 39.471751], [45.739978, 39.473999], [45.735379, 39.319719], [46.143623, 38.741201]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□", "bbox": [39.955009, 43.434998], "geometry": {"type": "Polygon", "coordinates": [[39.955009, 43.434998], [40.076965, 43.553104], [40.92219, 43.38215], [42.3944, 43.2203], [43.75599, 42.74083], [43.93121, 42.55496], [44.537623, 42.711993], [45.470279, 42.502781], [45.7764, 42.09244], [46.404951, 41.860675], [46.145432, 41.722802], [46.637908, 41.181673], [46.501637, 41.064445], [45.962601, 41.123873], [45.217426, 41.411452], [44.97248, 41.248129], [43.582746, 41.092143], [42.619549, 41.583173], [41.554084, 41.535656], [41.703171, 41.962943], [41.45347, 42.645123], [40.875469, 43.013628], [40.321394, 43.128634], [39.955009, 43.434998]]]]], {"type": "Feature", "properties": {"name": "□□□□□□□", "bbox": [117.174275, 5.581003, 126.537424, 18.505227], "geometry": {"type": "MultiPolygon", "coordinates": [[[[120.833896, 12.704496], [120.323436, 13.466413], [121.180128, 13.429697], [121.527394, 13.06959], [121.26219, 12.20556], [120.833896, 12.704496]]], [[122.586089, 9.981045], [122.837081, 10
```



.735679,68.649557],[25.689213,69.092114],[26.179622,69.825299],[27.732292,70.164193],[29.015573,69.766491],[28.59193,69.064777]]],{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[16.879983,47.758429,22.558138,49.571574],"geometry":{"type":"Polygon","coordinates":[[[22.558138,49.085738],[22.280842,48.825392],[22.085608,48.422264],[21.872236,48.319971],[20.801294,48.623854],[20.473562,48.56285],[20.239054,48.327567],[19.769471,48.202691],[19.661364,48.266615],[19.174365,48.111379],[18.777025,48.081768],[18.696513,47.880954],[17.857133,47.758429],[17.488473,47.867466],[16.979667,48.123497],[16.879983,48.470013],[16.960288,48.596982],[17.101985,48.816969],[17.545007,48.800019],[17.886485,48.903475],[17.913512,48.996493],[18.104973,49.043983],[18.170498,49.271515],[18.399994,49.315001],[18.554971,49.495015],[18.853144,49.49623],[18.909575,49.435846],[19.320713,49.571574],[19.825023,49.217125],[20.415839,49.431453],[20.887955,49.328772],[21.607808,49.470107],[22.558138,49.085738]]]]},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[12.240111,48.555395,18.853144,51.11268],"geometry":{"type":"Polygon","coordinates":[[[15.016996,51.106674],[15.490972,50.78473],[16.238627,50.697733],[16.176253,50.422607],[16.719476,50.215747],[16.868769,50.473974],[17.554567,50.362146],[17.649445,50.049038],[18.392914,49.988629],[18.853144,49.49623],[18.554971,49.495015],[18.399994,49.315001],[18.170498,49.271515],[18.104973,49.043983],[17.913512,48.996493],[17.886485,48.903475],[17.545007,48.800019],[17.101985,48.816969],[16.960288,48.596982],[16.499283,48.785808],[16.029647,48.733899],[15.253416,49.039074],[14.901447,48.964402],[14.338898,48.555305],[13.595946,48.877172],[13.031329,49.307068],[12.521024,49.547415],[12.415191,49.969121],[12.240111,50.266338],[12.966837,50.484076],[13.338132,50.733234],[14.056228,50.926918],[14.307013,51.117268],[14.570718,51.002339],[15.016996,51.106674]]]]},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[36.32322,12.455416,43.081226,17.998307],"geometry":{"type":"Polygon","coordinates":[[[36.42951,14.42211],[36.32322,14.82249],[36.75389,16.29186],[36.85253,16.95655],[37.16747,17.26314],[37.904,17.42754],[38.41009,17.998307],[38.990623,16.840626],[39.26611,15.922723],[39.814294,15.435647],[41.179275,14.49108],[41.734952,13.921037],[42.276831,13.343992],[42.589576,13.000421],[43.081226,12.699639],[42.779642,12.455416],[42.35156,12.54223],[42.00975,12.86582],[41.59856,13.45209],[41.1552,13.77333],[40.8966,14.11864],[40.02625,14.51959],[39.34061,14.53155],[39.0994,14.74064],[38.51295,14.50547],[37.90607,14.95943],[37.59377,14.2131],[36.42951,14.42211]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[129.408463,31.029579,145.543137,45.551483],"geometry":{"type":"MultiPolygon","coordinates":[[[[141.884601,39.180865],[140.959489,38.174001],[140.976388,37.142074],[140.59977,36.343983],[140.774074,35.842877],[140.253279,35.138114],[138.975528,34.6676],[137.217599,34.606286],[135.792983,33.464805],[135.120983,33.849071],[135.079435,34.596455],[133.340316,34.375938],[132.156771,33.904933],[130.986145,33.885761],[132.000036,33.149992],[131.33279,31.450355],[130.686318,31.029579],[130.20242,31.418238],[130.447676,32.319475],[129.814692,32.61031],[129.408463,33.296056],[130.353935,33.604151],[130.878451,34.232743],[131.884229,34.749714],[132.617673,35.433933],[134.608301,35.731618],[135.677538,35.527134],[136.723831,37.304984],[137.390612,36.827391],[138.857602,37.827485],[139.426405,38.215962],[140.05479,39.438807],[139.883379,40.563312],[140.305783,41.195005],[141.368973,41.37856],[141.914263,39.991616],[141.884601,39.180865]]],[[144.613427,43.960883],[145.320825,44.384733],[145.543137,43.262088],[144.059662,42.988358],[143.18385,41.995215],[141.611491,42.678791],[141.067286,41.584594],[139.955106,41.569556],[139.817544,42.563759],[140.312087,43.33273],[141.380549,43.388825],[141.671952,44.772125],[141.967645,45.551483],[143.14287,45.510358],[143.910162,44.1741],[144.613427,43.960883]]],[[132.371176,33.463642],[132.924373,34.060299],[133.492968,33.944621],[133.904106,34.364931],[134.638428,34.149234],[134.766379,33.806335],[134.203416,33.201178],[133.79295,33.521985],[133.280268,33.28957],[133.014858,32.704567],[132.363115,32.989382],[132.371176,33.463642]]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[-62.685057,-27.548499,-54.29296,-19.342747],"geometry":{"type":"Polygon","coordinates":[[[-58.166392,-20.176701],[57.870674,-20.732688],[57.937156,-22.090176],[56.88151,-22.282154],[56.473317,-22.0863],[55.797958,-22.35693],[55.610683,-22.655619],[55.517639,-23.571998],[55.400747,-23.956935],[55.027902,-24.001274],[54.652834,-23.839578],[54.29296,-24.021014],[54.293476,-24.5708],[54.428946,-25.162185],[54.625291,-25.739255],[54.788795,-26.621786],[55.695846,-27.387837],[56.486702,-27.548499],[57.60976,-27.395899],[58.618174,-27.123719],[57.63366,-25.603657],[57.777217,-25.16234],[58.807128,-24.771459],[60.028966,-24.032796],[60.846565,-23.880713],[62.685057,-22.249029],[62.291179,-21.051635],[62.265961,-20.513735],[61.786326,-19.633737],[60.043565,-19.342747],[59.115042,-19.356906],[58.183471,-19.868399],[58.166392,-20.176701]]]]}},{"type":"Feature","properties":{"name":"□□□□","bbox":[42.604873,12.58595,53.108573,19.000003],"geometry":{"type":"Polygon","coordinates":[[[52.00001,19.000003],[52.782184,17.349742],[53.108573,16.651051],[52.385206,16.382411],[52.191729,15.938433],[52.168165,15.59742],[51.172515,15.17525],[49.574576,14.708767],[48.679231,14.003202],[48.238947,13.94809],[47.938914,14.007233],[47.354454,13.59222],[46.717076,13.399699],[45.877593,13.347764],[45.62505,13.290946],[45.406459,13.026905],[45.144356,12.953938],[44.989533,12.699587],[44.494576,12.721653],[44.175113,12.58595],[43.482959,12.6368],[43.22871,13.22095],[43.251448,13.767584],[43.087944,14.06263],[42.892245,14.802249],[42.604873,15.213335],[42.805015,15.261963],[42.702438,15.718886],[42.823671,15.911742],[42.779332,16.347891],[43.218375,16.66689],[43.115798,17.08844],[43.380794,17.579987],[43.791519,17.319977],[44.062613,17.410359],[45.216651,17.433329],[45.399999,17.333335],[46.366659,17.233315],[46.749994,17.283338],[47.000005,16.949999],[47.466695,17.116682],[48.183344,18.166669],[49.116672,18.616668],[52.00001,19.000003]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□□□□□□□","bbox":[34.632336,16.347891,55.666659,32.161009],"geometry":{"type":"Polygon","coordinates":[[[34.956037,29.356555],[36.068941,29.197495],[36.501214,29.505254],[36.740528,29.865283],[37.503582,30.003776],[37.66812,30.338665],[37.998849,30.5085],[37.002166,31.508413],[39.004886,32.10217],[39.195468,32.161009],[40.399994,31.889992],[41.889981,31.190009],[44.709499,29.178891],[46.568713,29.099025],[47.459822,29.002519],[47.708851,28.526063],[48.416094,28.552004],[48.807595,27.689628],[49.299554,27.461218],[49.470914,27.109999],[50.152422,26.689663],[50.212935,26.277027],[50.113303,25.943972],[50.239859,25.608055],[50.527387,25.327888],[50.660557,24.999896],[50.810108,24.754743],[51.112415,24.556331],[51.389608,24.627386],[51.579519,24.245497],[51.617708,24.014219],[52.000733,23.001154],[55.006803,22.496948],[55.208341,22.70833],[55.666659,22.000001],[54.999982,19.999994],[52.00001,19.000003],[49.116672,18.616668],[48.183344,18.166669],[47.466695,17.116682],[47.000005,16.949999],[46.749994,17.283338],[46.366659,17.233315],[45.399999,17.333335],[45.216651,17.43329],[44.062613,17.410359],[43.791519,17.319977],[43.380794,17.579987],[43.115798,17.08844],[43.218375,16.66689],[42.779332,16.347891],[42.649573,16.774635],[42.347989,17.075806],[42.270888,17.474722],[41.754382,17.833046],[41.221391,18.6716],[40.939341,19.486485],[40.247652,20.174635],[39.801685,20.338862],[39.139399,21.291905],[39.023696,21.986875],[39.066329,22.579656],[38.492772,23.688451],[38.02386,24.078686],[37.483635,24.285495],[37.154818,24.858483],[37.209491,25.004542],[36.931627,25.602959],[36.639604,25.826228],[36.249137,26.570136],[35.640182,27.37652],[35.130187,28.063352],[34.632336,28.058546],[34.787779,28.607427],[34.83222,28.957483],[34.956037,29.356555]]]]}},{"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[-180,-90,180,-63.27066],"geometry":{"type":"MultiPolygon","coordinates":[[[-48.660616,-78.04707],[45.154758,-78.04707],[43.920828,-78.478103],[43.48995,-79.08556],[43.372438,-79.516645],[43.333267,-80.026123],[44.880537,-80.339644],[46.506174,-80.594357],[48.386421,-80.829485],[50.482107,-81.025442],[52.851988,-80.966685],[54.164259,-80.633528],[53.987991,-80.222028],[51.853134,-79.94773],[50.991326,-79.614623],[50.364595,-79.183487],[49.914131,-78.811209],[49.306959,-78.458569],[48.660616,-78.047018],[48.660616,-78.047019]]],[[-66.290031,-80.255773],[64.037688,-80.294944],[61.883246,-80.39287],[61.138976,-79.981371],[60.610119,-79.628679],[59.572095,-80.040179],[59.865849,-80.549657],[60.159656,-81.000327],[62.255393,-80.863178],[64.488125,-80.921934],[65.741666,-80.588827],[65.741666,-80.549657],[66.290031,-80.255773]]],[[-73.915819,-71.269344],[73.230331,-71.15178],[72.074717,-71.190951],[71.780962,-70.681473],[71.72218,-70.309196],[71.741791,-69.505782],[71.173815,-69.035475],[70.253252,-68.87874],[69.724447,-69.251017],[69.489422,-69.623346],[69.058518,-70.074016],[68.725541,-70.505153],[68.451346,-70.955823],[68.33834,-71.406493],[68.510128,-71.798407],[68.784297,-72.170736],[69.959471,-72.307885],[71.075889,-72.503842],[72.388134,-72.484257],[71.8985,-72.092343],[73.073622,-72.229492],[74.19004,-72.366693],[74.953895,-72.307275],[75.012625,-71.661258],[73.915819,-71.269345]]],[[-102.330725,-71.894164],[102.330725,-71.894164],[101.703967,-71.717792],[100.430919,-71.854993],[98.98155,-71.93334],[97.884743,-72.070535],[96.787937,-71.952971],[96.20035,-72.521205],[96.983765,-72.442864],[98.198083,-72.482035],[99.432013,-72.442864],[100.783455,-72.50162],[101.801868,-72.305663],[102.330725,-71.894164]]],[[-122.621735,-73.657778],[122.621735,-73.657777],[122.406245,-73.324619],[121.211511,-73.500099],[119.918851,-73.657725],[118.724

[illegible]

.619419], [77.644904, -69.462684], [78.134539, -69.07077], [78.428371, -68.698441], [79.113859, -68.326216], [80.093127, -68.071503], [81.093535, -67.875546], [81.483792, -67.542388], [82.051767, -67.366068], [82.776426, -67.209282], [83.775331, -67.30726], [84.676206, -67.209282], [85.655527, -67.091718], [86.752359, -67.150474], [87.477017, -66.876175], [87.986289, -66.209911], [88.358411, -66.484261], [88.828408, -66.954568], [89.67063, -67.150474], [90.630365, -67.228867], [91.5901, -67.111303], [92.608539, -67.189696], [93.548637, -67.209282], [94.17542, -67.111303], [95.017591, -67.170111], [95.781472, -67.385653], [96.682399, -67.248504], [97.759646, -67.248504], [98.68021, -67.111303], [99.718182, -67.248504], [100.384188, -66.915346], [100.893356, -66.58224], [101.578896, -66.30789], [102.832411, -65.563284], [103.478676, -65.700485], [104.242557, -65.974783], [104.90846, -66.327527], [106.181561, -66.934931], [107.160881, -66.954568], [108.081393, -66.954568], [109.15864, -66.837004], [110.235835, -66.699804], [111.058472, -66.425505], [111.74396, -66.13157], [112.860378, -66.092347], [113.604673, -65.876805], [114.388088, -66.072762], [114.897308, -66.386283], [115.602381, -66.699804], [116.699161, -66.660633], [117.384701, -66.915346], [118.57946, -67.170111], [119.832924, -67.268089], [120.871, -67.189696], [121.654415, -66.876175], [122.320369, -66.562654], [123.221296, -66.484261], [124.122274, -66.621462], [125.160247, -66.719389], [126.100396, -66.562654], [127.001427, -66.562654], [127.882768, -66.660633], [128.80328, -66.758611], [129.704259, -66.58224], [130.781454, -66.425505], [131.799945, -66.386283], [132.935896, -66.386283], [133.85646, -66.288304], [134.757387, -66.209963], [135.031582, -65.72007], [135.070753, -65.308571], [135.697485, -65.582869], [135.873805, -66.033591], [136.206705, -66.44509], [136.618049, -66.778197], [137.460271, -66.954568], [138.596223, -66.895761], [139.908442, -66.876175], [140.809421, -66.817367], [142.121692, -66.817367], [143.061842, -66.779782], [144.374061, -66.837004], [145.490427, -66.915346], [146.195552, -67.228867], [145.999699, -67.601196], [146.646067, -67.895131], [147.723263, -68.130259], [148.839629, -68.385024], [150.132314, -68.561292], [151.483705, -68.71813], [152.502247, -68.874813], [153.638199, -68.894502], [154.284567, -68.561292], [155.165857, -68.835642], [155.92979, -69.149215], [156.811132, -69.384291], [158.025528, -69.482269], [159.181013, -69.59983], [159.670699, -69.991747], [160.80665, -70.226875], [161.570479, -70.579618], [162.686897, -70.736353], [163.842434, -70.716768], [164.919681, -70.775524], [166.11444, -70.755938], [167.309095, -70.834332], [168.425616, -70.971481], [169.463589, -71.20666], [170.501665, -71.402617], [171.20679, -71.696501], [171.089227, -72.088415], [170.560422, -72.441159], [170.109958, -72.891829], [169.75737, -73.24452], [169.28731, -73.65602], [167.975101, -73.812806], [167.387489, -74.165498], [166.094803, -74.38104], [165.644391, -74.772954], [164.958851, -75.145283], [164.234193, -75.458804], [163.822797, -75.870303], [163.568239, -76.24258], [163.47026, -76.693302], [163.489897, -77.065579], [164.057873, -77.457442], [164.273363, -77.82977], [164.743464, -78.182514], [166.604126, -78.319611], [166.995781, -78.750748], [165.193876, -78.907483], [163.666217, -79.123025], [161.766385, -79.162248], [160.924162, -79.730482], [160.747894, -80.200737], [160.316964, -80.573066], [159.788211, -80.945395], [161.120016, -81.278501], [161.629287, -81.690001], [162.490992, -82.062278], [163.705336, -82.395435], [165.095949, -82.708956], [166.604126, -83.022477], [168.895665, -83.335998], [169.404782, -83.825891], [172.283394, -84.041433], [172.477049, -84.117914], [173.224083, -84.41371], [175.985672, -84.158997], [178.277212, -84.472518], [180.8471338]]]]], {"type": "Feature", "properties": {"name": "□□□□□□ □□□ □□□□ □□□□", "bbox": [32.73178, 35.000345, 34.576474, 35.671596], "geometry": {"type": "Polygon", "coordinates": [[[32.73178, 35.140026], [32.802474, 35.145504], [32.946961, 35.386703], [33.667227, 35.373216], [34.576474, 35.671596], [33.900804, 35.245756], [33.973617, 35.058506], [33.86644, 35.093595], [33.675392, 35.017863], [33.525685, 35.038688], [33.475817, 35.000345], [33.455922, 35.101424], [33.383833, 35.162712], [33.190977, 35.173125], [32.919572, 35.087833], [32.73178, 35.140026]]]]}}, {"type": "Feature", "properties": {"name": "□□□□□□", "bbox": [32.256667, 34.571869, 34.004881, 35.173125], "geometry": {"type": "Polygon", "coordinates": [[[32.73178, 35.140026], [32.919572, 35.087833], [33.190977, 35.173125], [33.383833, 35.162712], [33.455922, 35.101424], [33.475817, 35.000345], [33.525685, 35.038688], [33.675392, 35.017863], [33.86644, 35.093595], [33.973617, 35.058506], [34.004881, 34.978098], [32.979827, 34.571869], [32.490296, 34.701655], [32.256667, 35.103232], [32.73178, 35.140026]]]]}}, {"type": "Feature", "properties": {"name": "□□□□□□", "bbox": [17.020428, 21.420734, -1.124551, 35.759988], "geometry": {"type": "Polygon", "coordinates": [[[2.169914, 35.168396], [-1.792986, 34.527919], [-1.733455, 33.919713], [-1.388049, 32.864015], [-1.124551, 32.651522], [-1.307899, 32.262889], [-2.616605, 32.094346], [-3.06898, 31.724498], [-3.647498, 31.637294], [-3.690441, 30.896952], [-4.859646, 30.501188], [-5.242129, 30.000443], [-6.060632, 29.7317], [-7.059228, 29.579228], [-8.674116, 28.841289], [-8.66559, 27.656426], [-8.817828, 27.656426], [-8.794884, 27.120666], [-9.413037, 27.088476], [-9.735343, 26.860945], [-10.189424, 26.860945], [-10.551263, 26.990808], [-11.392555, 26.883424], [-11.71822, 26.104092], [-12.030759, 26.030866], [-12.500963, 24.770116], [-13.89111, 23.691009], [-14.221168, 22.310163], [-14.630833, 21.86094], [-14.750955, 21.5006], [-17.002962, 21.420734], [-17.020428, 21.42231], [-16.973248, 21.885745], [-16.589137, 22.158234], [-16.261922, 22.67934], [-16.326414, 23.017768], [-15.982611, 23.723358], [-15.426004, 24.359134], [-15.089332, 24.520261], [-14.824645, 25.103533], [-14.800926, 25.636265], [-14.43994, 26.254418], [-13.773805, 26.618892], [-13.139942, 27.640148], [-13.121613, 27.654148], [-12.618837, 28.038186], [-11.688919, 28.148644], [-10.900957, 28.832142], [-10.399592, 29.098586], [-9.564811, 29.933574], [-9.814718, 31.177736], [-9.434793, 32.038096], [-9.300693, 32.564679], [-8.657476, 33.240245], [-7.654178, 33.697065], [-6.912544, 34.110476], [-6.244342, 35.145865], [-5.929994, 35.759988], [-5.193863, 35.755182], [-4.591006, 35.330712], [-3.640057, 35.399855], [-2.604306, 35.179093], [-2.169914, 35.168396]]]]], {"type": "Feature", "properties": {"name": "□□□□", "bbox": [22.747007, 22, 36.86623, 31.58568], "geometry": {"type": "Polygon", "coordinates": [[[36.86623, 22], [32.9, 22], [29.02, 22], [25.22], [25, 25.6825], [25, 29.238655], [24.70007, 30.04419], [24.95762, 30.6616], [24.80287, 31.08929], [25.16482, 31.56915], [26.4953, 31.58568], [27.45762, 31.32126], [28.45048, 31.02577], [28.91353, 30.87005], [29.68342, 31.18686], [30.09503, 31.4734], [30.97693, 31.55586], [31.68796, 31.4296], [31.96041, 30.9336], [32.19247, 31.26034], [32.99392, 31.02407], [33.7734, 30.96746], [34.265435, 31.219357], [34.26544, 31.21936], [34.823243, 29.761081], [34.9226, 29.50133], [34.64174, 29.09942], [34.42655, 28.34399], [34.15451, 27.8233], [33.92136, 27.6487], [33.58811, 27.97136], [33.13676, 28.41765], [32.42323, 29.85108], [32.32046, 29.76043], [32.73482, 28.70523], [33.34876, 27.69989], [34.10455, 26.14227], [34.47387, 25.59856], [34.79507, 25.03375], [35.69241, 23.92671], [35.49372, 23.75237], [35.52598, 23.10244], [36.69069, 22.20485], [36.86623, 22]]]]}}, {"type": "Feature", "properties": {"name": "□□□□□□", "bbox": [9.319411, 19.58047, 25.16482, 33.136996], "geometry": {"type": "Polygon", "coordinates": [[[25, 22], [25, 20.00304], [23.85, 20], [23.83766, 19.58047], [19.84926, 21.49509], [15.86085, 23.40972], [14.8513, 22.86295], [14.143871, 22.491289], [13.581425, 23.040506], [11.999506, 23.471668], [11.560669, 24.097909], [10.771364, 24.562532], [10.303847, 24.379313], [9.948261, 24.936954], [9.910693, 25.365455], [9.319411, 26.094325], [9.716286, 26.512206], [9.629056, 27.140953], [9.756128, 27.688259], [9.683885, 28.144174], [9.859998, 28.95999], [9.805634, 29.424638], [9.48214, 30.307556], [9.970017, 30.539325], [10.056575, 30.961831], [9.950225, 31.37607], [10.636901, 31.761421], [10.94479, 32.081815], [11.432253, 32.368903], [11.488787, 33.136996], [12.66331, 32.79278], [13.08326, 32.87882], [13.91868, 32.71196], [15.24563, 32.26508], [15.71394, 31.37626], [16.61162, 31.18218], [18.02109, 30.76357], [19.08641, 30.26639], [19.57404, 30.52582], [20.05335, 30.98576], [19.82033, 31.75179], [20.13397, 32.2382], [20.85452, 32.7068], [21.54298, 32.8432], [22.89576, 32.63858], [23.2368, 32.19149], [23.60913, 32.18726], [23.9275, 32.01667], [24.92114, 31.89936], [25.16482, 31.56915], [24.80287, 31.08929], [24.95762, 30.6616], [24.70007, 30.04419], [25, 29.238655], [25, 25.6825], [25, 22]]]]}}, {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [32.95418, 3.42206, 47.78942, 14.95943], "geometry": {"type": "Polygon", "coordinates": [[[47.78942, 8.003], [44.9636, 5.00162], [43.66087, 4.95755], [42.76967, 4.25259], [42.12861, 4.23413], [41.855083, 3.918912], [41.1718, 3.91909], [40.76848, 4.25702], [39.85494, 3.83879], [39.559384, 3.42206], [38.89251, 3.50074], [38.67114, 3.61607], [38.43697, 3.58851], [38.120915, 3.598605], [36.855093, 4.447864], [36.159079, 4.447864], [35.817448, 4.776966], [35.817448, 5.338232], [35.298007, 5.506], [34.70702, 6.59422], [34.25032, 6.82607], [34.0751, 7.22595], [33.56829, 7.71334], [32.95418, 7.78497], [33.2948, 8.35458], [33.8255, 8.37916], [33.97498, 8.68456], [33.96162, 9.58358], [34.25745, 10.63009], [34.73115, 10.91017], [34.83163, 11.31896], [35.26049, 12.08286], [35.86363, 12.57828], [36.27022, 13.56333], [36.42951, 14.42211], [37.59377, 14.2131], [37.90607, 14.95943], [38.51295, 14.50547], [39.0994, 14.74064], [39.34061, 14.53155], [40.02625, 14.51959], [40.8966, 14.11864], [41.1552, 13.77333], [41.5985, 6.1345209], [42.00975, 12.86582], [42.35156, 12.54223], [42, 12.1], [41.66176, 11.6312], [41.73959, 11.35511], [41.75557, 11.05091], [42, 3.1414, 11.0342], [42.55493, 11.10511], [42.776852, 10.926879], [42.55876, 10.57258], [42.92812, 10.02194], [43.29699, 9.54048], [43.67875, 9.18358], [46.94834, 7.99688], [47.78942, 8.003]]]]}}, {"type": "Feature", "properties": {"name": "□□□□□□□□", "bbox": [41.66176, 10.926879, 43.317852, 12.699639], "geometry": {"type": "Polygon", "coordinates": [[[42.35156, 12.54223], [42.779642, 12.455416], [43.081226, 12.699639], [43.317852, 12.390148], [43.286381, 11.974928], [42.715874, 11.735641], [43.145305, 11.46204], [42.776852, 10.926879], [42.55493, 11.10511], [42, 3.1414, 11.0342], [41.75557, 11.05091], [41.73959, 11.35511], [41.66176, 11.6312], [42, 12.1], [42.35156, 12.54223]]]]}}, {"type": "Feature", "properties": {"name": "□□□□□□□□□□", "bbox": [42.55876, 7.99688, 48.948206, 11.462], "geometry": {"type": "Polygon", "coordinates": [[[48.948205, 11.410617], [48.948205, 11.410617], [48.942005, 11.394266], [48.9

```
38491,10.982327],[48.938233,9.9735],[48.93813,9.451749],[48.486736,8.837626],[47.78942,8.003],[46.94834,7.99688],[43.67875,9.18358],[43.29699,9.54048],[42.92812,10.02194],[42.55876,10.57258],[42.776852,10.926879],[43.145305,11.46204],[43.47066,11.27771],[43.666668,10.864169],[44.117804,10.445538],[44.614259,10.442205],[45.556941,10.698029],[46.645401,10.816549],[47.525658,11.127228],[48.021596,11.193064],[48.378784,11.375482],[48.948206,11.410622],[48.948205,11.410617]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[29.579466,-1.443322,35.035994,2.429885],"geometry":{"type":"Polygon","coordinates":[[[33.903711,-0.95],[31.86617,-1.02736],[30.76986,-1.01455],[30.419105,-1.134659],[29.821519,-1.443322],[29.579466,-1.341313],[29.587838,-0.587406],[29.819503,-0.20531],[29.875779,0.59738],[30.086154,1.062313],[30.468508,1.583805],[30.85267,1.849396],[31.174149,2.204465],[30.773347,2.339883],[30.83386,3.509166],[30.833852,3.509172],[31.24556,3.7819],[31.88145,3.55827],[32.68642,3.79232],[33.39,3.79],[34.005,4.249885],[34.47913,3.5556],[34.59607,3.05374],[35.03599,1.90584],[34.6721,1.17694],[34.18,0.515],[33.893569,0.109814],[33.903711,-0.95]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□","bbox":[29.024926,-2.917858,30.816135,-1.134659],"geometry":{"type":"Polygon","coordinates":[[[30.419105,-1.134659],[30.816135,-1.698914],[30.758309,-2.28725],[30.46967,-2.41383],[30.469674,-2.413855],[29.938359,-2.348487],[29.632176,-2.917858],[29.024926,-2.839258],[29.117479,-2.292211],[29.254835,-2.21511],[29.291887,-1.620056],[29.579466,-1.341313],[29.821519,-1.443322],[30.419105,-1.134659]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[15.750026,42.65,19.59976,45.233777],"geometry":{"type":"Polygon","coordinates":[[[18.56,42.65],[17.674922,43.028517],[17.297373,43.446341],[16.916156,43.667722],[16.456443,44.04124],[16.23966,44.351143],[15.750026,44.818712],[15.959367,45.233777],[16.318157,45.004127],[16.534939,45.211608],[17.002146,45.233777],[17.861783,45.06774],[18.553214,45.08159],[19.005485,44.860234],[19.00548,44.86023],[19.36803,44.863],[19.11761,44.42307],[19.59976,44.03847],[19.454,43.5681],[19.21852,43.52384],[19.03165,43.42533],[18.70648,43.20011],[18.56,42.65]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":[20.463175,40.842727,22.952377,42.32026],"geometry":{"type":"Polygon","coordinates":[[[22.380526,42.32026],[22.881374,41.999297],[22.952377,41.337994],[22.76177,41.3048],[22.597308,41.130487],[22.055378,41.149866],[21.674161,40.931275],[21.02004,40.842727],[20.605182,41.086226],[20.463175,41.515089],[20.590247,41.855404],[20.590247,41.855409],[20.71731,41.84711],[20.76216,42.05186],[21.3527,42.2068],[21.576636,42.245224],[21.91708,42.30364],[22.380526,42.32026]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□","bbox":[18.829825,42.245224,22.986019,46.17173],"geometry":{"type":"Polygon","coordinates":[[[18.829825,45.908877],[18.829838,45.908878],[19.596045,46.17173],[20.220192,46.127469],[20.762175,45.734573],[20.874313,45.416375],[21.483526,45.18117],[21.562023,44.768947],[22.145088,44.478422],[22.459022,44.702517],[22.705726,44.578003],[22.474008,44.409228],[22.65715,44.234923],[22.410446,44.008063],[22.500157,43.642814],[22.986019,43.211161],[22.604801,42.898519],[22.436595,42.580321],[22.545012,42.461362],[22.380526,42.32026],[21.91708,42.30364],[21.576636,42.245224],[21.54332,42.32025],[21.66292,42.43922],[21.77505,42.6827],[21.77505,42.6827],[21.63302,42.67717],[21.43866,42.86255],[21.27421,42.90959],[21.143395,43.068685],[20.95651,43.13094],[20.81448,43.27205],[20.63508,43.21671],[20.49679,42.88469],[20.25758,42.81275],[20.3398,42.89852],[19.95857,43.10604],[19.63,43.21378],[19.48389,43.35229],[19.21852,43.52384],[19.454,43.5681],[19.59976,44.03847],[19.11761,44.42307],[19.36803,44.863],[19.00548,44.86023],[19.005485,44.860234],[19.390476,45.236516],[19.072769,45.521511],[18.829825,45.908877]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":[18.450017,41.877551,20.3398,43.52384],"geometry":{"type":"Polygon","coordinates":[[[20.0707,42.58863],[19.801613,42.500093],[19.738051,42.688427],[19.304486,42.195745],[19.371768,41.877551],[19.16246,41.95502],[18.8214,42.28151],[18.450017,42.479992],[18.56,42.65],[18.70648,43.20011],[19.03165,43.42533],[19.21852,43.52384],[19.48389,43.35229],[19.63,43.21378],[19.95857,43.10604],[20.3398,42.89852],[20.25758,42.81275],[20.0707,42.58863]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":[20.0707,41.84711,21.77505,43.27205],"geometry":{"type":"Polygon","coordinates":[[[20.590247,41.855409],[20.76216,42.05186],[20.71731,41.84711],[20.590247,41.855409]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□□□□□□□","bbox":[-61.95,10,-60.895,10.89],"geometry":{"type":"Polygon","coordinates":[[[-61.68,10.76],[61.105,10.89],[61.895,10.855],[61.935,10.11],[61.7,10],[61.95,10.09],[61.66,10.365],[61.68,10.76]]]}},{{"type":"Feature","properties":{"name":"□□□□□□□□□□□□","bbox":[23.88698,3.509172,35.298007,12.248008],"geometry":{"type":"Polygon","coordinates":[[[30.833852,3.509172],[29.9535,4.173699],[29.715995,4.600805],[29.159078,4.389267],[28.69678,4.455077],[28.428994,4.28715
```

```

input,
select,
textarea,
button {
 font-family: inherit;
}

/* thin scrollbars for tables */
.scroll-x {
 overflow-x: auto;
 -webkit-overflow-scrolling: touch;
}
.scroll-x::-webkit-scrollbar {
 height: 6px;
}
.scroll-x::-webkit-scrollbar-thumb {
 background: #94a3b8;
 border-radius: 99px;
}

.leaflet-container {
 font-family: inherit;
 z-index: 0;
 background: #e8f0ed;
}

/* 0000 000000 00000000000 000000 0000 00000 00000 */
.leaflet-tooltip.sek-province-label {
 border: 0;
 border-radius: 7px;
 background: rgba(255, 255, 255, 0.86);
 box-shadow: 0 1px 4px rgba(15, 23, 42, 0.18);
 color: #334155;
 direction: rtl;
 font-family: inherit;
 font-size: 9px;
 font-weight: 800;
 line-height: 1;
 padding: 4px 6px;
 pointer-events: none;
 white-space: nowrap;
}

.leaflet-tooltip.sek-province-label::before {
 display: none;
}

.leaflet-tooltip.sek-province-label.is-active {
 background: #0f766e;
 color: white;
 font-size: 10px;
}

.leaflet-tooltip.sek-point-tooltip {
 border: 0;
 border-radius: 9px;
 direction: rtl;
 font-family: inherit;
 font-size: 11px;
 font-weight: 800;
 padding: 6px 9px;
}

.sek-map-marker {
 background: transparent !important;
 border: 0 !important;
}

@keyframes fadeIn {
 from {
 opacity: 0;
 transform: translateY(6px);
 }
 to {
 opacity: 1;
 transform: none;
 }
}

.fade-in {
 animation: fadeIn 0.18s ease-out;
}

```